

AI ENGINEERING FROM SCRATCH.

Foundations: Math, Tooling, and Classical Machine Learning

VOLUME I / VI · 52 CHAPTERS · PHASES 00 · 01 · 02

EDITION 2026.08 · A SNAPSHOT OF A LIVING COURSE · NEWEST BUILD ALWAYS AT THE LINK BELOW

aiengineeringfromscratch.com

github.com/rohitg00/ai-engineering-from-scratch

Contents

About This Volume	1
How to Use This Book	2
Part I — Setup & Tooling	3
Dev Environment	4
Git & Collaboration	8
GPU Setup & Cloud	11
APIs & Keys	14
Jupyter Notebooks	18
Python Environments	23
Docker for AI	28
Editor Setup	36
Data Management	41
Terminal & Shell	47
Linux for AI	54
Debugging and Profiling	61
Part II — Math Foundations	69
Linear Algebra Intuition	70
Vectors, Matrices & Operations	79
Matrix Transformations	86
Calculus for Machine Learning	94
Chain Rule & Automatic Differentiation	107
Probability and Distributions	118
Bayes' Theorem	127
Optimization	135
Information Theory	143

Dimensionality Reduction	152
Singular Value Decomposition	161
Tensor Operations	173
Numerical Stability	181
Norms and Distances	193
Statistics for Machine Learning	204
Sampling Methods	214
Linear Systems	228
Convex Optimization	240
Complex Numbers for AI	252
The Fourier Transform	261
Graph Theory for Machine Learning	271
Stochastic Processes	281
Part III — ML Fundamentals	291
What Is Machine Learning	292
Linear Regression	301
Logistic Regression	312
Decision Trees and Random Forests	322
Support Vector Machines	332
K-Nearest Neighbors and Distances	340
Unsupervised Learning	348
Feature Engineering & Selection	359
Model Evaluation	371
Bias-Variance Tradeoff	385
Ensemble Methods	395
Hyperparameter Tuning	403
ML Pipelines	415
Naive Bayes	423
Time Series Fundamentals	433
Anomaly Detection	443

Handling Imbalanced Data	453
Feature Selection	464

About This Volume

This is Volume 1 of *AI Engineering from Scratch*, a six-volume compilation of the open course of the same name. Each volume stands alone; cross-references cite course phase numbers, which map to volumes like this:

Vol	Title	Course phases
1	Foundations — Math, Tooling, and Classical Machine Learning	00, 01, 02
2	Deep Learning — Networks, Vision, and Speech	03, 04, 06
3	Language — NLP Foundations and the Transformer	05, 07
4	Large Language Models — Generation, Reinforcement, Pretraining, and Engineering	08, 09, 10, 11
5	Agents — Multimodality, Protocols, Autonomy, and Swarms	12, 13, 14, 15, 16
6	Production — Infrastructure, Safety, and Capstones	17, 18, 19

The chapters in this volume come from course phases 00, 01, 02. Chapter prerequisites name phases, not volumes; use the table above to translate.

How to Use This Book

This volume is one loop of a larger machine, and it works best when you run the whole loop:

1. **Read the chapter here.** The prose, the derivations, and the code walkthroughs are complete on the page.
2. **Run the code from the repository.** Every chapter has a code/ directory with a working implementation you can run and break: <https://github.com/rohitg00/ai-engineering-from-scratch>
3. **Open the web edition for what paper cannot do.** Animated figures you can watch and drag, and a quiz per chapter that grades itself: <https://aiengineeringfromscratch.com>

The repository is the living edition. Lessons are updated as the field moves; the book is a snapshot with a version number. When they disagree, the repo is right.

Learning with an AI

This course is built to be read by agents as well as people. The machine-readable index of every lesson lives at <https://aiengineeringfromscratch.com/llms.txt>. If you learn with an AI assistant, paste this and go:

I am working through *AI Engineering from Scratch, Volume 1: Foundations*. Fetch <https://aiengineeringfromscratch.com/llms.txt>, find the lesson I name, and act as my tutor: quiz me on its Key Terms, review my solutions to its Exercises, and walk me through its code from the repository.

Part I — Setup & Tooling

Course phase 00. Live edition with animated figures and quizzes: <https://aiengineeringfromscratch.com/catalog.html>

Dev Environment

Your tools shape your thinking. Set them up once, set them up right.

Type: Build **Languages:** Python, Node.js, Rust **Prerequisites:** None **Time:** ~45 minutes

Learning Objectives

- Set up Python 3.11+, Node.js 20+, and Rust toolchains from scratch
- Configure virtual environments and package managers for reproducible builds
- Verify GPU access with CUDA/MPS and run a test tensor operation
- Understand the four-layer stack: system, packages, runtimes, AI libraries

The Problem

You're about to learn AI engineering across 200+ lessons using Python, TypeScript, Rust, and Julia. If your environment is broken, every single lesson becomes a fight against tooling instead of learning.

Most people skip environment setup. Then they spend hours debugging import errors, version conflicts, and missing CUDA drivers. We're going to do this once, properly.

The Concept

An AI engineering environment has four layers:

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/00-setup-and-tooling/01-dev-environment>

We install bottom-up. Each layer depends on the one below it.

Interactive figure: s0-env-stack. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/00-setup-and-tooling/01-dev-environment>

Build It

Step 1: System Foundation

Check your system and install the basics.

```
# macOS
xcode-select --install
brew install git curl wget
```

```
# Ubuntu/Debian
```



```
sudo apt update && sudo apt install -y build-essential git curl wget
```

```
# Windows (use WSL2)
wsl --install -d Ubuntu-24.04
```

Step 2: Python with uv

We use uv — it's 10-100x faster than pip and handles virtual environments automatically.

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

```
uv python install 3.12
```

```
uv venv
source .venv/bin/activate # or .venv\Scripts\activate on Windows
```

```
uv pip install numpy matplotlib jupyter
```

Verify:

```
import sys
print(f"Python {sys.version}")

import numpy as np
print(f"NumPy {np.__version__}")
a = np.array([1, 2, 3])
print(f"Vector: {a}, dot product with itself: {np.dot(a, a)}")
```

Step 3: Node.js with pnpm

For TypeScript lessons (agents, MCP servers, web apps).

```
curl -fsSL https://fnm.vercel.app/install | bash
fnm install 22
fnm use 22
```

```
npm install -g pnpm
```

```
node -e "console.log('Node', process.version)"
```

macOS / Apple Silicon (M1/M2/M3/M4): If the installer stops with Error: Cannot install under Rosetta 2 in ARM default prefix (/opt/homebrew), your terminal is running under Rosetta 2 (arch prints i386) while Homebrew is a native arm64 build. Install fnm forcing arm64, wire it into your shell, then rerun the commands above from fnm install 22:

```
arch -arm64 brew install fnm
echo 'eval "$(fnm env --use-on-cd)"' >> ~/.zshrc
source ~/.zshrc
```

Step 4: Rust

For performance-critical lessons (inference, systems).

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

```
rustc --version
cargo --version
```

Step 5: Julia (Optional)

For math-heavy lessons where Julia shines.

```
curl -fsSL https://install.julia-lang.org | sh
```

```
julia -e 'println("Julia ", VERSION)'
```

Step 6: GPU Setup (If You Have One)

NVIDIA (Linux / Windows):

```
nvidia-smi
```

Install PyTorch with CUDA

```
uv pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu124
```

macOS / Apple Silicon (M1/M2/M3/M4): There is no CUDA on a Mac — that's expected, not a failure. Do **not** pass `--index-url .../cuXXX` (those wheels are Linux/Windows only, so the install fails). Install the plain build, which includes Apple's MPS (Metal) GPU backend:

```
uv pip install torch torchvision torchaudio
```

Verify (works on any platform):

```
import torch
print(f"CUDA available: {torch.cuda.is_available()}")           # False on macOS – expected
print(f"MPS available: {torch.backends.mps.is_available()}")    # True on Apple Silicon
if torch.cuda.is_available():
    print(f"GPU: {torch.cuda.get_device_name(0)}")
```

No GPU? No problem. Most lessons work on CPU. For training-heavy lessons, use Google Colab or cloud GPUs.

Step 7: Verify Everything

Run the verification script:

```
python phases/00-setup-and-tooling/01-dev-environment/code/verify.py
```

Use It

Your environment is now ready for every lesson in this course. Here's what you'll use where:

Language	Used In	Package Manager
Python	Phases 1-12 (ML, DL, NLP, Vision, Audio, LLMs)	uv
TypeScript	Phases 13-17 (Tools, Agents, Swarms, Infra)	pnpm
Rust	Phases 12, 15-17 (Performance-critical systems)	cargo
Julia	Phase 1 (Math foundations)	Pkg

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/00-setup-and-tooling/01-dev-environment>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/00-setup-and-tooling/01-dev-environment/code>

1. Run the verification script and fix any failures
2. Create a Python virtual environment for this course and install PyTorch
3. Write a "hello world" in all four languages and run each one

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/00-setup-and-tooling/01-dev-environment>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/00-setup-and-tooling/01-dev-environment/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/00-setup-and-tooling/01-dev-environment>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Git & Collaboration

Version control is not optional. Every experiment, every model, every lesson you build here gets tracked.

Type: Learn **Languages:** - **Prerequisites:** Phase 0, Lesson 01 **Time:** ~30 minutes

Learning Objectives

- Configure git identity and use the daily workflow of add, commit, and push
- Create and merge branches for isolated experiments without breaking main
- Write a .gitignore that excludes model checkpoints and large binary files
- Navigate the commit history with `git log` to understand project evolution

The Problem

You're about to write hundreds of code files across 20 phases. Without version control you will lose work, break things you can't undo, and have no way to collaborate with others.

Git is the tool. GitHub is where the code lives. This lesson covers what you need for this course and nothing more.

The Concept

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/00-setup-and-tooling/02-git-and-collaboration>

Three things to remember: 1. Save often (`git commit`) 2. Push to remote (`git push`) 3. Branch for experiments (`git checkout -b experiment`)

Interactive figure: s0-commit-dag. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/00-setup-and-tooling/02-git-and-collaboration>

Build It

Step 1: Configure git

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Step 2: The daily workflow

```
git status
git add file.py
```

```
git commit -m "Add perceptron implementation"
git push origin main
```

Step 3: Branching for experiments

```
git checkout -b experiment/new-optimizer
```

... make changes, commit ...

```
git checkout main
git merge experiment/new-optimizer
```

Step 4: Working with this course repo

You can't push to the course repo itself — only maintainers have write access. Fork it on GitHub first (the Fork button, top right) so origin points at your own copy:

```
git clone https://github.com/YOUR-USERNAME/ai-engineering-from-scratch.git
cd ai-engineering-from-scratch
```

```
git checkout -b my-progress
# work through lessons, commit your code
git push origin my-progress
```

Use It

For this course, you need exactly these commands:

Command	When
git clone	Get the course repo
git add + git commit	Save your work
git push	Back it up to GitHub
git checkout -b	Try something without breaking main
git log --oneline	See what you've done

That's it. You don't need rebase, cherry-pick, or submodules for this course.

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/00-setup-and-tooling/02-git-and-collaboration/code>

1. Fork this repo, clone your fork, create a branch called my-progress, make a file, commit it, push it
2. Create a .gitignore that excludes model checkpoint files (.pt, .pth, .safetensors)
3. Look at the commit history of this repo with `git log --oneline` and read how lessons were added

Key Terms

Term	What people say	What it actually means
Commit	“Saving”	A snapshot of your entire project at a point in time
Branch	“A copy”	A pointer to a commit that moves forward as you work
Merge	“Combining code”	Taking changes from one branch and applying them to another
Remote	“The cloud”	A copy of your repo hosted somewhere else (GitHub, GitLab)

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/00-setup-and-tooling/02-git-and-collaboration>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/00-setup-and-tooling/02-git-and-collaboration/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/00-setup-and-tooling/02-git-and-collaboration>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

GPU Setup & Cloud

Training on CPU is fine for learning. Training for real needs a GPU.

Type: Build **Languages:** Python **Prerequisites:** Phase 0, Lesson 01 **Time:** ~45 minutes

Learning Objectives

- Verify local GPU availability using `nvidia-smi` and PyTorch's CUDA API
- Configure Google Colab with a T4 GPU for free cloud-based experiments
- Benchmark matrix multiplication on CPU vs GPU and measure the speedup
- Estimate the largest model that fits in your VRAM using the fp16 rule of thumb

The Problem

Most lessons in phases 1-3 run fine on CPU. But once you start training CNNs, transformers, or LLMs (phases 4+), you need GPU acceleration. A training run that takes 8 hours on CPU takes 10 minutes on GPU.

You have three options: local GPU, cloud GPU, or Google Colab (free).

The Concept

Your options:

1. Local NVIDIA GPU
Cost: \$0 (you already have it)
Setup: Install CUDA + cuDNN
Best for: Regular use, large datasets
2. Google Colab (free tier)
Cost: \$0
Setup: None
Best for: Quick experiments, no GPU at home
3. Cloud GPU (Lambda, RunPod, Vast.ai)
Cost: \$0.20-2.00/hr
Setup: SSH + install
Best for: Serious training, large models

Interactive figure: s0-gpu-dispatch. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/00-setup-and-tooling/03-gpu-setup-and-cloud>

Build It

Option 1: Local NVIDIA GPU

Check if you have one:

```
nvidia-smi
```

Install PyTorch with CUDA:

```
import torch

print(f"CUDA available: {torch.cuda.is_available()}")
print(f"CUDA version: {torch.version.cuda}")
if torch.cuda.is_available():
    print(f"GPU: {torch.cuda.get_device_name(0)}")
    print(f"Memory: {torch.cuda.get_device_properties(0).total_memory / 1e9:.1f} GB")
```

Option 2: Google Colab

1. Go to colab.research.google.com
2. Runtime > Change runtime type > T4 GPU
3. Run `!nvidia-smi` to verify

Upload notebooks from this course directly to Colab.

Option 3: Cloud GPU

For Lambda Labs, RunPod, or Vast.ai:

```
ssh user@your-gpu-instance
```

```
pip install torch torchvision torchaudio
python -c "import torch; print(torch.cuda.get_device_name(0))"
```

No GPU? No problem.

Most lessons work on CPU. The ones that need GPU will say so and include Colab links.

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using: {device}")
```

Build It: GPU vs CPU benchmark

```
import torch
import time

size = 5000

a_cpu = torch.randn(size, size)
b_cpu = torch.randn(size, size)

start = time.time()
c_cpu = a_cpu @ b_cpu
cpu_time = time.time() - start
```



```
print(f"CPU: {cpu_time:.3f}s")

if torch.cuda.is_available():
    a_gpu = a_cpu.to("cuda")
    b_gpu = b_cpu.to("cuda")

    torch.cuda.synchronize()
    start = time.time()
    c_gpu = a_gpu @ b_gpu
    torch.cuda.synchronize()
    gpu_time = time.time() - start
    print(f"GPU: {gpu_time:.3f}s")
    print(f"Speedup: {cpu_time / gpu_time:.0f}x")
```

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/00-setup-and-tooling/03-gpu-setup-and-cloud/code>

1. Run the benchmark above and compare CPU vs GPU times
2. If you don't have a GPU, run it on Google Colab and compare
3. Check how much GPU memory you have and estimate the largest model you can fit (rule of thumb: 2 bytes per parameter for fp16)

Key Terms

Term	What people say	What it actually means
CUDA	"GPU programming"	NVIDIA's parallel computing platform that lets you run code on the GPU
VRAM	"GPU memory"	Video RAM on the GPU, separate from system RAM. Limits model size.
fp16	"Half precision"	16-bit floating point, uses half the memory of fp32 with minimal accuracy loss
Tensor Core	"Fast matrix hardware"	Specialized GPU cores for matrix multiplication, 4-8x faster than regular cores

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/00-setup-and-tooling/03-gpu-setup-and-cloud>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/00-setup-and-tooling/03-gpu-setup-and-cloud/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/00-setup-and-tooling/03-gpu-setup-and-cloud>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

APIs & Keys

Every AI API works the same way: send a request, get a response. The details change, the pattern doesn't.

Type: Build **Languages:** Python, TypeScript **Prerequisites:** Phase 0, Lesson 01 **Time:** ~30 minutes

Learning Objectives

- Store API keys securely using environment variables and `.env` files
- Make an LLM API call using both the Anthropic Python SDK and raw HTTP
- Compare SDK-based and raw HTTP request/response formats for debugging
- Identify and handle common API errors including authentication and rate limits

The Problem

Starting from Phase 11, you'll call LLM APIs (Anthropic, OpenAI, Google). In Phase 13-16 you'll build agents that use these APIs in loops. You need to know how API keys work, how to store them safely, and how to make your first API call.

The Concept

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/00-setup-and-tooling/04-apis-and-keys>

Every API call has: 1. An endpoint (URL) 2. An API key (authentication) 3. A request body (what you want) 4. A response body (what you get back)

Interactive figure: s0-secret-inject. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/00-setup-and-tooling/04-apis-and-keys>

Build It

Step 1: Store API keys safely

Never put API keys in code. Use environment variables.

```
export ANTHROPIC_API_KEY="sk-ant-..."
export OPENAI_API_KEY="sk-..."
```

Or use a `.env` file (add it to `.gitignore`):

```
ANTHROPIC_API_KEY=sk-ant-...
OPENAI_API_KEY=sk-...
```

Step 2: First API call (Python)

```
import os

import anthropic

client = anthropic.Anthropic()

MODEL = os.environ.get("LLM_MODEL", "claude-sonnet-5")

response = client.messages.create(
    model=MODEL,
    max_tokens=256,
    messages=[{"role": "user", "content": "What is a neural network in one sentence?"}]
)

print(response.content[0].text)
```

LLM_MODEL selects the Anthropic model id, and the default is the un-dated Sonnet alias. Other providers (OpenAI, Google, and others) follow the same pattern of a key plus a model id, but each has its own SDK, endpoint, and request/response schema.

Step 3: First API call (TypeScript)

```
import Anthropic from "@anthropic-ai/sdk";

const client = new Anthropic();

const MODEL = process.env.LLM_MODEL ?? "claude-sonnet-5";

const response = await client.messages.create({
  model: MODEL,
  max_tokens: 256,
  messages: [{ role: "user", content: "What is a neural network in one sentence?" }],
});

console.log(response.content[0].text);
```

Step 4: Raw HTTP (no SDK)

```
import os
import urllib.request
import json

url = "https://api.anthropic.com/v1/messages"
headers = {
  "Content-Type": "application/json",
  "x-api-key": os.environ["ANTHROPIC_API_KEY"],
  "anthropic-version": "2023-06-01",
}
body = json.dumps({
  "model": os.environ.get("LLM_MODEL", "claude-sonnet-5"),
  "max_tokens": 256,
  "messages": [{"role": "user", "content": "What is a neural network in one sentence?"}],
})
```

```

}).encode()

req = urllib.request.Request(url, data=body, headers=headers, method="POST")
with urllib.request.urlopen(req) as resp:
    result = json.loads(resp.read())
    print(result["content"][0]["text"])

```

This is what the SDKs do under the hood. Understanding the raw HTTP call helps when debugging.

Use It

For this course:

API	When you need it	Free tier
Anthropic (Claude)	Phases 11-16 (agents, tools)	\$5 credit on signup
OpenAI	Phase 11 (comparison)	\$5 credit on signup
Hugging Face	Phases 4-10 (models, datasets)	Free

You don't need all of them right now. Set them up when the lesson requires it.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/00-setup-and-tooling/04-apis-and-keys>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/00-setup-and-tooling/04-apis-and-keys/code>

1. Get an Anthropic API key and make your first API call
2. Try the raw HTTP version and compare the response format to the SDK version
3. Intentionally use a wrong API key and read the error message

Key Terms

Term	What people say	What it actually means
API key	"Password for the API"	A unique string that identifies your account and authorizes requests
Rate limit	"They're throttling me"	Maximum requests per minute/hour to prevent abuse and ensure fair usage
Token	"A word" (in API context)	A billing unit: input and output tokens are counted and charged separately
Streaming	"Real-time responses"	Getting the response word by word instead of waiting for the full response

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/00-setup-and-tooling/04-apis-and-keys>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/00-setup-and-tooling/04-apis-and-keys/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/00-setup-and-tooling/04-apis-and-keys>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Jupyter Notebooks

Notebooks are the lab bench of AI engineering. You prototype here, then move what works into production.

Type: Build **Languages:** Python **Prerequisites:** Phase 0, Lesson 01 **Time:** ~30 minutes

Learning Objectives

- Install and launch JupyterLab, Jupyter Notebook, or VS Code with the Jupyter extension
- Use magic commands (`%timeit`, `%time`, `%matplotlib inline`) to benchmark and visualize inline
- Distinguish when to use notebooks vs scripts and apply the “explore in notebooks, ship in scripts” workflow
- Identify and avoid common notebook traps: out-of-order execution, hidden state, and memory leaks

The Problem

Every AI paper, tutorial, and Kaggle competition uses Jupyter notebooks. They let you run code in pieces, see outputs inline, mix code with explanations, and iterate fast. If you try to learn AI without notebooks, you’re doing math homework without scratch paper.

But notebooks have real traps. People use them for everything, including things they’re terrible at. Knowing when to use a notebook and when to use a script will save you from debugging nightmares later.

The Concept

A notebook is a list of cells. Each cell is either code or text.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/00-setup-and-tooling/05-jupyter-notebooks>

The kernel is a Python process running in the background. When you run a cell, it sends the code to the kernel, which executes it and sends back the result. All cells share the same kernel, so variables persist between cells.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/00-setup-and-tooling/05-jupyter-notebooks>

That “whatever order you click” part is both the superpower and the foot-gun.

Interactive figure: s0-cell-order. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/00-setup-and-tooling/05-jupyter-notebooks>

Build It

Step 1: Pick your interface

Three options, one format:

Interface	Install	Best for
JupyterLab	<code>pip install jupyterlab</code> then <code>jupyter lab</code>	Full IDE experience, multiple tabs, file browser, terminal
Jupyter Notebook	<code>pip install notebook</code> then <code>jupyter notebook</code>	Simple, lightweight, one notebook at a time
VS Code	Install “Jupyter” extension	Already in your editor, git integration, debugging

All three read and write the same `.ipynb` file. Pick whatever you like. JupyterLab is the most common in AI work.

```
pip install jupyterlab
jupyter lab
```

Step 2: Keyboard shortcuts that matter

You operate in two modes. Press Escape for command mode (blue bar on the left), Enter for edit mode (green bar).

Command mode (most used):

Key	Action
Shift+Enter	Run cell, move to next
A	Insert cell above
B	Insert cell below
DD	Delete cell
M	Convert to markdown
Y	Convert to code
Z	Undo cell operation
Ctrl+Shift+H	Show all shortcuts

Edit mode:

Key	Action
Tab	Autocomplete
Shift+Tab	Show function signature
Ctrl+/	Toggle comment

Shift+Enter is the one you’ll use a thousand times a day. Learn it first.

Step 3: Cell types

Code cells run Python and show the output:

```
import numpy as np
data = np.random.randn(1000)
data.mean(), data.std()
```

Output: (0.0032, 0.9987)

Markdown cells render formatted text. Use them to document what you're doing and why. Supports headers, bold, italic, LaTeX math ($E = mc^2$), tables, and images.

Step 4: Magic commands

These aren't Python. They're Jupyter-specific commands that start with % (line magic) or %% (cell magic).

Time your code:

```
%timeit np.random.randn(10000)
```

Output: 45.2 us +/- 1.3 us per loop

```
%%time
model.fit(X_train, y_train, epochs=10)
```

Output: Wall time: 2.34 s

%timeit runs the code many times and averages. %%time runs it once. Use %timeit for microbenchmarks, %%time for training runs.

Enable inline plots:

```
%matplotlib inline
```

Every plt.plot() or plt.show() now renders directly in the notebook.

Install packages without leaving the notebook:

```
!pip install scikit-learn
```

The ! prefix runs any shell command.

Check environment variables:

```
%env CUDA_VISIBLE_DEVICES
```

Step 5: Display rich output inline

Notebooks auto-display the last expression in a cell. But you can control it:

```
import pandas as pd
```

```
df = pd.DataFrame({
    "model": ["Linear", "Random Forest", "Neural Net"],
    "accuracy": [0.72, 0.89, 0.94],
    "training_time": [0.1, 2.3, 45.6]
})
df
```

This renders a formatted HTML table, not a text dump. Same with plots:

```
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(8, 4))
plt.plot([1, 2, 3, 4], [1, 4, 2, 3])
```



```
plt.title("Inline Plot")
plt.show()
```

The plot appears right below the cell. This is why notebooks dominate AI work. You see the data, the plot, and the code together.

For images:

```
from IPython.display import Image, display
display(Image(filename="architecture.png"))
```

Step 6: Google Colab

Colab is a free Jupyter notebook in the cloud. It gives you a GPU, pre-installed libraries, and Google Drive integration. No setup required.

1. Go to colab.research.google.com
2. Upload any .ipynb file from this course
3. Runtime > Change runtime type > T4 GPU (free)

Colab differences from local Jupyter: - Files don't persist between sessions (save to Drive or download) - Pre-installed: numpy, pandas, matplotlib, torch, tensorflow, sklearn - from google.colab import files to upload/download files - from google.colab import drive; drive.mount('/content/drive') for persistent storage - Sessions time out after 90 minutes of inactivity (free tier)

Use It

Notebooks vs Scripts: When to use which

Use notebooks for	Use scripts for
Exploring a dataset	Training pipelines
Prototyping a model	Reusable utilities
Visualizing results	Anything with <code>if __name__</code>
Explaining your work	Code that runs on a schedule
Quick experiments	Production code
Course exercises	Packages and libraries

The rule: **explore in notebooks, ship in scripts.**

A common workflow in AI: 1. Explore data in a notebook 2. Prototype your model in the notebook 3. Once it works, move the code to .py files 4. Import those .py files back into the notebook for further experiments

Common traps

Out-of-order execution. You run cell 5, then cell 2, then cell 7. The notebook works on your machine but breaks when someone runs it top to bottom. Fix: Kernel > Restart & Run All before sharing.

Hidden state. You delete a cell but the variable it created is still in memory. The notebook looks clean but depends on a ghost cell. Fix: Restart the kernel regularly.

Memory leaks. Loading a 4GB dataset, training a model, loading another dataset. Nothing gets freed. Fix: `del variable_name` and `gc.collect()`, or restart the kernel.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/00-setup-and-tooling/05-jupyter-notebooks>

Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/00-setup-and-tooling/05-jupyter-notebooks/code>

1. Open JupyterLab, create a notebook, and use `%timeit` to compare list comprehension vs numpy for creating an array of 100,000 random numbers
2. Create a notebook with both markdown and code cells that loads a CSV, displays a dataframe, and plots a chart. Then run Kernel > Restart & Run All to verify it works top to bottom
3. Take the code from `code/notebook_tips.py`, paste it into a Colab notebook, and run it with a free GPU

Key Terms

Term	What people say	What it actually means
Kernel	“The thing running my code”	A separate Python process that executes cells and keeps variables in memory
Cell	“A code block”	An independently runnable unit in a notebook, either code or markdown
Magic command	“Jupyter tricks”	Special commands prefixed with <code>%</code> or <code>%%</code> that control the notebook environment
.ipynb	“Notebook file”	A JSON file containing cells, outputs, and metadata. Stands for IPython Notebook

Further Reading

- JupyterLab Docs for the full feature set
- Google Colab FAQ for Colab-specific limits and features
- 28 Jupyter Notebook Tips for power-user shortcuts

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/00-setup-and-tooling/05-jupyter-notebooks>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/00-setup-and-tooling/05-jupyter-notebooks/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/00-setup-and-tooling/05-jupyter-notebooks>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Python Environments

Dependency hell is real. Virtual environments are the cure.

Type: Build **Languages:** Shell **Prerequisites:** Phase 0, Lesson 01 **Time:** ~30 minutes

Learning Objectives

- Create isolated virtual environments using `uv`, `venv`, or `conda`
- Write a `pyproject.toml` with optional dependency groups and generate lockfiles for reproducibility
- Diagnose and fix common pitfalls: global installs, `pip`/`conda` mixing, CUDA version mismatches
- Implement a per-phase environment strategy for projects with conflicting dependencies

The Problem

You install PyTorch 2.4 for a fine-tuning project. Next week, a different project needs PyTorch 2.1 because its CUDA build is pinned. You upgrade globally, and the first project breaks. You downgrade, and the second one breaks.

This is dependency hell. It happens constantly in AI/ML work because:

- PyTorch, JAX, and TensorFlow each ship their own CUDA bindings
- Model libraries pin specific framework versions
- A global `pip install` overwrites whatever was there before
- CUDA 11.8 builds don't work with CUDA 12.x drivers (and vice versa)

The fix: every project gets its own isolated environment with its own packages.

The Concept

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/00-setup-and-tooling/06-python-environments>

Interactive figure: `s0-env-isolation`. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/00-setup-and-tooling/06-python-environments>

Build It

Option 1: `uv venv` (Recommended)

`uv` is the fastest Python package manager (10-100x faster than `pip`). It handles virtual environments, Python versions, and dependency resolution in one tool.

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

```
uv python install 3.12
```

```
cd your-project
uv venv
source .venv/bin/activate
```

Install packages:

```
uv pip install torch numpy
```

Create a project with `pyproject.toml` in one step:

```
uv init my-ai-project
cd my-ai-project
uv add torch numpy matplotlib
```

Option 2: venv (Built-in)

If you can't install `uv`, Python ships with `venv`:

```
python3 -m venv .venv
source .venv/bin/activate # Linux/macOS
.venv\Scripts\activate   # Windows
```

```
pip install torch numpy
```

Slower than `uv`, but works everywhere Python is installed.

Option 3: conda (When You Need It)

Conda manages non-Python dependencies like CUDA toolkits, cuDNN, and C libraries. Use it when:

- You need a specific CUDA toolkit version without installing it system-wide
- You're on a shared cluster where you can't install system packages
- A library's install instructions say "use conda"

Install miniconda (not the full Anaconda)

```
curl -LsSf https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh -o miniconda.s
bash miniconda.sh -b
```

```
conda create -n myproject python=3.12
conda activate myproject
```

```
conda install pytorch torchvision torchaudio pytorch-cuda=12.4 -c pytorch -c nvidia
```

One rule: if you use `conda` for an environment, use `conda` for all packages in that environment. Mixing `pip install` into a `conda` env causes dependency conflicts that are painful to debug.

For This Course: Per-Phase Strategy

You could create one environment for the whole course. Don't. Different phases need different (sometimes conflicting) dependencies.

Strategy:

```

ai-engineering-from-scratch/
├── .venv/                                <-- shared lightweight env for phases 0-3
├── phases/
│   ├── 04-neural-networks/
│   │   └── .venv/                        <-- PyTorch env
│   ├── 05-cnns/
│   │   └── .venv/                        <-- same PyTorch env (symlink or shared)
│   ├── 08-transformers/
│   │   └── .venv/                        <-- might need different transformer versions
│   └── 11-llm-apis/
│       └── .venv/                        <-- API SDKs, no torch needed

```

The script in `code/env_setup.sh` creates the base environment for this course.

pyproject.toml Basics

Every Python project should have a `pyproject.toml`. It replaces `setup.py`, `setup.cfg`, and `requirements.txt` in one file.

```

[project]
name = "ai-engineering-from-scratch"
version = "0.1.0"
requires-python = ">=3.11"
dependencies = [
    "numpy>=1.26",
    "matplotlib>=3.8",
    "jupyter>=1.0",
    "scikit-learn>=1.4",
]

[project.optional-dependencies]
torch = ["torch>=2.3", "torchvision>=0.18"]
llm = ["anthropic>=0.39", "openai>=1.50"]

```

Then install:

```

uv pip install -e ".[torch]"      # base + PyTorch
uv pip install -e ".[llm]"        # base + LLM SDKs
uv pip install -e ".[torch,llm]"  # everything

```

Lockfiles

A lockfile pins every dependency (including transitive ones) to exact versions. This guarantees reproducibility: anyone who installs from the lockfile gets exactly the same packages.

```

# uv generates uv.lock automatically when using uv add
uv add numpy

```

```

# pip-tools approach
uv pip compile pyproject.toml -o requirements.lock
uv pip install -r requirements.lock

```

Commit your lockfile to git. When someone clones the repo, they install from the lockfile and get identical versions.

Common Mistakes

1. Installing globally

```
pip install torch # BAD: installs to system Python
```

```
source .venv/bin/activate
```

```
pip install torch # GOOD: installs to virtual environment
```

Check where your packages go:

```
which python # should show .venv/bin/python, not /usr/bin/python
```

```
which pip # should show .venv/bin/pip
```

2. Mixing pip and conda

```
conda create -n myenv python=3.12
```

```
conda activate myenv
```

```
conda install pytorch -c pytorch
```

```
pip install some-other-package # BAD: can break conda's dependency tracking
```

```
conda install some-other-package # GOOD: let conda manage everything
```

If you must use pip inside conda (some packages are pip-only), install all conda packages first, then pip packages last.

3. Forgetting to activate

```
python train.py # uses system Python, missing packages
```

```
source .venv/bin/activate
```

```
python train.py # uses project Python, packages found
```

Your shell prompt should show the environment name:

```
(.venv) $ python train.py
```

4. Committing .venv to git

```
echo ".venv/" >> .gitignore
```

Virtual environments are 200MB-2GB. They're local, not portable between machines. Commit `pyproject.toml` and the lockfile instead.

5. CUDA version mismatch

```
nvidia-smi # shows driver CUDA version (e.g., 12.4)
```

```
python -c "import torch; print(torch.version.cuda)" # shows PyTorch CUDA version
```

```
# These must be compatible.
```

```
# PyTorch CUDA version must be <= driver CUDA version.
```

Use It

Run the setup script to create your course environment:

```
bash phases/00-setup-and-tooling/06-python-environments/code/env_setup.sh
```

This creates a `.venv` at the repo root with core dependencies installed and verified.

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/00-setup-and-tooling/06-python-environments/code>

1. Run `env_setup.sh` and verify all checks pass
2. Create a second virtual environment, install a different version of `numpy` in it, and confirm the two environments are isolated
3. Write a `pyproject.toml` for a project that needs both `PyTorch` and the `Anthropic SDK`
4. Deliberately install a package globally (without activating a `venv`), notice where it goes, then uninstall it

Key Terms

Term	What people say	What it actually means
Virtual environment	"A <code>venv</code> "	An isolated directory containing a Python interpreter and packages, separate from the system Python
Lockfile	"Pinned dependencies"	A file listing every package and its exact version, guaranteeing identical installs across machines
<code>pyproject.toml</code>	"The new <code>setup.py</code> "	The standard Python project configuration file, replacing <code>setup.py/setup.cfg/requirements.txt</code>
Transitive dependency	"A dependency of a dependency"	Package B depends on C; if you install A which depends on B, C is a transitive dependency of A
CUDA mismatch	"My GPU isn't working"	<code>PyTorch</code> was compiled for a different CUDA version than what your GPU driver supports

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/00-setup-and-tooling/06-python-environments>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/00-setup-and-tooling/06-python-environments/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/00-setup-and-tooling/06-python-environments>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Docker for AI

Containers make “works on my machine” a thing of the past.

Type: Build **Languages:** Docker **Prerequisites:** Phase 0, Lessons 01 and 03 **Time:** ~60 minutes

Learning Objectives

- Build a GPU-enabled Docker image with CUDA, PyTorch, and AI libraries from a Dockerfile
- Mount host directories as volumes to persist models, datasets, and code across container rebuilds
- Configure the NVIDIA Container Toolkit to expose GPUs inside containers
- Orchestrate multi-service AI applications (inference server + vector database) using Docker Compose

The Problem

You trained a model on your laptop with PyTorch 2.3, CUDA 12.4, and Python 3.12. Your colleague has PyTorch 2.1, CUDA 11.8, and Python 3.10. Your model crashes on their machine. Your Dockerfile works on both.

AI projects are dependency nightmares. A typical stack includes Python, PyTorch, CUDA drivers, cuDNN, system-level C libraries, and specialized packages like flash-attn that need exact compiler versions. Docker packages all of this into a single image that runs identically everywhere.

The Concept

Docker wraps your code, runtime, libraries, and system tools into an isolated unit called a container. Think of it as a lightweight virtual machine, except it shares the host OS kernel instead of running its own, so it starts in seconds instead of minutes.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/00-setup-and-tooling/07-docker-for-ai>

Why AI projects need Docker more than most

1. **GPU drivers are fragile.** CUDA 12.4 code does not run on CUDA 11.8. Docker isolates the CUDA toolkit inside the container while sharing the host GPU driver through the NVIDIA Container Toolkit.
2. **Model weights are large.** A 7B parameter model is 14 GB in fp16. You do not want to re-download it every time you rebuild. Docker volumes let you mount a models directory from the host.

3. **Multi-service architectures are common.** A real AI application is not just a Python script. It is an inference server, a vector database for RAG, maybe a web frontend. Docker Compose orchestrates all of these with one command.

Key vocabulary

Term	What it means
Image	A read-only template. Your recipe. Built from a Dockerfile.
Container	A running instance of an image. Your kitchen.
Dockerfile	Instructions to build an image. Layer by layer.
Volume	Persistent storage that survives container restarts.
docker-compose	A tool for defining multi-container applications in YAML.

Common container patterns in AI

Dev Container

Full toolkit. Editor support. Jupyter. Debugging tools.
Used during development and experimentation.

Training Container

Minimal. Just the training script and dependencies.
Runs on GPU clusters. No editor, no Jupyter.

Inference Container

Optimized for serving. Small image. Fast cold start.
Runs behind a load balancer in production.

Interactive figure: s0-image-layers. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/00-setup-and-tooling/07-docker-for-ai>

Build It

Step 1: Install Docker

macOS

```
brew install --cask docker
open /Applications/Docker.app
```

Ubuntu

```
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker $USER
# Log out and back in for group change to take effect
```

Verify:

```
docker --version
docker run hello-world
```

Step 2: Install NVIDIA Container Toolkit (Linux with NVIDIA GPU)

This lets Docker containers access your GPU. macOS and Windows (WSL2) users can skip this; Docker Desktop handles GPU passthrough differently on those platforms.

```
distribution=$(. /etc/os-release;echo $ID$VERSION_ID)
curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey | sudo gpg --dearmor -o /usr/share
curl -s -L https://nvidia.github.io/libnvidia-container/$distribution/libnvidia-container.list
    sed 's#deb https://#deb [signed-by=/usr/share/keyrings/nvidia-container-toolkit-keyring.gpg
    sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.list
```

```
sudo apt-get update
sudo apt-get install -y nvidia-container-toolkit
sudo nvidia-ctk runtime configure --runtime=docker
sudo systemctl restart docker
```

Test GPU access inside a container:

```
docker run --rm --gpus all nvidia/cuda:12.4.1-base-ubuntu22.04 nvidia-smi
```

If you see your GPU info, the toolkit is working.

Step 3: Understand base images

Choosing the right base image saves hours of debugging.

```
nvidia/cuda:12.4.1-devel-ubuntu22.04
  Full CUDA toolkit. Compilers included.
  Use for: building packages that need nvcc (flash-attn, bitsandbytes)
  Size: ~4 GB
```

```
nvidia/cuda:12.4.1-runtime-ubuntu22.04
  CUDA runtime only. No compilers.
  Use for: running pre-built code
  Size: ~1.5 GB
```

```
pytorch/pytorch:2.6.0-cuda12.4-cudnn9-runtime
  PyTorch pre-installed on top of CUDA.
  Use for: skipping the PyTorch install step
  Size: ~6 GB
```

```
python:3.12-slim
  No CUDA. CPU only.
  Use for: inference on CPU, lightweight tools
  Size: ~150 MB
```

Step 4: Write a Dockerfile for AI development

Here is the Dockerfile in code/Dockerfile. Walk through it:

```
FROM nvidia/cuda:12.4.1-devel-ubuntu22.04

ENV DEBIAN_FRONTEND=noninteractive
ENV PYTHONUNBUFFERED=1

RUN apt-get update && apt-get install -y --no-install-recommends \
    software-properties-common \
    git \
    curl \
    build-essential \
    && add-apt-repository -y ppa:deadsnakes/ppa \
```

```

    && apt-get update && apt-get install -y --no-install-recommends \
    python3.12 \
    python3.12-venv \
    python3.12-dev \
    && rm -rf /var/lib/apt/lists/*

RUN update-alternatives --install /usr/bin/python python /usr/bin/python3.12 1

RUN curl -sSL https://raw.githubusercontent.com/pypa/get-pip/3b73145063be545b649ad9ca83ea8da5fc
    && echo "a341e1a43e38001c551a1508a73ff23636a11970b61d901d9a1cad2a18f57055 /tmp/get-pip.py"
    && python /tmp/get-pip.py \
    && rm /tmp/get-pip.py \
    && update-alternatives --install /usr/bin/pip pip /usr/local/bin/pip3.12 1

RUN python -m pip install --no-cache-dir --upgrade pip setuptools wheel

RUN python -m pip install --no-cache-dir \
    torch==2.6.0+cu124 \
    torchvision==0.21.0+cu124 \
    torchaudio==2.6.0+cu124 \
    --index-url https://download.pytorch.org/whl/cu124

RUN python -m pip install --no-cache-dir \
    numpy \
    pandas \
    scikit-learn \
    matplotlib \
    jupyter \
    transformers \
    datasets \
    accelerate \
    safetensors

WORKDIR /workspace

VOLUME ["/workspace", "/models"]

EXPOSE 8888

CMD ["python"]

```

Build it:

```
docker build -t ai-dev -f phases/00-setup-and-tooling/07-docker-for-ai/code/Dockerfile .
```

This takes a while the first time (downloading CUDA base image + PyTorch). Subsequent builds use cached layers.

Run it:

```
docker run --rm -it --gpus all \
    -v $(pwd):/workspace \
    -v ~/models:/models \
    ai-dev python -c "import torch; print(f'PyTorch {torch.__version__}, CUDA: {torch.cuda.is_a
```

Run Jupyter inside the container:

```
docker run --rm -it --gpus all \
  -v $(pwd):/workspace \
  -v ~/models:/models \
  -p 8888:8888 \
  ai-dev jupyter notebook --ip=0.0.0.0 --port=8888 --no-browser --allow-root
```

Step 5: Volume mounts for data and models

Volume mounts are critical for AI work. Without them, your 14 GB model downloads vanish when the container stops.

```
# Mount your code
-v $(pwd):/workspace

# Mount a shared models directory
-v ~/models:/models

# Mount datasets
-v ~/datasets:/data
```

Inside your training script, load from the mounted path:

```
from transformers import AutoModel
```

```
model = AutoModel.from_pretrained("/models/llama-7b")
```

The model lives on your host filesystem. Rebuild the container as often as you want without re-downloading.

Step 6: Docker Compose for multi-service AI apps

A real RAG application needs an inference server and a vector database. Docker Compose runs both with one command.

See code/docker-compose.yml:

```
services:
  ai-dev:
    build:
      context: .
      dockerfile: Dockerfile
    deploy:
      resources:
        reservations:
          devices:
            - driver: nvidia
              count: all
              capabilities: [gpu]
    volumes:
      - ../../../../:/workspace
      - ~/models:/models
      - ~/datasets:/data
    ports:
      - "8888:8888"
    stdin_open: true
    tty: true
    command: jupyter notebook --ip=0.0.0.0 --port=8888 --no-browser --allow-root
```

```

qdrant:
  image: qdrant/qdrant:v1.12.5
  ports:
    - "6333:6333"
    - "6334:6334"
  volumes:
    - qdrant_data:/qdrant/storage

```

```

volumes:
  qdrant_data:

```

Start everything:

```

cd phases/00-setup-and-tooling/07-docker-for-ai/code
docker compose up -d

```

Now your AI dev container can reach the vector database at <http://qdrant:6333> by service name. Docker Compose creates a shared network automatically.

Test the connection from inside the AI container:

```

from qdrant_client import QdrantClient

client = QdrantClient(host="qdrant", port=6333)
print(client.get_collections())

```

Stop everything:

```

docker compose down

```

Add -v to also delete the qdrant volume:

```

docker compose down -v

```

Step 7: Useful Docker commands for AI work

```

# List running containers
docker ps

```

```

# List all images and their sizes
docker images

```

```

# Remove unused images (reclaim disk space)
docker system prune -a

```

```

# Check GPU usage inside a running container
docker exec -it <container_id> nvidia-smi

```

```

# Copy a file from container to host
docker cp <container_id>:/workspace/results.csv ./results.csv

```

```

# View container logs
docker logs -f <container_id>

```

Use It

You now have a reproducible AI development environment. For the rest of this course:

- Use `docker compose up` to start your dev environment and vector database together
- Mount your code, models, and data as volumes so nothing is lost between rebuilds
- When a lesson requires a new Python package, add it to the Dockerfile and rebuild
- Share your Dockerfile with teammates. They get the exact same environment.

No GPU?

Remove the `--gpus all` flag and the NVIDIA deploy block. The container still works for CPU-based lessons. PyTorch detects the absence of CUDA and falls back to CPU automatically.

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/00-setup-and-tooling/07-docker-for-ai/code>

1. Build the Dockerfile and run `python -c "import torch; print(torch.__version__)"` inside the container
2. Start the docker-compose stack and verify Qdrant is accessible from the AI container at <http://qdrant:6333/collections>
3. Add flask to the Dockerfile, rebuild, and run a simple API server on port 5000. Map the port with `-p 5000:5000`
4. Measure the image size with `docker images`. Try switching the base image from `devel` to `runtime` and compare sizes

Key Terms

Term	What people say	What it actually means
Container	"Lightweight VM"	An isolated process using the host kernel, with its own filesystem and network
Image layer	"Cached step"	Each Dockerfile instruction creates a layer. Unchanged layers are cached, so rebuilds are fast.
NVIDIA Container Toolkit	"GPU in Docker"	A runtime hook that exposes host GPUs to containers via <code>--gpus</code> flag
Volume mount	"Shared folder"	A directory on the host mapped into the container. Changes persist after the container stops.
Base image	"Starting point"	The FROM image your Dockerfile builds on top of. Determines what is pre-installed.

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/00-setup-and-tooling/07-docker-for-ai>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/00-setup-and-tooling/07-docker-for-ai/code>

- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/00-setup-and-tooling/07-docker-for-ai>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Editor Setup

Your editor is your co-pilot. Configure it once so it stays out of your way and starts pulling its weight.

Type: Build **Languages:** - **Prerequisites:** Phase 0, Lesson 01 **Time:** ~20 minutes

Learning Objectives

- Install VS Code with essential extensions for Python, Jupyter, linting, and remote SSH
- Configure format-on-save, type checking, and notebook output scrolling for AI workflows
- Set up Remote SSH to edit and debug code on remote GPU machines as if they were local
- Evaluate editor alternatives (Cursor, Windsurf, Neovim) and their tradeoffs for AI work

The Problem

You'll spend thousands of hours inside your editor writing Python, running notebooks, debugging training loops, and SSH-ing into GPU boxes. A misconfigured editor turns every session into friction: no autocomplete, no type hints, no inline errors, manual formatting, and a clunky terminal workflow.

The right setup takes 20 minutes. Skipping it costs you 20 minutes every day.

The Concept

An AI engineering editor setup needs five things:

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/00-setup-and-tooling/08-editor-setup>

Interactive figure: s0-lsp-roundtrip. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/00-setup-and-tooling/08-editor-setup>

Build It

Step 1: Install VS Code

VS Code is the recommended editor. It is free, runs on every OS, has first-class Jupyter notebook support, and the extension ecosystem covers everything you need for AI work.

Download it from code.visualstudio.com.

Verify from the terminal:

```
code --version
```


If code is not found on macOS, open VS Code, press Cmd+Shift+P, type “Shell Command”, and select “Install ‘code’ command in PATH”.

Step 2: Install Essential Extensions

Open the integrated terminal in VS Code (Ctrl+` on every platform) and install the extensions that matter for AI work:

```
code --install-extension ms-python.python
code --install-extension ms-python.vscode-pylance
code --install-extension ms-toolsai.jupyter
code --install-extension eamodio.gitlens
code --install-extension ms-vscode-remote.remote-ssh
code --install-extension ms-python.debugpy
code --install-extension ms-python.black-formatter
code --install-extension charliermarsh.ruff
```

What each one does:

Extension	Why
Python	Language support, virtual env detection, run/debug
Pylance	Fast type checking, autocomplete, import resolution
Jupyter	Run notebooks inside VS Code, variable explorer
GitLens	See who changed what, inline git blame
Remote SSH	Open a folder on a remote GPU box as if it were local
Debugpy	Step-through debugging for Python
Black Formatter	Auto-format on save, consistent style
Ruff	Fast linting, catches common mistakes

The file `code/.vscode/extensions.json` in this lesson contains the full recommendations list. When you open the project folder, VS Code will prompt you to install them.

Step 3: Configure Settings

Copy the settings from `code/.vscode/settings.json` in this lesson, or apply them manually through Settings > Open Settings (JSON).

The key settings for AI work:

```
{
  "python.analysis.typeCheckingMode": "basic",
  "editor.formatOnSave": true,
  "editor.rulers": [88, 120],
  "notebook.output.scrolling": true,
  "files.autoSave": "afterDelay"
}
```

Why these matter:

- **Type checking on basic:** Catches wrong argument types before you run. Saves debugging time on tensor shape mismatches and wrong API parameters.
- **Format on save:** Never think about formatting again. Black handles it.
- **Rulers at 88 and 120:** Black wraps at 88. The 120 marker shows when docstrings and comments are getting too long.
- **Notebook output scrolling:** Training loops print thousands of lines. Without scrolling, the output panel explodes.
- **Auto-save:** You will forget to save. Your training script will run stale code. Auto-save prevents that.

Step 4: Terminal Integration

VS Code's integrated terminal is where you run training scripts, monitor GPUs, and manage environments.

Set it up properly:

```
{
  "terminal.integrated.defaultProfile.osx": "zsh",
  "terminal.integrated.defaultProfile.linux": "bash",
  "terminal.integrated.fontSize": 13,
  "terminal.integrated.scrollback": 10000
}
```

Useful shortcuts:

Action	macOS	Linux/Windows
Toggle terminal	Ctrl+`	Ctrl+`
New terminal	Ctrl+Shift+`	Ctrl+Shift+`
Split terminal	Cmd+\	Ctrl+Shift+5

Split terminals are useful: one for running your script, one for monitoring GPU with `nvidia-smi -l 1` or `watch -n 1 nvidia-smi`.

Step 5: Remote Development (SSH into GPU Boxes)

This is the most important extension for AI work. You will run training on remote machines (cloud VMs, lab servers, Lambda, Vast.ai). Remote SSH lets you open the remote filesystem, edit files, run terminals, and debug as if everything were local.

Setup:

1. Install the Remote SSH extension (done in Step 2).
2. Press Ctrl+Shift+P (or Cmd+Shift+P), type "Remote-SSH: Connect to Host".
3. Enter `user@your-gpu-box-ip`.
4. VS Code installs its server component on the remote machine automatically.

For passwordless access, set up SSH keys:

```
ssh-keygen -t ed25519 -C "your-email@example.com"
ssh-copy-id user@your-gpu-box-ip
```

Add the host to `~/.ssh/config` for convenience:

```
Host gpu-box
  HostName 203.0.113.50
  User ubuntu
  IdentityFile ~/.ssh/id_ed25519
  ForwardAgent yes
```

Now Remote-SSH: Connect to Host > gpu-box connects instantly.

Alternatives

Cursor

cursor.com is a VS Code fork with built-in AI code generation. It uses the same extension ecosystem and settings format. If you use Cursor, everything in this lesson still applies. Import the same `settings.json` and `extensions.json`.

Windsurf

windsurf.com is another AI-first VS Code fork. Same story: same extensions, same settings format, same Remote SSH support.

Vim/Neovim

If you already use Vim or Neovim and are productive in it, stay there. The minimum setup for AI Python work:

- **pyright** or **pylsp** for type checking (via Mason or manual install)
- **nvim-lspconfig** for language server integration
- **jupyter-vim** or **molten-nvim** for notebook-like execution
- **telescope.nvim** for file/symbol search
- **none-ls.nvim** with black and ruff for formatting/linting

If you do not already use Vim, do not start now. The learning curve will compete with learning AI engineering. Use VS Code.

Use It

With this setup, your daily workflow looks like:

1. Open the project folder in VS Code (or connect via Remote SSH to a GPU box).
2. Write Python in the editor with autocomplete, type hints, and inline errors.
3. Run Jupyter notebooks inline with the Jupyter extension.
4. Use the integrated terminal for training scripts, `uv pip install`, and GPU monitoring.
5. Review changes with GitLens before committing.

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/00-setup-and-tooling/08-editor-setup/code>

1. Install VS Code and all extensions listed in Step 2
2. Copy the `settings.json` from this lesson into your VS Code config
3. Open a Python file and verify that Pylance shows type hints and Black formats on save

4. If you have access to a remote machine, set up Remote SSH and open a folder on it

Key Terms

Term	What people say	What it actually means
LSP	“Autocomplete engine”	Language Server Protocol: a standard for editors to get type info, completions, and diagnostics from a language-specific server Microsoft’s Python language server using Pyright for type checking and IntelliSense VS Code extension that runs a lightweight server on a remote machine and streams the UI to your local editor The editor runs a formatter (Black, Ruff) every time you save, so code style is always consistent
Pylance	“The Python plugin”	
Remote SSH	“Working on the server”	
Format on save	“Auto-prettier”	

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/00-setup-and-tooling/08-editor-setup>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/00-setup-and-tooling/08-editor-setup/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/00-setup-and-tooling/08-editor-setup>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Data Management

Data is the fuel. How you manage it determines how fast you go.

Type: Build **Language:** Python **Prerequisites:** Phase 0, Lesson 01 **Time:** ~45 minutes

Learning Objectives

- Load, stream, and cache datasets using the Hugging Face datasets library
- Convert between CSV, JSON, Parquet, and Arrow formats and explain their tradeoffs
- Create reproducible train/validation/test splits with fixed random seeds
- Manage large model and dataset files using .gitignore, Git LFS, or DVC

The Problem

Every AI project starts with data. You need to find datasets, download them, convert between formats, split them for training and evaluation, and version them so experiments are reproducible. Doing this manually every time is slow and error-prone. You need a repeatable workflow.

The Concept

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/00-setup-and-tooling/09-data-management>

The Hugging Face datasets library is the standard way to load data for AI work. It handles downloading, caching, format conversion, and streaming out of the box.

Interactive figure: s0-data-pipeline. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/00-setup-and-tooling/09-data-management>

Build It

Step 1: Install the datasets library

```
pip install datasets huggingface_hub
```

Step 2: Load a dataset

```
from datasets import load_dataset

dataset = load_dataset("stanfordnlp/imdb")
print(dataset)
print(dataset["train"][0])
```

This downloads the IMDB movie review dataset. After the first download, it loads from cache at `~/.cache/huggingface/datasets/`.

Step 3: Stream large datasets

Some datasets are too large to fit on disk. Streaming loads them row by row without downloading the full thing.

```
dataset = load_dataset("wikimedia/wikipedia", "20220301.en", split="train", streaming=True)

for i, example in enumerate(dataset):
    print(example["title"])
    if i >= 4:
        break
```

Streaming gives you an `IterableDataset`. You process rows as they arrive. Memory usage stays constant regardless of dataset size.

Step 4: Dataset formats

The datasets library uses Apache Arrow under the hood. You can convert to other formats depending on what your pipeline needs.

```
dataset = load_dataset("stanfordnlp/imdb", split="train")
```

```
dataset.to_csv("imdb_train.csv")
dataset.to_json("imdb_train.json")
dataset.to_parquet("imdb_train.parquet")
```

Format comparison:

Format	Size	Read Speed	Best For
CSV	Large	Slow	Human readability, spreadsheets
JSON	Large	Slow	APIs, nested data
Parquet	Small	Fast	Analytics, columnar queries
Arrow	Small	Fastest	In-memory processing (what datasets uses internally)

For AI work, Parquet is the best storage format. Arrow is what you work with in memory. CSV and JSON are for interchange.

Step 5: Data splits

Every ML project needs three splits:

- **Train:** The model learns from this (typically 80%)
- **Validation:** You check progress during training (typically 10%)
- **Test:** Final evaluation after training is done (typically 10%)

Some datasets come pre-split. When they don't, split them yourself:

```
dataset = load_dataset("stanfordnlp/imdb", split="train")

split = dataset.train_test_split(test_size=0.2, seed=42)
train_val = split["train"].train_test_split(test_size=0.125, seed=42)

train_ds = train_val["train"]
val_ds = train_val["test"]
test_ds = split["test"]

print(f"Train: {len(train_ds)}, Val: {len(val_ds)}, Test: {len(test_ds)}")
```

Always set a seed for reproducibility. The same seed produces the same split every time.

Step 6: Download and cache models

Models are large files. The `huggingface_hub` library handles downloading and caching.

```
from huggingface_hub import hf_hub_download, snapshot_download

model_path = hf_hub_download(
    repo_id="sentence-transformers/all-MiniLM-L6-v2",
    filename="config.json"
)
print(f"Cached at: {model_path}")

model_dir = snapshot_download("sentence-transformers/all-MiniLM-L6-v2")
print(f"Full model at: {model_dir}")
```

Models cache to `~/.cache/huggingface/hub/`. Once downloaded, they load instantly on subsequent runs.

Step 7: Handle large files

Model weights and large datasets should not go into git. Three options:

Option A: .gitignore (simplest)

```
*.bin
*.safetensors
*.pt
*.onnx
data/*.parquet
data/*.csv
models/
```

Option B: Git LFS (track large files in git)

```
git lfs install
git lfs track "*.bin"
git lfs track "*.safetensors"
git add .gitattributes
```

Git LFS stores pointers in your repo and the actual files on a separate server. GitHub gives you 1 GB free.

Option C: DVC (data version control)

```
pip install dvc
dvc init
```

```
dvc add data/training_set.parquet
git add data/training_set.parquet.dvc data/.gitignore
git commit -m "Track training data with DVC"
```

DVC creates small `.dvc` files that point to your data. The data itself lives in S3, GCS, or another remote storage backend.

Approach	Complexity	Best For
<code>.gitignore</code>	Low	Personal projects, downloaded data you can re-fetch
Git LFS	Medium	Teams sharing model weights via git
DVC	High	Reproducible experiments, large datasets, teams

For this course, `.gitignore` is enough. Use DVC when you need to reproduce exact experiments across machines.

Step 8: Storage patterns

Local storage works for datasets under ~10 GB. The HF cache handles this automatically.

Cloud storage is for anything larger or shared across machines:

```
import os
```

```
local_path = os.path.expanduser("~/cache/huggingface/datasets/")
```

```
# s3_path = "s3://my-bucket/datasets/"
# gcs_path = "gs://my-bucket/datasets/"
```

DVC integrates with S3 and GCS directly:

```
dvc remote add -d myremote s3://my-bucket/dvc-store
dvc push
```

For this course, local storage is sufficient. Cloud storage becomes relevant when you fine-tune on remote GPU instances.

Datasets Used in This Course

Dataset	Lessons	Size	What It Teaches
IMDB	Tokenization, classification	84 MB	Text classification basics
WikiText	Language modeling	181 MB	Next-token prediction
SQuAD	QA systems	35 MB	Question answering, spans
Common Crawl (subset)	Embeddings	Varies	Large-scale text processing
MNIST	Vision basics	21 MB	Image classification fundamentals
COCO (subset)	Multimodal	Varies	Image-text pairs

You do not need to download all of these now. Each lesson specifies what it needs.

Use It

Run the utility script to verify everything works:

```
python code/data_utils.py
```

This downloads a small dataset, converts it, splits it, and prints a summary.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/00-setup-and-tooling/09-data-management>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/00-setup-and-tooling/09-data-management/code>

1. Load the glue dataset with the mrpc config and inspect the first 5 examples
2. Stream the c4 dataset and count how many examples you can process in 10 seconds
3. Convert a dataset to Parquet and compare the file size to CSV
4. Create a 70/15/15 train/val/test split with a fixed seed and verify the sizes

Key Terms

Term	What people say	What it actually means
Dataset split	"Training data"	A named subset (train/val/test) used at different stages of the ML lifecycle
Streaming	"Load it lazily"	Processing data row by row from a remote source without downloading the full dataset
Parquet	"Compressed CSV"	A columnar file format optimized for analytical queries and storage efficiency
Arrow	"Fast dataframe"	An in-memory columnar format used internally by the datasets library for zero-copy reads
Git LFS	"Git for big files"	An extension that stores large files outside the git repo while keeping pointers in version control
DVC	"Git for data"	A version control system for datasets and models that integrates with cloud storage
Cache	"Already downloaded"	A local copy of previously fetched data, stored at ~/.cache/huggingface/ by default

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/00-setup-and-tooling/09-data-management>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/00-setup-and-tooling/09-data-management/code>

- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/00-setup-and-tooling/09-data-management>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Terminal & Shell

The terminal is where AI engineers live. Get comfortable here.

Type: Learn **Languages:** - **Prerequisites:** Phase 0, Lesson 01 **Time:** ~35 minutes

Learning Objectives

- Use piping, redirects, and grep to filter and process training logs from the command line
- Create persistent tmux sessions with multiple panes for concurrent training and GPU monitoring
- Monitor system and GPU resources with htop, nvidia-smi, and nvidia-smi
- Transfer files between local and remote machines using SSH, scp, and rsync

The Problem

You will spend more time in the terminal than in any editor. Training runs, GPU monitoring, log tailing, remote SSH sessions, environment management. Every AI workflow touches the shell. If you're slow here, you're slow everywhere.

This lesson covers the terminal skills that matter for AI work. No history of Unix. No deep-dive into Bash scripting. Just what you need.

The Concept

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/00-setup-and-tooling/10-terminal-and-shell>

Three things running at once. One terminal. You can detach, go home, SSH back in, and reattach. The training keeps running.

Interactive figure: s0-shell-pipeline. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/00-setup-and-tooling/10-terminal-and-shell>

Build It

Step 1: Know your shell

Check which shell you're running:

```
echo $SHELL
```

Most systems use bash or zsh. Both work fine. The commands in this course work in either.

Key things to know:

```

# Move around
cd ~/projects/ai-engineering-from-scratch
pwd
ls -la

# History search (most useful shortcut you'll learn)
# Ctrl+R then type part of a previous command
# Press Ctrl+R again to cycle through matches

# Clear terminal
clear # or Ctrl+L

# Cancel a running command
# Ctrl+C

# Suspend a running command (resume with fg)
# Ctrl+Z

```

Step 2: Piping and redirects

Piping connects commands together. This is how you process logs, filter output, and chain tools. You will use this constantly.

```

# Count how many times "loss" appears in a log
cat train.log | grep "loss" | wc -l

# Extract just the loss values from training output
grep "loss:" train.log | awk '{print $NF}' > losses.txt

# Watch a log file update in real time, filtering for errors
tail -f train.log | grep --line-buffered "ERROR"

# Sort experiments by final accuracy
grep "final_accuracy" results/*.log | sort -t= -k2 -n -r

# Redirect stdout and stderr to separate files
python train.py > output.log 2> errors.log

# Redirect both to the same file
python train.py > train_full.log 2>&1

```

The three redirects you need:

Symbol	What it does
>	Write stdout to file (overwrite)
>>	Append stdout to file
2>	Write stderr to file
2>&1	Send stderr to same place as stdout
\	Send stdout of one command as stdin to the next

Step 3: Background processes

Training runs take hours. You don't want to keep your terminal open the whole time.

```

# Run in background (output still goes to terminal)
python train.py &

# Run in background, immune to hangup (closing terminal won't kill it)
nohup python train.py > train.log 2>&1 &

# Check what's running in background
jobs
ps aux | grep train.py

# Bring a background job to foreground
fg %1

# Kill a background process
kill %1
# or find its PID and kill that
kill $(pgrep -f "train.py")

```

The difference between &, nohup, and screen/tmux:

Method	Survives terminal close?	Can reattach?
command &	No	No
nohup command &	Yes	No (check log file)
screen / tmux	Yes	Yes

For anything longer than a few minutes, use tmux.

Step 4: tmux

tmux lets you create persistent terminal sessions with multiple panes. This is the single most useful tool for managing training runs.

```

# Install
# macOS
brew install tmux
# Ubuntu
sudo apt install tmux

# Start a named session
tmux new -s training

# Split horizontally
# Ctrl+B then "

# Split vertically
# Ctrl+B then %

# Navigate between panes
# Ctrl+B then arrow keys

# Detach (session keeps running)
# Ctrl+B then d

```

```
# Reattach
tmux attach -t training

# List sessions
tmux ls

# Kill a session
tmux kill-session -t training

A typical AI workflow session:

tmux new -s train

# Pane 1: start training
python train.py --epochs 100 --lr 1e-4

# Ctrl+B, " to split, then run GPU monitor
watch -n1 nvidia-smi

# Ctrl+B, % to split vertically, tail the logs
tail -f logs/experiment.log

# Now detach with Ctrl+B, d
# SSH out, go get coffee, come back
# tmux attach -t train
```

Step 5: Monitoring with htop and nvidia-smi

```
# System processes (better than top)
htop

# GPU processes (if you have NVIDIA GPU)
# Install: sudo apt install nvidia-smi (Ubuntu) or brew install nvidia-smi (macOS)
nvidia-smi

# Quick GPU check without nvidia-smi
nvidia-smi

# Watch GPU usage update every second
watch -n1 nvidia-smi

# See which processes are using the GPU
nvidia-smi --query=compute_applications --format=csv

htop keybindings you'll use: - F6 or > to sort by column (sort by memory to find memory leaks)
- F5 to toggle tree view (see child processes) - F9 to kill a process - / to search for a process
name
```

Step 6: SSH for remote GPU boxes

When you rent a cloud GPU (Lambda, RunPod, Vast.ai), you connect via SSH.

```
# Basic connection
ssh user@gpu-box-ip
```

```

# With a specific key
ssh -i ~/.ssh/my_gpu_key user@gpu-box-ip

# Copy files to remote
scp model.pt user@gpu-box-ip:~/models/

# Copy files from remote
scp user@gpu-box-ip:~/results/metrics.json ./

# Sync a whole directory (faster for many files)
rsync -avz ./data/ user@gpu-box-ip:~/data/

# Port forward (access remote Jupyter/TensorBoard locally)
ssh -L 8888:localhost:8888 user@gpu-box-ip
# Now open localhost:8888 in your browser

# SSH config for convenience
# Add to ~/.ssh/config:
# Host gpu
#     HostName 192.168.1.100
#     User ubuntu
#     IdentityFile ~/.ssh/gpu_key
#
# Then just:
# ssh gpu

```

Step 7: Useful aliases for AI work

Add these to your ~/.bashrc or ~/.zshrc:

```
source phases/00-setup-and-tooling/10-terminal-and-shell/code/shell_aliases.sh
```

Or copy the ones you want. The key aliases:

```

# GPU status at a glance
alias gpu='nvidia-smi --query-gpu=index,name,utilization,gpu,memory.used,memory.total,temperature

# Kill all Python training processes
alias killtraining='pkill -f "python.*train"'

# Quick virtual environment activate
alias ae='source .venv/bin/activate'

# Watch training loss
alias watchloss='tail -f logs/*.log | grep --line-buffered "loss"'

```

See code/shell_aliases.sh for the full set.

Step 8: Common AI terminal patterns

These come up repeatedly in practice:

```

# Run training, log everything, notify when done
python train.py 2>&1 | tee train.log; echo "DONE" | mail -s "Training complete" you@email.com

# Compare two experiment logs side by side

```

```

diff <(grep "accuracy" exp1.log) <(grep "accuracy" exp2.log)

# Find the largest model files (clean up disk space)
find . -name "*.pt" -o -name "*.safetensors" | xargs du -h | sort -rh | head -20

# Download a model from Hugging Face
wget https://huggingface.co/model/resolve/main/model.safetensors

# Untar a dataset
tar xzf dataset.tar.gz -C ./data/

# Count lines in all Python files (see how big your project is)
find . -name "*.py" | xargs wc -l | tail -1

# Check disk space (training data fills disks fast)
df -h
du -sh ./data/*

# Environment variable check before training
env | grep -i cuda
env | grep -i torch

```

Use It

Here's when each tool comes into play during this course:

Tool	When you use it
tmux	Every training run (Phases 3+)
tail -f + grep	Monitoring training logs
nohup / &	Quick background tasks
htop / nvtop	Debugging slow training, OOM errors
SSH + rsync	Working on cloud GPUs
Piping + redirects	Processing experiment results
Aliases	Saving time on repetitive commands

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/00-setup-and-tooling/10-terminal-and-shell/code>

1. Install tmux, create a session with three panes, and run htop in one, watch -n1 date in another, and a Python script in the third. Detach and reattach.
2. Add the aliases from code/shell_aliases.sh to your shell config and reload with source ~/.zshrc (or ~/.bashrc).
3. Create a fake training log with for i in \$(seq 1 100); do echo "epoch \$i loss: \$(echo "scale=4; 1/\$i" | bc)"; sleep 0.1; done > fake_train.log and then use grep, tail, and awk to extract just the loss values.
4. Set up an SSH config entry for a server you have access to (or use localhost to practice the syntax).

Key Terms

Term	What people say	What it actually means
Shell	“The terminal”	The program that interprets your commands (bash, zsh, fish)
tmux	“Terminal multiplexer”	A program that lets you run multiple terminal sessions inside one window, and detach/reattach
Pipe	“The bar thing”	The <code> </code> operator that sends one command’s output as input to another
PID	“Process ID”	A unique number assigned to every running process, used to monitor or kill it
nohup	“No hangup”	Runs a command immune to the hangup signal, so closing the terminal won’t kill it
SSH	“Connecting to the server”	Secure Shell, an encrypted protocol for running commands on a remote machine

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/00-setup-and-tooling/10-terminal-and-shell>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/00-setup-and-tooling/10-terminal-and-shell/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/00-setup-and-tooling/10-terminal-and-shell>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Linux for AI

Most AI runs on Linux. You need to know enough to not be stuck.

Type: Learn **Languages:** - **Prerequisites:** Phase 0, Lesson 01 **Time:** ~30 minutes

Learning Objectives

- Navigate the Linux file system and perform essential file operations from the command line
- Manage file permissions with `chmod` and `chown` to resolve “Permission denied” errors
- Install system packages with `apt` and set up a fresh GPU box for AI work
- Identify macOS-to-Linux differences that commonly trip up developers working on remote machines

The Problem

You develop on macOS or Windows. But the moment you SSH into a cloud GPU box, rent a Lambda instance, or spin up an EC2 machine, you land in Ubuntu. The terminal is your only interface. There is no Finder, no Explorer, no GUI. If you can’t navigate the file system, install packages, and manage processes from the command line, you’re stuck paying for idle GPU hours while googling “how to unzip a file in Linux.”

This is a survival guide. It covers exactly what you need to operate on a remote Linux machine for AI work. Nothing more.

File System Layout

Linux organizes everything under a single root `/`. There is no `C:\` or `/Volumes`. The directories you’ll actually touch:

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/00-setup-and-tooling/11-linux-for-ai>

Your home directory is `~` or `/home/your-username`. Almost everything you do happens here.

Essential Commands

These are the 15 commands that cover 95% of what you’ll do on a remote GPU box.

Moving Around

<code>pwd</code>	<i># Where am I?</i>
<code>ls</code>	<i># What's here?</i>
<code>ls -la</code>	<i># What's here, including hidden files with details?</i>

```
cd /path/to/dir      # Go there
cd ~                 # Go home
cd ..                # Go up one level
```

Files and Directories

```
mkdir my-project      # Create a directory
mkdir -p a/b/c         # Create nested directories in one shot

cp file.txt backup.txt # Copy a file
cp -r src/ src-backup/ # Copy a directory (recursive)

mv old.txt new.txt     # Rename a file
mv file.txt /tmp/      # Move a file

rm file.txt           # Delete a file (no trash, it's gone)
rm -rf my-dir/        # Delete a directory and everything inside

rm -rf is permanent. There is no undo. Double-check the path before hitting enter.
```

Reading Files

```
cat file.txt          # Print entire file
head -20 file.txt      # First 20 lines
tail -20 file.txt      # Last 20 lines
tail -f log.txt        # Follow a log file in real time (Ctrl+C to stop)
less file.txt          # Scroll through a file (q to quit)
```

Searching

```
grep "error" training.log # Find lines containing "error"
grep -r "learning_rate" . # Search all files in current directory
grep -i "cuda" config.yaml # Case-insensitive search

find . -name "*.py" # Find all Python files under current dir
find . -name "*.ckpt" -size +1G # Find checkpoint files larger than 1GB
```

Permissions

Every file in Linux has an owner and permission bits. You'll run into this when scripts won't execute or you can't write to a directory.

```
ls -l train.py
# -rwxr-xr-- 1 user group 2048 Mar 19 10:00 train.py
#   ^^^          owner permissions: read, write, execute
#   ^^^          group permissions: read, execute
#   ^^           everyone else: read only
```

Common fixes:

```
chmod +x train.sh # Make a script executable
chmod 755 deploy.sh # Owner: full, others: read+execute
chmod 644 config.yaml # Owner: read+write, others: read only
```

```
chown user:group file.txt    # Change who owns a file (needs sudo)
```

When something says “Permission denied,” it’s almost always a permissions issue. `chmod +x` or `sudo` will fix most cases.

Package Management (apt)

Ubuntu uses apt. This is how you install system-level software.

```
sudo apt update                # Refresh the package list (always do this first)
sudo apt install -y htop       # Install a package (-y skips confirmation)
sudo apt install -y build-essential # C compiler, make, etc. Needed by many Python packages
sudo apt install -y tmux       # Terminal multiplexer (keep sessions alive after disconnect)

apt list --installed          # What's installed?
sudo apt remove htop          # Uninstall
```

Common packages you’ll install on a fresh GPU box:

```
sudo apt update && sudo apt install -y \
    build-essential \
    git \
    curl \
    wget \
    tmux \
    htop \
    unzip \
    python3-venv
```

Users and sudo

You’re usually logged in as a regular user. Some operations need root (admin) access.

```
whoami                # What user am I?
sudo command          # Run a single command as root
sudo su               # Become root (exit to go back, use sparingly)
```

On cloud GPU instances, you’re typically the only user and already have sudo access. Don’t run everything as root. Use sudo only when needed.

Processes and systemd

When your training hangs, or you need to check what’s running:

```
htop                  # Interactive process viewer (q to quit)
ps aux | grep python  # Find running Python processes
kill 12345            # Gracefully stop process with PID 12345
kill -9 12345         # Force kill (use when graceful doesn't work)
nvidia-smi            # GPU processes and memory usage
```

systemd manages services (background daemons). You’ll use it if you run inference servers:

```
sudo systemctl start nginx    # Start a service
sudo systemctl stop nginx     # Stop it
```

```

sudo systemctl restart nginx      # Restart it
sudo systemctl status nginx       # Check if it's running
sudo systemctl enable nginx       # Start automatically on boot

```

Disk Space

GPU boxes often have limited disk space. Models and datasets fill it fast.

```

df -h                            # Disk usage for all mounted drives
df -h /home                      # Disk usage for /home specifically

du -sh *                        # Size of each item in current directory
du -sh ~/.cache                 # Size of your cache (pip, huggingface models land here)
du -sh /data/checkpoints/      # Check how big your checkpoints are

# Find the biggest space hogs
du -h --max-depth=1 / 2>/dev/null | sort -hr | head -20

```

Common space savers:

```

# Clear pip cache
pip cache purge

# Clear apt cache
sudo apt clean

# Remove old checkpoints you don't need
rm -rf checkpoints/epoch_01/ checkpoints/epoch_02/

```

Networking

You'll download models, transfer files, and hit APIs from the command line.

```

# Download files
wget https://example.com/model.bin      # Download a file
curl -O https://example.com/data.tar.gz # Same thing with curl
curl -s https://api.example.com/health | python3 -m json.tool # Hit an API, pretty-print JSON

# Transfer files between machines
scp model.bin user@remote:/data/         # Copy file to remote machine
scp user@remote:/data/results.csv .      # Copy file from remote to local
scp -r user@remote:/data/checkpoints/ ./local-dir/ # Copy directory

# Sync directories (faster than scp for large transfers, resumes on failure)
rsync -avz --progress ./data/ user@remote:/data/
rsync -avz --progress user@remote:/results/ ./results/

```

Use rsync over scp for anything large. It only transfers changed bytes and handles interrupted connections.

tmux: Keep Sessions Alive

When you SSH into a remote box, closing your laptop kills your training run. tmux prevents this.

```
tmux new -s train          # Start a new session named "train"
# ... start your training, then:
# Ctrl+B, then D          # Detach (training keeps running)

tmux ls                   # List sessions
tmux attach -t train      # Reattach to session

# Inside tmux:
# Ctrl+B, then %          # Split pane vertically
# Ctrl+B, then "          # Split pane horizontally
# Ctrl+B, then arrow keys # Switch between panes
```

Always run long training jobs inside tmux. Always.

WSL2 for Windows Users

If you're on Windows, WSL2 gives you a real Linux environment without dual-booting.

```
# In PowerShell (admin)
wsl --install -d Ubuntu-24.04

# After restart, open Ubuntu from Start menu
sudo apt update && sudo apt upgrade -y
```

WSL2 runs a real Linux kernel. Everything in this lesson works inside it. Your Windows files are at /mnt/c/Users/YourName/ from inside WSL.

GPU passthrough works with NVIDIA drivers installed on the Windows side. Install the Windows NVIDIA driver (not the Linux one), and CUDA will be available inside WSL2.

Gotchas: macOS to Linux

Things that will trip you up if you're coming from macOS:

macOS	Linux	Notes
brew install	sudo apt install	Different package names sometimes. brew install htop vs sudo apt install htop works the same, but brew install readline vs sudo apt install libreadline-dev does not. But you won't have a GUI on a remote box. Use cat or less.
open file.txt	xdg-open file.txt	
pbcopy / pbpaste	Not available	Pipe to/from clipboard doesn't exist over SSH.

macOS	Linux	Notes
<code>~/.zshrc</code>	<code>~/.bashrc</code>	macOS defaults to zsh. Most Linux servers use bash.
<code>/opt/homebrew/</code>	<code>/usr/bin/, /usr/local/bin/</code>	Binaries live in different places.
<code>sed -i '' 's/a/b/' file</code>	<code>sed -i 's/a/b/' file</code>	macOS sed needs an empty string after <code>-i</code> . Linux does not.
Case-insensitive filesystem	Case-sensitive filesystem	<code>Model.py</code> and <code>model.py</code> are two different files on Linux.
Line endings <code>\n</code>	Line endings <code>\n</code>	Same. But Windows uses <code>\r\n</code> , which breaks bash scripts. Run <code>dos2unix</code> to fix.

Quick Reference Card

Navigation: `pwd, ls, cd, find`
 Files: `cp, mv, rm, mkdir, cat, head, tail, less`
 Search: `grep, find`
 Permissions: `chmod, chown, sudo`
 Packages: `apt update, apt install`
 Processes: `htop, ps, kill, nvidia-smi`
 Services: `systemctl start/stop/restart/status`
 Disk: `df -h, du -sh`
 Network: `curl, wget, scp, rsync`
 Sessions: `tmux new/attach/detach`

Interactive figure: s0-process-fork. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/00-setup-and-tooling/11-linux-for-ai>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/00-setup-and-tooling/11-linux-for-ai/code>

1. SSH into any Linux machine (or open WSL2) and navigate to your home directory. Create a project folder, create three empty files inside it with `touch`, then list them with `ls -la`.
2. Install `htop` with `apt`, run it, and identify which process is using the most memory.
3. Start a `tmux` session, run `sleep 300` inside it, detach, list sessions, and reattach.
4. Use `df -h` to check available disk space, then use `du -sh ~/.cache/*` to find what's taking up space in your cache.
5. Transfer a file from your local machine to a remote one using `scp`, then do the same transfer with `rsync` and compare the experience.

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/00-setup-and-tooling/11-linux-for-ai>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/00-setup-and-tooling/11-linux-for-ai/code>

- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/00-setup-and-tooling/11-linux-for-ai>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Debugging and Profiling

The worst AI bugs don't crash. They train silently on garbage and report a beautiful loss curve.

Type: Build **Language:** Python **Prerequisites:** Lesson 1 (Dev Environment), basic PyTorch familiarity **Time:** ~60 minutes

Learning Objectives

- Use conditional breakpoint() and debug_print to inspect tensor shapes, dtypes, and NaN values mid-training
- Profile training loops with cProfile, line_profiler, and tracemalloc to find bottlenecks
- Detect common AI bugs: shape mismatches, NaN loss, data leakage, and wrong-device tensors
- Set up TensorBoard to visualize loss curves, weight histograms, and gradient distributions

The Problem

AI code fails differently than regular code. A web app crashes with a stack trace. A misconfigured training loop runs for 8 hours, burns \$200 in GPU time, and produces a model that predicts the mean of every input. The code never errored. The bug was a tensor on the wrong device, a forgotten .detach(), or labels leaking into features.

You need debugging tools that catch these silent failures before they waste your time and compute.

The Concept

AI debugging operates at three levels:

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/00-setup-and-tooling/12-debugging-and-profiling>

Most people jump straight to level 3 (staring at TensorBoard). But 80% of AI bugs live at levels 1 and 2.

Interactive figure: s0-flame-hot. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/00-setup-and-tooling/12-debugging-and-profiling>

Build It

Part 1: Print Debugging (Yes, It Works)

Print debugging gets dismissed. It shouldn't. For tensor code, a targeted print statement beats stepping through a debugger because you need to see shapes, dtypes, and value ranges all at once.

```
def debug_print(name, tensor):
    print(f"{name}: shape={tensor.shape}, dtype={tensor.dtype}, "
          f"device={tensor.device}, "
          f"min={tensor.min().item():.4f}, max={tensor.max().item():.4f}, "
          f"mean={tensor.mean().item():.4f}, "
          f"has_nan={tensor.isnan().any().item()}")
```

Call this after every suspicious operation. When the bug is found, remove the prints. Simple.

Part 2: Python Debugger (pdb and breakpoint)

The built-in debugger is underrated for AI work. Drop `breakpoint()` into your training loop and inspect tensors interactively.

```
def training_step(model, batch, criterion, optimizer):
    inputs, labels = batch
    outputs = model(inputs)
    loss = criterion(outputs, labels)

    if loss.item() > 100 or torch.isnan(loss):
        breakpoint()

    loss.backward()
    optimizer.step()
```

When the debugger drops you in, useful commands:

- `p outputs.shape` to check shapes
- `p loss.item()` to see the loss value
- `p torch.isnan(outputs).sum()` to count NaNs
- `p model.fc1.weight.grad` to check gradients
- `c` to continue, `q` to quit

This is conditional debugging. You only stop when something looks wrong. For a 10,000-step training run, that matters.

Part 3: Python Logging

Replace print statements with logging when your debugging goes beyond a quick check.

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s",
    handlers=[
        logging.FileHandler("training.log"),
        logging.StreamHandler()
    ]
)
```

```

)
logger = logging.getLogger(__name__)

logger.info("Starting training: lr=%.4f, batch_size=%d", lr, batch_size)
logger.warning("Loss spike detected: %.4f at step %d", loss.item(), step)
logger.error("NaN loss at step %d, stopping", step)

```

Logging gives you timestamps, severity levels, and file output. When a training run fails at 3 AM, you want a log file, not terminal output that scrolled off screen.

Part 4: Timing Code Sections

Knowing where time goes is the first step to optimization.

```

import time

class Timer:
    def __init__(self, name=""):
        self.name = name

    def __enter__(self):
        self.start = time.perf_counter()
        return self

    def __exit__(self, *args):
        elapsed = time.perf_counter() - self.start
        print(f"[{self.name}] {elapsed:.4f}s")

with Timer("data loading"):
    batch = next(data_loader_iter)

with Timer("forward pass"):
    outputs = model(batch)

with Timer("backward pass"):
    loss.backward()

```

Common finding: data loading takes 60% of training time. The fix is `num_workers > 0` in your `DataLoader`, not a faster GPU.

Part 5: cProfile and line_profiler

When you need more than manual timers:

```
python -m cProfile -s cumtime train.py
```

This shows every function call sorted by cumulative time. For line-by-line profiling:

```
pip install line_profiler
```

```

@profile
def train_step(model, data, target):
    output = model(data)
    loss = F.cross_entropy(output, target)
    loss.backward()
    return loss

```

Run with: kernprof -l -v train.py

Part 6: Memory Profiling

CPU Memory with tracemalloc

```
import tracemalloc

tracemalloc.start()

# your code here
model = build_model()
data = load_dataset()

snapshot = tracemalloc.take_snapshot()
top_stats = snapshot.statistics("lineno")
for stat in top_stats[:10]:
    print(stat)
```

CPU Memory with memory_profiler

```
pip install memory_profiler

from memory_profiler import profile

@profile
def load_data():
    raw = read_csv("data.csv")           # watch memory jump here
    processed = preprocess(raw)          # and here
    return processed
```

Run with `python -m memory_profiler your_script.py` to see line-by-line memory usage.

GPU Memory with PyTorch

```
import torch

if torch.cuda.is_available():
    print(torch.cuda.memory_summary())

    print(f"Allocated: {torch.cuda.memory_allocated() / 1e9:.2f} GB")
    print(f"Cached: {torch.cuda.memory_reserved() / 1e9:.2f} GB")
```

When you hit OOM (Out of Memory):

1. Reduce batch size (first thing to try, always)
2. Use `torch.cuda.empty_cache()` to free cached memory
3. Use `del tensor` followed by `torch.cuda.empty_cache()` for large intermediates
4. Use mixed precision (`torch.cuda.amp`) to halve memory usage
5. Use gradient checkpointing for very deep models

Part 7: Common AI Bugs and How to Catch Them

Shape Mismatch

The most frequent bug. A tensor has shape [batch, features] when the model expects [batch, channels, height, width].

```
def check_shapes(model, sample_input):
    print(f"Input: {sample_input.shape}")
    hooks = []

    def make_hook(name):
        def hook(module, inp, out):
            in_shape = inp[0].shape if isinstance(inp, tuple) else inp.shape
            out_shape = out.shape if hasattr(out, "shape") else type(out)
            print(f" {name}: {in_shape} -> {out_shape}")
        return hook

    for name, module in model.named_modules():
        hooks.append(module.register_forward_hook(make_hook(name)))

    with torch.no_grad():
        model(sample_input)

    for h in hooks:
        h.remove()
```

Run this once with a sample batch. It maps every shape transformation in your model.

NaN Loss

NaN loss means something exploded. Common causes:

- Learning rate too high
- Division by zero in custom loss
- Log of zero or negative number
- Exploding gradients in RNNs

```
def detect_nan(model, loss, step):
    if torch.isnan(loss):
        print(f"NaN loss at step {step}")
        for name, param in model.named_parameters():
            if param.grad is not None:
                if torch.isnan(param.grad).any():
                    print(f" NaN gradient in {name}")
                if torch.isinf(param.grad).any():
                    print(f" Inf gradient in {name}")
        return True
    return False
```

Data Leakage

Your model gets 99% accuracy on the test set. Sounds great. It's a bug.

```
def check_data_leakage(train_set, test_set, id_column="id"):
    train_ids = set(train_set[id_column].tolist())
    test_ids = set(test_set[id_column].tolist())
```

```

overlap = train_ids & test_ids
if overlap:
    print(f"DATA LEAKAGE: {len(overlap)} samples in both train and test")
    return True
return False

```

Also check for temporal leakage: using future data to predict the past. Sort by timestamp before splitting.

Wrong Device

Tensors on different devices (CPU vs GPU) cause runtime errors. But sometimes a tensor silently stays on CPU while everything else is on GPU, and training just runs slowly.

```

def check_devices(model, *tensors):
    model_device = next(model.parameters()).device
    print(f"Model device: {model_device}")
    for i, t in enumerate(tensors):
        if t.device != model_device:
            print(f" WARNING: tensor {i} on {t.device}, model on {model_device}")

```

Part 8: TensorBoard Basics

TensorBoard shows you what's happening inside training over time.

pip install tensorboard

```
from torch.utils.tensorboard import SummaryWriter
```

```
writer = SummaryWriter("runs/experiment_1")
```

```

for step in range(num_steps):
    loss = train_step(model, batch)

    writer.add_scalar("loss/train", loss.item(), step)
    writer.add_scalar("lr", optimizer.param_groups[0]["lr"], step)

    if step % 100 == 0:
        for name, param in model.named_parameters():
            writer.add_histogram(f"weights/{name}", param, step)
            if param.grad is not None:
                writer.add_histogram(f"grads/{name}", param.grad, step)

writer.close()

```

Launch it:

```
tensorboard --logdir=runs
```

What to look for:

- **Loss not decreasing:** Learning rate too low, or model architecture issue
- **Loss oscillating wildly:** Learning rate too high
- **Loss goes to NaN:** Numerical instability (see NaN section above)
- **Train loss decreasing, val loss increasing:** Overfitting
- **Weight histograms collapsing to zero:** Vanishing gradients
- **Gradient histograms exploding:** Need gradient clipping

Part 9: VS Code Debugger

For interactive debugging, configure VS Code with a `launch.json`:

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Debug Training",
      "type": "debugpy",
      "request": "launch",
      "program": "${file}",
      "console": "integratedTerminal",
      "justMyCode": false
    }
  ]
}
```

Set breakpoints by clicking the gutter. Use the Variables pane to inspect tensor properties. The Debug Console lets you run arbitrary Python expressions mid-execution.

Useful for stepping through data preprocessing pipelines where you want to see each transformation.

Use It

Here's the debugging workflow that catches most AI bugs:

1. **Before training:** Run `check_shapes` with a sample batch. Verify input and output dimensions match expectations.
2. **First 10 steps:** Use `debug_print` on loss, outputs, and gradients. Confirm nothing is NaN and values are in reasonable ranges.
3. **During training:** Log loss, learning rate, and gradient norms. Use TensorBoard for visualization.
4. **When something breaks:** Drop `breakpoint()` at the failure point. Inspect tensors interactively.
5. **For performance:** Time your data loading vs forward vs backward pass. Profile memory if you're near OOM.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/00-setup-and-tooling/12-debugging-and-profiling>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/00-setup-and-tooling/12-debugging-and-profiling/code>

1. Run `debug_tools.py` and read through each section's output. Modify the dummy model to introduce a NaN (hint: divide by zero in the forward pass) and watch the detector catch it.
2. Profile a training loop with `cProfile` and identify the slowest function.
3. Use `tracemalloc` to find which line in your data loading pipeline allocates the most memory.

4. Set up TensorBoard for a simple training run and identify whether the model is overfitting.
5. Use `breakpoint()` inside a training loop. Practice inspecting tensor shapes, devices, and gradient values from the debugger prompt.

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/00-setup-and-tooling/12-debugging-and-profiling>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/00-setup-and-tooling/12-debugging-and-profiling/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/00-setup-and-tooling/12-debugging-and-profiling>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Part II — Math Foundations

Course phase 01. Live edition with animated figures and quizzes: <https://aiengineeringfromscratch.com/catalog.html>

Linear Algebra Intuition

Every AI model is just matrix math wearing a fancy hat.

Type: Learn **Languages:** Python, Julia **Prerequisites:** Phase 0 **Time:** ~60 minutes

Learning Objectives

- Implement vector and matrix operations (addition, dot product, matrix multiply) from scratch in Python
- Explain geometrically what the dot product, projection, and Gram-Schmidt process do
- Determine linear independence, rank, and basis of a set of vectors using row reduction
- Connect linear algebra concepts to their AI applications: embeddings, attention scores, and LoRA

The Problem

Open any ML paper. Within the first page, you'll see vectors, matrices, dot products, and transformations. Without linear algebra intuition, these are just symbols. With it, you can see what a neural network is actually doing – moving points around in space.

You don't need to be a mathematician. You need to see what these operations mean geometrically, then code them yourself.

The Concept

Vectors Are Points (and Directions)

A vector is just a list of numbers. But those numbers mean something – they're coordinates in space.

2D vector [3, 2]:

x	y	Point
3	2	The vector points from origin (0,0) to (3, 2) on the plane

The vector has magnitude $\sqrt{3^2 + 2^2} = \sqrt{13}$ and points up and to the right.

In AI, vectors represent everything: - A word → a vector of 768 numbers (its “meaning” in embedding space) - An image → a vector of millions of pixel values - A user → a vector of preferences

Matrices Are Transformations

A matrix transforms one vector into another. It can rotate, scale, stretch, or project.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/01-linear-algebra-intuition>

In AI, matrices ARE the model: - Neural network weights → matrices that transform input into output - Attention scores → matrices that decide what to focus on - Embeddings → matrices that map words to vectors

The Dot Product Measures Similarity

The dot product of two vectors tells you how similar they are.

$$a \cdot b = a_1 \times b_1 + a_2 \times b_2 + \dots + a_n \times b_n$$

Same direction: $a \cdot b > 0$ (similar)
 Perpendicular: $a \cdot b = 0$ (unrelated)
 Opposite direction: $a \cdot b < 0$ (dissimilar)

This is literally how search engines, recommendation systems, and RAG work – find vectors with high dot products.

Linear Independence

Vectors are linearly independent if no vector in the set can be written as a combination of the others. If v_1, v_2, v_3 are independent, they span a 3D space. If one is a combination of the others, they only span a plane.

Why it matters for AI: your feature matrix should have linearly independent columns. If two features are perfectly correlated (linearly dependent), the model cannot distinguish their effects. This causes multicollinearity in regression – the weight matrix becomes unstable, and small input changes produce wild output swings.

Concrete example:

$$\begin{aligned} v_1 &= [1, 0, 0] \\ v_2 &= [0, 1, 0] \\ v_3 &= [2, 1, 0] \quad \# \quad v_3 = 2 \cdot v_1 + v_2 \end{aligned}$$

v_1 and v_2 are independent – neither is a scalar multiple or combination of the other. But $v_3 = 2 \cdot v_1 + v_2$, so $\{v_1, v_2, v_3\}$ is a dependent set. These three vectors all lie in the xy-plane. No matter how you combine them, you cannot reach $[0, 0, 1]$. You have three vectors but only two dimensions of freedom.

In a dataset: if $\text{feature}_3 = 2 \cdot \text{feature}_1 + \text{feature}_2$, adding feature_3 gives the model zero new information. Worse, it makes the normal equations singular – there is no unique solution for the weights.

Basis and Rank

A basis is a minimal set of linearly independent vectors that span the entire space. The number of basis vectors is the dimension of the space.

The standard basis for 3D space is $\{[1,0,0], [0,1,0], [0,0,1]\}$. But any three independent vectors in 3D form a valid basis. The choice of basis is a choice of coordinate system.

Rank of a matrix = number of linearly independent columns = number of linearly independent rows. If $\text{rank} < \min(\text{rows}, \text{cols})$, the matrix is rank-deficient. This means: - The system has infinitely many solutions (or none) - Information is lost in the transformation - The matrix cannot be inverted

Situation	Rank	What it means for ML
Full rank ($\text{rank} = \min(m, n)$)	Maximum possible	Unique least-squares solution exists. Model is well-conditioned.
Rank deficient ($\text{rank} < \min(m, n)$)	Below maximum	Features are redundant. Infinitely many weight solutions. Regularization needed.
Rank 1	1	Every column is a scaled copy of one vector. All data lies on a line.
Near rank-deficient (small singular values)	Numerically low	Matrix is ill-conditioned. Tiny input noise causes large output changes. Use SVD truncation or ridge regression.

Projection

Projecting vector **a** onto vector **b** gives the component of **a** in the direction of **b**:

$$\text{proj}_b(a) = (a \cdot b / b \cdot b) * b$$

The residual $(a - \text{proj}_b(a))$ is perpendicular to **b**. This orthogonal decomposition is the foundation of least-squares fitting.

Projection is everywhere in ML: - Linear regression minimizes the distance from observations to the column space - the solution IS a projection - PCA projects data onto the directions of maximum variance - Attention in transformers computes projections of queries onto keys

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/01-linear-algebra-intuition>

Example: $a = [3, 4]$, $b = [1, 0]$

$$\text{proj}_b(a) = (3 \cdot 1 + 4 \cdot 0) / (1 \cdot 1 + 0 \cdot 0) * [1, 0] = 3 * [1, 0] = [3, 0]$$

The projection drops the y-component. This is dimensionality reduction in its simplest form - throw away the directions you don't care about.

Gram-Schmidt Process

Converting any set of independent vectors into an orthonormal basis. Orthonormal means every vector has length 1 and every pair is perpendicular.

The algorithm: 1. Take the first vector, normalize it 2. Take the second vector, subtract its projection onto the first, normalize 3. Take the third vector, subtract its projections onto all previous vectors, normalize 4. Repeat for remaining vectors

Input: $v_1, v_2, v_3, \dots$ (linearly independent)

$$u_1 = v_1 / |v_1|$$

$$w_2 = v_2 - (v_2 \cdot u_1) * u_1$$

$$u_2 = w_2 / |w_2|$$

$$w_3 = v_3 - (v_3 \cdot u_1) * u_1 - (v_3 \cdot u_2) * u_2$$

$$u_3 = w_3 / |w_3|$$

Output: $u_1, u_2, u_3, \dots$ (orthonormal basis)

This is how QR decomposition works internally. Q is the orthonormal basis, R captures the projection coefficients. QR decomposition is used in: - Solving linear systems (more stable than Gaussian elimination) - Computing eigenvalues (QR algorithm) - Least-squares regression (the standard numerical method)

Interactive figure: eigen-directions. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/01-linear-algebra-intuition>

Build It

Step 1: Vectors from scratch (Python)

```
class Vector:
    def __init__(self, components):
        self.components = list(components)
        self.dim = len(self.components)

    def __add__(self, other):
        return Vector([a + b for a, b in zip(self.components, other.components)])

    def __sub__(self, other):
        return Vector([a - b for a, b in zip(self.components, other.components)])

    def dot(self, other):
        return sum(a * b for a, b in zip(self.components, other.components))

    def magnitude(self):
        return sum(x**2 for x in self.components) ** 0.5

    def normalize(self):
        mag = self.magnitude()
        return Vector([x / mag for x in self.components])

    def cosine_similarity(self, other):
        return self.dot(other) / (self.magnitude() * other.magnitude())

    def __repr__(self):
        return f"Vector({self.components})"

a = Vector([1, 2, 3])
b = Vector([4, 5, 6])

print(f"a + b = {a + b}")
print(f"a · b = {a.dot(b)}")
print(f"|a| = {a.magnitude():.4f}")
print(f"cosine similarity = {a.cosine_similarity(b):.4f}")
```

Step 2: Matrices from scratch (Python)

```
class Matrix:
    def __init__(self, rows):
        self.rows = [list(row) for row in rows]
        self.shape = (len(self.rows), len(self.rows[0]))

    def __matmul__(self, other):
        if isinstance(other, Vector):
            return Vector([
                sum(self.rows[i][j] * other.components[j] for j in range(self.shape[1]))
                for i in range(self.shape[0])
            ])
        rows = []
        for i in range(self.shape[0]):
            row = []
            for j in range(other.shape[1]):
                row.append(sum(
                    self.rows[i][k] * other.rows[k][j]
                    for k in range(self.shape[1])
                ))
            rows.append(row)
        return Matrix(rows)

    def transpose(self):
        return Matrix([
            [self.rows[j][i] for j in range(self.shape[0])]
            for i in range(self.shape[1])
        ])

    def __repr__(self):
        return f"Matrix({self.rows})"
```

```
rotation_90 = Matrix([[0, -1], [1, 0]])
point = Vector([3, 1])
```

```
rotated = rotation_90 @ point
print(f"Original: {point}")
print(f"Rotated 90°: {rotated}")
```

Step 3: Why this matters for AI

```
import random

random.seed(42)
weights = Matrix([[random.gauss(0, 0.1) for _ in range(3)] for _ in range(2)])
input_vector = Vector([1.0, 0.5, -0.3])

output = weights @ input_vector
print(f"Input (3D): {input_vector}")
print(f"Output (2D): {output}")
print("This is what a neural network layer does -- matrix multiplication.")
```

Step 4: Julia version

```

a = [1.0, 2.0, 3.0]
b = [4.0, 5.0, 6.0]

println("a + b = ", a + b)
println("a · b = ", a · b)           # Julia supports unicode operators
println("|a| = ", √(a · a))
println("cosine = ", (a · b) / (√(a · a) * √(b · b)))

# Matrix-vector multiplication
W = [0.1 -0.2 0.3; 0.4 0.5 -0.1]
x = [1.0, 0.5, -0.3]
println("Wx = ", W * x)
println("This is a neural network layer.")

```

Step 5: Linear independence and projection from scratch (Python)

```

def is_linearly_independent(vectors):
    n = len(vectors)
    dim = len(vectors[0].components)
    mat = Matrix([v.components[:] for v in vectors])
    rows = [row[:] for row in mat.rows]
    rank = 0
    for col in range(dim):
        pivot = None
        for row in range(rank, len(rows)):
            if abs(rows[row][col]) > 1e-10:
                pivot = row
                break
        if pivot is None:
            continue
        rows[rank], rows[pivot] = rows[pivot], rows[rank]
        scale = rows[rank][col]
        rows[rank] = [x / scale for x in rows[rank]]
        for row in range(len(rows)):
            if row != rank and abs(rows[row][col]) > 1e-10:
                factor = rows[row][col]
                rows[row] = [rows[row][j] - factor * rows[rank][j] for j in range(dim)]
        rank += 1
    return rank == n

def project(a, b):
    scalar = a.dot(b) / b.dot(b)
    return Vector([scalar * x for x in b.components])

def gram_schmidt(vectors):
    orthonormal = []
    for v in vectors:
        w = v
        for u in orthonormal:
            proj = project(w, u)

```

```

        w = w - proj
        if w.magnitude() < 1e-10:
            continue
        orthonormal.append(w.normalize())
    return orthonormal

v1 = Vector([1, 0, 0])
v2 = Vector([1, 1, 0])
v3 = Vector([1, 1, 1])
basis = gram_schmidt([v1, v2, v3])
for i, u in enumerate(basis):
    print(f"u{i+1} = {u}")
    print(f"    |u{i+1}| = {u.magnitude():.6f}")

print(f"u1 · u2 = {basis[0].dot(basis[1]):.6f}")
print(f"u1 · u3 = {basis[0].dot(basis[2]):.6f}")
print(f"u2 · u3 = {basis[1].dot(basis[2]):.6f}")

```

Use It

Now the same thing with NumPy – what you’ll actually use in practice:

```

import numpy as np

a = np.array([1, 2, 3], dtype=float)
b = np.array([4, 5, 6], dtype=float)

print(f"a + b = {a + b}")
print(f"a · b = {np.dot(a, b)}")
print(f"|a| = {np.linalg.norm(a):.4f}")
print(f"cosine = {np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b)):.4f}")

W = np.random.randn(2, 3) * 0.1
x = np.array([1.0, 0.5, -0.3])
print(f"Wx = {W @ x}")

```

Rank, Projection, and QR with NumPy

```

import numpy as np

A = np.array([[1, 2], [2, 4]])
print(f"Rank: {np.linalg.matrix_rank(A)}")

a = np.array([3, 4])
b = np.array([1, 0])
proj = (np.dot(a, b) / np.dot(b, b)) * b
print(f"Projection of {a} onto {b}: {proj}")

Q, R = np.linalg.qr(np.random.randn(3, 3))
print(f"Q is orthogonal: {np.allclose(Q @ Q.T, np.eye(3))}")
print(f"R is upper triangular: {np.allclose(R, np.triu(R))}")

```


PyTorch - Tensors Are Vectors with Autodiff

```
import torch

x = torch.randn(3, requires_grad=True)
y = torch.tensor([1.0, 0.0, 0.0])

similarity = torch.dot(x, y)
similarity.backward()

print(f"x = {x.data}")
print(f"y = {y.data}")
print(f"dot product = {similarity.item():.4f}")
print(f"d(dot)/dx = {x.grad}")
```

The gradient of the dot product with respect to x is just y . PyTorch computed this automatically. Every operation in a neural network is built from operations like this – matrix multiplies, dot products, projections – and autodiff tracks gradients through all of them.

You just built from scratch what NumPy does in one line. Now you know what's happening under the hood.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/01-linear-algebra-intuition>

Connections

Everything in this lesson connects to specific parts of modern AI:

Concept	Where it shows up
Dot product	Attention scores in transformers, cosine similarity in RAG
Matrix multiply	Every neural network layer, every linear transformation
Linear independence	Feature selection, avoiding multicollinearity
Rank	Determining if a system is solvable, LoRA (low-rank adaptation)
Projection	Linear regression (projecting onto column space), PCA
Gram-Schmidt / QR	Numerical solvers, eigenvalue computation
Orthonormal basis	Stable numerical computation, whitening transforms

LoRA deserves special mention. It fine-tunes large language models by decomposing weight updates into low-rank matrices. Instead of updating a 4096×4096 weight matrix (16M parameters), LoRA updates two matrices of size 4096×16 and 16×4096 (131K parameters). The rank-16 constraint means LoRA assumes the weight update lives in a 16-dimensional subspace of the full 4096-dimensional space. That is linear algebra doing real work.

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/01-linear-algebra-intuition/code>

1. Implement `Vector.angle_between(other)` that returns the angle in degrees between two vectors

2. Create a 2D scaling matrix that doubles the x-coordinate and triples the y-coordinate, then apply it to the vector $[1, 1]$
3. Given 5 random word-like vectors (dimension 50), find the two most similar using cosine similarity
4. Verify that the Gram-Schmidt output is truly orthonormal: check that every pair has dot product 0 and every vector has magnitude 1
5. Create a 3×3 matrix with rank 2. Verify using the `rank()` method. Then explain what geometric object the columns span.
6. Project the vector $[1, 2, 3]$ onto $[1, 1, 1]$. What does the result represent geometrically?

Key Terms

Term	What people say	What it actually means
Vector	"An arrow"	A list of numbers representing a point or direction in n-dimensional space
Matrix	"A table of numbers"	A transformation that maps vectors from one space to another
Dot product	"Multiply and sum"	A measure of how aligned two vectors are – the core of similarity search
Embedding	"Some AI magic"	A vector that represents the meaning of something (word, image, user)
Linear independence	"They don't overlap"	No vector in the set can be written as a combination of the others
Rank	"How many dimensions"	The number of linearly independent columns (or rows) in a matrix
Projection	"The shadow"	The component of one vector in the direction of another
Basis	"The coordinate axes"	A minimal set of independent vectors that span the space
Orthonormal	"Perpendicular unit vectors"	Vectors that are mutually perpendicular and each have length 1

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/01-linear-algebra-intuition>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/01-linear-algebra-intuition/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/01-linear-algebra-intuition>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Vectors, Matrices & Operations

Every neural network is just matrix multiplication with extra steps.

Type: Build **Languages:** Python, Julia **Prerequisites:** Phase 1, Lesson 01 (Linear Algebra Intuition) **Time:** ~60 minutes

Learning Objectives

- Build a Matrix class with element-wise operations, matrix multiplication, transpose, determinant, and inverse
- Distinguish element-wise multiplication from matrix multiplication and explain when each applies
- Implement a single dense neural network layer ($\text{relu}(W @ x + b)$) using only the from-scratch Matrix class
- Explain broadcasting rules and how bias addition works in neural network frameworks

The Problem

You want to build a neural network. You read the code and see this:

```
output = activation(weights @ input + bias)
```

That `@` is matrix multiplication. The weights are a matrix. The input is a vector. If you do not know what those operations do, this line is magic. If you do know, it is the entire forward pass of a layer in three operations.

Every image your model processes is a matrix of pixel values. Every word embedding is a vector. Every layer of every neural network is a matrix transformation. You cannot build AI systems without being fluent in matrix operations the same way you cannot write code without understanding variables.

This lesson builds that fluency from scratch.

The Concept

Vectors: ordered lists of numbers

A vector is a list of numbers with a direction and magnitude. In AI, vectors represent data points, features, or parameters.

```
v = [3, 4]      -- a 2D vector  
w = [1, 0, -2]  -- a 3D vector
```

A 2D vector `[3, 4]` points to coordinates (3, 4) on a plane. Its length (magnitude) is 5 (the 3-4-5 triangle).

Matrices: grids of numbers

A matrix is a 2D grid. Rows and columns. An $m \times n$ matrix has m rows and n columns.

$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$ -- 2x3 matrix (2 rows, 3 columns)

In neural networks, weight matrices transform input vectors into output vectors. A layer with 784 inputs and 128 outputs uses a 128x784 weight matrix.

Why shapes matter

Matrix multiplication has a strict rule: $(m \times n) @ (n \times p) = (m \times p)$. The inner dimensions must match.

$(128 \times 784) @ (784 \times 1) = (128 \times 1)$
 weights input output

Inner dimensions: 784 = 784 -- valid

If you get a shape mismatch error in PyTorch, this is why.

The operations map

Operation	What it does	Neural network use
Addition	Element-wise combine	Adding bias to output
Scalar multiply	Scale every element	Learning rate * gradients
Matrix multiply	Transform vectors	Layer forward pass
Transpose	Flip rows and columns	Backpropagation
Determinant	Single number summary	Checking invertibility
Inverse	Undo a transformation	Solving linear systems
Identity	Do-nothing matrix	Initialization, residual connections

Element-wise vs matrix multiplication

This distinction trips up beginners constantly.

Element-wise: multiply matching positions. Both matrices must be the same shape.

$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} * \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} = \begin{bmatrix} 5 & 12 \\ 21 & 32 \end{bmatrix}$

Matrix multiplication: dot products of rows and columns. Inner dimensions must match.

$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} @ \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} = \begin{bmatrix} 1*5+2*7 & 1*6+2*8 \\ 3*5+4*7 & 3*6+4*8 \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$

Different operations, different results, different rules.

Broadcasting

When you add a bias vector to a matrix of outputs, the shapes do not match. Broadcasting stretches the smaller array to fit.

$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} + [10, 20, 30]$

Broadcasting stretches the vector across rows:

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} + \begin{bmatrix} 10 & 20 & 30 \\ 10 & 20 & 30 \end{bmatrix} = \begin{bmatrix} 11 & 22 & 33 \\ 14 & 25 & 36 \end{bmatrix}$$

Every modern framework does this automatically. Understanding it prevents confusion when shapes seem wrong but the code runs.

Interactive figure: vector-projection. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/02-vectors-matrices-operations>

Build It

Step 1: Vector class

```
class Vector:
    def __init__(self, data):
        self.data = list(data)
        self.size = len(self.data)

    def __repr__(self):
        return f"Vector({self.data})"

    def __add__(self, other):
        return Vector([a + b for a, b in zip(self.data, other.data)])

    def __sub__(self, other):
        return Vector([a - b for a, b in zip(self.data, other.data)])

    def __mul__(self, scalar):
        return Vector([x * scalar for x in self.data])

    def dot(self, other):
        return sum(a * b for a, b in zip(self.data, other.data))

    def magnitude(self):
        return sum(x ** 2 for x in self.data) ** 0.5
```

Step 2: Matrix class with core operations

```
class Matrix:
    def __init__(self, data):
        self.data = [list(row) for row in data]
        self.rows = len(self.data)
        self.cols = len(self.data[0])
        self.shape = (self.rows, self.cols)

    def __repr__(self):
        rows_str = "\n ".join(str(row) for row in self.data)
        return f"Matrix({self.shape}):\n {rows_str}"

    def __add__(self, other):
        return Matrix([
```

```

        [self.data[i][j] + other.data[i][j] for j in range(self.cols)]
        for i in range(self.rows)
    ])

    def __sub__(self, other):
        return Matrix([
            [self.data[i][j] - other.data[i][j] for j in range(self.cols)]
            for i in range(self.rows)
        ])

    def scalar_multiply(self, scalar):
        return Matrix([
            [self.data[i][j] * scalar for j in range(self.cols)]
            for i in range(self.rows)
        ])

    def element_wise_multiply(self, other):
        return Matrix([
            [self.data[i][j] * other.data[i][j] for j in range(self.cols)]
            for i in range(self.rows)
        ])

    def matmul(self, other):
        return Matrix([
            [
                sum(self.data[i][k] * other.data[k][j] for k in range(self.cols))
                for j in range(other.cols)
            ]
            for i in range(self.rows)
        ])

    def transpose(self):
        return Matrix([
            [self.data[j][i] for j in range(self.rows)]
            for i in range(self.cols)
        ])

    def determinant(self):
        if self.shape == (1, 1):
            return self.data[0][0]
        if self.shape == (2, 2):
            return self.data[0][0] * self.data[1][1] - self.data[0][1] * self.data[1][0]
        det = 0
        for j in range(self.cols):
            minor = Matrix([
                [self.data[i][k] for k in range(self.cols) if k != j]
                for i in range(1, self.rows)
            ])
            det += ((-1) ** j) * self.data[0][j] * minor.determinant()
        return det

    def inverse_2x2(self):
        det = self.determinant()
        if det == 0:

```

```

        raise ValueError("Matrix is singular, no inverse exists")
    return Matrix([
        [self.data[1][1] / det, -self.data[0][1] / det],
        [-self.data[1][0] / det, self.data[0][0] / det]
    ])

    @staticmethod
    def identity(n):
        return Matrix([
            [1 if i == j else 0 for j in range(n)]
            for i in range(n)
        ])

```

Step 3: See it work

```

A = Matrix([[1, 2], [3, 4]])
B = Matrix([[5, 6], [7, 8]])

print("A + B =", (A + B).data)
print("A @ B =", A.matmul(B).data)
print("A^T =", A.transpose().data)
print("det(A) =", A.determinant())
print("A^-1 =", A.inverse_2x2().data)

I = Matrix.identity(2)
print("A @ A^-1 =", A.matmul(A.inverse_2x2()).data)

```

Step 4: Connect to neural networks

```

import random

inputs = Matrix([[0.5], [0.8], [0.2]])
weights = Matrix([
    [random.uniform(-1, 1) for _ in range(3)]
    for _ in range(2)
])
bias = Matrix([[0.1], [0.1]])

def relu_matrix(m):
    return Matrix([[max(0, val) for val in row] for row in m.data])

pre_activation = weights.matmul(inputs) + bias
output = relu_matrix(pre_activation)

print(f"Input shape: {inputs.shape}")
print(f"Weight shape: {weights.shape}")
print(f"Output shape: {output.shape}")
print(f"Output: {output.data}")

```

This is a single dense layer: $\text{output} = \text{relu}(W @ x + b)$. Every dense layer in every neural network does exactly this.

Use It

NumPy does everything above in fewer lines and orders of magnitude faster.

```
import numpy as np

A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

print("A + B =\n", A + B)
print("A * B (element-wise) =\n", A * B)
print("A @ B (matrix multiply) =\n", A @ B)
print("A^T =\n", A.T)
print("det(A) =", np.linalg.det(A))
print("A^-1 =\n", np.linalg.inv(A))
print("I =\n", np.eye(2))

inputs = np.random.randn(3, 1)
weights = np.random.randn(2, 3)
bias = np.array([[0.1], [0.1]])
output = np.maximum(0, weights @ inputs + bias)

print(f"\nNeural network layer: {weights.shape} @ {inputs.shape} = {output.shape}")
print(f"Output:\n{output}")
```

The @ operator in Python calls `__matmul__`. NumPy implements it with optimized BLAS routines written in C and Fortran. Same math, 100x faster.

Broadcasting in NumPy:

```
matrix = np.array([[1, 2, 3], [4, 5, 6]])
bias = np.array([10, 20, 30])
print(matrix + bias)
```

NumPy automatically broadcasts the 1D bias across both rows. This is how bias addition works in every neural network framework.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/02-vectors-matrices-operations>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/02-vectors-matrices-operations/code>

1. **Verify the inverse.** Multiply `A @ A.inverse_2x2()` and confirm you get the identity matrix. Try it with three different 2x2 matrices. What happens when the determinant is zero?
2. **Implement 3x3 inverse.** Extend the Matrix class to compute inverses for 3x3 matrices using the adjugate method. Test it against NumPy's `np.linalg.inv`.
3. **Build a two-layer network.** Using only your Matrix class (no NumPy), create a two-layer neural network: input (3) -> hidden (4) -> output (2). Initialize random weights,

run a forward pass, and verify all shapes are correct.

Key Terms

Term	What people say	What it actually means
Vector	“An arrow”	An ordered list of numbers. In AI: a point in high-dimensional space.
Matrix	“A table of numbers”	A linear transformation. It maps vectors from one space to another.
Matrix multiply	“Just multiply the numbers”	Dot products between every row of the first matrix and every column of the second. Order matters.
Transpose	“Flip it”	Swap rows and columns. Turns an $m \times n$ matrix into $n \times m$. Critical in backpropagation.
Determinant	“Some number from the matrix”	Measures how much the matrix scales area (2D) or volume (3D). Zero means the transformation crushes a dimension.
Inverse	“Undo the matrix”	The matrix that reverses the transformation. Only exists when the determinant is not zero.
Identity matrix	“The boring matrix”	The matrix equivalent of multiplying by 1. Used in residual connections (ResNets).
Broadcasting	“Magic shape fixing”	Stretching a smaller array to match a larger one by repeating along missing dimensions.
Element-wise	“Regular multiplication”	Multiply matching positions. Both arrays must have the same shape (or be broadcastable).

Further Reading

- 3Blue1Brown: Essence of Linear Algebra - visual intuition for every operation covered here
- NumPy documentation on broadcasting - the exact rules NumPy follows
- Stanford CS229 Linear Algebra Review - concise reference for ML-specific linear algebra

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/02-vectors-matrices-operations>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/02-vectors-matrices-operations/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/02-vectors-matrices-operations>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Matrix Transformations

A matrix is a machine that reshapes space. Learn what it does to every point, and you understand the whole transformation.

Type: Build **Languages:** Python, Julia **Prerequisites:** Phase 1, Lessons 01-02 (Linear Algebra Intuition, Vectors & Matrices Operations) **Time:** ~75 minutes

Learning Objectives

- Construct rotation, scaling, shearing, and reflection matrices and apply them to 2D and 3D points
- Compose multiple transformations by matrix multiplication and verify that order matters
- Compute eigenvalues and eigenvectors of 2x2 matrices from the characteristic equation
- Explain why eigenvalues determine PCA directions, RNN stability, and spectral clustering behavior

The Problem

You read about PCA and see “find the eigenvectors of the covariance matrix.” You read about model stability and see “check if all eigenvalues have magnitude less than 1.” You read about data augmentation and see “apply a random rotation.” None of this makes sense until you understand what matrices do to space geometrically.

Matrices are not just grids of numbers. They are spatial machines. A rotation matrix spins points. A scaling matrix stretches them. A shearing matrix tilts them. Every transformation a neural network applies to data is one of these operations or a composition of them. This lesson makes those operations concrete.

The Concept

Transformations as matrices

Every linear transformation in 2D can be written as a 2x2 matrix. The matrix tells you exactly where the basis vectors $[1, 0]$ and $[0, 1]$ end up. Everything else follows.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/03-matrix-transformations>

Rotation

A 2D rotation by angle θ keeps distances and angles intact. It moves every point along a circular arc.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/03-matrix-transformations>

In 3D, you rotate around an axis. Each axis has its own rotation matrix:

$$R_z(\theta) = \begin{vmatrix} \cos & -\sin & 0 \\ \sin & \cos & 0 \\ 0 & 0 & 1 \end{vmatrix} \quad \begin{array}{l} \text{Rotate around z-axis} \\ \text{(x-y plane spins, z stays)} \end{array}$$

$$R_x(\theta) = \begin{vmatrix} 1 & 0 & 0 \\ 0 & \cos & -\sin \\ 0 & \sin & \cos \end{vmatrix} \quad \begin{array}{l} \text{Rotate around x-axis} \\ \text{(y-z plane spins, x stays)} \end{array}$$

$$R_y(\theta) = \begin{vmatrix} \cos & 0 & \sin \\ 0 & 1 & 0 \\ -\sin & 0 & \cos \end{vmatrix} \quad \begin{array}{l} \text{Rotate around y-axis} \\ \text{(x-z plane spins, y stays)} \end{array}$$

Scaling

Scaling stretches or compresses along each axis independently.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/03-matrix-transformations>

Shearing

Shearing tilts one axis while keeping the other fixed. It turns rectangles into parallelograms.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/03-matrix-transformations>

Shear matrices: - $Sh_x = \begin{bmatrix} 1 & k \\ 0 & 1 \end{bmatrix}$ shifts x by $k * y$ - $Sh_y = \begin{bmatrix} 1 & 0 \\ k & 1 \end{bmatrix}$ shifts y by $k * x$

Reflection

Reflection mirrors points across an axis or line.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/03-matrix-transformations>

Reflection matrices: - Reflect across y-axis: $\begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}$ - Reflect across x-axis: $\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$

Composition: chaining transformations

Applying transformation A then B is the same as multiplying their matrices: $result = B @ A$ @ point. Order matters. Rotate then scale gives different results than scale then rotate.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/03-matrix-transformations>

Composed: $S @ R = \begin{bmatrix} 0 & -2 \\ 0.5 & 0 \end{bmatrix}$

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/03-matrix-transformations>

Composed: $R @ S = \begin{bmatrix} 0 & -0.5 \\ 2 & 0 \end{bmatrix}$

Different results. Matrix multiplication is not commutative.

Eigenvalues and eigenvectors

Most vectors change direction when a matrix hits them. Eigenvectors are special: the matrix only scales them, never rotates them. The scaling factor is the eigenvalue.

$$A @ v = \lambda * v$$

v is the eigenvector (direction that survives)
 λ is the eigenvalue (how much it stretches)

$$\text{Example: } A = \begin{vmatrix} 2 & 1 \\ 1 & 2 \end{vmatrix}$$

Eigenvector $[1, 1]$ with eigenvalue 3:

$$A @ [1, 1] = [3, 3] = 3 * [1, 1] \quad (\text{same direction, scaled by 3})$$

Eigenvector $[1, -1]$ with eigenvalue 1:

$$A @ [1, -1] = [1, -1] = 1 * [1, -1] \quad (\text{same direction, unchanged})$$

The matrix stretches space by 3x along $[1, 1]$ and keeps $[1, -1]$ unchanged. Every other direction is a mix of these two.

Eigendecomposition

If a matrix has n linearly independent eigenvectors, it can be decomposed:

$$A = V @ D @ V^{-1}$$

V = matrix whose columns are eigenvectors

D = diagonal matrix of eigenvalues

V^{-1} = inverse of V

This says: rotate into eigenvector coordinates, scale along each axis, rotate back.

Why eigenvalues matter

PCA. The eigenvectors of the covariance matrix are the principal components. The eigenvalues tell you how much variance each component captures. Sort by eigenvalue, keep the top k , and you have dimensionality reduction.

Stability. In recurrent networks and dynamical systems, eigenvalues with magnitude > 1 cause outputs to explode. Magnitude < 1 causes them to vanish. This is the vanishing/exploding gradient problem stated in one sentence.

Spectral methods. Graph neural networks use eigenvalues of the adjacency matrix. Spectral clustering uses eigenvalues of the Laplacian. The eigenvectors reveal the structure of the graph.

Determinant as volume scaling factor

The determinant of a transformation matrix tells you how much it scales area (2D) or volume (3D).

$\det = 1$: area preserved (rotation)

$\det = 2$: area doubled

$\det = 0$: space crushed to lower dimension (singular)

$\det = -1$: area preserved but orientation flipped (reflection)

```
| det(Rotation) | = 1          (always)
| det(Scale sx, sy) | = sx * sy
| det(Shear) | = 1            (area preserved)
| det(Reflection) | = -1      (orientation flipped)
```

Interactive figure: matrix-transform. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/03-matrix-transformations>

Build It

Step 1: Transformation matrices from scratch (Python)

```
import math

def rotation_2d(theta):
    c, s = math.cos(theta), math.sin(theta)
    return [[c, -s], [s, c]]

def scaling_2d(sx, sy):
    return [[sx, 0], [0, sy]]

def shearing_2d(kx, ky):
    return [[1, kx], [ky, 1]]

def reflection_x():
    return [[1, 0], [0, -1]]

def reflection_y():
    return [[-1, 0], [0, 1]]

def mat_vec_mul(matrix, vector):
    return [
        sum(matrix[i][j] * vector[j] for j in range(len(vector)))
        for i in range(len(matrix))
    ]

def mat_mul(a, b):
    rows_a, cols_b = len(a), len(b[0])
    cols_a = len(a[0])
    return [
        [sum(a[i][k] * b[k][j] for k in range(cols_a)) for j in range(cols_b)]
        for i in range(rows_a)
    ]

point = [1.0, 0.0]
angle = math.pi / 4

rotated = mat_vec_mul(rotation_2d(angle), point)
print(f"Rotate (1,0) by 45 deg: ({rotated[0]:.4f}, {rotated[1]:.4f})")

scaled = mat_vec_mul(scaling_2d(2, 3), [1.0, 1.0])
```

```

print(f"Scale (1,1) by (2,3): ({scaled[0]:.1f}, {scaled[1]:.1f})")

sheared = mat_vec_mul(shearing_2d(1, 0), [1.0, 1.0])
print(f"Shear (1,1) kx=1: ({sheared[0]:.1f}, {sheared[1]:.1f})")

reflected = mat_vec_mul(reflection_y(), [2.0, 1.0])
print(f"Reflect (2,1) across y: ({reflected[0]:.1f}, {reflected[1]:.1f})")

```

Step 2: Composition of transformations

```

R = rotation_2d(math.pi / 2)
S = scaling_2d(2, 0.5)

rotate_then_scale = mat_mul(S, R)
scale_then_rotate = mat_mul(R, S)

point = [1.0, 0.0]
result1 = mat_vec_mul(rotate_then_scale, point)
result2 = mat_vec_mul(scale_then_rotate, point)

print(f"Rotate 90 then scale: ({result1[0]:.2f}, {result1[1]:.2f})")
print(f"Scale then rotate 90: ({result2[0]:.2f}, {result2[1]:.2f})")
print(f"Same? {result1 == result2}")

```

Step 3: Eigenvalues from scratch (2x2)

For a 2x2 matrix $\begin{bmatrix} a & b \\ c & d \end{bmatrix}$, eigenvalues solve the characteristic equation: $\lambda^2 - (a+d)\lambda + (ad - bc) = 0$.

```

def eigenvalues_2x2(matrix):
    a, b = matrix[0]
    c, d = matrix[1]
    trace = a + d
    det = a * d - b * c
    discriminant = trace ** 2 - 4 * det
    if discriminant < 0:
        real = trace / 2
        imag = (-discriminant) ** 0.5 / 2
        return (complex(real, imag), complex(real, -imag))
    sqrt_disc = discriminant ** 0.5
    return ((trace + sqrt_disc) / 2, (trace - sqrt_disc) / 2)

def eigenvector_2x2(matrix, eigenvalue):
    a, b = matrix[0]
    c, d = matrix[1]
    if abs(b) > 1e-10:
        v = [b, eigenvalue - a]
    elif abs(c) > 1e-10:
        v = [eigenvalue - d, c]
    else:
        if abs(a - eigenvalue) < 1e-10:
            v = [1, 0]
        else:
            v = [0, 1]

```

```

mag = (v[0] ** 2 + v[1] ** 2) ** 0.5
return [v[0] / mag, v[1] / mag]

A = [[2, 1], [1, 2]]
vals = eigenvalues_2x2(A)
print(f"Matrix: {A}")
print(f"Eigenvalues: {vals[0]:.4f}, {vals[1]:.4f}")

for val in vals:
    vec = eigenvector_2x2(A, val)
    result = mat_vec_mul(A, vec)
    scaled = [val * vec[0], val * vec[1]]
    print(f"    lambda={val:.1f}, v=[{round(x,4) for x in vec}]")
    print(f"    A@v = [{round(x,4) for x in result}]")
    print(f"    l*v = [{round(x,4) for x in scaled}]")

```

Step 4: Determinant as volume scaling factor

```

def det_2x2(matrix):
    return matrix[0][0] * matrix[1][1] - matrix[0][1] * matrix[1][0]

print(f"det(rotation 45) = {det_2x2(rotation_2d(math.pi/4)):.4f}")
print(f"det(scale 2,3)    = {det_2x2(scaling_2d(2, 3)):.1f}")
print(f"det(shear kx=1)   = {det_2x2(shearing_2d(1, 0)):.1f}")
print(f"det(reflect y)     = {det_2x2(reflection_y()):.1f}")

singular = [[1, 2], [2, 4]]
print(f"det(singular)      = {det_2x2(singular):.1f}")
print("Singular: columns are proportional, space collapses to a line.")

```

Use It

NumPy handles all of this with optimized routines.

```

import numpy as np

theta = np.pi / 4
R = np.array([[np.cos(theta), -np.sin(theta)],
              [np.sin(theta),  np.cos(theta)]])

point = np.array([1.0, 0.0])
print(f"Rotate (1,0) by 45 deg: {R @ point}")

S = np.diag([2.0, 3.0])
composed = S @ R
print(f"Scale(2,3) after Rotate(45): {composed @ point}")

A = np.array([[2, 1], [1, 2]], dtype=float)
eigenvalues, eigenvectors = np.linalg.eig(A)
print(f"\nEigenvalues: {eigenvalues}")
print(f"Eigenvectors (columns):\n{eigenvectors}")

for i in range(len(eigenvalues)):

```

```

v = eigenvectors[:, i]
lam = eigenvalues[i]
print(f" A @ v{i} = {A @ v}, lambda * v{i} = {lam * v}")

print(f"\ndet(R) = {np.linalg.det(R):.4f}")
print(f"\ndet(S) = {np.linalg.det(S):.1f}")

B = np.array([[3, 1], [0, 2]], dtype=float)
vals, vecs = np.linalg.eig(B)
D = np.diag(vals)
V = vecs
reconstructed = V @ D @ np.linalg.inv(V)
print(f"\nEigendecomposition A = V @ D @ V^-1:")
print(f"Original:\n{B}")
print(f"Reconstructed:\n{reconstructed}")

```

3D rotations with NumPy

```

def rotation_3d_z(theta):
    c, s = np.cos(theta), np.sin(theta)
    return np.array([[c, -s, 0], [s, c, 0], [0, 0, 1]])

def rotation_3d_x(theta):
    c, s = np.cos(theta), np.sin(theta)
    return np.array([[1, 0, 0], [0, c, -s], [0, s, c]])

point_3d = np.array([1.0, 0.0, 0.0])
rotated_z = rotation_3d_z(np.pi / 2) @ point_3d
rotated_x = rotation_3d_x(np.pi / 2) @ point_3d

print(f"\n3D point: {point_3d}")
print(f"Rotate 90 around z: {np.round(rotated_z, 4)}")
print(f"Rotate 90 around x: {np.round(rotated_x, 4)}")

```

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/03-matrix-transformations>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/03-matrix-transformations/code>

1. Apply rotation, scaling, and shearing to a unit square (corners at [0,0], [1,0], [1,1], [0,1]). Print the transformed corners for each. Verify that rotation preserves distances between corners.
2. Find the eigenvalues of the matrix $\begin{bmatrix} 4 & 2 \\ 1 & 3 \end{bmatrix}$ by hand using the characteristic equation. Then verify with your from-scratch function and with NumPy.
3. Create a composition of three transformations (rotate 30 degrees, scale by [1.5, 0.8], shear with $k_x=0.3$) and apply it to 8 points arranged in a circle. Print before and after coordinates. Compute the determinant of the composed matrix and verify it equals the product of the individual determinants.

Key Terms

Term	What people say	What it actually means
Rotation matrix	“Spins things”	An orthogonal matrix that moves points along circular arcs while preserving distances and angles. Determinant is always 1.
Scaling matrix	“Makes things bigger”	A diagonal matrix that stretches or compresses independently along each axis. Determinant is the product of scale factors.
Shearing matrix	“Slants things”	A matrix that shifts one coordinate proportionally to another, turning rectangles into parallelograms. Determinant is 1.
Reflection	“Mirrors things”	A matrix that flips space across an axis or plane. Determinant is -1.
Composition	“Do two things”	Multiplying transformation matrices to chain operations. Order matters: $B @ A$ means apply A first, then B.
Eigenvector	“Special direction”	A direction that the matrix only scales, never rotates. The transformation’s fingerprint.
Eigenvalue	“How much it stretches”	The scalar factor by which the matrix scales its eigenvector. Can be negative (flip) or complex (rotation).
Eigendecomposition	“Break the matrix apart”	Writing a matrix as $V @ D @ V^{-1}$, separating it into its fundamental scaling directions and magnitudes.
Determinant	“A single number from a matrix”	The factor by which the transformation scales area (2D) or volume (3D). Zero means the transformation is irreversible.
Characteristic equation	“Where eigenvalues come from”	$\det(A - \lambda I) = 0$. The polynomial whose roots are the eigenvalues.

Further Reading

- 3Blue1Brown: Linear Transformations – visual intuition for how matrices reshape space
- 3Blue1Brown: Eigenvectors and Eigenvalues – the best visual explanation of what eigenvectors mean geometrically
- MIT 18.06 Lecture 21: Eigenvalues and Eigenvectors – Gilbert Strang’s classic treatment

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/03-matrix-transformations>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/03-matrix-transformations/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/03-matrix-transformations>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Calculus for Machine Learning

Derivatives tell you which way is downhill. That is all a neural network needs to learn.

Type: Learn **Language:** Python **Prerequisites:** Phase 1, Lessons 01-03 **Time:** ~60 minutes

Learning Objectives

- Compute numerical and analytical derivatives for common ML functions (x^2 , sigmoid, cross-entropy)
- Implement gradient descent from scratch to minimize a loss function in 1D and 2D
- Derive the gradient of a linear regression model and train it via manual weight updates
- Explain the Hessian matrix, Taylor series approximations, and their connection to optimization methods

The Problem

You have a neural network with millions of weights. Each weight is a knob. You need to figure out which direction to turn every single knob to make the model slightly less wrong. Calculus gives you that direction.

Without calculus, training a neural network would mean trying random changes and hoping for the best. With derivatives, you know exactly how each weight affects the error. You turn every knob the right way, every time.

The Concept

What is a derivative?

A derivative measures the rate of change. For a function $y = f(x)$, the derivative $f'(x)$ tells you: if you nudge x by a tiny amount, how much does y change?

Geometrically, the derivative is the slope of the tangent line at a point.

$f(x) = x^2$:

x	f(x)	f'(x) (slope)
0	0	0 (flat, at the bottom)
1	1	2
2	4	4 (tangent line slope at this point)
3	9	6

At $x=2$, the slope is 4. If you move x a tiny bit to the right, y increases by about 4 times that amount. At $x=0$, the slope is 0. You are at the bottom of the bowl.

The formal definition:

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

In code, you skip the limit and just use a very small h . That is the numerical derivative.

Partial derivatives: one variable at a time

Real functions have many inputs. A neural network loss depends on thousands of weights. A partial derivative holds all variables constant except one, then takes the derivative with respect to that one.

$$f(x, y) = x^2 + 3xy + y^2$$

$$df/dx = 2x + 3y \quad (\text{treat } y \text{ as a constant})$$

$$df/dy = 3x + 2y \quad (\text{treat } x \text{ as a constant})$$

Each partial derivative answers: if I nudge just this one weight, how does the loss change?

The gradient: vector of all partial derivatives

The gradient collects every partial derivative into one vector. For a function $f(x, y, z)$, the gradient is:

$$\text{grad } f = [df/dx, df/dy, df/dz]$$

The gradient points in the direction of steepest ascent. To minimize a function, go in the opposite direction.

Contour plot of $f(x,y) = x^2 + y^2$:

The function forms a bowl shape with concentric circles as contour lines. The minimum is at (0, 0).

Point	grad f	-grad f (descent direction)
(1, 1)	[2, 2] (points uphill, away from minimum)	[-2, -2] (points downhill, toward minimum)
(0, 0)	[0, 0] (flat, at the minimum)	[0, 0]

This is gradient descent in a picture. Compute the gradient, negate it, take a step.

The connection to optimization

Training a neural network is optimization. You have a loss function $L(w_1, w_2, \dots, w_n)$ that measures how wrong the model is. You want to minimize it.

Gradient descent update rule:

$$w_{\text{new}} = w_{\text{old}} - \text{learning_rate} * dL/dw$$

For every weight:

1. Compute the partial derivative of loss with respect to that weight

2. Subtract a small multiple of it from the weight
3. Repeat

The learning rate controls step size. Too big and you overshoot. Too small and you crawl.

Loss landscape (1D slice):

The loss function $L(w)$ forms a curve with peaks and valleys as the weight w varies.

Feature	Description
Global minimum	The lowest point on the entire curve – the best solution
Local minimum	A valley that is lower than its neighbors but not the lowest overall
Slope	Gradient descent follows the slope downhill from any starting point

Gradient descent follows the slope downhill. It can get stuck in local minima, but in high-dimensional spaces (millions of weights) this is rarely a practical problem.

Numerical vs analytical derivatives

There are two ways to compute a derivative.

Analytical: apply calculus rules by hand. For $f(x) = x^2$, the derivative is $f'(x) = 2x$. Exact. Fast.

Numerical: approximate using the definition. Compute $f(x+h)$ and $f(x-h)$ for a tiny h , then use the difference.

Numerical (central difference):

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}$$

$h = 0.0001$ works well in practice

Numerical derivatives are slower but work for any function. Analytical derivatives are fast but require you to derive the formula. Neural network frameworks use a third approach: automatic differentiation, which computes exact derivatives mechanically. You will see that in Phase 3.

Derivatives by hand for simple functions

These are the derivatives you will see over and over in ML.

Function	Derivative	Used in
-----	-----	-----
$f(x) = x^2$	$f'(x) = 2x$	Loss functions (MSE)
$f(x) = wx + b$	$f'(w) = x$	Linear layer (gradient w.r.t. weight)
	$f'(b) = 1$	Linear layer (gradient w.r.t. bias)
	$f'(x) = w$	Linear layer (gradient w.r.t. input)
$f(x) = e^x$	$f'(x) = e^x$	Softmax, attention
$f(x) = \ln(x)$	$f'(x) = 1/x$	Cross-entropy loss
$f(x) = 1/(1+e^{-x})$	$f'(x) = f(x)(1-f(x))$	Sigmoid activation

For $f(x) = x^2$:

$$f(x) = x^2 \quad f'(x) = 2x$$

x	f(x)	f'(x)	meaning
-2	4	-4	slope tilts left (decreasing)
-1	1	-2	slope tilts left (decreasing)
0	0	0	flat (minimum!)
1	1	2	slope tilts right (increasing)
2	4	4	slope tilts right (increasing)

For $f(w) = wx + b$ with $x=3$, $b=1$:

$$f(w) = 3w + 1 \quad f'(w) = 3$$

The derivative with respect to w is just x .

If x is big, a small change in w causes a big change in output.

The chain rule

When functions are composed, the chain rule tells you how to differentiate.

If $y = f(g(x))$, then $dy/dx = f'(g(x)) * g'(x)$

Example: $y = (3x + 1)^2$

$$\text{outer: } f(u) = u^2 \quad f'(u) = 2u$$

$$\text{inner: } g(x) = 3x + 1 \quad g'(x) = 3$$

$$dy/dx = 2(3x + 1) * 3 = 6(3x + 1)$$

Neural networks are chains of functions: input -> linear -> activation -> linear -> activation -> loss. Backpropagation is the chain rule applied repeatedly from output to input. That is the entire algorithm.

The Hessian Matrix

The gradient tells you the slope. The Hessian tells you the curvature.

The Hessian is the matrix of second-order partial derivatives. For a function $f(x_1, x_2, \dots, x_n)$, entry (i, j) of the Hessian is:

$$H[i][j] = d^2f / (dx_i * dx_j)$$

For a 2-variable function $f(x, y)$:

$$H = \begin{vmatrix} d^2f/dx^2 & d^2f/dxdy \\ d^2f/dydx & d^2f/dy^2 \end{vmatrix}$$

What the Hessian tells you at a critical point (where gradient = 0):

Hessian property	Meaning	Example surface
Positive definite (all eigenvalues > 0)	Local minimum	Bowl pointing up
Negative definite (all eigenvalues < 0)	Local maximum	Bowl pointing down
Indefinite (mixed eigenvalues)	Saddle point	Horse saddle shape

Example: $f(x, y) = x^2 - y^2$ (a saddle function)

$$\begin{aligned} df/dx &= 2x & df/dy &= -2y \\ d^2f/dx^2 &= 2 & d^2f/dy^2 &= -2 & d^2f/dxdy &= 0 \end{aligned}$$

$$H = \begin{vmatrix} 2 & 0 \\ 0 & -2 \end{vmatrix}$$

Eigenvalues: 2 and -2 (one positive, one negative)
--> Saddle point at (0, 0)

Compare with $f(x, y) = x^2 + y^2$ (a bowl):

$$H = \begin{vmatrix} 2 & 0 \\ 0 & 2 \end{vmatrix}$$

Eigenvalues: 2 and 2 (both positive)
--> Local minimum at (0, 0)

Why the Hessian matters in ML:

Newton's method uses the Hessian to take better optimization steps than gradient descent. Instead of just following the slope, it accounts for curvature:

Newton's update: $w_{\text{new}} = w_{\text{old}} - H^{-1} * \text{gradient}$
Gradient descent: $w_{\text{new}} = w_{\text{old}} - \text{lr} * \text{gradient}$

Newton's method converges faster because the Hessian "rescales" the gradient – steep directions get smaller steps, flat directions get larger steps.

The catch: for a neural network with N parameters, the Hessian is $N \times N$. A model with 1 million parameters would need a 1 trillion-entry matrix. That is why we use approximations.

Method	What it uses	Cost	Convergence
Gradient descent	First derivatives only	$O(N)$ per step	Slow (linear)
Newton's method	Full Hessian	$O(N^3)$ per step	Fast (quadratic)
L-BFGS	Approximate Hessian from gradient history	$O(N)$ per step	Medium (superlinear)
Adam	Per-parameter adaptive rates (diagonal Hessian approx)	$O(N)$ per step	Medium
Natural gradient	Fisher information matrix (statistical Hessian)	$O(N^2)$ per step	Fast

In practice, Adam is the default optimizer for deep learning. It approximates second-order information cheaply by tracking the running mean and variance of gradients per parameter.

Taylor Series Approximation

Any smooth function can be approximated locally by a polynomial:

$$f(x + h) = f(x) + f'(x)h + (1/2)f''(x)h^2 + (1/6)f'''(x)h^3 + \dots$$

The more terms you include, the better the approximation – but only near the point x .

Why Taylor series matter for ML:

- **First-order Taylor = gradient descent.** When you use $f(x + h) \sim f(x) + f'(x)h$, you are making a linear approximation. Gradient descent minimizes this linear model to choose $h = -1/r \ f'(x)$.
- **Second-order Taylor = Newton's method.** Using $f(x + h) \sim f(x) + f'(x)h + (1/2)f''(x)h^2$, you get a quadratic model. Minimizing it gives $h = -f'(x)/f''(x)$ - Newton's step.
- **Loss function design.** MSE and cross-entropy are smooth, which means their Taylor expansions are well-behaved. This is not an accident. Smooth losses make optimization predictable.

Approximation order	What it captures	Optimization method
-----	-----	-----
0th order (constant)	Just the value	Random search
1st order (linear)	Slope	Gradient descent
2nd order (quadratic)	Curvature	Newton's method
Higher orders	Finer structure	Rarely used in ML

The key insight: all gradient-based optimization is really about approximating the loss function locally and stepping to the minimum of that approximation.

Integrals in ML

Derivatives tell you rates of change. Integrals compute accumulations - area under a curve.

In ML, you rarely compute integrals by hand, but the concept is everywhere:

Probability. For a continuous random variable with density $p(x)$:

$$P(a < X < b) = \text{integral from } a \text{ to } b \text{ of } p(x) \, dx$$

The area under the probability density curve between a and b is the probability of landing in that range.

Expected value. The average outcome weighted by probability:

$$E[f(X)] = \text{integral of } f(x) * p(x) \, dx$$

The expected loss over a data distribution is an integral. Training minimizes an empirical approximation of this.

KL divergence. Measures how different two distributions are:

$$KL(p || q) = \text{integral of } p(x) * \log(p(x) / q(x)) \, dx$$

Used in VAEs, knowledge distillation, and Bayesian inference.

Normalization constants. In Bayesian inference:

$$p(w | \text{data}) = p(\text{data} | w) * p(w) / \text{integral of } p(\text{data} | w) * p(w) \, dw$$

The denominator is an integral over all possible parameter values. It is often intractable, which is why we use approximations like MCMC and variational inference.

Integral concept	Where it appears in ML
Area under curve	Probability from density functions
Expected value	Loss functions, risk minimization
KL divergence	VAEs, policy optimization, distillation
Normalization	Bayesian posteriors, softmax denominator
Marginal likelihood	Model comparison, evidence lower bound (ELBO)

Multivariable Chain Rule in a Computation Graph

The chain rule does not just apply to scalar functions in a line. In a neural network, variables fan out and merge. Here is how derivatives flow through a simple forward pass:

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/04-calculus-for-ml>

The backward pass computes gradients right to left:

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/04-calculus-for-ml>

Each arrow multiplies by the local derivative. The gradient for any parameter is the product of all local derivatives along the path from loss to that parameter. When paths branch and merge, you sum the contributions (multivariate chain rule).

This is all backpropagation is: the chain rule applied systematically through a computation graph, from output to inputs.

The Jacobian matrix

When a function maps a vector to a vector (like a neural network layer), its derivative is a matrix. The Jacobian contains every partial derivative of every output with respect to every input.

For $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$, the Jacobian J is an $m \times n$ matrix:

	x_1	x_2	...	x_n
f_1	df_1/dx_1	df_1/dx_2	...	df_1/dx_n
f_2	df_2/dx_1	df_2/dx_2	...	df_2/dx_n
...	...	...	...	...
f_m	df_m/dx_1	df_m/dx_2	...	df_m/dx_n

You will not compute Jacobians by hand for neural networks. PyTorch handles it. But knowing it exists helps you understand shapes in backpropagation: if a layer maps $\mathbb{R}^n$ to $\mathbb{R}^m$, its Jacobian is $m \times n$. The gradient flows backward through the transpose of this matrix.

Why this matters for neural networks

Every weight in a neural network gets a gradient. The gradient tells you how to adjust that weight to reduce the loss.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/04-calculus-for-ml>

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/04-calculus-for-ml>

Each weight update: $-W_1 = W_1 - \eta r * dL/dW_1$ $-W_2 = W_2 - \eta r * dL/dW_2$

The forward pass computes the prediction and loss. The backward pass computes the gradient of the loss with respect to every weight. Then every weight takes a small step downhill. Repeat for millions of steps. That is deep learning.

Interactive figure: derivative-tangent. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/04-calculus-for-ml>

Build It

Step 1: Numerical derivative from scratch

```
def numerical_derivative(f, x, h=1e-7):
    return (f(x + h) - f(x - h)) / (2 * h)

def f(x):
    return x ** 2

for x in [-2, -1, 0, 1, 2]:
    numerical = numerical_derivative(f, x)
    analytical = 2 * x
    print(f"x={x:2d}  f'(x) numerical={numerical:.6f}  analytical={analytical:.1f}")
```

The numerical derivative matches the analytical one to many decimal places.

Step 2: Partial derivatives and gradients

```
def numerical_gradient(f, point, h=1e-7):
    gradient = []
    for i in range(len(point)):
        point_plus = list(point)
        point_minus = list(point)
        point_plus[i] += h
        point_minus[i] -= h
        partial = (f(point_plus) - f(point_minus)) / (2 * h)
        gradient.append(partial)
    return gradient

def f_multi(point):
    x, y = point
    return x**2 + 3*x*y + y**2

grad = numerical_gradient(f_multi, [1.0, 2.0])
print(f"Numerical gradient at (1,2): {[f'{g:.4f}' for g in grad]}")
print(f"Analytical gradient at (1,2): [2*1+3*2, 3*1+2*2] = [{2*1+3*2}, {3*1+2*2}]")
```

Step 3: Gradient descent to find the minimum of $f(x) = x^2$

```
x = 5.0
lr = 0.1
for step in range(20):
    grad = 2 * x
    x = x - lr * grad
    print(f"step {step:2d}  x={x:8.4f}  f(x)={x**2:10.6f}")
```

Starting at $x=5$, each step moves closer to $x=0$ (the minimum).

Step 4: Gradient descent on a 2D function

```
def f_2d(point):
    x, y = point
    return x**2 + y**2
```

```

point = [4.0, 3.0]
lr = 0.1
for step in range(30):
    grad = numerical_gradient(f_2d, point)
    point = [p - lr * g for p, g in zip(point, grad)]
    loss = f_2d(point)
    if step % 5 == 0 or step == 29:
        print(f"step {step:2d} point=({point[0]:7.4f}, {point[1]:7.4f}) f={loss:.6f}")

```

Step 5: Comparing numerical and analytical derivatives

```

import math

test_functions = [
    ("x^2", lambda x: x**2, lambda x: 2*x),
    ("x^3", lambda x: x**3, lambda x: 3*x**2),
    ("sin(x)", lambda x: math.sin(x), lambda x: math.cos(x)),
    ("e^x", lambda x: math.exp(x), lambda x: math.exp(x)),
    ("1/x", lambda x: 1/x, lambda x: -1/x**2),
]

x = 2.0
print(f"{'Function':<12} {'Numerical':>12} {'Analytical':>12} {'Error':>12}")
print("-" * 50)
for name, f, df in test_functions:
    num = numerical_derivative(f, x)
    ana = df(x)
    err = abs(num - ana)
    print(f"{name:<12} {num:12.6f} {ana:12.6f} {err:12.2e}")

```

Step 6: Computing the Hessian numerically

```

def hessian_2d(f, x, y, h=1e-5):
    fxx = (f(x + h, y) - 2 * f(x, y) + f(x - h, y)) / (h ** 2)
    fyy = (f(x, y + h) - 2 * f(x, y) + f(x, y - h)) / (h ** 2)
    fxy = (f(x + h, y + h) - f(x + h, y - h) - f(x - h, y + h) + f(x - h, y - h)) / (4 * h ** 2)
    return [[fxx, fxy], [fxy, fyy]]

def saddle(x, y):
    return x ** 2 - y ** 2

def bowl(x, y):
    return x ** 2 + y ** 2

H_saddle = hessian_2d(saddle, 0.0, 0.0)
H_bowl = hessian_2d(bowl, 0.0, 0.0)
print(f"Saddle Hessian: {H_saddle}") # [[2, 0], [0, -2]] -- mixed signs
print(f"Bowl Hessian: {H_bowl}") # [[2, 0], [0, 2]] -- both positive

```

The Hessian of the saddle function has eigenvalues 2 and -2 (mixed signs, confirming a saddle point). The bowl has eigenvalues 2 and 2 (both positive, confirming a minimum).

Step 7: Taylor approximation in action

```
import math

def taylor_approx(f, f_prime, f_double_prime, x0, h, order=2):
    result = f(x0)
    if order >= 1:
        result += f_prime(x0) * h
    if order >= 2:
        result += 0.5 * f_double_prime(x0) * h ** 2
    return result

x0 = 0.0
for h in [0.1, 0.5, 1.0, 2.0]:
    true_val = math.sin(h)
    t1 = taylor_approx(math.sin, math.cos, lambda x: -math.sin(x), x0, h, order=1)
    t2 = taylor_approx(math.sin, math.cos, lambda x: -math.sin(x), x0, h, order=2)
    print(f"h={h:.1f} sin(h)={true_val:.4f} order1={t1:.4f} order2={t2:.4f}")
```

Near $x_0=0$, $\sin(x) \sim x$ (first-order Taylor). The approximation is excellent for small h but breaks down for large h . This is why gradient descent works best with small learning rates – each step assumes the linear approximation is accurate.

Step 8: Why this matters for a neural network

```
import random

random.seed(42)

w = random.gauss(0, 1)
b = random.gauss(0, 1)
lr = 0.01

xs = [1.0, 2.0, 3.0, 4.0, 5.0]
ys = [3.0, 5.0, 7.0, 9.0, 11.0]

for epoch in range(200):
    total_loss = 0
    dw = 0
    db = 0
    for x, y in zip(xs, ys):
        pred = w * x + b
        error = pred - y
        total_loss += error ** 2
        dw += 2 * error * x
        db += 2 * error
    dw /= len(xs)
    db /= len(xs)
    total_loss /= len(xs)
    w -= lr * dw
    b -= lr * db
    if epoch % 40 == 0 or epoch == 199:
        print(f"epoch {epoch:3d} w={w:.4f} b={b:.4f} loss={total_loss:.6f}")

print(f"\nLearned: y = {w:.2f}x + {b:.2f}")
```

```
print(f"Actual: y = 2x + 1")
```

Every gradient-based training loop follows this pattern: predict, compute loss, compute gradients, update weights.

Use It

With NumPy, the same operations are faster and more concise:

```
import numpy as np

x = np.array([1, 2, 3, 4, 5], dtype=float)
y = np.array([3, 5, 7, 9, 11], dtype=float)

w, b = np.random.randn(), np.random.randn()
lr = 0.01

for epoch in range(200):
    pred = w * x + b
    error = pred - y
    loss = np.mean(error ** 2)
    dw = np.mean(2 * error * x)
    db = np.mean(2 * error)
    w -= lr * dw
    b -= lr * db

print(f"Learned: y = {w:.2f}x + {b:.2f}")
```

You just built gradient descent from scratch. PyTorch automates the gradient computation, but the update loop is identical.

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/04-calculus-for-ml/code>

1. Implement `numerical_second_derivative(f, x)` using `numerical_derivative` called twice. Verify that the second derivative of x^3 at $x=2$ is 12.
2. Use gradient descent to find the minimum of $f(x, y) = (x - 3)^2 + (y + 1)^2$. Start from $(0, 0)$. The answer should converge to $(3, -1)$.
3. Add momentum to the gradient descent loop: maintain a velocity vector that accumulates past gradients. Compare convergence speed with and without momentum on $f(x) = x^4 - 3x^2$.

Key Terms

Term	What people say	What it actually means
Derivative	"The slope"	The rate of change of a function at a point. Tells you how much the output changes per unit change in input.

Term	What people say	What it actually means
Partial derivative	“Derivative of one variable”	The derivative with respect to one variable while all others are held constant.
Gradient	“Direction of steepest ascent”	A vector of all partial derivatives. Points in the direction that increases the function fastest.
Gradient descent	“Go downhill”	Subtract the gradient (times a learning rate) from the parameters to reduce the loss. The core of neural network training.
Learning rate	“Step size”	A scalar that controls how big each gradient descent step is. Too large: diverge. Too small: converge slowly.
Chain rule	“Multiply the derivatives”	The rule for differentiating composed functions: $df/dx = df/dg * dg/dx$. The mathematical basis of backpropagation.
Jacobian	“Matrix of derivatives”	When a function maps vectors to vectors, the Jacobian is the matrix of all partial derivatives of outputs with respect to inputs.
Numerical derivative	“Finite differences”	Approximating a derivative by evaluating the function at two nearby points and computing the slope between them.
Backpropagation	“Reverse-mode autodiff”	Computing gradients layer by layer from output to input using the chain rule. How neural networks learn.
Hessian	“Matrix of second derivatives”	The matrix of all second-order partial derivatives. Describes the curvature of a function. Positive definite Hessian at a critical point means local minimum.
Taylor series	“Polynomial approximation”	Approximating a function near a point using its derivatives: $f(x+h) \sim f(x) + f'(x)h + (1/2)f''(x)h^2 + \dots$. The basis for understanding why gradient descent and Newton’s method work.
Integral	“Area under the curve”	The accumulation of a quantity over a range. In ML, integrals define probabilities, expected values, and KL divergence.

Further Reading

- 3Blue1Brown: Essence of Calculus - visual intuition for derivatives, integrals, and the chain rule
- Stanford CS231n: Backpropagation - how gradients flow through neural network layers

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/04-calculus-for-ml>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/04-calculus-for-ml/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/04-calculus-for-ml>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Chain Rule & Automatic Differentiation

The chain rule is the engine behind every neural network that learns.

Type: Build **Language:** Python **Prerequisites:** Phase 1, Lesson 04 (Derivatives & Gradients)
Time: ~90 minutes

Learning Objectives

- Build a minimal autograd engine (Value class) that records operations and computes gradients via reverse-mode autodiff
- Implement forward and backward passes through a computation graph using topological sort
- Construct and train a multi-layer perceptron on XOR using only the from-scratch autograd engine
- Verify autodiff correctness using gradient checking against numerical finite differences

The Problem

You can compute derivatives of simple functions. But a neural network is not a simple function. It is hundreds of functions composed together: matrix multiply, add bias, apply activation, matrix multiply again, softmax, cross-entropy loss. The output is a function of a function of a function.

To train the network, you need the gradient of the loss with respect to every single weight. Doing this by hand is impossible for millions of parameters. Doing it numerically (finite differences) is too slow.

The chain rule gives you the math. Automatic differentiation gives you the algorithm. Together they let you compute exact gradients through arbitrary compositions of functions in time proportional to a single forward pass.

This is how PyTorch, TensorFlow, and JAX work. You will build a miniature version from scratch.

The Concept

The Chain Rule

If $y = f(g(x))$, the derivative of y with respect to x is:

$$dy/dx = dy/dg * dg/dx = f'(g(x)) * g'(x)$$

Multiply the derivatives along the chain. Each link contributes its local derivative.

Example: $y = \sin(x^2)$

$$\begin{aligned} g(x) &= x^2 & g'(x) &= 2x \\ f(g) &= \sin(g) & f'(g) &= \cos(g) \end{aligned}$$

$$dy/dx = \cos(x^2) * 2x$$

For deeper compositions, the chain extends:

$$y = f(g(h(x)))$$

$$dy/dx = f'(g(h(x))) * g'(h(x)) * h'(x)$$

Every layer in a neural network is one link in this chain.

Computational Graphs

A computational graph makes the chain rule visual. Every operation becomes a node. Data flows forward through the graph. Gradients flow backward.

Forward pass (compute values):

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/05-chain-rule-and-autodiff>

Backward pass (compute gradients):

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/05-chain-rule-and-autodiff>

The backward pass applies the chain rule at every node, propagating gradients from output to inputs.

Forward Mode vs Reverse Mode

There are two ways to apply the chain rule through a graph.

Forward mode starts at the inputs and pushes derivatives forward. It computes $dx/dx = 1$ and propagates through each operation. Good when you have few inputs and many outputs.

Forward mode: seed $dx/dx = 1$, propagate forward

$$\begin{aligned} x &= 2 & (dx/dx &= 1) \\ a &= x^2 & (da/dx &= 2x = 4) \\ y &= \sin(a) & (dy/dx &= \cos(a) * da/dx = \cos(4) * 4 = -2.615) \end{aligned}$$

Reverse mode starts at the output and pulls gradients backward. It computes $dy/dy = 1$ and propagates through each operation in reverse. Good when you have many inputs and few outputs.

Reverse mode: seed $dy/dy = 1$, propagate backward

$$\begin{aligned} y &= \sin(a) & (dy/dy &= 1) \\ a &= x^2 & (dy/da &= \cos(a) = \cos(4) = -0.654) \\ x &= 2 & (dy/dx &= dy/da * da/dx = -0.654 * 4 = -2.615) \end{aligned}$$

Neural networks have millions of inputs (weights) and one output (loss). Reverse mode computes all gradients in one backward pass. This is why backpropagation uses reverse mode.

Mode	Seed	Direction	Best when
Forward	$dx_i/dx_i = 1$	Input to output	Few inputs, many outputs

Mode	Seed	Direction	Best when
Reverse	$dy/dy = 1$	Output to input	Many inputs, few outputs (neural nets)

Dual Numbers for Forward Mode

Forward mode can be implemented elegantly with dual numbers. A dual number has the form $a + b \cdot \epsilon$ where $\epsilon^2 = 0$.

Dual number: (value, derivative)

(2, 1) means: value is 2, derivative w.r.t. x is 1

Arithmetic rules:

$$\begin{aligned}(a, a') + (b, b') &= (a+b, a'+b') \\ (a, a') * (b, b') &= (a*b, a'*b + a*b') \\ \sin(a, a') &= (\sin(a), \cos(a)*a')\end{aligned}$$

Seed the input variable with derivative 1. The derivative propagates automatically through every operation.

Building an Autograd Engine

An autograd engine needs three things:

1. **Value wrapping.** Wrap every number in an object that stores its value and gradient.
2. **Graph recording.** Every operation records its inputs and the local gradient function.
3. **Backward pass.** Topological sort the graph, then walk it in reverse, applying the chain rule at each node.

This is exactly what PyTorch's autograd does. The `torch.Tensor` class wraps values, records operations when `requires_grad=True`, and computes gradients when you call `.backward()`.

How PyTorch Autograd Works Under the Hood

When you write PyTorch code:

```
x = torch.tensor(2.0, requires_grad=True)
y = x ** 2 + 3 * x + 1
y.backward()
print(x.grad)  # 7.0 = 2*x + 3 = 2*2 + 3
```

PyTorch internally:

1. Creates a Tensor node for x with `requires_grad=True`
2. Every operation (`**`, `*`, `+`) creates a new node and records the backward function
3. `y.backward()` triggers reverse-mode autodiff through the recorded graph
4. Each node's `grad_fn` computes local gradients and passes them to parent nodes
5. Gradients accumulate in `.grad` attributes via addition (not replacement)

The graph is dynamic (define-by-run). A new graph is built on every forward pass. This is why PyTorch supports control flow (if/else, loops) inside models.

Interactive figure: chain-rule. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/05-chain-rule-and-autodiff>

Build It

Step 1: The Value class

```
class Value:
    def __init__(self, data, children=(), op=''):
        self.data = data
        self.grad = 0.0
        self._backward = lambda: None
        self._prev = set(children)
        self._op = op

    def __repr__(self):
        return f"Value(data={self.data:.4f}, grad={self.grad:.4f})"
```

Every Value stores its numeric data, its gradient (initially zero), a backward function, and pointers to child nodes that produced it.

Step 2: Arithmetic operations with gradient tracking

```
def __add__(self, other):
    other = other if isinstance(other, Value) else Value(other)
    out = Value(self.data + other.data, (self, other), '+')
    def _backward():
        self.grad += out.grad
        other.grad += out.grad
    out._backward = _backward
    return out

def __mul__(self, other):
    other = other if isinstance(other, Value) else Value(other)
    out = Value(self.data * other.data, (self, other), '*')
    def _backward():
        self.grad += other.data * out.grad
        other.grad += self.data * out.grad
    out._backward = _backward
    return out

def relu(self):
    out = Value(max(0, self.data), (self,), 'relu')
    def _backward():
        self.grad += (1.0 if out.data > 0 else 0.0) * out.grad
    out._backward = _backward
    return out
```

Each operation creates a closure that knows how to compute local gradients and multiply by the upstream gradient (`out.grad`). The `+=` handles the case where a value is used in multiple operations.

Step 3: The backward pass

```
def backward(self):
    topo = []
    visited = set()
```

```

def build_topo(v):
    if v not in visited:
        visited.add(v)
        for child in v._prev:
            build_topo(child)
        topo.append(v)
build_topo(self)

self.grad = 1.0
for v in reversed(topo):
    v._backward()

```

Topological sort ensures every node's gradient is fully computed before it propagates to its children. The seed gradient is 1.0 ($dy/dy = 1$).

Step 4: More operations for a complete engine

The basic Value class handles addition, multiplication, and relu. A real autograd engine needs more. Here are the operations you need to build neural networks:

```

def __neg__(self):
    return self * -1

def __sub__(self, other):
    return self + (-other)

def __radd__(self, other):
    return self + other

def __rmul__(self, other):
    return self * other

def __rsub__(self, other):
    return other + (-self)

def __pow__(self, n):
    out = Value(self.data ** n, (self,), f'**{n}')
    def _backward():
        self.grad += n * (self.data ** (n - 1)) * out.grad
    out._backward = _backward
    return out

def __truediv__(self, other):
    return self * (other ** -1) if isinstance(other, Value) else self * (Value(other) ** -1)

def exp(self):
    import math
    e = math.exp(self.data)
    out = Value(e, (self,), 'exp')
    def _backward():
        self.grad += e * out.grad
    out._backward = _backward
    return out

def log(self):

```

```

import math
out = Value(math.log(self.data), (self,), 'log')
def _backward():
    self.grad += (1.0 / self.data) * out.grad
out._backward = _backward
return out

def tanh(self):
    import math
    t = math.tanh(self.data)
    out = Value(t, (self,), 'tanh')
    def _backward():
        self.grad += (1 - t ** 2) * out.grad
    out._backward = _backward
    return out

```

Why each operation matters:

Operation	Backward rule	Used in
<code>__sub__</code>	Reuses add + neg	Loss computation (pred - target)
<code>__pow__</code>	$n * x^{(n-1)}$	Polynomial activations, MSE (error ²)
<code>__truediv__</code>	Reuses mul + pow(-1)	Normalization, learning rate scaling
exp	exp(x) * upstream	Softmax, log-likelihood
log	(1/x) * upstream	Cross-entropy loss, log probabilities
tanh	(1 - tanh ²) * upstream	Classic activation function

The clever part: `__sub__` and `__truediv__` are defined in terms of existing operations. They get correct gradients for free because the chain rule composes through the underlying add/mul/pow operations.

Step 5: Mini MLP from scratch

With a complete Value class, you can build a neural network. No PyTorch. No NumPy. Just Values and the chain rule.

```

import random

class Neuron:
    def __init__(self, n_inputs):
        self.w = [Value(random.uniform(-1, 1)) for _ in range(n_inputs)]
        self.b = Value(0.0)

    def __call__(self, x):
        act = sum((wi * xi for wi, xi in zip(self.w, x)), self.b)
        return act.tanh()

    def parameters(self):
        return self.w + [self.b]

```

```

class Layer:
    def __init__(self, n_inputs, n_outputs):
        self.neurons = [Neuron(n_inputs) for _ in range(n_outputs)]

    def __call__(self, x):
        return [n(x) for n in self.neurons]

    def parameters(self):
        return [p for n in self.neurons for p in n.parameters()]

class MLP:
    def __init__(self, sizes):
        self.layers = [Layer(sizes[i], sizes[i+1]) for i in range(len(sizes)-1)]

    def __call__(self, x):
        for layer in self.layers:
            x = layer(x)
        return x[0] if len(x) == 1 else x

    def parameters(self):
        return [p for layer in self.layers for p in layer.parameters()]

```

A Neuron computes $\tanh(w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + b)$. A Layer is a list of neurons. An MLP stacks layers. Every weight is a Value, so calling `loss.backward()` propagates gradients to every parameter.

Training on XOR:

```

random.seed(42)
model = MLP([2, 4, 1]) # 2 inputs, 4 hidden neurons, 1 output

xs = [[0, 0], [0, 1], [1, 0], [1, 1]]
ys = [-1, 1, 1, -1] # XOR pattern (using -1/1 for tanh)

for step in range(100):
    preds = [model(x) for x in xs]
    loss = sum((p - y) ** 2 for p, y in zip(preds, ys))

    for p in model.parameters():
        p.grad = 0.0
    loss.backward()

    lr = 0.05
    for p in model.parameters():
        p.data -= lr * p.grad

    if step % 20 == 0:
        print(f"step {step:3d}  loss = {loss.data:.4f}")

print("\nPredictions after training:")
for x, y in zip(xs, ys):
    print(f"  input={x}  target={y:2d}  pred={model(x).data:6.3f}")

```

This is micrograd. A complete neural network training loop in pure Python with automatic differentiation. Every commercial deep learning framework does the same thing at massive

scale.

Step 6: Gradient checking

How do you know your autograd is correct? Compare it against numerical derivatives. This is gradient checking.

```
def gradient_check(build_expr, x_val, h=1e-7):
    x = Value(x_val)
    y = build_expr(x)
    y.backward()
    autodiff_grad = x.grad

    y_plus = build_expr(Value(x_val + h)).data
    y_minus = build_expr(Value(x_val - h)).data
    numerical_grad = (y_plus - y_minus) / (2 * h)

    diff = abs(autodiff_grad - numerical_grad)
    return autodiff_grad, numerical_grad, diff
```

Test it on a complex expression:

```
def expr(x):
    return (x ** 3 + x * 2 + 1).tanh()

ad, num, diff = gradient_check(expr, 0.5)
print(f"Autodiff: {ad:.8f}")
print(f"Numerical: {num:.8f}")
print(f"Difference: {diff:.2e}")
# Difference should be < 1e-5
```

Gradient checking is essential when implementing new operations. If your backward pass has a bug, the numerical check catches it. Every serious deep learning implementation runs gradient checks during development.

When to use gradient checking:

Situation	Do gradient check?
Adding a new operation to your autograd	Yes, always
Debugging a training loop that won't converge	Yes, check gradients first
Production training	No, too slow (2x forward passes per parameter)
Unit tests for autograd code	Yes, automate it

Step 7: Verify against manual calculation

```
x1 = Value(2.0)
x2 = Value(3.0)
a = x1 * x2          # a = 6.0
b = a + Value(1.0)    # b = 7.0
y = b.relu()          # y = 7.0

y.backward()
```

```
print(f"y = {y.data}")           # 7.0
print(f"dy/dx1 = {x1.grad}")     # 3.0 (= x2)
print(f"dy/dx2 = {x2.grad}")     # 2.0 (= x1)
```

Manual check: $y = \text{relu}(x_1 \cdot x_2 + 1)$. Since $x_1 \cdot x_2 + 1 = 7 > 0$, relu is identity. $dy/dx_1 = x_2 = 3$. $dy/dx_2 = x_1 = 2$. The engine matches.

Use It

Verify against PyTorch

```
import torch

x1 = torch.tensor(2.0, requires_grad=True)
x2 = torch.tensor(3.0, requires_grad=True)
a = x1 * x2
b = a + 1.0
y = torch.relu(b)
y.backward()

print(f"PyTorch dy/dx1 = {x1.grad.item()}") # 3.0
print(f"PyTorch dy/dx2 = {x2.grad.item()}") # 2.0
```

Same gradients. Your engine computes the same result as PyTorch because the math is the same: reverse-mode autodiff via the chain rule.

A more complex expression

```
a = Value(2.0)
b = Value(-3.0)
c = Value(10.0)
f = (a * b + c).relu() # relu(2*(-3) + 10) = relu(4) = 4

f.backward()
print(f"df/da = {a.grad}") # -3.0 (= b)
print(f"df/db = {b.grad}") # 2.0 (= a)
print(f"df/dc = {c.grad}") # 1.0
```

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/05-chain-rule-and-autodiff>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/05-chain-rule-and-autodiff/code>

1. Add `__pow__` to the `Value` class so you can compute $x ** n$. Verify that $d/dx(x^3)$ at $x=2$ equals 12.0.
2. Add $\tanh$ as an activation function. Verify that $\tanh'(0) = 1$ and $\tanh'(2) = 0.0707$ (approx).
3. Build a computation graph for a single neuron: $y = \text{relu}(w_1 \cdot x_1 + w_2 \cdot x_2 + b)$. Compute all five gradients and verify against PyTorch.

4. Implement forward-mode autodiff using dual numbers. Create a Dual class and verify it gives the same derivatives as your reverse-mode engine.

Key Terms

Term	What people say	What it actually means
Chain rule	“Multiply the derivatives”	The derivative of composed functions equals the product of each function’s local derivative, evaluated at the right point
Computational graph	“The network diagram”	A directed acyclic graph where nodes are operations and edges carry values (forward) or gradients (backward)
Forward mode	“Push derivatives forward”	Autodiff that propagates derivatives from inputs to outputs. One pass per input variable.
Reverse mode	“Backpropagation”	Autodiff that propagates gradients from outputs to inputs. One pass per output variable.
Autograd	“Automatic gradients”	A system that records operations on values, builds a graph, and computes exact gradients via the chain rule
Dual numbers	“Value plus derivative”	Numbers of the form $a + b \cdot \epsilon$ ($\epsilon^2 = 0$) that carry derivative information through arithmetic
Topological sort	“Dependency order”	Ordering graph nodes so every node comes after all its dependencies. Required for correct gradient propagation.
Gradient accumulation	“Add, don’t replace”	When a value feeds into multiple operations, its gradient is the sum of all incoming gradient contributions
Dynamic graph	“Define by run”	A computation graph rebuilt on every forward pass, allowing Python control flow inside models (PyTorch style)
Gradient checking	“Numerical verification”	Comparing autodiff gradients against numerical finite-difference gradients to verify correctness. Essential for debugging.
MLP	“Multi-layer perceptron”	A neural network with one or more hidden layers of neurons. Each neuron computes a weighted sum plus bias, then applies an activation function.
Neuron	“Weighted sum + activation”	The basic unit: $\text{output} = \text{activation}(w_1x_1 + w_2x_2 + \dots + b)$. The weights and bias are learnable parameters.

Further Reading

- 3Blue1Brown: Backpropagation calculus – visual explanation of the chain rule in neural networks
- PyTorch Autograd mechanics – how the real system works

- Baydin et al., Automatic Differentiation in Machine Learning: a Survey - comprehensive reference

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/05-chain-rule-and-autodiff>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/05-chain-rule-and-autodiff/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/05-chain-rule-and-autodiff>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Probability and Distributions

Probability is the language AI uses to express uncertainty.

Type: Learn **Language:** Python **Prerequisites:** Phase 1, Lessons 01-04 **Time:** ~75 minutes

Learning Objectives

- Implement PMFs and PDFs from scratch for Bernoulli, categorical, Poisson, uniform, and normal distributions
- Compute expected value, variance, and use the Central Limit Theorem to explain why Gaussians dominate
- Build softmax and log-softmax functions with the numerical stability trick (subtract max logit)
- Calculate cross-entropy loss from logits and connect it to negative log-likelihood

The Problem

A classifier outputs $[0.03, 0.91, 0.06]$. A language model picks the next word from 50,000 candidates. A diffusion model generates images by sampling from learned distributions. All of these are probability in action.

Every prediction a model makes is a probability distribution. Every loss function measures how far the predicted distribution is from the true one. Every training step adjusts parameters to make one distribution look more like another. Without probability, you cannot read a single ML paper, debug a single model, or understand why your training loss is NaN.

The Concept

Events, Sample Spaces, and Probability

The sample space S is the set of all possible outcomes. An event is a subset of the sample space. Probability maps events to numbers between 0 and 1.

Coin flip:

$$S = \{H, T\}$$

$$P(H) = 0.5, \quad P(T) = 0.5$$

Single die roll:

$$S = \{1, 2, 3, 4, 5, 6\}$$

$$P(\text{even}) = P(\{2, 4, 6\}) = 3/6 = 0.5$$

Three axioms define all of probability: 1. $P(A) \geq 0$ for any event A 2. $P(S) = 1$ (something always happens) 3. $P(A \text{ or } B) = P(A) + P(B)$ when A and B cannot both occur

Everything else (Bayes' theorem, expectations, distributions) follows from these three rules.

Conditional Probability and Independence

$P(A|B)$ is the probability of A given that B happened.

$$P(A|B) = P(A \text{ and } B) / P(B)$$

Example: deck of cards

$$\begin{aligned} P(\text{King} \mid \text{Face card}) &= P(\text{King and Face card}) / P(\text{Face card}) \\ &= (4/52) / (12/52) \\ &= 4/12 = 1/3 \end{aligned}$$

Two events are independent when knowing one tells you nothing about the other:

Independent: $P(A|B) = P(A)$

Equivalent to: $P(A \text{ and } B) = P(A) * P(B)$

Coin flips are independent. Drawing cards without replacement is not.

Probability Mass Functions vs Probability Density Functions

Discrete random variables have a probability mass function (PMF). Each outcome has a specific probability that you can read off directly.

PMF: $P(X = k)$

Fair die:

$$P(X = 1) = 1/6$$

$$P(X = 2) = 1/6$$

...

$$P(X = 6) = 1/6$$

Sum of all probabilities = 1

Continuous random variables have a probability density function (PDF). The density at a single point is not a probability. Probability comes from integrating the density over an interval.

PDF: $f(x)$

$P(a \leq X \leq b) = \text{integral of } f(x) \text{ from } a \text{ to } b$

$f(x)$ can be greater than 1 (density, not probability)

integral from $-\infty$ to $+\infty$ of $f(x) dx = 1$

This distinction matters in ML. Classification outputs are PMFs (discrete choices). VAE latent spaces use PDFs (continuous).

Common Distributions

Bernoulli: one trial, two outcomes. Models binary classification.

$$P(X = 1) = p$$

$$P(X = 0) = 1 - p$$

$$\text{Mean} = p, \text{ Variance} = p(1-p)$$

Categorical: one trial, k outcomes. Models multi-class classification (softmax output).

$$P(X = i) = p_i, \text{ where } \sum p_i = 1$$

$$\text{Example: } P(\text{cat}) = 0.7, P(\text{dog}) = 0.2, P(\text{bird}) = 0.1$$

Uniform: all outcomes equally likely. Used for random initialization.

Discrete: $P(X = k) = 1/n$ for k in $\{1, \dots, n\}$

Continuous: $f(x) = 1/(b-a)$ for x in $[a, b]$

Normal (Gaussian): the bell curve. Parameterized by mean (μ) and variance (σ^2).

$$f(x) = (1 / \sqrt{2\pi\sigma^2}) * \exp(-(x - \mu)^2 / (2\sigma^2))$$

Standard normal: $\mu = 0$, $\sigma = 1$

68% of data within 1 sigma

95% within 2 sigma

99.7% within 3 sigma

Poisson: counts of rare events in a fixed interval. Models event rates.

$$P(X = k) = (\lambda^k * e^{-\lambda}) / k!$$

Mean = λ , Variance = λ

Expected Value and Variance

Expected value is the weighted average outcome.

Discrete: $E[X] = \sum x_i * P(X = x_i)$

Continuous: $E[X] = \int x * f(x) dx$

Variance measures spread around the mean.

$$\text{Var}(X) = E[(X - E[X])^2] = E[X^2] - (E[X])^2$$

$$\text{Standard deviation} = \sqrt{\text{Var}(X)}$$

In ML, expected value appears as the loss function (average loss over the data distribution).

Variance tells you about model stability. High variance in gradients means noisy training.

Joint and Marginal Distributions

A joint distribution $P(X, Y)$ describes two random variables together.

Joint PMF example (X = weather, Y = umbrella):

	Y=0 (no umbrella)	Y=1 (umbrella)	Marginal P(X)
X=0 (sun)	0.40	0.10	$P(X=0) = 0.50$
X=1 (rain)	0.05	0.45	$P(X=1) = 0.50$
Marginal P(Y)	$P(Y=0) = 0.45$	$P(Y=1) = 0.55$	1.00

The marginal distribution sums out the other variable:

$$P(X = x) = \sum \text{over all } y \text{ of } P(X = x, Y = y)$$

The row and column totals in the table above are the marginals.

Why the Normal Distribution Shows Up Everywhere

The Central Limit Theorem: the sum (or average) of many independent random variables converges to a normal distribution, regardless of the original distribution.

Roll 1 die: uniform distribution (flat)

Average of 2 dice: triangular (peaked)

Average of 30 dice: nearly perfect bell curve

This works for ANY starting distribution.

This is why: - Measurement errors are approximately normal (many small independent sources) - Weight initializations in neural networks use normal distributions - Gradient noise in SGD is approximately normal (sum of many sample gradients) - The normal distribution is the maximum entropy distribution for a given mean and variance

Log Probabilities

Raw probabilities cause numerical problems. Multiplying many small probabilities together quickly underflows to zero.

```
P(sentence) = P(word1) * P(word2) * ... * P(word_n)
              = 0.01 * 0.003 * 0.02 * ...
              -> 0.0 (underflow after ~30 terms)
```

Log probabilities fix this. Multiplications become additions.

```
log P(sentence) = log P(word1) + log P(word2) + ... + log P(word_n)
                 = -4.6 + -5.8 + -3.9 + ...
                 -> finite number (no underflow)
```

Rules: - $\log(a * b) = \log(a) + \log(b)$ - log probabilities are always ≤ 0 (since $0 < P \leq 1$) - More negative = less likely - Cross-entropy loss is the negative log probability of the correct class

Softmax as a Probability Distribution

Neural networks output raw scores (logits). Softmax converts them into a valid probability distribution.

```
softmax(z_i) = exp(z_i) / sum(exp(z_j) for all j)
```

Properties:

- All outputs are in (0, 1)
- All outputs sum to 1
- Preserves relative ordering of inputs
- exp() amplifies differences between logits

The softmax trick: subtract the max logit before exponentiating to prevent overflow.

```
z = [100, 101, 102]
exp(102) = overflow
```

```
z_shifted = z - max(z) = [-2, -1, 0]
exp(0) = 1 (safe)
```

Same result, no overflow.

Log-softmax combines softmax and log for numerical stability. PyTorch uses this internally for cross-entropy loss.

Sampling

Sampling means drawing random values from a distribution. In ML: - Dropout randomly samples which neurons to zero out - Data augmentation samples random transformations - Language models sample the next token from the predicted distribution - Diffusion models sample noise and progressively denoise

Sampling from arbitrary distributions requires techniques like inverse transform sampling, rejection sampling, or the reparameterization trick (used in VAEs).

Interactive figure: gaussian-pdf. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/06-probability-and-distributions>

Build It

Step 1: Probability basics

```
import math
import random

def factorial(n):
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result

def combinations(n, k):
    return factorial(n) // (factorial(k) * factorial(n - k))

def conditional_probability(p_a_and_b, p_b):
    return p_a_and_b / p_b

p_king_given_face = conditional_probability(4/52, 12/52)
print(f"P(King | Face card) = {p_king_given_face:.4f}")
```

Step 2: PMF and PDF from scratch

```
def bernoulli_pmf(k, p):
    return p if k == 1 else (1 - p)

def categorical_pmf(k, probs):
    return probs[k]

def poisson_pmf(k, lam):
    return (lam ** k) * math.exp(-lam) / factorial(k)

def uniform_pdf(x, a, b):
    if a <= x <= b:
        return 1.0 / (b - a)
    return 0.0

def normal_pdf(x, mu, sigma):
    coeff = 1.0 / (sigma * math.sqrt(2 * math.pi))
    exponent = -0.5 * ((x - mu) / sigma) ** 2
    return coeff * math.exp(exponent)
```

Step 3: Expected value and variance

```
def expected_value(values, probabilities):
    return sum(v * p for v, p in zip(values, probabilities))

def variance(values, probabilities):
    mu = expected_value(values, probabilities)
    return sum(p * (v - mu) ** 2 for v, p in zip(values, probabilities))

die_values = [1, 2, 3, 4, 5, 6]
die_probs = [1/6] * 6
mu = expected_value(die_values, die_probs)
var = variance(die_values, die_probs)
print(f"Die: E[X] = {mu:.4f}, Var(X) = {var:.4f}, SD = {var**0.5:.4f}")
```

Step 4: Sampling from distributions

```
def sample_bernoulli(p, n=1):
    return [1 if random.random() < p else 0 for _ in range(n)]

def sample_categorical(probs, n=1):
    cumulative = []
    total = 0
    for p in probs:
        total += p
        cumulative.append(total)
    samples = []
    for _ in range(n):
        r = random.random()
        for i, c in enumerate(cumulative):
            if r <= c:
                samples.append(i)
                break
    return samples

def sample_normal_box_muller(mu, sigma, n=1):
    samples = []
    for _ in range(n):
        u1 = random.random()
        u2 = random.random()
        z = math.sqrt(-2 * math.log(u1)) * math.cos(2 * math.pi * u2)
        samples.append(mu + sigma * z)
    return samples
```

Step 5: Softmax and log probabilities

```
def softmax(logits):
    max_logit = max(logits)
    shifted = [z - max_logit for z in logits]
    exps = [math.exp(z) for z in shifted]
    total = sum(exps)
    return [e / total for e in exps]

def log_softmax(logits):
```

```

max_logit = max(logits)
shifted = [z - max_logit for z in logits]
log_sum_exp = max_logit + math.log(sum(math.exp(z) for z in shifted))
return [z - log_sum_exp for z in logits]

def cross_entropy_loss(logits, target_index):
    log_probs = log_softmax(logits)
    return -log_probs[target_index]

```

Step 6: Central Limit Theorem demonstration

```

def demonstrate_clt(dist_fn, n_samples, n_averages):
    averages = []
    for _ in range(n_averages):
        samples = [dist_fn() for _ in range(n_samples)]
        averages.append(sum(samples) / len(samples))
    return averages

```

Step 7: Visualization

```

import matplotlib.pyplot as plt

xs = [mu + sigma * (i - 500) / 100 for i in range(1001)]
ys = [normal_pdf(x, mu, sigma) for x, mu, sigma in ...]
plt.plot(xs, ys)

```

Full implementations with all visualizations are in `code/probability.py`.

Use It

With NumPy and SciPy, everything above is one-liners:

```

import numpy as np
from scipy import stats

normal = stats.norm(loc=0, scale=1)
samples = normal.rvs(size=10000)
print(f"Mean: {np.mean(samples):.4f}, Std: {np.std(samples):.4f}")
print(f"P(X < 1.96) = {normal.cdf(1.96):.4f}")

logits = np.array([2.0, 1.0, 0.1])
from scipy.special import softmax, log_softmax
probs = softmax(logits)
log_probs = log_softmax(logits)
print(f"Softmax: {probs}")
print(f"Log-softmax: {log_probs}")

```

You built these from scratch. Now you know what the library calls are doing.

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/06-probability-and->

distributions/code

1. Implement inverse transform sampling for the exponential distribution. Verify by sampling 10,000 values and comparing the histogram to the true PDF.
2. Build a joint distribution table for two loaded dice. Compute the marginal distributions and check whether the dice are independent.
3. Compute the cross-entropy loss for a 5-class classifier that outputs logits $[2.0, 0.5, -1.0, 3.0, 0.1]$ when the correct class is index 3. Then verify your answer with PyTorch's `nn.CrossEntropyLoss`.
4. Write a function that takes a list of log probabilities and returns the most likely sequence, the total log probability, and the equivalent raw probability. Test it with a sentence of 50 words where each word has probability 0.01.

Key Terms

Term	What people say	What it actually means
Sample space	"All the possibilities"	The set S of every possible outcome of an experiment
PMF	"The probability function"	A function that gives the exact probability of each discrete outcome, summing to 1
PDF	"The probability curve"	A density function for continuous variables. Integrate it over an interval to get probability
Conditional probability	"Probability given something"	$P(A B) = P(A \text{ and } B) / P(B)$. The foundation of Bayesian thinking and Bayes' theorem
Independence	"They don't affect each other"	$P(A \text{ and } B) = P(A) * P(B)$. Knowing one event tells you nothing about the other
Expected value	"The average"	The probability-weighted sum of all outcomes. The loss function is an expected value
Variance	"How spread out"	The expected squared deviation from the mean. High variance = noisy, unstable estimates
Normal distribution	"The bell curve"	$f(x) = (1/\sqrt{2\pi\sigma^2}) * \exp(-(x-\mu)^2/(2*\sigma^2))$. Appears everywhere due to the CLT
Central Limit Theorem	"Averages become normal"	The mean of many independent samples converges to a normal distribution regardless of the source
Joint distribution	"Two variables together"	$P(X, Y)$ describes the probability of every combination of X and Y outcomes
Marginal distribution	"Sum out the other variable"	$P(X) = \sum_y P(X, Y)$. Recovers one variable's distribution from the joint
Log probability	"Log of the probability"	$\log P(x)$. Turns products into sums, preventing numerical underflow in long sequences
Softmax	"Turn scores into probabilities"	$\text{softmax}(z_i) = \exp(z_i) / \sum(\exp(z_j))$. Maps real-valued logits to a valid probability distribution

Term	What people say	What it actually means
Cross-entropy	"The loss function"	$-\sum(p_{\text{true}} * \log(p_{\text{predicted}}))$. Measures how different two distributions are. Lower is better
Logits	"Raw model outputs"	Unnormalized scores before softmax. Named after the logistic function
Sampling	"Drawing random values"	Generating values according to a probability distribution. How models generate output

Further Reading

- 3Blue1Brown: But what is the Central Limit Theorem? - visual proof of why averages become normal
- Stanford CS229 Probability Review - concise reference covering everything here and more
- The Log-Sum-Exp Trick - why numerical stability matters and how to achieve it

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/06-probability-and-distributions>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/06-probability-and-distributions/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/06-probability-and-distributions>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Bayes' Theorem

Probability is about what you expect. Bayes' theorem is about what you learn.

Type: Build **Language:** Python **Prerequisites:** Phase 1, Lesson 06 (Probability Fundamentals) **Time:** ~75 minutes

Learning Objectives

- Apply Bayes' theorem to compute posterior probabilities from priors, likelihoods, and evidence
- Build a Naive Bayes text classifier from scratch with Laplace smoothing and log-space computation
- Compare MLE and MAP estimation and explain how MAP corresponds to L2 regularization
- Implement sequential Bayesian updating using Beta-Binomial conjugate priors for A/B testing

The Problem

A medical test is 99% accurate. You test positive. What are the chances you actually have the disease?

Most people say 99%. The real answer depends on how rare the disease is. If 1 in 10,000 people have it, a positive result only gives you about a 1% chance of being sick. The other 99% of positive results are false alarms from healthy people.

This is not a trick question. It is Bayes' theorem. Every spam filter, every medical diagnostic, every machine learning model that quantifies uncertainty uses this exact reasoning. You start with a belief. You see evidence. You update.

If you build ML systems without understanding this, you will misinterpret model outputs, set bad thresholds, and ship overconfident predictions.

The Concept

From joint probability to Bayes

You already know from Lesson 06 that conditional probability is:

$$P(A|B) = P(A \text{ and } B) / P(B)$$

And symmetrically:

$$P(B|A) = P(A \text{ and } B) / P(A)$$

Both expressions share the same numerator: $P(A \text{ and } B)$. Set them equal and rearrange:

$$P(A \text{ and } B) = P(A|B) * P(B) = P(B|A) * P(A)$$

Therefore:

$$P(A|B) = P(B|A) * P(A) / P(B)$$

That is Bayes' theorem. Four quantities, one equation.

The four parts

Part	Name	What it means
$P(A B)$	Posterior	Your updated belief about A after seeing evidence B
$P(B A)$	Likelihood	How probable the evidence B is if A is true
$P(A)$	Prior	Your belief about A before seeing any evidence
$P(B)$	Evidence	Total probability of seeing B under all possibilities

The evidence term $P(B)$ acts as a normalizer. You can expand it using the law of total probability:

$$P(B) = P(B|A) * P(A) + P(B|\text{not } A) * P(\text{not } A)$$

Medical test example

A disease affects 1 in 10,000 people. The test is 99% accurate (catches 99% of sick people, gives false positives 1% of the time).

$$\begin{aligned} P(\text{sick}) &= 0.0001 && \text{(prior: disease is rare)} \\ P(\text{positive}|\text{sick}) &= 0.99 && \text{(likelihood: test catches it)} \\ P(\text{positive}|\text{healthy}) &= 0.01 && \text{(false positive rate)} \end{aligned}$$

$$\begin{aligned} P(\text{positive}) &= P(\text{positive}|\text{sick}) * P(\text{sick}) + P(\text{positive}|\text{healthy}) * P(\text{healthy}) \\ &= 0.99 * 0.0001 + 0.01 * 0.9999 \\ &= 0.000099 + 0.009999 \\ &= 0.010098 \end{aligned}$$

$$\begin{aligned} P(\text{sick}|\text{positive}) &= P(\text{positive}|\text{sick}) * P(\text{sick}) / P(\text{positive}) \\ &= 0.99 * 0.0001 / 0.010098 \\ &= 0.0098 \\ &= 0.98\% \end{aligned}$$

Less than 1%. The prior dominates. When a condition is rare, even accurate tests produce mostly false positives. This is why doctors order confirmation tests.

Spam filter example

You receive an email containing the word "lottery". Is it spam?

$$\begin{aligned} P(\text{spam}) &= 0.3 && \text{(30\% of email is spam)} \\ P(\text{"lottery"}|\text{spam}) &= 0.05 && \text{(5\% of spam emails contain "lottery")} \\ P(\text{"lottery"}|\text{not spam}) &= 0.001 && \text{(0.1\% of legitimate emails contain "lottery")} \end{aligned}$$

$$P(\text{"lottery"}) = 0.05 * 0.3 + 0.001 * 0.7$$

$$\begin{aligned}
 &= 0.015 + 0.0007 \\
 &= 0.0157
 \end{aligned}$$

$$\begin{aligned}
 P(\text{spam} | \text{"lottery"}) &= 0.05 * 0.3 / 0.0157 \\
 &= 0.955 \\
 &= 95.5\%
 \end{aligned}$$

One word shifts the probability from 30% to 95.5%. A real spam filter applies Bayes across hundreds of words simultaneously.

Naive Bayes: independence assumption

Naive Bayes extends this to multiple features by assuming all features are conditionally independent given the class:

$$\begin{aligned}
 P(\text{class} | \text{feature}_1, \text{feature}_2, \dots, \text{feature}_n) \\
 &= P(\text{class}) * P(\text{feature}_1 | \text{class}) * P(\text{feature}_2 | \text{class}) * \dots * P(\text{feature}_n | \text{class}) \\
 &\quad / P(\text{feature}_1, \text{feature}_2, \dots, \text{feature}_n)
 \end{aligned}$$

The “naive” part is the independence assumption. In text, word occurrences are not independent (“New” and “York” are correlated). But the assumption works surprisingly well in practice because the classifier only needs to rank classes, not produce calibrated probabilities.

Since the denominator is the same for all classes, you can skip it and just compare numerators:

$$\text{score}(\text{class}) = P(\text{class}) * \text{product of } P(\text{feature}_i | \text{class})$$

Pick the class with the highest score.

Maximum likelihood estimation (MLE)

How do you get $P(\text{feature} | \text{class})$ from training data? Count.

$$P(\text{"free"} | \text{spam}) = (\text{number of spam emails containing "free"}) / (\text{total spam emails})$$

This is MLE: choose the parameter values that make the observed data most likely. You are maximizing the likelihood function, which for discrete counts reduces to relative frequency.

Problem: if a word never appears in spam during training, MLE gives it probability zero. One unseen word kills the entire product. Fix this with Laplace smoothing:

$$P(\text{word} | \text{class}) = (\text{count}(\text{word}, \text{class}) + 1) / (\text{total_words_in_class} + \text{vocabulary_size})$$

Adding 1 to every count ensures no probability is ever zero.

Maximum a posteriori (MAP)

MLE asks: what parameters maximize $P(\text{data} | \text{parameters})$?

MAP asks: what parameters maximize $P(\text{parameters} | \text{data})$?

By Bayes' theorem:

$$P(\text{parameters} | \text{data}) \text{ proportional to } P(\text{data} | \text{parameters}) * P(\text{parameters})$$

MAP adds a prior over the parameters themselves. If you believe parameters should be small, you encode that as a prior that penalizes large values. This is identical to L2 regularization in ML. The “ridge” penalty in ridge regression is literally a Gaussian prior on the weights.

Estimation	Optimizes	ML equivalent
MLE	$P(\text{data} \text{params})$	Unregularized training
MAP	$P(\text{data} \text{params}) * P(\text{params})$	L2 / L1 regularization

Bayesian vs frequentist: the practical difference

Frequentists treat parameters as fixed unknowns. They ask: "If I repeated this experiment many times, what would happen?"

Bayesians treat parameters as distributions. They ask: "Given what I have observed, what do I believe about the parameters?"

For building ML systems, the practical difference:

Aspect	Frequentist	Bayesian
Output	Point estimate	Distribution over values
Uncertainty	Confidence intervals (about procedure)	Credible intervals (about parameter)
Small data	Can overfit	Prior acts as regularization
Computation	Usually faster	Often requires sampling (MCMC)

Most production ML is frequentist (SGD, point estimates). Bayesian methods shine when you need calibrated uncertainty (medical decisions, safety-critical systems) or when data is scarce (few-shot learning, cold start).

Why Bayesian thinking matters for ML

The connection is deeper than analogy:

Priors are regularization. A Gaussian prior on weights is L2 regularization. A Laplace prior is L1. Every time you add a regularization term, you are making a Bayesian statement about what parameter values you expect.

Posteriors are uncertainty. A single predicted probability tells you nothing about how confident the model is in that estimate. Bayesian methods give you a distribution: "I think $P(\text{spam})$ is between 0.8 and 0.95."

Bayes updates are online learning. Today's posterior becomes tomorrow's prior. When your model sees new data, it updates its beliefs incrementally instead of retraining from scratch.

Model comparison is Bayesian. Bayesian information criterion (BIC), marginal likelihood, and Bayes factors all use Bayesian reasoning to choose between models without overfitting.

Interactive figure: bayes-update. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/07-bayes-theorem>

Build It

Step 1: Bayes theorem function

```
def bayes(prior, likelihood, false_positive_rate):
    evidence = likelihood * prior + false_positive_rate * (1 - prior)
    posterior = likelihood * prior / evidence
    return posterior

result = bayes(prior=0.0001, likelihood=0.99, false_positive_rate=0.01)
print(f"P(sick|positive) = {result:.4f}")
```

Step 2: Naive Bayes classifier

```
import math
from collections import defaultdict

class NaiveBayes:
    def __init__(self, smoothing=1.0):
        self.smoothing = smoothing
        self.class_counts = defaultdict(int)
        self.word_counts = defaultdict(lambda: defaultdict(int))
        self.class_word_totals = defaultdict(int)
        self.vocab = set()

    def train(self, documents, labels):
        for doc, label in zip(documents, labels):
            self.class_counts[label] += 1
            words = doc.lower().split()
            for word in words:
                self.word_counts[label][word] += 1
                self.class_word_totals[label] += 1
                self.vocab.add(word)

    def predict(self, document):
        words = document.lower().split()
        total_docs = sum(self.class_counts.values())
        vocab_size = len(self.vocab)
        best_class = None
        best_score = float("-inf")
        for cls in self.class_counts:
            score = math.log(self.class_counts[cls] / total_docs)
            for word in words:
                count = self.word_counts[cls].get(word, 0)
                total = self.class_word_totals[cls]
                score += math.log((count + self.smoothing) / (total + self.smoothing * vocab_size))
            if score > best_score:
                best_score = score
                best_class = cls
        return best_class
```

Log probabilities prevent underflow. Multiplying many small probabilities produces numbers too tiny for floating point. Summing log-probabilities is numerically stable and mathematically equivalent.

Step 3: Train on spam data

```

train_docs = [
    "win free money now",
    "free lottery ticket winner",
    "claim your prize today free",
    "urgent offer free cash",
    "congratulations you won free",
    "meeting tomorrow at noon",
    "project update attached",
    "can we schedule a call",
    "quarterly report review",
    "lunch on thursday sounds good",
    "team standup notes attached",
    "please review the pull request",
]

train_labels = [
    "spam", "spam", "spam", "spam", "spam",
    "ham", "ham", "ham", "ham", "ham", "ham", "ham",
]

classifier = NaiveBayes()
classifier.train(train_docs, train_labels)

test_messages = [
    "free money waiting for you",
    "meeting rescheduled to friday",
    "you won a free prize",
    "please review the attached report",
]

for msg in test_messages:
    print(f"    '{msg}' -> {classifier.predict(msg)}")

```

Step 4: Inspect the learned probabilities

```

def show_top_words(classifier, cls, n=5):
    vocab_size = len(classifier.vocab)
    total = classifier.class_word_totals[cls]
    probs = {}
    for word in classifier.vocab:
        count = classifier.word_counts[cls].get(word, 0)
        probs[word] = (count + classifier.smoothing) / (total + classifier.smoothing * vocab_size)
    sorted_words = sorted(probs.items(), key=lambda x: x[1], reverse=True)
    for word, prob in sorted_words[:n]:
        print(f"    {word}: {prob:.4f}")

print("\nTop spam words:")
show_top_words(classifier, "spam")
print("\nTop ham words:")
show_top_words(classifier, "ham")

```


Use It

Scikit-learn ships production-ready naive Bayes implementations:

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import classification_report
```

```
vectorizer = CountVectorizer()
X_train = vectorizer.fit_transform(train_docs)
clf = MultinomialNB()
clf.fit(X_train, train_labels)
```

```
X_test = vectorizer.transform(test_messages)
predictions = clf.predict(X_test)
for msg, pred in zip(test_messages, predictions):
    print(f" '{msg}' -> {pred}")
```

Same algorithm. CountVectorizer handles tokenization and vocabulary building. MultinomialNB handles smoothing and log-probabilities internally. Your from-scratch version does the same thing in 40 lines.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/07-bayes-theorem>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/07-bayes-theorem/code>

1. **Multiple tests.** A patient tests positive twice on independent tests (both 99% accurate, disease prevalence 1 in 10,000). What is $P(\text{sick})$ after both tests? Use the posterior from the first test as the prior for the second.
2. **Smoothing impact.** Run the spam classifier with smoothing values of 0.01, 0.1, 1.0, and 10.0. How do the top word probabilities change? What happens with smoothing=0 and a word that appears only in ham?
3. **Add features.** Extend the NaiveBayes class to also use message length (short/long) as a feature alongside word counts. Estimate $P(\text{short}|\text{spam})$ and $P(\text{short}|\text{ham})$ from the training data and fold it into the prediction score.
4. **MAP by hand.** Given observed data (7 heads in 10 coin flips), compute the MAP estimate of the bias using a Beta(2,2) prior. Compare it to the MLE estimate (7/10).

Key Terms

Term	What people say	What it actually means
Prior	"My initial guess"	$P(\text{hypothesis})$ before observing evidence. In ML: the regularization term.

Term	What people say	What it actually means
Likelihood	"How well the data fits"	$P(\text{evidence} \text{hypothesis})$. How probable the observed data is under a specific hypothesis.
Posterior	"My updated belief"	$P(\text{hypothesis} \text{evidence})$. The prior multiplied by the likelihood, then normalized.
Evidence	"The normalizing constant"	$P(\text{data})$ across all hypotheses. Ensures the posterior sums to 1.
Naive Bayes	"That simple text classifier"	A classifier that assumes features are independent given the class. Works well despite the false assumption.
Laplace smoothing	"Add-one smoothing"	Adding a small count to every feature to prevent zero probabilities from unseen data.
MLE	"Just use the frequencies"	Choose parameters that maximize $P(\text{data} \text{parameters})$. No prior. Can overfit with small data.
MAP	"MLE with a prior"	Choose parameters that maximize $P(\text{data} \text{parameters}) * P(\text{parameters})$. Equivalent to regularized MLE.
Log-probability	"Work in log space"	Using $\log(P)$ instead of P to avoid floating-point underflow when multiplying many small numbers.
False positive	"A wrong alarm"	The test says positive, but the true state is negative. Drives the base rate fallacy.

Further Reading

- 3Blue1Brown: Bayes' theorem - visual explanation with the medical test example
- Stanford CS229: Generative Learning Algorithms - naive Bayes and its connection to discriminative models
- Think Bayes - free book, Bayesian statistics with Python code
- scikit-learn Naive Bayes - production implementations and when to use each variant

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/07-bayes-theorem>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/07-bayes-theorem/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/07-bayes-theorem>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Optimization

Training a neural network is nothing more than finding the bottom of a valley.

Type: Build **Language:** Python **Prerequisites:** Phase 1, Lessons 04-05 (Derivatives, Gradients) **Time:** ~75 minutes

Learning Objectives

- Implement vanilla gradient descent, SGD with momentum, and Adam from scratch
- Compare optimizer convergence on the Rosenbrock function and explain why Adam adapts per-weight learning rates
- Distinguish convex from non-convex loss landscapes and explain the role of saddle points in high dimensions
- Configure learning rate schedules (step decay, cosine annealing, warmup) for training stability

The Problem

You have a loss function. It tells you how wrong your model is. You have gradients. They tell you which direction makes the loss worse. Now you need a strategy for walking downhill.

The naive approach is simple: move opposite the gradient. Scale the step by some number called the learning rate. Repeat. This is gradient descent, and it works. But “works” has caveats. Too large a learning rate and you overshoot the valley entirely, bouncing between walls. Too small and you crawl toward the answer over thousands of unnecessary steps. Hit a saddle point and you stop moving even though you have not found a minimum.

Every optimizer in deep learning is an answer to the same question: how do you get to the bottom of the valley faster and more reliably?

The Concept

What optimization means

Optimization is finding the input values that minimize (or maximize) a function. In machine learning, the function is the loss. The inputs are the model’s weights. Training is optimization.

minimize $L(w)$ where:

L = loss function

w = model weights (could be millions of parameters)

Gradient descent (vanilla)

The simplest optimizer. Compute the gradient of the loss with respect to every weight. Move each weight in the opposite direction of its gradient. Scale the step by the learning rate.

```
w = w - lr * gradient
```

That is the entire algorithm. One line.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/08-optimization>

Learning rate: the most important hyperparameter

The learning rate controls step size. It determines everything about convergence.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/08-optimization>

There is no formula for the right learning rate. You find it by experiment. Common starting points: 0.001 for Adam, 0.01 for SGD with momentum.

SGD vs batch vs mini-batch

Vanilla gradient descent computes the gradient over the entire dataset before taking one step. This is called batch gradient descent. It is stable but slow.

Stochastic gradient descent (SGD) computes the gradient on a single random sample and steps immediately. It is noisy but fast.

Mini-batch gradient descent splits the difference. Compute the gradient over a small batch (32, 64, 128, 256 samples), then step. This is what everyone actually uses.

Variant	Batch size	Gradient quality	Speed per step	Noise
Batch GD	Entire dataset	Exact	Slow	None
SGD	1 sample	Very noisy	Fast	High
Mini-batch	32-256	Good estimate	Balanced	Moderate

The noise in SGD and mini-batch is not a bug. It helps escape shallow local minima and saddle points.

Momentum: the ball rolling downhill

Vanilla gradient descent only looks at the current gradient. If the gradient zigzags (common in narrow valleys), progress is slow. Momentum fixes this by accumulating past gradients into a velocity term.

```
v = beta * v + gradient
w = w - lr * v
```

The analogy: a ball rolling downhill. It does not stop and restart at every bump. It builds speed in consistent directions and dampens oscillations.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/08-optimization>

beta (typically 0.9) controls how much history to keep. Higher beta means more momentum, smoother paths, but slower response to direction changes.

Adam: adaptive learning rates

Different weights need different learning rates. A weight that rarely gets large gradients should take bigger steps when it finally does. A weight that gets huge gradients constantly should take smaller steps.

Adam (Adaptive Moment Estimation) tracks two things per weight:

1. First moment (m): running average of gradients (like momentum)
2. Second moment (v): running average of squared gradients (gradient magnitude)

```
m = beta1 * m + (1 - beta1) * gradient
v = beta2 * v + (1 - beta2) * gradient^2
```

```
m_hat = m / (1 - beta1^t)    bias correction
v_hat = v / (1 - beta2^t)    bias correction
```

```
w = w - lr * m_hat / (sqrt(v_hat) + epsilon)
```

The division by $\sqrt{v_hat}$ is the key insight. Weights with large gradients get divided by a large number (small effective step). Weights with small gradients get divided by a small number (large effective step). Each weight gets its own adaptive learning rate.

Default hyperparameters: $lr=0.001$, $\beta_1=0.9$, $\beta_2=0.999$, $\epsilon=1e-8$. These defaults work well for most problems.

Learning rate schedules

A fixed learning rate is a compromise. Early in training, you want large steps to make fast progress. Late in training, you want small steps to fine-tune near the minimum.

Common schedules:

Schedule	Formula	Use case
Step decay	$lr = lr * \text{factor every } N \text{ epochs}$	Simple, manual control
Exponential decay	$lr = lr_0 * \text{decay}^t$	Smooth reduction
Cosine annealing	$lr = lr_min + 0.5 * (lr_max - lr_min) * (1 + \cos(\pi * t / T))$	Transformers, modern training
Warmup + decay	Linear ramp up, then decay	Large models, prevents early instability

Convex vs non-convex

A convex function has one minimum. Gradient descent always finds it. A quadratic like $f(x) = x^2$ is convex.

Neural network loss functions are non-convex. They have many local minima, saddle points, and flat regions.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/08-optimization>

In practice, local minima in high-dimensional neural networks are rarely a problem. Most local minima have loss values close to the global minimum. Saddle points (flat in some directions, curved in others) are the real obstacle. Momentum and noise from mini-batches help escape them.

Loss landscape visualization

The loss is a function of all weights. For a model with 1 million weights, the loss landscape lives in 1,000,001-dimensional space. We visualize it by picking two random directions in weight space and plotting the loss along those directions, producing a 2D surface.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/08-optimization>

Sharp minima generalize poorly. Flat minima generalize well. This is one reason SGD with momentum often outperforms Adam on final test accuracy: its noise prevents settling into sharp minima.

Interactive figure: gradient-descent. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/08-optimization>

Build It

Step 1: Define a test function

The Rosenbrock function is a classic optimization benchmark. Its minimum is at (1, 1) inside a narrow curved valley that is easy to find but hard to follow.

$$f(x, y) = (1 - x)^2 + 100 * (y - x^2)^2$$

```
def rosenbrock(params):
    x, y = params
    return (1 - x) ** 2 + 100 * (y - x ** 2) ** 2

def rosenbrock_gradient(params):
    x, y = params
    df_dx = -2 * (1 - x) + 200 * (y - x ** 2) * (-2 * x)
    df_dy = 200 * (y - x ** 2)
    return [df_dx, df_dy]
```

Step 2: Vanilla gradient descent

```
class GradientDescent:
    def __init__(self, lr=0.001):
        self.lr = lr

    def step(self, params, grads):
        return [p - self.lr * g for p, g in zip(params, grads)]
```

Step 3: SGD with momentum

```
class SGDMomentum:
    def __init__(self, lr=0.001, momentum=0.9):
        self.lr = lr
        self.momentum = momentum
        self.velocity = None

    def step(self, params, grads):
        if self.velocity is None:
```

```

        self.velocity = [0.0] * len(params)
    self.velocity = [
        self.momentum * v + g
        for v, g in zip(self.velocity, grads)
    ]
    return [p - self.lr * v for p, v in zip(params, self.velocity)]

```

Step 4: Adam

```

class Adam:
    def __init__(self, lr=0.001, beta1=0.9, beta2=0.999, epsilon=1e-8):
        self.lr = lr
        self.beta1 = beta1
        self.beta2 = beta2
        self.epsilon = epsilon
        self.m = None
        self.v = None
        self.t = 0

    def step(self, params, grads):
        if self.m is None:
            self.m = [0.0] * len(params)
            self.v = [0.0] * len(params)

        self.t += 1

        self.m = [
            self.beta1 * m + (1 - self.beta1) * g
            for m, g in zip(self.m, grads)
        ]
        self.v = [
            self.beta2 * v + (1 - self.beta2) * g ** 2
            for v, g in zip(self.v, grads)
        ]

        m_hat = [m / (1 - self.beta1 ** self.t) for m in self.m]
        v_hat = [v / (1 - self.beta2 ** self.t) for v in self.v]

        return [
            p - self.lr * mh / (vh ** 0.5 + self.epsilon)
            for p, mh, vh in zip(params, m_hat, v_hat)
        ]

```

Step 5: Run and compare

```

def optimize(optimizer, func, grad_func, start, steps=5000):
    params = list(start)
    history = [params[:]]
    for _ in range(steps):
        grads = grad_func(params)
        params = optimizer.step(params, grads)
        history.append(params[:])
    return history

```

```

start = [-1.0, 1.0]

gd_history = optimize(GradientDescent(lr=0.0005), rosenbrock, rosenbrock_gradient, start)
sgd_history = optimize(SGDMomentum(lr=0.0001, momentum=0.9), rosenbrock, rosenbrock_gradient, start)
adam_history = optimize(Adam(lr=0.01), rosenbrock, rosenbrock_gradient, start)

for name, history in [("GD", gd_history), ("SGD+M", sgd_history), ("Adam", adam_history)]:
    final = history[-1]
    loss = rosenbrock(final)
    print(f"{name:6s} -> x={final[0]:.6f}, y={final[1]:.6f}, loss={loss:.8f}")

```

Expected output: Adam converges fastest. SGD with momentum follows a smoother path. Vanilla GD makes slow progress along the narrow valley.

Use It

In practice, use PyTorch or JAX optimizers. They handle parameter groups, weight decay, gradient clipping, and GPU acceleration.

```

import torch

model = torch.nn.Linear(784, 10)

sgd = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9)
adam = torch.optim.Adam(model.parameters(), lr=0.001)
adamw = torch.optim.AdamW(model.parameters(), lr=0.001, weight_decay=0.01)

scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(adam, T_max=100)

```

Rules of thumb:

- Start with Adam (lr=0.001). It works for most problems without tuning.
- Switch to SGD with momentum (lr=0.01, momentum=0.9) when you need the best final accuracy and can afford more tuning.
- Use AdamW (Adam with decoupled weight decay) for transformers.
- Always use a learning rate schedule for training runs longer than a few epochs.
- If training is unstable, reduce the learning rate. If training is too slow, increase it.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/08-optimization>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/08-optimization/code>

1. **Learning rate sweep.** Run vanilla gradient descent on the Rosenbrock function with learning rates [0.0001, 0.0005, 0.001, 0.005, 0.01]. Plot or print the final loss after 5000 steps for each. Find the largest learning rate that still converges.
2. **Momentum comparison.** Run SGD with momentum values [0.0, 0.5, 0.9, 0.99] on the Rosenbrock function. Track the loss at every step. Which momentum value converges fastest? Which overshoots?

3. **Saddle point escape.** Define the function $f(x, y) = x^2 - y^2$ (a saddle point at the origin). Start at (0.01, 0.01). Compare how vanilla GD, SGD with momentum, and Adam behave. Which escapes the saddle point?
4. **Implement learning rate decay.** Add an exponential decay schedule to the Gradient-Descent class: $lr = lr_0 * 0.999^{step}$. Compare convergence with and without decay on the Rosenbrock function.

Key Terms

Term	What people say	What it actually means
Gradient descent	“Go downhill”	Update weights by subtracting the gradient scaled by the learning rate. The most basic optimizer.
Learning rate	“Step size”	A scalar that controls how far each update moves the weights. Too large causes divergence. Too small wastes compute.
Momentum	“Keep rolling”	Accumulate past gradients into a velocity vector. Dampens oscillations and accelerates movement through consistent directions.
SGD	“Random sampling”	Stochastic gradient descent. Compute gradient on a random subset instead of the full dataset. Almost always means mini-batch SGD in practice.
Mini-batch	“A chunk of data”	A small subset of training data (32-256 samples) used to estimate the gradient.
Adam	“The default optimizer”	Balances speed and gradient accuracy. Adaptive Moment Estimation. Tracks per-weight running averages of gradients and squared gradients to give each weight its own learning rate.
Bias correction	“Fix the cold start”	Adam’s first and second moments are initialized to zero. Bias correction divides by $(1 - \beta^t)$ to compensate during early steps.
Learning rate schedule	“Change lr over time”	A function that adjusts the learning rate during training. Large steps early, small steps late.
Convex function	“One valley”	A function where any local minimum is the global minimum. Gradient descent always finds it. Neural network losses are not convex.
Saddle point	“Flat but not a minimum”	A point where the gradient is zero but it is a minimum in some directions and a maximum in others. Common in high dimensions.
Loss landscape	“The terrain”	The loss function plotted over weight space. Visualized by slicing along two random directions.

Term	What people say	What it actually means
Convergence	“Getting there”	The optimizer has reached a point where further steps do not meaningfully reduce the loss.

Further Reading

- Sebastian Ruder: An overview of gradient descent optimization algorithms - comprehensive survey of all major optimizers
- Why Momentum Really Works (Distill) - interactive visualization of momentum dynamics
- Adam: A Method for Stochastic Optimization (Kingma & Ba, 2014) - the original Adam paper, readable and short
- Visualizing the Loss Landscape of Neural Nets (Li et al., 2018) - the paper that showed sharp vs flat minima

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/08-optimization>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/08-optimization/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/08-optimization>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Information Theory

Information theory measures surprise. Loss functions are built on it.

Type: Learn **Language:** Python **Prerequisites:** Phase 1, Lesson 06 (Probability) **Time:** ~60 minutes

Learning Objectives

- Compute entropy, cross-entropy, and KL divergence from scratch and explain their relationship
- Derive why minimizing cross-entropy loss is equivalent to maximizing log-likelihood
- Calculate mutual information between features and a target to rank feature importance
- Explain perplexity as the effective vocabulary size a language model chooses from

The Problem

You call `CrossEntropyLoss()` in every classification model you train. You see “perplexity” in every language model paper. You read about KL divergence in VAEs, distillation, and RLHF. These are not disconnected concepts. They are all the same idea wearing different hats.

Information theory gives you the language to reason about uncertainty, compression, and prediction. Claude Shannon invented it in 1948 to solve communication problems. Turns out, training a neural network is a communication problem: the model is trying to transmit the correct label through a noisy channel of learned weights.

This lesson builds every formula from scratch so you see where they come from and why they work.

The Concept

Information Content (Surprise)

When something unlikely happens, it carries more information. A coin landing heads? Not surprising. A lottery win? Very surprising.

The information content of an event with probability p is:

$$I(x) = -\log(p(x))$$

Using log base 2 gives you bits. Using natural log gives you nats. Same idea, different units.

Event	Probability	Surprise (bits)
Fair coin heads	0.5	1.0
Rolling a 6	0.167	2.58
1-in-1000 event	0.001	9.97
Certain event	1.0	0.0

Certain events carry zero information. You already knew they would happen.

Entropy (Average Surprise)

Entropy is the expected surprise across all possible outcomes of a distribution.

$$H(P) = -\sum (p(x) * \log(p(x))) \text{ for all } x$$

A fair coin has maximum entropy for a binary variable: 1 bit. A biased coin (99% heads) has low entropy: 0.08 bits. You already know what will happen, so each flip tells you almost nothing.

$$\text{Fair coin: } H = -(0.5 * \log_2(0.5) + 0.5 * \log_2(0.5)) = 1.0 \text{ bit}$$

$$\text{Biased coin: } H = -(0.99 * \log_2(0.99) + 0.01 * \log_2(0.01)) = 0.08 \text{ bits}$$

Entropy measures the irreducible uncertainty in a distribution. You cannot compress below it.

Cross-Entropy (The Loss Function You Use Every Day)

Cross-entropy measures the average surprise when you use distribution Q to encode events that actually come from distribution P.

$$H(P, Q) = -\sum (p(x) * \log(q(x))) \text{ for all } x$$

P is the true distribution (the labels). Q is your model's predictions. If Q matches P perfectly, cross-entropy equals entropy. Any mismatch makes it larger.

In classification, P is a one-hot vector (the true class has probability 1, everything else 0). This simplifies cross-entropy to:

$$H(P, Q) = -\log(q(\text{true_class}))$$

That is the entire cross-entropy loss formula for classification. Maximize the predicted probability of the correct class.

KL Divergence (Distance Between Distributions)

KL divergence measures how much extra surprise you get from using Q instead of P.

$$\begin{aligned} D_{\text{KL}}(P \parallel Q) &= \sum (p(x) * \log(p(x) / q(x))) \text{ for all } x \\ &= H(P, Q) - H(P) \end{aligned}$$

Cross-entropy is entropy plus KL divergence. Since entropy of the true distribution is constant during training, minimizing cross-entropy is the same as minimizing KL divergence. You are pushing your model's distribution toward the true distribution.

KL divergence is not symmetric: $D_{\text{KL}}(P \parallel Q) \neq D_{\text{KL}}(Q \parallel P)$. It is not a true distance metric.

Mutual Information

Mutual information measures how much knowing one variable tells you about another.

$$\begin{aligned} I(X; Y) &= H(X) - H(X|Y) \\ &= H(X) + H(Y) - H(X, Y) \end{aligned}$$

If X and Y are independent, mutual information is zero. Knowing one tells you nothing about the other. If they are perfectly correlated, mutual information equals the entropy of either variable.

In feature selection, high mutual information between a feature and the target means the feature is useful. Low mutual information means it is noise.

Conditional Entropy

$H(Y|X)$ measures how much uncertainty remains about Y after you observe X .

$$H(Y|X) = H(X, Y) - H(X)$$

Two extremes: - If X completely determines Y , then $H(Y|X) = 0$. Knowing X eliminates all uncertainty about Y . Example: X = temperature in Celsius, Y = temperature in Fahrenheit. - If X tells you nothing about Y , then $H(Y|X) = H(Y)$. Knowing X does not reduce your uncertainty at all. Example: X = coin flip, Y = tomorrow's weather.

Conditional entropy is always non-negative and never exceeds $H(Y)$:

$$0 \leq H(Y|X) \leq H(Y)$$

In machine learning, conditional entropy appears in decision trees. At each split, the algorithm picks the feature X that minimizes $H(Y|X)$ - the feature that removes the most uncertainty about the label Y .

Joint Entropy

$H(X, Y)$ is the entropy of the joint distribution of X and Y together.

$$H(X, Y) = -\sum_x \sum_y p(x, y) \log(p(x, y)) \quad \text{for all } x, y$$

Key property:

$$H(X, Y) \leq H(X) + H(Y)$$

Equality holds when X and Y are independent. If they share information, the joint entropy is less than the sum of individual entropies. The "missing" entropy is exactly the mutual information.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/09-information-theory>

The relationships: - $H(X, Y) = H(X) + H(Y|X) = H(Y) + H(X|Y)$ - $I(X; Y) = H(X) - H(X|Y) = H(Y) - H(Y|X)$ - $H(X, Y) = H(X) + H(Y) - I(X; Y)$

Mutual Information (Deep Dive)

Mutual information $I(X; Y)$ quantifies how much knowing one variable reduces uncertainty about the other.

$$\begin{aligned} I(X; Y) &= H(X) - H(X|Y) \\ &= H(Y) - H(Y|X) \\ &= H(X) + H(Y) - H(X, Y) \\ &= \sum_x \sum_y p(x, y) \log(p(x, y) / (p(x) * p(y))) \end{aligned}$$

Properties: - $I(X; Y) \geq 0$ always. You never lose information by observing something. - $I(X; Y) = 0$ if and only if X and Y are independent. - $I(X; Y) = I(Y; X)$. It is symmetric, unlike KL divergence. - $I(X; X) = H(X)$. A variable shares all its information with itself.

Mutual information for feature selection. In ML, you want features that are informative about the target. Mutual information gives you a principled way to rank features:

1. For each feature X_i , compute $I(X_i; Y)$ where Y is the target variable.
2. Rank features by MI score.
3. Keep the top k features.

This works for any relationship between feature and target – linear, nonlinear, monotonic, or not. Correlation only catches linear relationships. MI catches everything.

Method	Detects	Computational cost	Handles categorical?
Pearson correlation	Linear relationships	$O(n)$	No
Spearman correlation	Monotonic relationships	$O(n \log n)$	No
Mutual information	Any statistical dependency	$O(n \log n)$ with binning	Yes

Label Smoothing and Cross-Entropy

Standard classification uses hard targets: $[0, 0, 1, 0]$. The true class gets probability 1, everything else gets 0. Label smoothing replaces these with soft targets:

$\text{soft_target} = (1 - \epsilon) * \text{hard_target} + \epsilon / \text{num_classes}$

With $\epsilon = 0.1$ and 4 classes: - Hard target: $[0, 0, 1, 0]$ - Soft target: $[0.025, 0.025, 0.925, 0.025]$

From an information theory perspective, label smoothing increases the entropy of the target distribution. Hard one-hot targets have entropy 0 – there is no uncertainty. Soft targets have positive entropy.

Why this helps: - Prevents the model from driving logits to extreme values (infinite logits would be needed to perfectly match a one-hot target under cross-entropy) - Acts as regularization: the model cannot be 100% confident - Improves calibration: predicted probabilities better reflect true uncertainty - Reduces the gap between training and inference behavior

The cross-entropy loss with label smoothing becomes:

$L = (1 - \epsilon) * \text{CE}(\text{hard_target}, \text{prediction}) + \epsilon * H_{\text{uniform}}(\text{prediction})$

The second term penalizes predictions that are far from uniform – a direct regularization on confidence.

Why Cross-Entropy Is THE Classification Loss

Three perspectives, same conclusion.

Information theory view. Cross-entropy measures how many bits you waste by using your model's distribution instead of the true distribution. Minimizing it makes your model the most efficient encoder of reality.

Maximum likelihood view. For N training samples with true classes y_i :

Likelihood = $\text{product}(q(y_i))$
 Log-likelihood = $\text{sum}(\log(q(y_i)))$
 Negative log-likelihood = $-\text{sum}(\log(q(y_i)))$

That last line is cross-entropy loss. Minimizing cross-entropy = maximizing the likelihood of the training data under your model.

Gradient view. The gradient of cross-entropy with respect to the logits is simply (predicted - true). Clean, stable, and fast to compute. This is why it pairs perfectly with softmax.

Bits vs Nats

The only difference is the log base.

```
log base 2    -> bits      (information theory tradition)
log base e    -> nats      (machine learning convention)
log base 10   -> hartleys  (rarely used)
```

1 nat = $1/\ln(2)$ bits = 1.4427 bits. PyTorch and TensorFlow use natural log (nats) by default.

Perplexity

Perplexity is the exponential of cross-entropy. It tells you the effective number of equally likely choices the model is uncertain between.

```
Perplexity = 2^H(P,Q)    (if using bits)
Perplexity = e^H(P,Q)    (if using nats)
```

A language model with perplexity 50 is, on average, as confused as if it had to pick uniformly from 50 possible next tokens. Lower is better.

GPT-2 achieved perplexity ~ 30 on common benchmarks. Modern models are in the single digits for well-represented domains.

Interactive figure: entropy-kl. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/09-information-theory>

Build It

Step 1: Information content and entropy

```
import math

def information_content(p, base=2):
    if p <= 0 or p > 1:
        return float('inf') if p <= 0 else 0.0
    return -math.log(p) / math.log(base)

def entropy(probs, base=2):
    return sum(
        p * information_content(p, base)
        for p in probs if p > 0
    )

fair_coin = [0.5, 0.5]
biased_coin = [0.99, 0.01]
fair_die = [1/6] * 6

print(f"Fair coin entropy: {entropy(fair_coin):.4f} bits")
print(f"Biased coin entropy: {entropy(biased_coin):.4f} bits")
print(f"Fair die entropy: {entropy(fair_die):.4f} bits")
```

Step 2: Cross-entropy and KL divergence

```
def cross_entropy(p, q, base=2):
    total = 0.0
    for pi, qi in zip(p, q):
        if pi > 0:
            if qi <= 0:
                return float('inf')
            total += pi * (-math.log(qi) / math.log(base))
    return total

def kl_divergence(p, q, base=2):
    return cross_entropy(p, q, base) - entropy(p, base)

true_dist = [0.7, 0.2, 0.1]
good_model = [0.6, 0.25, 0.15]
bad_model = [0.1, 0.1, 0.8]

print(f"Entropy of true dist:      {entropy(true_dist):.4f} bits")
print(f"CE (good model):           {cross_entropy(true_dist, good_model):.4f} bits")
print(f"CE (bad model):              {cross_entropy(true_dist, bad_model):.4f} bits")
print(f"KL divergence (good):        {kl_divergence(true_dist, good_model):.4f} bits")
print(f"KL divergence (bad):          {kl_divergence(true_dist, bad_model):.4f} bits")
```

Step 3: Cross-entropy as classification loss

```
def softmax(logits):
    max_logit = max(logits)
    exps = [math.exp(z - max_logit) for z in logits]
    total = sum(exps)
    return [e / total for e in exps]

def cross_entropy_loss(true_class, logits):
    probs = softmax(logits)
    return -math.log(probs[true_class])

logits = [2.0, 1.0, 0.1]
true_class = 0

probs = softmax(logits)
loss = cross_entropy_loss(true_class, logits)

print(f"Logits:      {logits}")
print(f"Softmax:      {[f'{p:.4f}' for p in probs]}")
print(f"True class:    {true_class}")
print(f"Loss:          {loss:.4f} nats")
print(f"Perplexity:    {math.exp(loss):.2f}")
```

Step 4: Cross-entropy equals negative log-likelihood

```
import random

random.seed(42)
```



```

n_samples = 1000
n_classes = 3
true_labels = [random.randint(0, n_classes - 1) for _ in range(n_samples)]
model_logits = [[random.gauss(0, 1) for _ in range(n_classes)] for _ in range(n_samples)]

ce_loss = sum(
    cross_entropy_loss(label, logits)
    for label, logits in zip(true_labels, model_logits)
) / n_samples

nll = -sum(
    math.log(softmax(logits)[label])
    for label, logits in zip(true_labels, model_logits)
) / n_samples

print(f"Cross-entropy loss:      {ce_loss:.6f}")
print(f"Negative log-likelihood: {nll:.6f}")
print(f"Difference:               {abs(ce_loss - nll):.2e}")

```

Step 5: Mutual information

```

def mutual_information(joint_probs, base=2):
    rows = len(joint_probs)
    cols = len(joint_probs[0])

    margin_x = [sum(joint_probs[i][j] for j in range(cols)) for i in range(rows)]
    margin_y = [sum(joint_probs[i][j] for i in range(rows)) for j in range(cols)]

    mi = 0.0
    for i in range(rows):
        for j in range(cols):
            pxy = joint_probs[i][j]
            if pxy > 0:
                mi += pxy * math.log(pxy / (margin_x[i] * margin_y[j])) / math.log(base)
    return mi

independent = [[0.25, 0.25], [0.25, 0.25]]
dependent = [[0.45, 0.05], [0.05, 0.45]]

print(f"MI (independent): {mutual_information(independent):.4f} bits")
print(f"MI (dependent):     {mutual_information(dependent):.4f} bits")

```

Use It

The same concepts using NumPy, the way you will use them in practice:

```

import numpy as np

def np_entropy(p):
    p = np.asarray(p, dtype=float)
    mask = p > 0
    result = np.zeros_like(p)
    result[mask] = p[mask] * np.log(p[mask])

```

```

    return -result.sum()

def np_cross_entropy(p, q):
    p, q = np.asarray(p, dtype=float), np.asarray(q, dtype=float)
    mask = p > 0
    return -(p[mask] * np.log(q[mask])).sum()

def np_kl_divergence(p, q):
    return np_cross_entropy(p, q) - np_entropy(p)

true = np.array([0.7, 0.2, 0.1])
pred = np.array([0.6, 0.25, 0.15])
print(f"Entropy:      {np_entropy(true):.4f} nats")
print(f"Cross-ent:    {np_cross_entropy(true, pred):.4f} nats")
print(f"KL div:        {np_kl_divergence(true, pred):.4f} nats")

```

You built from scratch what `torch.nn.CrossEntropyLoss()` does internally. Now you know why the loss goes down during training: your model's predicted distribution is getting closer to the true distribution, measured in nats of wasted information.

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/09-information-theory/code>

1. Compute the entropy of the English alphabet assuming uniform distribution (26 letters). Then estimate it using actual letter frequencies. Which is higher and why?
2. A model outputs logits [5.0, 2.0, 0.5] for a sample with true class 1. Compute the cross-entropy loss by hand, then verify with your `cross_entropy_loss` function. What logits would give zero loss?
3. Show that KL divergence is not symmetric. Pick two distributions P and Q and compute $D_{KL}(P \parallel Q)$ and $D_{KL}(Q \parallel P)$. Explain why they differ.
4. Build a function that computes perplexity for a sequence of token predictions. Given a list of (true_token_index, predicted_logits) pairs, return the perplexity of the sequence.

Key Terms

Term	What people say	What it actually means
Information content	"Surprise"	The number of bits (or nats) needed to encode an event: $-\log(p)$
Entropy	"Randomness"	The average surprise across all outcomes of a distribution. Measures irreducible uncertainty.
Cross-entropy	"The loss function"	Average surprise when using model distribution Q to encode events from true distribution P.
KL divergence	"Distance between distributions"	Extra bits wasted by using Q instead of P. Equals cross-entropy minus entropy. Not symmetric.

Term	What people say	What it actually means
Mutual information	“How related are X and Y”	Reduction in uncertainty about X from knowing Y. Zero means independent.
Softmax	“Turn logits into probabilities”	Exponentiate and normalize. Maps any real-valued vector to a valid probability distribution.
Perplexity	“How confused the model is”	Exponential of cross-entropy. The effective vocabulary size the model is choosing from at each step.
Bits	“Shannon’s unit”	Information measured with log base 2. One bit resolves one fair coin flip.
Nats	“ML’s unit”	Information measured with natural log. Used by PyTorch and TensorFlow by default.
Negative log-likelihood	“NLL loss”	Identical to cross-entropy loss for one-hot labels. Minimizing it maximizes the probability of correct predictions.

Further Reading

- Shannon 1948: A Mathematical Theory of Communication - the original paper, still readable
- Visual Information Theory (Chris Olah) - best visual explanation of entropy and KL divergence
- PyTorch CrossEntropyLoss docs - how the framework implements what you just built

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/09-information-theory>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/09-information-theory/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/09-information-theory>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Dimensionality Reduction

High-dimensional data has structure. You find it by looking from the right angle.

Type: Build **Language:** Python **Prerequisites:** Phase 1, Lessons 01 (Linear Algebra Intuition), 02 (Vectors, Matrices & Operations), 03 (Eigenvalues & Eigenvectors), 06 (Probability & Distributions) **Time:** ~90 minutes

Learning Objectives

- Implement PCA from scratch: center data, compute the covariance matrix, eigendecompose, and project
- Use explained variance ratio and the elbow method to choose the number of principal components
- Compare PCA, t-SNE, and UMAP for visualizing MNIST digits in 2D and explain their tradeoffs
- Apply kernel PCA with an RBF kernel to separate nonlinear data structures that standard PCA cannot handle

The Problem

You have a dataset with 784 features per sample. Maybe it is pixel values of handwritten digits. Maybe it is gene expression levels. Maybe it is user behavior signals. You cannot visualize 784 dimensions. You cannot plot them. You cannot even think about them.

But most of those 784 features are redundant. The actual information lives on a much smaller surface. A handwritten “7” does not need 784 independent numbers to describe it. It needs a few: the angle of the stroke, the length of the crossbar, how much it leans. The rest is noise.

Dimensionality reduction finds that smaller surface. It takes your 784-dimensional data and compresses it to 2, 10, or 50 dimensions while keeping the structure that matters.

The Concept

The curse of dimensionality

High-dimensional spaces are unintuitive. Three things break as dimensions grow.

Distance becomes meaningless. In high dimensions, the distance between any two random points converges to the same value. If every point is roughly the same distance from every other point, nearest-neighbor search stops working.

Dimension	Avg distance ratio (max/min between random points)
2	~5.0
10	~1.8
100	~1.2
1000	~1.02

Volume concentrates in corners. A unit hypercube in d dimensions has 2^d corners. In 100 dimensions, nearly all the volume is in the corners, far from the center. Data points spread to the edges and your models starve for data in the interior.

You need exponentially more data. To maintain the same density of samples in a space, going from 2D to 20D means you need 10^{18} times more data. You never have enough. Reducing dimensions brings the data density back to something workable.

PCA: find the directions that matter

Principal Component Analysis (PCA) finds the axes along which your data varies the most. It rotates your coordinate system so the first axis captures the most variance, the second captures the next most, and so on.

The algorithm:

1. Center the data (subtract the mean from each feature)
2. Compute covariance (how features move together)
3. Eigendecomposition (find the principal directions)
4. Sort by eigenvalue (biggest variance first)
5. Project (keep top k eigenvectors, drop the rest)

Why eigendecomposition? The covariance matrix is symmetric and positive semi-definite. Its eigenvectors are orthogonal directions in feature space. The eigenvalues tell you how much variance each direction captures. The eigenvector with the largest eigenvalue points along the direction of maximum variance.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/10-dimensionality-reduction>

- **Before PCA:** Data cloud is spread diagonally across both x and y axes
- **After PCA:** Coordinate system is rotated so PC1 aligns with the direction of maximum variance (elongated spread) and PC2 aligns with the direction of minimum variance (narrow spread)
- **Dimensionality reduction:** Dropping PC2 projects the data onto PC1, losing very little information

Explained variance ratio

Each principal component captures a fraction of the total variance. The explained variance ratio tells you how much.

Component	Eigenvalue	Explained ratio	Cumulative
PC1	4.73	0.473	0.473
PC2	2.51	0.251	0.724
PC3	1.12	0.112	0.836
PC4	0.89	0.089	0.925
...			

When the cumulative explained variance reaches 0.95, you know that many components capture 95% of the information. Everything after that is mostly noise.

Choosing the number of components

Three strategies:

1. **Threshold.** Keep enough components to explain 90-95% of the variance.
2. **Elbow method.** Plot explained variance per component. Look for a sharp drop-off.

3. **Downstream performance.** Use PCA as preprocessing. Sweep k and measure your model's accuracy. The best k is wherever accuracy plateaus.

t-SNE: preserve neighborhoods

t-Distributed Stochastic Neighbor Embedding (t-SNE) is designed for visualization. It maps high-dimensional data to 2D (or 3D) while preserving which points are near each other.

The intuition: in the original space, compute a probability distribution over pairs of points based on their distances. Near points get high probability. Far points get low probability. Then find a 2D arrangement where the same probability distribution holds. Points that were neighbors in 784 dimensions stay neighbors in 2D.

Key properties of t-SNE: - Non-linear. It can unfold complex manifolds that PCA cannot. - Stochastic. Different runs produce different layouts. - Perplexity parameter controls how many neighbors to consider (typical range: 5-50). - Distances between clusters in the output are not meaningful. Only the clusters themselves are. - Slow on large datasets. $O(n^2)$ by default.

UMAP: faster, better global structure

Uniform Manifold Approximation and Projection (UMAP) works similarly to t-SNE but with two advantages: - Faster. It uses approximate nearest-neighbor graphs instead of computing all pairwise distances. - Better global structure. The relative positions of clusters in the output tend to be more meaningful than in t-SNE.

UMAP builds a weighted graph in high-dimensional space (the "fuzzy topological representation") and then finds a low-dimensional layout that preserves this graph as well as possible.

Key parameters: - `n_neighbors`: how many neighbors define local structure (similar to perplexity). Higher values preserve more global structure. - `min_dist`: how tightly points pack together in the output. Lower values create denser clusters.

When to use which

Method	Use case	Preserves	Speed
PCA	Preprocessing before training	Global variance	Fast (exact), works on millions of samples
PCA	Quick exploratory visualization	Linear structure	Fast
t-SNE	Publication-quality 2D plots	Local neighborhoods	Slow (< 10k samples ideal)
UMAP	2D visualization at scale	Local + some global structure	Medium (handles millions)
PCA	Feature reduction for models	Variance-ranked features	Fast
t-SNE / UMAP	Understanding cluster structure	Cluster separation	Medium to slow

Rule of thumb: use PCA for preprocessing and data compression. Use t-SNE or UMAP when you need to visualize structure in 2D.

Kernel PCA

Standard PCA finds linear subspaces. It rotates your coordinate system and drops axes. But what if the data lies on a nonlinear manifold? A circle in 2D cannot be separated by any line. Standard PCA will not help.

Kernel PCA applies PCA in a high-dimensional feature space induced by a kernel function, without explicitly computing the coordinates in that space. This is the kernel trick - the same idea behind SVMs.

The algorithm: 1. Compute the kernel matrix K where $K_{ij} = k(x_i, x_j)$ 2. Center the kernel matrix in feature space 3. Eigendecompose the centered kernel matrix 4. The top eigenvectors (scaled by $1/\sqrt{\text{eigenvalue}}$) are the projections

Common kernel functions:

Kernel	Formula	Good for
RBf (Gaussian)	$\exp(-\gamma * x - y ^2)$	Most nonlinear data, smooth manifolds
Polynomial	$(x \cdot y + c)^d$	Polynomial relationships
Sigmoid	$\tanh(\alpha * x \cdot y + c)$	Neural network-like mappings

When to use kernel PCA vs standard PCA:

Criterion	Standard PCA	Kernel PCA
Data structure	Linear subspace	Nonlinear manifold
Speed	$O(\min(n^2 d, d^2 n))$	$O(n^2 d + n^3)$
Interpretability	Components are linear combinations of features	Components lack direct feature interpretation
Scalability	Works on millions of samples	Kernel matrix is $n \times n$, memory-limited
Reconstruction	Direct inverse transform	Requires pre-image approximation

The classic example: concentric circles in 2D. Two rings of points, one inside the other. Standard PCA projects both onto the same line - useless for classification. Kernel PCA with an RBF kernel maps the inner circle and outer circle to different regions, making them linearly separable.

Reconstruction Error

How good is your dimensionality reduction? You compressed 784 dimensions to 50. What did you lose?

Measure reconstruction error: 1. Project data to k dimensions: $X_{\text{reduced}} = X @ W_k$ 2. Reconstruct: $\hat{X} = X_{\text{reduced}} @ W_k^T$ 3. Compute MSE: $\text{mean}((X - \hat{X})^2)$

For PCA, reconstruction error has a clean relationship to explained variance:

Reconstruction error = sum of eigenvalues NOT included

Total variance = sum of ALL eigenvalues

Fraction lost = (sum of dropped eigenvalues) / (sum of all eigenvalues)

The explained variance ratio for each component is:

```
explained_ratio_k = eigenvalue_k / sum(all_eigenvalues)
```

Plotting cumulative explained variance against number of components gives you the “elbow” curve. The right number of components is where: - The curve flattens out (diminishing returns) - Cumulative variance crosses your threshold (usually 0.90 or 0.95) - Downstream task performance plateaus

Reconstruction error is useful beyond choosing k. You can use it for anomaly detection: samples with high reconstruction error are outliers that do not fit the learned subspace. This is the basis of PCA-based anomaly detection in production systems.

Interactive figure: pca-axes. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/10-dimensionality-reduction>

Build It

Step 1: PCA from scratch

```
import numpy as np

class PCA:
    def __init__(self, n_components):
        self.n_components = n_components
        self.components = None
        self.mean = None
        self.eigenvalues = None
        self.explained_variance_ratio_ = None

    def fit(self, X):
        self.mean = np.mean(X, axis=0)
        X_centered = X - self.mean

        cov_matrix = np.cov(X_centered, rowvar=False)

        eigenvalues, eigenvectors = np.linalg.eigh(cov_matrix)

        sorted_idx = np.argsort(eigenvalues)[::-1]
        eigenvalues = eigenvalues[sorted_idx]
        eigenvectors = eigenvectors[:, sorted_idx]

        self.components = eigenvectors[:, :self.n_components].T
        self.eigenvalues = eigenvalues[:self.n_components]
        total_var = np.sum(eigenvalues)
        self.explained_variance_ratio_ = self.eigenvalues / total_var

        return self

    def transform(self, X):
        X_centered = X - self.mean
        return X_centered @ self.components.T

    def fit_transform(self, X):
        self.fit(X)
```



```
return self.transform(X)
```

Step 2: Test on synthetic data

```
np.random.seed(42)
n_samples = 500

t = np.random.uniform(0, 2 * np.pi, n_samples)
x1 = 3 * np.cos(t) + np.random.normal(0, 0.2, n_samples)
x2 = 3 * np.sin(t) + np.random.normal(0, 0.2, n_samples)
x3 = 0.5 * x1 + 0.3 * x2 + np.random.normal(0, 0.1, n_samples)

X_synthetic = np.column_stack([x1, x2, x3])

pca = PCA(n_components=2)
X_reduced = pca.fit_transform(X_synthetic)

print(f"Original shape: {X_synthetic.shape}")
print(f"Reduced shape: {X_reduced.shape}")
print(f"Explained variance ratios: {pca.explained_variance_ratio_}")
print(f"Total variance captured: {sum(pca.explained_variance_ratio_):.4f}")
```

Step 3: MNIST digits in 2D

```
from sklearn.datasets import fetch_openml

mnist = fetch_openml("mnist_784", version=1, as_frame=False, parser="auto")
X_mnist = mnist.data[:5000].astype(float)
y_mnist = mnist.target[:5000].astype(int)

pca_mnist = PCA(n_components=50)
X_pca50 = pca_mnist.fit_transform(X_mnist)
print(f"50 components capture {sum(pca_mnist.explained_variance_ratio_):.2%} of variance")

pca_2d = PCA(n_components=2)
X_pca2d = pca_2d.fit_transform(X_mnist)
print(f"2 components capture {sum(pca_2d.explained_variance_ratio_):.2%} of variance")
```

Step 4: Compare with sklearn

```
from sklearn.decomposition import PCA as SklearnPCA
from sklearn.manifold import TSNE

sklearn_pca = SklearnPCA(n_components=2)
X_sklearn_pca = sklearn_pca.fit_transform(X_mnist)

print(f"\nOur PCA explained variance: {pca_2d.explained_variance_ratio_}")
print(f"Sklearn PCA explained variance: {sklearn_pca.explained_variance_ratio_}")

diff = np.abs(np.abs(X_pca2d) - np.abs(X_sklearn_pca))
print(f"Max absolute difference: {diff.max():.10f}")

tsne = TSNE(n_components=2, perplexity=30, random_state=42)
```

```
X_tsne = tsne.fit_transform(X_mnist)
print(f"\nt-SNE output shape: {X_tsne.shape}")
```

Step 5: UMAP comparison

```
try:
    from umap import UMAP

    reducer = UMAP(n_components=2, n_neighbors=15, min_dist=0.1, random_state=42)
    X_umap = reducer.fit_transform(X_mnist)
    print(f"UMAP output shape: {X_umap.shape}")
except ImportError:
    print("Install umap-learn: pip install umap-learn")
```

Use It

PCA as preprocessing before a classifier:

```
from sklearn.decomposition import PCA as SklearnPCA
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

X_train, X_test, y_train, y_test = train_test_split(
    X_mnist, y_mnist, test_size=0.2, random_state=42
)

results = {}
for k in [10, 30, 50, 100, 200]:
    pca_k = SklearnPCA(n_components=k)
    X_tr = pca_k.fit_transform(X_train)
    X_te = pca_k.transform(X_test)

    clf = LogisticRegression(max_iter=1000, random_state=42)
    clf.fit(X_tr, y_train)
    acc = accuracy_score(y_test, clf.predict(X_te))
    var_captured = sum(pca_k.explained_variance_ratio_)
    results[k] = (acc, var_captured)
    print(f"k={k:>3d} accuracy={acc:.4f} variance={var_captured:.4f}")
```

Performance plateaus well before 784 dimensions. That plateau is your operating point.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/10-dimensionality-reduction>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/10-dimensionality-reduction/code>

1. Modify the PCA class to support `inverse_transform`. Reconstruct MNIST digits from 10, 50, and 200 components. Print the reconstruction error (mean squared difference from the original) for each.
2. Run t-SNE on the same MNIST subset with perplexity values of 5, 30, and 100. Describe how the output changes. Why does perplexity affect cluster tightness?
3. Take a dataset with 50 features where only 5 are informative (generate one with `sklearn.datasets.make_classification`). Apply PCA and check whether the explained variance curve correctly identifies that the data is effectively 5-dimensional.

Key Terms

Term	What people say	What it actually means
Curse of dimensionality	“Too many features”	Distances, volumes, and data density all behave counterintuitively as dimensions grow. Models need exponentially more data to compensate.
PCA	“Reduce dimensions”	Rotate your coordinate system so the axes align with the directions of maximum variance, then drop the low-variance axes.
Principal component	“An important direction”	An eigenvector of the covariance matrix. The direction in feature space along which the data varies most.
Explained variance ratio	“How much info this component has”	The fraction of total variance captured by one principal component. Sum the top k ratios to see how much k components preserve.
Covariance matrix	“How features correlate”	A symmetric matrix where entry (i,j) measures how feature i and feature j move together. Diagonal entries are individual variances.
t-SNE	“That cluster plot”	A nonlinear method that maps high-dimensional data to 2D by preserving pairwise neighborhood probabilities. Good for visualization, not for preprocessing.
UMAP	“Faster t-SNE”	A nonlinear method based on topological data analysis. Preserves both local and some global structure. Scales better than t-SNE.
Perplexity	“A t-SNE knob”	Controls the effective number of neighbors each point considers. Low perplexity focuses on very local structure. High perplexity captures broader patterns.
Manifold	“The surface the data lives on”	A lower-dimensional surface embedded in a higher-dimensional space. A sheet of paper crumpled in 3D is a 2D manifold.

Further Reading

- A Tutorial on Principal Component Analysis (Shlens) - clear derivation of PCA from the

ground up

- How to Use t-SNE Effectively (Wattenberg et al.) - interactive guide to t-SNE pitfalls and parameter choices
- UMAP documentation - theory and practical guidance from the UMAP authors

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/10-dimensionality-reduction>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/10-dimensionality-reduction/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/10-dimensionality-reduction>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Singular Value Decomposition

SVD is the Swiss Army knife of linear algebra. Every matrix has one. Every data scientist needs one.

Type: Build **Languages:** Python, Julia **Prerequisites:** Phase 1, Lessons 01 (Linear Algebra Intuition), 02 (Vectors & Matrices Operations), 03 (Matrix Transformations) **Time:** ~120 minutes

Learning Objectives

- Implement SVD via power iteration and explain the geometric meaning of U, Sigma, and V^T
- Apply truncated SVD for image compression and measure the compression ratio vs reconstruction error
- Compute the Moore-Penrose pseudoinverse via SVD to solve overdetermined least-squares systems
- Connect SVD to PCA, recommendation systems (latent factors), and Latent Semantic Analysis in NLP

The Problem

You have a 1000x2000 matrix. Maybe it is user-movie ratings. Maybe it is a document-term frequency table. Maybe it is the pixel values of an image. You need to compress it, denoise it, find hidden structure in it, or solve a least-squares system with it. Eigendecomposition only works on square matrices. Even then, it requires the matrix to have a full set of linearly independent eigenvectors.

SVD works on any matrix. Any shape. Any rank. No conditions. It decomposes the matrix into three factors that reveal the geometry of what the matrix does to space. It is the most general and most useful factorization in all of linear algebra.

The Concept

What SVD does geometrically

Every matrix, regardless of shape, performs three operations in sequence: rotate, scale, rotate. SVD makes this decomposition explicit.

$$A = U * \text{Sigma} * V^T$$

$$\begin{array}{cccc} m \times n & m \times m & m \times n & n \times n \\ \text{(any)} & \text{(rotate)} & \text{(scale)} & \text{(rotate)} \end{array}$$

Given any matrix A, SVD factors it into: - V^T rotates vectors in the input space (n-dimensional) - Sigma scales along each axis (stretches or compresses) - U rotates the result into the output space (m-dimensional)

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/11-singular-value-decomposition>

Think of it this way. You hand SVD a matrix. It tells you: “This matrix takes a sphere of inputs, first rotates it by V^T , then stretches it into an ellipsoid by Sigma, then rotates the ellipsoid by U .” The singular values are the lengths of the ellipsoid’s axes.

The full decomposition

For a matrix A with shape $m \times n$:

$$A = U * \text{Sigma} * V^T$$

where:

U is $m \times m$, orthogonal ($U^T U = I$)
 Sigma is $m \times n$, diagonal (singular values on the diagonal)
 V is $n \times n$, orthogonal ($V^T V = I$)

The singular values $\text{sigma}_1 \geq \text{sigma}_2 \geq \dots \geq \text{sigma}_r > 0$
 where $r = \text{rank}(A)$

The columns of U are called left singular vectors. The columns of V are called right singular vectors. The diagonal entries of Sigma are called singular values. They are always non-negative and conventionally sorted in decreasing order.

Left singular vectors, singular values, right singular vectors

Each component of the SVD has a distinct geometric meaning.

Right singular vectors (columns of V): These form an orthonormal basis for the input space ($\mathbb{R}^n$). They are the directions in input space that the matrix maps to orthogonal directions in output space. Think of them as the natural coordinate system for the domain.

Singular values (diagonal of Sigma): These are the scaling factors. The i -th singular value tells you how much the matrix stretches vectors along the i -th right singular vector. A singular value of zero means the matrix crushes that direction entirely.

Left singular vectors (columns of U): These form an orthonormal basis for the output space ($\mathbb{R}^m$). The i -th left singular vector is the direction in output space where the i -th right singular vector lands (after scaling).

The relationship between them:

$$A * v_i = \text{sigma}_i * u_i$$

The matrix A takes the i -th right singular vector v_i , scales it by sigma_i , and maps it to the i -th left singular vector u_i .

This gives you a coordinate-by-coordinate picture of what any matrix does.

Outer product form

The SVD can be written as a sum of rank-1 matrices:

$$A = \text{sigma}_1 * u_1 * v_1^T + \text{sigma}_2 * u_2 * v_2^T + \dots + \text{sigma}_r * u_r * v_r^T$$

Each term $\text{sigma}_i * u_i * v_i^T$ is a rank-1 matrix (an outer product). The full matrix is the sum of r such matrices, where r is the rank.

This form is the foundation of low-rank approximation. Each term adds one layer of structure. The first term captures the single most important pattern. The second captures the next most important. And so on. Truncating this sum gives you the best possible approximation at any given rank.

Rank-1 approx: $A_1 = \sigma_1 * u_1 * v_1^T$
(captures the dominant pattern)

Rank-2 approx: $A_2 = \sigma_1 * u_1 * v_1^T + \sigma_2 * u_2 * v_2^T$
(captures the two most important patterns)

Rank-k approx: $A_k = \text{sum of top } k \text{ terms}$
(optimal by the Eckart-Young theorem)

Relationship to eigendecomposition

SVD and eigendecomposition are deeply connected. The singular values and vectors of A come directly from the eigenvalues and eigenvectors of $A^T A$ and $A A^T$.

$$\begin{aligned} A^T A &= V * \Sigma^T * U^T * U * \Sigma * V^T \\ &= V * \Sigma^T * \Sigma * V^T \\ &= V * D * V^T \end{aligned}$$

where $D = \Sigma^T * \Sigma$ is a diagonal matrix with σ_i^2 on the diagonal.

So:

- The right singular vectors (V) are eigenvectors of $A^T A$
- The singular values squared (σ_i^2) are eigenvalues of $A^T A$

Similarly:

$$\begin{aligned} A A^T &= U * \Sigma * V^T * V * \Sigma^T * U^T \\ &= U * \Sigma * \Sigma^T * U^T \end{aligned}$$

So:

- The left singular vectors (U) are eigenvectors of $A A^T$
- The eigenvalues of $A A^T$ are also σ_i^2

This connection tells you three things: 1. Singular values are always real and non-negative (they are square roots of eigenvalues of a positive semi-definite matrix). 2. You could compute SVD via eigendecomposition of $A^T A$, but this squares the condition number and loses numerical precision. Dedicated SVD algorithms avoid this. 3. When A is square and symmetric positive semi-definite, SVD and eigendecomposition are the same thing.

Truncated SVD: low-rank approximation

The Eckart-Young-Mirsky theorem states that the best rank- k approximation to A (in both Frobenius and spectral norm) is obtained by keeping only the top k singular values and their corresponding vectors:

$$A_k = U_k * \Sigma_k * V_k^T$$

where:

U_k is $m \times k$ (first k columns of U)
 Σ_k is $k \times k$ (top-left $k \times k$ block of Σ)
 V_k is $n \times k$ (first k columns of V)

Approximation error = σ_{k+1} (in spectral norm)
 = $\sqrt{\sigma_{k+1}^2 + \dots + \sigma_r^2}$ (in Frobenius norm)

This is not just “a good” approximation. It is provably the best possible approximation of rank k . No other rank- k matrix is closer to A .

Component	Relative magnitude	Kept in rank-3 approx?
σ_1	Largest	Yes
σ_2	Large	Yes
σ_3	Medium-large	Yes
σ_4	Medium	No (error)
σ_5	Medium-small	No (error)
σ_6	Small	No (error)
σ_7	Very small	No (error)
σ_8	Tiny	No (error)

Keep top 3: A_3 captures the three largest singular values. Error = remaining values (σ_4 through σ_8).

If singular values decay fast, a small k captures most of the matrix. If they decay slowly, the matrix has no low-rank structure.

Image compression with SVD

A grayscale image is a matrix of pixel intensities. An 800x600 image has 480,000 values. SVD lets you approximate it with far fewer.

Original image: $800 \times 600 = 480,000$ values

SVD with rank k :

U_k : 800 $\times$ k values
 Σ_k : k values
 V_k : 600 $\times$ k values
 Total: $k * (800 + 600 + 1) = k * 1401$ values

$k=10$: 14,010 values (2.9% of original)
 $k=50$: 70,050 values (14.6% of original)
 $k=100$: 140,100 values (29.2% of original)

The compression ratio improves as k gets smaller, but visual quality degrades.

The key insight: natural images have rapidly decaying singular values. The first few singular values capture the broad structure (shapes, gradients). The later ones capture fine detail and noise. Truncating at rank 50 often produces an image that looks nearly identical to the original while using 85% less storage.

SVD for recommendation systems

The Netflix Prize made this famous. You have a user-movie ratings matrix where most entries are missing.

	Movie1	Movie2	Movie3	Movie4	Movie5
User1	[5	?	3	?	1]
User2	[?	4	?	2	?]

User3	[	3	?	5	?	?	]
User4	[	?	?	?	4	3	]

? = unknown rating

The idea: this ratings matrix has low rank. Users do not have completely independent tastes. There are a handful of latent factors (action vs. drama, old vs. new, cerebral vs. visceral) that explain most preferences.

SVD on the (filled-in) ratings matrix decomposes it into: - U: user profiles in latent factor space - Sigma: importance of each latent factor - V^T : movie profiles in latent factor space

A user's predicted rating for a movie is the dot product of their user profile with the movie's profile (weighted by singular values). The low-rank approximation fills in the missing entries.

In practice, you use variants like Simon Funk's incremental SVD or ALS (alternating least squares) that handle missing data directly. But the core idea is the same: latent factor decomposition via SVD.

SVD in NLP: Latent Semantic Analysis

Latent Semantic Analysis (LSA), also called Latent Semantic Indexing (LSI), applies SVD to a term-document matrix.

	Doc1	Doc2	Doc3	Doc4
"cat"	[3	0	1	0]
"dog"	[2	0	0	1]
"fish"	[0	4	1	0]
"pet"	[1	1	1	1]
"ocean"	[0	3	0	0]

After SVD with rank $k=2$:

Each document becomes a point in 2D "concept space."

Each term becomes a point in the same 2D space.

Documents about similar topics cluster together.

Terms with similar meanings cluster together.

"cat" and "dog" end up near each other (land pets).

"fish" and "ocean" end up near each other (water concepts).

Doc1 and Doc3 cluster if they share similar topics.

LSA was one of the first successful methods for capturing semantic similarity from raw text. It works because synonymous terms tend to appear in similar documents, so SVD groups them into the same latent dimensions. Modern word embeddings (Word2Vec, GloVe) can be seen as descendants of this idea.

SVD for noise reduction

Noisy data has signal concentrated in the top singular values and noise spread across all singular values. Truncating removes the noise floor.

Clean signal singular values:

Component	Magnitude	Type
sigma_1	Very large	Signal

Component	Magnitude	Type
sigma_2	Large	Signal
sigma_3	Medium	Signal
sigma_4	Near zero	Negligible
sigma_5	Near zero	Negligible

Noisy signal singular values (noise adds to all):

Component	Magnitude	Type
sigma_1	Very large	Signal
sigma_2	Large	Signal
sigma_3	Medium	Signal
sigma_4	Small	Noise
sigma_5	Small	Noise
sigma_6	Small	Noise
sigma_7	Small	Noise

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/11-singular-value-decomposition>

This is used in signal processing, scientific measurement, and data cleaning. Any time you have a matrix corrupted by additive noise, truncated SVD is a principled way to separate signal from noise.

Pseudoinverse via SVD

The Moore-Penrose pseudoinverse A^+ generalizes matrix inversion to non-square and singular matrices. SVD makes computing it trivial.

If $A = U * \text{Sigma} * V^T$, then:

$$A^+ = V * \text{Sigma}^+ * U^T$$

where Sigma^+ is formed by:

1. Transpose Sigma (swap rows and columns)
2. Replace each non-zero diagonal entry sigma_i with $1/\text{sigma}_i$
3. Leave zeros as zeros

For A ($m \times n$): A^+ is ($n \times m$)

For Sigma ($m \times n$): Sigma^+ is ($n \times m$)

The pseudoinverse solves least-squares problems. If $Ax = b$ has no exact solution (overdetermined system), then $x = A^+ b$ is the least-squares solution (minimizes $\|Ax - b\|$).

Overdetermined system (more equations than unknowns):

$$\begin{bmatrix} 1 & 1 \\ 2 & 1 \\ 3 & 1 \end{bmatrix} x = \begin{bmatrix} 3 \\ 5 \\ 6 \end{bmatrix} \quad \text{No exact solution exists.}$$

$$x_{ls} = A^+ b = V * \text{Sigma}^+ * U^T * b$$

This gives the x that minimizes the sum of squared residuals.
 Same result as the normal equations $(A^T A)^{-1} A^T b$,
 but numerically more stable.

Numerical stability advantages

Computing eigendecomposition of $A^T A$ squares the singular values (eigenvalues of $A^T A$ are σ_i^2). This squares the condition number, amplifying numerical errors.

Example:

A has singular values [1000, 1, 0.001]
 Condition number of A: $1000 / 0.001 = 10^6$

$A^T A$ has eigenvalues [10^6 , 1, 10^{-6}]
 Condition number of $A^T A$: $10^6 / 10^{-6} = 10^{12}$

Computing SVD directly: works with condition number 10^6
 Computing via $A^T A$: works with condition number 10^{12}
 (6 extra digits of precision lost)

Modern SVD algorithms (Golub-Kahan bidiagonalization) work directly on A , never forming $A^T A$. This is why you should always prefer `np.linalg.svd(A)` over `np.linalg.eig(A.T @ A)`.

Connection to PCA

PCA IS SVD on centered data. This is not an analogy. It is literally the same computation.

Given data matrix X ($n_{\text{samples}} \times n_{\text{features}}$), centered (mean subtracted):

Covariance matrix: $C = (1/(n-1)) * X^T X$

PCA finds eigenvectors of C . But:

$$X = U * \text{Sigma} * V^T \quad (\text{SVD of } X)$$

$$X^T X = V * \text{Sigma}^2 * V^T$$

$$C = (1/(n-1)) * V * \text{Sigma}^2 * V^T$$

So the principal components are exactly the right singular vectors V .
 The explained variance for each component is $\sigma_i^2 / (n-1)$.

In sklearn, PCA is implemented using SVD, not eigendecomposition.
 It is faster and more numerically stable.

This means everything you learned about dimensionality reduction in Lesson 10 is SVD under the hood. PCA is the most common application of SVD in machine learning.

Interactive figure: svd-rank-reconstruction. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/11-singular-value-decomposition>

Build It

Step 1: SVD from scratch using power iteration

The idea: to find the largest singular value and its vectors, use power iteration on $A^T A$ (or $A A^T$). Then deflate the matrix and repeat for the next singular value.

```
import numpy as np

def power_iteration(M, num_iters=100):
    n = M.shape[1]
    v = np.random.randn(n)
    v = v / np.linalg.norm(v)

    for _ in range(num_iters):
        Mv = M @ v
        v = Mv / np.linalg.norm(Mv)

    eigenvalue = v @ M @ v
    return eigenvalue, v

def svd_from_scratch(A, k=None):
    m, n = A.shape
    if k is None:
        k = min(m, n)

    sigmas = []
    us = []
    vs = []

    A_residual = A.copy().astype(float)

    for _ in range(k):
        AtA = A_residual.T @ A_residual
        eigenvalue, v = power_iteration(AtA, num_iters=200)

        if eigenvalue < 1e-10:
            break

        sigma = np.sqrt(eigenvalue)
        u = A_residual @ v / sigma

        sigmas.append(sigma)
        us.append(u)
        vs.append(v)

        A_residual = A_residual - sigma * np.outer(u, v)

    U = np.column_stack(us) if us else np.empty((m, 0))
    S = np.array(sigmas)
    V = np.column_stack(vs) if vs else np.empty((n, 0))

    return U, S, V
```

Step 2: Test and compare with NumPy

```
np.random.seed(42)
A = np.random.randn(5, 4)

U_ours, S_ours, V_ours = svd_from_scratch(A)
U_np, S_np, Vt_np = np.linalg.svd(A, full_matrices=False)

print("Our singular values:", np.round(S_ours, 4))
print("NumPy singular values:", np.round(S_np, 4))

A_reconstructed = U_ours @ np.diag(S_ours) @ V_ours.T
print(f"Reconstruction error: {np.linalg.norm(A - A_reconstructed):.8f}")
```

Step 3: Image compression demo

```
def compress_image_svd(image_matrix, k):
    U, S, Vt = np.linalg.svd(image_matrix, full_matrices=False)
    compressed = U[:, :k] @ np.diag(S[:k]) @ Vt[:k, :]
    return compressed

image = np.random.seed(42)
rows, cols = 200, 300
image = np.random.randn(rows, cols)

for k in [1, 5, 10, 20, 50]:
    compressed = compress_image_svd(image, k)
    error = np.linalg.norm(image - compressed) / np.linalg.norm(image)
    original_size = rows * cols
    compressed_size = k * (rows + cols + 1)
    ratio = compressed_size / original_size
    print(f"k={k:>3d}  error={error:.4f}  storage={ratio:.1%}")
```

Step 4: Noise reduction

```
np.random.seed(42)
clean = np.outer(np.sin(np.linspace(0, 4*np.pi, 100)),
                 np.cos(np.linspace(0, 2*np.pi, 80)))
noise = 0.3 * np.random.randn(100, 80)
noisy = clean + noise

U, S, Vt = np.linalg.svd(noisy, full_matrices=False)
denoised = U[:, :5] @ np.diag(S[:5]) @ Vt[:5, :]

print(f"Noisy error:      {np.linalg.norm(noisy - clean):.4f}")
print(f"Denoised error: {np.linalg.norm(denoised - clean):.4f}")
print(f"Improvement:      {(1 - np.linalg.norm(denoised - clean) / np.linalg.norm(noisy - clean))
```

Step 5: Pseudoinverse

```
A = np.array([[1, 1], [2, 1], [3, 1]], dtype=float)
b = np.array([3, 5, 6], dtype=float)

U, S, Vt = np.linalg.svd(A, full_matrices=False)
```

```

S_inv = np.diag(1.0 / S)
A_pinv = Vt.T @ S_inv @ U.T

x_svd = A_pinv @ b
x_lstsq = np.linalg.lstsq(A, b, rcond=None)[0]
x_pinv = np.linalg.pinv(A) @ b

print(f"SVD pseudoinverse solution: {x_svd}")
print(f"np.linalg.lstsq solution: {x_lstsq}")
print(f"np.linalg.pinv solution: {x_pinv}")

```

Use It

Full working demos are in `code/svd.py`. Run it to see SVD applied to image compression, recommendation systems, latent semantic analysis, and noise reduction.

```
python svd.py
```

The Julia version in `code/svd.jl` demonstrates the same concepts using Julia's native `svd()` function and `LinearAlgebra` package.

```
julia svd.jl
```

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/11-singular-value-decomposition>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/11-singular-value-decomposition/code>

1. Implement the full SVD from scratch without using power iteration. Instead, compute the eigendecomposition of $A^T A$ to get V and the singular values, then compute $U = A V \Sigma^{-1}$. Compare numerical accuracy with your power iteration version and with NumPy.
2. Load a real grayscale image (or convert one to grayscale). Compress it at ranks 1, 5, 10, 25, 50, 100. For each rank, compute the compression ratio and the relative error. Find the rank where the image becomes visually acceptable.
3. Build a tiny recommendation system. Create a 10x8 user-movie ratings matrix with some known entries. Fill missing entries with row means. Compute SVD and reconstruct a rank-3 approximation. Use the reconstructed matrix to predict the missing ratings. Verify that the predictions are reasonable.
4. Create a 100x50 document-term matrix with 3 synthetic topics. Each topic has 5 associated terms. Add noise. Apply SVD and verify that the top 3 singular values are much larger than the rest. Project documents into the 3D latent space and check that documents from the same topic cluster together.
5. Generate a clean low-rank matrix (rank 3, size 50x40) and add Gaussian noise at different levels ($\sigma = 0.1, 0.5, 1.0, 2.0$). For each noise level, find the optimal truncation rank

by sweeping k from 1 to 40 and measuring reconstruction error against the clean matrix. Plot how the optimal k changes with noise level.

Key Terms

Term	What people say	What it actually means
SVD	"Factor any matrix"	Decompose A into $U \Sigma V^T$ where U and V are orthogonal and Σ is diagonal with non-negative entries. Works for any matrix of any shape.
Singular value	"How important this component is"	The i -th diagonal entry of Σ . Measures how much the matrix stretches along the i -th principal direction. Always non-negative, sorted in decreasing order.
Left singular vector	"Output direction"	A column of U . The direction in output space that the i -th right singular vector maps to (after scaling by σ_i).
Right singular vector	"Input direction"	A column of V . The direction in input space that the matrix maps to the i -th left singular vector (after scaling by σ_i).
Truncated SVD	"Low-rank approximation"	Keep only the top k singular values and their vectors. Produces the provably best rank- k approximation to the original matrix (Eckart-Young theorem).
Rank	"True dimensionality"	The number of non-zero singular values. Tells you how many independent directions the matrix actually uses.
Pseudoinverse	"Generalized inverse"	$V \Sigma^+ U^T$. Inverts non-zero singular values, leaves zeros as zeros. Solves least-squares problems for non-square or singular matrices.
Condition number	"How sensitive to errors"	$\sigma_{\max} / \sigma_{\min}$. A large condition number means small input changes cause large output changes. SVD reveals this directly.
Latent factor	"Hidden variable"	A dimension in the low-rank space discovered by SVD. In recommendations, a latent factor might correspond to genre preference. In NLP, it might correspond to a topic.
Frobenius norm	"Total matrix size"	Square root of the sum of squared entries. Equals the square root of the sum of squared singular values. Used to measure approximation error.
Eckart-Young theorem	"SVD gives the best compression"	For any target rank k , the truncated SVD minimizes the approximation error over all possible rank- k matrices.
Power iteration	"Find the biggest eigenvector"	Repeatedly multiply a random vector by the matrix and normalize. Converges to the eigenvector with the largest eigenvalue. The building block of many SVD algorithms.

Further Reading

- Gilbert Strang: Linear Algebra and Its Applications, Chapter 7 - thorough treatment of SVD with applications
- 3Blue1Brown: But what is the SVD? - geometric intuition for SVD
- We Recommend a Singular Value Decomposition - accessible overview from the American Mathematical Society
- Netflix Prize and Matrix Factorization - Simon Funk's original blog post on SVD for recommendations
- Latent Semantic Analysis - the original NLP application of SVD
- Numerical Linear Algebra by Trefethen and Bau - the gold standard for understanding SVD algorithms and their numerical properties

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/11-singular-value-decomposition>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/11-singular-value-decomposition/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/11-singular-value-decomposition>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Tensor Operations

Tensors are the common language between data and deep learning. Every image, every sentence, every gradient flows through them.

Type: Build **Language:** Python **Prerequisites:** Phase 1, Lessons 01 (Linear Algebra Intuition), 02 (Vectors, Matrices & Operations) **Time:** ~90 minutes

Learning Objectives

- Implement a tensor class with shape, strides, reshape, transpose, and element-wise operations from scratch
- Apply broadcasting rules to operate on tensors of different shapes without copying data
- Write einsum expressions for dot products, matrix multiplications, outer products, and batched operations
- Trace the exact tensor shapes through every step of multi-head attention

The Problem

You build a transformer. The forward pass looks clean. You run it and get: `RuntimeError: mat1 and mat2 shapes cannot be multiplied (32x768 and 512x768)`. You stare at the shapes. You try a transpose. Now it says `Expected 4D input (got 3D input)`. You add an `unsqueeze`. Something else breaks.

Shape errors are the most common bug in deep learning code. They are not hard conceptually – each operation has a shape contract – but they multiply fast. A transformer has dozens of reshapes, transposes, and broadcasts chained together. One wrong axis and the error cascades. Worse, some shape mistakes do not throw errors at all. They silently produce garbage by broadcasting along the wrong dimension or summing over the wrong axis.

Matrices handle pairwise relationships between two sets of things. Real data does not fit into two dimensions. A batch of 32 RGB images at 224x224 is a 4D tensor: (32, 3, 224, 224). Self-attention with 12 heads is also 4D: (batch, heads, seq_len, head_dim). You need a data structure that generalizes to any number of dimensions, with operations that compose cleanly across all of them. That structure is the tensor. Master its operations and shape errors become trivially debuggable.

The Concept

What a tensor is

A tensor is a multi-dimensional array of numbers with a uniform data type. The number of dimensions is the **rank** (or **order**). Each dimension is an **axis**. The **shape** is a tuple listing the size along each axis.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/12-tensor-operations>

Total elements = product of all sizes. A shape (2, 3, 4) holds $2 * 3 * 4 = 24$ elements.

Tensor shapes in deep learning

Different data types map to specific tensor shapes by convention.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/12-tensor-operations>

PyTorch uses NCHW (channels-first). TensorFlow defaults to NHWC (channels-last). Mismatched layouts cause silent slowdowns or errors.

How memory layout works

A 2D array in memory is a 1D sequence of bytes. **Strides** tell you how many elements to skip to move one step along each axis.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/12-tensor-operations>

Transpose does not move data. It swaps the strides, making the tensor **non-contiguous** - the elements for a row are no longer adjacent in memory.

Broadcasting rules

Broadcasting lets you operate on tensors of different shapes without copying data. Align shapes from the right. Two dimensions are compatible when they are equal or one is 1. Fewer dimensions get padded with 1s on the left.

```
Tensor A:      (8, 1, 6, 1)
Tensor B:      (7, 1, 5)
Padded B:      (1, 7, 1, 5)
Result:        (8, 7, 6, 5)
```

Einsum: the universal tensor operation

Einstein summation labels each axis with a letter. Axes in the input but not the output get summed. Axes in both are kept.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/12-tensor-operations>

Key patterns: $i, i \rightarrow$ (dot product), $i, j \rightarrow ij$ (outer product), $ii \rightarrow$ (trace), $ij \rightarrow ji$ (transpose), $bij, bjk \rightarrow bik$ (batch matmul), $bhtd, bhsd \rightarrow bhts$ (attention scores).

Interactive figure: tensor-broadcast. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/12-tensor-operations>

Build It

The code lives in `code/tensors.py`. Each step references the implementation there.

Step 1: Tensor storage and strides

A tensor stores a flat list of numbers plus shape metadata. Strides tell the indexing logic how to map multi-dimensional indices to flat positions.

```
class Tensor:
    def __init__(self, data, shape=None):
        if isinstance(data, (list, tuple)):
            self._data, self._shape = self._flatten_nested(data)
        elif isinstance(data, np.ndarray):
            self._data = data.flatten().tolist()
            self._shape = tuple(data.shape)
        else:
            self._data = [data]
            self._shape = ()

        if shape is not None:
            total = reduce(lambda a, b: a * b, shape, 1)
            if total != len(self._data):
                raise ValueError(
                    f"Cannot reshape {len(self._data)} elements into shape {shape}"
                )
            self._shape = tuple(shape)

        self._strides = self._compute_strides(self._shape)

    @staticmethod
    def _compute_strides(shape):
        if len(shape) == 0:
            return ()
        strides = [1] * len(shape)
        for i in range(len(shape) - 2, -1, -1):
            strides[i] = strides[i + 1] * shape[i + 1]
        return tuple(strides)
```

For shape (3, 4), strides are (4, 1) - skip 4 elements to advance one row, skip 1 element to advance one column.

Step 2: Reshape, squeeze, unsqueeze

Reshape changes the shape without changing element order. The total number of elements must stay the same. Use -1 for one dimension to infer its size.

```
t = Tensor(list(range(12)), shape=(2, 6))
r = t.reshape((3, 4))
r = t.reshape((-1, 3))
```

Squeeze removes axes of size 1. Unsqueeze inserts one. Unsquizing is critical for broadcasting - a bias vector (D,) added to a batch (B, T, D) needs unsquizing to (1, 1, D).

```
t = Tensor(list(range(6)), shape=(1, 3, 1, 2))
s = t.squeeze()
v = Tensor([1, 2, 3])
u = v.unsqueeze(0)
```

Step 3: Transpose and permute

Transpose swaps two axes. Permute reorders all axes. This is how you convert between NCHW and NHWC.

```
mat = Tensor(list(range(6)), shape=(2, 3))
tr = mat.transpose(0, 1)

t4d = Tensor(list(range(24)), shape=(1, 2, 3, 4))
perm = t4d.permute((0, 2, 3, 1))
```

After transpose or permute, the tensor is non-contiguous in memory. In PyTorch, view fails on non-contiguous tensors – use reshape or call .contiguous() first.

Step 4: Element-wise operations and reductions

Element-wise ops (add, multiply, subtract) apply independently to each element and preserve shape. Reductions (sum, mean, max) collapse one or more axes.

```
a = Tensor([[1, 2], [3, 4]])
b = Tensor([[10, 20], [30, 40]])
c = a + b
d = a * 2
s = a.sum(axis=0)
```

Global average pooling in a CNN: (B, C, H, W).mean(axis=[2, 3]) produces (B, C). Sequence mean pooling in NLP: (B, T, D).mean(axis=1) produces (B, D).

Step 5: Broadcasting with NumPy

The demo_broadcasting_numpy() function in tensors.py shows the core patterns.

```
activations = np.random.randn(4, 3)
bias = np.array([0.1, 0.2, 0.3])
result = activations + bias

images = np.random.randn(2, 3, 4, 4)
scale = np.array([0.5, 1.0, 1.5]).reshape(1, 3, 1, 1)
result = images * scale

a = np.array([1, 2, 3]).reshape(-1, 1)
b = np.array([10, 20, 30, 40]).reshape(1, -1)
outer = a * b
```

Pairwise distance via broadcasting: reshape (M, 2) to (M, 1, 2) and (N, 2) to (1, N, 2), subtract, square, sum along last axis, take square root. Result: (M, N).

Step 6: Einsum operations

The demo_einsum() and demo_einsum_gallery() functions walk through every common pattern.

```
a = np.array([1.0, 2.0, 3.0])
b = np.array([4.0, 5.0, 6.0])
dot = np.einsum("i,i->", a, b)

A = np.array([[1, 2], [3, 4], [5, 6]], dtype=float)
```

```
B = np.array([[7, 8, 9], [10, 11, 12]], dtype=float)
matmul = np.einsum("ik,kj->ij", A, B)
```

```
batch_A = np.random.randn(4, 3, 5)
batch_B = np.random.randn(4, 5, 2)
batch_mm = np.einsum("bij,bjk->bik", batch_A, batch_B)
```

The computational cost of a contraction is the product of all index sizes (kept and summed). For $bij, bjk \rightarrow bik$ with $B=32, I=128, J=64, K=128$: $32 * 128 * 64 * 128 = 33,554,432$ multiply-adds.

Step 7: Attention mechanism via einsum

The `demo_attention_einsum()` function implements multi-head attention end to end.

```
B, H, T, D = 2, 4, 8, 16
E = H * D
```

```
X = np.random.randn(B, T, E)
W_q = np.random.randn(E, E) * 0.02
```

```
Q = np.einsum("bte,ek->btk", X, W_q)
Q = Q.reshape(B, T, H, D).transpose(0, 2, 1, 3)
```

```
scores = np.einsum("bhtd,bhsd->bhts", Q, K) / np.sqrt(D)
weights = softmax(scores, axis=-1)
attn_output = np.einsum("bhts,bhsd->bhtd", weights, V)
```

```
concat = attn_output.transpose(0, 2, 1, 3).reshape(B, T, E)
output = np.einsum("bte,ek->btk", concat, W_o)
```

Every step is a tensor operation: projection (matmul via einsum), head splitting (reshape + transpose), attention scores (batch matmul via einsum), weighted sum (batch matmul via einsum), head merging (transpose + reshape), output projection (matmul via einsum).

Use It

Scratch vs NumPy

Operation	Scratch (Tensor class)	NumPy
Create	<code>Tensor([[1,2],[3,4]])</code>	<code>np.array([[1,2],[3,4]])</code>
Reshape	<code>t.reshape((3,4))</code>	<code>a.reshape(3,4)</code>
Transpose	<code>t.transpose(0,1)</code>	<code>a.T</code> or <code>a.transpose(0,1)</code>
Squeeze	<code>t.squeeze(0)</code>	<code>np.squeeze(a, 0)</code>
Sum	<code>t.sum(axis=0)</code>	<code>a.sum(axis=0)</code>
Einsum	N/A	<code>np.einsum("ij,jk->ik", a, b)</code>

Scratch vs PyTorch

```
import torch
```

```

t = torch.tensor([[1, 2, 3], [4, 5, 6]], dtype=torch.float32)
t.shape
t.stride()
t.is_contiguous()

t.reshape(3, 2)
t.unsqueeze(0)
t.transpose(0, 1)
t.transpose(0, 1).contiguous()

torch.einsum("ik,kj->ij", A, B)

```

PyTorch adds autograd, GPU support, and optimized BLAS kernels. The shape semantics are identical. If you understand the scratch version, PyTorch shape errors become readable.

Every neural network layer as a tensor operation

Operation	Tensor Form	Einsum
Linear layer	$Y = X @ W.T + b$	"bd,od->bo" + bias
Attention QKV	$Q = X @ W_q$	"btd,dh->bth"
Attention scores	$Q @ K.T / \sqrt{d}$	"bhtd,bhsd->bhts"
Attention output	$\text{softmax(scores)} @ V$	"bhts,bhsd->bhtd"
Batch norm	$(X - \mu) / \sigma * \gamma$	element-wise + broadcast
Softmax	$\exp(x) / \sum(\exp(x))$	element-wise + reduction

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/12-tensor-operations>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/12-tensor-operations/code>

1. **Easy - Reshape round-trip.** Take a tensor of shape (2, 3, 4). Reshape it to (6, 4), then to (24,), then back to (2, 3, 4). Verify element order is preserved at each step by printing the flat data.
2. **Medium - Implement broadcasting.** Extend the Tensor class with a `broadcast_to(shape)` method that expands dimensions of size 1 to match a target shape. Then modify `_elementwise_op` to automatically broadcast before operating. Test with shapes (3, 1) and (1, 4) producing (3, 4).
3. **Hard - Build einsum from scratch.** Implement a basic `einsum(subscripts, *tensors)` function that handles at least: dot product (i,i->), matrix multiply (ij,jk->ik), outer product (i,j->ij), and transpose (ij->ji). Parse the subscript string, identify contracted indices, and loop over all index combinations. Compare your results against `np.einsum`.
4. **Hard - Attention shape tracker.** Write a function that takes `batch_size`, `seq_len`, `embed_dim`, and `num_heads` as inputs and prints the exact shape at every step of

multi-head attention: input, Q/K/V projection, head split, attention scores, softmax weights, weighted sum, head merge, output projection. Verify against the `demo_attention_einsum()` output.

Key Terms

Term	What people say	What it actually means
Tensor	"A matrix but more dimensions"	A multi-dimensional array with uniform type and defined shape, strides, and operations
Rank	"The number of dimensions"	The number of axes. A matrix has rank 2, not rank equal to its matrix rank
Shape	"The size of the tensor"	A tuple listing the size along each axis. (2, 3) means 2 rows, 3 columns
Stride	"How memory is laid out"	The number of elements to skip to advance one position along each axis
Broadcasting	"It just works when shapes differ"	A strict set of rules: align from right, dimensions must be equal or one must be 1
Contiguous	"The tensor is normal"	Elements stored sequentially in memory with no gaps or reordering from the logical layout
Einsum	"A fancy way to write matmul"	A general notation that expresses any tensor contraction, outer product, trace, or transpose in one line
View	"Same as reshape"	A tensor sharing the same memory buffer but with different shape/stride metadata. Fails on non-contiguous data
Contraction	"Summing over an index"	The general operation where a shared index between tensors is multiplied and summed, producing a lower-rank result
NCHW / NHWC	"PyTorch vs TensorFlow format"	Memory layout conventions for image tensors. NCHW puts channels before spatial dims, NHWC puts them after

Further Reading

- NumPy Broadcasting - The canonical rules with visual examples

- PyTorch Tensor Views - When views work and when they copy
- einops - A library that makes tensor reshaping readable and safe
- The Illustrated Transformer - Visualizes the tensor shapes flowing through attention
- Einstein Summation in NumPy - Full einsum documentation with examples

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/12-tensor-operations>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/12-tensor-operations/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/12-tensor-operations>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Numerical Stability

Floating point is a leaky abstraction. It will bite you during training, and you will not see it coming.

Type: Build **Language:** Python **Prerequisites:** Phase 1, Lessons 01-04 **Time:** ~120 minutes

Learning Objectives

- Implement numerically stable softmax and log-sum-exp using the max-subtraction trick
- Identify overflow, underflow, and catastrophic cancellation in floating-point computations
- Verify analytical gradients against numerical gradients using centered finite differences
- Explain why bfloat16 is preferred over float16 for training and how loss scaling prevents gradient underflow

The Problem

Your model trains for three hours, then the loss becomes NaN. You add a print statement. The logits are fine at step 9,000. At step 9,001 they are inf. By step 9,002 every gradient is nan and training is dead.

Or: your model trains to completion but accuracy is 2% worse than the paper claims. You check everything. Architecture matches. Hyperparameters match. Data matches. The problem is that the paper used float32 and you used float16 without the right scaling. Thirty-two bits of accumulated rounding error quietly ate your accuracy.

Or: you implement cross-entropy loss from scratch. It works on small logits. When logits exceed 100, it returns inf. The softmax overflowed because $\exp(100)$ is larger than float32 can represent. Every ML framework handles this with a two-line trick. You did not know the trick existed.

Numerical stability is not a theoretical concern. It is the difference between a training run that succeeds and one that silently fails. Every serious ML bug you will debug eventually comes down to floating point.

The Concept

IEEE 754: How Computers Store Real Numbers

Computers store real numbers as floating point values following the IEEE 754 standard. A float has three parts: a sign bit, an exponent, and a mantissa (significand).

Float32 layout (32 bits total):
[1 sign] [8 exponent] [23 mantissa]

Value = $(-1)^{\text{sign}} * 2^{(\text{exponent} - 127)} * 1.\text{mantissa}$

The mantissa determines precision (how many significant digits). The exponent determines range (how large or small a number can be).

Format	Bits	Exponent	Mantissa	Decimal digits	Range (approx)
float64	64	11	52	~15-16	+/- 1.8e308
float32	32	8	23	~7-8	+/- 3.4e38
float16	16	5	10	~3-4	+/- 65,504
bfloat16	16	8	7	~2-3	+/- 3.4e38

float32 gives you about 7 decimal digits of precision. That means it can tell apart 1.0000001 and 1.0000002, but not 1.00000001 and 1.00000002. After 7 digits, everything is rounding noise.

float16 gives you about 3 digits. The largest number it can represent is 65,504. That is disturbingly small for ML where logits, gradients, and activations routinely exceed this.

bfloat16 is Google's answer to float16's range problem. It has the same 8-bit exponent as float32 (same range, up to 3.4e38) but only 7 mantissa bits (less precision than float16). For training neural networks, range matters more than precision, so bfloat16 usually wins.

Why $0.1 + 0.2 \neq 0.3$

The number 0.1 cannot be represented exactly in binary floating point. In base 2, it is a repeating fraction:

0.1 in binary = 0.0001100110011001100110011... (repeating forever)

Float32 truncates this to 23 bits of mantissa. The stored value is approximately 0.100000001490116. Similarly, 0.2 is stored as approximately 0.200000002980232. Their sum is 0.300000004470348, not 0.3.

In Python:

```
>>> 0.1 + 0.2
0.30000000000000004
```

```
>>> 0.1 + 0.2 == 0.3
False
```

This matters for ML because:

1. Loss comparisons like `if loss < threshold` can give wrong answers
2. Accumulating many small values (gradient updates over thousands of steps) drifts from the true sum
3. Checksums and reproducibility tests fail if you compare floats with `==`

The fix: never compare floats with `==`. Use `abs(a - b) < epsilon` or `math.isclose()`.

Catastrophic Cancellation

When you subtract two nearly equal floating point numbers, the significant digits cancel and you are left with rounding noise promoted to leading digits.

```
a = 1.0000001      (stored as 1.00000011920929 in float32)
b = 1.0000000      (stored as 1.00000000000000 in float32)
```

```
True difference: 0.0000001
Computed:       0.00000011920929
```

Relative error: 19.2%

That is a 19% relative error from a single subtraction. In ML, this happens whenever you:

- Compute variance of data with a large mean: $E[x^2] - E[x]^2$ when $E[x]$ is large
- Subtract nearly equal log-probabilities
- Compute finite-difference gradients with too-small epsilon

The fix: rearrange formulas to avoid subtracting large, nearly equal numbers. For variance, use the Welford algorithm or center the data first. For log-probabilities, work in log-space throughout.

Overflow and Underflow

Overflow happens when a result is too large to represent. Underflow happens when it is too small (closer to zero than the smallest representable positive number).

Float32 boundaries:

```
Maximum: 3.4028235e+38
Minimum positive (normal): 1.175e-38
Minimum positive (denorm): 1.401e-45
Overflow: anything > 3.4e38 becomes inf
Underflow: anything < 1.4e-45 becomes 0.0
```

The `exp()` function is the primary source of overflow in ML:

```
exp(88.7) = 3.40e+38    (barely fits in float32)
exp(89.0) = inf         (overflow)
exp(-87.3) = 1.18e-38   (barely above underflow)
exp(-104) = 0.0         (underflow to zero)
```

The `log()` function hits the other direction:

```
log(0.0) = -inf
log(-1.0) = nan
log(1e-45) = -103.3      (fine)
log(1e-46) = -inf        (input underflowed to 0, then log(0) = -inf)
```

In ML, `exp()` appears in softmax, sigmoid, and probability computations. `log()` appears in cross-entropy, log-likelihoods, and KL divergence. The combination `log(exp(x))` is a mine-field without the right tricks.

The Log-Sum-Exp Trick

Computing `log(sum(exp(x_i)))` directly is numerically dangerous. If any x_i is large, `exp(x_i)` overflows. If all x_i are very negative, every `exp(x_i)` underflows to zero and `log(0)` is `-inf`.

The trick: subtract the maximum value before exponentiating.

```
log(sum(exp(x_i))) = max(x) + log(sum(exp(x_i - max(x))))
```

Why this works: after subtracting `max(x)`, the largest exponent is `exp(0) = 1`. No overflow is possible. At least one term in the sum is 1, so the sum is at least 1, and `log(1) = 0`. No underflow to `-inf` is possible.

Proof:

```
log(sum(exp(x_i)))
= log(sum(exp(x_i - c + c)))           (add and subtract c)
= log(sum(exp(x_i - c) * exp(c)))      (exp(a+b) = exp(a)*exp(b))
= log(exp(c) * sum(exp(x_i - c)))      (factor out exp(c))
```

$= c + \log(\sum(\exp(x_i - c)))$ $(\log(a*b) = \log(a) + \log(b))$

Set $c = \max(x)$ and overflow is eliminated.

This trick appears everywhere in ML: - Softmax normalization - Cross-entropy loss computation - Log-probability summation in sequence models - Mixture of Gaussians - Variational inference

Why Softmax Needs the Max-Subtraction Trick

Softmax converts logits to probabilities:

$\text{softmax}(x_i) = \exp(x_i) / \sum(\exp(x_j))$

Without the trick, logits of [100, 101, 102] cause overflow:

```
exp(100) = 2.69e43
exp(101) = 7.31e43
exp(102) = 1.99e44
sum      = 2.99e44
```

These overflow float32 (max ~3.4e38)? No, $2.69e43 < 3.4e38$? Actually:
 $\exp(88.7)$ is already at the float32 limit.
 $\exp(100) = \text{inf}$ in float32.

With the trick, subtract $\max(x) = 102$:

```
exp(100 - 102) = exp(-2) = 0.135
exp(101 - 102) = exp(-1) = 0.368
exp(102 - 102) = exp(0)  = 1.000
sum = 1.503
```

$\text{softmax} = [0.090, 0.245, 0.665]$

The probabilities are identical. The computation is safe. This is not an optimization. It is a requirement for correctness.

NaN and Inf: Detection and Prevention

nan (Not a Number) and inf (infinity) propagate virally through computation. One nan in a gradient update makes the weight nan, which makes every subsequent output nan. Training is dead within one step.

How inf appears: - $\exp()$ of a large positive number - Division by zero: $1.0 / 0.0$ - float32 overflow in accumulations

How nan appears: - $0.0 / 0.0$ - $\text{inf} - \text{inf} - \text{inf} * 0$ - $\text{sqrt}()$ of a negative number - $\log()$ of a negative number - Any arithmetic involving an existing nan

Detection:

```
import math
```

```
math.isnan(x)      # True if x is nan
math.isinf(x)      # True if x is +inf or -inf
math.isfinite(x)   # True if x is neither nan nor inf
```

Prevention strategies:

1. Clamp inputs to $\exp()$: $\exp(\text{clamp}(x, -80, 80))$

2. Add epsilon to denominators: $x / (y + 1e-8)$
3. Add epsilon inside $\log()$: $\log(x + 1e-8)$
4. Use stable implementations (log-sum-exp, stable softmax)
5. Gradient clipping to prevent weight explosion
6. Check for nan/inf after every forward pass during debugging

Numerical Gradient Checking

Analytical gradients (from backpropagation) can have bugs. Numerical gradient checking verifies them by computing gradients with finite differences.

The centered difference formula:

$$df/dx \approx (f(x + h) - f(x - h)) / (2h)$$

This is $O(h^2)$ accurate, much better than the forward difference $(f(x+h) - f(x)) / h$ which is only $O(h)$.

Choosing h : too large and the approximation is wrong. Too small and catastrophic cancellation destroys the answer. $h = 1e-5$ to $1e-7$ is typical.

The check: compute the relative difference between analytical and numerical gradients.

```
relative_error = |grad_analytical - grad_numerical| / max(|grad_analytical|, |grad_numerical|, 1e-8)
```

Rules of thumb: - $\text{relative_error} < 1e-7$: perfect, gradient is correct - $\text{relative_error} < 1e-5$: acceptable, probably correct - $\text{relative_error} > 1e-3$: something is wrong - $\text{relative_error} > 1$: gradient is completely wrong

Always check gradients when implementing a new layer or loss function. PyTorch provides `torch.autograd.gradcheck()` for this.

Mixed Precision Training

Modern GPUs have specialized hardware (Tensor Cores) that compute float16 matrix multiplications 2-8x faster than float32. Mixed precision training exploits this:

1. Maintain float32 master copy of weights
2. Forward pass in float16 (fast)
3. Compute loss in float32 (prevents overflow)
4. Backward pass in float16 (fast)
5. Scale gradients to float32
6. Update float32 master weights

The problem with pure float16 training: gradients are often very small ($1e-8$ or smaller). Float16 underflows anything below $\sim 6e-8$ to zero. Your model stops learning because all gradient updates are zero.

The fix is loss scaling:

1. Multiply loss by a large scale factor (e.g., 1024)
2. Backward pass computes gradients of $(\text{loss} * 1024)$
3. All gradients are 1024x larger (pushed above float16 underflow)
4. Divide gradients by 1024 before updating weights
5. Net effect: same update, but no underflow

Dynamic loss scaling adjusts the scale factor automatically. Start with a large value (65536). If gradients overflow to inf, halve it. If N steps pass without overflow, double it.

bfloat16 vs float16: Why bfloat16 Wins for Training

float16: [1 sign] [5 exponent] [10 mantissa]
 bfloat16: [1 sign] [8 exponent] [7 mantissa]

float16 has more precision (10 mantissa bits vs 7) but limited range (max ~65,504). bfloat16 has less precision but the same range as float32 (max ~3.4e38).

For training neural networks:

- Activations and logits regularly exceed 65,504 during training spikes. float16 overflows; bfloat16 handles it.
- Loss scaling is required with float16 but usually unnecessary with bfloat16 because its range covers the gradient magnitude spectrum.
- bfloat16 is a simple truncation of float32: drop the bottom 16 bits of the mantissa. Conversion is trivial and lossless in the exponent.

float16 is preferred for inference where values are bounded and precision matters more. bfloat16 is preferred for training where range matters more. This is why TPUs and modern NVIDIA GPUs (A100, H100) have native bfloat16 support.

Gradient Clipping

Exploding gradients happen when gradients grow exponentially through many layers (common in RNNs, deep networks, and transformers). A single large gradient can corrupt all weights in one step.

Two types of clipping:

Clip by value: clamp each gradient element independently.

```
grad = clamp(grad, -max_val, max_val)
```

Simple but can change the direction of the gradient vector.

Clip by norm: scale the entire gradient vector so its norm does not exceed a threshold.

```
if ||grad|| > max_norm:
    grad = grad * (max_norm / ||grad||)
```

Preserves the direction of the gradient. This is what `torch.nn.utils.clip_grad_norm_()` does. It is the standard choice.

Typical values: `max_norm=1.0` for transformers, `max_norm=0.5` for RL, `max_norm=5.0` for simpler networks.

Gradient clipping is not a hack. It is a safety mechanism. Without it, a single outlier batch can produce a gradient large enough to ruin weeks of training.

Normalization Layers as Numerical Stabilizers

Batch normalization, layer normalization, and RMS normalization are usually presented as regularizers that help training converge. They are also numerical stabilizers.

Without normalization, activations can grow or shrink exponentially through layers:

Layer 1: values in [0, 1]
 Layer 5: values in [0, 100]
 Layer 10: values in [0, 10,000]
 Layer 50: values in [0, inf]

Normalization recenters and rescales activations at every layer:

$$\text{LayerNorm}(x) = (x - \text{mean}(x)) / (\text{std}(x) + \text{epsilon}) * \text{gamma} + \text{beta}$$

The `epsilon` (typically $1e-5$) prevents division by zero when all activations are identical. The learned parameters `gamma` and `beta` let the network restore any scale it needs.

This keeps values in a numerically safe range throughout the network, preventing both overflow in the forward pass and gradient explosion in the backward pass.

Common ML Numerical Bugs

Bug: Loss is NaN after a few epochs. Cause: logits grew too large, softmax overflowed. Or learning rate is too high and weights diverged. Fix: use stable softmax (max subtraction), reduce learning rate, add gradient clipping.

Bug: Loss is stuck at $\log(\text{num_classes})$. Cause: model outputs are near-uniform probabilities. Often means gradients are vanishing or the model is not learning at all. Fix: check that data labels are correct, verify the loss function, check for dead ReLUs.

Bug: Validation accuracy is lower than expected by 1-3%. Cause: mixed precision without proper loss scaling. Gradient underflow silently zeroes out small updates. Fix: enable dynamic loss scaling, or switch to `bfloat16`.

Bug: Gradient norms are 0.0 for some layers. Cause: dead ReLU neurons (all inputs negative), or `float16` underflow. Fix: use LeakyReLU or GELU, use gradient scaling, check weight initialization.

Bug: Model works on one GPU but gives different results on another. Cause: non-deterministic floating point accumulation order. GPU parallel reductions sum in different orders on different hardware, and floating point addition is not associative. Fix: accept small differences ($1e-6$), or set `torch.use_deterministic_algorithms(True)` and accept the speed penalty.

Bug: `exp()` returns `inf` in loss computation. Cause: raw logits passed to `exp()` without the max-subtraction trick. Fix: use `torch.nn.functional.log_softmax()` which implements log-sum-exp internally.

Bug: Training diverges after switching from `float32` to `float16`. Cause: `float16` cannot represent gradient magnitudes below $6e-8$ or activations above 65,504. Fix: use mixed precision with loss scaling (AMP), or use `bfloat16` instead.

Interactive figure: logsumexp-stability. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/13-numerical-stability>

Build It

Step 1: Demonstrate floating point precision limits

```
print("=== Floating Point Precision ===")
print(f"0.1 + 0.2 = {0.1 + 0.2}")
print(f"0.1 + 0.2 == 0.3? {0.1 + 0.2 == 0.3}")
print(f"Difference: {(0.1 + 0.2) - 0.3:.2e}")
```

Step 2: Implement naive vs stable softmax

```
import math
```

```

def softmax_naive(logits):
    exps = [math.exp(z) for z in logits]
    total = sum(exps)
    return [e / total for e in exps]

def softmax_stable(logits):
    max_logit = max(logits)
    exps = [math.exp(z - max_logit) for z in logits]
    total = sum(exps)
    return [e / total for e in exps]

safe_logits = [2.0, 1.0, 0.1]
print(f"Naive: {softmax_naive(safe_logits)}")
print(f"Stable: {softmax_stable(safe_logits)}")

dangerous_logits = [100.0, 101.0, 102.0]
print(f"Stable: {softmax_stable(dangerous_logits)}")
# softmax_naive(dangerous_logits) would return [nan, nan, nan]

```

Step 3: Implement stable log-sum-exp

```

def logsumexp_naive(values):
    return math.log(sum(math.exp(v) for v in values))

def logsumexp_stable(values):
    c = max(values)
    return c + math.log(sum(math.exp(v - c) for v in values))

safe = [1.0, 2.0, 3.0]
print(f"Naive: {logsumexp_naive(safe):.6f}")
print(f"Stable: {logsumexp_stable(safe):.6f}")

large = [500.0, 501.0, 502.0]
print(f"Stable: {logsumexp_stable(large):.6f}")
# logsumexp_naive(large) returns inf

```

Step 4: Implement stable cross-entropy

```

def cross_entropy_naive(true_class, logits):
    probs = softmax_naive(logits)
    return -math.log(probs[true_class])

def cross_entropy_stable(true_class, logits):
    max_logit = max(logits)
    shifted = [z - max_logit for z in logits]
    log_sum_exp = math.log(sum(math.exp(s) for s in shifted))
    log_prob = shifted[true_class] - log_sum_exp
    return -log_prob

logits = [2.0, 5.0, 1.0]
true_class = 1
print(f"Naive: {cross_entropy_naive(true_class, logits):.6f}")
print(f"Stable: {cross_entropy_stable(true_class, logits):.6f}")

```


Step 5: Gradient checking

```
def numerical_gradient(f, x, h=1e-5):
    grad = []
    for i in range(len(x)):
        x_plus = x[:]
        x_minus = x[:]
        x_plus[i] += h
        x_minus[i] -= h
        grad.append((f(x_plus) - f(x_minus)) / (2 * h))
    return grad

def check_gradient(analytical, numerical, tolerance=1e-5):
    for i, (a, n) in enumerate(zip(analytical, numerical)):
        denom = max(abs(a), abs(n), 1e-8)
        rel_error = abs(a - n) / denom
        status = "OK" if rel_error < tolerance else "FAIL"
        print(f" param {i}: analytical={a:.8f} numerical={n:.8f} "
              f"rel_error={rel_error:.2e} [{status}]")

def f(params):
    x, y = params
    return x**2 + 3*x*y + y**3

def f_grad(params):
    x, y = params
    return [2*x + 3*y, 3*x + 3*y**2]

point = [2.0, 1.0]
analytical = f_grad(point)
numerical = numerical_gradient(f, point)
check_gradient(analytical, numerical)
```

Use It

Mixed precision simulation

```
import struct

def float32_to_float16_round(x):
    packed = struct.pack('f', x)
    f32 = struct.unpack('f', packed)[0]
    packed16 = struct.pack('e', f32)
    return struct.unpack('e', packed16)[0]

def simulate_bfloat16(x):
    packed = struct.pack('f', x)
    as_int = int.from_bytes(packed, 'little')
    truncated = as_int & 0xFFFF0000
    repacked = truncated.to_bytes(4, 'little')
    return struct.unpack('f', repacked)[0]
```

Gradient clipping

```
def clip_by_norm(gradients, max_norm):
    total_norm = math.sqrt(sum(g**2 for g in gradients))
    if total_norm > max_norm:
        scale = max_norm / total_norm
        return [g * scale for g in gradients]
    return gradients

grads = [10.0, 20.0, 30.0]
clipped = clip_by_norm(grads, max_norm=5.0)
print(f"Original norm: {math.sqrt(sum(g**2 for g in grads)):.2f}")
print(f"Clipped norm: {math.sqrt(sum(g**2 for g in clipped)):.2f}")
print(f"Direction preserved: {[c/clipped[0] for c in clipped]} == {[g/grads[0] for g in grads]}")
```

NaN/Inf detection

```
def check_tensor(name, values):
    has_nan = any(math.isnan(v) for v in values)
    has_inf = any(math.isinf(v) for v in values)
    if has_nan or has_inf:
        print(f"WARNING {name}: nan={has_nan} inf={has_inf}")
        return False
    return True
```

```
check_tensor("good", [1.0, 2.0, 3.0])
check_tensor("bad", [1.0, float('nan'), 3.0])
check_tensor("ugly", [1.0, float('inf'), 3.0])
```

See code/numerical.py for complete implementations with all edge cases demonstrated.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/13-numerical-stability>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/13-numerical-stability/code>

1. **Catastrophic cancellation.** Compute the variance of [1000000.0, 1000001.0, 1000002.0] using the naive formula $E[x^2] - E[x]^2$ in float32. Then compute it using Welford's online algorithm. Compare the errors against the true variance (0.6667).
2. **Precision hunt.** Find the smallest positive float32 value x such that $1.0 + x == 1.0$ in Python. This is the machine epsilon. Verify it matches `numpy.finfo(numpy.float32).eps`.
3. **Log-sum-exp edge cases.** Test your `logsumexp_stable` function with: (a) all values equal, (b) one value much larger than the rest, (c) all values very negative (-1000). Verify it gives correct results where the naive version fails.
4. **Gradient checking a neural network layer.** Implement a single linear layer $y = Wx + b$ and its analytical backward pass. Use `numerical_gradient` to verify correctness for a 3x2 weight matrix.

5. **Loss scaling experiment.** Simulate training with float16: create random gradients in the range $[1e-9, 1e-3]$, convert to float16, and measure what fraction become zero. Then apply loss scaling (multiply by 1024), convert to float16, scale back, and measure the zero fraction again.

Key Terms

Term	What people say	What it actually means
IEEE 754	"The float standard"	International standard defining binary floating point formats, rounding rules, and special values (inf, nan). Every modern CPU and GPU implements it.
Machine epsilon	"The precision limit"	The smallest value e such that $1.0 + e \neq 1.0$ in a given float format. For float32, it is about $1.19e-7$.
Catastrophic cancellation	"Precision loss from subtraction"	When subtracting nearly equal floating point numbers, significant digits cancel and rounding noise dominates the result.
Overflow	"Number too big"	A result exceeds the maximum representable value and becomes inf. $\exp(89)$ overflows float32.
Underflow	"Number too small"	A result is closer to zero than the smallest representable positive number and becomes 0.0. $\exp(-104)$ underflows float32.
Log-sum-exp trick	"Subtract the max first"	Computing $\log(\sum(\exp(x)))$ by factoring out $\exp(\max(x))$ to prevent overflow and underflow. Used in softmax, cross-entropy, and log-probability math.
Stable softmax	"Softmax that does not explode"	Subtracting $\max(\text{logits})$ before exponentiating. Numerically identical result, no overflow possible.
Gradient checking	"Verify your backprop"	Comparing analytical gradients from backpropagation against numerical gradients from finite differences to catch implementation bugs.
Mixed precision	"Float16 forward, float32 backward"	Using lower-precision floats for speed-critical operations and higher-precision floats for numerically sensitive operations. Typical speedup is 2-3x.
Loss scaling	"Prevent gradient underflow"	Multiplying the loss by a large constant before backprop so gradients stay in float16's representable range, then dividing by the same constant before weight updates.
bfloat16	"Brain floating point"	Google's 16-bit format with 8 exponent bits (same range as float32) and 7 mantissa bits (less precision than float16). Preferred for training.

Term	What people say	What it actually means
Gradient clipping	“Cap the gradient norm”	Scaling the gradient vector so its norm does not exceed a threshold. Prevents exploding gradients from ruining weights.
NaN	“Not a Number”	Special float value from undefined operations (0/0, inf-inf, sqrt(-1)). Propagates through all subsequent arithmetic.
Inf	“Infinity”	Special float value from overflow or division by zero. Can combine to produce NaN (inf - inf, inf * 0).
Numerical gradient	“Brute force derivative”	Approximating a derivative by evaluating $f(x+h)$ and $f(x-h)$ and dividing by $2h$. Slow but reliable for verification.

Further Reading

- What Every Computer Scientist Should Know About Floating-Point Arithmetic (Goldberg 1991) – the definitive reference, dense but complete
- Mixed Precision Training (Micikevicius et al., 2018) – the NVIDIA paper that introduced loss scaling for float16 training
- AMP: Automatic Mixed Precision (PyTorch docs) – practical guide to mixed precision in PyTorch
- bfloat16 format (Google Cloud TPU docs) – why Google chose this format for TPUs
- Kahan Summation (Wikipedia) – algorithm for reducing rounding error in floating point sums

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/13-numerical-stability>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/13-numerical-stability/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/13-numerical-stability>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Norms and Distances

Your distance function defines what “similar” means. Choose wrong and everything downstream breaks.

Type: Build **Language:** Python **Prerequisites:** Phase 1, Lessons 01 (Linear Algebra Intuition), 02 (Vectors, Matrices & Operations) **Time:** ~90 minutes

Learning Objectives

- Implement L1, L2, cosine, Mahalanobis, Jaccard, and edit distance functions from scratch
- Select the appropriate distance metric for a given ML task and explain why alternatives fail
- Connect L1 and L2 norms to LASSO and Ridge regularization and their geometric constraint regions
- Demonstrate how the same dataset produces different nearest neighbors under different metrics

The Problem

You have two vectors. Maybe they are word embeddings. Maybe they are user profiles. Maybe they are pixel arrays. You need to know: how close are they?

The answer depends entirely on which distance function you pick. Two data points can be nearest neighbors under one metric and far apart under another. Your KNN classifier, your recommendation engine, your vector database, your clustering algorithm, your loss function – they all depend on this choice. Get it wrong and your model optimizes for the wrong thing.

There is no universal best distance. L2 works for spatial data. Cosine similarity dominates NLP. Jaccard handles sets. Edit distance handles strings. Mahalanobis accounts for correlations. Wasserstein moves probability mass. Each one encodes a different assumption about what “similar” means.

This lesson builds every major distance function from scratch, shows you when each one is the right tool, and demonstrates how the same data produces completely different nearest neighbors depending on which metric you use.

The Concept

Norms: measuring vector magnitude

A norm measures the “size” of a vector. Every distance function between two vectors can be written as the norm of their difference: $d(a, b) = \|a - b\|$. So understanding norms is understanding distances.

L1 Norm (Manhattan distance)

The L1 norm sums the absolute values of all components.

$$||x||_1 = |x_1| + |x_2| + \dots + |x_n|$$

It is called Manhattan distance because it measures how far you walk on a city grid where you can only move along axes. No diagonals.

Point A = (1, 1)

Point B = (4, 5)

$$\text{L1 distance} = |4-1| + |5-1| = 3 + 4 = 7$$

On a grid, you walk 3 blocks east and 4 blocks north.

When to use L1: - High-dimensional sparse data (text features, one-hot encodings) - When you want robustness to outliers (a single huge difference does not dominate) - Feature selection problems (L1 regularization promotes sparsity)

Connection to L1 regularization (Lasso): adding $||w||_1$ to your loss function penalizes the sum of absolute weight values. This pushes small weights to exactly zero, performing automatic feature selection. The L1 penalty creates diamond-shaped constraint regions in weight space, and the corners of diamonds lie on the axes where some weights are zero.

Connection to loss functions: Mean Absolute Error (MAE) is the average L1 distance between predictions and targets. It penalizes all errors linearly, making it robust to outliers compared to MSE.

L2 Norm (Euclidean distance)

The L2 norm is the straight-line distance. Square root of the sum of squared components.

$$||x||_2 = \sqrt{x_1^2 + x_2^2 + \dots + x_n^2}$$

This is the distance you learned in geometry class. Pythagoras in n dimensions.

Point A = (1, 1)

Point B = (4, 5)

$$\text{L2 distance} = \sqrt{(4-1)^2 + (5-1)^2} = \sqrt{9 + 16} = \sqrt{25} = 5.0$$

The straight line, cutting diagonally through the grid.

When to use L2: - Low-to-medium dimensional continuous data - When the feature scales are comparable - Physical distances (spatial data, sensor readings) - Image similarity at the pixel level

Connection to L2 regularization (Ridge): adding $||w||_2^2$ to your loss function penalizes large weights. Unlike L1, it does not push weights to zero. It shrinks all weights toward zero proportionally. The L2 penalty creates circular constraint regions, so there are no corners on axes. Weights get small but rarely exactly zero.

Connection to loss functions: Mean Squared Error (MSE) is the average of L2 distances squared. Squaring penalizes large errors more heavily than small ones.

MAE (L1 loss): $|y - \hat{y}|$

Linear penalty. Robust to outliers.

MSE (L2 loss): $(y - \hat{y})^2$

Quadratic penalty. Sensitive to outliers.

Lp Norms: the general family

L1 and L2 are special cases of the Lp norm:

$$||x||_p = (|x_1|^p + |x_2|^p + \dots + |x_n|^p)^{1/p}$$

Different values of p produce different shaped “unit balls” (the set of all points at distance 1 from the origin):

p=1:	Diamond shape	(corners on axes)
p=2:	Circle/sphere	(the usual round ball)
p=3:	Superellipse	(rounded square)
p=inf:	Square/hypercube	(flat sides along axes)

L-infinity Norm (Chebyshev distance)

As p approaches infinity, the Lp norm converges to the maximum absolute component.

$$||x||_{\infty} = \max(|x_1|, |x_2|, \dots, |x_n|)$$

The distance between two points is determined by the single dimension where they differ the most. All other dimensions are ignored.

Point A = (1, 1)

Point B = (4, 5)

$$L\text{-inf distance} = \max(|4-1|, |5-1|) = \max(3, 4) = 4$$

When to use L-infinity: - When the worst-case deviation in any single dimension matters - Game boards (a king in chess moves in L-infinity: one step in any direction costs 1) - Manufacturing tolerances (every dimension must be within spec)

Cosine Similarity and Cosine Distance

Cosine similarity measures the angle between two vectors, ignoring their magnitudes.

$$\text{cos_sim}(a, b) = (a \cdot b) / (||a||_2 * ||b||_2)$$

It ranges from -1 (opposite directions) to +1 (same direction). Perpendicular vectors have cosine similarity 0.

Cosine distance converts it to a distance: $\text{cosine_distance} = 1 - \text{cosine_similarity}$. This ranges from 0 (identical direction) to 2 (opposite direction).

$$a = (1, 0) \quad b = (1, 1)$$

$$\begin{aligned} \text{cos_sim} &= (1*1 + 0*1) / (1 * \sqrt{2}) = 1/\sqrt{2} = 0.707 \\ \text{cos_dist} &= 1 - 0.707 = 0.293 \end{aligned}$$

Why cosine dominates NLP and embeddings: in text, document length should not affect similarity. A document about cats that is twice as long as another document about cats should still be “similar.” Cosine similarity ignores magnitude (length) and only cares about direction. Two documents with the same word distribution but different lengths point in the same direction and get cosine similarity 1.0.

When to use cosine similarity: - Text similarity (TF-IDF vectors, word embeddings, sentence embeddings) - Any domain where magnitude is noise and direction is signal - Recommendation systems (user preference vectors) - Embedding search (vector databases almost always use cosine or dot product)

Dot Product Similarity vs Cosine Similarity

The dot product of two vectors is:

$$\begin{aligned} a \cdot b &= a_1 \cdot b_1 + a_2 \cdot b_2 + \dots + a_n \cdot b_n \\ &= ||a|| \cdot ||b|| \cdot \cos(\text{angle}) \end{aligned}$$

Cosine similarity is the dot product normalized by both magnitudes. When both vectors are already unit-normalized (magnitude = 1), dot product and cosine similarity are identical.

If $||a|| = 1$ and $||b|| = 1$:

$$a \cdot b = \cos(\text{angle between } a \text{ and } b)$$

When they differ: dot product includes magnitude information. A vector with larger magnitude gets a higher dot product score. This matters in some retrieval systems where you want “popular” items to rank higher. The magnitude acts as an implicit quality or importance signal.

$$a = (3, 0) \quad b = (1, 0) \quad c = (0, 1)$$

$$\begin{aligned} \text{dot}(a, b) &= 3 & \text{dot}(a, c) &= 0 \\ \cos(a, b) &= 1.0 & \cos(a, c) &= 0.0 \end{aligned}$$

Both agree on direction, but dot product also reflects magnitude.

In practice: - Use cosine similarity when you want pure directional similarity - Use dot product when magnitudes carry meaningful information - Many vector databases (Pinecone, Weaviate, Qdrant) let you choose between them - If your embeddings are L2-normalized, the choice does not matter

Mahalanobis Distance

Euclidean distance treats all dimensions equally. But if your features are correlated or have different scales, L2 gives misleading results.

Mahalanobis distance accounts for the covariance structure of the data.

$$d_M(x, y) = \sqrt{(x - y)^T \cdot S^{-1} \cdot (x - y)}$$

where S is the covariance matrix of the data.

Intuitively: Mahalanobis distance first decorrelates and normalizes the data (whitening), then computes L2 distance in that transformed space. If S is the identity matrix (uncorrelated, unit variance features), Mahalanobis distance reduces to Euclidean distance.

Example: height and weight are correlated.

Someone 6'2" and 180 lbs is not unusual.

Someone 5'0" and 180 lbs is unusual.

Euclidean distance might say they are equally far from the mean.

Mahalanobis distance correctly identifies the second as an outlier because it accounts for the height-weight correlation.

When to use Mahalanobis distance: - Outlier detection (points with large Mahalanobis distance from the mean are outliers) - Classification when features have different scales and correlations - When you have enough data to estimate a reliable covariance matrix - Quality control in manufacturing (multivariate process monitoring)

Jaccard Similarity (for sets)

Jaccard similarity measures overlap between two sets.

$$J(A, B) = |A \cap B| / |A \cup B|$$

It ranges from 0 (no overlap) to 1 (identical sets). Jaccard distance = 1 - Jaccard similarity.

A = {cat, dog, fish}

B = {cat, bird, fish, snake}

Intersection = {cat, fish} size = 2

Union = {cat, dog, fish, bird, snake} size = 5

Jaccard similarity = $2/5 = 0.4$

Jaccard distance = 0.6

When to use Jaccard: - Comparing sets of tags, categories, or features - Document similarity based on word presence (not frequency) - Near-duplicate detection (MinHash approximation of Jaccard) - Comparing binary feature vectors (presence/absence data) - Evaluating segmentation models (Intersection over Union = Jaccard)

Edit Distance (Levenshtein Distance)

Edit distance counts the minimum number of single-character operations needed to transform one string into another. The operations are: insert, delete, or substitute.

"kitten" -> "sitting"

kitten -> sitten (substitute k -> s)

sitten -> sittin (substitute e -> i)

sittin -> sitting (insert g)

Edit distance = 3

Computed using dynamic programming. Fill a matrix where entry (i, j) is the edit distance between the first i characters of string A and the first j characters of string B.

		s	i	t	t	i	n	g
" "	0	1	2	3	4	5	6	7
k	1	1	2	3	4	5	6	7
i	2	2	1	2	3	4	5	6
t	3	3	2	1	2	3	4	5
t	4	4	3	2	1	2	3	4
e	5	5	4	3	2	2	3	4
n	6	6	5	4	3	3	2	3

When to use edit distance: - Spell checking and correction - DNA sequence alignment (with weighted operations) - Fuzzy string matching - Deduplication of messy text data

KL Divergence (not a distance, but used like one)

KL divergence measures how one probability distribution differs from another. Covered in Lesson 09, but it belongs in this discussion because people use it as a "distance" despite it not being one.

$$D_{KL}(P || Q) = \sum(p(x) * \log(p(x) / q(x)))$$

Critical property: KL divergence is NOT symmetric.

$$D_{KL}(P \parallel Q) \neq D_{KL}(Q \parallel P)$$

This means it fails the basic requirement of a distance metric. It also does not satisfy the triangle inequality. It is a divergence, not a distance.

Forward KL ($D_{KL}(P \parallel Q)$) is “mean-seeking”: Q tries to cover all modes of P. Reverse KL ($D_{KL}(Q \parallel P)$) is “mode-seeking”: Q focuses on a single mode of P.

When you see KL divergence: - VAEs (the KL term in the ELBO pushes the latent distribution toward a prior) - Knowledge distillation (student tries to match teacher’s distribution) - RLHF (the KL penalty keeps the fine-tuned model close to the base model) - Policy gradient methods (constraining policy updates)

Wasserstein Distance (Earth Mover’s Distance)

Wasserstein distance measures the minimum “work” needed to transform one probability distribution into another. Think of it as: if one distribution is a pile of dirt and the other is a hole, how much dirt do you have to move and how far?

$$W(P, Q) = \inf \text{ over all transport plans } \gamma \text{ of } E[d(x, y)]$$

For 1D distributions, it simplifies to the integral of the absolute difference of the cumulative distribution functions:

$$W_1(P, Q) = \int |CDF_P(x) - CDF_Q(x)| dx$$

Why Wasserstein matters: - It is a true metric (symmetric, satisfies triangle inequality) - It provides gradients even when distributions do not overlap (KL divergence goes to infinity) - This property made it central to Wasserstein GANs (WGANs), which solved the training instability of original GANs

Distributions with no overlap:

$$P: [1, 0, 0, 0, 0] \quad Q: [0, 0, 0, 0, 1]$$

KL divergence: infinity (log of zero)

Wasserstein: 4 (move all mass 4 bins)

Wasserstein gives a meaningful gradient. KL does not.

When to use Wasserstein: - GAN training (WGAN, WGAN-GP) - Comparing distributions that may not overlap - Optimal transport problems - Image retrieval (comparing color histograms)

Why Different Tasks Need Different Distances

Task	Best distance	Why
Text similarity	Cosine	Magnitude is noise, direction is meaning
Image pixel comparison	L2	Spatial relationships matter, features are comparable scale
Sparse high-dim features	L1	Robust, does not amplify rare large differences

Task	Best distance	Why
Set overlap (tags, categories)	Jaccard	Data is naturally set-valued, not vectorial
String matching	Edit distance	Operations map to human editing intuition
Outlier detection	Mahalanobis	Accounts for feature correlations and scales
Comparing distributions	KL divergence	Measures information lost by using Q instead of P
GAN training	Wasserstein	Provides gradients even when distributions do not overlap
Embeddings (vector DB)	Cosine or dot product	Embeddings are trained to encode meaning in direction
Recommendation	Dot product	Magnitude can encode popularity or confidence
DNA sequences	Weighted edit distance	Substitution costs vary by nucleotide pair
Manufacturing QC	L-infinity	Worst-case deviation in any dimension matters

Connection to Loss Functions

Loss functions are distance functions applied to predictions vs targets.

Loss function	Distance it uses	Behavior
MSE	L2 squared	Penalizes large errors heavily
MAE	L1	Penalizes all errors equally
Huber loss	L1 for large errors, L2 for small errors	Best of both: robust to outliers, smooth gradient near zero
Cross-entropy	KL divergence	Measures distribution mismatch
Hinge loss	$\max(0, \text{margin} - d)$	Only penalizes below margin
Triplet loss	L2 (typically)	Pulls positives close, pushes negatives away
Contrastive loss	L2	Similar pairs close, dissimilar pairs beyond margin

Connection to Regularization

Regularization adds a norm penalty on the weights to the loss function.

L1 regularization (Lasso): $\text{loss} + \lambda * ||w||_1$
 -> Sparse weights. Some weights become exactly zero.
 -> Automatic feature selection.
 -> Solution has corners (non-differentiable at zero).

L2 regularization (Ridge): $\text{loss} + \lambda * ||w||_2^2$
 -> Small weights. All weights shrink toward zero.
 -> No feature selection (nothing goes to exactly zero).
 -> Smooth solution everywhere.

Elastic Net: $\text{loss} + \lambda_1 * ||w||_1 + \lambda_2 * ||w||_2^2$
 -> Combines sparsity of L1 with stability of L2.
 -> Groups of correlated features are kept or dropped together.

Why L1 produces sparsity but L2 does not: picture the constraint region in 2D weight space. L1 is a diamond, L2 is a circle. The loss function's contours (ellipses) are most likely to touch the diamond at a corner, where one weight is zero. They touch the circle at a smooth point, where both weights are nonzero.

Nearest Neighbor Search

Every distance function implies a nearest neighbor search problem: given a query point, find the closest points in a dataset.

Exact nearest neighbor search is $O(n * d)$ per query in a dataset of n points with d dimensions. For large datasets, this is too slow.

Approximate Nearest Neighbor (ANN) algorithms trade a small amount of accuracy for massive speed gains:

Algorithm	Approach	Used by
KD-trees	Axis-aligned space partition	scikit-learn (low-dim)
Ball trees	Nested hyperspheres	scikit-learn (medium-dim)
LSH	Random hash projections	Near-duplicate detection
HNSW	Hierarchical navigable small-world graph	FAISS, Qdrant, Weaviate
IVF	Inverted file index with cluster-based search	FAISS (billion-scale)
Product quant.	Compress vectors, search in compressed space	FAISS (memory-constrained)

HNSW (Hierarchical Navigable Small World) is the dominant algorithm in modern vector databases. It builds a multi-layer graph where each node connects to its approximate nearest neighbors. Search starts at the top layer (sparse, long jumps) and descends to the bottom layer (dense, short jumps).

Interactive figure: norm-unit-balls. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/14-norms-and-distances>

Build It

Step 1: All norm and distance functions

See code/distances.py for the complete implementation. Every function is built from scratch using only basic Python math.

Step 2: Same data, different distances, different neighbors

The demo in distances.py creates a dataset, picks a query point, and shows how the nearest neighbor changes depending on the distance metric. The point that is “closest” under L1 may not be closest under L2 or cosine.

Step 3: Embedding similarity search

The code includes a mock embedding similarity search that finds the most similar “documents” to a query using cosine similarity vs L2 distance, showing that the rankings can differ.

Use It

The most common practical use: finding similar items in a vector database.

```
import numpy as np

def cosine_similarity_matrix(X):
    norms = np.linalg.norm(X, axis=1, keepdims=True)
    norms = np.where(norms == 0, 1, norms)
    X_normalized = X / norms
    return X_normalized @ X_normalized.T

embeddings = np.random.randn(1000, 768)

sim_matrix = cosine_similarity_matrix(embeddings)

query_idx = 0
similarities = sim_matrix[query_idx]
top_k = np.argsort(similarities)[::-1][1:6]
print(f"Top 5 most similar to item 0: {top_k}")
print(f"Similarities: {similarities[top_k]}")
```

When you call `model.encode(text)` and then search a vector database, this is what happens under the hood. The embedding model maps text to vectors. The vector database computes cosine similarity (or dot product) between your query vector and every stored vector, using ANN algorithms to avoid checking all of them.

Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/14-norms-and-distances/code>

1. Compute L1, L2, and L-infinity distances between (1, 2, 3) and (4, 0, 6). Verify that $L\text{-inf} \leq L2 \leq L1$ always holds for any pair of points. Prove why this ordering is guaranteed.

2. Create two vectors where cosine similarity is high (> 0.9) but L2 distance is large (> 10). Explain geometrically what is happening. Then create two vectors where cosine similarity is low (< 0.3) but L2 distance is small (< 0.5).
3. Implement a function that takes a dataset and a query point and returns the nearest neighbor under L1, L2, cosine, and Mahalanobis distance. Find a dataset where all four disagree on which point is nearest.
4. Compute the Wasserstein distance between $[0.5, 0.5, 0, 0]$ and $[0, 0, 0.5, 0.5]$ by hand using the CDF method. Then compute it between $[0.25, 0.25, 0.25, 0.25]$ and $[0, 0, 0.5, 0.5]$. Which is larger and why?
5. Implement MinHash for approximate Jaccard similarity. Generate 100 random sets, compute exact Jaccard for all pairs, and compare with MinHash approximation using 50, 100, and 200 hash functions. Plot the approximation error.

Key Terms

Term	What people say	What it actually means
Norm	"Size of a vector"	A function that maps a vector to a non-negative scalar, satisfying triangle inequality, absolute homogeneity, and zero only for the zero vector
L1 norm	"Manhattan distance"	Sum of absolute component values. Produces sparsity in optimization. Robust to outliers
L2 norm	"Euclidean distance"	Square root of sum of squared components. The straight-line distance in Euclidean space
Lp norm	"Generalized norm"	The p-th root of the sum of p-th powers of absolute components. L1 and L2 are special cases
L-infinity norm	"Max norm" or "Chebyshev distance"	The maximum absolute component value. The limit of Lp as p approaches infinity
Cosine similarity	"Angle between vectors"	Dot product normalized by both magnitudes. Ranges from -1 to +1. Ignores vector length
Cosine distance	"1 minus cosine similarity"	Converts cosine similarity to a distance. Ranges from 0 to 2
Dot product	"Unnormalized cosine"	Sum of component-wise products. Equals cosine similarity times both magnitudes
Mahalanobis distance	"Correlation-aware distance"	L2 distance in a space that has been whitened (decorrelated and normalized) using the data covariance matrix
Jaccard similarity	"Set overlap"	Size of intersection divided by size of union. For sets, not vectors
Edit distance	"Levenshtein distance"	Minimum insertions, deletions, and substitutions to transform one string into another
KL divergence	"Distance between distributions"	Not a true distance (not symmetric). Measures extra bits from using Q to encode P

Term	What people say	What it actually means
Wasserstein distance	“Earth mover’s distance”	Minimum work to transport mass from one distribution to another. A true metric
Approximate nearest neighbor	“ANN search”	Algorithms (HNSW, LSH, IVF) that find approximately closest points much faster than exact search
HNSW	“The vector DB algorithm”	Hierarchical Navigable Small World graph. Multi-layer graph for fast approximate nearest neighbor search
L1 regularization	“Lasso”	Adding the L1 norm of weights to the loss. Drives weights to zero (sparsity)
L2 regularization	“Ridge” or “weight decay”	Adding the squared L2 norm of weights to the loss. Shrinks weights toward zero without sparsity
Elastic Net	“L1 + L2”	Combines L1 and L2 regularization. Handles correlated feature groups better than either alone

Further Reading

- FAISS: A Library for Efficient Similarity Search - Meta’s library for billion-scale ANN search
- Wasserstein GAN (Arjovsky et al., 2017) - the paper that introduced Earth Mover’s distance to GANs
- Locality-Sensitive Hashing (Indyk & Motwani, 1998) - foundational ANN algorithm
- Efficient Estimation of Word Representations (Mikolov et al., 2013) - Word2Vec, where cosine similarity became the default for embeddings
- sklearn.neighbors documentation - practical guide to distance metrics and neighbor algorithms in scikit-learn

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/14-norms-and-distances>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/14-norms-and-distances/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/14-norms-and-distances>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Statistics for Machine Learning

Statistics is how you know if your model actually works or just got lucky.

Type: Build **Language:** Python **Prerequisites:** Phase 1, Lessons 06 (Probability and Distributions), 07 (Bayes' Theorem) **Time:** ~120 minutes

Learning Objectives

- Compute descriptive statistics, Pearson/Spearman correlation, and covariance matrices from scratch
- Perform hypothesis tests (t-test, chi-squared) and interpret p-values and confidence intervals correctly
- Use bootstrap resampling to construct confidence intervals for any metric without distributional assumptions
- Distinguish statistical significance from practical significance using effect size measures

The Problem

You trained two models. Model A scores 0.87 on your test set. Model B scores 0.89. You deploy Model B. Three weeks later, production metrics are worse than before. What happened?

Model B did not actually outperform Model A. The 0.02 difference was noise. Your test set was too small, or the variance too high, or both. You shipped randomness dressed up as improvement.

This happens constantly. Kaggle leaderboard shakeups. Papers that fail to reproduce. A/B tests that declare winners based on a few hundred samples. The root cause is always the same: someone skipped the statistics.

Statistics gives you the tools to distinguish signal from noise. It tells you when a difference is real, how confident you should be, and how much data you need before you can trust a result. Every ML pipeline, every model comparison, every experiment needs statistics. Without it, you are guessing.

The Concept

Descriptive Statistics: Summarizing Your Data

Before you model anything, you need to know what your data looks like. Descriptive statistics compress a dataset into a few numbers that capture its shape.

Measures of central tendency answer “where is the middle?”

Mean: $\text{sum of all values} / \text{count}$
 $\mu = (1/n) * \text{sum}(x_i)$

Median: middle value when sorted

Robust to outliers. If you have [1, 2, 3, 4, 1000], the mean is 202 but the median is 3.

Mode: most frequent value

Useful for categorical data. For continuous data, rarely informative.

The mean is the balance point. The median is the halfway mark. When they diverge, your distribution is skewed. Income distributions have mean » median (right skew from billionaires). Loss distributions during training often have mean « median (left skew from easy samples).

Measures of spread answer “how dispersed is the data?”

Variance: average squared deviation from the mean

$$\sigma^2 = (1/n) * \sum((x_i - \mu)^2)$$

Standard deviation: square root of variance

$$\sigma = \sqrt{\sigma^2}$$

Same units as the data, so more interpretable.

Range: max - min

Sensitive to outliers. Almost never useful alone.

IQR: Q3 - Q1 (interquartile range)

The range of the middle 50% of the data.

Robust to outliers. Used for box plots and outlier detection.

Percentiles divide sorted data into 100 equal parts. The 25th percentile (Q1) means 25% of values fall below this point. The 50th percentile is the median. The 75th percentile is Q3.

For latency monitoring:

P50 = median latency (typical user experience)

P95 = 95th percentile (bad but not worst case)

P99 = 99th percentile (tail latency, often 10x the median)

In ML, you care about percentiles for inference latency, prediction confidence distributions, and understanding error distributions. A model with low average error but terrible P99 error might be useless for safety-critical applications.

Sample vs population statistics. When computing variance from a sample, divide by (n-1) instead of n. This is Bessel’s correction. It compensates for the fact that your sample mean is not the true population mean. With n in the denominator, you systematically underestimate the true variance. With (n-1), the estimate is unbiased.

Population variance: $\sigma^2 = (1/N) * \sum((x_i - \mu)^2)$

Sample variance: $s^2 = (1/(n-1)) * \sum((x_i - \bar{x})^2)$

In practice: if n is large (thousands of samples), the difference is negligible. If n is small (dozens of samples), it matters.

Correlation: How Variables Move Together

Correlation measures the strength and direction of a linear relationship between two variables.

Pearson correlation coefficient measures linear association:

$$r = \frac{\sum((x_i - \bar{x})(y_i - \bar{y}))}{(n * s_x * s_y)}$$

r = +1: perfect positive linear relationship

$r = -1$: perfect negative linear relationship
 $r = 0$: no linear relationship (but there might be a nonlinear one!)

Range: $[-1, 1]$

Pearson assumes the relationship is linear and both variables are roughly normally distributed. It is sensitive to outliers. A single extreme point can drag r from 0.1 to 0.9.

Spearman rank correlation measures monotonic association:

1. Replace each value with its rank (1, 2, 3, ...)
2. Compute Pearson correlation on the ranks

Spearman catches any monotonic relationship, not just linear.
 If $y = x^3$, Pearson gives $r < 1$ but Spearman gives $\rho = 1$.

When to use each:

Pearson: Both variables are continuous and roughly normal.
 You care about the linear relationship specifically.
 No extreme outliers.

Spearman: Ordinal data (rankings, ratings).
 Data is not normally distributed.
 You suspect a monotonic but not linear relationship.
 Outliers are present.

The golden rule: correlation does not imply causation. Ice cream sales and drowning deaths are correlated because both increase in summer. Your model's accuracy and the number of parameters are correlated, but adding parameters does not automatically improve accuracy (see: overfitting).

Covariance Matrix

The covariance between two variables measures how they vary together:

$$\text{Cov}(X, Y) = (1/n) * \sum((x_i - \bar{x})(y_i - \bar{y}))$$

$\text{Cov}(X, Y) > 0$: X and Y tend to increase together
 $\text{Cov}(X, Y) < 0$: when X increases, Y tends to decrease
 $\text{Cov}(X, Y) = 0$: no linear co-movement

For d features, the covariance matrix C is a $d \times d$ matrix where $C[i][j] = \text{Cov}(\text{feature}_i, \text{feature}_j)$. The diagonal entries $C[i][i]$ are the variances of each feature.

$$C = \begin{vmatrix} \text{Var}(x_1) & \text{Cov}(x_1, x_2) & \text{Cov}(x_1, x_3) \\ \text{Cov}(x_2, x_1) & \text{Var}(x_2) & \text{Cov}(x_2, x_3) \\ \text{Cov}(x_3, x_1) & \text{Cov}(x_3, x_2) & \text{Var}(x_3) \end{vmatrix}$$

Properties:

- Symmetric: $C[i][j] = C[j][i]$
- Positive semi-definite: all eigenvalues ≥ 0
- Diagonal = variances
- Off-diagonal = covariances

Connection to PCA. PCA eigendecomposes the covariance matrix. The eigenvectors are the principal components (directions of maximum variance). The eigenvalues tell you how much variance each component captures. This is exactly what Lesson 10 covered, but now you

see why the covariance matrix is the right thing to decompose: it encodes all pairwise linear relationships in your data.

Connection to correlation. The correlation matrix is the covariance matrix of standardized variables (each divided by its standard deviation). Correlation normalizes covariance so all values fall in $[-1, 1]$.

Hypothesis Testing

Hypothesis testing is a framework for making decisions under uncertainty. You start with a claim, collect data, and determine if the data is consistent with the claim.

The setup:

Null hypothesis (H_0): the default assumption, usually "no effect"
 Alternative hypothesis (H_1): what you are trying to show

Example:

H_0 : Model A and Model B have the same accuracy
 H_1 : Model B has higher accuracy than Model A

The p-value is the probability of seeing data as extreme as what you observed, assuming H_0 is true. It is NOT the probability that H_0 is true. This is the single most common misunderstanding in statistics.

$p\text{-value} = P(\text{data this extreme} \mid H_0 \text{ is true})$

If $p\text{-value} < \alpha$ (typically 0.05):

Reject H_0 . The result is "statistically significant."

If $p\text{-value} \geq \alpha$:

Fail to reject H_0 . You do not have enough evidence.

This does NOT mean H_0 is true.

Confidence intervals give a range of plausible values for a parameter:

95% confidence interval for the mean:

$\bar{x} \pm z * (s / \sqrt{n})$

where $z = 1.96$ for 95% confidence

Interpretation: if you repeated this experiment many times, 95% of the computed intervals would contain the true mean. It does NOT mean there is a 95% probability the true mean is in this specific interval.

The width of the confidence interval tells you about precision. Wide intervals mean high uncertainty. Narrow intervals mean your estimate is precise (but not necessarily accurate, if your data is biased).

The t-test

The t-test compares means. There are several flavors.

One-sample t-test: is the population mean different from a hypothesized value?

$t = (\bar{x} - \mu_0) / (s / \sqrt{n})$

degrees of freedom = $n - 1$

Two-sample t-test (independent): are two group means different?

$$t = (x_{\text{bar}_1} - x_{\text{bar}_2}) / \sqrt{s_1^2/n_1 + s_2^2/n_2}$$

This is Welch's t-test, which does not assume equal variances. Always use Welch's unless you have a specific reason for equal variances.

Paired t-test: when measurements come in pairs (same model evaluated on same data splits):

Compute $d_i = x_i - y_i$ for each pair

Then run a one-sample t-test on the d_i values against $\mu_0 = 0$

In ML, the paired t-test is common: you run both models on the same 10 cross-validation folds and compare their scores pairwise.

Chi-squared Test

The chi-squared test checks if observed frequencies match expected frequencies. Useful for categorical data.

$$\chi^2 = \sum ((\text{observed} - \text{expected})^2 / \text{expected})$$

Example: does a language model's output distribution match the training distribution across categories?

Category	Observed	Expected
Positive	120	100
Negative	80	100

$$\chi^2 = (120-100)^2/100 + (80-100)^2/100 = 4 + 4 = 8$$

With 1 degree of freedom, $\chi^2 = 8$ gives $p < 0.005$. The difference is significant.

A/B Testing for ML Models

A/B testing in ML is not the same as web A/B testing. Model comparison has specific challenges:

1. Same test set: Both models must be evaluated on identical data. Different test sets make comparison meaningless.
2. Multiple metrics: Accuracy alone is not enough. You need precision, recall, F1, latency, and fairness metrics.
3. Variance: Use cross-validation or bootstrap to estimate the variance of each metric, not just point estimates.
4. Data leakage: If the test set was used during model selection, your comparison is biased. Hold out a final test set.

The procedure:

1. Define your metric and significance level ($\alpha = 0.05$)
2. Run both models on the same k-fold cross-validation splits
3. Collect paired scores: $[(a_1, b_1), (a_2, b_2), \dots, (a_k, b_k)]$
4. Compute differences: $d_i = b_i - a_i$
5. Run a paired t-test on the differences
6. Check: is the mean difference significantly different from 0?

7. Compute a confidence interval for the mean difference
8. Compute effect size (Cohen's d) to judge practical significance

Statistical Significance vs Practical Significance

A result can be statistically significant but practically meaningless. With enough data, even a trivial difference becomes statistically significant.

Example:

Model A accuracy: 0.9234
 Model B accuracy: 0.9237
 n = 1,000,000 test samples
 p-value = 0.001

Statistically significant? Yes.

Practically significant? A 0.03% improvement is not worth the engineering cost of deploying a new model.

Effect size quantifies how big the difference is, independent of sample size:

Cohen's d = (mean_1 - mean_2) / pooled_std

d = 0.2: small effect
 d = 0.5: medium effect
 d = 0.8: large effect

Always report both the p-value and the effect size. The p-value tells you if the difference is real. The effect size tells you if it matters.

Multiple Comparison Problem

When you test many hypotheses, some will be “significant” by chance. If you test 20 things at $\alpha = 0.05$, you expect 1 false positive even when nothing is real.

$P(\text{at least one false positive}) = 1 - (1 - \alpha)^m$

m = 20 tests, $\alpha = 0.05$:
 $P(\text{false positive}) = 1 - 0.95^{20} = 0.64$

You have a 64% chance of at least one false positive.

Bonferroni correction: divide alpha by the number of tests.

Adjusted alpha = $\alpha / m = 0.05 / 20 = 0.0025$

Only reject H_0 if p-value < 0.0025.

Conservative but simple. Works when tests are independent.

In ML, this matters when you compare a model across multiple metrics, test many hyperparameter configurations, or evaluate on multiple datasets.

Bootstrap Methods

Bootstrapping estimates the sampling distribution of a statistic by resampling your data with replacement. No assumptions about the underlying distribution required.

The algorithm:

1. You have n data points
2. Draw n samples WITH replacement (some points appear multiple times, some not at all)
3. Compute your statistic on this bootstrap sample
4. Repeat B times (typically $B = 1000$ to 10000)
5. The distribution of bootstrap statistics approximates the sampling distribution

Bootstrap confidence interval (percentile method):

Sort the B bootstrap statistics

95% CI = [2.5th percentile, 97.5th percentile]

Why bootstrap matters for ML:

- Test set accuracy is a point estimate. Bootstrap gives you confidence intervals.
- You cannot assume metric distributions are normal (especially for AUC, F1, precision at k).
- Bootstrap works for ANY statistic: median, ratio of two means, difference in AUC between two models.
- No closed-form formula needed.

Bootstrap for model comparison:

1. You have predictions from Model A and Model B on the same test set
2. For each bootstrap iteration:
 - a. Resample test indices with replacement
 - b. Compute metric_A and metric_B on the resampled set
 - c. Store $\text{diff} = \text{metric_B} - \text{metric_A}$
3. 95% CI for the difference:

[2.5th percentile of diffs, 97.5th percentile of diffs]
4. If the CI does not contain 0, the difference is significant

This is more robust than the paired t-test because it makes no distributional assumptions.

Parametric vs Non-parametric Tests

Parametric tests assume a specific distribution (usually normal):

t-test: assumes normally distributed data (or large n by CLT)
 ANOVA: assumes normality and equal variances
 Pearson r : assumes bivariate normality

Non-parametric tests make no distributional assumptions:

Mann-Whitney U: compares two groups (replaces independent t-test)
 Wilcoxon signed-rank: compares paired data (replaces paired t-test)
 Spearman rho: correlation on ranks (replaces Pearson)
 Kruskal-Wallis: compares multiple groups (replaces ANOVA)

When to use non-parametric:

- Small sample size ($n < 30$) and data is clearly non-normal
- Ordinal data (ratings, rankings)
- Heavy outliers you cannot remove
- Skewed distributions

When to use parametric:

- Large sample size (CLT makes the test statistic approximately normal)
- Data is roughly symmetric without extreme outliers
- More statistical power (better at detecting real differences)

In ML experiments, you typically have small n (5 or 10 cross-validation folds), so non-parametric tests like Wilcoxon signed-rank are often more appropriate than t-tests.

Central Limit Theorem: Practical Implications

The CLT says the distribution of sample means approaches a normal distribution as n grows, regardless of the underlying population distribution.

If $X_1, X_2, \dots, X_n$ are iid with mean μ and variance σ^2 :

$$\bar{X} \sim \text{Normal}(\mu, \sigma^2 / n) \quad \text{as } n \rightarrow \text{infinity}$$

Works for $n \geq 30$ in most cases.

For highly skewed distributions, you might need $n \geq 100$.

Why this matters for ML:

1. Justifies confidence intervals and t-tests on aggregated metrics
2. Explains why averaging over cross-validation folds gives stable estimates even when individual folds vary wildly
3. Mini-batch gradient descent works because the average gradient over a batch approximates the true gradient (CLT in action)
4. Ensemble methods: averaging predictions from many models gives more stable output than any single model

What CLT does NOT do:

- Does NOT make your data normal. It makes the MEAN of samples normal.
- Does NOT work for heavy-tailed distributions with infinite variance (Cauchy distribution).
- Does NOT apply to dependent data (time series without correction).

Common Statistical Mistakes in ML Papers

1. **Testing on the training set.** Guarantees overfitting. Always hold out data the model never sees during training.
2. **No confidence intervals.** Reporting a single accuracy number without uncertainty makes results unreproducible and unverifiable.
3. **Ignoring multiple comparisons.** Testing 50 configurations and reporting the best one without correction inflates false positive rates.
4. **Confusing statistical and practical significance.** A p-value of 0.001 on a 0.01% accuracy improvement is not meaningful.
5. **Using accuracy on imbalanced data.** 99% accuracy on a dataset with 99% negative class means the model learned nothing. Use precision, recall, F1, or AUC.
6. **Cherry-picking metrics.** Reporting only the metric where your model wins. Honest evaluation reports all relevant metrics.
7. **Leaking information across train/test splits.** Normalizing before splitting, or using future data to predict the past.

8. **Small test sets with no variance estimates.** Evaluating on 100 samples and claiming 2% improvement is noise, not signal.
9. **Assuming independence when data is not independent.** Medical images from the same patient, multiple sentences from the same document. Observations within a group are correlated.
10. **P-hacking.** Trying different tests, subsets, or exclusion criteria until you get $p < 0.05$. The result is an artifact of the search.

Building It

You will implement:

1. **Descriptive statistics from scratch** (mean, median, mode, standard deviation, percentiles, IQR)
2. **Correlation functions** (Pearson and Spearman, with the covariance matrix)
3. **Hypothesis tests** (one-sample t-test, two-sample t-test, chi-squared test)
4. **Bootstrap confidence intervals** (for any statistic, no assumptions needed)
5. **A/B test simulator** (generate data, test, check for Type I and Type II errors)
6. **Statistical vs practical significance demo** (showing that large n makes everything “significant”)

All from scratch, using only math and random. No numpy, no scipy.

Interactive figure: f3-bootstrap-resample. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/15-statistics-for-ml>

Key Terms

Term	Definition
Mean	Sum of values divided by count. Sensitive to outliers.
Median	Middle value of sorted data. Robust to outliers.
Standard deviation	Square root of variance. Measures spread in original units.
Percentile	Value below which a given percentage of data falls.
IQR	Interquartile range. $Q3$ minus $Q1$. The spread of the middle 50%.
Pearson correlation	Measures linear association between two variables. Range $[-1, 1]$.
Spearman correlation	Measures monotonic association using ranks.
Covariance matrix	Matrix of pairwise covariances between all features.
Null hypothesis	Default assumption of no effect or no difference.
p-value	Probability of data this extreme given the null hypothesis is true.

Term	Definition
Confidence interval	Range of plausible values for a parameter at a given confidence level.
t-test	Tests whether means differ significantly. Uses the t-distribution.
Chi-squared test	Tests whether observed frequencies differ from expected frequencies.
Effect size	Magnitude of a difference, independent of sample size. Cohen's d is common.
Bonferroni correction	Divides significance threshold by number of tests to control false positives.
Bootstrap	Resampling with replacement to estimate sampling distributions.
Type I error	False positive. Rejecting H_0 when it is true.
Type II error	False negative. Failing to reject H_0 when it is false.
Statistical power	Probability of correctly rejecting a false H_0 . Power = 1 minus Type II error rate.
Central limit theorem	Sample means converge to a normal distribution as sample size grows.
Parametric test	Assumes a specific distribution for the data (usually normal).
Non-parametric test	Makes no distributional assumptions. Works on ranks or signs.

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/15-statistics-for-ml>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/15-statistics-for-ml/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/15-statistics-for-ml>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Sampling Methods

Sampling is how AI explores the space of possibilities.

Type: Build **Language:** Python **Prerequisites:** Phase 1, Lessons 06-07 (Probability, Bayes' Theorem) **Time:** ~120 minutes

Learning Objectives

- Implement inverse CDF, rejection, and importance sampling from scratch using only uniform random numbers
- Build temperature, top-k, and top-p (nucleus) sampling for language model token generation
- Explain the reparameterization trick and why it enables backpropagation through sampling in VAEs
- Run Metropolis-Hastings MCMC to sample from an unnormalized target distribution

The Problem

A language model finishes processing your prompt and produces a vector of 50,000 logits. One for every token in its vocabulary. Now it has to pick one. How?

If it always picks the highest-probability token, every response is identical. Deterministic. Boring. If it picks uniformly at random, the output is gibberish. The answer lives somewhere between these extremes, and that somewhere is controlled by sampling.

Sampling is not limited to text generation. Reinforcement learning estimates policy gradients by sampling trajectories. VAEs learn latent representations by sampling from learned distributions and backpropagating through the randomness. Diffusion models generate images by sampling noise and iteratively denoising. Monte Carlo methods estimate integrals that have no closed-form solution. MCMC algorithms explore high-dimensional posterior distributions that are impossible to enumerate.

Every generative AI system is a sampling system. The sampling strategy determines the quality, diversity, and controllability of the output. This lesson builds every major sampling method from scratch, starting from uniform random numbers and ending with the techniques that power modern LLMs and generative models.

The Concept

Why Sampling Matters

Sampling appears in four fundamental roles across AI and machine learning:

Generation. Language models, diffusion models, and GANs all produce output by sampling. The sampling algorithm directly controls creativity, coherence, and diversity. Temperature, top-k, and nucleus sampling are the knobs that engineers turn daily.

Training. Stochastic gradient descent samples mini-batches. Dropout samples neurons to deactivate. Data augmentation samples random transformations. Importance sampling reweights samples to reduce gradient variance in reinforcement learning (PPO, TRPO).

Estimation. Many quantities in ML have no closed-form solution. The expected loss over a data distribution, the partition function of an energy-based model, the evidence in Bayesian inference. Monte Carlo estimation approximates all of these by averaging over samples.

Exploration. MCMC algorithms explore posterior distributions in Bayesian inference. Evolutionary strategies sample parameter perturbations. Thompson sampling balances exploration and exploitation in bandits.

The core challenge: you can only sample directly from simple distributions (uniform, normal). For everything else, you need a method to convert simple samples into samples from your target distribution.

Uniform Random Sampling

Every sampling method starts here. A uniform random number generator produces values in $[0, 1)$ where every sub-interval of equal length has equal probability.

$U \sim \text{Uniform}(0, 1)$

$P(a \leq U \leq b) = b - a \quad \text{for } 0 \leq a \leq b \leq 1$

Properties:

$E[U] = 0.5$

$\text{Var}(U) = 1/12$

To sample uniformly from a discrete set of n items, generate U and return $\text{floor}(n * U)$. To sample from a continuous range $[a, b]$, compute $a + (b - a) * U$.

The key insight: a single uniform random number contains exactly the right amount of randomness to produce one sample from any distribution. The trick is finding the right transformation.

Inverse CDF Method (Inverse Transform Sampling)

The cumulative distribution function (CDF) maps values to probabilities:

$F(x) = P(X \leq x)$

Properties:

F is non-decreasing

$F(-\infty) = 0$

$F(+\infty) = 1$

F maps the real line to $[0, 1]$

The inverse CDF maps probabilities back to values. If $U \sim \text{Uniform}(0, 1)$, then $X = F_{\text{inverse}}(U)$ follows the target distribution.

Algorithm:

1. Generate $u \sim \text{Uniform}(0, 1)$
2. Return $F_{\text{inverse}}(u)$

Why it works:

$P(X \leq x) = P(F_{\text{inverse}}(U) \leq x) = P(U \leq F(x)) = F(x)$

Exponential distribution example:

PDF: $f(x) = \lambda \exp(-\lambda x)$, $x \geq 0$
 CDF: $F(x) = 1 - \exp(-\lambda x)$

Solve $F(x) = u$ for x :
 $u = 1 - \exp(-\lambda x)$
 $\exp(-\lambda x) = 1 - u$
 $x = -\ln(1 - u) / \lambda$

Since $(1 - U)$ and U have the same distribution:
 $x = -\ln(u) / \lambda$

This works perfectly when you can write down F_{inverse} in closed form. For the normal distribution, there is no closed-form inverse CDF, so we use other methods (Box-Muller, or numerical approximation).

Discrete version: For discrete distributions, build the CDF as a cumulative sum, generate U , and find the first index where the cumulative sum exceeds U . This is how `sample_categorical` works in Lesson 06.

Rejection Sampling

When you cannot invert the CDF but can evaluate the target PDF up to a constant, rejection sampling works.

Target distribution: $p(x)$ (can evaluate, possibly unnormalized)
 Proposal distribution: $q(x)$ (can sample from)
 Bound: M such that $p(x) \leq M * q(x)$ for all x

Algorithm:

1. Sample $x \sim q(x)$
2. Sample $u \sim \text{Uniform}(0, 1)$
3. If $u < p(x) / (M * q(x))$, accept x
4. Otherwise, reject and go to step 1

Acceptance rate = $1/M$

The tighter the bound M , the higher the acceptance rate. In low dimensions (1-3), rejection sampling works well. In high dimensions, the acceptance rate drops exponentially because most of the proposal volume gets rejected. This is the curse of dimensionality for rejection sampling.

Example: sampling from a truncated normal. Use a uniform proposal over the truncated range. The envelope M is the maximum of the normal PDF in that range.

Example: sampling from a semicircle. Propose uniformly in the bounding rectangle. Accept if the point falls inside the semicircle. This is how Monte Carlo computes π : the acceptance rate equals the area ratio $\pi/4$.

Importance Sampling

Sometimes you do not need samples from the target distribution $p(x)$. You need to estimate an expectation under $p(x)$, and you have samples from a different distribution $q(x)$.

Goal: estimate $E_p[f(x)] = \text{integral of } f(x) * p(x) \, dx$

Rewrite:
 $E_p[f(x)] = \text{integral of } f(x) * (p(x)/q(x)) * q(x) \, dx$

$$= E_q[f(x) * w(x)]$$

where $w(x) = p(x) / q(x)$ are the importance weights.

Estimator:

$$E_p[f(x)] \sim (1/N) * \sum(f(x_i) * w(x_i)) \quad \text{where } x_i \sim q(x)$$

This is critical in reinforcement learning. In PPO (Proximal Policy Optimization), you collect trajectories under an old policy π_{old} but want to optimize a new policy π_{new} . The importance weight is $\pi_{\text{new}}(a|s) / \pi_{\text{old}}(a|s)$. PPO clips these weights to prevent the new policy from diverging too far from the old one.

The variance of the importance sampling estimator depends on how similar q is to p . If q is very different from p , a few samples get enormous weights and dominate the estimate. Self-normalized importance sampling divides by the sum of weights to reduce this problem:

$$E_p[f(x)] \sim \sum(w_i * f(x_i)) / \sum(w_i)$$

Monte Carlo Estimation

Monte Carlo estimation approximates integrals by averaging random samples. The law of large numbers guarantees convergence.

Goal: estimate $I = \text{integral of } g(x) \text{ dx over domain } D$

Method:

1. Sample $x_1, \dots, x_N$ uniformly from D
2. $I \sim (\text{Volume of } D / N) * \sum(g(x_i))$

Error: $O(1 / \sqrt{N})$ regardless of dimension

The error rate is dimension-independent. This is why Monte Carlo methods dominate in high dimensions where grid-based integration is impossible.

Estimating π :

Sample (x, y) uniformly from $[-1, 1] \times [-1, 1]$
 Count how many fall inside the unit circle: $x^2 + y^2 \leq 1$
 $\pi \sim 4 * (\text{count inside}) / (\text{total count})$

Estimating expectations:

$$E[f(X)] \sim (1/N) * \sum(f(x_i)) \quad \text{where } x_i \sim p(x)$$

The sample mean converges to the true expectation.

Variance of the estimator = $\text{Var}(f(X)) / N$

Markov Chain Monte Carlo (MCMC): Metropolis-Hastings

MCMC constructs a Markov chain whose stationary distribution is the target distribution $p(x)$. After enough steps, samples from the chain are (approximately) samples from $p(x)$.

Target: $p(x)$ (known up to a normalizing constant)

Proposal: $q(x'|x)$ (how to propose the next state given the current state)

Metropolis-Hastings algorithm:

1. Start at some x_0
2. For $t = 1, 2, \dots, T$:

- a. Propose $x' \sim q(x'|x_t)$
- b. Compute acceptance ratio:
 $\alpha = [p(x') * q(x_t|x')] / [p(x_t) * q(x'|x_t)]$
- c. Accept with probability $\min(1, \alpha)$:
 - If $u < \alpha$ ($u \sim \text{Uniform}(0,1)$): $x_{t+1} = x'$
 - Otherwise: $x_{t+1} = x_t$
3. Discard first B samples (burn-in)
4. Return remaining samples

For symmetric proposals ($q(x'|x) = q(x|x')$), the ratio simplifies to $p(x')/p(x)$. This is the original Metropolis algorithm.

Why it works. The acceptance rule ensures detailed balance: the probability of being at x and moving to x' equals the probability of being at x' and moving to x . Detailed balance implies that $p(x)$ is the stationary distribution of the chain.

Practical considerations: - Burn-in: discard early samples before the chain reaches equilibrium - Thinning: keep every k -th sample to reduce autocorrelation - Proposal scale: too small and the chain moves slowly (high acceptance, slow exploration); too large and most proposals are rejected (low acceptance, stuck in place) - The optimal acceptance rate for a Gaussian proposal in high dimensions is approximately 0.234

Gibbs Sampling

Gibbs sampling is a special case of MCMC for multivariate distributions. Instead of proposing a move in all dimensions at once, it updates one variable at a time from its conditional distribution.

Target: $p(x_1, x_2, \dots, x_d)$

Algorithm:

```

For each iteration t:
  Sample  $x_1^{t+1} \sim p(x_1 | x_2^t, x_3^t, \dots, x_d^t)$ 
  Sample  $x_2^{t+1} \sim p(x_2 | x_1^{t+1}, x_3^t, \dots, x_d^t)$ 
  ...
  Sample  $x_d^{t+1} \sim p(x_d | x_1^{t+1}, x_2^{t+1}, \dots, x_{d-1}^{t+1})$ 

```

Gibbs sampling requires that you can sample from each conditional distribution $p(x_i | x_{-i})$. This is straightforward for many models: - Bayesian networks: conditionals follow from the graph structure - Gaussian mixtures: conditionals are Gaussian - Ising models: each spin's conditional depends only on its neighbors

The acceptance rate is always 1 (every proposal is accepted) because sampling from the exact conditional automatically satisfies detailed balance.

Limitation. When variables are highly correlated, Gibbs sampling mixes slowly because updating one variable at a time cannot make large diagonal moves through the distribution.

Temperature Sampling (Used in LLMs)

Language models output logits $z_1, \dots, z_V$ for each token in the vocabulary. Softmax converts these to probabilities. Temperature rescales the logits before softmax:

$$p_i = \exp(z_i / T) / \sum(\exp(z_j / T))$$

$T = 1.0$: standard softmax (original distribution)

$T \rightarrow 0$: argmax (deterministic, always picks highest logit)

$T \rightarrow \infty$: uniform (all tokens equally likely)
 $T < 1.0$: sharpens the distribution (more confident, less diverse)
 $T > 1.0$: flattens the distribution (less confident, more diverse)

Why it works. Dividing logits by $T < 1$ amplifies differences between logits. If $z_1 = 2$ and $z_2 = 1$, dividing by $T = 0.5$ gives $z_1/T = 4$ and $z_2/T = 2$, making the gap larger. After softmax, the highest-logit token gets a much larger share.

In practice: - $T = 0.0$: greedy decoding, best for factual Q&A - $T = 0.3-0.7$: slightly creative, good for code generation - $T = 0.7-1.0$: balanced, good for general conversation - $T = 1.0-1.5$: creative writing, brainstorming - $T > 1.5$: increasingly random, rarely useful

Temperature does not change which tokens are possible. It changes the probability mass allocated to each token.

Top-k Sampling

Top-k sampling restricts the candidate set to the k tokens with the highest probabilities, then renormalizes and samples from that restricted set.

Algorithm:

1. Compute softmax probabilities for all V tokens
2. Sort tokens by probability (descending)
3. Keep only the top k tokens
4. Renormalize: $p_i' = p_i / \sum(p_j \text{ for } j \text{ in top-}k)$
5. Sample from the renormalized distribution

$k = 1$: greedy decoding

$k = V$: no filtering (standard sampling)

$k = 40$: typical setting, removes long tail of unlikely tokens

Top-k prevents the model from selecting extremely unlikely tokens (typos, nonsense) that exist in the long tail of the vocabulary distribution. The problem: k is fixed regardless of context. When the model is confident (one token has 95% probability), $k = 40$ still allows 39 alternatives. When the model is uncertain (probability is spread across 1000 tokens), $k = 40$ cuts off plausible options.

Top-p (Nucleus) Sampling

Top-p sampling dynamically adjusts the candidate set size. Instead of keeping a fixed number of tokens, it keeps the smallest set of tokens whose cumulative probability exceeds p .

Algorithm:

1. Compute softmax probabilities for all V tokens
2. Sort tokens by probability (descending)
3. Find smallest k such that sum of top- k probabilities $\geq p$
4. Keep only those k tokens
5. Renormalize and sample

$p = 0.9$: keeps tokens covering 90% of probability mass

$p = 1.0$: no filtering

$p = 0.1$: very restrictive, nearly greedy

When the model is confident, nucleus sampling keeps few tokens (maybe 2-3). When the model is uncertain, it keeps many (maybe 200). This adaptive behavior is why nucleus sampling generally produces better text than top-k.

Common combinations: - Temperature 0.7 + top-p 0.9: good general-purpose setting - Temperature 0.0 (greedy): best for deterministic tasks - Temperature 1.0 + top-k 50: Fan et al. (2018) original paper setting

Top-k and top-p can be combined. Apply top-k first, then top-p on the remaining set.

Reparameterization Trick (Used in VAEs)

Variational autoencoders (VAEs) learn by encoding inputs into a distribution in latent space, sampling from that distribution, and decoding the sample back. The problem: you cannot backpropagate through a sampling operation.

Standard sampling (not differentiable):

$z \sim N(\mu, \sigma^2)$

The randomness blocks gradient flow.

$d/d_{\mu} [\text{sample from } N(\mu, \sigma^2)] = ???$

The reparameterization trick separates the randomness from the parameters:

Reparameterized sampling:

$\epsilon \sim N(0, 1)$ (fixed random noise, no parameters)

$z = \mu + \sigma * \epsilon$ (deterministic function of parameters)

Now z is a deterministic, differentiable function of μ and σ .

$d(z)/d(\mu) = 1$

$d(z)/d(\sigma) = \epsilon$

Gradients flow through μ and σ .

This works because $N(\mu, \sigma^2)$ has the same distribution as $\mu + \sigma * N(0, 1)$. The key insight: move the randomness to a parameter-free source (ϵ), then express the sample as a differentiable transformation of the parameters.

In the VAE training loop: 1. Encoder outputs μ and $\log(\sigma^2)$ for each input 2. Sample $\epsilon \sim N(0, 1)$ 3. Compute $z = \mu + \sigma * \epsilon$ 4. Decode z to reconstruct the input 5. Backpropagate through steps 4, 3, 2, 1 (possible because step 3 is differentiable)

Without the reparameterization trick, VAEs cannot be trained with standard backpropagation. This single insight made VAEs practical.

Gumbel-Softmax (Differentiable Categorical Sampling)

The reparameterization trick works for continuous distributions (Gaussian). For discrete categorical distributions, we need a different approach. Gumbel-Softmax provides a differentiable approximation to categorical sampling.

The Gumbel-Max trick (non-differentiable):

To sample from a categorical distribution with log-probabilities $\log(p_1), \dots, \log(p_k)$:

1. Sample $g_i \sim \text{Gumbel}(0, 1)$ for each category
($g = -\log(-\log(u))$, where $u \sim \text{Uniform}(0, 1)$)
2. Return $\text{argmax}(\log(p_i) + g_i)$

This produces exact categorical samples.

Gumbel-Softmax (differentiable approximation):

Replace the hard argmax with a soft softmax:

$$y_i = \exp((\log(p_i) + g_i) / \tau) / \sum(\exp((\log(p_j) + g_j) / \tau))$$

τ (temperature) controls the approximation:

$\tau \rightarrow 0$: approaches a one-hot vector (hard categorical)

$\tau \rightarrow \infty$: approaches uniform ($1/k, 1/k, \dots, 1/k$)

$\tau = 1.0$: soft approximation

Gumbel-Softmax produces a continuous relaxation of a discrete sample. The output is a probability vector (soft one-hot) instead of a hard one-hot. Gradients flow through the softmax. During the forward pass in training, you can use the “straight-through” estimator: use the hard argmax for the forward pass but the soft Gumbel-Softmax gradients for the backward pass.

Applications: - Discrete latent variables in VAEs - Neural architecture search (choosing discrete operations) - Hard attention mechanisms - Reinforcement learning with discrete actions

Stratified Sampling

Standard Monte Carlo sampling can leave gaps in the sample space by chance. Stratified sampling forces even coverage by dividing the space into strata and sampling from each.

Standard Monte Carlo:

Sample N points uniformly from $[0, 1]$

Some regions may have clusters, others gaps

Stratified sampling:

Divide $[0, 1]$ into N equal strata: $[0, 1/N), [1/N, 2/N), \dots, [(N-1)/N, 1]$

Sample one point uniformly within each stratum

$x_i = (i + u_i) / N$ where $u_i \sim \text{Uniform}(0, 1)$, $i = 0, \dots, N-1$

Stratified sampling always has lower or equal variance compared to standard Monte Carlo:

$\text{Var}(\text{stratified}) \leq \text{Var}(\text{standard Monte Carlo})$

The improvement is largest when $f(x)$ varies smoothly.

For piecewise-constant functions, stratified sampling is exact.

Applications: - Numerical integration (quasi-Monte Carlo) - Training data splits (ensuring class balance in each fold) - Importance sampling with stratification (combining both techniques) - NeRF (Neural Radiance Fields) uses stratified sampling along camera rays

Connection to Diffusion Models

Diffusion models generate images through a sampling process. The forward process adds Gaussian noise to an image over T steps until it becomes pure noise. The reverse process learns to denoise, recovering the original image step by step.

Forward process (known):

$x_t = \sqrt{\alpha_t} * x_{t-1} + \sqrt{1 - \alpha_t} * \epsilon$
where $\epsilon \sim N(0, I)$

After T steps: $x_T \sim N(0, I)$ (pure noise)

Reverse process (learned):

$x_{t-1} = (1/\sqrt{\alpha_t}) * (x_t - (1 - \alpha_t)/\sqrt{1 - \alpha_{t-1}} * \epsilon_{\theta}(x_t, t))$
where $z \sim N(0, I)$

Each denoising step is a sampling step.

The connection to the methods in this lesson: - Each denoising step uses the reparameterization trick (sample noise, apply deterministic transform) - The noise schedule $\{\alpha_t\}$ controls a form of temperature annealing - Training uses Monte Carlo estimation to approximate the ELBO (evidence lower bound) - Ancestral sampling in diffusion models is a Markov chain (each step depends only on the current state)

The entire image generation process is iterative sampling: start from noise, and at each step, sample a slightly less noisy version conditioned on the learned denoising model.

Interactive figure: monte-carlo-pi. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/16-sampling-methods>

Build It

Step 1: Uniform and inverse CDF sampling

```
import math
import random

def sample_uniform(a, b):
    return a + (b - a) * random.random()

def sample_exponential_inverse_cdf(lam):
    u = random.random()
    return -math.log(u) / lam
```

Generate 10,000 exponential samples and verify the mean is $1/\lambda$.

Step 2: Rejection sampling

```
def rejection_sample(target_pdf, proposal_sample, proposal_pdf, M):
    while True:
        x = proposal_sample()
        u = random.random()
        if u < target_pdf(x) / (M * proposal_pdf(x)):
            return x
```

Use rejection sampling to draw from a truncated normal distribution. Verify the shape by histogramming the samples.

Step 3: Importance sampling

```
def importance_sampling_estimate(f, target_pdf, proposal_pdf, proposal_sample, n):
    total = 0
    for _ in range(n):
        x = proposal_sample()
        w = target_pdf(x) / proposal_pdf(x)
        total += f(x) * w
    return total / n
```

Estimate $E[X^2]$ under a normal distribution using a uniform proposal. Compare to the known answer ($\mu^2 + \sigma^2$).

Step 4: Monte Carlo estimation of pi

```
def monte_carlo_pi(n):
    inside = 0
    for _ in range(n):
        x = random.uniform(-1, 1)
        y = random.uniform(-1, 1)
        if x*x + y*y <= 1:
            inside += 1
    return 4 * inside / n
```

Step 5: Metropolis-Hastings MCMC

```
def metropolis_hastings(target_log_pdf, proposal_sample, proposal_log_pdf, x0, n_samples, burn_in):
    samples = []
    x = x0
    for i in range(n_samples + burn_in):
        x_new = proposal_sample(x)
        log_alpha = (target_log_pdf(x_new) + proposal_log_pdf(x, x_new)
                     - target_log_pdf(x) - proposal_log_pdf(x_new, x))
        if math.log(random.random()) < log_alpha:
            x = x_new
        if i >= burn_in:
            samples.append(x)
    return samples
```

Sample from a bimodal distribution (mixture of two Gaussians). Visualize the chain's trajectory.

Step 6: Gibbs sampling

```
def gibbs_sampling_2d(conditional_x_given_y, conditional_y_given_x, x0, y0, n_samples, burn_in):
    x, y = x0, y0
    samples = []
    for i in range(n_samples + burn_in):
        x = conditional_x_given_y(y)
        y = conditional_y_given_x(x)
        if i >= burn_in:
            samples.append((x, y))
    return samples
```

Step 7: Temperature sampling

```
def softmax(logits):
    max_l = max(logits)
    exps = [math.exp(z - max_l) for z in logits]
    total = sum(exps)
    return [e / total for e in exps]

def temperature_sample(logits, temperature):
    scaled = [z / temperature for z in logits]
    probs = softmax(scaled)
    return sample_from_probs(probs)
```

Show how temperature changes the output distribution for a set of token logits.

Step 8: Top-k and top-p sampling

```
def top_k_sample(logits, k):
    indexed = sorted(enumerate(logits), key=lambda x: -x[1])
    top = indexed[:k]
    top_logits = [l for _, l in top]
    probs = softmax(top_logits)
    idx = sample_from_probs(probs)
    return top[idx][0]

def top_p_sample(logits, p):
    probs = softmax(logits)
    indexed = sorted(enumerate(probs), key=lambda x: -x[1])
    cumsum = 0
    selected = []
    for token_idx, prob in indexed:
        cumsum += prob
        selected.append((token_idx, prob))
        if cumsum >= p:
            break
    sel_probs = [pr for _, pr in selected]
    total = sum(sel_probs)
    sel_probs = [pr / total for pr in sel_probs]
    idx = sample_from_probs(sel_probs)
    return selected[idx][0]
```

Step 9: Reparameterization trick

```
def reparam_sample(mu, sigma):
    epsilon = random.gauss(0, 1)
    return mu + sigma * epsilon

def reparam_gradient(mu, sigma, epsilon):
    dz_dmu = 1.0
    dz_dsigma = epsilon
    return dz_dmu, dz_dsigma
```

Demonstrate that gradients flow through the reparameterized sample but not through direct sampling.

Step 10: Gumbel-Softmax

```
def gumbel_sample():
    u = random.random()
    return -math.log(-math.log(u))

def gumbel_softmax(logits, temperature):
    gumbels = [math.log(p) + gumbel_sample() for p in logits]
    return softmax([g / temperature for g in gumbels])
```

Show how decreasing temperature makes the output approach a one-hot vector.

Full implementations with all visualizations are in `code/sampling.py`.

Use It

With NumPy and SciPy, the production versions:

```
import numpy as np

rng = np.random.default_rng(42)

exponential_samples = rng.exponential(scale=2.0, size=10000)
print(f"Exponential mean: {exponential_samples.mean():.4f} (expected 2.0)")

from scipy import stats
normal = stats.norm(loc=0, scale=1)
print(f"CDF at 1.96: {normal.cdf(1.96):.4f}")
print(f"Inverse CDF at 0.975: {normal.ppf(0.975):.4f}")

logits = np.array([2.0, 1.0, 0.5, 0.1, -1.0])
temperature = 0.7
scaled = logits / temperature
probs = np.exp(scaled - scaled.max()) / np.exp(scaled - scaled.max()).sum()
token = rng.choice(len(logits), p=probs)
print(f"Sampled token index: {token}")
```

For MCMC at scale, use dedicated libraries: - PyMC: full Bayesian modeling with NUTS (adaptive HMC) - emcee: ensemble MCMC sampler - NumPyro/JAX: GPU-accelerated MCMC

You built these from scratch. Now you know what the library calls are doing.

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/16-sampling-methods/code>

1. Implement inverse CDF sampling for the Cauchy distribution. The CDF is $F(x) = 0.5 + \arctan(x)/\pi$. Generate 10,000 samples and plot the histogram against the true PDF. Notice the heavy tails (extreme values far from center).
2. Use rejection sampling to generate samples from a Beta(2, 5) distribution using a Uniform(0, 1) proposal. Plot the accepted samples against the true Beta PDF. What is the theoretical acceptance rate?
3. Estimate the integral of $\sin(x)$ from 0 to π using Monte Carlo with 1,000, 10,000, and 100,000 samples. Compare the error at each level. Verify that the error scales as $O(1/\sqrt{N})$.
4. Implement Metropolis-Hastings to sample from a 2D distribution $p(x, y)$ proportional to $\exp(-(x^2 * y^2 + x^2 + y^2 - 8x - 8y) / 2)$. Plot the samples and the chain trajectory. Experiment with different proposal standard deviations.
5. Build a complete text generation demo: given a vocabulary of 10 words with logits, generate sequences of 20 tokens using (a) greedy, (b) temperature=0.7, (c) top-k=3, (d) top-p=0.9. Compare the diversity of outputs across 5 runs.

Key Terms

Term	What people say	What it actually means
Sampling	"Drawing random values"	Generating values according to a probability distribution. The mechanism behind all generative AI
Uniform distribution	"All equally likely"	Every value in $[a, b]$ has equal probability density $1/(b-a)$. The starting point for all sampling methods
Inverse CDF	"Probability transform"	$F_{\text{inverse}}(U)$ converts a uniform sample into a sample from any distribution with known CDF. Exact and efficient
Rejection sampling	"Propose and accept/reject"	Generate from a simple proposal, accept with probability proportional to target/proposal ratio. Exact but wastes samples
Importance sampling	"Reweight samples"	Estimate expectations under $p(x)$ using samples from $q(x)$ by weighting each sample by $p(x)/q(x)$. Core to PPO in RL
Monte Carlo	"Average random samples"	Approximate integrals as sample averages. Error $O(1/\sqrt{N})$ regardless of dimension
MCMC	"Random walk that converges"	Construct a Markov chain whose stationary distribution is the target. Metropolis-Hastings is the foundational algorithm
Metropolis-Hastings	"Accept uphill, sometimes downhill"	Propose moves, accept based on density ratio. Detailed balance ensures convergence to target distribution
Gibbs sampling	"One variable at a time"	Update each variable from its conditional distribution holding others fixed. 100% acceptance rate
Temperature	"Confidence knob"	Divides logits by T before softmax. $T < 1$ sharpens (more confident), $T > 1$ flattens (more diverse)
Top-k sampling	"Keep the k best"	Zero out all but the k highest-probability tokens, renormalize, sample. Fixed candidate set size
Nucleus sampling (top-p)	"Keep the probable ones"	Keep the smallest set of tokens whose cumulative probability exceeds p . Adaptive candidate set size
Reparameterization trick	"Move randomness outside"	Write $z = \mu + \sigma * \epsilon$ where $\epsilon \sim N(0,1)$. Makes sampling differentiable. Essential for VAE training
Gumbel-Softmax	"Soft categorical sampling"	Differentiable approximation to categorical sampling using Gumbel noise + softmax with temperature
Stratified sampling	"Forced coverage"	Divide sample space into strata, sample from each. Always lower variance than naive Monte Carlo
Burn-in	"Warm-up period"	Initial MCMC samples discarded before the chain reaches its stationary distribution
Detailed balance	"Reversibility condition"	$p(x) * T(x \rightarrow y) = p(y) * T(y \rightarrow x)$. Sufficient condition for p to be the stationary distribution of a Markov chain

Term	What people say	What it actually means
Diffusion sampling	“Iterative denoising”	Generate data by starting from noise and applying learned denoising steps. Each step is a conditional sampling operation

Further Reading

- Holbrook (2023): The Metropolis-Hastings Algorithm - detailed tutorial on MCMC foundations
- Jang, Gu, Poole (2017): Categorical Reparameterization with Gumbel-Softmax - original Gumbel-Softmax paper
- Holtzman et al. (2020): The Curious Case of Neural Text Degeneration - nucleus (top-p) sampling paper
- Kingma & Welling (2014): Auto-Encoding Variational Bayes - VAE paper introducing the reparameterization trick
- Ho, Jain, Abbeel (2020): Denoising Diffusion Probabilistic Models - DDPM connects sampling to image generation

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/16-sampling-methods>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/16-sampling-methods/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/16-sampling-methods>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Linear Systems

Solving $Ax = b$ is the oldest problem in mathematics that still runs your neural network.

Type: Build **Language:** Python **Prerequisites:** Phase 1, Lessons 01 (Linear Algebra Intuition), 02 (Vectors & Matrices), 03 (Matrix Transformations) **Time:** ~120 minutes

Learning Objectives

- Solve $Ax = b$ using Gaussian elimination with partial pivoting and back substitution
- Factor matrices with LU, QR, and Cholesky decompositions and explain when each is appropriate
- Derive the normal equations for least squares and connect them to linear and ridge regression
- Diagnose ill-conditioned systems using the condition number and apply regularization to stabilize them

The Problem

Every time you train a linear regression, you solve a linear system. Every time you compute a least-squares fit, you solve a linear system. Every time a neural network layer computes $y = Wx + b$, it is evaluating one side of a linear system. When you add regularization, you modify the system. When you use Gaussian processes, you factor a matrix. When you invert a covariance matrix for Mahalanobis distance, you solve a linear system.

The equation $Ax = b$ appears everywhere. A is a matrix of known coefficients. b is a vector of known outputs. x is the vector of unknowns you want to find. In linear regression, A is your data matrix, b is your target vector, and x is the weight vector. The entire model reduces to: find x such that Ax is as close to b as possible.

This lesson builds every major method for solving that equation from scratch. You will understand why some methods are fast and others are stable, why some work only for square systems and others handle overdetermined ones, and why the condition number of your matrix determines whether your answer means anything at all.

The Concept

What $Ax = b$ means geometrically

A system of linear equations has a geometric interpretation. Each equation defines a hyperplane. The solution is the point (or set of points) where all hyperplanes intersect.

$2x + y = 5$	Two lines in 2D.
$x - y = 1$	They intersect at $x=2, y=1$.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/17-linear-systems>

Three things can happen:

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/17-linear-systems>

In matrix form, “one solution” means A is invertible. “No solution” means the system is inconsistent. “Infinite solutions” means A has a null space. Most ML problems fall in the “no exact solution” category because you have more equations (data points) than unknowns (parameters). That is where least squares comes in.

Column picture vs row picture

There are two ways to read $Ax = b$.

Row picture. Each row of A defines one equation. Each equation is a hyperplane. The solution is where they all intersect.

Column picture. Each column of A is a vector. The question becomes: what linear combination of the columns of A produces b ?

$$A = \begin{bmatrix} 2 & 1 \\ 1 & -1 \end{bmatrix} \quad b = \begin{bmatrix} 5 \\ 1 \end{bmatrix}$$

Row picture: solve $2x + y = 5$ and $x - y = 1$ simultaneously.

Column picture: find x_1, x_2 such that:

$$\begin{aligned} x_1 * [2, 1] + x_2 * [1, -1] &= [5, 1] \\ 2 * [2, 1] + 1 * [1, -1] &= [4+1, 2-1] = [5, 1] \quad \text{check.} \end{aligned}$$

The column picture is more fundamental. If b lies in the column space of A , the system has a solution. If b does not, you find the closest point in the column space. That closest point is the least-squares solution.

Gaussian elimination

Gaussian elimination transforms $Ax = b$ into an upper triangular system $Ux = c$ that you solve by back substitution. It is the most direct method.

The algorithm:

1. For each column k (the pivot column):
 - a. Find the largest entry in column k at or below row k (partial pivoting).
 - b. Swap that row with row k .
 - c. For each row i below k :
 - Compute multiplier $m = A[i][k] / A[k][k]$
 - Subtract m times row k from row i .
2. Back substitute: solve from the last equation upward.

Example:

Original:

$$\begin{array}{l} \begin{bmatrix} 2 & 1 & 1 & | & 8 \\ 4 & 3 & 3 & | & 20 \\ 2 & 3 & 1 & | & 12 \end{bmatrix} \quad \begin{array}{l} R2 = R2 - (2)R1 \\ R3 = R3 - (1)R1 \end{array} \quad \begin{bmatrix} 2 & 1 & 1 & | & 8 \\ 0 & 1 & 1 & | & 4 \\ 0 & 2 & 0 & | & 4 \end{bmatrix} \\ R3 = R3 - (2)R2 \quad \begin{bmatrix} 2 & 1 & 1 & | & 8 \\ 0 & 1 & 1 & | & 4 \\ 0 & 0 & -2 & | & -4 \end{bmatrix} \end{array}$$

$$\rightarrow \left| \begin{array}{ccc|c} 0 & 1 & 1 & 4 \\ 0 & 0 & -2 & -4 \end{array} \right|$$

Back substitute:

$$\begin{aligned} -2 * x_3 &= -4 & \rightarrow x_3 &= 2 \\ x_2 + 2 &= 4 & \rightarrow x_2 &= 2 \\ 2*x_1 + 2 + 2 &= 8 & \rightarrow x_1 &= 2 \end{aligned}$$

Gaussian elimination costs $O(n^3)$ operations. For a 1000x1000 system, that is about a billion floating-point operations. Fast, but you can do better if you need to solve multiple systems with the same A.

Partial pivoting: why it matters

Without pivoting, Gaussian elimination can fail or produce garbage. If a pivot element is zero, you divide by zero. If it is small, you amplify rounding errors.

Bad pivot:

$$\left| \begin{array}{cc|c} 0.001 & 1 & 1.001 \\ 1 & 1 & 2 \end{array} \right|$$

$$m = 1/0.001 = 1000$$

$$R2 = R2 - 1000*R1$$

$$\left| \begin{array}{cc|c} 0.001 & 1 & 1.001 \\ 0 & -999 & -999.0 \end{array} \right|$$

$$x_2 = 1.000 \text{ (correct)}$$

$$\begin{aligned} x_1 &= (1.001 - 1)/0.001 \\ &= 0.001/0.001 = 1.000 \end{aligned}$$

With partial pivoting:

Swap rows first:

$$\left| \begin{array}{cc|c} 1 & 1 & 2 \\ 0.001 & 1 & 1.001 \end{array} \right|$$

$$m = 0.001/1 = 0.001$$

$$R2 = R2 - 0.001*R1$$

$$\left| \begin{array}{cc|c} 1 & 1 & 2 \\ 0 & 0.999 & 0.999 \end{array} \right|$$

$$x_2 = 1.000 \text{ (correct)}$$

$$x_1 = (2 - 1)/1 = 1.000 \text{ (correct)}$$

Stable because the multiplier is small.

In floating-point arithmetic with limited precision, the unpivoted version can lose significant digits. Partial pivoting always selects the largest available pivot to minimize error amplification.

LU decomposition

LU decomposition factors A into a lower triangular matrix L and an upper triangular matrix U: $A = LU$. The L matrix stores the multipliers from Gaussian elimination. The U matrix is the result of elimination.

$$A = L @ U$$

$$\left| \begin{array}{ccc|ccc|ccc} 2 & 1 & 1 & 1 & 0 & 0 & 2 & 1 & 1 \\ 4 & 3 & 3 & 2 & 1 & 0 & 0 & 1 & 1 \\ 2 & 3 & 1 & 1 & 2 & 1 & 0 & 0 & -2 \end{array} \right|$$

Why factor instead of just eliminating? Because once you have L and U, solving $Ax = b$ for any new b costs only $O(n^2)$:

$$Ax = b$$

$$LUx = b$$

$$\text{Let } y = Ux:$$

$$Ly = b \quad (\text{forward substitution, } O(n^2))$$

$$Ux = y \quad (\text{back substitution, } O(n^2))$$

The $O(n^3)$ cost is paid once during factorization. Every subsequent solve is $O(n^2)$. If you need to solve 1000 systems with the same A but different b vectors, LU saves a factor of

1000/3 in total work.

With partial pivoting, you get $PA = LU$ where P is a permutation matrix recording the row swaps.

QR decomposition

QR decomposition factors A into an orthogonal matrix Q and an upper triangular matrix R : $A = QR$.

An orthogonal matrix has the property $Q^T Q = I$. Its columns are orthonormal vectors. Multiplying by Q preserves lengths and angles.

$$A = Q @ R$$

Q has orthonormal columns: $Q^T Q = I$

R is upper triangular

To solve $Ax = b$:

$$QRx = b$$

$$Rx = Q^T b \quad (\text{just multiply by } Q^T, \text{ no inversion needed})$$

Back substitute to get x .

QR is numerically more stable than LU for solving least-squares problems. The Gram-Schmidt process builds Q column by column:

Given columns $a_1, a_2, \dots$ of A :

$$q_1 = a_1 / ||a_1||$$

$$\begin{aligned} q_2 &= a_2 - (a_2 \cdot q_1) * q_1 && (\text{subtract projection onto } q_1) \\ q_2 &= q_2 / ||q_2|| && (\text{normalize}) \end{aligned}$$

$$\begin{aligned} q_3 &= a_3 - (a_3 \cdot q_1) * q_1 - (a_3 \cdot q_2) * q_2 \\ q_3 &= q_3 / ||q_3|| \end{aligned}$$

$$R[i][j] = q_i \cdot a_j \quad \text{for } i \leq j$$

Each step removes the component along all previous q vectors, leaving only the new orthogonal direction.

Cholesky decomposition

When A is symmetric ($A = A^T$) and positive definite (all eigenvalues positive), you can factor it as $A = L L^T$ where L is lower triangular. This is the Cholesky decomposition.

$$A = L @ L^T$$

$$\begin{bmatrix} 4 & 2 \\ 2 & 5 \end{bmatrix} = \begin{bmatrix} 2 & 0 \\ 1 & 2 \end{bmatrix} @ \begin{bmatrix} 2 & 1 \\ 0 & 2 \end{bmatrix}$$

$$\begin{aligned} L[i][i] &= \sqrt{A[i][i] - \sum(L[i][k]^2 \text{ for } k < i)} \\ L[i][j] &= (A[i][j] - \sum(L[i][k]*L[j][k] \text{ for } k < j)) / L[j][j] \quad \text{for } i > j \end{aligned}$$

Cholesky is twice as fast as LU and requires half the storage. It only works for symmetric positive definite matrices, but those show up constantly:

- Covariance matrices are symmetric positive semi-definite (positive definite with regularization).
- The kernel matrix in Gaussian processes is symmetric positive definite.
- The Hessian of a convex function at a minimum is symmetric positive definite.
- $A^T A$ is always symmetric positive semi-definite.

In Gaussian processes, you factor the kernel matrix K with Cholesky, then solve $K \alpha = y$ to get the predictive mean. The Cholesky factor also gives you the log-determinant for the marginal likelihood: $\log \det(K) = 2 * \sum(\log(\text{diag}(L)))$.

Least squares: when $Ax = b$ has no exact solution

If A is $m \times n$ with $m > n$ (more equations than unknowns), the system is overdetermined. There is no exact solution. Instead, you minimize the squared error:

minimize $\|Ax - b\|^2$

This is the sum of squared residuals:

$\sum((A[i,:] @ x - b[i])^2 \text{ for } i \text{ in range}(m))$

The minimizer satisfies the normal equations:

$$A^T A x = A^T b$$

Derivation: expand $\|Ax - b\|^2 = (Ax - b)^T (Ax - b) = x^T A^T A x - 2 x^T A^T b + b^T b$. Take the gradient with respect to x , set it to zero: $2 A^T A x - 2 A^T b = 0$.

Original system (overdetermined, 4 equations, 2 unknowns):

$$\begin{array}{c|c} \begin{array}{cc} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \end{array} & x \\ \hline \end{array} = \begin{array}{c} 3 \\ 5 \\ 6 \\ 8 \end{array}$$

No exact x satisfies all 4 equations.

Normal equations:

$$A^T A = \begin{array}{cc} 4 & 10 \\ 10 & 30 \end{array} \quad A^T b = \begin{array}{c} 22 \\ 63 \end{array}$$

Solve: $x = [1.5, 1.7]$

This is linear regression. $x[0]$ is the intercept, $x[1]$ is the slope.

Normal equations = linear regression

The connection is exact. In linear regression, your data matrix X has one row per sample and one column per feature. Your target vector y has one entry per sample. The weight vector w satisfies:

$$X^T X w = X^T y$$

$$w = (X^T X)^{-1} X^T y$$

This is the closed-form solution to linear regression. Every call to `sklearn.linear_model.LinearRegression` computes this (or an equivalent via QR or SVD).

Add a regularization term $\lambda * I$ to the matrix and you get ridge regression:

$$(X^T X + \lambda * I) w = X^T y$$

$$w = (X^T X + \lambda * I)^{-1} X^T y$$

The regularization makes the matrix better conditioned (easier to invert accurately) and prevents overfitting by shrinking the weights toward zero. The matrix $X^T X + \lambda I$ is always symmetric positive definite when $\lambda > 0$, so you can use Cholesky to solve it.

Pseudoinverse (Moore-Penrose)

The pseudoinverse A^+ generalizes matrix inversion to non-square and singular matrices. For any matrix A :

$$x = A^+ b$$

where $A^+ = V \Sigma^+ U^T$ (computed via SVD)

Σ^+ is formed by taking the reciprocal of each nonzero singular value and transposing the result. If $A = U \Sigma V^T$, then $A^+ = V \Sigma^+ U^T$.

$$A = U \Sigma V^T \quad (\text{SVD})$$

$$\Sigma = \begin{bmatrix} 5 & 0 \\ 0 & 2 \\ 0 & 0 \end{bmatrix} \quad \Sigma^+ = \begin{bmatrix} 1/5 & 0 & 0 \\ 0 & 1/2 & 0 \end{bmatrix}$$

$$A^+ = V \Sigma^+ U^T$$

The pseudoinverse gives the minimum-norm least-squares solution. If the system has: - One solution: $A^+ b$ gives it. - No solution: $A^+ b$ gives the least-squares solution. - Infinite solutions: $A^+ b$ gives the one with the smallest $\|x\|$.

NumPy's `np.linalg.lstsq` and `np.linalg.pinv` both use the SVD internally.

Condition number

The condition number measures how sensitive the solution is to small changes in the input. For a matrix A , the condition number is:

$$\kappa(A) = \|A\| * \|A^{-1}\| = \sigma_{\max} / \sigma_{\min}$$

where $\sigma_{\max}$ and $\sigma_{\min}$ are the largest and smallest singular values.

Well-conditioned ($\kappa \sim 1$):

Small change in $b \rightarrow$

small change in x

Ill-conditioned ($\kappa \sim 10^{15}$):

Small change in $b \rightarrow$

huge change in x

$$\begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix} \quad \kappa = 2/1 = 2$$

safe to solve

$$\begin{bmatrix} 1 & 1 \\ 1 & 1+10^{-15} \end{bmatrix} \quad \kappa \sim 10^{15}$$

solution is garbage

Rules of thumb: - $\kappa < 100$: safe, solution is accurate. - $\kappa \sim 10^k$: you lose about k digits of precision from your floating-point arithmetic. - $\kappa \sim 10^{16}$ (for float64): the solution is meaningless. The matrix is effectively singular.

In ML, ill-conditioning happens when features are nearly collinear. Regularization (adding λI) improves the condition number from $\sigma_{\max} / \sigma_{\min}$ to $(\sigma_{\max} + \lambda) / (\sigma_{\min} + \lambda)$.

Iterative methods: conjugate gradient

For very large sparse systems (millions of unknowns), direct methods like LU or Cholesky are too expensive. Iterative methods approximate the solution by improving a guess over many

iterations.

Conjugate gradient (CG) solves $Ax = b$ when A is symmetric positive definite. It finds the exact solution in at most n iterations (in exact arithmetic), but typically converges much faster if the eigenvalues of A are clustered.

Algorithm sketch:

```
x0 = initial guess (often zero)
r0 = b - A x0          (residual)
p0 = r0                (search direction)
```

```
For k = 0, 1, 2, ...:
    alpha = (rk . rk) / (pk . A pk)
    x_{k+1} = xk + alpha * pk
    r_{k+1} = rk - alpha * A pk
    beta = (r_{k+1} . r_{k+1}) / (rk . rk)
    p_{k+1} = r_{k+1} + beta * pk
    if ||r_{k+1}|| < tolerance: stop
```

CG is used in: - Large-scale optimization (Newton-CG method) - Solving PDE discretizations
- Kernel methods where the kernel matrix is too large to factor - Preconditioning for other iterative solvers

The convergence rate depends on the condition number. Better conditioned systems converge faster, which is another reason regularization helps.

The full picture: which method when

Method	Requirements	Cost	Use case
Gaussian elimination	Square, nonsingular A	$O(n^3)$	One-off solve of a square system
LU decomposition	Square, nonsingular A	$O(n^3)$ factor + $O(n^2)$ solve	Multiple solves with the same A
QR decomposition	Any A ($m \geq n$)	$O(mn^2)$	Least squares, numerically stable
Cholesky	Symmetric positive definite A	$O(n^3/3)$	Covariance matrices, Gaussian processes, ridge regression
Normal equations	Overdetermined ($m > n$)	$O(mn^2 + n^3)$	Linear regression (small n)
SVD / pseudoinverse	Any A	$O(mn^2)$	Rank-deficient systems, minimum-norm solutions
Conjugate gradient	Symmetric positive definite, sparse A	$O(n * k * nnz)$	Large sparse systems, k = iterations

Connection to ML

Every method in this lesson appears in production ML:

Linear regression. The closed-form solution solves the normal equations $X^T X w = X^T y$. This is done via Cholesky (if n is small) or QR (if numerical stability matters) or SVD (if the matrix might be rank-deficient).

Ridge regression. Adds $\lambda * I$ to $X^T X$. The regularized system $(X^T X + \lambda * I) w = X^T y$ is always solvable via Cholesky because $X^T X + \lambda * I$ is symmetric positive definite for $\lambda > 0$.

Gaussian processes. The predictive mean requires solving $K \alpha = y$ where K is the kernel matrix. Cholesky factorization of K is the standard approach. The log marginal likelihood uses $\log \det(K) = 2 \sum (\log(\text{diag}(L)))$.

Neural network initialization. Orthogonal initialization uses QR decomposition to create weight matrices whose columns are orthonormal. This prevents signal collapse in deep networks.

Preconditioning. Large-scale optimizers use incomplete Cholesky or incomplete LU as preconditioners for conjugate gradient solvers.

Feature engineering. The condition number of $X^T X$ tells you if your features are collinear. If kappa is large, drop features or add regularization.

Interactive figure: linear-system-conditioning. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/17-linear-systems>

Build It

Step 1: Gaussian elimination with partial pivoting

```
import numpy as np

def gaussian_elimination(A, b):
    n = len(b)
    Ab = np.hstack([A.astype(float), b.reshape(-1, 1).astype(float)])

    for k in range(n):
        max_row = k + np.argmax(np.abs(Ab[k:, k]))
        Ab[[k, max_row]] = Ab[[max_row, k]]

        if abs(Ab[k, k]) < 1e-12:
            raise ValueError(f"Matrix is singular or nearly singular at pivot {k}")

        for i in range(k + 1, n):
            m = Ab[i, k] / Ab[k, k]
            Ab[i, k:] -= m * Ab[k, k:]

    x = np.zeros(n)
    for i in range(n - 1, -1, -1):
        x[i] = (Ab[i, -1] - Ab[i, i+1:n] @ x[i+1:n]) / Ab[i, i]

    return x
```

Step 2: LU decomposition

```
def lu_decompose(A):
    n = A.shape[0]
    L = np.eye(n)
    U = A.astype(float).copy()
```

```

P = np.eye(n)

for k in range(n):
    max_row = k + np.argmax(np.abs(U[k:, k]))
    if max_row != k:
        U[[k, max_row]] = U[[max_row, k]]
        P[[k, max_row]] = P[[max_row, k]]
        if k > 0:
            L[[k, max_row], :k] = L[[max_row, k], :k]

    for i in range(k + 1, n):
        L[i, k] = U[i, k] / U[k, k]
        U[i, k:] -= L[i, k] * U[k, k:]

return P, L, U

def lu_solve(P, L, U, b):
    n = len(b)
    Pb = P @ b.astype(float)

    y = np.zeros(n)
    for i in range(n):
        y[i] = Pb[i] - L[i, :i] @ y[:i]

    x = np.zeros(n)
    for i in range(n - 1, -1, -1):
        x[i] = (y[i] - U[i, i+1:] @ x[i+1:]) / U[i, i]

    return x

```

Step 3: Cholesky decomposition

```

def cholesky(A):
    n = A.shape[0]
    L = np.zeros_like(A, dtype=float)

    for i in range(n):
        for j in range(i + 1):
            s = A[i, j] - L[i, :j] @ L[j, :j]
            if i == j:
                if s <= 0:
                    raise ValueError("Matrix is not positive definite")
                L[i, j] = np.sqrt(s)
            else:
                L[i, j] = s / L[j, j]

    return L

```

Step 4: Least squares via normal equations

```

def least_squares_normal(A, b):
    AtA = A.T @ A
    Atb = A.T @ b
    return gaussian_elimination(AtA, Atb)

```



```
def ridge_regression(A, b, lam):
    n = A.shape[1]
    AtA = A.T @ A + lam * np.eye(n)
    Atb = A.T @ b
    L = cholesky(AtA)
    y = np.zeros(n)
    for i in range(n):
        y[i] = (Atb[i] - L[i, :i] @ y[:i]) / L[i, i]
    x = np.zeros(n)
    for i in range(n - 1, -1, -1):
        x[i] = (y[i] - L.T[i, i+1:] @ x[i+1:]) / L.T[i, i]
    return x
```

Step 5: Condition number

```
def condition_number(A):
    U, S, Vt = np.linalg.svd(A)
    return S[0] / S[-1]
```

Use It

Putting the pieces together for linear regression and ridge regression on real data:

```
np.random.seed(42)
X_raw = np.random.randn(100, 3)
w_true = np.array([2.0, -1.0, 0.5])
y = X_raw @ w_true + np.random.randn(100) * 0.1

X = np.column_stack([np.ones(100), X_raw])

w_ols = least_squares_normal(X, y)
print(f"OLS weights (ours): {w_ols}")

w_np = np.linalg.lstsq(X, y, rcond=None)[0]
print(f"OLS weights (numpy): {w_np}")
print(f"Max difference: {np.max(np.abs(w_ols - w_np)):.2e}")

w_ridge = ridge_regression(X, y, lam=1.0)
print(f"Ridge weights (ours): {w_ridge}")

from sklearn.linear_model import Ridge
ridge_sk = Ridge(alpha=1.0, fit_intercept=False)
ridge_sk.fit(X, y)
print(f"Ridge weights (sklearn): {ridge_sk.coef_}")
```

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/17-linear-systems>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/17-linear-systems/code>

1. Solve the system $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 10 \end{bmatrix} x = \begin{bmatrix} 6 \\ 15 \\ 27 \end{bmatrix}$ using your Gaussian elimination, your LU solver, and `np.linalg.solve`. Verify all three give the same answer within floating-point tolerance.
2. Generate a 50x5 random matrix X and target $y = X @ w_{\text{true}} + \text{noise}$. Solve for w using normal equations, QR (via `np.linalg.qr`), SVD (via `np.linalg.svd`), and `np.linalg.lstsq`. Compare all four solutions. Measure the condition number of $X^T X$ and explain how it affects which method you trust.
3. Create a nearly singular matrix by making two columns almost identical (e.g., column 2 = column 1 + $1e-10 * \text{noise}$). Compute its condition number. Solve $Ax = b$ with and without regularization (add $0.01 * I$). Compare the solutions and residuals. Explain why regularization helps.
4. Implement the conjugate gradient algorithm for a 100x100 random symmetric positive definite matrix. Count how many iterations it takes to converge to tolerance $1e-8$. Compare with the theoretical maximum of n iterations.
5. Time your Cholesky solver vs your LU solver vs `np.linalg.solve` on symmetric positive definite matrices of size 10, 50, 200, 500. Plot the results. Verify Cholesky is roughly 2x faster than LU.

Key Terms

Term	What people say	What it actually means
Linear system	"Solve for x "	A set of linear equations $Ax = b$. Finding x means finding the input that produces output b under transformation A .
Gaussian elimination	"Row reduce"	Systematically zero out entries below the diagonal using row operations, producing an upper triangular system solvable by back substitution. $O(n^3)$.
Partial pivoting	"Swap rows for stability"	Before eliminating in column k , swap the row with the largest absolute value in that column to the pivot position. Prevents division by small numbers.
LU decomposition	"Factor into triangles"	Write $A = LU$ where L is lower triangular (stores multipliers) and U is upper triangular (the eliminated matrix). Amortizes the $O(n^3)$ cost over multiple solves.
QR decomposition	"Orthogonal factorization"	Write $A = QR$ where Q has orthonormal columns and R is upper triangular. More stable than LU for least squares.
Cholesky decomposition	"Square root of a matrix"	For symmetric positive definite A , write $A = LL^T$. Half the cost of LU. Used for covariance matrices, kernel matrices, and ridge regression.

Term	What people say	What it actually means
Least squares	"Best fit when exact is impossible"	Minimize the sum of squared residuals
Normal equations	"The calculus shortcut"	$A^T A x = A^T b$. Setting the gradient of
Pseudoinverse	"Inversion for non-square matrices"	$A^+ = V \Sigma^+ U^T$ via SVD. Gives the minimum-norm least-squares solution for any matrix, square or rectangular, singular or not.
Condition number	"How trustworthy is this answer"	$\kappa = \sigma_{\max} / \sigma_{\min}$. Measures sensitivity to input perturbations. Lose about $\log_{10}(\kappa)$ digits of precision.
Ridge regression	"Regularized least squares"	Solve $(X^T X + \lambda I) w = X^T y$. Adding λI improves conditioning and shrinks weights toward zero. Prevents overfitting.
Conjugate gradient	"Iterative $Ax=b$ for big matrices"	An iterative solver for symmetric positive definite systems. Converges in at most n steps. Practical for large sparse systems where factorization is too expensive.
Overdetermined system	"More data than parameters"	$m > n$ in an m -by- n system. No exact solution exists. Least squares finds the best approximation. This is every regression problem.
Back substitution	"Solve from the bottom up"	Given an upper triangular system, solve the last equation first, then substitute backward. $O(n^2)$.
Forward substitution	"Solve from the top down"	Given a lower triangular system, solve the first equation first, then substitute forward. $O(n^2)$. Used in the L step of LU solves.

Further Reading

- MIT 18.06: Linear Algebra (Gilbert Strang) - the definitive course on linear systems and matrix factorizations
- Numerical Linear Algebra (Trefethen & Bau) - the standard reference for understanding numerical stability, conditioning, and why algorithms fail
- Matrix Computations (Golub & Van Loan) - the encyclopedic reference for every matrix algorithm
- 3Blue1Brown: Inverse Matrices - visual intuition for what solving $Ax = b$ means geometrically

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/17-linear-systems>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/17-linear-systems/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/17-linear-systems>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Convex Optimization

Convex problems have one valley. Neural networks have millions. Knowing the difference matters.

Type: Build **Language:** Python **Prerequisites:** Phase 1, Lessons 04 (Calculus for ML), 08 (Optimization) **Time:** ~90 minutes

Learning Objectives

- Test whether a function is convex using the definition, second derivative, and Hessian criteria
- Implement Newton's method and compare its quadratic convergence against gradient descent
- Solve constrained optimization problems using Lagrange multipliers and interpret KKT conditions
- Explain why neural network loss landscapes are non-convex yet SGD still finds good solutions

The Problem

Lesson 08 taught you gradient descent, momentum, and Adam. Those optimizers walk downhill on any surface. But they come with no guarantees. Gradient descent on a non-convex landscape might land in a bad local minimum, get stuck on a saddle point, or oscillate forever. You used it anyway because neural networks are non-convex and there is no alternative.

But many problems in machine learning are convex. Linear regression, logistic regression, SVMs, LASSO, ridge regression. For these, something stronger exists: optimization with mathematical guarantees. A convex problem has exactly one valley. Any algorithm that walks downhill will reach the global minimum. No restarts needed. No learning rate schedules. No prayer.

Understanding convexity does three things. First, it tells you when your problem is easy (convex) versus hard (non-convex). Second, it gives you faster tools like Newton's method for convex problems. Third, it explains concepts that appear throughout ML: regularization as a constraint, duality in SVMs, and why deep learning works despite violating every nice property convexity gives you.

The Concept

Convex sets

A set S is convex if for any two points in S , the line segment between them also lies entirely in S .

Convex sets	Not convex
Rectangle: any two points inside can be connected by a line segment that stays inside	Star/crescent shape: a line between two interior points can pass outside the set
Triangle: same property holds for all interior points	Donut/annulus: the hole means some line segments leave the set
The line segment between any two points stays within the set	The line segment between some pairs of points exits the set

Formal test: for any points x, y in S and any t in $[0, 1]$, the point $tx + (1-t)y$ is also in S .

Examples of convex sets: - A line, a plane, all of $\mathbb{R}^n$ - A ball (circle, sphere, hypersphere) - A halfspace: $\{x : a^T x \leq b\}$ - The intersection of any number of convex sets

Examples of non-convex sets: - A donut (annulus) - The union of two disjoint circles - Any set with a "dent" or "hole"

Convex functions

A function f is convex if its domain is a convex set and for any two points x, y in its domain and any t in $[0, 1]$:

$$f(tx + (1-t)y) \leq t f(x) + (1-t)f(y)$$

Geometrically: the line segment between any two points on the graph lies above or on the graph.

Property	Convex function	Non-convex function
Line segment test	The line between any two points on the graph lies above or on the curve	The line between some points on the graph dips below the curve
Shape	Single bowl/valley curving upward	Multiple peaks and valleys with mixed curvature
Local minima	Every local minimum is the global minimum	Multiple local minima may exist at different heights

Common convex functions: - $f(x) = x^2$ (parabola) - $f(x) = |x|$ (absolute value) - $f(x) = e^x$ (exponential) - $f(x) = \max(0, x)$ (ReLU, though piecewise linear) - $f(x) = -\log(x)$ for $x > 0$ (negative log) - Any linear function $f(x) = a^T x + b$ (both convex and concave)

Testing for convexity

Three practical tests, from easiest to most rigorous.

Test 1: Second derivative test (1D). If $f''(x) \geq 0$ for all x , then f is convex.

- $f(x) = x^2$: $f''(x) = 2 \geq 0$. Convex.
- $f(x) = x^3$: $f''(x) = 6x$. Negative for $x < 0$. Not convex.
- $f(x) = e^x$: $f''(x) = e^x > 0$. Convex.

Test 2: Hessian test (multivariate). If the Hessian matrix $H(x)$ is positive semidefinite for all x , then f is convex. The Hessian is the matrix of second partial derivatives.

Test 3: Definition test. Check the inequality $f(tx + (1-t)y) \leq tf(x) + (1-t)f(y)$ directly. Useful for functions where derivatives are hard to compute.

Why convexity matters

The central theorem of convex optimization:

For a convex function, every local minimum is a global minimum.

This means gradient descent cannot get trapped. Any downhill path leads to the same answer. The algorithm is guaranteed to converge to the optimal solution.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/18-convex-optimization>

Consequences: - No need for random restarts - No need for sophisticated learning rate schedules - Convergence proofs are possible (rate depends on function properties) - The solution is unique (up to flat regions)

Convex vs non-convex in ML

Problem	Convex?	Why
Linear regression (MSE)	Yes	Loss is quadratic in weights
Logistic regression	Yes	Log-loss is convex in weights
SVM (hinge loss)	Yes	Maximum of linear functions
LASSO (L1 regression)	Yes	Sum of convex functions is convex
Ridge regression (L2)	Yes	Quadratic + quadratic = convex
Neural network (any loss)	No	Nonlinear activations create non-convex landscape
k-means clustering	No	Discrete assignment step
Matrix factorization	No	Product of unknowns

Linear models with convex losses are convex. The moment you add hidden layers with non-linear activations, convexity breaks.

The Hessian matrix

The Hessian H of a function $f: \mathbb{R}^n \rightarrow \mathbb{R}$ is the $n \times n$ matrix of second partial derivatives.

$$H[i][j] = d^2 f / (dx_i dx_j)$$

For $f(x, y) = x^2 + 3xy + y^2$:

$$\begin{aligned} df/dx &= 2x + 3y & d^2f/dx^2 &= 2 & d^2f/dxdy &= 3 \\ df/dy &= 3x + 2y & d^2f/dydx &= 3 & d^2f/dy^2 &= 2 \end{aligned}$$

$$H = \begin{bmatrix} 2 & 3 \\ 3 & 2 \end{bmatrix}$$

The Hessian tells you about curvature: - Eigenvalues all positive: the function curves upward in every direction (convex at that point) - Eigenvalues all negative: curves downward in every direction (concave, a local max) - Mixed signs: saddle point (curves up in some directions, down in others) - Zero eigenvalue: flat in that direction (degenerate)

For convexity, the Hessian must be positive semidefinite (all eigenvalues ≥ 0) everywhere, not just at one point.

Newton's method

Gradient descent uses first-order information (the gradient). Newton's method uses second-order information (the Hessian). It fits a quadratic approximation at the current point and jumps directly to the minimum of that quadratic.

Update rule:

$$x_{\text{new}} = x - H^{-1} * \text{gradient}$$

Compare to gradient descent:

$$x_{\text{new}} = x - \text{lr} * \text{gradient}$$

Newton's method replaces the scalar learning rate with the inverse Hessian. This automatically adjusts the step size and direction based on local curvature.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/18-convex-optimization>

Advantages: - Quadratic convergence near the minimum (error squares each step) - No learning rate to tune - Scale-invariant (works regardless of how you parameterize the problem)

Disadvantages: - Computing the Hessian costs $O(n^2)$ memory and $O(n^3)$ to invert - For a neural network with 1 million weights, that is 10^{12} entries and 10^{18} operations - Not practical for deep learning

Constrained optimization

Unconstrained optimization: minimize $f(x)$ over all x . Constrained optimization: minimize $f(x)$ subject to constraints.

Real problems have constraints. You want to minimize cost but your budget is limited. You want to minimize error but your model complexity is bounded.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/18-convex-optimization>

Lagrange multipliers

The method of Lagrange multipliers converts a constrained problem into an unconstrained one.

Problem: minimize $f(x)$ subject to $g(x) = 0$.

Solution: introduce a new variable (the Lagrange multiplier λ) and solve the unconstrained problem:

$$L(x, \lambda) = f(x) + \lambda * g(x)$$

At the solution, the gradient of L is zero:

$$dL/dx = df/dx + \lambda * dg/dx = 0$$

$$dL/d\lambda = g(x) = 0$$

Geometric intuition: at the constrained minimum, the gradient of f must be parallel to the gradient of the constraint g . If they were not parallel, you could move along the constraint surface and reduce f further.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/18-convex-optimization>

Example: minimize $f(x,y) = x^2 + y^2$ subject to $x + y = 1$.

$$L = x^2 + y^2 + \lambda(x + y - 1)$$

$$dL/dx = 2x + \lambda = 0 \Rightarrow x = -\lambda/2$$

$$dL/dy = 2y + \lambda = 0 \Rightarrow y = -\lambda/2$$

$$dL/d\lambda = x + y - 1 = 0$$

From first two: $x = y$

Substituting: $2x = 1$, so $x = y = 0.5$, $\lambda = -1$

The closest point on the line $x + y = 1$ to the origin is $(0.5, 0.5)$.

KKT conditions

The Karush-Kuhn-Tucker conditions extend Lagrange multipliers to inequality constraints.

Problem: minimize $f(x)$ subject to $g_i(x) \leq 0$ for $i = 1, \dots, m$.

The KKT conditions (necessary for optimality):

1. Stationarity: $df/dx + \sum(\lambda_i * dg_i/dx) = 0$
2. Primal feasibility: $g_i(x) \leq 0$ for all i
3. Dual feasibility: $\lambda_i \geq 0$ for all i
4. Complementary slackness: $\lambda_i * g_i(x) = 0$ for all i

Complementary slackness is the key insight: either the constraint is active ($g_i = 0$, the solution sits on the boundary) or the multiplier is zero (the constraint does not matter). A constraint that does not affect the solution has $\lambda = 0$.

KKT conditions are central to SVMs. The support vectors are the data points where the constraint is active ($\lambda > 0$). All other data points have $\lambda = 0$ and do not affect the decision boundary.

Regularization as constrained optimization

L1 and L2 regularization are not arbitrary tricks. They are constrained optimization problems in disguise.

L2 regularization (Ridge):

$$\text{minimize } \text{Loss}(w) \quad \text{subject to } ||w||^2 \leq t$$

Equivalent unconstrained form:

$$\text{minimize } \text{Loss}(w) + \lambda * ||w||^2$$

The constraint $||w||^2 \leq t$ defines a ball (circle in 2D, sphere in 3D). The solution is where the loss contours first touch this ball.

L1 regularization (LASSO):

$$\text{minimize } \text{Loss}(w) \quad \text{subject to } ||w||_1 \leq t$$

Equivalent unconstrained form:
 $\text{minimize } \text{Loss}(w) + \lambda * ||w||_1$

The constraint $||w||_1 \leq t$ defines a diamond (rotated square in 2D).

Property	L2 constraint (circle)	L1 constraint (diamond)
Constraint shape	Circle (sphere in higher dims)	Diamond (rotated square in 2D)
Where loss contour touches	Smooth boundary — any point on the circle	Corner — aligned with an axis
Solution behavior	Weights are small but nonzero	Some weights are exactly zero (sparse)
Result	Weight shrinkage	Feature selection

This explains why L1 produces sparse models (feature selection) while L2 only shrinks weights. The diamond has corners aligned with axes. Loss contours are more likely to touch a corner, setting one or more weights exactly to zero.

Duality

Every constrained optimization problem (the primal) has a companion problem (the dual). For convex problems, the primal and dual have the same optimal value. This is strong duality.

The Lagrangian dual function:

Primal: minimize $f(x)$ subject to $g(x) \leq 0$
 Lagrangian: $L(x, \lambda) = f(x) + \lambda * g(x)$
 Dual function: $d(\lambda) = \min_x L(x, \lambda)$
 Dual problem: maximize $d(\lambda)$ subject to $\lambda \geq 0$

Why duality matters: - The dual problem is sometimes easier to solve than the primal - SVMs are solved in their dual form, where the problem depends on dot products between data points (enabling the kernel trick) - The dual provides a lower bound on the primal optimum, useful for checking solution quality

For SVMs specifically:

Primal: find w, b that maximize the margin $2/||w||$ subject to
 $y_i(w^T x_i + b) \geq 1$ for all i

Dual: maximize $\sum(\alpha_i) - 0.5 * \sum_{ij}(\alpha_i * \alpha_j * y_i * y_j * x_i^T x_j)$
 subject to $\alpha_i \geq 0$ and $\sum(\alpha_i * y_i) = 0$

The dual only involves dot products $x_i^T x_j$.
 Replace $x_i^T x_j$ with $K(x_i, x_j)$ to get the kernel trick.

Why deep learning works despite non-convexity

Neural network loss functions are wildly non-convex. By every classical measure, optimizing them should fail. Yet stochastic gradient descent finds good solutions reliably. Several factors explain this.

Most local minima are good enough. In high-dimensional spaces, random critical points (where the gradient is zero) are overwhelmingly saddle points, not local minima. The few local minima that exist tend to have loss values close to the global minimum. Getting trapped

in a terrible local minimum is extremely unlikely when the parameter space has millions of dimensions.

Saddle points, not local minima, are the real obstacle. In a function with n parameters, a saddle point has a mix of positive and negative curvature directions. For a random critical point in high dimensions, the probability of all n eigenvalues being positive (local minimum) is roughly $2^{(-n)}$. Almost all critical points are saddle points. SGD's noise helps escape them.

Overparameterization smooths the landscape. Networks with more parameters than training examples have smoother, more connected loss surfaces. Wider networks have fewer bad local minima. This is counterintuitive but empirically consistent.

Loss landscape structure:

Property	Low-dimensional space	High-dimensional space
Landscape	Many isolated peaks and valleys	Smoothly connected valleys
Minima	Many isolated local minima	Few bad local minima; most are near-optimal
Navigation	Hard to find global minimum	Many paths lead to good solutions
Critical points	Mix of local minima and saddle points	Overwhelmingly saddle points, not local minima

Stochastic noise acts as implicit regularization. Mini-batch SGD adds noise that prevents settling into sharp minima. Sharp minima overfit; flat minima generalize. The noise biases optimization toward flat regions of the loss landscape.

Second-order methods in practice

Pure Newton's method is impractical for large models. Several approximations make second-order information usable.

L-BFGS (Limited-memory BFGS): Approximates the inverse Hessian using the last m gradient differences. Requires $O(mn)$ memory instead of $O(n^2)$. Works well for problems with up to $\sim 10,000$ parameters. Used in classical ML (logistic regression, CRFs) but not deep learning.

Natural gradient: Uses the Fisher information matrix (expected Hessian of the log-likelihood) instead of the standard Hessian. This accounts for the geometry of probability distributions. K-FAC (Kronecker-Factored Approximate Curvature) approximates the Fisher matrix as a Kronecker product, making it practical for neural networks.

Hessian-free optimization: Uses conjugate gradient to solve $Hx = g$ without ever forming H . Only requires Hessian-vector products, which can be computed in $O(n)$ time via automatic differentiation.

Diagonal approximations: Adam's second moment is a diagonal approximation of the Hessian's diagonal. AdaHessian extends this by using actual Hessian diagonal elements via Hutchinson's estimator.

Method	Memory	Per-step cost	When to use
Gradient descent	$O(n)$	$O(n)$	Baseline, large models

Method	Memory	Per-step cost	When to use
Newton's method	$O(n^2)$	$O(n^3)$	Small convex problems
L-BFGS	$O(mn)$	$O(mn)$	Medium convex problems
Adam	$O(n)$	$O(n)$	Deep learning default
K-FAC	$O(n)$	$O(n)$ per layer	Research, large-batch training

Interactive figure: convex-vs-nonconvex. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/18-convex-optimization>

Build It

Step 1: Convexity checker

Build a function that tests convexity empirically by sampling points and checking the definition.

```
import random
import math

def check_convexity(f, dim, bounds=(-5, 5), samples=1000):
    violations = 0
    for _ in range(samples):
        x = [random.uniform(*bounds) for _ in range(dim)]
        y = [random.uniform(*bounds) for _ in range(dim)]
        t = random.uniform(0, 1)
        mid = [t * xi + (1 - t) * yi for xi, yi in zip(x, y)]
        lhs = f(mid)
        rhs = t * f(x) + (1 - t) * f(y)
        if lhs > rhs + 1e-10:
            violations += 1
    return violations == 0, violations
```

Step 2: Newton's method for 2D

Implement Newton's method using an explicit Hessian. Compare convergence speed against gradient descent.

```
def newtons_method(f, grad_f, hessian_f, x0, steps=50, tol=1e-12):
    x = list(x0)
    history = [x[:]]
    for _ in range(steps):
        g = grad_f(x)
        H = hessian_f(x)
        det = H[0][0] * H[1][1] - H[0][1] * H[1][0]
        if abs(det) < 1e-15:
            break
        H_inv = [
            [H[1][1] / det, -H[0][1] / det],
            [-H[1][0] / det, H[0][0] / det],
        ]
```

```

]
dx = [
    H_inv[0][0] * g[0] + H_inv[0][1] * g[1],
    H_inv[1][0] * g[0] + H_inv[1][1] * g[1],
]
x = [x[0] - dx[0], x[1] - dx[1]]
history.append(x[:])
if sum(gi ** 2 for gi in g) < tol:
    break
return history

```

Step 3: Lagrange multiplier solver

Solve constrained optimization using gradient descent on the Lagrangian.

```

def lagrange_solve(f_grad, g_val, g_grad, x0, lr=0.01,
                  lr_lambda=0.01, steps=5000):
    x = list(x0)
    lam = 0.0
    history = []
    for _ in range(steps):
        fg = f_grad(x)
        gv = g_val(x)
        gg = g_grad(x)
        x = [
            xi - lr * (fgi + lam * ggi)
            for xi, fgi, ggi in zip(x, fg, gg)
        ]
        lam = lam + lr_lambda * gv
        history.append((x[:], lam, gv))
    return history

```

Step 4: Compare first-order vs second-order

Run gradient descent and Newton's method on the same quadratic function. Count the steps to convergence.

```

def quadratic(x):
    return 5 * x[0] ** 2 + x[1] ** 2

def quadratic_grad(x):
    return [10 * x[0], 2 * x[1]]

def quadratic_hessian(x):
    return [[10, 0], [0, 2]]

```

Newton's method will converge in 1 step (it is exact for quadratics). Gradient descent will take hundreds of steps because the eigenvalues of the Hessian differ by a factor of 5, creating an elongated valley.

Use It

Convexity analysis applies directly when choosing ML models and solvers.

For convex problems (logistic regression, SVMs, LASSO): - Use dedicated solvers (liblinear, CVXPY, `scipy.optimize.minimize` with `method='L-BFGS-B'`) - Expect a unique global solution - Second-order methods are practical and fast

For non-convex problems (neural networks): - Use first-order methods (SGD, Adam) - Accept that the solution depends on initialization and randomness - Use overparameterization, noise, and learning rate schedules as implicit regularization - Do not waste time searching for the global minimum. A good local minimum is sufficient.

```
from scipy.optimize import minimize

result = minimize(
    fun=lambda w: sum((y - X @ w) ** 2) + 0.1 * sum(w ** 2),
    x0=np.zeros(d),
    method='L-BFGS-B',
    jac=lambda w: -2 * X.T @ (y - X @ w) + 0.2 * w,
)
```

For SVMs, the dual formulation lets you use the kernel trick:

```
from sklearn.svm import SVC

svm = SVC(kernel='rbf', C=1.0)
svm.fit(X_train, y_train)
print(f"Support vectors: {svm.n_support_}")
```

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/18-convex-optimization/code>

1. **Convexity gallery.** Test these functions for convexity using the checker: $f(x) = x^4$, $f(x) = \sin(x)$, $f(x,y) = x^2 + y^2$, $f(x,y) = x*y$, $f(x) = \max(x, 0)$. Explain why each result makes sense.
2. **Newton vs gradient descent race.** Run both methods on $f(x,y) = 50*x^2 + y^2$ from the starting point (10, 10). How many steps does each need to reach $\text{loss} < 1e-10$? What happens to gradient descent when the condition number (ratio of largest to smallest Hessian eigenvalue) increases?
3. **Lagrange multiplier geometry.** Minimize $f(x,y) = (x-3)^2 + (y-3)^2$ subject to $x + 2y = 4$. Verify the solution by checking that the gradient of f is parallel to the gradient of g at the solution.
4. **Regularization constraint.** Implement L1-constrained optimization: minimize $(x-3)^2 + (y-2)^2$ subject to $|x| + |y| \leq 1$. Show that the solution has one coordinate equal to zero (sparsity from the diamond constraint).
5. **Hessian eigenvalue analysis.** Compute the Hessian of the Rosenbrock function at (1,1) and at (-1,1). Compute eigenvalues at both points. What do the eigenvalues tell you about the curvature at the minimum versus far from it?

Key Terms

Term	What it means
Convex set	A set where the line segment between any two points in the set stays inside the set
Convex function	A function where the line between any two points on its graph lies above or on the graph. Equivalently, Hessian is positive semidefinite everywhere
Local minimum	A point lower than all nearby points. For convex functions, every local minimum is the global minimum
Global minimum	The lowest point of a function over its entire domain
Hessian matrix	The matrix of all second partial derivatives. Encodes curvature information
Positive semidefinite	A matrix whose eigenvalues are all non-negative. The multidimensional analogue of “second derivative ≥ 0 ”
Condition number	Ratio of largest to smallest eigenvalue of the Hessian. High condition number means elongated valleys and slow gradient descent
Newton’s method	Second-order optimizer that uses the inverse Hessian to determine step direction and size. Quadratic convergence near the minimum
Lagrange multiplier	A variable introduced to convert a constrained optimization problem into an unconstrained one
KKT conditions	Necessary conditions for optimality with inequality constraints. Generalize Lagrange multipliers
Complementary slackness	At the solution, either a constraint is active or its multiplier is zero. Never both nonzero
Duality	Every constrained problem has a companion dual problem. For convex problems, both have the same optimal value
Strong duality	Primal and dual optimal values are equal. Holds for convex problems satisfying Slater’s condition
L-BFGS	Approximate second-order method that stores the last m gradient differences instead of the full Hessian
Saddle point	A point where the gradient is zero but it is a minimum in some directions and a maximum in others
Overparameterization	Using more parameters than training examples. Smooths the loss landscape and reduces bad local minima

Further Reading

- Boyd & Vandenberghe: Convex Optimization - the standard textbook, freely available online
- Bottou, Curtis, Nocedal: Optimization Methods for Large-Scale Machine Learning (2018) - bridges convex optimization theory and deep learning practice
- Choromanska et al.: The Loss Surfaces of Multilayer Networks (2015) - why non-convex neural network landscapes are not as bad as they seem
- Nocedal & Wright: Numerical Optimization - comprehensive reference for Newton’s method, L-BFGS, and constrained optimization

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/18-convex-optimization>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/18-convex-optimization/code>

- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/18-convex-optimization>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Complex Numbers for AI

The square root of -1 is not imaginary. It is the key to rotations, frequencies, and half of signal processing.

Type: Learn **Language:** Python **Prerequisites:** Phase 1, Lessons 01-04 (linear algebra, calculus) **Time:** ~60 minutes

Learning Objectives

- Perform complex arithmetic (add, multiply, divide, conjugate) in both rectangular and polar form
- Apply Euler's formula to convert between complex exponentials and trigonometric functions
- Implement the Discrete Fourier Transform using complex roots of unity
- Explain how complex rotations underlie RoPE and sinusoidal positional encodings in transformers

The Problem

You open a paper on Fourier transforms and there is i everywhere. You look at transformer positional encodings and see $\sin$ and $\cos$ at different frequencies – the real and imaginary parts of complex exponentials. You read about quantum computing and find everything expressed in complex vector spaces.

Complex numbers seem abstract. A number system built on the square root of -1 feels like a mathematical trick. But it is not a trick. It is the natural language of rotations and oscillations. Every time something spins, vibrates, or oscillates, complex numbers are the right tool.

Without understanding complex numbers, you cannot understand the Discrete Fourier Transform. You cannot understand FFT. You cannot understand how RoPE (Rotary Position Embedding) works in modern language models. You cannot understand why sinusoidal positional encodings in the original Transformer paper use the frequencies they do.

This lesson builds complex arithmetic from scratch, connects it to geometry, and shows you exactly where complex numbers appear in machine learning.

The Concept

What is a complex number?

A complex number has two parts: a real part and an imaginary part.

$$z = a + bi$$

where:

a is the real part

b is the imaginary part
i is the imaginary unit, defined by $i^2 = -1$

That is it. You extend the number line into a plane. The real numbers sit on one axis. The imaginary numbers sit on the other. Every complex number is a point in this plane.

Complex arithmetic

Addition. Add the real parts together, add the imaginary parts together.

$$(a + bi) + (c + di) = (a + c) + (b + d)i$$

Example: $(3 + 2i) + (1 + 4i) = 4 + 6i$

Multiplication. Use the distributive law and remember that $i^2 = -1$.

$$\begin{aligned}(a + bi)(c + di) &= ac + adi + bci + bdi^2 \\ &= ac + adi + bci - bd \\ &= (ac - bd) + (ad + bc)i\end{aligned}$$

$$\begin{aligned}\text{Example: } (3 + 2i)(1 + 4i) &= 3 + 12i + 2i + 8i^2 \\ &= 3 + 14i - 8 \\ &= -5 + 14i\end{aligned}$$

Conjugate. Flip the sign of the imaginary part.

$$\text{conjugate of } (a + bi) = a - bi$$

The product of a complex number and its conjugate is always real:

$$(a + bi)(a - bi) = a^2 + b^2$$

Division. Multiply numerator and denominator by the conjugate of the denominator.

$$(a + bi) / (c + di) = (a + bi)(c - di) / (c^2 + d^2)$$

This eliminates the imaginary part from the denominator, giving you a clean complex number.

The complex plane

The complex plane maps every complex number to a 2D point. The horizontal axis is the real axis, the vertical axis is the imaginary axis.

$z = 3 + 2i$ corresponds to the point (3, 2)

$z = -1 + 0i$ corresponds to the point (-1, 0) on the real axis

$z = 0 + 4i$ corresponds to the point (0, 4) on the imaginary axis

A complex number is simultaneously a point and a vector from the origin. This dual interpretation is what makes complex numbers useful for geometry.

Polar form

Any point in the plane can be described by its distance from the origin and its angle from the positive real axis.

$$z = r * (\cos(\theta) + i\sin(\theta))$$

where:

$$\begin{aligned}r &= |z| = \sqrt{a^2 + b^2} && \text{(magnitude, or modulus)} \\ \theta &= \text{atan2}(b, a) && \text{(phase, or argument)}\end{aligned}$$

Rectangular form ($a + bi$) is good for addition. Polar form (r, θ) is good for multiplication.

Multiplication in polar form. Multiply the magnitudes, add the angles.

$$\begin{aligned} z_1 &= r_1 * e^{i\theta_1} \\ z_2 &= r_2 * e^{i\theta_2} \end{aligned}$$

$$z_1 * z_2 = (r_1 * r_2) * e^{i(\theta_1 + \theta_2)}$$

This is why complex numbers are perfect for rotations. Multiplying by a complex number with magnitude 1 is a pure rotation.

Euler's formula

The bridge between complex exponentials and trigonometry:

$$e^{i\theta} = \cos(\theta) + i\sin(\theta)$$

This is the most important formula in this lesson. When $\theta = \pi$:

$$e^{i\pi} = \cos(\pi) + i\sin(\pi) = -1 + 0i = -1$$

$$\text{Therefore: } e^{i\pi} + 1 = 0$$

Five fundamental constants ($e, i, \pi, 1, 0$) linked in one equation.

Why Euler's formula matters for ML

Euler's formula says that $e^{i\theta}$ traces the unit circle as θ varies. At $\theta = 0$, you are at $(1, 0)$. At $\theta = \pi/2$, you are at $(0, 1)$. At $\theta = \pi$, you are at $(-1, 0)$. At $\theta = 3\pi/2$, you are at $(0, -1)$. A full rotation is $\theta = 2\pi$.

This means complex exponentials ARE rotations. And rotations are everywhere in signal processing and ML.

Connection to 2D rotations

Multiplying the complex number $(x + yi)$ by $e^{i\theta}$ rotates the point (x, y) by angle θ around the origin.

Rotation via complex multiplication:

$$\begin{aligned} (x + yi) * (\cos(\theta) + i\sin(\theta)) \\ = (x\cos(\theta) - y\sin(\theta)) + (x\sin(\theta) + y\cos(\theta))i \end{aligned}$$

Rotation via matrix multiplication:

$$\begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} x\cos(\theta) - y\sin(\theta) \\ x\sin(\theta) + y\cos(\theta) \end{bmatrix}$$

They produce identical results. Complex multiplication IS 2D rotation. The rotation matrix is just complex multiplication written in matrix notation.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/19-complex-numbers>

Phasors and rotating signals

A complex exponential $e^{i\omega t}$ is a point rotating around the unit circle at angular frequency ω . As t increases, the point traces the circle.

The real part of this rotating point is $\cos(\omega t)$. *The imaginary part is $\sin(\omega t)$.* A sinusoidal signal is the shadow of a rotating complex number.

$$e^{i\omega t} = \cos(\omega t) + i\sin(\omega t)$$

Real part: $\cos(\omega t)$ -- a cosine wave

Imaginary part: $\sin(\omega t)$ -- a sine wave

This is the phasor representation. Instead of tracking a wiggly sine wave, you track a smoothly rotating arrow. Phase shifts become angle offsets. Amplitude changes become magnitude changes. Addition of signals becomes vector addition.

Roots of unity

The N-th roots of unity are N points equally spaced on the unit circle:

$$w_k = e^{2\pi i k/N} \quad \text{for } k = 0, 1, 2, \dots, N-1$$

For $N = 4$, the roots are: 1, i , -1 , $-i$ (the four compass points). For $N = 8$, you get the four compass points plus the four diagonals.

Roots of unity are the foundation of the Discrete Fourier Transform. The DFT decomposes a signal into components at these N equally-spaced frequencies.

Connection to the DFT

The Discrete Fourier Transform of a signal $x[0], x[1], \dots, x[N-1]$ is:

$$X[k] = \sum_{n=0}^{N-1} x[n] * e^{-2\pi i k n/N}$$

Each $X[k]$ measures how much the signal correlates with the k-th root of unity – a complex sinusoid at frequency k. The DFT breaks a signal into N rotating phasors and tells you the amplitude and phase of each one.

Why i is not imaginary

The word “imaginary” is a historical accident. Descartes used it dismissively. But i is no more imaginary than negative numbers were when people first rejected them. Negative numbers answer “what do you subtract 5 from 3 to get?” The imaginary unit answers “what do you square to get -1?”

More usefully: i is a 90-degree rotation operator. Multiply a real number by i once, you rotate 90 degrees to the imaginary axis. Multiply by i again (i^2), you rotate another 90 degrees – now you are pointing in the negative real direction. That is why $i^2 = -1$. It is not mysterious. It is a half-turn built from two quarter-turns.

This is why complex numbers are everywhere in engineering. Anything that rotates – electromagnetic waves, quantum states, signal oscillations, positional encodings – is naturally described by complex numbers.

Complex exponentials vs trigonometric functions

Before Euler’s formula, engineers wrote signals as $A\cos(\omega t + \phi)$ – amplitude A, frequency ω , phase ϕ . This works but makes arithmetic painful. Adding two cosines with different phases requires trigonometric identities.

With complex exponentials, the same signal is $Ae^{i(\omega t + \phi)}$. Adding two signals is just adding two complex numbers. Multiplying (modulating) is just multiplying magnitudes

and adding angles. Phase shifts become angle additions. Frequency shifts become multiplications by phasors.

The entire field of signal processing switched to complex exponential notation because the math is cleaner. The “real signal” is always just the real part of the complex representation. The imaginary part is carried along as bookkeeping, making all the algebra work out naturally.

Connection to transformers

Sinusoidal positional encodings (original Transformer paper):

```
PE(pos, 2i) = sin(pos / 10000^(2i/d))
PE(pos, 2i+1) = cos(pos / 10000^(2i/d))
```

The sin and cos pairs are the real and imaginary parts of complex exponentials at different frequencies. Each frequency provides a different “resolution” for encoding position. Low frequencies change slowly (coarse position). High frequencies change quickly (fine position). Together they give each position a unique frequency fingerprint.

RoPE (Rotary Position Embedding) takes this further. It explicitly multiplies query and key vectors by complex rotation matrices. The relative position between two tokens becomes a rotation angle. Attention is computed using these rotated vectors, making the model sensitive to relative position through complex multiplication.

Operation	Algebraic Form	Geometric Meaning
Addition	$(a+c) + (b+d)i$	Vector addition in the plane
Multiplication	$(ac-bd) + (ad+bc)i$	Rotate and scale
Conjugate	$a - bi$	Reflect over real axis
Magnitude	$\sqrt{a^2 + b^2}$	Distance from origin
Phase	$\text{atan2}(b, a)$	Angle from positive real axis
Division	multiply by conjugate	Reverse rotation and rescale
Power	$r^n * e^{(in\theta)}$	Rotate n times, scale by r^n

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/19-complex-numbers>

Interactive figure: roots-of-unity. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/19-complex-numbers>

Build It

Step 1: Complex class

Build a Complex number class that supports arithmetic, magnitude, phase, and conversion between rectangular and polar forms.

```
import math

class Complex:
    def __init__(self, real, imag=0.0):
        self.real = real
        self.imag = imag
```

```

def __add__(self, other):
    return Complex(self.real + other.real, self.imag + other.imag)

def __mul__(self, other):
    r = self.real * other.real - self.imag * other.imag
    i = self.real * other.imag + self.imag * other.real
    return Complex(r, i)

def __truediv__(self, other):
    denom = other.real ** 2 + other.imag ** 2
    r = (self.real * other.real + self.imag * other.imag) / denom
    i = (self.imag * other.real - self.real * other.imag) / denom
    return Complex(r, i)

def magnitude(self):
    return math.sqrt(self.real ** 2 + self.imag ** 2)

def phase(self):
    return math.atan2(self.imag, self.real)

def conjugate(self):
    return Complex(self.real, -self.imag)

```

Step 2: Polar conversion and Euler's formula

```

def to_polar(z):
    return z.magnitude(), z.phase()

def from_polar(r, theta):
    return Complex(r * math.cos(theta), r * math.sin(theta))

def euler(theta):
    return Complex(math.cos(theta), math.sin(theta))

```

Verify: `euler(theta).magnitude()` should always be 1.0. `euler(0)` should give (1, 0). `euler(pi)` should give (-1, 0).

Step 3: Rotation

Rotating a point (x, y) by angle theta is one complex multiplication:

```

point = Complex(3, 4)
rotated = point * euler(math.pi / 4)

```

The magnitude stays the same. Only the angle changes.

Step 4: DFT from complex arithmetic

```

def dft(signal):
    N = len(signal)
    result = []
    for k in range(N):
        total = Complex(0, 0)
        for n in range(N):
            angle = -2 * math.pi * k * n / N

```

```

        total = total + Complex(signal[n], 0) * euler(angle)
    result.append(total)
    return result

```

This is the $O(N^2)$ DFT. Each output $X[k]$ is the sum of the signal samples multiplied by roots of unity.

Step 5: Inverse DFT

The inverse DFT reconstructs the original signal from its spectrum. The only changes from the forward DFT: flip the sign in the exponent and divide by N .

```

def idft(spectrum):
    N = len(spectrum)
    result = []
    for n in range(N):
        total = Complex(0, 0)
        for k in range(N):
            angle = 2 * math.pi * k * n / N
            total = total + spectrum[k] * euler(angle)
        result.append(Complex(total.real / N, total.imag / N))
    return result

```

This gives you perfect reconstruction. Apply DFT, then IDFT, and you get back the original signal to machine precision. No information is lost.

Step 6: Roots of unity

```

def roots_of_unity(N):
    return [euler(2 * math.pi * k / N) for k in range(N)]

```

Verify two properties: - Every root has magnitude exactly 1. - The sum of all N roots is zero (they cancel out by symmetry).

These properties are what make the DFT invertible. The roots of unity form an orthogonal basis for the frequency domain.

Use It

Python has built-in complex number support. The literal j represents the imaginary unit.

```

z = 3 + 2j
w = 1 + 4j

print(z + w)
print(z * w)
print(abs(z))

import cmath
print(cmath.phase(z))
print(cmath.exp(1j * cmath.pi))

```

For arrays, numpy handles complex numbers natively:

```

import numpy as np

```

```

z = np.array([1+2j, 3+4j, 5+6j])
print(np.abs(z))
print(np.angle(z))
print(np.conj(z))
print(np.real(z))
print(np.imag(z))

signal = np.sin(2 * np.pi * 5 * np.linspace(0, 1, 128))
spectrum = np.fft.fft(signal)
freqs = np.fft.fftfreq(128, d=1/128)

```

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/19-complex-numbers>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/19-complex-numbers/code>

1. **Complex arithmetic by hand.** Compute $(2 + 3i) * (4 - i)$ and verify with the code. Then compute $(5 + 2i) / (1 - 3i)$. Draw both results on the complex plane and check that multiplication rotated and scaled the first number.
2. **Rotation sequence.** Start with the point $(1, 0)$. Multiply by $e^{i\pi/6}$ twelve times. Verify that you return to $(1, 0)$ after 12 multiplications. Print the coordinates at each step and confirm they trace a regular 12-gon.
3. **DFT of a known signal.** Create a signal that is the sum of $\sin(2\pi 3t)$ and $0.5\sin(2\pi 7t)$ sampled at 32 points. Run your DFT. Verify that the magnitude spectrum has peaks at frequencies 3 and 7, with the peak at 7 being half the height of the peak at 3.
4. **Roots of unity visualization.** Compute the 8th roots of unity. Verify that they sum to zero. Verify that multiplying any root by the primitive root $e^{2\pi i/8}$ gives the next root.
5. **Rotation matrix equivalence.** For 10 random angles and 10 random points, verify that complex multiplication gives the same result as matrix-vector multiplication with the 2x2 rotation matrix. Print the maximum numerical difference.

Key Terms

Term	What it means
Complex number	A number $a + bi$ where a is the real part, b is the imaginary part, and $i^2 = -1$
Imaginary unit	The number i , defined by $i^2 = -1$. Not imaginary in the philosophical sense - it is a rotation operator
Complex plane	The 2D plane where the x-axis is real and the y-axis is imaginary. Also called the Argand plane
Magnitude (modulus)	The distance from the origin: $\sqrt{a^2 + b^2}$. Written as $ z $
Phase (argument)	The angle from the positive real axis: $\text{atan2}(b, a)$. Written as $\arg(z)$
Conjugate	The mirror image across the real axis: conjugate of $a + bi$ is $a - bi$

Term	What it means
Polar form	Expressing z as $r * e^{(i*\theta)}$ instead of $a + bi$. Makes multiplication easy
Euler's formula	$e^{(i\theta)} = \cos(\theta) + i\sin(\theta)$. Connects exponentials to trigonometry
Phasor	A rotating complex number $e^{(i\omega t)}$ representing a sinusoidal signal
Roots of unity	The N complex numbers $e^{(2\pi i * k / N)}$ for $k = 0$ to $N-1$. N equally spaced points on the unit circle
DFT	Discrete Fourier Transform. Decomposes a signal into complex sinusoidal components using roots of unity
RoPE	Rotary Position Embedding. Uses complex multiplication to encode relative position in transformer attention

Further Reading

- Visual Introduction to Euler's Formula - builds geometric intuition without heavy notation
- Su et al.: RoFormer (2021) - the paper introducing Rotary Position Embedding using complex rotations
- Vaswani et al.: Attention Is All You Need (2017) - the original Transformer paper with sinusoidal positional encodings
- 3Blue1Brown: Euler's formula with introductory group theory - visual explanation of why $e^{(i*\pi)} = -1$
- Needham: Visual Complex Analysis - the best visual treatment of complex numbers, full of geometric insight
- Strang: Introduction to Linear Algebra, Ch. 10 - complex numbers in the context of linear algebra and eigenvalues

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/19-complex-numbers>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/19-complex-numbers/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/19-complex-numbers>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

The Fourier Transform

Every signal is a sum of sine waves. The Fourier transform tells you which ones.

Type: Build **Language:** Python **Prerequisites:** Phase 1, Lessons 01-04, 19 (complex numbers) **Time:** ~90 minutes

Learning Objectives

- Implement the DFT from scratch and verify it against the $O(N \log N)$ Cooley-Tukey FFT
- Interpret frequency coefficients: extract amplitude, phase, and power spectrum from a signal
- Apply the convolution theorem to perform convolution via FFT multiplication
- Connect Fourier frequency decomposition to transformer positional encodings and CNN convolution layers

The Problem

An audio recording is a sequence of pressure measurements over time. A stock price is a sequence of values over days. An image is a grid of pixel intensities over space. All of these are data in the time domain (or space domain). You see values changing over some index.

But many patterns are invisible in the time domain. Is this audio signal a pure tone or a chord? Does this stock price have a weekly cycle? Does this image have a repeating texture? These questions are about frequency content, and the time domain hides it.

The Fourier transform converts data from the time domain to the frequency domain. It takes a signal and decomposes it into sine waves of different frequencies. Each sine wave has an amplitude (how strong it is) and a phase (where it starts). The Fourier transform tells you both.

This matters for ML because frequency-domain thinking appears everywhere. Convolutional neural networks perform convolution, which is multiplication in the frequency domain. Transformer positional encodings use frequency decomposition to represent position. Audio models (speech recognition, music generation) operate on spectrograms – frequency representations of sound. Time series models look for periodic patterns. Understanding the Fourier transform gives you the vocabulary to work with all of these.

The Concept

The DFT definition

Given N samples $x[0], x[1], \dots, x[N-1]$, the Discrete Fourier Transform produces N frequency coefficients $X[0], X[1], \dots, X[N-1]$:

$$X[k] = \sum_{n=0}^{N-1} x[n] * e^{(-2\pi i * k * n / N)}$$

for $k = 0, 1, \dots, N-1$

Each $X[k]$ is a complex number. Its magnitude $|X[k]|$ tells you the amplitude of frequency k . Its phase angle($X[k]$) tells you the phase offset of that frequency.

The key insight: $e^{(-2\pi i k n/N)}$ is a rotating phasor at frequency k . The DFT computes the correlation between the signal and each of N equally-spaced frequencies. If the signal contains energy at frequency k , the correlation is large. If not, it is near zero.

What each coefficient means

$X[0]$: the DC component. This is the sum of all samples – proportional to the mean. It represents the constant (zero-frequency) offset of the signal.

$$X[0] = \sum_{n=0}^{N-1} x[n] * e^{0} = \text{sum of all samples}$$

$X[k]$ for $1 \leq k \leq N/2$: positive frequencies. $X[k]$ represents frequency k cycles per N samples. Higher k means higher frequency (faster oscillation).

$X[N/2]$: the Nyquist frequency. The highest frequency you can represent with N samples. Above this, you get aliasing – high frequencies masquerading as low ones.

$X[k]$ for $N/2 < k < N$: negative frequencies. For real-valued signals, $X[N-k] = \text{conj}(X[k])$. The negative frequencies are mirror images of the positive ones. This is why the useful information is in the first $N/2 + 1$ coefficients.

Inverse DFT

The inverse DFT reconstructs the original signal from its frequency coefficients:

$$x[n] = (1/N) * \sum_{k=0}^{N-1} X[k] * e^{(2\pi i k n/N)}$$

for $n = 0, 1, \dots, N-1$

The only differences from the forward DFT: the sign in the exponent is positive (not negative), and there is a $1/N$ normalization factor.

The inverse DFT is perfect reconstruction. No information is lost. You can go from time domain to frequency domain and back without any error. The DFT is a change of basis – it re-expresses the same information in a different coordinate system.

The FFT: making it fast

The DFT as defined above is $O(N^2)$: for each of N output coefficients, you sum over N input samples. For $N = 1$ million, that is 10^{12} operations.

The Fast Fourier Transform (FFT) computes the same result in $O(N \log N)$. For $N = 1$ million, that is about 20 million operations instead of a trillion. This is what makes frequency analysis practical.

The Cooley-Tukey algorithm (the most common FFT) works by divide and conquer:

1. Split the signal into even-indexed and odd-indexed samples.
2. Compute the DFT of each half recursively.
3. Combine the two half-size DFTs using “twiddle factors” $e^{(-2\pi i k/N)}$.

$$\begin{aligned} X[k] &= E[k] + e^{(-2\pi i k/N)} * O[k] && \text{for } k = 0, \dots, N/2 - 1 \\ X[k + N/2] &= E[k] - e^{(-2\pi i k/N)} * O[k] && \text{for } k = 0, \dots, N/2 - 1 \end{aligned}$$

where E = DFT of even-indexed samples

0 = DFT of odd-indexed samples

The symmetry means each level of recursion does $O(N)$ work, and there are $\log_2(N)$ levels. Total: $O(N \log N)$.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/20-fourier-transform>

The FFT requires the signal length to be a power of 2. In practice, signals are zero-padded to the next power of 2.

Spectral analysis

The **power spectrum** is $|X[k]|^2$ – the squared magnitude of each frequency coefficient. It shows how much energy is at each frequency.

The **phase spectrum** is $\angle(X[k])$ – the phase offset of each frequency. For most analysis tasks, you care about the power spectrum and ignore the phase.

Power at frequency k : $P[k] = |X[k]|^2 = X[k].\text{real}^2 + X[k].\text{imag}^2$

Phase at frequency k : $\phi[k] = \text{atan2}(X[k].\text{imag}, X[k].\text{real})$

Frequency resolution

The frequency resolution of the DFT depends on the number of samples N and the sampling rate f_s .

Frequency of bin k : $f_k = k * f_s / N$

Frequency resolution: $\Delta f = f_s / N$

Maximum frequency: $f_{\text{max}} = f_s / 2$ (Nyquist)

To resolve two frequencies that are close together, you need more samples. To capture high frequencies, you need a higher sampling rate.

The convolution theorem

This is one of the most important results in signal processing and directly relevant to CNNs.

Convolution in the time domain equals pointwise multiplication in the frequency domain.

$x * h = \text{IFFT}(\text{FFT}(x) \cdot \text{FFT}(h))$

where $*$ is convolution and $\cdot$ is element-wise multiplication

Why this matters:

- Direct convolution of two signals of length N and M takes $O(N*M)$ operations.
- FFT-based convolution takes $O(N \log N)$: transform both, multiply, transform back.
- For large kernels, FFT convolution is dramatically faster.
- This is exactly what happens in convolutional layers with large receptive fields.

Note: the DFT computes circular convolution (the signal wraps around). For linear convolution (no wraparound), zero-pad both signals to length $N + M - 1$ before computing.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/20-fourier-transform>

Windowing

The DFT assumes the signal is periodic – it treats the N samples as one period of an infinitely repeating signal. If the signal does not start and end at the same value, this creates a discontinuity at the boundary, which shows up as spurious high-frequency content. This is called spectral leakage.

Windowing reduces leakage by tapering the signal to zero at both ends before computing the DFT.

Common windows:

Window	Shape	Main lobe width	Side lobe level	Use case
Rectangular	Flat (no window)	Narrowest	Highest (-13 dB)	When signal is exactly periodic in N samples
Hann	Raised cosine	Moderate	Low (-31 dB)	General purpose spectral analysis
Hamming	Modified cosine	Moderate	Lower (-42 dB)	Audio processing, speech analysis
Blackman	Triple cosine	Wide	Very low (-58 dB)	When side lobe suppression is critical

Hann window: $w[n] = 0.5 * (1 - \cos(2\pi n / (N-1)))$

Hamming window: $w[n] = 0.54 - 0.46 * \cos(2\pi n / (N-1))$

Apply the window by multiplying it element-wise with the signal before the DFT: $X = \text{DFT}(x * w)$.

DFT properties

Property	Time Domain	Frequency Domain
Linearity	$ax + by$	$aX + bY$
Time shift	$x[n - k]$	$X[f] * e^{(-2\pi i f k / N)}$
Frequency shift	$x[n] * e^{(2\pi i f_0 n / N)}$	$X[f - f_0]$
Convolution	$x * h$	$X * H$ (pointwise)
Multiplication	$x * h$ (pointwise)	$X * H$ (circular convolution, scaled by $1/N$)
Parseval's theorem	$\sum x[n] ^2$	$(1/N) * \sum X[k] ^2$
Conjugate symmetry (real input)	$x[n]$ real	$X[k] = \text{conj}(X[N-k])$

Parseval's theorem says the total energy is the same in both domains. Energy is conserved through the transform.

Connection to positional encodings

The original Transformer uses sinusoidal positional encodings:

$$\begin{aligned} \text{PE}(\text{pos}, 2i) &= \sin(\text{pos} / 10000^{(2i/d_{\text{model}})}) \\ \text{PE}(\text{pos}, 2i+1) &= \cos(\text{pos} / 10000^{(2i/d_{\text{model}})}) \end{aligned}$$

Each dimension pair $(2i, 2i+1)$ oscillates at a different frequency. The frequencies are geometrically spaced from high (dimension 0,1) to low (last dimensions). This gives each position a unique pattern across all frequency bands – similar to how Fourier coefficients uniquely identify a signal.

The key properties this provides:

- **Uniqueness:** No two positions have the same encoding.
- **Bounded values:** sin and cos are always in $[-1, 1]$.
- **Relative position:** The encoding of position $p+k$ can be expressed as a linear function of the encoding at position p . The model can learn to attend to relative positions.

Connection to CNNs

A convolution layer applies a learned filter (kernel) to the input by sliding it across the signal or image. Mathematically, this is the convolution operation.

By the convolution theorem, this is equivalent to: 1. FFT the input 2. FFT the kernel 3. Multiply in frequency domain 4. IFFT the result

Standard CNN implementations use direct convolution (faster for small 3x3 kernels). But for large kernels or global convolution, FFT-based approaches are significantly faster. Some architectures (like FNet) replace attention entirely with FFT, achieving competitive accuracy with $O(N \log N)$ instead of $O(N^2)$ complexity.

Spectrograms and the Short-Time Fourier Transform

A single FFT gives you the frequency content of the entire signal, but tells you nothing about when those frequencies occur. A chirp (a signal whose frequency increases over time) and a chord (all frequencies present simultaneously) can have the same magnitude spectrum.

The Short-Time Fourier Transform (STFT) solves this by computing FFTs on overlapping windows of the signal. The result is a spectrogram: a 2D representation with time on one axis and frequency on the other. The intensity at each point shows the energy at that frequency at that time.

STFT procedure:

1. Choose a window size (e.g., 1024 samples)
2. Choose a hop size (e.g., 256 samples -- 75% overlap)
3. For each window position:
 - a. Extract the windowed segment
 - b. Apply a Hann/Hamming window
 - c. Compute FFT
 - d. Store the magnitude spectrum as one column of the spectrogram

Spectrograms are the standard input representation for audio ML models. Speech recognition models (Whisper, DeepSpeech) operate on mel-spectrograms – spectrograms with frequencies mapped to the mel scale, which better matches human pitch perception.

Aliasing

If a signal contains frequencies above $f_s/2$ (the Nyquist frequency), sampling at rate f_s will create aliased copies. A 90 Hz signal sampled at 100 Hz looks identical to a 10 Hz signal. There is no way to distinguish them from the samples alone.

Example:

True signal: 90 Hz sine wave
 Sampling rate: 100 Hz
 Apparent frequency: $100 - 90 = 10$ Hz

The samples from the 90 Hz signal at 100 Hz sampling rate are identical to the samples from a 10 Hz signal.
 No amount of math can recover the original 90 Hz.

This is why analog-to-digital converters include anti-aliasing filters that remove frequencies above Nyquist before sampling. In ML, aliasing appears when downsampling feature maps without proper low-pass filtering – some architectures address this with anti-aliased pooling layers.

Zero-padding does not increase resolution

A common misconception: zero-padding a signal before FFT improves frequency resolution. It does not. Zero-padding interpolates between existing frequency bins, giving you a smoother-looking spectrum. But it cannot reveal frequency detail that was not present in the original samples.

True frequency resolution depends only on the observation time $T = N / f_s$. To resolve two frequencies separated by Δf , you need at least $T = 1 / \Delta f$ seconds of data. No amount of zero-padding changes this fundamental limit.

Interactive figure: fourier-synthesis. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/20-fourier-transform>

Build It

Step 1: DFT from scratch

The $O(N^2)$ DFT follows directly from the definition.

```
import math

class Complex:
    ...

def dft(x):
    N = len(x)
    result = []
    for k in range(N):
        total = Complex(0, 0)
        for n in range(N):
            angle = -2 * math.pi * k * n / N
            w = Complex(math.cos(angle), math.sin(angle))
            xn = x[n] if isinstance(x[n], Complex) else Complex(x[n])
            total = total + xn * w
```

```

        result.append(total)
    return result

```

Step 2: Inverse DFT

Same structure, positive exponent, divide by N.

```

def idft(X):
    N = len(X)
    result = []
    for n in range(N):
        total = Complex(0, 0)
        for k in range(N):
            angle = 2 * math.pi * k * n / N
            w = Complex(math.cos(angle), math.sin(angle))
            total = total + X[k] * w
        result.append(Complex(total.real / N, total.imag / N))
    return result

```

Step 3: FFT (Cooley-Tukey)

The recursive FFT requires power-of-2 length. Split into even and odd, recurse, combine with twiddle factors.

```

def fft(x):
    N = len(x)
    if N <= 1:
        return [x[0] if isinstance(x[0], Complex) else Complex(x[0])]
    if N % 2 != 0:
        return dft(x)

    even = fft([x[i] for i in range(0, N, 2)])
    odd = fft([x[i] for i in range(1, N, 2)])

    result = [Complex(0)] * N
    for k in range(N // 2):
        angle = -2 * math.pi * k / N
        twiddle = Complex(math.cos(angle), math.sin(angle))
        t = twiddle * odd[k]
        result[k] = even[k] + t
        result[k + N // 2] = even[k] - t
    return result

```

Step 4: Spectral analysis helpers

```

def power_spectrum(X):
    return [xk.real ** 2 + xk.imag ** 2 for xk in X]

def convolve_fft(x, h):
    N = len(x) + len(h) - 1
    padded_N = 1
    while padded_N < N:
        padded_N *= 2

```

```

x_padded = x + [0.0] * (padded_N - len(x))
h_padded = h + [0.0] * (padded_N - len(h))

X = fft(x_padded)
H = fft(h_padded)

Y = [xk * hk for xk, hk in zip(X, H)]

y = idft(Y)
return [y[n].real for n in range(N)]

```

Use It

For real work, use numpy's FFT which is backed by highly optimized C libraries.

```

import numpy as np

signal = np.sin(2 * np.pi * 5 * np.arange(256) / 256)
spectrum = np.fft.fft(signal)
freqs = np.fft.fftfreq(256, d=1/256)

power = np.abs(spectrum) ** 2

positive_freqs = freqs[:len(freqs)//2]
positive_power = power[:len(power)//2]

```

For windowing and more advanced spectral analysis:

```

from scipy.signal import windows, stft

window = windows.hann(256)
windowed = signal * window
spectrum = np.fft.fft(windowed)

```

For convolution:

```

from scipy.signal import fftconvolve

result = fftconvolve(signal, kernel, mode='full')

```

For spectrograms:

```

from scipy.signal import stft

frequencies, times, Zxx = stft(signal, fs=sample_rate, nperseg=256)
spectrogram = np.abs(Zxx) ** 2

```

The spectrogram matrix has shape (n_frequencies, n_time_frames). Each column is the power spectrum at one time window. This is what audio ML models consume as input.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/20-fourier-transform>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/20-fourier-transform/code>

1. **Pure tone identification.** Create a signal with a single sine wave at an unknown frequency (between 1 and 50 Hz), sampled at 128 Hz for 1 second. Use your DFT to identify the frequency. Verify the answer matches. Now add Gaussian noise with standard deviation 0.5 and repeat. How does noise affect the spectrum?
2. **FFT vs DFT verification.** Generate a random signal of length 64. Compute both DFT ($O(N^2)$) and FFT. Verify that all coefficients match to within $1e-10$. Time both functions on signals of length 256, 512, 1024, and 2048. Plot the ratio of DFT time to FFT time.
3. **Convolution theorem proof by example.** Create signal $x = [1, 2, 3, 4, 0, 0, 0, 0]$ and filter $h = [1, 1, 1, 0, 0, 0, 0, 0]$. Compute their circular convolution directly (nested loop). Then compute it via FFT (transform, multiply, inverse transform). Verify the results match. Now do linear convolution by zero-padding appropriately.
4. **Windowing effects.** Create a signal that is the sum of two sine waves at 10 Hz and 12 Hz (very close). Sample at 128 Hz for 1 second. Compute the power spectrum with no window, Hann window, and Hamming window. Which window makes it easiest to distinguish the two peaks? Why?
5. **Positional encoding analysis.** Generate the sinusoidal positional encodings for $d_{\text{model}} = 128$ and $\text{max_pos} = 512$. For each pair of positions $(p1, p2)$, compute the dot product of their encodings. Show that the dot product depends only on $|p1 - p2|$, not on the absolute positions. What happens to the dot product as the distance increases?

Key Terms

Term	What it means
DFT (Discrete Fourier Transform)	Converts N time-domain samples into N frequency-domain coefficients. Each coefficient is the correlation with a complex sinusoid at that frequency
FFT (Fast Fourier Transform)	An $O(N \log N)$ algorithm to compute the DFT. The Cooley-Tukey algorithm splits even/odd indices recursively
Inverse DFT	Reconstructs the time-domain signal from frequency coefficients. Same formula as DFT with flipped exponent sign and $1/N$ scaling
Frequency bin	Each index k in the DFT output represents frequency $k \cdot f_s / N$ Hz. The "bin" is the discrete frequency slot
DC component	$X[0]$, the zero-frequency coefficient. Proportional to the signal mean
Nyquist frequency	$f_s/2$, the maximum frequency representable at sampling rate f_s . Frequencies above this alias
Power spectrum	$ X[k] ^2$, the squared magnitude of each frequency coefficient. Shows energy distribution across frequencies
Phase spectrum	$\text{angle}(X[k])$, the phase offset of each frequency component. Often ignored in analysis
Spectral leakage	Spurious frequency content caused by treating a non-periodic signal as periodic. Reduced by windowing
Window function	A tapering function (Hann, Hamming, Blackman) applied before DFT to reduce spectral leakage

Term	What it means
Twiddle factor	The complex exponential $e^{-2\pi i k/N}$ used to combine sub-DFTs in the FFT butterfly computation
Convolution theorem	Convolution in time domain equals pointwise multiplication in frequency domain. Fundamental to signal processing and CNNs
Circular convolution	Convolution where the signal wraps around. This is what the DFT naturally computes
Linear convolution	Standard convolution without wraparound. Achieved by zero-padding before DFT
Parseval's theorem	Total energy is preserved through the Fourier transform. $\sum x[n] ^2 = (1/N) \sum X[k] ^2$
Aliasing	When frequencies above Nyquist appear as lower frequencies due to insufficient sampling rate

Further Reading

- Cooley & Tukey: An Algorithm for the Machine Calculation of Complex Fourier Series (1965) - the original FFT paper that changed computing
- 3Blue1Brown: But what is the Fourier Transform? - the best visual introduction to Fourier transforms
- Lee-Thorp et al.: FNet: Mixing Tokens with Fourier Transforms (2021) - replaces self-attention with FFT in transformers
- Smith: The Scientist and Engineer's Guide to Digital Signal Processing - free online textbook covering FFT, windowing, and spectral analysis in depth
- Vaswani et al.: Attention Is All You Need (2017) - sinusoidal positional encodings derived from Fourier frequency decomposition
- Radford et al.: Whisper (2022) - speech recognition using mel-spectrograms as input representation

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/20-fourier-transform>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/20-fourier-transform/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/20-fourier-transform>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Graph Theory for Machine Learning

Graphs are the data structure of relationships. If your data has connections, you need graph theory.

Type: Build **Language:** Python **Prerequisites:** Phase 1, Lessons 01-03 (linear algebra, matrices) **Time:** ~90 minutes

Learning Objectives

- Build a graph class with adjacency matrix/list representations and implement BFS and DFS traversals
- Compute the graph Laplacian and use its eigenvalues to detect connected components and cluster nodes
- Implement one round of GNN-style message passing as a normalized adjacency matrix multiplication
- Apply spectral clustering to partition a graph using the Fiedler vector

The Problem

Social networks, molecules, knowledge bases, citation networks, road maps – all are graphs. Traditional ML treats data as flat tables. Each row is independent. Each feature is a column. But when the structure of connections matters, tables fail.

Consider a social network. You want to predict what product a user will buy. Their purchase history matters. But their friends' purchase history matters more. The connections carry signal.

Or consider a molecule. You want to predict if it binds to a protein. The atoms matter, but what really matters is how atoms are bonded to each other. The structure is the data.

Graph Neural Networks (GNNs) are the fastest-growing area in deep learning. They power drug discovery, social recommendation, fraud detection, and knowledge graph reasoning. Every GNN builds on the same foundation: basic graph theory.

You need four things: 1. A way to represent graphs as matrices (so you can multiply them) 2. Traversal algorithms to explore graph structure 3. The Laplacian – the single most important matrix in spectral graph theory 4. Message passing – the operation that makes GNNs work

The Concept

Graphs: Nodes and Edges

A graph $G = (V, E)$ consists of vertices (nodes) V and edges E . Each edge connects two nodes.

Directed vs undirected. In an undirected graph, edge (u, v) means u connects to v AND v connects to u . In a directed graph (digraph), edge (u, v) means u points to v , but not necessarily the reverse.

Weighted vs unweighted. In an unweighted graph, edges either exist or they don't. In a weighted graph, each edge has a numerical weight – a distance, a cost, a strength.

Graph type	Example
Undirected, unweighted	Facebook friendship network
Directed, unweighted	Twitter follow network
Undirected, weighted	Road map (distances)
Directed, weighted	Web page links (PageRank scores)

The Adjacency Matrix

The adjacency matrix A is the core representation. For a graph with n nodes:

$A[i][j] = 1$ if there is an edge from node i to node j
 $A[i][j] = 0$ otherwise

For undirected graphs, A is symmetric: $A[i][j] = A[j][i]$. For weighted graphs, $A[i][j] = \text{weight of edge } (i, j)$.

Example - a triangle:

Nodes: 0, 1, 2
 Edges: (0,1), (1,2), (0,2)

$A = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{bmatrix}$

The adjacency matrix is the input to every GNN. Matrix operations on A correspond to operations on the graph.

Degree

The degree of a node is the number of edges connected to it. For directed graphs, you have in-degree (edges coming in) and out-degree (edges going out).

The degree matrix D is diagonal:

$D[i][i] = \text{degree of node } i$
 $D[i][j] = 0$ for $i \neq j$

For the triangle example: $D = \text{diag}(2, 2, 2)$ because every node connects to two others.

Degree tells you about node importance. High degree = hub node. The degree distribution of a network reveals its structure. Social networks follow power laws (few hubs, many leaf nodes). Random graphs have Poisson-distributed degrees.

BFS and DFS

The two fundamental graph traversal algorithms. You need both.

Breadth-First Search (BFS): Explore all neighbors first, then neighbors' neighbors. Uses a queue (FIFO).

BFS from node 0:

Visit 0
 Queue: [1, 2] (neighbors of 0)
 Visit 1

```

Queue: [2, 3]      (add neighbors of 1)
Visit 2
Queue: [3]        (neighbors of 2 already visited)
Visit 3
Queue: []         (done)

```

BFS finds shortest paths in unweighted graphs. The distance from the start to any node equals the BFS level at which that node is first discovered. This is why BFS is used for hop-count distances in social networks.

Depth-First Search (DFS): Go as deep as possible before backtracking. Uses a stack (LIFO) or recursion.

DFS from node 0:

```

Visit 0
Stack: [1, 2]     (neighbors of 0)
Visit 2           (pop from stack)
Stack: [1, 3]     (add neighbors of 2)
Visit 3           (pop from stack)
Stack: [1]
Visit 1           (pop from stack)
Stack: []         (done)

```

DFS is useful for: - Finding connected components (run DFS from unvisited nodes) - Cycle detection (back edges in DFS tree) - Topological sorting (reverse DFS finish order)

Algorithm	Data structure	Finds	Use case
BFS	Queue	Shortest paths	Social network distance, knowledge graph traversal
DFS	Stack	Components, cycles	Connectivity, topological sort

The Graph Laplacian

$L = D - A$. The most important matrix in spectral graph theory.

For the triangle:

```

D = [[2, 0, 0],    A = [[0, 1, 1],    L = [[2, -1, -1],
    [0, 2, 0],      [1, 0, 1],      [-1, 2, -1],
    [0, 0, 2]]      [1, 1, 0]]      [-1, -1, 2]]

```

The Laplacian has remarkable properties:

1. **L is positive semi-definite.** All eigenvalues are ≥ 0 .
2. **The number of zero eigenvalues equals the number of connected components.** A connected graph has exactly one zero eigenvalue. A graph with 3 disconnected components has three zero eigenvalues.
3. **The smallest non-zero eigenvalue (Fiedler value) measures connectivity.** A large Fiedler value means the graph is well-connected. A small Fiedler value means the graph has a weak point – a bottleneck.

4. **The eigenvector of the Fiedler value (Fiedler vector) reveals the best split.** Nodes with positive values go in one group, nodes with negative values go in the other. This is spectral clustering.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/21-graph-theory>

Spectral Properties

The eigenvalues of the adjacency matrix and Laplacian reveal structural properties without any traversal.

Spectral clustering works like this: 1. Compute the Laplacian L 2. Find the k smallest eigenvectors of L (skip the first, which is all-ones for connected graphs) 3. Use those eigenvectors as new coordinates for each node 4. Run k -means on those coordinates

Why does this work? The eigenvectors of L encode the “smoothest” functions on the graph. Nodes that are well-connected get similar eigenvector values. Nodes separated by a bottleneck get different values. The eigenvectors naturally separate clusters.

Random walk connection. The normalized Laplacian relates to random walks on the graph. The stationary distribution of a random walk is proportional to node degree. The mixing time (how fast the walk converges) depends on the spectral gap.

Message Passing

The core operation of Graph Neural Networks. Each node collects messages from its neighbors, aggregates them, and updates its own state.

$$h_v^{(k+1)} = \text{UPDATE}(h_v^{(k)}, \text{AGGREGATE}(\{h_u^{(k)} : u \in \text{neighbors}(v)\}))$$

In the simplest form, AGGREGATE = mean, and UPDATE = linear transform + activation:

$$h_v^{(k+1)} = \text{sigma}(W * \text{mean}(\{h_u^{(k)} : u \in \text{neighbors}(v)\}))$$

This is matrix multiplication in disguise. If H is the matrix of all node features and A is the adjacency matrix:

$$H^{(k+1)} = \text{sigma}(A_{\text{norm}} * H^{(k)} * W)$$

where A_{norm} is the normalized adjacency matrix (each row sums to 1).

One round of message passing lets each node “see” its immediate neighbors. Two rounds let it see neighbors of neighbors. K rounds give each node information from its K -hop neighborhood.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/21-graph-theory>

Concepts and ML Applications

Concept	ML Application
Adjacency matrix	GNN input representation
Graph Laplacian	Spectral clustering, community detection
BFS/DFS	Knowledge graph traversal, path finding
Degree distribution	Node importance, feature engineering
Message passing	GNN layers (GCN, GAT, GraphSAGE)
Eigenvalues of L	Community detection, graph partitioning

Concept	ML Application
Spectral clustering	Unsupervised node grouping
PageRank	Node importance, web search

Interactive figure: graph-degree-distribution. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/21-graph-theory>

Build It

Step 1: Graph class from scratch

```
class Graph:
    def __init__(self, n_nodes, directed=False):
        self.n = n_nodes
        self.directed = directed
        self.adj = {i: {} for i in range(n_nodes)}

    def add_edge(self, u, v, weight=1.0):
        self.adj[u][v] = weight
        if not self.directed:
            self.adj[v][u] = weight

    def neighbors(self, node):
        return list(self.adj[node].keys())

    def degree(self, node):
        return len(self.adj[node])

    def adjacency_matrix(self):
        import numpy as np
        A = np.zeros((self.n, self.n))
        for u in range(self.n):
            for v, w in self.adj[u].items():
                A[u][v] = w
        return A

    def degree_matrix(self):
        import numpy as np
        D = np.zeros((self.n, self.n))
        for i in range(self.n):
            D[i][i] = self.degree(i)
        return D

    def laplacian(self):
        return self.degree_matrix() - self.adjacency_matrix()
```

The adjacency list (`self.adj`) stores neighbors efficiently. The adjacency matrix conversion uses numpy because all the spectral operations need it.

Step 2: BFS and DFS

```

from collections import deque

def bfs(graph, start):
    visited = set()
    order = []
    distances = {}
    queue = deque([(start, 0)])
    visited.add(start)
    while queue:
        node, dist = queue.popleft()
        order.append(node)
        distances[node] = dist
        for neighbor in graph.neighbors(node):
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor, dist + 1))
    return order, distances

def dfs(graph, start):
    visited = set()
    order = []
    stack = [start]
    while stack:
        node = stack.pop()
        if node in visited:
            continue
        visited.add(node)
        order.append(node)
        for neighbor in reversed(graph.neighbors(node)):
            if neighbor not in visited:
                stack.append(neighbor)
    return order

```

BFS uses a deque (double-ended queue) for $O(1)$ `popleft`. DFS uses a list as a stack. Both visit every node exactly once – $O(V + E)$ time.

Step 3: Connected components and Laplacian eigenvalues

```

def connected_components(graph):
    visited = set()
    components = []
    for node in range(graph.n):
        if node not in visited:
            order, _ = bfs(graph, node)
            visited.update(order)
            components.append(order)
    return components

def laplacian_eigenvalues(graph):
    import numpy as np
    L = graph.laplacian()

```



```
eigenvalues = np.linalg.eigvalsh(L)
return eigenvalues
```

eigvalsh is for symmetric matrices – the Laplacian is always symmetric for undirected graphs. It returns eigenvalues in ascending order. Count the zeros to find the number of connected components.

Step 4: Spectral clustering

```
def spectral_clustering(graph, k=2):
    import numpy as np
    L = graph.laplacian()
    eigenvalues, eigenvectors = np.linalg.eigh(L)
    features = eigenvectors[:, 1:k+1]

    labels = np.zeros(graph.n, dtype=int)
    for i in range(graph.n):
        if features[i, 0] >= 0:
            labels[i] = 0
        else:
            labels[i] = 1
    return labels
```

For $k=2$, the sign of the Fiedler vector splits the graph into two clusters. For $k>2$, you would run k-means on the first k eigenvectors (excluding the trivial all-ones eigenvector).

Step 5: Message passing

```
def message_passing(graph, features, weight_matrix):
    import numpy as np
    A = graph.adjacency_matrix()
    row_sums = A.sum(axis=1, keepdims=True)
    row_sums[row_sums == 0] = 1
    A_norm = A / row_sums
    aggregated = A_norm @ features
    output = aggregated @ weight_matrix
    return output
```

This is one round of GNN message passing. Each node's new features are the weighted average of its neighbors' features, transformed by the weight matrix. Stack multiple rounds to propagate information further.

Use It

With networkx and numpy, the same operations are one-liners:

```
import networkx as nx
import numpy as np

G = nx.karate_club_graph()

A = nx.adjacency_matrix(G).toarray()
L = nx.laplacian_matrix(G).toarray()
```

```

eigenvalues = np.linalg.eigvalsh(L.astype(float))
print(f"Smallest eigenvalues: {eigenvalues[:5]}")
print(f"Connected components: {nx.number_connected_components(G)}")

communities = nx.community.greedy_modularity_communities(G)
print(f"Communities found: {len(communities)}")

pr = nx.pagerank(G)
top_nodes = sorted(pr.items(), key=lambda x: x[1], reverse=True)[:5]
print(f"Top 5 PageRank nodes: {top_nodes}")

```

networkx handles graphs of any size with optimized C backends. Use it in production. Use your from-scratch implementation to understand what it does.

numpy spectral analysis

```

import numpy as np

A = np.array([
    [0, 1, 1, 0, 0],
    [1, 0, 1, 0, 0],
    [1, 1, 0, 1, 0],
    [0, 0, 1, 0, 1],
    [0, 0, 0, 1, 0]
])

D = np.diag(A.sum(axis=1))
L = D - A

eigenvalues, eigenvectors = np.linalg.eigh(L)
print(f"Eigenvalues: {np.round(eigenvalues, 4)}")
print(f"Fiedler value: {eigenvalues[1]:.4f}")
print(f"Fiedler vector: {np.round(eigenvectors[:, 1], 4)}")

fiedler = eigenvectors[:, 1]
group_a = np.where(fiedler >= 0)[0]
group_b = np.where(fiedler < 0)[0]
print(f"Cluster A: {group_a}")
print(f"Cluster B: {group_b}")

```

The Fiedler vector does the heavy lifting. Positive entries in one cluster, negative in the other. No iterative optimization needed – just one eigendecomposition.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/21-graph-theory>

Connections

Concept	Where it shows up
Adjacency matrix	GCN, GAT, GraphSAGE input
Laplacian	Spectral clustering, ChebNet filters
BFS	Knowledge graph traversal, shortest path queries

Concept	Where it shows up
Message passing	Every GNN layer, neural message passing
Spectral gap	Graph connectivity, mixing time of random walks
Degree distribution	Power-law networks, node feature engineering
Connected components	Preprocessing, handling disconnected graphs
PageRank	Node importance ranking, attention initialization

GNNs deserve special mention. The graph convolution operation in GCN (Kipf & Welling, 2017) uses the adjacency matrix with added self-loops, $A_{\text{hat}} = A + I$:

$$H^{(l+1)} = \text{sigma}(D_{\text{hat}}^{-1/2} * A_{\text{hat}} * D_{\text{hat}}^{-1/2} * H^{(l)} * W^{(l)})$$

where $A_{\text{hat}} = A + I$ (adjacency plus self-loops) and D_{hat} is the degree matrix of A_{hat} . The self-loops ensure each node includes its own features during aggregation. This is exactly message passing with symmetric normalization. $D_{\text{hat}}^{-1/2} * A_{\text{hat}} * D_{\text{hat}}^{-1/2}$ is the normalized adjacency matrix. The Laplacian shows up because this normalization is related to $L_{\text{sym}} = I - D^{-1/2} * A * D^{-1/2}$. Understanding the Laplacian means understanding why GCNs work.

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/21-graph-theory/code>

1. **Implement PageRank from scratch.** Start with uniform scores. At each step: $\text{score}(v) = (1-d)/n + d * \sum(\text{score}(u)/\text{out_degree}(u))$ for all u pointing to v . Use $d=0.85$. Run until convergence (change $< 1e-6$). Test on a small web graph.
2. **Find communities using spectral clustering.** Create a graph with two clearly separated clusters (e.g., two cliques connected by a single edge). Run spectral clustering and verify it finds the right split. What happens as you add more cross-cluster edges?
3. **Implement Dijkstra's algorithm** for shortest paths in weighted graphs. Compare results to BFS on the same graph with uniform weights.
4. **Build a 2-layer message passing network.** Apply message passing twice with different weight matrices. Show that after 2 rounds, each node has information from its 2-hop neighborhood.
5. **Analyze a real-world graph.** Use the Karate Club graph (34 nodes, 78 edges). Compute degree distribution, Laplacian eigenvalues, and spectral clustering. Compare the spectral clustering result to the known ground truth split.

Key Terms

Term	What people say	What it actually means
Graph	"Nodes and edges"	A mathematical structure $G=(V,E)$ encoding pairwise relationships
Adjacency matrix	"The connection table"	An $n \times n$ matrix where $A[i][j] = 1$ if nodes i and j are connected
Degree	"How connected a node is"	The number of edges touching a node

Term	What people say	What it actually means
Laplacian	"D minus A"	$L = D - A$, the matrix whose eigenvalues reveal graph structure
Fiedler value	"The algebraic connectivity"	The smallest non-zero eigenvalue of L , measuring how well-connected the graph is
BFS	"Level-by-level search"	Traversal that visits all neighbors before going deeper, finds shortest paths
DFS	"Go deep first"	Traversal that follows one path to its end before backtracking
Message passing	"Nodes talk to neighbors"	Each node aggregates information from its neighbors, the core of GNNs
Spectral clustering	"Cluster by eigenvectors"	Partition a graph using eigenvectors of its Laplacian
Connected component	"A separate piece"	A maximal subgraph where every node can reach every other node

Further Reading

- **Kipf & Welling (2017)** - "Semi-Supervised Classification with Graph Convolutional Networks." The paper that launched modern GNNs. Shows that spectral graph convolutions simplify to message passing.
- **Spielman (2012)** - "Spectral Graph Theory" lecture notes. The definitive introduction to Laplacians, spectral gaps, and graph partitioning.
- **Hamilton (2020)** - "Graph Representation Learning." Book covering GNNs from fundamentals to applications.
- **Bronstein et al. (2021)** - "Geometric Deep Learning: Grids, Groups, Graphs, Geodesics, and Gauges." The unifying framework paper.
- **Veličković et al. (2018)** - "Graph Attention Networks." Extends message passing with attention mechanisms.

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/21-graph-theory>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/21-graph-theory/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/21-graph-theory>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Stochastic Processes

Randomness with structure. The math behind random walks, Markov chains, and diffusion models.

Type: Learn **Language:** Python **Prerequisites:** Phase 1, Lessons 06-07 (probability, Bayes)
Time: ~75 minutes

Learning Objectives

- Simulate 1D and 2D random walks and verify the $\sqrt{n}$ scaling of displacement
- Build a Markov chain simulator and compute its stationary distribution via eigendecomposition
- Implement Metropolis-Hastings MCMC and Langevin dynamics for sampling from target distributions
- Connect the forward diffusion process to Brownian motion and explain how the reverse process generates data

The Problem

Many AI systems involve randomness that evolves over time. Not static randomness – structured, sequential randomness where each step depends on what came before.

Language models generate tokens one at a time. Each token depends on the previous context. The model outputs a probability distribution, samples from it, and moves on. That is a stochastic process.

Diffusion models add noise to an image step by step until it becomes pure static. Then they reverse the process, denoising step by step until a new image emerges. The forward process is a Markov chain. The reverse process is a learned Markov chain running backward.

Reinforcement learning agents take actions in an environment. Each action leads to a new state with some probability. The agent follows a random policy in a random world. The whole thing is a Markov decision process.

MCMC sampling – the backbone of Bayesian inference – constructs a Markov chain whose stationary distribution is the posterior you want to sample from.

All of these build on four foundational ideas: 1. Random walks – the simplest stochastic process 2. Markov chains – structured randomness with a transition matrix 3. Langevin dynamics – gradient descent with noise 4. Metropolis-Hastings – sampling from any distribution

The Concept

Random Walks

Start at position 0. At each step, flip a fair coin. Heads: move right (+1). Tails: move left (-1).

After n steps, your position is the sum of n random ± 1 values. The expected position is 0 (the walk is unbiased). But the expected distance from the origin grows as $\sqrt{n}$.

This is counterintuitive. The walk is fair - no drift in either direction. But over time, it wanders further and further from where it started. The standard deviation after n steps is $\sqrt{n}$.

Step 0: Position = 0

Step 1: Position = +1 or -1

Step 2: Position = +2, 0, or -2

...

Step 100: Expected distance from origin ~ 10 ($\sqrt{100}$)

Step 10000: Expected distance from origin ~ 100 ($\sqrt{10000}$)

In 2D, the walk moves up, down, left, or right with equal probability. The same $\sqrt{n}$ scaling applies to the distance from the origin. The path traces a fractal-like pattern.

Why $\sqrt{n}$? Each step is ± 1 with equal probability. After n steps, the position $S_n = X_1 + X_2 + \dots + X_n$ where each X_i is ± 1 . The variance of each step is 1, and the steps are independent, so $\text{Var}(S_n) = n$. Standard deviation = $\sqrt{n}$. By the central limit theorem, $S_n / \sqrt{n}$ converges to a standard normal distribution.

This $\sqrt{n}$ scaling shows up everywhere in ML. SGD noise scales as $1/\sqrt{\text{batch_size}}$. Embedding dimensions scale as $\sqrt{d}$. The square root is the signature of independent random additions.

Connection to Brownian motion. Take a random walk with step size $1/\sqrt{n}$ and n steps per unit time. As n goes to infinity, the walk converges to Brownian motion $B(t)$ - a continuous-time process where $B(t)$ is normally distributed with mean 0 and variance t .

Brownian motion is the mathematical foundation of diffusion. It models the random jiggling of particles in a fluid, the fluctuations of stock prices, and - crucially - the noise process in diffusion models.

Gambler's ruin. A random walker starting at position k , with absorbing barriers at 0 and N . What is the probability of reaching N before 0? For a fair walk: $P(\text{reach } N) = k/N$. This is surprisingly simple and elegant. It connects to the theory of martingales - the fair random walk is a martingale (expected future value = current value).

Markov Chains

A Markov chain is a system that transitions between states according to fixed probabilities. The key property: the next state depends only on the current state, not on the history.

$$P(X_{t+1} = j \mid X_t = i, X_{t-1} = \dots) = P(X_{t+1} = j \mid X_t = i)$$

This is the Markov property. It means you can describe the entire dynamics with a transition matrix P :

$P[i][j]$ = probability of going from state i to state j

Each row of P sums to 1 (you must go somewhere).

Example - Weather:

States: Sunny (0), Rainy (1), Cloudy (2)

```
P = [[0.7, 0.1, 0.2], (if sunny: 70% sunny, 10% rainy, 20% cloudy)
      [0.3, 0.4, 0.3], (if rainy: 30% sunny, 40% rainy, 30% cloudy)
      [0.4, 0.2, 0.4]] (if cloudy: 40% sunny, 20% rainy, 40% cloudy)
```

Start in any state. After many transitions, the distribution of states converges to the stationary distribution π , where $\pi * P = \pi$. This is the left eigenvector of P with eigenvalue 1.

For the weather chain, the stationary distribution is [0.55, 0.18, 0.27] – over the long run, it is sunny 55% of the time regardless of the starting state.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/22-stochastic-processes>

Computing the stationary distribution. There are two approaches:

1. **Power method:** multiply any initial distribution by P repeatedly. After enough iterations, it converges.
2. **Eigenvalue method:** find the left eigenvector of P with eigenvalue 1. This is the eigenvector of P^T with eigenvalue 1.

Both approaches require the chain to satisfy convergence conditions.

Convergence conditions. A Markov chain converges to a unique stationary distribution if it is: - **Irreducible:** every state is reachable from every other state - **Aperiodic:** the chain does not cycle with a fixed period

Most chains you encounter in ML satisfy both conditions.

Absorbing states. A state is absorbing if once you enter it, you never leave ($P[i][i] = 1$). Absorbing Markov chains model processes with terminal states – a game that ends, a customer who churns, a token sequence that hits the end-of-text token.

Mixing time. How many steps until the chain is “close” to the stationary distribution? Formally, the number of steps until the total variation distance from stationarity drops below some threshold. Fast mixing = few steps needed. The spectral gap of P (1 minus the second-largest eigenvalue) controls the mixing time. Larger gap = faster mixing.

Connection to Language Models

Token generation in a language model is approximately a Markov process. Given the current context, the model outputs a distribution over the next token. Temperature controls the sharpness:

$$P(\text{token}_i) = \exp(\text{logit}_i / \text{temperature}) / \sum(\exp(\text{logit}_j / \text{temperature}))$$

- Temperature = 1.0: standard distribution
- Temperature < 1.0: sharper (more deterministic)
- Temperature > 1.0: flatter (more random)
- Temperature $\rightarrow 0$: argmax (greedy)

Top-k sampling truncates to the k highest-probability tokens. Top-p (nucleus) sampling truncates to the smallest set of tokens whose cumulative probability exceeds p . Both modify the Markov transition probabilities.

Brownian Motion

The continuous-time limit of the random walk. Position $B(t)$ has three properties: 1. $B(0) = 0$ 2. $B(t) - B(s)$ is normally distributed with mean 0 and variance $t - s$ (for $t > s$) 3. Increments on non-overlapping intervals are independent

Brownian motion is continuous but nowhere differentiable – it jiggles at every scale. The path has fractal dimension 2 in the plane.

In discrete simulation, you approximate Brownian motion by:

$$B(t + dt) = B(t) + \sqrt{dt} * z, \quad \text{where } z \sim N(0, 1)$$

The $\sqrt{dt}$ scaling is important. It comes from the central limit theorem applied to random walks.

Langevin Dynamics

Gradient descent finds the minimum of a function. Langevin dynamics finds the probability distribution proportional to $\exp(-U(x)/T)$, where U is an energy function and T is temperature.

$$x_{t+1} = x_t - dt * \text{gradient}(U(x_t)) + \sqrt{2 * T * dt} * z_t$$

Two forces act on the particle: 1. **Gradient force** ($-dt * \text{gradient}(U)$): pushes toward low energy (like gradient descent) 2. **Random force** ($\sqrt{2Tdt} * z$): pushes in random directions (exploration)

At temperature $T = 0$, this is pure gradient descent. At high temperature, it is nearly a random walk. At the right temperature, the particle explores the energy landscape and spends more time in low-energy regions.

Connection to diffusion models. The forward process of a diffusion model is:

$$x_t = \sqrt{\alpha_t} * x_{t-1} + \sqrt{1 - \alpha_t} * \text{noise}$$

This is a Markov chain that gradually mixes the data with noise. After enough steps, x_T is pure Gaussian noise.

The reverse process – going from noise back to data – is also a Markov chain, but its transition probabilities are learned by a neural network. The network learns to predict the noise that was added at each step, then subtracts it.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/22-stochastic-processes>

MCMC: Markov Chain Monte Carlo

Sometimes you need to sample from a distribution $p(x)$ that you can evaluate (up to a constant) but cannot sample from directly. Bayesian posteriors are the classic example – you know the likelihood times the prior, but the normalizing constant is intractable.

Metropolis-Hastings constructs a Markov chain whose stationary distribution is $p(x)$:

1. Start at some position x
2. Propose a new position x' from a proposal distribution $Q(x'|x)$
3. Compute acceptance ratio: $a = p(x') * Q(x|x') / (p(x) * Q(x'|x))$
4. Accept x' with probability $\min(1, a)$. Otherwise stay at x .
5. Repeat.

If Q is symmetric (e.g., $Q(x'|x) = Q(x|x') = N(x, \sigma^2)$), the ratio simplifies to $a = p(x') / p(x)$. You only need the ratio of probabilities – the normalizing constant cancels.

The chain is guaranteed to converge to $p(x)$ under mild conditions. But convergence can be slow if the proposal is too small (random walk) or too large (high rejection). Tuning the proposal is the art of MCMC.

Why it works. The acceptance ratio ensures detailed balance: the probability of being at x and moving to x' equals the probability of being at x' and moving to x . Detailed balance implies that $p(x)$ is the stationary distribution of the chain. So after enough steps, the samples come from $p(x)$.

Practical considerations: - **Burn-in:** discard the first N samples. The chain needs time to reach the stationary distribution from its starting point. - **Thinning:** keep every k -th sample to reduce autocorrelation. - **Multiple chains:** run several chains from different starting points. If they converge to the same distribution, you have evidence of convergence. - **Acceptance rate:** for Gaussian proposals in d dimensions, the optimal acceptance rate is about 23% (Roberts & Rosenthal, 2001). Too high means the chain barely moves. Too low means it rejects everything.

Stochastic Processes in AI

Process	AI Application
Random walk	Exploration in RL, Node2Vec embeddings
Markov chain	Text generation, MCMC sampling
Brownian motion	Diffusion models (forward process)
Langevin dynamics	Score-based generative models, SGLD
Markov decision process	Reinforcement learning
Metropolis-Hastings	Bayesian inference, posterior sampling

Interactive figure: random-walk-diffusion. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/22-stochastic-processes>

Build It

Step 1: Random walk simulator

```
import numpy as np

def random_walk_1d(n_steps, seed=None):
    rng = np.random.RandomState(seed)
    steps = rng.choice([-1, 1], size=n_steps)
    positions = np.concatenate([[0], np.cumsum(steps)])
    return positions

def random_walk_2d(n_steps, seed=None):
    rng = np.random.RandomState(seed)
    directions = rng.choice(4, size=n_steps)
    dx = np.zeros(n_steps)
    dy = np.zeros(n_steps)
    dx[directions == 0] = 1    # right
    dx[directions == 1] = -1  # left
    dy[directions == 2] = 1    # up
    dy[directions == 3] = -1  # down
    x = np.concatenate([[0], np.cumsum(dx)])
    y = np.concatenate([[0], np.cumsum(dy)])
    return x, y
```

The 1D walk stores cumulative sums. Each step is $+1$ or -1 . After n steps, the position is the sum. The variance grows linearly with n , so the standard deviation grows as $\sqrt{n}$.

Step 2: Markov chain

```
class MarkovChain:
    def __init__(self, transition_matrix, state_names=None):
        self.P = np.array(transition_matrix, dtype=float)
        self.n_states = len(self.P)
        self.state_names = state_names or [str(i) for i in range(self.n_states)]

    def step(self, current_state, rng=None):
        if rng is None:
            rng = np.random.RandomState()
        probs = self.P[current_state]
        return rng.choice(self.n_states, p=probs)

    def simulate(self, start_state, n_steps, seed=None):
        rng = np.random.RandomState(seed)
        states = [start_state]
        current = start_state
        for _ in range(n_steps):
            current = self.step(current, rng)
            states.append(current)
        return states

    def stationary_distribution(self):
        eigenvalues, eigenvectors = np.linalg.eig(self.P.T)
        idx = np.argmin(np.abs(eigenvalues - 1.0))
        stationary = np.real(eigenvectors[:, idx])
        stationary = stationary / stationary.sum()
        return np.abs(stationary)
```

The stationary distribution is the left eigenvector of P with eigenvalue 1. We find it by computing eigenvectors of P^T (transposing turns left eigenvectors into right eigenvectors).

Step 3: Langevin dynamics

```
def langevin_dynamics(grad_U, x0, dt, temperature, n_steps, seed=None):
    rng = np.random.RandomState(seed)
    x = np.array(x0, dtype=float)
    trajectory = [x.copy()]
    for _ in range(n_steps):
        noise = rng.randn(*x.shape)
        x = x - dt * grad_U(x) + np.sqrt(2 * temperature * dt) * noise
        trajectory.append(x.copy())
    return np.array(trajectory)
```

The gradient pushes x toward low energy. The noise prevents it from getting stuck. At equilibrium, the distribution of samples is proportional to $\exp(-U(x)/\text{temperature})$.

Step 4: Metropolis-Hastings

```
def metropolis_hastings(target_log_prob, proposal_std, x0, n_samples, seed=None):
    rng = np.random.RandomState(seed)
    x = np.array(x0, dtype=float)
    samples = [x.copy()]
    accepted = 0
```

```

for _ in range(n_samples - 1):
    x_proposed = x + rng.randn(*x.shape) * proposal_std
    log_ratio = target_log_prob(x_proposed) - target_log_prob(x)
    if np.log(rng.rand()) < log_ratio:
        x = x_proposed
        accepted += 1
    samples.append(x.copy())
acceptance_rate = accepted / (n_samples - 1)
return np.array(samples), acceptance_rate

```

The algorithm proposes a new point, checks if it has higher probability (or accepts with probability proportional to the ratio), and repeats. The acceptance rate should be around 23-50% for good mixing.

Use It

In practice, you use established libraries for these algorithms. But understanding the mechanics matters for debugging and tuning.

```

import numpy as np

rng = np.random.RandomState(42)
walk = np.cumsum(rng.choice([-1, 1], size=10000))
print(f"Final position: {walk[-1]}")
print(f"Expected distance: {np.sqrt(10000):.1f}")
print(f"Actual distance: {abs(walk[-1])}")

```

numpy for transition matrices

```

import numpy as np

P = np.array([[0.7, 0.1, 0.2],
              [0.3, 0.4, 0.3],
              [0.4, 0.2, 0.4]])

distribution = np.array([1.0, 0.0, 0.0])
for _ in range(100):
    distribution = distribution @ P

print(f"Stationary distribution: {np.round(distribution, 4)}")

```

Multiply the initial distribution by P repeatedly. After enough iterations, it converges to the stationary distribution regardless of where you started. This is the power method for finding the dominant left eigenvector.

Connections to real frameworks

- **PyTorch diffusion:** The DDPM Scheduler in Hugging Face diffusers implements the forward and reverse Markov chains
- **NumPyro / PyMC:** Use MCMC (NUTS sampler, which improves on Metropolis-Hastings) for Bayesian inference
- **Gymnasium (RL):** The environment step function defines a Markov decision process

Verifying Markov chain convergence

```
import numpy as np

P = np.array([[0.9, 0.1], [0.3, 0.7]])

eigenvalues = np.linalg.eigvals(P)
spectral_gap = 1 - sorted(np.abs(eigenvalues))[-2]
print(f"Eigenvalues: {eigenvalues}")
print(f"Spectral gap: {spectral_gap:.4f}")
print(f"Approximate mixing time: {1/spectral_gap:.1f} steps")
```

The spectral gap tells you how fast the chain forgets its initial state. A gap of 0.2 means roughly 5 steps to mix. A gap of 0.01 means roughly 100 steps. Always check this before running long simulations – a slowly mixing chain wastes compute.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/22-stochastic-processes>

Connections

Concept	Where it shows up
Random walk	Node2Vec graph embeddings, exploration in RL
Markov chain	Token generation in LLMs, MCMC sampling
Brownian motion	Forward diffusion process in DDPM, SDE-based models
Langevin dynamics	Score-based generative models, stochastic gradient Langevin dynamics (SGLD)
Stationary distribution	MCMC convergence target, PageRank
Metropolis-Hastings	Bayesian posterior sampling, simulated annealing
Temperature	LLM sampling, Boltzmann exploration in RL, simulated annealing
Mixing time	Convergence speed of MCMC, spectral gap analysis
Absorbing state	End-of-sequence token, terminal states in RL
Detailed balance	Correctness guarantee for MCMC samplers

Diffusion models deserve special attention. DDPM (Ho et al., 2020) defines a forward Markov chain:

$$q(x_t | x_{t-1}) = N(x_t; \sqrt{1-\beta_t} * x_{t-1}, \beta_t * I)$$

where β_t is a noise schedule. After T steps, x_T is approximately $N(0, I)$. The reverse process is parameterized by a neural network that predicts the noise:

$$p_{\theta}(x_{t-1} | x_t) = N(x_{t-1}; \mu_{\theta}(x_t, t), \sigma_t^2 * I)$$

Every step of generation is a step in a learned Markov chain. Understanding Markov chains means understanding how and why diffusion models generate data.

SGLD (Stochastic Gradient Langevin Dynamics) combines mini-batch gradient descent with Langevin noise. Instead of computing the full gradient, you use a stochastic estimate and add calibrated noise. As learning rate decays, SGLD transitions from optimization to sampling – you get approximate Bayesian posterior samples for free. This is one of the simplest ways to get uncertainty estimates from a neural network.

The key insight across all these connections: stochastic processes are not just theoretical tools. They are the computational mechanisms inside modern AI systems. When you tune the temperature of an LLM, you are adjusting a Markov chain. When you train a diffusion model, you are learning to reverse a Brownian-motion-like process. When you run Bayesian inference, you are constructing a chain that converges to the posterior.

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/22-stochastic-processes/code>

1. **Simulate 1000 random walks of 10000 steps.** Plot the distribution of final positions. Verify it is approximately Gaussian with mean 0 and standard deviation $\sqrt{10000} = 100$.
2. **Build a text generator using a Markov chain.** Train on a small corpus: for each word, count transitions to the next word. Build the transition matrix. Generate new sentences by sampling from the chain.
3. **Implement simulated annealing** using Metropolis-Hastings. Start at high temperature (accept almost everything) and gradually cool down (accept only improvements). Use it to find the minimum of a function with many local minima.
4. **Compare Langevin dynamics at different temperatures.** Sample from a double-well potential $U(x) = (x^2 - 1)^2$. At low temperature, samples cluster in one well. At high temperature, they spread across both. Find the critical temperature where the chain mixes between wells.
5. **Implement the forward diffusion process.** Start with a 1D signal (e.g., a sine wave). Add noise progressively over 100 steps with a linear noise schedule. Show how the signal degrades to pure noise. Then implement a simple denoiser that reverses the process (even a naive one that just subtracts the estimated noise).

Key Terms

Term	What people say	What it actually means
Random walk	"Coin-flip movement"	A process where position changes by random increments at each step
Markov property	"Memoryless"	The future depends only on the present state, not on the history
Transition matrix	"The probability table"	$P[i][j]$ = probability of moving from state i to state j
Stationary distribution	"The long-run average"	The distribution π where $\pi * P = \pi$ - the chain's equilibrium
Brownian motion	"Random jiggling"	The continuous-time limit of a random walk, $B(t) \sim N(0, t)$
Langevin dynamics	"Gradient descent with noise"	Update rule that combines deterministic gradient and random perturbation
MCMC	"Walking toward the target"	Constructing a Markov chain whose stationary distribution is the one you want
Metropolis-Hastings	"Propose and accept/reject"	MCMC algorithm that uses acceptance ratios to ensure convergence

Term	What people say	What it actually means
Temperature	“The randomness knob”	Parameter controlling the tradeoff between exploration and exploitation
Diffusion process	“Noise in, noise out”	Forward: gradually add noise. Reverse: gradually remove it. Generates data.

Further Reading

- **Ho, Jain, Abbeel (2020)** - “Denoising Diffusion Probabilistic Models.” The DDPM paper that launched the diffusion model revolution. Clear derivation of the forward and reverse Markov chains.
- **Song & Ermon (2019)** - “Generative Modeling by Estimating Gradients of the Data Distribution.” Score-based approach using Langevin dynamics for sampling.
- **Roberts & Rosenthal (2004)** - “General state space Markov chains and MCMC algorithms.” The theory behind when and why MCMC works.
- **Norris (1997)** - “Markov Chains.” The standard textbook. Covers convergence, stationary distributions, and hitting times.
- **Welling & Teh (2011)** - “Bayesian Learning via Stochastic Gradient Langevin Dynamics.” Combines SGD with Langevin dynamics for scalable Bayesian inference.

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/22-stochastic-processes>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/01-math-foundations/22-stochastic-processes/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/01-math-foundations/22-stochastic-processes>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Part III — ML Fundamentals

Course phase 02. Live edition with animated figures and quizzes: <https://aiengineeringfromscratch.com/catalog.html>

What Is Machine Learning

Machine learning is teaching computers to find patterns in data instead of writing rules by hand.

Type: Learn **Languages:** Python **Prerequisites:** Phase 1 (Math Foundations) **Time:** ~45 minutes

Learning Objectives

- Explain the difference between supervised, unsupervised, and reinforcement learning and identify which type applies to a given problem
- Implement a nearest centroid classifier from scratch and evaluate it against a random baseline
- Distinguish between classification and regression tasks and select the appropriate loss function for each
- Evaluate whether a given business problem is suitable for ML or better solved with deterministic rules

The Problem

You want to build a spam filter. The traditional approach: sit down and write hundreds of rules. “If the email contains ‘FREE MONEY’, mark it spam. If it has more than 3 exclamation marks, mark it spam.” You spend weeks writing rules. Then spammers change their wording. Your rules break. You write more rules. The cycle never ends.

Machine learning flips this. Instead of writing rules, you give the computer thousands of labeled emails (“spam” or “not spam”) and let it figure out the rules on its own. The computer finds patterns you never would have thought of. When spammers change tactics, you retrain on new data instead of rewriting code.

This shift from “programming rules” to “learning from data” is the core of machine learning. Every recommendation engine, voice assistant, self-driving car, and language model works this way.

The Concept

Learning From Data, Not Rules

Traditional programming and machine learning solve problems in opposite directions.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/01-what-is-machine-learning>

Traditional programming: you write the rules. The program applies them to data to produce output.

Machine learning: you provide data and expected outputs. The algorithm discovers the rules.

The “model” that comes out of training IS the rules, encoded as numbers (weights, parameters). It generalizes from examples it has seen to make predictions on data it has never seen.

The Three Types of Machine Learning

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/01-what-is-machine-learning>

Supervised Learning: You have input-output pairs. The model learns to map inputs to outputs. - “Here are 10,000 photos labeled cat or dog. Learn to tell them apart.” - “Here are house features and prices. Learn to predict the price.”

Unsupervised Learning: You have inputs only. No labels. The model finds structure on its own. - “Here are 10,000 customer purchase histories. Find natural groupings.” - “Here are 1,000 dimensional data points. Reduce to 2 dimensions while keeping structure.”

Reinforcement Learning: An agent takes actions in an environment and receives rewards or penalties. It learns a strategy (policy) to maximize total reward. - “Play this game. +1 for winning, -1 for losing. Figure out a strategy.” - “Control this robot arm. +1 for picking up the object, -0.01 for each second wasted.”

Most of what you will build in practice uses supervised learning. Unsupervised learning is common for preprocessing and exploration. Reinforcement learning powers game AI, robotics, and RLHF for language models.

Beyond the Big Three

The three categories above are clean, but real-world ML often blurs the lines.

Semi-supervised learning uses a small set of labeled data and a large set of unlabeled data. You might have 100 labeled medical images and 100,000 unlabeled ones. Techniques include:

- **Label propagation:** Build a graph connecting similar data points. Labels spread from labeled nodes to unlabeled neighbors through the graph.
- **Pseudo-labeling:** Train a model on the labeled data, use it to predict labels for unlabeled data, then retrain on everything. The model bootstraps its own training set.
- **Consistency regularization:** The model should give the same prediction for an input and a slightly perturbed version of that input. This works even without labels.

Self-supervised learning creates supervision from the data itself. No human labels needed at all. The model creates its own prediction task from the structure of the data.

- **Masked language modeling (BERT):** Hide 15% of words in a sentence, train the model to predict the missing words. The “labels” come from the original text.
- **Contrastive learning (SimCLR):** Take an image, create two augmented versions. Train the model to recognize they came from the same image while distinguishing them from augmented versions of other images.
- **Next-token prediction (GPT):** Predict the next word given all previous words. Every text document becomes a training example.

These are not separate categories from the big three. They are strategies that combine supervised and unsupervised ideas. Self-supervised learning is technically supervised (the model predicts something), but the labels are generated automatically, not by humans.

Classification vs Regression

These are the two main supervised learning tasks.

Aspect	Classification	Regression
Output	Discrete categories	Continuous numbers
Example	"Is this email spam?"	"What will the house price be?"
Output space	{cat, dog, bird}	Any real number
Loss function	Cross-entropy, accuracy	Mean squared error, MAE
Decision	Boundaries between classes	A curve that fits the data

Classification answers "which category?" Regression answers "how much?"

Some problems can be framed either way. Predicting if a stock goes up or down is classification. Predicting the exact price is regression.

The ML Workflow

Every machine learning project follows the same pipeline, regardless of the algorithm.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/01-what-is-machine-learning>

Collect Data: Gather raw data. More data is almost always better, but quality matters more than quantity.

Clean & Explore: Handle missing values, remove duplicates, visualize distributions, spot anomalies. This step often takes 60-80% of total project time.

Feature Engineering: Transform raw data into features the model can use. Turn dates into day-of-week. Normalize numerical columns. Encode categorical variables. Good features matter more than fancy algorithms.

Split Data: Divide into training, validation, and test sets. The model trains on training data, you tune hyperparameters on validation data, and you report final performance on test data.

Train Model: Feed training data into an algorithm. The algorithm adjusts internal parameters to minimize a loss function.

Evaluate: Measure performance on validation/test data. If performance is not acceptable, go back and try different features, algorithms, or hyperparameters.

Deploy: Put the model into production where it makes predictions on new data.

Monitor: Track performance over time. Data distributions change (data drift), and models degrade. When performance drops, retrain.

Training, Validation, and Test Splits

This is the most important concept beginners get wrong. You must evaluate your model on data it has never seen during training. Otherwise you are measuring memorization, not learning.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/01-what-is-machine-learning>

Split	Purpose	When used	Typical size
Training	Model learns from this data	During training	60-80%

Split	Purpose	When used	Typical size
Validation	Tune hyperparameters, compare models	After each training run	10-20%
Test	Final unbiased performance estimate	Once, at the very end	10-20%

The test set is sacred. You look at it exactly once. If you keep adjusting your model based on test performance, you are effectively training on the test set and your reported numbers are meaningless.

For small datasets, use k-fold cross-validation: split data into k parts, train on k-1 parts, validate on the remaining part, rotate, and average results.

Overfitting vs Underfitting

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/01-what-is-machine-learning>

Underfitting: The model is too simple to capture the patterns in the data. A straight line trying to fit a curved relationship. Training error is high. Test error is high.

Overfitting: The model is too complex and memorizes the training data, including its noise. A wiggly curve that passes through every training point but fails on new data. Training error is low. Test error is high.

Good fit: The model captures real patterns without memorizing noise. Training error and test error are both reasonably low.

Signs of overfitting: - Training accuracy is much higher than validation accuracy - The model performs well on training data but poorly on new data - Adding more training data improves performance (the model was memorizing, not learning)

Fixes for overfitting: - Get more training data - Reduce model complexity (fewer parameters, simpler architecture) - Regularization (add a penalty for large weights) - Dropout (randomly zero out neurons during training) - Early stopping (stop training when validation error starts increasing)

Fixes for underfitting: - Use a more complex model - Add more features - Reduce regularization - Train longer

The Bias-Variance Tradeoff

This is the mathematical framework behind overfitting and underfitting.

Bias: Error from wrong assumptions in the model. A linear model has high bias when the true relationship is nonlinear. High bias leads to underfitting.

Variance: Error from sensitivity to small fluctuations in the training data. A model with high variance gives very different predictions when trained on different subsets of data. High variance leads to overfitting.

Model complexity	Bias	Variance	Result
Too low (linear model for curved data)	High	Low	Underfitting
Just right	Medium	Medium	Good generalization
Too high (degree-20 polynomial for 10 points)	Low	High	Overfitting

Total error = Bias² + Variance + Irreducible noise

You cannot reduce irreducible noise (it is randomness in the data itself). You want to find the sweet spot where bias² + variance is minimized.

No Free Lunch Theorem

There is no single algorithm that works best for every problem. An algorithm that performs well on one class of problems will perform poorly on another. This is why data scientists try multiple algorithms and compare results.

In practice, the choice depends on: - How much data you have - How many features there are - Whether the relationship is linear or nonlinear - Whether you need interpretability - How much compute you can afford

When NOT to Use Machine Learning

ML is powerful but not always the right tool. Before reaching for a model, ask whether you actually need one.

Do not use ML when:

- **Rules are simple and well-defined.** Tax calculation, sorting algorithms, unit conversions. If you can write the logic in a few if-statements, a model adds complexity for no benefit.
- **You have no data or very little data.** ML needs examples to learn from. With 10 data points, you cannot train anything meaningful. Collect data first.
- **The cost of being wrong is catastrophic and you need guaranteed correctness.** Medical dosage calculation, nuclear reactor control, cryptographic verification. ML models are probabilistic. They will sometimes be wrong. If “sometimes wrong” is unacceptable, use deterministic methods.
- **A lookup table or heuristic solves the problem.** If a simple threshold or table covers 99% of cases, adding ML increases maintenance cost without meaningful improvement.
- **You cannot explain the decision and explainability is required.** Regulated industries (lending, insurance, criminal justice) sometimes require that every decision be fully explainable. Some ML models are interpretable (linear regression, small decision trees). Most are not.
- **The problem changes faster than you can retrain.** If the rules change daily and retraining takes a week, the model is always stale.

Use this decision flowchart:

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/01-what-is-machine-learning>

Interactive figure: f3-learning-boundary. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/01-what-is-machine-learning>

Build It

The code in `code/ml_intro.py` implements a nearest centroid classifier from scratch, the simplest possible ML algorithm. It demonstrates the core idea: learn from data, then predict on new data.

Step 1: Nearest Centroid Classifier from Scratch

The nearest centroid classifier computes the center (mean) of each class in the training data. To predict, it assigns each new point to the class whose center is closest.

```
class NearestCentroid:
    def fit(self, X, y):
        self.classes = np.unique(y)
        self.centroids = np.array([
            X[y == c].mean(axis=0) for c in self.classes
        ])

    def predict(self, X):
        distances = np.array([
            np.sqrt((X - c) ** 2).sum(axis=1))
            for c in self.centroids
        ])
        return self.classes[distances.argmin(axis=0)]
```

That is the entire algorithm. Fit computes two means. Predict computes distances. No gradient descent, no iteration, no hyperparameters.

Step 2: Train on Synthetic Data

We generate a 2D classification dataset with two classes that overlap slightly. The centroid classifier draws a linear decision boundary between the class centers.

```
rng = np.random.RandomState(42)
X_class0 = rng.randn(100, 2) + np.array([1.0, 1.0])
X_class1 = rng.randn(100, 2) + np.array([-1.0, -1.0])
X = np.vstack([X_class0, X_class1])
y = np.array([0] * 100 + [1] * 100)
```

Step 3: Compare Against a Baseline

Every ML model should be compared against a trivial baseline. Here, the baseline predicts a random class. If your ML model does not beat random guessing, something is wrong.

```
baseline_preds = rng.choice([0, 1], size=len(y_test))
baseline_acc = np.mean(baseline_preds == y_test)
```

The centroid classifier should get around 90%+ accuracy on this clean dataset. Random baseline gets around 50%.

Why This Matters

The nearest centroid classifier is trivially simple. It has no hyperparameters, no iteration, no gradient descent. Yet it captures the fundamental ML pattern:

1. **Learn** a representation from training data (the centroids)

2. **Predict** on new data using that representation (nearest distance)
3. **Evaluate** against a baseline (random guessing)

Every ML algorithm, from logistic regression to transformers, follows this same three-step pattern. The representation gets more complex, but the workflow stays the same.

Step 4: What the Centroid Classifier Cannot Do

The nearest centroid classifier assumes each class forms a single blob. It draws linear decision boundaries. It fails when:

- Classes have multiple clusters (e.g., the digit “1” can be written in several different ways)
- The decision boundary is nonlinear (e.g., one class wraps around another)
- Features have very different scales (distance is dominated by the largest-scale feature)

These limitations motivate every other algorithm you will learn. K-nearest neighbors handles multiple clusters. Decision trees handle nonlinear boundaries. Feature scaling fixes the scale problem. Each lesson builds on the limitations of the previous one.

Use It

sklearn provides NearestCentroid and synthetic data generators:

```
from sklearn.neighbors import NearestCentroid
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split

X, y = make_classification(
    n_samples=500, n_features=2, n_redundant=0,
    n_clusters_per_class=1, random_state=42
)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)

clf = NearestCentroid()
clf.fit(X_train, y_train)
print(f"Accuracy: {clf.score(X_test, y_test):.3f}")
```

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/01-what-is-machine-learning>

Key Terms

Term	What people say	What it actually means
Model	“The AI”	A mathematical function with learnable parameters that maps inputs to outputs
Training	“Teaching the AI”	Running an optimization algorithm to adjust model parameters so predictions match known outputs
Feature	“An input column”	A measurable property of the data that the model uses to make predictions

Term	What people say	What it actually means
Label	"The answer"	The known output for a training example, used to compute the error signal
Hyperparameter	"A setting you tweak"	A parameter set before training that controls the learning process (learning rate, number of layers)
Loss function	"How wrong the model is"	A function that measures the gap between predicted and actual outputs, which training tries to minimize
Overfitting	"It memorized the test"	The model learned training-specific noise instead of general patterns, so it fails on new data
Underfitting	"It didn't learn anything"	The model is too simple to capture the real patterns in the data
Generalization	"It works on new data"	The model's ability to make accurate predictions on data it was not trained on
Cross-validation	"Testing on different chunks"	Repeatedly splitting data into train/test folds and averaging results, giving a more robust performance estimate
Regularization	"Keeping weights small"	Adding a penalty term to the loss function that discourages overly complex models
Data drift	"The world changed"	The statistical distribution of incoming data shifts over time, degrading model performance

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/01-what-is-machine-learning/code>

1. Take any dataset (e.g., Iris, Titanic). Split it 70/15/15 into train/validation/test. Explain why you should not tune hyperparameters on the test set.
2. List three real-world problems. For each one, identify whether it is classification, regression, or clustering, and whether it is supervised or unsupervised.
3. A model gets 99% accuracy on training data but 60% on test data. Diagnose the problem and list three things you would try to fix it.

Further Reading

- An Introduction to Statistical Learning - free textbook covering all classical ML methods with practical examples
- Google's Machine Learning Crash Course - concise visual introduction to ML concepts
- Scikit-learn User Guide - the practical reference for implementing ML in Python

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/01-what-is-machine-learning>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/01-what-is-machine-learning/code>

- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/01-what-is-machine-learning>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Linear Regression

Linear regression draws the best straight line through your data. It is the “hello world” of machine learning.

Type: Build **Languages:** Python **Prerequisites:** Phase 1 (Linear Algebra, Calculus, Optimization), Phase 2 Lesson 1 **Time:** ~90 minutes

Learning Objectives

- Derive the gradient descent update rules for mean squared error and implement linear regression from scratch
- Compare gradient descent and the normal equation in terms of computational complexity and when to use each
- Build a multiple linear regression model with feature standardization and interpret the learned weights
- Explain how Ridge regression (L2 regularization) prevents overfitting by penalizing large weights

The Problem

You have data: house sizes and their sale prices. You want to predict the price of a new house given its size. You could eyeball it on a scatter plot, but you need a formula. You need a line that best fits the data so you can plug in any size and get a price prediction.

Linear regression gives you that line. More importantly, it introduces the entire ML training loop: define a model, define a cost function, optimize the parameters. Every ML algorithm follows this same pattern. Master it here with the simplest case, and you will recognize it everywhere.

This is not just for simple problems. Linear regression is used in production systems for demand forecasting, A/B test analysis, financial modeling, and as a baseline for every regression task.

The Concept

The Model

Linear regression assumes a linear relationship between input (x) and output (y):

$$y = wx + b$$

- w (weight/slope): how much y changes when x increases by 1
- b (bias/intercept): the value of y when x = 0

For multiple inputs (features), this extends to:

$$y = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

Or in vector form: $y = w^T * x + b$

The goal: find the values of w and b that make the predicted y as close as possible to the actual y across all training examples.

The Cost Function (Mean Squared Error)

How do you measure “as close as possible”? You need a single number that captures how wrong your predictions are. The most common choice is Mean Squared Error (MSE):

$$\text{MSE} = (1/n) * \sum((y_{\text{predicted}} - y_{\text{actual}})^2)$$

Why squared? Two reasons. First, it penalizes large errors more than small errors (an error of 10 is 100x worse than an error of 1, not 10x). Second, the squared function is smooth and differentiable everywhere, which makes optimization straightforward.

The cost function creates a surface. For a single weight w and bias b , the MSE surface looks like a bowl (a convex paraboloid). The bottom of the bowl is where MSE is minimized. Training means finding that bottom.

Gradient Descent

Gradient descent finds the bottom of the bowl by taking steps downhill.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/02-linear-regression>

The gradients tell you two things: which direction to move each parameter, and how much to move.

For MSE with $y_{\text{hat}} = wx + b$:

$$d\text{MSE}/dw = (2/n) * \sum((y_{\text{hat}} - y) * x)$$

$$d\text{MSE}/db = (2/n) * \sum(y_{\text{hat}} - y)$$

The update rule:

$$w = w - \text{learning_rate} * d\text{MSE}/dw$$

$$b = b - \text{learning_rate} * d\text{MSE}/db$$

The learning rate controls step size. Too large: you overshoot the minimum and diverge. Too small: training takes forever. Typical starting values: 0.01, 0.001, or 0.0001.

The Normal Equation (Closed-Form Solution)

For linear regression specifically, there is a direct formula that gives the optimal weights without any iteration:

$$w = (X^T * X)^{-1} * X^T * y$$

This inverts a matrix to solve for w in one step. It works perfectly for small datasets. For large datasets (millions of rows or thousands of features), gradient descent is preferred because matrix inversion is $O(n^3)$ in the number of features.

Multiple Linear Regression

With multiple features, the model becomes:

$$y = w_1 * x_1 + w_2 * x_2 + \dots + w_n * x_n + b$$

Everything works the same: MSE is the cost function, gradient descent updates all weights simultaneously. The only difference is that you are fitting a hyperplane instead of a line.

Feature scaling matters here. If one feature ranges from 0 to 1 and another ranges from 0 to 1,000,000, gradient descent will struggle because the cost surface becomes elongated. Standardize features (subtract mean, divide by standard deviation) before training.

Polynomial Regression

What if the relationship is not linear? You can still use linear regression by creating polynomial features:

$$y = w_1x + w_2x^2 + w_3x^3 + b$$

This is still “linear” regression because the model is linear in the weights (w_1 , w_2 , w_3). You are just using nonlinear features of x .

Higher-degree polynomials can fit more complex curves but risk overfitting. A degree-10 polynomial will pass through every point in a 10-point dataset but predict poorly on new data.

R-Squared Score

MSE tells you how wrong you are, but the number depends on the scale of y . R-squared (R^2) gives a scale-independent measure:

$$\begin{aligned} R^2 &= 1 - (\text{sum of squared residuals}) / (\text{sum of squared deviations from mean}) \\ &= 1 - SS_{\text{res}} / SS_{\text{tot}} \end{aligned}$$

- $R^2 = 1.0$: perfect predictions
- $R^2 = 0.0$: the model is no better than predicting the mean every time
- $R^2 < 0.0$: the model is worse than predicting the mean

Regularization Preview (Ridge Regression)

When you have many features, the model can overfit by assigning large weights. Ridge regression (L2 regularization) adds a penalty:

$$\text{Cost} = \text{MSE} + \lambda \sum (w_i^2)$$

The penalty term discourages large weights. The hyperparameter λ controls the trade-off: higher λ means smaller weights and more regularization. This is covered in depth in a later lesson. For now, know that it exists and why it helps.

Interactive figure: linear-regression-fit. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/02-linear-regression>

Build It

Step 1: Generate sample data

```
import random
import math

random.seed(42)
```

```

TRUE_W = 3.0
TRUE_B = 7.0
N_SAMPLES = 100

X = [random.uniform(0, 10) for _ in range(N_SAMPLES)]
y = [TRUE_W * x + TRUE_B + random.gauss(0, 2.0) for x in X]

print(f"Generated {N_SAMPLES} samples")
print(f"True relationship: y = {TRUE_W}x + {TRUE_B} (+ noise)")
print(f"First 5 points: {[round(X[i], 2), round(y[i], 2)] for i in range(5)}")

```

Step 2: Linear regression from scratch with gradient descent

```

class LinearRegression:
    def __init__(self, learning_rate=0.01):
        self.w = 0.0
        self.b = 0.0
        self.lr = learning_rate
        self.cost_history = []

    def predict(self, X):
        return [self.w * x + self.b for x in X]

    def compute_cost(self, X, y):
        predictions = self.predict(X)
        n = len(y)
        cost = sum((pred - actual) ** 2 for pred, actual in zip(predictions, y)) / n
        return cost

    def compute_gradients(self, X, y):
        predictions = self.predict(X)
        n = len(y)
        dw = (2 / n) * sum((pred - actual) * x for pred, actual, x in zip(predictions, y, X))
        db = (2 / n) * sum(pred - actual for pred, actual in zip(predictions, y))
        return dw, db

    def fit(self, X, y, epochs=1000, print_every=200):
        for epoch in range(epochs):
            dw, db = self.compute_gradients(X, y)
            self.w -= self.lr * dw
            self.b -= self.lr * db
            cost = self.compute_cost(X, y)
            self.cost_history.append(cost)
            if epoch % print_every == 0:
                print(f" Epoch {epoch:4d} | Cost: {cost:.4f} | w: {self.w:.4f} | b: {self.b:.4f}")
        return self

    def r_squared(self, X, y):
        predictions = self.predict(X)
        y_mean = sum(y) / len(y)
        ss_res = sum((actual - pred) ** 2 for actual, pred in zip(y, predictions))
        ss_tot = sum((actual - y_mean) ** 2 for actual in y)
        return 1 - (ss_res / ss_tot)

```

```

print("=== Training Linear Regression (Gradient Descent) ===")
model = LinearRegression(learning_rate=0.005)
model.fit(X, y, epochs=1000, print_every=200)
print(f"\nLearned: y = {model.w:.4f}x + {model.b:.4f}")
print(f"True:      y = {TRUE_W}x + {TRUE_B}")
print(f"R-squared: {model.r_squared(X, y):.4f}")

```

Step 3: Normal equation (closed-form solution)

```

class LinearRegressionNormal:
    def __init__(self):
        self.w = 0.0
        self.b = 0.0

    def fit(self, X, y):
        n = len(X)
        x_mean = sum(X) / n
        y_mean = sum(y) / n
        numerator = sum((X[i] - x_mean) * (y[i] - y_mean) for i in range(n))
        denominator = sum((X[i] - x_mean) ** 2 for i in range(n))
        self.w = numerator / denominator
        self.b = y_mean - self.w * x_mean
        return self

    def predict(self, X):
        return [self.w * x + self.b for x in X]

    def r_squared(self, X, y):
        predictions = self.predict(X)
        y_mean = sum(y) / len(y)
        ss_res = sum((actual - pred) ** 2 for actual, pred in zip(y, predictions))
        ss_tot = sum((actual - y_mean) ** 2 for actual in y)
        return 1 - (ss_res / ss_tot)

print("\n=== Normal Equation (Closed-Form) ===")
model_normal = LinearRegressionNormal()
model_normal.fit(X, y)
print(f"Learned: y = {model_normal.w:.4f}x + {model_normal.b:.4f}")
print(f"R-squared: {model_normal.r_squared(X, y):.4f}")

```

Step 4: Multiple linear regression

```

class MultipleLinearRegression:
    def __init__(self, n_features, learning_rate=0.01):
        self.weights = [0.0] * n_features
        self.bias = 0.0
        self.lr = learning_rate
        self.cost_history = []

    def predict_single(self, x):
        return sum(w * xi for w, xi in zip(self.weights, x)) + self.bias

```

```

def predict(self, X):
    return [self.predict_single(x) for x in X]

def compute_cost(self, X, y):
    predictions = self.predict(X)
    n = len(y)
    return sum((pred - actual) ** 2 for pred, actual in zip(predictions, y)) / n

def fit(self, X, y, epochs=1000, print_every=200):
    n = len(y)
    n_features = len(X[0])
    for epoch in range(epochs):
        predictions = self.predict(X)
        errors = [pred - actual for pred, actual in zip(predictions, y)]
        for j in range(n_features):
            grad = (2 / n) * sum(errors[i] * X[i][j] for i in range(n))
            self.weights[j] -= self.lr * grad
        grad_b = (2 / n) * sum(errors)
        self.bias -= self.lr * grad_b
        cost = self.compute_cost(X, y)
        self.cost_history.append(cost)
        if epoch % print_every == 0:
            print(f" Epoch {epoch:4d} | Cost: {cost:.4f}")
    return self

def r_squared(self, X, y):
    predictions = self.predict(X)
    y_mean = sum(y) / len(y)
    ss_res = sum((actual - pred) ** 2 for actual, pred in zip(y, predictions))
    ss_tot = sum((actual - y_mean) ** 2 for actual in y)
    return 1 - (ss_res / ss_tot)

random.seed(42)
N = 100
X_multi = []
y_multi = []
for _ in range(N):
    size = random.uniform(500, 3000)
    bedrooms = random.randint(1, 5)
    age = random.uniform(0, 50)
    price = 50 * size + 10000 * bedrooms - 1000 * age + 50000 + random.gauss(0, 20000)
    X_multi.append([size, bedrooms, age])
    y_multi.append(price)

def standardize(X):
    n_features = len(X[0])
    means = [sum(X[i][j] for i in range(len(X))) / len(X) for j in range(n_features)]
    stds = []
    for j in range(n_features):
        variance = sum((X[i][j] - means[j]) ** 2 for i in range(len(X))) / len(X)
        stds.append(variance ** 0.5)
    X_scaled = []

```

```

    for i in range(len(X)):
        row = [(X[i][j] - means[j]) / stds[j] if stds[j] > 0 else 0 for j in range(n_features)]
        X_scaled.append(row)
    return X_scaled, means, stds

y_mean_val = sum(y_multi) / len(y_multi)
y_std_val = (sum((yi - y_mean_val) ** 2 for yi in y_multi) / len(y_multi)) ** 0.5
y_scaled = [(yi - y_mean_val) / y_std_val for yi in y_multi]

X_scaled, x_means, x_stds = standardize(X_multi)

print("\n=== Multiple Linear Regression (3 features) ===")
print("Features: house size, bedrooms, age")
multi_model = MultipleLinearRegression(n_features=3, learning_rate=0.01)
multi_model.fit(X_scaled, y_scaled, epochs=1000, print_every=200)

print(f"\nWeights (standardized): {[round(w, 4) for w in multi_model.weights]}")
print(f"Bias (standardized): {multi_model.bias:.4f}")
print(f"R-squared: {multi_model.r_squared(X_scaled, y_scaled):.4f}")

```

Step 5: Polynomial regression

```

class PolynomialRegression:
    def __init__(self, degree, learning_rate=0.01):
        self.degree = degree
        self.weights = [0.0] * degree
        self.bias = 0.0
        self.lr = learning_rate

    def make_features(self, X):
        return [[x ** (d + 1) for d in range(self.degree)] for x in X]

    def predict(self, X):
        features = self.make_features(X)
        return [sum(w * f for w, f in zip(self.weights, row)) + self.bias for row in features]

    def fit(self, X, y, epochs=1000, print_every=200):
        features = self.make_features(X)
        n = len(y)
        for epoch in range(epochs):
            predictions = [sum(w * f for w, f in zip(self.weights, row)) + self.bias for row in features]
            errors = [pred - actual for pred, actual in zip(predictions, y)]
            for j in range(self.degree):
                grad = (2 / n) * sum(errors[i] * features[i][j] for i in range(n))
                self.weights[j] -= self.lr * grad
            grad_b = (2 / n) * sum(errors)
            self.bias -= self.lr * grad_b
            if epoch % print_every == 0:
                cost = sum(e ** 2 for e in errors) / n
                print(f"Epoch {epoch:4d} | Cost: {cost:.6f}")
        return self

    def r_squared(self, X, y):

```

```

    predictions = self.predict(X)
    y_mean = sum(y) / len(y)
    ss_res = sum((actual - pred) ** 2 for actual, pred in zip(y, predictions))
    ss_tot = sum((actual - y_mean) ** 2 for actual in y)
    return 1 - (ss_res / ss_tot)

random.seed(42)
X_poly = [x / 10.0 for x in range(0, 50)]
y_poly = [0.5 * x ** 2 - 2 * x + 3 + random.gauss(0, 1.0) for x in X_poly]

x_max = max(abs(x) for x in X_poly)
X_poly_norm = [x / x_max for x in X_poly]
y_poly_mean = sum(y_poly) / len(y_poly)
y_poly_std = (sum((yi - y_poly_mean) ** 2 for yi in y_poly) / len(y_poly)) ** 0.5
y_poly_norm = [(yi - y_poly_mean) / y_poly_std for yi in y_poly]

print("\n=== Polynomial Regression (degree 2 vs degree 5) ===")
print("True relationship: y = 0.5x^2 - 2x + 3")

print("\nDegree 2:")
poly2 = PolynomialRegression(degree=2, learning_rate=0.1)
poly2.fit(X_poly_norm, y_poly_norm, epochs=2000, print_every=500)
print(f" R-squared: {poly2.r_squared(X_poly_norm, y_poly_norm):.4f}")

print("\nDegree 5:")
poly5 = PolynomialRegression(degree=5, learning_rate=0.1)
poly5.fit(X_poly_norm, y_poly_norm, epochs=2000, print_every=500)
print(f" R-squared: {poly5.r_squared(X_poly_norm, y_poly_norm):.4f}")

print("\nDegree 2 fits the true curve well. Degree 5 fits training data slightly better")
print("but risks overfitting on new data.")

```

Step 6: Ridge regression (L2 regularization)

```

class RidgeRegression:
    def __init__(self, n_features, learning_rate=0.01, alpha=1.0):
        self.weights = [0.0] * n_features
        self.bias = 0.0
        self.lr = learning_rate
        self.alpha = alpha

    def predict_single(self, x):
        return sum(w * xi for w, xi in zip(self.weights, x)) + self.bias

    def predict(self, X):
        return [self.predict_single(x) for x in X]

    def fit(self, X, y, epochs=1000, print_every=200):
        n = len(y)
        n_features = len(X[0])
        for epoch in range(epochs):
            predictions = self.predict(X)
            errors = [pred - actual for pred, actual in zip(predictions, y)]

```



```

mse = sum(e ** 2 for e in errors) / n
reg_term = self.alpha * sum(w ** 2 for w in self.weights)
cost = mse + reg_term
for j in range(n_features):
    grad = (2 / n) * sum(errors[i] * X[i][j] for i in range(n))
    grad += 2 * self.alpha * self.weights[j]
    self.weights[j] -= self.lr * grad
grad_b = (2 / n) * sum(errors)
self.bias -= self.lr * grad_b
if epoch % print_every == 0:
    print(f" Epoch {epoch:4d} | Cost: {cost:.4f} | L2 penalty: {reg_term:.4f}")
return self

print("\n=== Ridge Regression (L2 Regularization) ===")
print("Same data as multiple regression, with alpha=0.1")
ridge = RidgeRegression(n_features=3, learning_rate=0.01, alpha=0.1)
ridge.fit(X_scaled, y_scaled, epochs=1000, print_every=200)
print(f"\nRidge weights: {[round(w, 4) for w in ridge.weights]}")
print(f"Plain weights: {[round(w, 4) for w in multi_model.weights]}")
print("Ridge weights are smaller (shrunk toward zero) due to the L2 penalty.")

```

Use It

Now the same thing with scikit-learn, which is what you will actually use in production.

```

from sklearn.linear_model import LinearRegression as SklearnLR
from sklearn.linear_model import Ridge
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np

```

```

np.random.seed(42)
X_sk = np.random.uniform(0, 10, (100, 1))
y_sk = 3.0 * X_sk.squeeze() + 7.0 + np.random.normal(0, 2.0, 100)

```

```

X_train, X_test, y_train, y_test = train_test_split(X_sk, y_sk, test_size=0.2, random_state=42)

```

```

lr = SklearnLR()
lr.fit(X_train, y_train)
y_pred = lr.predict(X_test)

```

```

print("=== Scikit-learn Linear Regression ===")
print(f"Coefficient (w): {lr.coef_[0]:.4f}")
print(f"Intercept (b): {lr.intercept_:.4f}")
print(f"R-squared (test): {r2_score(y_test, y_pred):.4f}")
print(f"MSE (test): {mean_squared_error(y_test, y_pred):.4f}")

```

```

poly = PolynomialFeatures(degree=2, include_bias=False)
X_poly_sk = poly.fit_transform(X_train)
X_poly_test = poly.transform(X_test)

```

```

lr_poly = SklearnLR()
lr_poly.fit(X_poly_sk, y_train)
print(f"\nPolynomial degree 2 R-squared: {r2_score(y_test, lr_poly.predict(X_poly_test)):.4f}")

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

ridge = Ridge(alpha=1.0)
ridge.fit(X_train_scaled, y_train)
print(f"Ridge R-squared: {r2_score(y_test, ridge.predict(X_test_scaled)):.4f}")
print(f"Ridge coefficient: {ridge.coef_[0]:.4f}")

```

Your from-scratch implementation and scikit-learn produce the same results. The difference: scikit-learn handles edge cases, numerical stability, and performance optimizations. Use the library for production. Use the from-scratch version to understand what is happening.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/02-linear-regression>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/02-linear-regression/code>

1. Implement batch gradient descent, stochastic gradient descent (SGD), and mini-batch gradient descent. Compare convergence speed on the same dataset. Which converges fastest? Which has the smoothest cost curve?
2. Generate data from a cubic function ($y = ax^3 + bx^2 + cx + d + \text{noise}$). Fit polynomials of degree 1, 3, and 10. Compare training R^2 and test R^2 . At what degree does overfitting become obvious?
3. Implement Lasso regression (L1 regularization: $\text{penalty} = \alpha * \sum(|w_i|)$). Train on the multi-feature housing data. Compare which weights go to zero vs Ridge. Why does L1 produce sparse solutions while L2 does not?

Key Terms

Term	What people say	What it actually means
Linear regression	"Draw a line through data"	Find weight w and bias b that minimize the sum of squared differences between $wx+b$ and actual y values
Cost function	"How bad the model is"	A function that maps model parameters to a single number measuring prediction error, which optimization minimizes
Mean squared error	"Average of squared errors"	$(1/n) * \text{sum of } (\text{predicted} - \text{actual})^2$, penalizing large errors disproportionately
Gradient descent	"Walk downhill"	Iteratively adjust parameters in the direction that reduces the cost function, using partial derivatives

Term	What people say	What it actually means
Learning rate	“Step size”	A scalar that controls how much parameters change per gradient descent step
Normal equation	“Solve it directly”	The closed-form solution $w = (X^T X)^{-1} X^T y$ that gives optimal weights without iteration
R-squared	“How good the fit is”	The fraction of variance in y explained by the model, ranging from negative infinity to 1.0
Feature scaling	“Make features comparable”	Transforming features to similar ranges (e.g., zero mean, unit variance) so gradient descent converges faster
Regularization	“Penalize complexity”	Adding a term to the cost function that shrinks weights, preventing overfitting
Ridge regression	“L2 regularization”	Linear regression with a penalty of $\lambda \sum (w_i^2)$ added to MSE
Polynomial regression	“Fitting curves with linear math”	Linear regression on polynomial features (x , x^2 , x^3 , ...), still linear in the weights
Overfitting	“Memorizing training data”	Using a model so complex that it fits noise in training data and fails on new data

Further Reading

- An Introduction to Statistical Learning (ISLR) – free PDF, chapters 3 and 6 cover linear regression and regularization with practical R examples
- The Elements of Statistical Learning (ESL) – free PDF, the more mathematical companion to ISLR with deeper treatment of ridge and lasso
- Stanford CS229 Lecture Notes on Linear Regression – Andrew Ng’s notes deriving the normal equation and gradient descent from first principles
- scikit-learn LinearRegression documentation – practical reference for LinearRegression, Ridge, Lasso, and ElasticNet with code examples

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/02-linear-regression>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/02-linear-regression/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/02-linear-regression>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Logistic Regression

Logistic regression bends a straight line into an S-curve to answer yes-or-no questions with probabilities.

Type: Build **Languages:** Python **Prerequisites:** Phase 2 Lesson 1-2 (What Is ML, Linear Regression) **Time:** ~90 minutes

Learning Objectives

- Implement logistic regression from scratch using the sigmoid function and binary cross-entropy loss
- Compute and interpret precision, recall, F1 score, and the confusion matrix for binary classification
- Explain why MSE fails for classification and why binary cross-entropy produces a convex cost surface
- Build a softmax regression model for multi-class classification and evaluate threshold tuning tradeoffs

The Problem

You want to predict whether a tumor is malignant or benign given its size. You try linear regression. It outputs numbers like 0.3 or 1.7 or -0.5. What do those mean? Is 1.7 “very malignant”? Is -0.5 “very benign”? Linear regression outputs unbounded numbers. Classification needs bounded probabilities between 0 and 1, and a clear decision: yes or no.

Logistic regression solves this. It takes the same linear combination ($wx + b$) and passes it through the sigmoid function, which squashes any number into the range (0, 1). The output is a probability. You set a threshold (usually 0.5) and make a decision.

This is one of the most widely used algorithms in practice. Despite its name, logistic regression is a classification algorithm, not a regression algorithm. The name comes from the logistic (sigmoid) function it uses.

The Concept

Why Linear Regression Fails for Classification

Imagine predicting pass/fail (1/0) based on study hours. Linear regression fits a line through the data:

hours:	1	2	3	4	5	6	7	8	9	10
actual:	0	0	0	0	1	1	1	1	1	1

A linear fit might produce predictions like -0.2 at hour 1 and 1.3 at hour 10. These values are not probabilities. They go below 0 and above 1. Worse, a single outlier (someone who studied 50 hours) would drag the entire line, changing predictions for everyone.

Classification needs a function that: - Outputs values between 0 and 1 (probabilities) - Creates a sharp transition (a decision boundary) - Is not distorted by outliers far from the boundary

The Sigmoid Function

The sigmoid function does exactly this:

$$\text{sigmoid}(z) = 1 / (1 + e^{(-z)})$$

Properties: - When z is large and positive, $\text{sigmoid}(z)$ approaches 1 - When z is large and negative, $\text{sigmoid}(z)$ approaches 0 - When $z = 0$, $\text{sigmoid}(z) = 0.5$ - The output is always between 0 and 1 - The function is smooth and differentiable everywhere

The derivative has a convenient form: $\text{sigmoid}'(z) = \text{sigmoid}(z) * (1 - \text{sigmoid}(z))$. This makes gradient computation efficient.

Logistic Regression = Linear Model + Sigmoid

The model computes $z = wx + b$ (same as linear regression), then applies sigmoid:

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/03-logistic-regression>

The output p is interpreted as $P(y=1 | x)$, the probability that the input belongs to class 1. The decision boundary is where $w x + b = 0$, which makes sigmoid output exactly 0.5.

Binary Cross-Entropy Loss

You cannot use MSE for logistic regression. MSE with a sigmoid creates a non-convex cost surface with many local minima. Instead, use binary cross-entropy (log loss):

$$\text{Loss} = -(1/n) * \sum(y * \log(p) + (1-y) * \log(1-p))$$

Why this works: - When $y=1$ and p is close to 1: $\log(1) = 0$, so loss is near 0 (correct, low cost) - When $y=1$ and p is close to 0: $\log(0)$ approaches negative infinity, so loss is huge (wrong, high cost) - When $y=0$ and p is close to 0: $\log(1) = 0$, so loss is near 0 (correct, low cost) - When $y=0$ and p is close to 1: $\log(0)$ approaches negative infinity, so loss is huge (wrong, high cost)

This loss function is convex for logistic regression, guaranteeing a single global minimum.

Gradient Descent for Logistic Regression

The gradients for binary cross-entropy with sigmoid have a clean form:

$$\begin{aligned} dL/dw &= (1/n) * \sum((p - y) * x) \\ dL/db &= (1/n) * \sum(p - y) \end{aligned}$$

These look identical to the linear regression gradients. The difference is that $p = \text{sigmoid}(wx + b)$ instead of $p = wx + b$. The sigmoid introduces the nonlinearity, but the gradient update rule stays the same.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/03-logistic-regression>

The Decision Boundary

For a 2D input (two features), the decision boundary is the line where:

$$w_1 \cdot x_1 + w_2 \cdot x_2 + b = 0$$

Points on one side get classified as 1, points on the other side as 0. Logistic regression always produces a linear decision boundary. If you need a curved boundary, you either add polynomial features or use a nonlinear model.

Multi-Class Classification with Softmax

Binary logistic regression handles two classes. For k classes, use the softmax function:

$$\text{softmax}(z_i) = e^{(z_i)} / \sum(e^{(z_j)} \text{ for all } j)$$

Each class has its own weight vector. The model computes a score z_i for each class, then softmax converts scores to probabilities that sum to 1. The predicted class is the one with the highest probability.

The loss function becomes categorical cross-entropy:

$$\text{Loss} = -(1/n) * \sum(\sum(y_k * \log(p_k)))$$

where y_k is 1 for the true class and 0 for all others (one-hot encoding).

Evaluation Metrics

Accuracy alone is not enough. For a dataset with 95% negative and 5% positive, a model that always predicts negative gets 95% accuracy but is useless.

Confusion Matrix:

	Predicted Positive	Predicted Negative
Actually Positive	True Positive (TP)	False Negative (FN)
Actually Negative	False Positive (FP)	True Negative (TN)

Precision: Of all predicted positives, how many are actually positive?

$$\text{Precision} = TP / (TP + FP)$$

Recall (Sensitivity): Of all actual positives, how many did we catch?

$$\text{Recall} = TP / (TP + FN)$$

F1 Score: Harmonic mean of precision and recall. Balances both metrics.

$$F1 = 2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall})$$

When to prioritize: - **Precision:** when false positives are costly (spam filter, you do not want to block legitimate email) - **Recall:** when false negatives are costly (cancer screening, you do not want to miss a tumor) - **F1:** when you need a single balanced metric

Interactive figure: logistic-sigmoid. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/03-logistic-regression>

Build It

Step 1: Sigmoid function and data generation

```
import random
import math

def sigmoid(z):
    z = max(-500, min(500, z))
    return 1.0 / (1.0 + math.exp(-z))

random.seed(42)
N = 200
X = []
y = []

for _ in range(N // 2):
    X.append([random.gauss(2, 1), random.gauss(2, 1)])
    y.append(0)

for _ in range(N // 2):
    X.append([random.gauss(5, 1), random.gauss(5, 1)])
    y.append(1)

combined = list(zip(X, y))
random.shuffle(combined)
X, y = zip(*combined)
X = list(X)
y = list(y)

print(f"Generated {N} samples (2 classes, 2 features)")
print(f"Class 0 center: (2, 2), Class 1 center: (5, 5)")
print(f"First 5 samples:")
for i in range(5):
    print(f"  Features: [{X[i][0]:.2f}, {X[i][1]:.2f}], Label: {y[i]}")
```

Step 2: Logistic regression from scratch

```
class LogisticRegression:
    def __init__(self, n_features, learning_rate=0.01):
        self.weights = [0.0] * n_features
        self.bias = 0.0
        self.lr = learning_rate
        self.loss_history = []

    def predict_proba(self, x):
        z = sum(w * xi for w, xi in zip(self.weights, x)) + self.bias
        return sigmoid(z)

    def predict(self, x, threshold=0.5):
        return 1 if self.predict_proba(x) >= threshold else 0

    def compute_loss(self, X, y):
```

```

n = len(y)
total = 0.0
for i in range(n):
    p = self.predict_proba(X[i])
    p = max(1e-15, min(1 - 1e-15, p))
    total += y[i] * math.log(p) + (1 - y[i]) * math.log(1 - p)
return -total / n

def fit(self, X, y, epochs=1000, print_every=200):
    n = len(y)
    n_features = len(X[0])
    for epoch in range(epochs):
        dw = [0.0] * n_features
        db = 0.0
        for i in range(n):
            p = self.predict_proba(X[i])
            error = p - y[i]
            for j in range(n_features):
                dw[j] += error * X[i][j]
            db += error
        for j in range(n_features):
            self.weights[j] -= self.lr * (dw[j] / n)
        self.bias -= self.lr * (db / n)
        loss = self.compute_loss(X, y)
        self.loss_history.append(loss)
        if epoch % print_every == 0:
            print(f" Epoch {epoch:4d} | Loss: {loss:.4f} | w: [{self.weights[0]:.3f}, {self.weights[1]:.3f}]")
    return self

def accuracy(self, X, y):
    correct = sum(1 for i in range(len(y)) if self.predict(X[i]) == y[i])
    return correct / len(y)

split = int(0.8 * N)
X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]

print("\n=== Training Logistic Regression ===")
model = LogisticRegression(n_features=2, learning_rate=0.1)
model.fit(X_train, y_train, epochs=1000, print_every=200)

print(f"\nTrain accuracy: {model.accuracy(X_train, y_train):.4f}")
print(f"Test accuracy: {model.accuracy(X_test, y_test):.4f}")
print(f"Weights: [{model.weights[0]:.4f}, {model.weights[1]:.4f}]")
print(f"Bias: {model.bias:.4f}")

```

Step 3: Confusion matrix and metrics from scratch

```

class ClassificationMetrics:
    def __init__(self, y_true, y_pred):
        self.tp = sum(1 for t, p in zip(y_true, y_pred) if t == 1 and p == 1)
        self.tn = sum(1 for t, p in zip(y_true, y_pred) if t == 0 and p == 0)
        self.fp = sum(1 for t, p in zip(y_true, y_pred) if t == 0 and p == 1)

```



```

        self.fn = sum(1 for t, p in zip(y_true, y_pred) if t == 1 and p == 0)

    def accuracy(self):
        total = self.tp + self.tn + self.fp + self.fn
        return (self.tp + self.tn) / total if total > 0 else 0

    def precision(self):
        denom = self.tp + self.fp
        return self.tp / denom if denom > 0 else 0

    def recall(self):
        denom = self.tp + self.fn
        return self.tp / denom if denom > 0 else 0

    def f1(self):
        p = self.precision()
        r = self.recall()
        return 2 * p * r / (p + r) if (p + r) > 0 else 0

    def print_confusion_matrix(self):
        print(f"\n Confusion Matrix:")
        print(f"                Predicted")
        print(f"                Pos    Neg")
        print(f" Actual Pos      {self.tp:4d} {self.fn:4d}")
        print(f" Actual Neg      {self.fp:4d} {self.tn:4d}")

    def print_report(self):
        self.print_confusion_matrix()
        print(f"\n Accuracy: {self.accuracy():.4f}")
        print(f" Precision: {self.precision():.4f}")
        print(f" Recall:    {self.recall():.4f}")
        print(f" F1 Score:  {self.f1():.4f}")

y_pred_test = [model.predict(x) for x in X_test]
print("\n=== Classification Report (Test Set) ===")
metrics = ClassificationMetrics(y_test, y_pred_test)
metrics.print_report()

```

Step 4: Decision boundary analysis

```

print("\n=== Decision Boundary ===")
w1, w2 = model.weights
b = model.bias
print(f"Decision boundary: {w1:.4f}*x1 + {w2:.4f}*x2 + {b:.4f} = 0")
if abs(w2) > 1e-10:
    print(f"Solved for x2:      x2 = {-w1/w2:.4f}*x1 + {-b/w2:.4f}")

print("\nSample predictions near the boundary:")
test_points = [
    [3.0, 3.0],
    [3.5, 3.5],
    [4.0, 4.0],
    [2.5, 2.5],

```

```

    [5.0, 5.0],
]
for point in test_points:
    prob = model.predict_proba(point)
    pred = model.predict(point)
    print(f" [{point[0]}, {point[1]}] -> prob={prob:.4f}, class={pred}")

```

Step 5: Multi-class with softmax

```

class SoftmaxRegression:
    def __init__(self, n_features, n_classes, learning_rate=0.01):
        self.n_features = n_features
        self.n_classes = n_classes
        self.lr = learning_rate
        self.weights = [[0.0] * n_features for _ in range(n_classes)]
        self.biases = [0.0] * n_classes

    def softmax(self, scores):
        max_score = max(scores)
        exp_scores = [math.exp(s - max_score) for s in scores]
        total = sum(exp_scores)
        return [e / total for e in exp_scores]

    def predict_proba(self, x):
        scores = [
            sum(self.weights[k][j] * x[j] for j in range(self.n_features)) + self.biases[k]
            for k in range(self.n_classes)
        ]
        return self.softmax(scores)

    def predict(self, x):
        probs = self.predict_proba(x)
        return probs.index(max(probs))

    def fit(self, X, y, epochs=1000, print_every=200):
        n = len(y)
        for epoch in range(epochs):
            grad_w = [[0.0] * self.n_features for _ in range(self.n_classes)]
            grad_b = [0.0] * self.n_classes
            total_loss = 0.0
            for i in range(n):
                probs = self.predict_proba(X[i])
                for k in range(self.n_classes):
                    target = 1.0 if y[i] == k else 0.0
                    error = probs[k] - target
                    for j in range(self.n_features):
                        grad_w[k][j] += error * X[i][j]
                    grad_b[k] += error
                true_prob = max(probs[y[i]], 1e-15)
                total_loss -= math.log(true_prob)
            for k in range(self.n_classes):
                for j in range(self.n_features):
                    self.weights[k][j] -= self.lr * (grad_w[k][j] / n)
                self.biases[k] -= self.lr * (grad_b[k] / n)

```

```

        if epoch % print_every == 0:
            print(f" Epoch {epoch:4d} | Loss: {total_loss / n:.4f}")
        return self

    def accuracy(self, X, y):
        correct = sum(1 for i in range(len(y)) if self.predict(X[i]) == y[i])
        return correct / len(y)

random.seed(42)
X_3class = []
y_3class = []

centers = [(1, 1), (5, 1), (3, 5)]
for label, (cx, cy) in enumerate(centers):
    for _ in range(50):
        X_3class.append([random.gauss(cx, 0.8), random.gauss(cy, 0.8)])
        y_3class.append(label)

combined = list(zip(X_3class, y_3class))
random.shuffle(combined)
X_3class, y_3class = zip(*combined)
X_3class = list(X_3class)
y_3class = list(y_3class)

split_3 = int(0.8 * len(X_3class))
X_train_3 = X_3class[:split_3]
y_train_3 = y_3class[:split_3]
X_test_3 = X_3class[split_3:]
y_test_3 = y_3class[split_3:]

print("\n=== Multi-class Softmax Regression (3 classes) ===")
softmax_model = SoftmaxRegression(n_features=2, n_classes=3, learning_rate=0.1)
softmax_model.fit(X_train_3, y_train_3, epochs=1000, print_every=200)
print(f"\nTrain accuracy: {softmax_model.accuracy(X_train_3, y_train_3):.4f}")
print(f"Test accuracy: {softmax_model.accuracy(X_test_3, y_test_3):.4f}")

print("\nSample predictions:")
for i in range(5):
    probs = softmax_model.predict_proba(X_test_3[i])
    pred = softmax_model.predict(X_test_3[i])
    print(f" True: {y_test_3[i]}, Predicted: {pred}, Probs: [{', '.join(f'{p:.3f}' for p in probs)}]")

```

Step 6: Threshold tuning

```

print("\n=== Threshold Tuning ===")
print("Default threshold: 0.5. Adjusting the threshold trades precision for recall.\n")

thresholds = [0.3, 0.4, 0.5, 0.6, 0.7]
print(f"{'Threshold':>10} {'Accuracy':>10} {'Precision':>10} {'Recall':>10} {'F1':>10}")
print("-" * 52)

for t in thresholds:
    y_pred_t = [1 if model.predict_proba(x) >= t else 0 for x in X_test]

```

```
m = ClassificationMetrics(y_test, y_pred_t)
print(f"t:>10.1f} {m.accuracy():>10.4f} {m.precision():>10.4f} {m.recall():>10.4f} {m.f1()")
```

Use It

Now the same thing with scikit-learn.

```
from sklearn.linear_model import LogisticRegression as SklearnLR
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.metrics import confusion_matrix, classification_report
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import numpy as np

np.random.seed(42)
X_0 = np.random.randn(100, 2) + [2, 2]
X_1 = np.random.randn(100, 2) + [5, 5]
X_sk = np.vstack([X_0, X_1])
y_sk = np.array([0] * 100 + [1] * 100)

X_tr, X_te, y_tr, y_te = train_test_split(X_sk, y_sk, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_tr_sc = scaler.fit_transform(X_tr)
X_te_sc = scaler.transform(X_te)

lr = SklearnLR()
lr.fit(X_tr_sc, y_tr)
y_pred = lr.predict(X_te_sc)

print("=== Scikit-learn Logistic Regression ===")
print(f"Accuracy: {accuracy_score(y_te, y_pred):.4f}")
print(f"Precision: {precision_score(y_te, y_pred):.4f}")
print(f"Recall: {recall_score(y_te, y_pred):.4f}")
print(f"F1: {f1_score(y_te, y_pred):.4f}")
print(f"\nConfusion Matrix:\n{confusion_matrix(y_te, y_pred)}")
print(f"\nClassification Report:\n{classification_report(y_te, y_pred)}")
```

Your from-scratch implementation produces the same decision boundary and metrics. Scikit-learn adds solver options (liblinear, lbfgs, saga), automatic regularization, multi-class strategies (one-vs-rest, multinomial), and numerical stability optimizations.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/03-logistic-regression>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/03-logistic-regression/code>

1. Generate a dataset that is NOT linearly separable (e.g., two concentric circles). Train logistic regression and observe its failure. Then add polynomial features (x_1^2 , x_2^2 , $x_1 \cdot x_2$) and train again. Show that the accuracy improves.

2. Implement a multi-class confusion matrix for the 3-class softmax model. Compute per-class precision and recall. Which class is hardest to classify?
3. Build an ROC curve from scratch. For 100 threshold values from 0 to 1, compute the true positive rate and false positive rate. Calculate the AUC (area under the curve) using the trapezoidal rule.

Key Terms

Term	What people say	What it actually means
Logistic regression	“Regression for classification”	A linear model followed by a sigmoid function that outputs class probabilities
Sigmoid function	“The S-curve”	The function $1/(1+e^{(-z)})$ that maps any real number to the range (0, 1)
Binary cross-entropy	“Log loss”	The loss function $-[y\log(p) + (1-y)\log(1-p)]$ that penalizes confident wrong predictions severely
Decision boundary	“The dividing line”	The surface where the model’s output probability equals 0.5, separating predicted classes
Softmax	“Multi-class sigmoid”	A function that converts a vector of scores into probabilities that sum to 1
Precision	“How many selected are relevant”	$TP / (TP + FP)$, the fraction of positive predictions that are actually positive
Recall	“How many relevant are selected”	$TP / (TP + FN)$, the fraction of actual positives that the model correctly identifies
F1 score	“Balanced accuracy”	The harmonic mean of precision and recall: $2PR / (P+R)$
Confusion matrix	“The error breakdown”	A table showing TP, TN, FP, FN counts for each class pair
Threshold	“The cutoff”	The probability value above which the model predicts class 1 (default 0.5, tunable)
One-hot encoding	“Binary columns for categories”	Representing class k as a vector of zeros with a 1 at position k
Categorical cross-entropy	“Multi-class log loss”	The extension of binary cross-entropy to k classes using one-hot encoded labels

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/03-logistic-regression>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/03-logistic-regression/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/03-logistic-regression>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Decision Trees and Random Forests

A decision tree is just a flowchart. But a forest of them is one of the most powerful tools in ML.

Type: Build **Language:** Python **Prerequisites:** Phase 1 (Lessons 09 Information Theory, 06 Probability) **Time:** ~90 minutes

Learning Objectives

- Implement Gini impurity, entropy, and information gain calculations to find optimal decision tree splits
- Build a decision tree classifier from scratch with pre-pruning controls (max depth, min samples)
- Construct a random forest using bootstrap sampling and feature randomization, and explain why it reduces variance
- Compare MDI feature importance with permutation importance and identify when MDI is biased

The Problem

You have tabular data. Rows are samples, columns are features, and there is a target column you want to predict. You could throw a neural network at it. But for tabular data, tree-based models (decision trees, random forests, gradient boosted trees) consistently outperform deep learning. Kaggle competitions on structured data are dominated by XGBoost and LightGBM, not transformers.

Why? Trees handle mixed feature types (numeric and categorical) without preprocessing. They handle nonlinear relationships without feature engineering. They are interpretable: you can look at the tree and see exactly why a prediction was made. And random forests, which average many trees, are highly resistant to overfitting on moderate-sized datasets.

This lesson builds decision trees from scratch using recursive splitting, then builds a random forest on top. You will implement the math behind split criteria (Gini impurity, entropy, information gain) and understand why an ensemble of weak learners becomes a strong one.

The Concept

What a decision tree does

A decision tree partitions the feature space into rectangular regions by asking a sequence of yes/no questions.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/04-decision-trees>

Each internal node tests a feature against a threshold. Each leaf node makes a prediction. To classify a new data point, you start at the root and follow the branches until you reach a leaf.

The tree is built top-down by choosing, at each node, the feature and threshold that best separate the data. “Best” is defined by a split criterion.

Split criteria: measuring impurity

At each node, we have a set of samples. We want to split them so that the resulting child nodes are as “pure” as possible, meaning each child contains mostly one class.

Gini impurity measures the probability that a randomly chosen sample would be misclassified if it were labeled according to the class distribution at that node.

$$\text{Gini}(S) = 1 - \sum(p_k^2)$$

where p_k is the proportion of class k in set S .

For a pure node (all one class), Gini = 0. For a binary split with 50/50 classes, Gini = 0.5. Lower is better.

Example: 6 cats, 4 dogs

$$\text{Gini} = 1 - (0.6^2 + 0.4^2) = 1 - (0.36 + 0.16) = 0.48$$

Entropy measures the information content (disorder) in a node. Covered in Phase 1 Lesson 09.

$$\text{Entropy}(S) = -\sum(p_k * \log_2(p_k))$$

For a pure node, entropy = 0. For a 50/50 binary split, entropy = 1.0. Lower is better.

Example: 6 cats, 4 dogs

$$\begin{aligned} \text{Entropy} &= -(0.6 * \log_2(0.6) + 0.4 * \log_2(0.4)) \\ &= -(0.6 * -0.737 + 0.4 * -1.322) \\ &= 0.442 + 0.529 \\ &= 0.971 \text{ bits} \end{aligned}$$

Information gain is the reduction in impurity (entropy or Gini) after a split.

$$\text{IG}(S, \text{feature}, \text{threshold}) = \text{Impurity}(S) - \text{weighted_avg}(\text{Impurity}(S_{\text{left}}), \text{Impurity}(S_{\text{right}}))$$

where the weights are the proportions of samples in each child.

The greedy algorithm at each node: try every feature and every possible threshold. Pick the (feature, threshold) pair that maximizes information gain.

How splitting works

For a dataset with n features and m samples at the current node:

1. For each feature j ($j = 1$ to n):
 - Sort the samples by feature j
 - Try every midpoint between consecutive distinct values as a threshold
 - Compute the information gain for each threshold
2. Select the feature and threshold with the highest information gain
3. Split the data into left (feature $\leq$ threshold) and right (feature $>$ threshold)
4. Recurse on each child

This greedy approach does not guarantee the globally optimal tree. Finding the optimal tree is NP-hard. But greedy splitting works well in practice.

Stopping conditions

Without stopping conditions, the tree grows until every leaf is pure (one sample per leaf). This perfectly memorizes the training data and generalizes terribly.

Pre-pruning stops the tree before it fully grows:

- Maximum depth: stop splitting when the tree reaches a set depth
- Minimum samples per leaf: stop if a node has fewer than k samples
- Minimum information gain: stop if the best split improves impurity by less than a threshold
- Maximum leaf nodes: limit the total number of leaves

Post-pruning grows the full tree, then trims it back:

- Cost-complexity pruning (used by scikit-learn): adds a penalty proportional to the number of leaves. Increase the penalty to get smaller trees
- Reduced error pruning: remove a subtree if the validation error does not increase

Pre-pruning is simpler and faster. Post-pruning often produces better trees because it does not prematurely stop splits that might lead to useful further splits.

Decision trees for regression

For regression, the leaf prediction is the mean of the target values in that leaf. The split criterion changes too:

Variance reduction replaces information gain:

$$VR(S, \text{feature}, \text{threshold}) = \text{Var}(S) - \text{weighted_avg}(\text{Var}(S_{\text{left}}), \text{Var}(S_{\text{right}}))$$

Pick the split that reduces variance the most. The tree partitions the input space into regions, and predicts a constant (the mean) in each region.

Random forests: the power of ensembles

A single decision tree is high variance. Small changes in the data can produce completely different trees. Random forests fix this by averaging many trees.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/04-decision-trees>

Two sources of randomness make the trees diverse:

Bagging (bootstrap aggregating): Each tree is trained on a bootstrap sample, a random sample with replacement from the training data. About 63% of the original samples appear in each bootstrap (the rest are out-of-bag samples that can be used for validation).

Feature randomization: At each split, only a random subset of features is considered. For classification, the default is $\sqrt{n_{\text{features}}}$. For regression, $n_{\text{features}}/3$. This prevents all trees from splitting on the same dominant feature.

The key insight: averaging many decorrelated trees reduces variance without increasing bias. Each individual tree may be mediocre. The ensemble is strong.

Feature importance

Random forests naturally provide feature importance scores. The most common method:

Mean Decrease in Impurity (MDI): For each feature, sum the total reduction in impurity across all trees and all nodes where that feature is used. Features that produce bigger impurity reductions at earlier splits are more important.

```
importance(feature_j) = sum over all nodes where feature_j is used:
    (n_samples_at_node / n_total_samples) * impurity_decrease
```

This is fast (computed during training) but biased toward high-cardinality features and features with many possible split points.

Permutation importance is the alternative: shuffle one feature's values and measure how much the model's accuracy drops. More reliable but slower.

When trees beat neural networks

Trees and forests dominate neural networks on tabular data. Several reasons:

Factor	Trees	Neural networks
Mixed types (numeric + categorical)	Native support	Need encoding
Small datasets (< 10k rows)	Work well	Overfit
Feature interactions	Found by splitting	Need architecture design
Interpretability	Full transparency	Black box
Training time	Minutes	Hours
Hyperparameter sensitivity	Low	High

Neural networks win when the data has spatial or sequential structure (images, text, audio). For flat tables of features, trees are the default.

Interactive figure: decision-tree-depth. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/04-decision-trees>

Build It

Step 1: Gini impurity and entropy

Build both split criteria from scratch and verify they agree on which splits are good.

```
import math
```

```
def gini_impurity(labels):
    n = len(labels)
    if n == 0:
        return 0.0
    counts = {}
    for label in labels:
        counts[label] = counts.get(label, 0) + 1
    return 1.0 - sum((c / n) ** 2 for c in counts.values())
```

```
def entropy(labels):
    n = len(labels)
```

```

if n == 0:
    return 0.0
counts = {}
for label in labels:
    counts[label] = counts.get(label, 0) + 1
return -sum(
    (c / n) * math.log2(c / n) for c in counts.values() if c > 0
)

```

Step 2: Find the best split

Try every feature and every threshold. Return the one with the highest information gain.

```

def information_gain(parent_labels, left_labels, right_labels, criterion="gini"):
    measure = gini_impurity if criterion == "gini" else entropy
    n = len(parent_labels)
    n_left = len(left_labels)
    n_right = len(right_labels)
    if n_left == 0 or n_right == 0:
        return 0.0
    parent_impurity = measure(parent_labels)
    child_impurity = (
        (n_left / n) * measure(left_labels) +
        (n_right / n) * measure(right_labels)
    )
    return parent_impurity - child_impurity

```

Step 3: Build the DecisionTree class

Recursive splitting, prediction, and feature importance tracking. `_build` is the heart of the tree: it stops when a node is pure or hits a pre-pruning limit, otherwise it takes the best split and recurses into both children.

```

import random

class DecisionTree:
    def __init__(self, max_depth=None, min_samples_split=2,
                 min_samples_leaf=1, criterion="gini",
                 max_features=None):
        self.max_depth = max_depth
        self.min_samples_split = min_samples_split
        self.min_samples_leaf = min_samples_leaf
        self.criterion = criterion
        self.max_features = max_features
        self.tree = None
        self.feature_importances_ = None

    def fit(self, X, y):
        self.n_features = len(X[0])
        self.feature_importances_ = [0.0] * self.n_features
        self.n_samples = len(X)
        self.tree = self._build(X, y, depth=0)
        total = sum(self.feature_importances_)
        if total > 0:
            self.feature_importances_ = [

```

```

        fi / total for fi in self.feature_importances_
    ]

def predict(self, X):
    return [self._predict_one(x, self.tree) for x in X]

def _build(self, X, y, depth):
    if len(set(y)) == 1:
        return {"leaf": True, "value": y[0]}

    if self.max_depth is not None and depth >= self.max_depth:
        return self._make_leaf(y)

    if len(y) < self.min_samples_split:
        return self._make_leaf(y)

    best_feature, best_threshold, best_gain = self._best_split(X, y)

    if best_feature is None or best_gain <= 0:
        return self._make_leaf(y)

    left_X, left_y, right_X, right_y = self._split_data(
        X, y, best_feature, best_threshold
    )

    if len(left_y) < self.min_samples_leaf or len(right_y) < self.min_samples_leaf:
        return self._make_leaf(y)

    weight = len(y) / self.n_samples
    self.feature_importances_[best_feature] += weight * best_gain

    return {
        "leaf": False,
        "feature": best_feature,
        "threshold": best_threshold,
        "left": self._build(left_X, left_y, depth + 1),
        "right": self._build(right_X, right_y, depth + 1),
    }

def _make_leaf(self, y):
    counts = {}
    for label in y:
        counts[label] = counts.get(label, 0) + 1
    return {"leaf": True, "value": max(counts, key=counts.get)}

def _best_split(self, X, y):
    best_feature = None
    best_threshold = None
    best_gain = -1.0

    if self.max_features == "sqrt":
        k = max(1, int(math.sqrt(self.n_features)))
        feature_indices = random.sample(range(self.n_features), k)
    elif isinstance(self.max_features, int):

```

```

        if self.max_features < 1:
            raise ValueError("max_features must be at least 1 when given as an integer")
        k = min(self.max_features, self.n_features)
        feature_indices = random.sample(range(self.n_features), k)
    else:
        feature_indices = list(range(self.n_features))

    for feature_idx in feature_indices:
        values = sorted(set(X[i][feature_idx] for i in range(len(X))))
        if len(values) <= 1:
            continue

        for i in range(len(values) - 1):
            threshold = (values[i] + values[i + 1]) / 2.0
            left_y = [y[j] for j in range(len(X)) if X[j][feature_idx] <= threshold]
            right_y = [y[j] for j in range(len(X)) if X[j][feature_idx] > threshold]

            if len(left_y) < self.min_samples_leaf or len(right_y) < self.min_samples_leaf:
                continue

            gain = information_gain(y, left_y, right_y, self.criterion)
            if gain > best_gain:
                best_gain = gain
                best_feature = feature_idx
                best_threshold = threshold

    return best_feature, best_threshold, best_gain

def _split_data(self, X, y, feature, threshold):
    left_X, left_y, right_X, right_y = [], [], [], []
    for i in range(len(X)):
        if X[i][feature] <= threshold:
            left_X.append(X[i])
            left_y.append(y[i])
        else:
            right_X.append(X[i])
            right_y.append(y[i])
    return left_X, left_y, right_X, right_y

def _predict_one(self, x, node):
    if node["leaf"]:
        return node["value"]
    if x[node["feature"]] <= node["threshold"]:
        return self._predict_one(x, node["left"])
    return self._predict_one(x, node["right"])

```

Step 4: Build the RandomForest class

Bootstrap sampling, feature randomization, and majority voting.

```

class RandomForest:
    def __init__(self, n_trees=100, max_depth=None,
                 min_samples_split=2, max_features="sqrt",
                 criterion="gini"):

```

```

self.n_trees = n_trees
self.max_depth = max_depth
self.min_samples_split = min_samples_split
self.max_features = max_features
self.criterion = criterion
self.trees = []

def fit(self, X, y):
    n = len(X)
    for _ in range(self.n_trees):
        indices = [random.randint(0, n - 1) for _ in range(n)]
        X_boot = [X[i] for i in indices]
        y_boot = [y[i] for i in indices]
        tree = DecisionTree(
            max_depth=self.max_depth,
            min_samples_split=self.min_samples_split,
            max_features=self.max_features,
            criterion=self.criterion,
        )
        tree.fit(X_boot, y_boot)
        self.trees.append(tree)

def predict(self, X):
    all_preds = [tree.predict(X) for tree in self.trees]
    predictions = []
    for i in range(len(X)):
        votes = {}
        for preds in all_preds:
            v = preds[i]
            votes[v] = votes.get(v, 0) + 1
        predictions.append(max(votes, key=votes.get))
    return predictions

```

See code/trees.py for the complete implementation with all helper methods.

Use It

With scikit-learn, training a random forest is three lines:

```

from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

```

```

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)

```

```

rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
print(f"Accuracy: {rf.score(X_test, y_test):.4f}")
print(f"Feature importances: {rf.feature_importances_}")

```

In practice, gradient boosted trees (XGBoost, LightGBM, CatBoost) are often stronger than random forests because they build trees sequentially, with each tree correcting the errors of the previous ones. But random forests are harder to misconfigure and require almost no

hyperparameter tuning.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/04-decision-trees>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/04-decision-trees/code>

1. Train a single decision tree on a 2D dataset with 3 classes. Manually trace the splits and draw the rectangular decision boundaries. Compare the boundaries at `max_depth=2` vs `max_depth=10`.
2. Implement variance reduction splitting for regression trees. Generate $y = \sin(x) + \text{noise}$ for 200 points and fit your regression tree. Plot the tree's piecewise-constant predictions against the true curve.
3. Build a random forest with 1, 5, 10, 50, and 200 trees. Plot training accuracy and test accuracy vs number of trees. Observe that test accuracy plateaus but does not decrease (forests resist overfitting).
4. Compare Gini impurity vs entropy as split criteria on 5 different datasets. Measure accuracy and tree depth. In most cases, they produce nearly identical results. Explain why.
5. Implement permutation importance. Compare it with MDI importance on a dataset where one feature is random noise but has high cardinality. MDI will rank the noise feature highly. Permutation importance will not.

Key Terms

Term	What people say	What it actually means
Decision tree	"A flowchart for predictions"	A model that partitions feature space into rectangular regions by learning a sequence of if/else splits
Gini impurity	"How mixed the node is"	Probability of misclassifying a random sample at a node. 0 = pure, 0.5 = maximum impurity for binary
Entropy	"The disorder in a node"	Information content at a node. 0 = pure, 1.0 = maximum uncertainty for binary. From information theory
Information gain	"How good a split is"	Reduction in impurity after a split. The greedy criterion for choosing splits
Pre-pruning	"Stop the tree early"	Stopping tree growth early by setting max depth, min samples, or min gain thresholds
Post-pruning	"Trim the tree after"	Growing the full tree, then removing subtrees that do not improve validation performance
Bagging	"Train on random subsets"	Bootstrap aggregating. Train each model on a different random sample with replacement

Term	What people say	What it actually means
Random forest	"A bunch of trees"	Ensemble of decision trees, each trained on a bootstrap sample with random feature subsets at each split
Feature importance (MDI)	"Which features matter"	Total impurity decrease contributed by each feature, summed across all trees and nodes
Permutation importance	"Shuffle and check"	Accuracy drop when a feature's values are randomly shuffled. More reliable than MDI for noisy features
Variance reduction	"The regression version of info gain"	The regression tree analogue of information gain. Picks the split that reduces target variance the most
Bootstrap sample	"Random sample with repeats"	A random sample drawn with replacement from the original dataset. Same size, but with duplicates

Further Reading

- Breiman: Random Forests (2001) - the original random forest paper
- Grinsztajn et al.: Why do tree-based models still outperform deep learning on tabular data? (2022) - rigorous comparison of trees vs neural networks on tabular tasks
- scikit-learn Decision Trees documentation - practical guide with visualization tools
- XGBoost: A Scalable Tree Boosting System (Chen & Guestrin, 2016) - the gradient boosting paper that dominates Kaggle

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/04-decision-trees>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/04-decision-trees/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/04-decision-trees>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Support Vector Machines

Find the widest street between two classes. That is the entire idea.

Type: Build **Language:** Python **Prerequisites:** Phase 1 (Lessons 08 Optimization, 14 Norms and Distances, 18 Convex Optimization) **Time:** ~90 minutes

Learning Objectives

- Implement a linear SVM from scratch using hinge loss and gradient descent on the primal formulation
- Explain the maximum margin principle and identify support vectors from a trained model
- Compare linear, polynomial, and RBF kernels and explain how the kernel trick avoids explicit high-dimensional mapping
- Evaluate the tradeoff controlled by the C parameter between margin width and classification errors

The Problem

You have two classes of data points and need to draw a line (or hyperplane) separating them. Infinitely many lines could work. Which one should you pick?

The one with the biggest margin. The margin is the distance between the decision boundary and the nearest data points on each side. A wider margin means the classifier is more confident and generalizes better to unseen data.

This intuition leads to Support Vector Machines, one of the most mathematically elegant algorithms in ML. SVMs were the dominant classification method before deep learning and remain the best choice for small datasets, high-dimensional data, and problems where you need a principled, well-understood model with theoretical guarantees.

SVMs connect directly to Phase 1: the optimization is convex (Lesson 18), the margin is measured with norms (Lesson 14), and the kernel trick exploits dot products to handle nonlinear boundaries without ever computing in the high-dimensional space.

The Concept

The maximum margin classifier

Given linearly separable data with labels y_i in $\{-1, +1\}$ and feature vectors x_i , we want a hyperplane $w^T x + b = 0$ that separates the classes.

The distance from a point x_i to the hyperplane is:

$$\text{distance} = |w^T x_i + b| / ||w||$$

For a correctly classified point: $y_i * (w^T x_i + b) > 0$. The margin is twice the distance from the hyperplane to the nearest point on either side.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/05-support-vector-machines>

The optimization problem:

```
maximize    2 / ||w||      (the margin width)
subject to  y_i * (w^T x_i + b) >= 1  for all i
```

Equivalently (minimizing $\|w\|^2$ is easier to optimize):

```
minimize    (1/2) ||w||^2
subject to  y_i * (w^T x_i + b) >= 1  for all i
```

This is a convex quadratic program. It has a unique global solution. The data points that sit exactly on the margin boundaries (where $y_i * (w^T x_i + b) = 1$) are the support vectors. They are the only points that determine the decision boundary. Move or remove any non-support-vector point, and the boundary does not change.

Support vectors: the critical few

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/05-support-vector-machines>

Most training points are irrelevant. Only the support vectors matter. This is why SVMs are memory-efficient at prediction time: you only need to store the support vectors, not the entire training set.

The number of support vectors also gives a bound on generalization error. Fewer support vectors relative to the dataset size means better generalization.

Soft margin: handling noise with the C parameter

Real data is rarely perfectly separable. Some points may be on the wrong side of the boundary, or inside the margin. The soft margin formulation allows violations by introducing slack variables.

```
minimize    (1/2) ||w||^2 + C * sum(xi_i)
subject to  y_i * (w^T x_i + b) >= 1 - xi_i
            xi_i >= 0  for all i
```

The slack variable xi_i measures how much point i violates the margin. C controls the trade-off:

C value	Behavior
Large C	Penalizes violations heavily. Narrow margin, fewer misclassifications. Overfits
Small C	Allows more violations. Wide margin, more misclassifications. Underfits

C is the regularization strength, inverted. Large C = less regularization. Small C = more regularization.

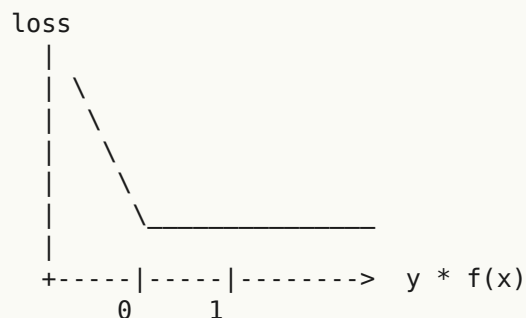
Hinge loss: the SVM loss function

The soft margin SVM can be rewritten as an unconstrained optimization:

```
minimize    (1/2) ||w||^2 + C * sum(max(0, 1 - y_i * (w^T x_i + b)))
```

The term $\max(0, 1 - y_i * f(x_i))$ is the hinge loss. It is zero when the point is correctly classified and beyond the margin. It is linear when the point is inside the margin or misclassified.

Hinge loss for a single point:



Zero loss when $y * f(x) \geq 1$ (correctly classified, outside margin).
Linear penalty when $y * f(x) < 1$.

Compare with logistic loss (logistic regression):

Hinge:	$\max(0, 1 - y * f(x))$	Hard cutoff at margin
Logistic:	$\log(1 + \exp(-y * f(x)))$	Smooth, never exactly zero

Hinge loss produces sparse solutions (only support vectors have nonzero contribution). Logistic loss uses all data points. This makes SVMs more memory-efficient at prediction time.

Training a linear SVM with gradient descent

You can train a linear SVM using gradient descent on the hinge loss plus L2 regularization, without solving the constrained QP:

$$L(w, b) = (\lambda/2) * ||w||^2 + (1/n) * \sum (\max(0, 1 - y_i * (w^T x_i + b)))$$

Gradient with respect to w :

If $y_i * (w^T x_i + b) \geq 1$:	$dL/dw = \lambda * w$
If $y_i * (w^T x_i + b) < 1$:	$dL/dw = \lambda * w - y_i * x_i$

Gradient with respect to b :

If $y_i * (w^T x_i + b) \geq 1$:	$dL/db = 0$
If $y_i * (w^T x_i + b) < 1$:	$dL/db = -y_i$

This is called the primal formulation. It runs in $O(n * d)$ per epoch, where n is the number of samples and d is the number of features. For large, sparse, high-dimensional data (text classification), this is fast.

The dual formulation and the kernel trick

The Lagrangian dual of the SVM problem (from Phase 1 Lesson 18, KKT conditions) is:

maximize $\sum(\alpha_i) - (1/2) * \sum_{ij}(\alpha_i * \alpha_j * y_i * y_j * (x_i \cdot x_j))$
subject to $0 \leq \alpha_i \leq C$
 $\sum(\alpha_i * y_i) = 0$

The dual only involves dot products $x_i \cdot x_j$ between data points. This is the key insight. Replace every dot product with a kernel function $K(x_i, x_j)$ and the SVM can learn nonlinear boundaries without ever computing the transformation explicitly.

Linear kernel: $K(x, z) = x \cdot z$
 Polynomial kernel: $K(x, z) = (x \cdot z + c)^d$
 RBF (Gaussian): $K(x, z) = \exp(-\gamma * ||x - z||^2)$

The RBF kernel maps data into an infinite-dimensional space. Points that are close in input space have kernel value near 1. Points that are far apart have kernel value near 0. It can learn any smooth decision boundary.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/05-support-vector-machines>

The kernel trick computes the dot product in the high-dimensional space without ever going there. For the polynomial kernel of degree d in D dimensions, the explicit feature space has $O(D^d)$ dimensions. But $K(x, z)$ is computed in $O(D)$ time.

SVM for regression (SVR)

Support Vector Regression fits a tube of width epsilon around the data. Points inside the tube have zero loss. Points outside the tube are penalized linearly.

```

minimize    (1/2) ||w||^2 + C * sum(xi_i + xi_i*)
subject to  y_i - (w^T x_i + b) <= epsilon + xi_i
            (w^T x_i + b) - y_i <= epsilon + xi_i*
            xi_i, xi_i* >= 0
  
```

The epsilon parameter controls the tube width. Wider tube = fewer support vectors = smoother fit. Narrower tube = more support vectors = tighter fit.

Why SVMs lost to deep learning (and when they still win)

SVMs dominated ML from the late 1990s through the early 2010s. Deep learning surpassed them for several reasons:

Factor	SVMs	Deep learning
Feature engineering	Requires it	Learns features
Scalability	$O(n^2)$ to $O(n^3)$ for kernel	$O(n)$ per epoch with SGD
Image/text/audio	Needs handcrafted features	Learns from raw data
Large datasets (>100k)	Slow	Scales well
GPU acceleration	Limited benefit	Massive speedup

SVMs still win in these situations: - Small datasets (hundreds to low thousands of samples) - High-dimensional sparse data (text with TF-IDF features) - When you need mathematical guarantees (margin bounds) - When training time must be minimal (linear SVM is very fast) - Binary classification with clear margin structure - Anomaly detection (one-class SVM)

Interactive figure: svm-margin. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/05-support-vector-machines>

Build It

Step 1: Hinge loss and gradient

The foundation. Compute hinge loss for a batch and its gradient.

```
def hinge_loss(X, y, w, b):
    n = len(X)
    total_loss = 0.0
    for i in range(n):
        margin = y[i] * (dot(w, X[i]) + b)
        total_loss += max(0.0, 1.0 - margin)
    return total_loss / n
```

Step 2: Linear SVM via gradient descent

Train by minimizing regularized hinge loss. No QP solver needed.

```
class LinearSVM:
    def __init__(self, lr=0.001, lambda_param=0.01, n_epochs=1000):
        self.lr = lr
        self.lambda_param = lambda_param
        self.n_epochs = n_epochs
        self.w = None
        self.b = 0.0

    def fit(self, X, y):
        n_features = len(X[0])
        self.w = [0.0] * n_features
        self.b = 0.0

        for epoch in range(self.n_epochs):
            for i in range(len(X)):
                margin = y[i] * (dot(self.w, X[i]) + self.b)
                if margin >= 1:
                    self.w = [wj - self.lr * self.lambda_param * wj
                               for wj in self.w]
                else:
                    self.w = [wj - self.lr * (self.lambda_param * wj - y[i] * X[i][j])
                               for j, wj in enumerate(self.w)]
                    self.b -= self.lr * (-y[i])

    def predict(self, X):
        return [1 if dot(self.w, x) + self.b >= 0 else -1 for x in X]
```

Step 3: Kernel functions

Implement linear, polynomial, and RBF kernels.

```
def linear_kernel(x, z):
    return dot(x, z)

def polynomial_kernel(x, z, degree=3, c=1.0):
    return (dot(x, z) + c) ** degree
```

```
def rbf_kernel(x, z, gamma=0.5):
    diff = [xi - zi for xi, zi in zip(x, z)]
    return math.exp(-gamma * dot(diff, diff))
```

Step 4: Margin and support vector identification

After training, identify which points are support vectors and compute the margin width.

```
def find_support_vectors(X, y, w, b, tol=1e-3):
    support_vectors = []
    for i in range(len(X)):
        margin = y[i] * (dot(w, X[i]) + b)
        if abs(margin - 1.0) < tol:
            support_vectors.append(i)
    return support_vectors
```

See code/svm.py for the complete implementation with all demos.

Use It

With scikit-learn:

```
from sklearn.svm import SVC, LinearSVC, SVR
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

clf = Pipeline([
    ("scaler", StandardScaler()),
    ("svm", SVC(kernel="rbf", C=1.0, gamma="scale")),
])
clf.fit(X_train, y_train)
print(f"Accuracy: {clf.score(X_test, y_test):.4f}")
print(f"Support vectors: {clf['svm'].n_support_}")
```

Important: always scale your features before training an SVM. SVMs are sensitive to feature magnitudes because the margin depends on $\|w\|$, and unscaled features distort the geometry.

For large datasets, use LinearSVC (primal formulation, $O(n)$ per epoch) instead of SVC (dual formulation, $O(n^2)$ to $O(n^3)$):

```
from sklearn.svm import LinearSVC

clf = Pipeline([
    ("scaler", StandardScaler()),
    ("svm", LinearSVC(C=1.0, max_iter=10000)),
])
```

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/05-support-vector-machines/code>

1. Generate a 2D linearly separable dataset. Train your LinearSVM and identify the support vectors. Verify that the support vectors are the points closest to the decision boundary.

2. Vary C from 0.001 to 1000 on a noisy dataset. Plot the decision boundary for each C value. Observe the transition from wide margin (underfitting) to narrow margin (overfitting).
3. Create a dataset where class boundaries are circular (not linear). Show that a linear SVM fails. Compute the RBF kernel matrix and show that the classes become separable in the kernel-induced feature space.
4. Compare hinge loss vs logistic loss on the same dataset. Train a linear SVM and logistic regression. Count how many training points contribute to each model's decision boundary (support vectors vs all points).
5. Implement SVR (epsilon-insensitive loss). Fit it to $y = \sin(x) + \text{noise}$. Plot the epsilon tube around the predictions and highlight the support vectors (points outside the tube).

Key Terms

Term	What it actually means
Support vectors	The training points closest to the decision boundary. The only points that determine the hyperplane
Margin	The distance between the decision boundary and the nearest support vectors. SVMs maximize this
Hinge loss	$\max(0, 1 - y \cdot f(x))$. Zero when correctly classified and outside the margin. Linear penalty otherwise
C parameter	Trade-off between margin width and classification errors. Large C = narrow margin, small C = wide margin
Soft margin	SVM formulation that allows margin violations via slack variables. Handles non-separable data
Kernel trick	Computing dot products in a high-dimensional feature space without explicitly mapping to that space
Linear kernel	$K(x, z) = x \cdot z$. Equivalent to standard dot product. For linearly separable data
RBF kernel	$K(x, z) = \exp(-\gamma \ x - z\ ^2)$. Maps to infinite dimensions. Learns any smooth boundary
Polynomial kernel	$K(x, z) = (x \cdot z + c)^d$. Maps to a feature space of polynomial combinations
Dual formulation	Reformulation of the SVM problem that depends only on dot products between data points. Enables kernels
SVR	Support Vector Regression. Fits an epsilon-tube around the data. Points inside the tube have zero loss
Slack variables	ξ_i : measures how much a point violates the margin. Zero for correctly classified points outside margin
Maximum margin	The principle of choosing the hyperplane that maximizes the distance to the nearest points of each class

Further Reading

- Vapnik: The Nature of Statistical Learning Theory (1995) - the foundational text on SVMs and statistical learning
- Cortes & Vapnik: Support-vector networks (1995) - the original SVM paper
- Platt: Sequential Minimal Optimization (1998) - the SMO algorithm that made SVM training practical

- scikit-learn SVM documentation - practical guide with implementation details
- LIBSVM: A Library for Support Vector Machines - the C++ library behind most SVM implementations

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/05-support-vector-machines>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/05-support-vector-machines/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/05-support-vector-machines>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

K-Nearest Neighbors and Distances

Store everything. Predict by looking at your neighbors. The simplest algorithm that actually works.

Type: Build **Language:** Python **Prerequisites:** Phase 1 (Lesson 14 Norms and Distances)

Time: ~90 minutes

Learning Objectives

- Implement KNN classification and regression from scratch with configurable K and distance-weighted voting
- Compare L1, L2, cosine, and Minkowski distance metrics and select the appropriate one for a given data type
- Explain the curse of dimensionality and demonstrate why KNN degrades in high-dimensional spaces
- Build a KD-tree for efficient nearest neighbor search and analyze when it outperforms brute-force

The Problem

You have a dataset. A new data point arrives. You need to classify it or predict its value. Instead of learning parameters from the data (like linear regression or SVMs), you just find the K training points closest to the new point and let them vote.

This is K-nearest neighbors. There is no training phase. No parameters to learn. No loss function to minimize. You store the entire training set and compute distances at prediction time.

It sounds too simple to work. But KNN is surprisingly competitive for many problems, especially with small to medium datasets, and understanding it deeply reveals fundamental concepts: the choice of distance metric (connecting to Phase 1 Lesson 14), the curse of dimensionality, and the difference between lazy and eager learning.

KNN also shows up everywhere in modern AI, just under different names. Vector databases do KNN search over embeddings. Retrieval-augmented generation (RAG) finds the K nearest document chunks. Recommendation systems find similar users or items. The algorithm is the same. The scale and the data structures are different.

The Concept

How KNN works

Given a dataset of labeled points and a new query point:

1. Compute the distance from the query to every point in the dataset
2. Sort by distance

3. Take the K closest points
4. For classification: majority vote among the K neighbors
5. For regression: average (or weighted average) of the K neighbors' values

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/06-knn-and-distances>

That is the entire algorithm. No fitting. No gradient descent. No epochs.

Choosing K

K is the single hyperparameter. It controls the bias-variance trade-off:

K	Behavior
K = 1	Decision boundary follows every point. Zero training error. High variance. Overfits
Small K (3-5)	Sensitive to local structure. Can capture complex boundaries
Large K	Smoother boundaries. More robust to noise. May underfit
K = N	Predicts the majority class for every point. Maximum bias

A common starting point is $K = \sqrt{N}$ for a dataset of N points. Use odd K for binary classification to avoid ties.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/06-knn-and-distances>

Distance metrics

The distance function defines what “near” means. Different metrics produce different neighbors, different predictions.

L2 (Euclidean) is the default. Straight-line distance.

$$d(a, b) = \sqrt{\sum (a_i - b_i)^2}$$

Sensitive to feature scale. Always standardize features before using L2 with KNN.

L1 (Manhattan) sums absolute differences. More robust to outliers than L2 because it does not square the differences.

$$d(a, b) = \sum |a_i - b_i|$$

Cosine distance measures the angle between vectors, ignoring magnitude. Essential for text and embedding data.

$$d(a, b) = 1 - (a \cdot b) / (||a|| * ||b||)$$

Minkowski generalizes L1 and L2 with parameter p.

$$d(a, b) = (\sum |a_i - b_i|^p)^{1/p}$$

p=1: Manhattan

p=2: Euclidean

p->inf: Chebyshev (max absolute difference)

Which metric to use depends on the data:

Data type	Best metric	Why
Numeric features, similar scale	L2 (Euclidean)	Default, works for spatial data
Numeric features, outliers	L1 (Manhattan)	Robust, does not amplify large differences
Text embeddings	Cosine	Magnitude is noise, direction is meaning
High-dimensional sparse	Cosine or L1	L2 suffers from curse of dimensionality
Mixed types	Custom distance	Combine metrics per feature type

Weighted KNN

Standard KNN gives equal weight to all K neighbors. But a neighbor at distance 0.1 should matter more than one at distance 5.0.

Distance-weighted KNN weights each neighbor inversely by distance:

$$\text{weight}_i = 1 / (\text{distance}_i + \text{epsilon})$$

For classification: weighted vote

For regression: $\text{weighted average} = \text{sum}(w_i * y_i) / \text{sum}(w_i)$

The epsilon prevents division by zero when a query point exactly matches a training point.

Weighted KNN is less sensitive to the choice of K because distant neighbors contribute very little regardless.

The curse of dimensionality

KNN performance degrades in high dimensions. This is not a vague concern. It is a mathematical fact.

Problem 1: distances converge. As dimensionality increases, the ratio of the maximum distance to the minimum distance approaches 1. All points become equally “far” from the query.

In d dimensions, for random uniform points:

d=2: $\text{max_dist} / \text{min_dist} = \text{varies widely}$

d=100: $\text{max_dist} / \text{min_dist} \sim 1.01$

d=1000: $\text{max_dist} / \text{min_dist} \sim 1.001$

When all distances are nearly equal, “nearest” is meaningless.

Problem 2: volume explodes. To capture K neighbors within a fixed fraction of the data, you need to extend your search radius to cover a much larger fraction of the feature space. The “neighborhood” in high dimensions encompasses most of the space.

Problem 3: corners dominate. In a unit hypercube in d dimensions, most of the volume is concentrated near the corners, not the center. A sphere inscribed in the cube contains a vanishing fraction of the volume as d grows.

Practical consequence: KNN works well up to about 20-50 features. Beyond that, you need dimensionality reduction (PCA, UMAP, t-SNE) before applying KNN, or you need to use tree-based search structures that exploit the data's intrinsic lower dimensionality.

KD-trees: fast nearest neighbor search

Brute-force KNN computes the distance from the query to every training point. That is $O(n * d)$ per query. For large datasets, this is too slow.

A KD-tree recursively partitions the space along feature axes. At each level, it splits along one dimension at the median value.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/06-knn-and-distances>

To find the nearest neighbor, traverse the tree to the leaf containing the query, then backtrack and check neighboring partitions only if they could contain closer points.

Average query time: $O(\log n)$ for low dimensions. But KD-trees degrade to $O(n)$ in high dimensions ($d > 20$) because the backtracking eliminates fewer and fewer branches.

Ball trees: better for moderate dimensions

Ball trees partition data into nested hyperspheres instead of axis-aligned boxes. Each node defines a ball (center + radius) that contains all points in that subtree.

Advantages over KD-trees: - Work better in moderate dimensions (up to ~ 50) - Handle non-axis-aligned structure - Tighter bounding volumes mean more branches are pruned during search

Both KD-trees and ball trees are exact algorithms. For truly large-scale search (millions of points, hundreds of dimensions), approximate nearest neighbor methods (HNSW, IVF, product quantization) are used instead. These are covered in Phase 1 Lesson 14.

Lazy learning vs eager learning

KNN is a lazy learner: it does no work at training time and all work at prediction time. Most other algorithms (linear regression, SVMs, neural networks) are eager learners: they do heavy computation at training time to build a compact model, then predictions are fast.

Aspect	Lazy (KNN)	Eager (SVM, neural net)
Training time	$O(1)$ just store data	$O(n * \text{epochs})$
Prediction time	$O(n * d)$ per query	$O(d)$ or $O(\text{parameters})$
Memory at prediction	Store entire training set	Store model parameters only
Adapts to new data	Add points instantly	Retrain the model
Decision boundary	Implicit, computed on the fly	Explicit, fixed after training

Lazy learning is ideal when: - The dataset changes frequently (add/remove points without retraining) - You need predictions for very few queries - You want zero training time - The dataset is small enough that brute-force search is fast

KNN for regression

Instead of majority voting, KNN for regression averages the target values of the K neighbors.

$$\text{prediction} = (1/K) * \sum(y_i \text{ for } i \text{ in } K \text{ nearest neighbors})$$

Or with distance weighting:

$$\text{prediction} = \sum(w_i * y_i) / \sum(w_i)$$

where $w_i = 1 / \text{distance}_i$

KNN regression produces piecewise-constant (or piecewise-smooth with weighting) predictions. It cannot extrapolate beyond the range of the training data. If the training targets are all between 0 and 100, KNN will never predict 200.

Interactive figure: knn-smoothness. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/06-knn-and-distances>

Build It

Step 1: Distance functions

Implement L1, L2, cosine, and Minkowski distances. These connect directly to Phase 1 Lesson 14.

```
import math

def l2_distance(a, b):
    return math.sqrt(sum((ai - bi) ** 2 for ai, bi in zip(a, b)))

def l1_distance(a, b):
    return sum(abs(ai - bi) for ai, bi in zip(a, b))

def cosine_distance(a, b):
    dot_val = sum(ai * bi for ai, bi in zip(a, b))
    norm_a = math.sqrt(sum(ai ** 2 for ai in a))
    norm_b = math.sqrt(sum(bi ** 2 for bi in b))
    if norm_a == 0 or norm_b == 0:
        return 1.0
    return 1.0 - dot_val / (norm_a * norm_b)

def minkowski_distance(a, b, p=2):
    if p == float('inf'):
        return max(abs(ai - bi) for ai, bi in zip(a, b))
    return sum(abs(ai - bi) ** p for ai, bi in zip(a, b)) ** (1 / p)
```

Step 2: KNN classifier and regressor

Build the full KNN with configurable K, distance metric, and optional distance weighting.

```

class KNN:
    def __init__(self, k=5, distance_fn=l2_distance, weighted=False,
                 task="classification"):
        self.k = k
        self.distance_fn = distance_fn
        self.weighted = weighted
        self.task = task
        self.X_train = None
        self.y_train = None

    def fit(self, X, y):
        self.X_train = X
        self.y_train = y

    def predict(self, X):
        return [self._predict_one(x) for x in X]

```

Step 3: KD-tree for efficient search

Build a KD-tree from scratch that recursively splits on the median of each dimension.

```

class KDTree:
    def __init__(self, X, indices=None, depth=0):
        # Recursively partition the data
        self.axis = depth % len(X[0])
        # Split on median of the current axis
        ...

    def query(self, point, k=1):
        # Traverse to leaf, then backtrack
        ...

```

See code/knn.py for the complete implementation with all helper methods and demos.

Step 4: Feature scaling

KNN requires feature scaling because distances are sensitive to feature magnitudes. A feature ranging from 0 to 1000 will dominate a feature ranging from 0 to 1.

```

def standardize(X):
    n = len(X)
    d = len(X[0])
    means = [sum(X[i][j] for i in range(n)) / n for j in range(d)]
    stds = [
        max(1e-10, (sum((X[i][j] - means[j]) ** 2 for i in range(n)) / n) ** 0.5)
        for j in range(d)
    ]
    return [((X[i][j] - means[j]) / stds[j]) for j in range(d)] for i in range(n)], means, stds

```

Use It

With scikit-learn:

```

from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

clf = Pipeline([
    ("scaler", StandardScaler()),
    ("knn", KNeighborsClassifier(n_neighbors=5, metric="euclidean")),
])
clf.fit(X_train, y_train)
print(f"Accuracy: {clf.score(X_test, y_test):.4f}")

```

Scikit-learn automatically uses KD-trees or ball trees when the dataset is large enough and the dimensionality is low enough. For high-dimensional data, it falls back to brute force. You can control this with the `algorithm` parameter.

For large-scale nearest neighbor search (millions of vectors), use FAISS, Annoy, or a vector database:

```

import faiss

index = faiss.IndexFlatL2(dimension)
index.add(embeddings)
distances, indices = index.search(query_vectors, k=5)

```

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/06-knn-and-distances/code>

1. Implement KNN classification on a 2D dataset with 3 classes. Plot the decision boundary for $K=1$, $K=5$, $K=15$, and $K=N$. Observe the transition from overfitting to underfitting.
2. Generate 1000 random points in 2, 5, 10, 50, 100, and 500 dimensions. For each dimensionality, compute the ratio of the maximum pairwise distance to the minimum pairwise distance. Plot the ratio vs dimensionality to visualize the curse of dimensionality.
3. Compare L1, L2, and cosine distance for KNN on a text classification problem (use TF-IDF vectors). Which metric gives the best accuracy? Why does cosine tend to win for text?
4. Implement a KD-tree and measure query time vs brute force for datasets of 1k, 10k, and 100k points in 2D, 10D, and 50D. At what dimensionality does the KD-tree stop being faster than brute force?
5. Build a weighted KNN regressor for $y = \sin(x) + \text{noise}$. Compare it with unweighted KNN for $K=3$, 10, 30. Show that weighting produces smoother predictions, especially for large K .

Key Terms

Term	What it actually means
K-nearest neighbors	Non-parametric algorithm that predicts by finding the K closest training points to a query

Term	What it actually means
Lazy learning	No computation at training time. All work happens at prediction time. KNN is the canonical example
Eager learning	Heavy computation at training time to build a compact model. Most ML algorithms are eager
Curse of dimensionality	In high dimensions, distances converge and neighborhoods expand to cover most of the space, making KNN ineffective
KD-tree	Binary tree that recursively partitions space along feature axes. $O(\log n)$ queries in low dimensions
Ball tree	Tree of nested hyperspheres. Works better than KD-trees in moderate dimensions (up to ~ 50)
Weighted KNN	Neighbors weighted inversely by distance. Closer neighbors have more influence on the prediction
Feature scaling	Normalizing features to comparable ranges. Required for distance-based methods like KNN
Majority vote	Classification by counting which class is most common among K neighbors
Brute force search	Computing distance to every training point. $O(n*d)$ per query. Exact but slow for large n
Approximate nearest neighbor	Algorithms (HNSW, LSH, IVF) that find approximately nearest points much faster than exact search
Voronoi diagram	The partition of space where each region contains all points closer to one training point than any other. $K=1$ KNN produces Voronoi boundaries

Further Reading

- Cover & Hart: Nearest Neighbor Pattern Classification (1967) - the foundational KNN paper proving it has error rate at most twice the Bayes optimal
- Friedman, Bentley, Finkel: An Algorithm for Finding Best Matches in Logarithmic Expected Time (1977) - the original KD-tree paper
- Beyer et al.: When Is “Nearest Neighbor” Meaningful? (1999) - formal analysis of the curse of dimensionality for nearest neighbor
- scikit-learn Nearest Neighbors documentation - practical guide with algorithm selection
- FAISS: A Library for Efficient Similarity Search - Meta’s library for billion-scale approximate nearest neighbor search

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/06-knn-and-distances>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/06-knn-and-distances/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/06-knn-and-distances>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Unsupervised Learning

No labels, no teacher. The algorithm finds structure on its own.

Type: Build **Languages:** Python **Prerequisites:** Phase 1 (Norms & Distances, Probability & Distributions), Phase 2 Lessons 1-6 **Time:** ~90 minutes

Learning Objectives

- Implement K-Means, DBSCAN, and Gaussian Mixture Models from scratch and compare their clustering behavior
- Evaluate cluster quality using the silhouette score and the elbow method to select the optimal K
- Explain when DBSCAN outperforms K-Means and identify which algorithm handles non-spherical clusters and outliers
- Build an anomaly detection pipeline using clustering methods to flag points that deviate from normal patterns

The Problem

Every ML lesson so far has assumed labeled data: “here is an input, here is the correct output.” In the real world, labels are expensive. A hospital has millions of patient records but no one has manually tagged each one with a disease category. An e-commerce site has millions of user sessions but no one has hand-labeled customer segments. A security team has network logs but nobody has flagged every anomaly.

Unsupervised learning finds patterns without being told what to look for. It groups similar data points, discovers hidden structures, and surfaces anomalies. If supervised learning is learning from a textbook with an answer key, unsupervised learning is staring at raw data until the patterns reveal themselves.

The catch: without labels, you cannot directly measure “right” or “wrong.” You need different tools to evaluate whether the structure your algorithm found is meaningful.

The Concept

Clustering: Grouping Similar Things Together

Clustering assigns each data point to a group (cluster) so that points within the same group are more similar to each other than to points in other groups. The question is always: what does “similar” mean?

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/07-unsupervised-learning>

K-Means: The Workhorse

K-Means partitions data into exactly K clusters. Each cluster has a centroid (its center of mass), and every point belongs to the nearest centroid.

Lloyd's algorithm:

1. Pick K random points as initial centroids
2. Assign each data point to the nearest centroid
3. Recompute each centroid as the mean of its assigned points
4. Repeat steps 2-3 until assignments stop changing

The objective function (inertia) measures the total squared distance from each point to its assigned centroid. K-Means minimizes this, but only finds a local minimum. Different initializations can give different results.

Choosing K

Two standard methods:

Elbow method: Run K-Means for $K = 1, 2, 3, \dots, n$. Plot inertia vs K. Look for the “elbow” where adding more clusters stops reducing inertia significantly.

Silhouette score: For each point, measure how similar it is to its own cluster (a) versus the nearest other cluster (b). The silhouette coefficient is $(b - a) / \max(a, b)$, ranging from -1 (wrong cluster) to +1 (well-clustered). Average across all points for a global score.

DBSCAN: Density-Based Clustering

K-Means assumes clusters are spherical and requires you to pick K upfront. DBSCAN makes neither assumption. It finds clusters as dense regions separated by sparse regions.

Two parameters: - **eps**: the radius of a neighborhood - **min_samples**: the minimum number of points needed to form a dense region

Three types of points: - **Core point**: has at least min_samples points within eps distance - **Border point**: within eps of a core point but not itself a core point - **Noise point**: neither core nor border. These are outliers.

DBSCAN connects core points that are within eps of each other into the same cluster. Border points join the cluster of a nearby core point. Noise points belong to no cluster.

Strengths: finds clusters of any shape, automatically determines the number of clusters, identifies outliers. Weakness: struggles with clusters of varying densities.

Hierarchical Clustering

Builds a tree (dendrogram) of nested clusters.

Agglomerative (bottom-up): 1. Start with each point as its own cluster 2. Merge the two closest clusters 3. Repeat until only one cluster remains 4. Cut the dendrogram at the desired level to get K clusters

The “closeness” between clusters can be measured as: - **Single linkage**: minimum distance between any two points in the two clusters - **Complete linkage**: maximum distance between any two points - **Average linkage**: average distance between all pairs - **Ward's method**: the merge that causes the smallest increase in total within-cluster variance

Gaussian Mixture Models (GMM)

K-Means gives hard assignments: each point belongs to exactly one cluster. GMM gives soft assignments: each point has a probability of belonging to each cluster.

GMM assumes the data is generated from a mixture of K Gaussian distributions, each with its own mean and covariance. The Expectation-Maximization (EM) algorithm alternates between:

- **E-step:** compute the probability that each point belongs to each Gaussian
- **M-step:** update the mean, covariance, and mixing weight of each Gaussian to maximize the likelihood of the data

GMM can model elliptical clusters (not just spherical like K-Means) and naturally handles overlapping clusters.

When to Use Which

Method	Best for	Avoid when
K-Means	Large datasets, spherical clusters, known K	Irregular shapes, outliers present
DBSCAN	Unknown K , arbitrary shapes, outlier detection	Varying densities, very high dimensions
Hierarchical	Small datasets, need dendrogram, unknown K	Large datasets ($O(n^2)$ memory)
GMM	Overlapping clusters, soft assignments needed	Very large datasets, too many dimensions

Anomaly Detection with Clustering

Clustering naturally supports anomaly detection: - **K-Means:** points far from any centroid are anomalies - **DBSCAN:** noise points are anomalies by definition - **GMM:** points with low probability under all Gaussians are anomalies

Interactive figure: kmeans-step. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/07-unsupervised-learning>

Build It

Step 1: K-Means from scratch

```
import math
import random

def euclidean_distance(a, b):
    return math.sqrt(sum((ai - bi) ** 2 for ai, bi in zip(a, b)))

def kmeans(data, k, max_iterations=100, seed=42):
    random.seed(seed)
    n_features = len(data[0])
```

```

centroids = random.sample(data, k)

for iteration in range(max_iterations):
    clusters = [[] for _ in range(k)]
    assignments = []

    for point in data:
        distances = [euclidean_distance(point, c) for c in centroids]
        nearest = distances.index(min(distances))
        clusters[nearest].append(point)
        assignments.append(nearest)

    new_centroids = []
    for cluster in clusters:
        if len(cluster) == 0:
            new_centroids.append(random.choice(data))
            continue
        centroid = [
            sum(point[j] for point in cluster) / len(cluster)
            for j in range(n_features)
        ]
        new_centroids.append(centroid)

    if all(
        euclidean_distance(old, new) < 1e-6
        for old, new in zip(centroids, new_centroids)
    ):
        print(f" Converged at iteration {iteration + 1}")
        break

    centroids = new_centroids

return assignments, centroids

```

Step 2: Elbow method and silhouette score

```

def compute_inertia(data, assignments, centroids):
    total = 0.0
    for point, cluster_id in zip(data, assignments):
        total += euclidean_distance(point, centroids[cluster_id]) ** 2
    return total

def silhouette_score(data, assignments):
    n = len(data)
    if n < 2:
        return 0.0

    clusters = {}
    for i, c in enumerate(assignments):
        clusters.setdefault(c, []).append(i)

    if len(clusters) < 2:

```

```

        return 0.0

scores = []
for i in range(n):
    own_cluster = assignments[i]
    own_members = [j for j in clusters[own_cluster] if j != i]

    if len(own_members) == 0:
        scores.append(0.0)
        continue

    a = sum(euclidean_distance(data[i], data[j]) for j in own_members) / len(own_members)

    b = float("inf")
    for cluster_id, members in clusters.items():
        if cluster_id == own_cluster:
            continue
        avg_dist = sum(euclidean_distance(data[i], data[j]) for j in members) / len(members)
        b = min(b, avg_dist)

    if max(a, b) == 0:
        scores.append(0.0)
    else:
        scores.append((b - a) / max(a, b))

return sum(scores) / len(scores)

def find_best_k(data, max_k=10):
    print("Elbow method:")
    inertias = []
    for k in range(1, max_k + 1):
        assignments, centroids = kmeans(data, k)
        inertia = compute_inertia(data, assignments, centroids)
        inertias.append(inertia)
        print(f" K={k}: inertia={inertia:.2f}")

    print("\nSilhouette scores:")
    for k in range(2, max_k + 1):
        assignments, centroids = kmeans(data, k)
        score = silhouette_score(data, assignments)
        print(f" K={k}: silhouette={score:.4f}")

    return inertias

```

Step 3: DBSCAN from scratch

```

def dbscan(data, eps, min_samples):
    n = len(data)
    labels = [-1] * n
    cluster_id = 0

    def region_query(point_idx):
        neighbors = []

```

```

    for i in range(n):
        if euclidean_distance(data[point_idx], data[i]) <= eps:
            neighbors.append(i)
    return neighbors

visited = [False] * n

for i in range(n):
    if visited[i]:
        continue
    visited[i] = True

    neighbors = region_query(i)

    if len(neighbors) < min_samples:
        labels[i] = -1
        continue

    labels[i] = cluster_id
    seed_set = list(neighbors)
    seed_set.remove(i)

    j = 0
    while j < len(seed_set):
        q = seed_set[j]

        if not visited[q]:
            visited[q] = True
            q_neighbors = region_query(q)
            if len(q_neighbors) >= min_samples:
                for nb in q_neighbors:
                    if nb not in seed_set:
                        seed_set.append(nb)

        if labels[q] == -1:
            labels[q] = cluster_id

        j += 1

    cluster_id += 1

return labels

```

Step 4: Gaussian Mixture Model (EM algorithm)

```

def gmm(data, k, max_iterations=100, seed=42):
    random.seed(seed)
    n = len(data)
    d = len(data[0])

    indices = random.sample(range(n), k)
    means = [list(data[i]) for i in indices]
    variances = [1.0] * k
    weights = [1.0 / k] * k

```

```

def gaussian_pdf(x, mean, variance):
    d = len(x)
    coeff = 1.0 / ((2 * math.pi * variance) ** (d / 2))
    exponent = -sum((xi - mi) ** 2 for xi, mi in zip(x, mean)) / (2 * variance)
    return coeff * math.exp(max(exponent, -500))

for iteration in range(max_iterations):
    responsibilities = []
    for i in range(n):
        probs = []
        for j in range(k):
            probs.append(weights[j] * gaussian_pdf(data[i], means[j], variances[j]))
        total = sum(probs)
        if total == 0:
            total = 1e-300
        responsibilities.append([p / total for p in probs])

    old_means = [list(m) for m in means]

    for j in range(k):
        r_sum = sum(responsibilities[i][j] for i in range(n))
        if r_sum < 1e-10:
            continue

        weights[j] = r_sum / n

        for dim in range(d):
            means[j][dim] = sum(
                responsibilities[i][j] * data[i][dim] for i in range(n)
            ) / r_sum

        variances[j] = sum(
            responsibilities[i][j]
            * sum((data[i][dim] - means[j][dim]) ** 2 for dim in range(d))
            for i in range(n)
        ) / (r_sum * d)
        variances[j] = max(variances[j], 1e-6)

    shift = sum(
        euclidean_distance(old_means[j], means[j]) for j in range(k)
    )
    if shift < 1e-6:
        print(f" GMM converged at iteration {iteration + 1}")
        break

assignments = []
for i in range(n):
    assignments.append(responsibilities[i].index(max(responsibilities[i])))

return assignments, means, weights, responsibilities

```

Step 5: Generate test data and run everything

```
def make_blobs(centers, n_per_cluster=50, spread=0.5, seed=42):
    random.seed(seed)
    data = []
    true_labels = []
    for label, (cx, cy) in enumerate(centers):
        for _ in range(n_per_cluster):
            x = cx + random.gauss(0, spread)
            y = cy + random.gauss(0, spread)
            data.append([x, y])
            true_labels.append(label)
    return data, true_labels

def make_moons(n_samples=200, noise=0.1, seed=42):
    random.seed(seed)
    data = []
    labels = []
    n_half = n_samples // 2
    for i in range(n_half):
        angle = math.pi * i / n_half
        x = math.cos(angle) + random.gauss(0, noise)
        y = math.sin(angle) + random.gauss(0, noise)
        data.append([x, y])
        labels.append(0)
    for i in range(n_half):
        angle = math.pi * i / n_half
        x = 1 - math.cos(angle) + random.gauss(0, noise)
        y = 1 - math.sin(angle) - 0.5 + random.gauss(0, noise)
        data.append([x, y])
        labels.append(1)
    return data, labels

if __name__ == "__main__":
    centers = [[2, 2], [8, 3], [5, 8]]
    data, true_labels = make_blobs(centers, n_per_cluster=50, spread=0.8)

    print("=== K-Means on 3 blobs ===")
    assignments, centroids = kmeans(data, k=3)
    print(f" Centroids: {[round(c, 2) for c in cent] for cent in centroids}")
    sil = silhouette_score(data, assignments)
    print(f" Silhouette score: {sil:.4f}")

    print("\n=== Elbow Method ===")
    find_best_k(data, max_k=6)

    print("\n=== DBSCAN on 3 blobs ===")
    db_labels = dbscan(data, eps=1.5, min_samples=5)
    n_clusters = len(set(db_labels) - {-1})
    n_noise = db_labels.count(-1)
    print(f" Found {n_clusters} clusters, {n_noise} noise points")
```

```

print("\n=== GMM on 3 blobs ===")
gmm_assignments, gmm_means, gmm_weights, _ = gmm(data, k=3)
print(f" Means: {[round(m, 2) for m in mean] for mean in gmm_means}")
print(f" Weights: {[round(w, 3) for w in gmm_weights]}")
gmm_sil = silhouette_score(data, gmm_assignments)
print(f" Silhouette score: {gmm_sil:.4f}")

print("\n=== DBSCAN on moons (non-spherical clusters) ===")
moon_data, moon_labels = make_moons(n_samples=200, noise=0.1)
moon_db = dbscan(moon_data, eps=0.3, min_samples=5)
n_moon_clusters = len(set(moon_db) - {-1})
n_moon_noise = moon_db.count(-1)
print(f" Found {n_moon_clusters} clusters, {n_moon_noise} noise points")

print("\n=== K-Means on moons (will fail to separate) ===")
moon_km, moon_centroids = kmeans(moon_data, k=2)
moon_sil = silhouette_score(moon_data, moon_km)
print(f" Silhouette score: {moon_sil:.4f}")
print(" K-Means splits moons poorly because they are not spherical")

print("\n=== Anomaly detection with DBSCAN ===")
anomaly_data = list(data)
anomaly_data.append([20.0, 20.0])
anomaly_data.append([-5.0, -5.0])
anomaly_data.append([15.0, 0.0])
anomaly_labels = dbscan(anomaly_data, eps=1.5, min_samples=5)
anomalies = [
    anomaly_data[i]
    for i in range(len(anomaly_labels))
    if anomaly_labels[i] == -1
]
print(f" Detected {len(anomalies)} anomalies")
for a in anomalies[-3:]:
    print(f" Point {[round(v, 2) for v in a]}")

```

Use It

With scikit-learn, the same algorithms are one-liners:

```

from sklearn.cluster import KMeans, DBSCAN, AgglomerativeClustering
from sklearn.mixture import GaussianMixture
from sklearn.metrics import silhouette_score as sklearn_silhouette

km = KMeans(n_clusters=3, random_state=42).fit(data)
db = DBSCAN(eps=1.5, min_samples=5).fit(data)
agg = AgglomerativeClustering(n_clusters=3).fit(data)
gmm_model = GaussianMixture(n_components=3, random_state=42).fit(data)

```

The from-scratch versions show you exactly what these libraries compute. K-Means iterates between assigning and recomputing. DBSCAN grows clusters from dense seeds. GMM alternates between expectation and maximization. The library versions add numerical stability, smarter initialization (K-Means++), and GPU acceleration, but the core logic is the same.

This chapter ships an artifact. The course version of this lesson produces a reusable

prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/07-unsupervised-learning>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/07-unsupervised-learning/code>

1. Implement K-Means++ initialization: instead of picking random centroids, pick the first randomly and each subsequent centroid with probability proportional to its squared distance from the nearest existing centroid. Compare convergence speed to random initialization.
2. Add hierarchical agglomerative clustering to the code. Implement Ward's linkage and produce a dendrogram (as a nested list of merges). Cut it at different levels and compare to K-Means results.
3. Build a simple anomaly detection pipeline: run DBSCAN and GMM on the same data, flag points that both methods agree are outliers (noise in DBSCAN, low probability in GMM). Measure the overlap and discuss when the methods disagree.

Key Terms

Term	What people say	What it actually means
Clustering	"Grouping similar things"	Partitioning data into subsets where within-group similarity exceeds between-group similarity, measured by a specific distance metric
Centroid	"The center of a cluster"	The mean of all points assigned to a cluster; used by K-Means as the cluster representative
Inertia	"How tight the clusters are"	Sum of squared distances from each point to its assigned centroid; lower is tighter
Silhouette score	"How well-separated clusters are"	For each point, $(b - a) / \max(a, b)$ where a is mean intra-cluster distance and b is mean nearest-cluster distance
Core point	"A point in a dense region"	A point with at least <code>min_samples</code> neighbors within <code>eps</code> distance, in DBSCAN
EM algorithm	"Soft K-Means"	Expectation-Maximization: iteratively compute membership probabilities (E-step) and update distribution parameters (M-step)
Dendrogram	"A tree of clusters"	A tree diagram showing the order and distance at which clusters were merged in hierarchical clustering
Anomaly	"An outlier"	A data point that does not conform to the expected pattern, identified as noise by DBSCAN or low-probability by GMM

Further Reading

- Stanford CS229 - Unsupervised Learning - Andrew Ng's lecture notes on clustering and

EM

- scikit-learn Clustering Guide - practical comparison of all clustering algorithms with visual examples
- DBSCAN original paper (Ester et al., 1996) - the paper that introduced density-based clustering

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/07-unsupervised-learning>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/07-unsupervised-learning/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/07-unsupervised-learning>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Feature Engineering & Selection

A good feature is worth a thousand data points.

Type: Build **Languages:** Python **Prerequisites:** Phase 1 (Statistics for ML, Linear Algebra), Phase 2 Lessons 1-7 **Time:** ~90 minutes

Learning Objectives

- Implement numerical transforms (standardization, min-max scaling, log transform, binning) and explain when each is appropriate
- Build one-hot, label, and target encoding for categorical features and identify the data leakage risk in target encoding
- Construct a TF-IDF vectorizer from scratch and explain why it outperforms raw word counts for text classification
- Apply filter-based feature selection (variance threshold, correlation, mutual information) to reduce dimensionality

The Problem

You have a dataset. You pick an algorithm. You train it. The results are mediocre. You try a fancier algorithm. Still mediocre. You spend a week tuning hyperparameters. Marginal improvement.

Then someone transforms the raw data into better features and a simple logistic regression beats your tuned gradient-boosted ensemble.

This happens constantly. In classical ML, the representation of the data matters more than the choice of algorithm. A house price model with “square footage” and “number of bedrooms” will beat a model with “address as a raw string” no matter how sophisticated the learner is. The algorithm can only work with what you give it.

Feature engineering is the process of transforming raw data into representations that make patterns easier for models to find. Feature selection is the process of throwing away features that add noise without adding signal. Together, they are the highest-leverage activity in classical ML.

The Concept

The Feature Pipeline

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/08-feature-engineering>

Numerical Features

Raw numbers are rarely model-ready. Common transforms:

Scaling: Put features on the same range so distance-based algorithms (K-Means, KNN, SVM) treat all features equally. Min-max scaling maps to [0, 1]. Standardization (z-score) maps to mean=0, std=1.

Log transform: Compresses right-skewed distributions (income, population, word counts). Turns multiplicative relationships into additive ones.

Binning: Converts continuous values into categories. Useful when the relationship between feature and target is non-linear but step-wise (e.g., age groups).

Polynomial features: Creates x^2 , x^3 , $x_1 \cdot x_2$ terms. Lets linear models capture non-linear relationships at the cost of more features.

Categorical Features

Models need numbers. Categories need encoding.

One-hot encoding: Creates a binary column for each category. “color = red/blue/green” becomes three columns: is_red, is_blue, is_green. Works well for low-cardinality features but explodes with many categories.

Label encoding: Maps each category to an integer: red=0, blue=1, green=2. Introduces false ordering (the model might think green > blue > red). Only appropriate for tree-based models that split on individual values.

Target encoding: Replaces each category with the mean of the target variable for that category. Powerful but dangerous: high risk of data leakage. Must be computed only on training data and applied to test data.

Text Features

Count vectorizer: Counts how many times each word appears in a document. “the cat sat on the mat” becomes {the: 2, cat: 1, sat: 1, on: 1, mat: 1}.

TF-IDF: Term Frequency-Inverse Document Frequency. Weighs words by how unique they are across documents. Common words like “the” get low weight. Rare, distinctive words get high weight.

$TF(\text{word}, \text{doc}) = \text{count}(\text{word in doc}) / \text{total words in doc}$

$IDF(\text{word}) = \log(\text{total docs} / \text{docs containing word})$

$TF\text{-}IDF = TF * IDF$

Missing Values

Real data has holes. Strategies:

- **Drop rows:** Only when missing data is rare and random
- **Mean/median imputation:** Simple, preserves distribution shape (median is more robust to outliers)
- **Mode imputation:** For categorical features
- **Indicator column:** Add a binary column “was_this_missing” before imputing. The fact that data is missing can itself be informative
- **Forward/backward fill:** For time series data

Feature Interaction

Sometimes the relationship is in the combination. “Height” and “weight” alone are less predictive than “BMI = weight / height²”. Feature interactions multiply the feature space, so use domain knowledge to pick the right ones.

Feature Selection

More features is not always better. Irrelevant features add noise, increase training time, and can cause overfitting.

Filter methods (pre-model): - Correlation: remove features highly correlated with each other (redundant) - Mutual information: measures how much knowing a feature reduces uncertainty about the target - Variance threshold: remove features that barely vary

Wrapper methods (model-based): - L1 regularization (Lasso): drives irrelevant feature weights to exactly zero - Recursive feature elimination: train, remove least important feature, repeat

Why selection matters: A model with 10 good features will usually outperform a model with 10 good features and 90 noisy ones. The noisy features give the model opportunities to overfit on training data patterns that do not generalize.

Interactive figure: feature-scaling. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/08-feature-engineering>

Build It

Step 1: Numerical transforms from scratch

```
import math

def min_max_scale(values):
    min_val = min(values)
    max_val = max(values)
    if max_val == min_val:
        return [0.0] * len(values)
    return [(v - min_val) / (max_val - min_val) for v in values]

def standardize(values):
    n = len(values)
    mean = sum(values) / n
    variance = sum((v - mean) ** 2 for v in values) / n
    std = math.sqrt(variance) if variance > 0 else 1.0
    return [(v - mean) / std for v in values]

def log_transform(values):
    return [math.log(v + 1) for v in values]

def bin_values(values, n_bins=5):
```

```

min_val = min(values)
max_val = max(values)
bin_width = (max_val - min_val) / n_bins
if bin_width == 0:
    return [0] * len(values)
result = []
for v in values:
    bin_idx = int((v - min_val) / bin_width)
    bin_idx = min(bin_idx, n_bins - 1)
    result.append(bin_idx)
return result

def polynomial_features(row, degree=2):
    n = len(row)
    result = list(row)
    if degree >= 2:
        for i in range(n):
            result.append(row[i] ** 2)
        for i in range(n):
            for j in range(i + 1, n):
                result.append(row[i] * row[j])
    return result

Step 2: Categorical encoding from scratch

def one_hot_encode(values):
    categories = sorted(set(values))
    cat_to_idx = {cat: i for i, cat in enumerate(categories)}
    n_cats = len(categories)

    encoded = []
    for v in values:
        row = [0] * n_cats
        row[cat_to_idx[v]] = 1
        encoded.append(row)

    return encoded, categories

def label_encode(values):
    categories = sorted(set(values))
    cat_to_int = {cat: i for i, cat in enumerate(categories)}
    return [cat_to_int[v] for v in values], cat_to_int

def target_encode(feature_values, target_values, smoothing=10):
    global_mean = sum(target_values) / len(target_values)

    category_stats = {}
    for feat, target in zip(feature_values, target_values):
        if feat not in category_stats:
            category_stats[feat] = {"sum": 0.0, "count": 0}
        category_stats[feat]["sum"] += target

```

```

        category_stats[feat]["count"] += 1

encoding = {}
for cat, stats in category_stats.items():
    cat_mean = stats["sum"] / stats["count"]
    weight = stats["count"] / (stats["count"] + smoothing)
    encoding[cat] = weight * cat_mean + (1 - weight) * global_mean

return [encoding[v] for v in feature_values], encoding

```

Step 3: Text features from scratch

```

def count_vectorize(documents):
    vocab = {}
    idx = 0
    for doc in documents:
        for word in doc.lower().split():
            if word not in vocab:
                vocab[word] = idx
                idx += 1

    vectors = []
    for doc in documents:
        vec = [0] * len(vocab)
        for word in doc.lower().split():
            vec[vocab[word]] += 1
        vectors.append(vec)

    return vectors, vocab

def tfidf(documents):
    n_docs = len(documents)

    vocab = {}
    idx = 0
    for doc in documents:
        for word in doc.lower().split():
            if word not in vocab:
                vocab[word] = idx
                idx += 1

    doc_freq = {}
    for doc in documents:
        seen = set()
        for word in doc.lower().split():
            if word not in seen:
                doc_freq[word] = doc_freq.get(word, 0) + 1
                seen.add(word)

    vectors = []
    for doc in documents:
        words = doc.lower().split()
        word_count = len(words)

```

```

tf_map = {}
for word in words:
    tf_map[word] = tf_map.get(word, 0) + 1

vec = [0.0] * len(vocab)
for word, count in tf_map.items():
    tf = count / word_count
    idf = math.log(n_docs / doc_freq[word])
    vec[vocab[word]] = tf * idf
vectors.append(vec)

return vectors, vocab

```

Step 4: Missing value imputation from scratch

```

def impute_mean(values):
    present = [v for v in values if v is not None]
    if not present:
        return [0.0] * len(values), 0.0
    mean = sum(present) / len(present)
    return [v if v is not None else mean for v in values], mean

def impute_median(values):
    present = sorted(v for v in values if v is not None)
    if not present:
        return [0.0] * len(values), 0.0
    n = len(present)
    if n % 2 == 0:
        median = (present[n // 2 - 1] + present[n // 2]) / 2
    else:
        median = present[n // 2]
    return [v if v is not None else median for v in values], median

def impute_mode(values):
    present = [v for v in values if v is not None]
    if not present:
        return values, None
    counts = {}
    for v in present:
        counts[v] = counts.get(v, 0) + 1
    mode = max(counts, key=counts.get)
    return [v if v is not None else mode for v in values], mode

def add_missing_indicator(values):
    return [0 if v is not None else 1 for v in values]

```

Step 5: Feature selection from scratch

```

def correlation(x, y):
    n = len(x)
    mean_x = sum(x) / n

```



```

mean_y = sum(y) / n
cov = sum((xi - mean_x) * (yi - mean_y) for xi, yi in zip(x, y)) / n
std_x = math.sqrt(sum((xi - mean_x) ** 2 for xi in x) / n)
std_y = math.sqrt(sum((yi - mean_y) ** 2 for yi in y) / n)
if std_x == 0 or std_y == 0:
    return 0.0
return cov / (std_x * std_y)

def mutual_information(feature, target, n_bins=10):
    feat_min = min(feature)
    feat_max = max(feature)
    bin_width = (feat_max - feat_min) / n_bins if feat_max != feat_min else 1.0
    feat_binned = [
        min(int((f - feat_min) / bin_width), n_bins - 1) for f in feature
    ]

    n = len(feature)
    target_classes = sorted(set(target))

    feat_bins = sorted(set(feat_binned))
    p_feat = {}
    for b in feat_bins:
        p_feat[b] = feat_binned.count(b) / n

    p_target = {}
    for t in target_classes:
        p_target[t] = target.count(t) / n

    mi = 0.0
    for b in feat_bins:
        for t in target_classes:
            joint_count = sum(
                1 for fb, tv in zip(feat_binned, target) if fb == b and tv == t
            )
            p_joint = joint_count / n
            if p_joint > 0:
                mi += p_joint * math.log(p_joint / (p_feat[b] * p_target[t]))

    return mi

def variance_threshold(features, threshold=0.01):
    n_features = len(features[0])
    n_samples = len(features)
    selected = []

    for j in range(n_features):
        col = [features[i][j] for i in range(n_samples)]
        mean = sum(col) / n_samples
        var = sum((v - mean) ** 2 for v in col) / n_samples
        if var >= threshold:
            selected.append(j)

```

```

    return selected

def remove_correlated(features, threshold=0.9):
    n_features = len(features[0])
    n_samples = len(features)

    to_remove = set()
    for i in range(n_features):
        if i in to_remove:
            continue
        col_i = [features[r][i] for r in range(n_samples)]
        for j in range(i + 1, n_features):
            if j in to_remove:
                continue
            col_j = [features[r][j] for r in range(n_samples)]
            corr = abs(correlation(col_i, col_j))
            if corr >= threshold:
                to_remove.add(j)

    return [i for i in range(n_features) if i not in to_remove]

```

Step 6: Full pipeline and demo

```
import random
```

```

def make_housing_data(n=200, seed=42):
    random.seed(seed)
    data = []
    for _ in range(n):
        sqft = random.uniform(500, 5000)
        bedrooms = random.choice([1, 2, 3, 4, 5])
        age = random.uniform(0, 50)
        neighborhood = random.choice(["downtown", "suburbs", "rural"])
        has_pool = random.choice([True, False])

        sqft_with_missing = sqft if random.random() > 0.05 else None
        age_with_missing = age if random.random() > 0.08 else None

        price = (
            50 * sqft
            + 20000 * bedrooms
            - 1000 * age
            + (50000 if neighborhood == "downtown" else 10000 if neighborhood == "suburbs" else 0)
            + (15000 if has_pool else 0)
            + random.gauss(0, 20000)
        )

        data.append({
            "sqft": sqft_with_missing,
            "bedrooms": bedrooms,
            "age": age_with_missing,
            "neighborhood": neighborhood,

```

```

        "has_pool": has_pool,
        "price": price,
    })
    return data

if __name__ == "__main__":
    data = make_housing_data(200)

    print("=== Raw Data Sample ===")
    for row in data[:3]:
        print(f" {row}")

    sqft_raw = [d["sqft"] for d in data]
    age_raw = [d["age"] for d in data]
    prices = [d["price"] for d in data]

    print("\n=== Missing Value Handling ===")
    sqft_missing = sum(1 for v in sqft_raw if v is None)
    age_missing = sum(1 for v in age_raw if v is None)
    print(f" sqft missing: {sqft_missing}/{len(sqft_raw)}")
    print(f" age missing: {age_missing}/{len(age_raw)}")

    sqft_indicator = add_missing_indicator(sqft_raw)
    age_indicator = add_missing_indicator(age_raw)
    sqft_imputed, sqft_fill = impute_median(sqft_raw)
    age_imputed, age_fill = impute_mean(age_raw)
    print(f" sqft filled with median: {sqft_fill:.0f}")
    print(f" age filled with mean: {age_fill:.1f}")

    print("\n=== Numerical Transforms ===")
    sqft_scaled = standardize(sqft_imputed)
    age_scaled = min_max_scale(age_imputed)
    sqft_log = log_transform(sqft_imputed)
    age_binned = bin_values(age_imputed, n_bins=5)
    print(f" sqft standardized: mean={sum(sqft_scaled)/len(sqft_scaled):.4f}, std={math.sqrt(s)}")
    print(f" age min-max: [{min(age_scaled):.2f}, {max(age_scaled):.2f}]")
    print(f" age bins: {sorted(set(age_binned))}")

    print("\n=== Categorical Encoding ===")
    neighborhoods = [d["neighborhood"] for d in data]

    ohe, ohe_cats = one_hot_encode(neighborhoods)
    print(f" One-hot categories: {ohe_cats}")
    print(f" Sample encoding: {neighborhoods[0]} -> {ohe[0]}")

    le, le_map = label_encode(neighborhoods)
    print(f" Label encoding map: {le_map}")

    te, te_map = target_encode(neighborhoods, prices, smoothing=10)
    print(f" Target encoding: {{(k: round(v) for k, v in te_map.items())}}")

    print("\n=== Text Features ===")
    descriptions = [

```

```

    "large modern house with pool",
    "small cozy cottage near downtown",
    "spacious family home with large yard",
    "modern apartment downtown with view",
    "rustic cabin in rural area",
]
cv, cv_vocab = count_vectorize(descriptions)
print(f" Vocabulary size: {len(cv_vocab)}")
print(f" Doc 0 non-zero features: {sum(1 for v in cv[0] if v > 0)}")

tf, tf_vocab = tfidf(descriptions)
print(f" TF-IDF vocabulary size: {len(tf_vocab)}")
top_words = sorted(tf_vocab.keys(), key=lambda w: tf[0][tf_vocab[w]], reverse=True)[:3]
print(f" Doc 0 top TF-IDF words: {top_words}")

print("\n=== Polynomial Features ===")
sample_row = [sqft_scaled[0], age_scaled[0]]
poly = polynomial_features(sample_row, degree=2)
print(f" Input: {[round(v, 4) for v in sample_row]}")
print(f" Polynomial: {[round(v, 4) for v in poly]}")
print(f" Features: [x1, x2, x1^2, x2^2, x1*x2]")

print("\n=== Feature Selection ===")
feature_matrix = [
    [sqft_scaled[i], age_scaled[i], float(sqft_indicator[i]), float(age_indicator[i])]
    + ohe[i]
    for i in range(len(data))
]

print(f" Total features: {len(feature_matrix[0])}")

surviving_var = variance_threshold(feature_matrix, threshold=0.01)
print(f" After variance threshold (0.01): {len(surviving_var)} features kept")

surviving_corr = remove_correlated(feature_matrix, threshold=0.9)
print(f" After correlation filter (0.9): {len(surviving_corr)} features kept")

binary_prices = [1 if p > sum(prices) / len(prices) else 0 for p in prices]
print("\n Mutual information with target:")
feature_names = ["sqft", "age", "sqft_missing", "age_missing"] + [f"neigh_{c}" for c in ohe]
for j in range(len(feature_matrix[0])):
    col = [feature_matrix[i][j] for i in range(len(feature_matrix))]
    mi = mutual_information(col, binary_prices, n_bins=10)
    print(f" {feature_names[j]}: MI={mi:.4f}")

print("\n Correlation with price:")
for j in range(len(feature_matrix[0])):
    col = [feature_matrix[i][j] for i in range(len(feature_matrix))]
    corr = correlation(col, prices)
    print(f" {feature_names[j]}: r={corr:.4f}")

```

Use It

With scikit-learn, these transforms are composable pipelines:

```
from sklearn.preprocessing import StandardScaler, OneHotEncoder, PolynomialFeatures
from sklearn.impute import SimpleImputer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.feature_selection import mutual_info_classif, VarianceThreshold
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
```

```
numeric_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
])
```

```
categorical_pipe = Pipeline([
    ("encoder", OneHotEncoder(sparse_output=False)),
])
```

```
preprocessor = ColumnTransformer([
    ("num", numeric_pipe, ["sqft", "age"]),
    ("cat", categorical_pipe, ["neighborhood"]),
])
```

The from-scratch versions show exactly what happens inside each transform. The library versions add edge-case handling, sparse matrix support, and pipeline composition, but the math is the same.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/08-feature-engineering>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/08-feature-engineering/code>

1. Add robust scaling (using median and interquartile range instead of mean and standard deviation) to the numerical transforms. Compare it to standard scaling on data with extreme outliers.
2. Implement leave-one-out target encoding: for each row, compute the target mean excluding that row's own target value. Show how this reduces overfitting compared to naive target encoding.
3. Build an automated feature selection pipeline that combines variance threshold, correlation filtering, and mutual information ranking. Apply it to the housing dataset and compare model performance (use a simple linear regression) with all features vs selected features.

Key Terms

Term	What people say	What it actually means
Feature engineering	“Making new columns”	Transforming raw data into representations that expose patterns to the model
Standardization	“Making it normal”	Subtracting the mean and dividing by standard deviation so the feature has mean=0 and std=1
One-hot encoding	“Making dummy variables”	Creating one binary column per category, where exactly one column is 1 for each row
Target encoding	“Using the answer to encode”	Replacing each category with the average target value for that category, with smoothing to prevent overfitting
TF-IDF	“Fancy word counts”	Term Frequency times Inverse Document Frequency: words weighted by how distinctive they are across the corpus
Imputation	“Filling in blanks”	Replacing missing values with estimated values (mean, median, mode, or model-predicted)
Feature selection	“Throwing out bad columns”	Removing features that add noise or redundancy, keeping only those with signal about the target
Mutual information	“How much one thing tells you about another”	A measure of the reduction in uncertainty about variable Y gained by observing variable X
Data leakage	“Accidentally cheating”	Using information during training that would not be available at prediction time, giving falsely optimistic results

Further Reading

- Feature Engineering and Selection (Max Kuhn & Kjell Johnson) - free online book covering the full landscape of feature engineering
- scikit-learn Preprocessing Guide - practical reference for all standard transforms
- Target Encoding Done Right (Micci-Barreca, 2001) - the original paper on target encoding with smoothing

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/08-feature-engineering>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/08-feature-engineering/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/08-feature-engineering>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Model Evaluation

A model is only as good as the way you measure it.

Type: Build **Languages:** Python **Prerequisites:** Phase 1 (Probability & Distributions, Statistics for ML), Phase 2 Lessons 1-8 **Time:** ~90 minutes

Learning Objectives

- Implement K-fold and stratified K-fold cross-validation from scratch and explain why stratification matters for imbalanced data
- Compute precision, recall, F1, AUC-ROC, and regression metrics (MSE, RMSE, MAE, R-squared) from scratch
- Interpret learning curves to diagnose whether a model suffers from high bias or high variance
- Identify common evaluation mistakes including data leakage, wrong metric selection, and test set contamination

The Problem

You trained a model. It gets 95% accuracy on your data. Is it good?

Maybe. Maybe not. If 95% of your data belongs to one class, a model that always predicts that class gets 95% accuracy while being completely useless. If you evaluated on the same data you trained on, the 95% number is meaningless because the model just memorized the answers. If your dataset has a time component and you randomly shuffled before splitting, your model might be using future data to predict the past.

Model evaluation is where most ML projects go wrong. The wrong metric makes a bad model look good. The wrong split lets a model cheat. The wrong comparison makes you pick the worse model. Getting evaluation right is not optional. It is the difference between a model that works in production and one that fails the moment it sees real data.

The Concept

Train, Validation, Test

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/09-model-evaluation>

Three splits, three purposes:

- **Training set:** the model learns from this data. It sees these examples during training.
- **Validation set:** used to tune hyperparameters and select between models. The model never trains on this data, but your decisions are influenced by it.

- **Test set:** touched exactly once, at the very end, to report final performance. If you look at test performance and then go back to change your model, it is no longer a test set. It has become a second validation set.

The test set is your hold-out guarantee that the reported performance reflects how the model will do on truly unseen data.

K-Fold Cross-Validation

With small datasets, a single train/validation split wastes data and gives noisy estimates. K-fold cross-validation uses all the data for both training and validation:

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/09-model-evaluation>

1. Split data into K equal-sized folds
2. For each fold, train on K-1 folds and validate on the remaining fold
3. Average the K validation scores

K=5 or K=10 are standard choices. Every data point gets used for validation exactly once. The average score is a more stable estimate than any single split.

Stratified K-fold: preserves the class distribution in each fold. If your dataset is 70% class A and 30% class B, each fold will have roughly the same ratio. This is important for imbalanced datasets where a random split might put all minority samples in one fold.

Classification Metrics

Confusion matrix: the foundation. For binary classification:

	Predicted Positive	Predicted Negative
Actually Positive	True Positive (TP)	False Negative (FN)
Actually Negative	False Positive (FP)	True Negative (TN)

From this matrix, all other metrics follow:

- **Accuracy** = $(TP + TN) / (TP + TN + FP + FN)$. Fraction of correct predictions. Misleading when classes are imbalanced.
- **Precision** = $TP / (TP + FP)$. Of all things predicted positive, how many actually were? Use when false positives are costly (e.g., spam filter marking real email as spam).
- **Recall** (sensitivity) = $TP / (TP + FN)$. Of all actual positives, how many did we catch? Use when false negatives are costly (e.g., cancer screening missing a tumor).
- **F1 score** = $2 * \text{precision} * \text{recall} / (\text{precision} + \text{recall})$. Harmonic mean of precision and recall. Balances both when neither clearly dominates.
- **AUC-ROC:** Area Under the Receiver Operating Characteristic curve. Plots true positive rate vs false positive rate at various classification thresholds. AUC = 0.5 means random guessing, AUC = 1.0 means perfect separation. Threshold-independent: it measures how well the model ranks positives above negatives, regardless of the cutoff you pick.

Regression Metrics

- **MSE** (Mean Squared Error) = $\text{mean}((y_{\text{true}} - y_{\text{pred}})^2)$. Penalizes large errors quadratically. Sensitive to outliers.
- **RMSE** (Root Mean Squared Error) = $\sqrt{\text{MSE}}$. Same units as the target variable. Easier to interpret than MSE.

- **MAE** (Mean Absolute Error) = $\text{mean}(|y_{\text{true}} - y_{\text{pred}}|)$. Treats all errors linearly. More robust to outliers than MSE.
- **R-squared** = $1 - \text{SS}_{\text{res}} / \text{SS}_{\text{tot}}$, where $\text{SS}_{\text{res}} = \sum (y_{\text{true}} - y_{\text{pred}})^2$ and $\text{SS}_{\text{tot}} = \sum (y_{\text{true}} - y_{\text{mean}})^2$. Fraction of variance explained by the model. $R^2 = 1.0$ is perfect. $R^2 = 0.0$ means the model is no better than always predicting the mean. R^2 can be negative if the model is worse than the mean.

Learning Curves

Plot training and validation scores as a function of training set size:

- **High bias (underfitting)**: both curves converge to a low score. Adding more data will not help. You need a more complex model.
- **High variance (overfitting)**: training score is high but validation score is much lower. The gap between them is large. Adding more data should help.

Validation Curves

Plot training and validation scores as a function of a hyperparameter:

- At low complexity: both scores are low (underfitting)
- At the right complexity: both scores are high and close together
- At high complexity: training score stays high but validation score drops (overfitting)

The optimal hyperparameter value is where the validation score peaks.

Common Evaluation Mistakes

Data leakage: information from the test set leaks into training. Examples: fitting a scaler on the full dataset before splitting, including future data in time series prediction, using a feature that is derived from the target. Always split first, then preprocess.

Class imbalance: 99% of transactions are legitimate, 1% are fraud. A model that always predicts “legitimate” gets 99% accuracy. Use precision, recall, F1, or AUC-ROC instead.

Wrong metric: optimizing accuracy when you should optimize recall (medical diagnosis), or optimizing RMSE when your data has heavy outliers (use MAE instead).

Not using stratified splits: with imbalanced data, a random split might put very few minority samples in the validation fold, giving unstable estimates.

Testing too often: every time you look at test performance and adjust, you overfit to the test set. The test set is single-use.

Interactive figure: precision-recall-threshold. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/09-model-evaluation>

Build It

Step 1: Train/validation/test split

```
import random
import math
```

```
def train_val_test_split(X, y, train_ratio=0.6, val_ratio=0.2, seed=42):
    random.seed(seed)
    n = len(X)
    indices = list(range(n))
    random.shuffle(indices)

    train_end = int(n * train_ratio)
    val_end = int(n * (train_ratio + val_ratio))

    train_idx = indices[:train_end]
    val_idx = indices[train_end:val_end]
    test_idx = indices[val_end:]

    X_train = [X[i] for i in train_idx]
    y_train = [y[i] for i in train_idx]
    X_val = [X[i] for i in val_idx]
    y_val = [y[i] for i in val_idx]
    X_test = [X[i] for i in test_idx]
    y_test = [y[i] for i in test_idx]

    return X_train, y_train, X_val, y_val, X_test, y_test
```

Step 2: K-fold and stratified K-fold cross-validation

```
def kfold_split(n, k=5, seed=42):
    random.seed(seed)
    indices = list(range(n))
    random.shuffle(indices)

    fold_size = n // k
    folds = []

    for i in range(k):
        start = i * fold_size
        end = start + fold_size if i < k - 1 else n
        val_idx = indices[start:end]
        train_idx = indices[:start] + indices[end:]
        folds.append((train_idx, val_idx))

    return folds

def stratified_kfold_split(y, k=5, seed=42):
    random.seed(seed)

    class_indices = {}
    for i, label in enumerate(y):
        class_indices.setdefault(label, []).append(i)

    for label in class_indices:
        random.shuffle(class_indices[label])

    folds = [{"train": [], "val": []} for _ in range(k)]
```

```

for label, indices in class_indices.items():
    fold_size = len(indices) // k
    for i in range(k):
        start = i * fold_size
        end = start + fold_size if i < k - 1 else len(indices)
        val_part = indices[start:end]
        train_part = indices[:start] + indices[end:]
        folds[i]["val"].extend(val_part)
        folds[i]["train"].extend(train_part)

return [(f["train"], f["val"]) for f in folds]

def cross_validate(X, y, model_fn, k=5, metric_fn=None, stratified=False):
    n = len(X)

    if stratified:
        folds = stratified_kfold_split(y, k)
    else:
        folds = kfold_split(n, k)

    scores = []
    for train_idx, val_idx in folds:
        X_train = [X[i] for i in train_idx]
        y_train = [y[i] for i in train_idx]
        X_val = [X[i] for i in val_idx]
        y_val = [y[i] for i in val_idx]

        model = model_fn()
        model.fit(X_train, y_train)
        predictions = [model.predict(x) for x in X_val]

        if metric_fn:
            score = metric_fn(y_val, predictions)
        else:
            score = sum(1 for yt, yp in zip(y_val, predictions) if yt == yp) / len(y_val)
        scores.append(score)

    return scores

```

Step 3: Confusion matrix and classification metrics

```

def confusion_matrix(y_true, y_pred):
    tp = sum(1 for yt, yp in zip(y_true, y_pred) if yt == 1 and yp == 1)
    tn = sum(1 for yt, yp in zip(y_true, y_pred) if yt == 0 and yp == 0)
    fp = sum(1 for yt, yp in zip(y_true, y_pred) if yt == 0 and yp == 1)
    fn = sum(1 for yt, yp in zip(y_true, y_pred) if yt == 1 and yp == 0)
    return tp, tn, fp, fn

def accuracy(y_true, y_pred):
    tp, tn, fp, fn = confusion_matrix(y_true, y_pred)
    total = tp + tn + fp + fn
    return (tp + tn) / total if total > 0 else 0.0

```

```

def precision(y_true, y_pred):
    tp, tn, fp, fn = confusion_matrix(y_true, y_pred)
    return tp / (tp + fp) if (tp + fp) > 0 else 0.0

def recall(y_true, y_pred):
    tp, tn, fp, fn = confusion_matrix(y_true, y_pred)
    return tp / (tp + fn) if (tp + fn) > 0 else 0.0

def f1_score(y_true, y_pred):
    p = precision(y_true, y_pred)
    r = recall(y_true, y_pred)
    return 2 * p * r / (p + r) if (p + r) > 0 else 0.0

def roc_curve(y_true, y_scores):
    thresholds = sorted(set(y_scores), reverse=True)
    tpr_list = []
    fpr_list = []

    total_positives = sum(y_true)
    total_negatives = len(y_true) - total_positives

    for threshold in thresholds:
        y_pred = [1 if s >= threshold else 0 for s in y_scores]
        tp = sum(1 for yt, yp in zip(y_true, y_pred) if yt == 1 and yp == 1)
        fp = sum(1 for yt, yp in zip(y_true, y_pred) if yt == 0 and yp == 1)

        tpr = tp / total_positives if total_positives > 0 else 0.0
        fpr = fp / total_negatives if total_negatives > 0 else 0.0

        tpr_list.append(tpr)
        fpr_list.append(fpr)

    return fpr_list, tpr_list, thresholds

def auc_roc(y_true, y_scores):
    fpr_list, tpr_list, _ = roc_curve(y_true, y_scores)

    pairs = sorted(zip(fpr_list, tpr_list))
    fpr_sorted = [p[0] for p in pairs]
    tpr_sorted = [p[1] for p in pairs]

    area = 0.0
    for i in range(1, len(fpr_sorted)):
        width = fpr_sorted[i] - fpr_sorted[i - 1]
        height = (tpr_sorted[i] + tpr_sorted[i - 1]) / 2
        area += width * height

    return area

```

Step 4: Regression metrics

```
def mse(y_true, y_pred):
    n = len(y_true)
    return sum((yt - yp) ** 2 for yt, yp in zip(y_true, y_pred)) / n

def rmse(y_true, y_pred):
    return math.sqrt(mse(y_true, y_pred))

def mae(y_true, y_pred):
    n = len(y_true)
    return sum(abs(yt - yp) for yt, yp in zip(y_true, y_pred)) / n

def r_squared(y_true, y_pred):
    mean_y = sum(y_true) / len(y_true)
    ss_res = sum((yt - yp) ** 2 for yt, yp in zip(y_true, y_pred))
    ss_tot = sum((yt - mean_y) ** 2 for yt in y_true)
    if ss_tot == 0:
        return 0.0
    return 1.0 - ss_res / ss_tot
```

Step 5: Learning curves

```
def learning_curve(X, y, model_fn, metric_fn, train_sizes=None, val_ratio=0.2, seed=42):
    random.seed(seed)
    n = len(X)
    indices = list(range(n))
    random.shuffle(indices)

    val_size = int(n * val_ratio)
    val_idx = indices[:val_size]
    pool_idx = indices[val_size:]

    X_val = [X[i] for i in val_idx]
    y_val = [y[i] for i in val_idx]

    if train_sizes is None:
        train_sizes = [int(len(pool_idx) * r) for r in [0.1, 0.2, 0.4, 0.6, 0.8, 1.0]]

    train_scores = []
    val_scores = []

    for size in train_sizes:
        subset = pool_idx[:size]
        X_train = [X[i] for i in subset]
        y_train = [y[i] for i in subset]

        model = model_fn()
        model.fit(X_train, y_train)

        train_pred = [model.predict(x) for x in X_train]
```

```

val_pred = [model.predict(x) for x in X_val]

train_scores.append(metric_fn(y_train, train_pred))
val_scores.append(metric_fn(y_val, val_pred))

return train_sizes, train_scores, val_scores

```

Step 6: A simple classifier for testing, plus the full demo

```

class SimpleLogistic:
    def __init__(self, lr=0.1, epochs=100):
        self.lr = lr
        self.epochs = epochs
        self.weights = None
        self.bias = 0.0

    def sigmoid(self, z):
        z = max(-500, min(500, z))
        return 1.0 / (1.0 + math.exp(-z))

    def fit(self, X, y):
        n_features = len(X[0])
        self.weights = [0.0] * n_features
        self.bias = 0.0

        for _ in range(self.epochs):
            for xi, yi in zip(X, y):
                z = sum(w * x for w, x in zip(self.weights, xi)) + self.bias
                pred = self.sigmoid(z)
                error = yi - pred
                for j in range(n_features):
                    self.weights[j] += self.lr * error * xi[j]
                self.bias += self.lr * error

    def predict_proba(self, x):
        z = sum(w * xi for w, xi in zip(self.weights, x)) + self.bias
        return self.sigmoid(z)

    def predict(self, x):
        return 1 if self.predict_proba(x) >= 0.5 else 0

class SimpleLinearRegression:
    def __init__(self, lr=0.001, epochs=200):
        self.lr = lr
        self.epochs = epochs
        self.weights = None
        self.bias = 0.0

    def fit(self, X, y):
        n_features = len(X[0])
        self.weights = [0.0] * n_features
        self.bias = 0.0
        n = len(X)

```

```

        for _ in range(self.epochs):
            for xi, yi in zip(X, y):
                pred = sum(w * x for w, x in zip(self.weights, xi)) + self.bias
                error = yi - pred
                for j in range(n_features):
                    self.weights[j] += self.lr * error * xi[j] / n
                self.bias += self.lr * error / n

    def predict(self, x):
        return sum(w * xi for w, xi in zip(self.weights, x)) + self.bias

def standardize(values):
    n = len(values)
    mean = sum(values) / n
    var = sum((v - mean) ** 2 for v in values) / n
    std = math.sqrt(var) if var > 0 else 1.0
    return [(v - mean) / std for v in values], mean, std

def make_classification_data(n=300, seed=42):
    random.seed(seed)
    X = []
    y = []
    for _ in range(n):
        x1 = random.gauss(0, 1)
        x2 = random.gauss(0, 1)
        label = 1 if (x1 + x2 + random.gauss(0, 0.5)) > 0 else 0
        X.append([x1, x2])
        y.append(label)
    return X, y

def make_regression_data(n=200, seed=42):
    random.seed(seed)
    X = []
    y = []
    for _ in range(n):
        x1 = random.uniform(0, 10)
        x2 = random.uniform(0, 5)
        target = 3 * x1 + 2 * x2 + random.gauss(0, 2)
        X.append([x1, x2])
        y.append(target)
    return X, y

def make_imbalanced_data(n=300, minority_ratio=0.05, seed=42):
    random.seed(seed)
    X = []
    y = []
    for _ in range(n):
        if random.random() < minority_ratio:
            x1 = random.gauss(3, 0.5)

```

```

        x2 = random.gauss(3, 0.5)
        label = 1
    else:
        x1 = random.gauss(0, 1)
        x2 = random.gauss(0, 1)
        label = 0
    X.append([x1, x2])
    y.append(label)
return X, y

if __name__ == "__main__":
    X_clf, y_clf = make_classification_data(300)

    print("=== Train/Validation/Test Split ===")
    X_train, y_train, X_val, y_val, X_test, y_test = train_val_test_split(X_clf, y_clf)
    print(f" Train: {len(X_train)}, Val: {len(X_val)}, Test: {len(X_test)}")
    print(f" Train class distribution: {sum(y_train)}/{len(y_train)} positive")
    print(f" Val class distribution: {sum(y_val)}/{len(y_val)} positive")

    model = SimpleLogistic(lr=0.1, epochs=200)
    model.fit(X_train, y_train)

    print("\n=== Classification Metrics ===")
    y_pred = [model.predict(x) for x in X_test]
    tp, tn, fp, fn = confusion_matrix(y_test, y_pred)
    print(f" Confusion matrix: TP={tp}, TN={tn}, FP={fp}, FN={fn}")
    print(f" Accuracy: {accuracy(y_test, y_pred):.4f}")
    print(f" Precision: {precision(y_test, y_pred):.4f}")
    print(f" Recall: {recall(y_test, y_pred):.4f}")
    print(f" F1 Score: {f1_score(y_test, y_pred):.4f}")

    y_scores = [model.predict_proba(x) for x in X_test]
    auc = auc_roc(y_test, y_scores)
    print(f" AUC-ROC: {auc:.4f}")

    print("\n=== K-Fold Cross-Validation (K=5) ===")
    cv_scores = cross_validate(
        X_clf, y_clf,
        model_fn=lambda: SimpleLogistic(lr=0.1, epochs=200),
        k=5,
        metric_fn=accuracy,
    )
    mean_cv = sum(cv_scores) / len(cv_scores)
    std_cv = math.sqrt(sum((s - mean_cv) ** 2 for s in cv_scores) / len(cv_scores))
    print(f" Fold scores: {[round(s, 4) for s in cv_scores]}")
    print(f" Mean: {mean_cv:.4f} (+/- {std_cv:.4f})")

    print("\n=== Stratified K-Fold Cross-Validation (K=5) ===")
    strat_scores = cross_validate(
        X_clf, y_clf,
        model_fn=lambda: SimpleLogistic(lr=0.1, epochs=200),
        k=5,
        metric_fn=accuracy,
    )

```



```

        stratified=True,
    )
    strat_mean = sum(strat_scores) / len(strat_scores)
    strat_std = math.sqrt(sum((s - strat_mean) ** 2 for s in strat_scores) / len(strat_scores))
    print(f" Fold scores: {[round(s, 4) for s in strat_scores]}")
    print(f" Mean: {strat_mean:.4f} (+/- {strat_std:.4f})")

    print("\n=== Imbalanced Data: Why Accuracy Lies ===")
    X_imb, y_imb = make_imbalanced_data(300, minority_ratio=0.05)
    positives = sum(y_imb)
    print(f" Class distribution: {positives} positive, {len(y_imb) - positives} negative ({pos

always_negative = [0] * len(y_imb)
print(f" Always-negative baseline:")
print(f" Accuracy: {accuracy(y_imb, always_negative):.4f}")
print(f" Precision: {precision(y_imb, always_negative):.4f}")
print(f" Recall: {recall(y_imb, always_negative):.4f}")
print(f" F1 Score: {f1_score(y_imb, always_negative):.4f}")

X_tr_i, y_tr_i, X_v_i, y_v_i, X_te_i, y_te_i = train_val_test_split(X_imb, y_imb)
model_imb = SimpleLogistic(lr=0.5, epochs=500)
model_imb.fit(X_tr_i, y_tr_i)
y_pred_imb = [model_imb.predict(x) for x in X_te_i]
print(f"\n Trained model on imbalanced data:")
print(f" Accuracy: {accuracy(y_te_i, y_pred_imb):.4f}")
print(f" Precision: {precision(y_te_i, y_pred_imb):.4f}")
print(f" Recall: {recall(y_te_i, y_pred_imb):.4f}")
print(f" F1 Score: {f1_score(y_te_i, y_pred_imb):.4f}")

print("\n=== Regression Metrics ===")
X_reg, y_reg = make_regression_data(200)

col0 = [x[0] for x in X_reg]
col1 = [x[1] for x in X_reg]
col0_s, m0, s0 = standardize(col0)
col1_s, m1, s1 = standardize(col1)
X_reg_scaled = [[col0_s[i], col1_s[i]] for i in range(len(X_reg))]

X_tr_r, y_tr_r, X_v_r, y_v_r, X_te_r, y_te_r = train_val_test_split(X_reg_scaled, y_reg)
reg_model = SimpleLinearRegression(lr=0.01, epochs=500)
reg_model.fit(X_tr_r, y_tr_r)
y_pred_r = [reg_model.predict(x) for x in X_te_r]

print(f" MSE: {mse(y_te_r, y_pred_r):.4f}")
print(f" RMSE: {rmse(y_te_r, y_pred_r):.4f}")
print(f" MAE: {mae(y_te_r, y_pred_r):.4f}")
print(f" R-squared: {r_squared(y_te_r, y_pred_r):.4f}")

mean_baseline = [sum(y_tr_r) / len(y_tr_r)] * len(y_te_r)
print(f"\n Mean baseline:")
print(f" MSE: {mse(y_te_r, mean_baseline):.4f}")
print(f" R-squared: {r_squared(y_te_r, mean_baseline):.4f}")

print("\n=== Learning Curve ===")

```

```

sizes, train_sc, val_sc = learning_curve(
    X_clf, y_clf,
    model_fn=lambda: SimpleLogistic(lr=0.1, epochs=200),
    metric_fn=accuracy,
)
print(f" {'Size':>6} {'Train':>8} {'Val':>8}")
for s, tr, va in zip(sizes, train_sc, val_sc):
    print(f" {s:>6} {tr:>8.4f} {va:>8.4f}")

print("\n=== Statistical Model Comparison ===")
model_a_scores = cross_validate(
    X_clf, y_clf,
    model_fn=lambda: SimpleLogistic(lr=0.1, epochs=100),
    k=5, metric_fn=accuracy,
)
model_b_scores = cross_validate(
    X_clf, y_clf,
    model_fn=lambda: SimpleLogistic(lr=0.1, epochs=500),
    k=5, metric_fn=accuracy,
)
diffs = [a - b for a, b in zip(model_a_scores, model_b_scores)]
mean_diff = sum(diffs) / len(diffs)
std_diff = math.sqrt(sum((d - mean_diff) ** 2 for d in diffs) / len(diffs))
t_stat = mean_diff / (std_diff / math.sqrt(len(diffs))) if std_diff > 0 else 0.0
print(f" Model A (100 epochs) mean: {sum(model_a_scores)/len(model_a_scores):.4f}")
print(f" Model B (500 epochs) mean: {sum(model_b_scores)/len(model_b_scores):.4f}")
print(f" Mean difference: {mean_diff:.4f}")
print(f" Paired t-statistic: {t_stat:.4f}")
print(f" (|t| > 2.78 for significance at p<0.05 with df=4)")

```

Use It

With scikit-learn, evaluation is built into the workflow:

```

from sklearn.model_selection import cross_val_score, StratifiedKFold, learning_curve
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, confusion_matrix, mean_squared_error, r2_score,
)
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()
scores = cross_val_score(model, X, y, cv=StratifiedKFold(5), scoring="f1")

```

The from-scratch versions show exactly what cross-validation does (no magic, just for-loops and index tracking), how each metric is computed (just counting TP/FP/TN/FN), and why stratification matters (preserving class ratios in each fold). The library versions add parallelism, more scoring options, and integration with pipelines.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/09-model-evaluation>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/09-model-evaluation/code>

1. Implement precision-recall curves: plot precision vs recall at different thresholds. Compute the average precision (area under the PR curve). Compare the PR curve to the ROC curve on an imbalanced dataset and explain when each is more informative.
2. Build a nested cross-validation loop: the outer loop evaluates model performance, the inner loop tunes hyperparameters. Use it to compare two models fairly without leaking validation data into the evaluation.
3. Implement a permutation test for model comparison: shuffle the labels, retrain, and measure performance. Repeat 100 times to build a null distribution. Compute the p-value for the observed model performance against this distribution.

Key Terms

Term	What people say	What it actually means
Overfitting	"Memorizing the training data"	The model captures noise in the training data, performing well on training but poorly on unseen data
Cross-validation	"Testing on different subsets"	Systematically rotating which portion of data is used for validation, averaging results across all rotations
Precision	"How many predicted positives are correct"	$TP / (TP + FP)$: the fraction of positive predictions that are actually positive
Recall	"How many actual positives we found"	$TP / (TP + FN)$: the fraction of actual positives that were correctly identified
AUC-ROC	"How well the model separates classes"	The area under the curve of true positive rate vs false positive rate across all thresholds, from 0.5 (random) to 1.0 (perfect)
R-squared	"How much variance is explained"	$1 - (\text{sum of squared residuals} / \text{total sum of squares})$: the fraction of target variance captured by the model
Data leakage	"The model cheated"	Using information during training that would not be available at prediction time, leading to optimistic evaluation
Learning curve	"How performance changes with more data"	A plot of training and validation scores vs training set size, revealing underfitting or overfitting
Stratified split	"Keeping class ratios balanced"	Splitting data so each subset has the same proportion of each class as the full dataset

Further Reading

- scikit-learn Model Selection Guide - comprehensive reference on cross-validation, metrics, and hyperparameter tuning
- Beyond Accuracy: Precision and Recall (Google ML Crash Course) - clear explanation with interactive examples

- A Survey of Cross-Validation Procedures (Arlot & Celisse, 2010) - rigorous treatment of when and why different CV strategies work

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/09-model-evaluation>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/09-model-evaluation/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/09-model-evaluation>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Bias-Variance Tradeoff

Every model error comes from one of three sources: bias, variance, or noise. You can only control the first two.

Type: Learn **Language:** Python **Prerequisites:** Phase 2, Lessons 01-09 (ML basics, regression, classification, evaluation) **Time:** ~75 minutes

Learning Objectives

- Derive the bias-variance decomposition of expected prediction error and explain the role of irreducible noise
- Diagnose whether a model suffers from high bias or high variance using training and test error patterns
- Explain how regularization techniques (L1, L2, dropout, early stopping) trade bias for variance
- Implement experiments that visualize the bias-variance tradeoff across models of increasing complexity

The Problem

You trained a model. It has some error on test data. Where does that error come from?

If your model is too simple (linear regression on a curved dataset), it will consistently miss the true pattern. That is bias. If your model is too complex (degree-20 polynomial on 15 data points), it will fit the training data perfectly but give wildly different predictions on new data. That is variance.

You cannot minimize both at the same time for a fixed model capacity. Push bias down and variance goes up. Push variance down and bias goes up. Understanding this tradeoff is the single most useful diagnostic skill in machine learning. It tells you whether to make your model more complex or less complex, whether to get more data or engineer better features, whether to regularize more or less.

The Concept

Bias: Systematic Error

Bias measures how far off your model's average prediction is from the true value. If you trained the same model on many different training sets drawn from the same distribution and averaged the predictions, bias is the gap between that average and the truth.

High bias means the model is too rigid to capture the real pattern. A straight line fit to a parabola will always miss the curve, no matter how much data you give it. This is underfitting.

High bias (underfitting):

Model always predicts roughly the same wrong thing.

Training error: HIGH
 Test error: HIGH
 Gap between them: SMALL

Variance: Sensitivity to Training Data

Variance measures how much your predictions change when you train on different subsets of data. If small changes in the training set cause large changes in the model, variance is high.

High variance means the model is fitting noise in the training data, not the underlying signal. A degree-20 polynomial will thread through every training point but oscillate wildly between them. This is overfitting.

High variance (overfitting):

Model fits training data perfectly but fails on new data.
 Training error: LOW
 Test error: HIGH
 Gap between them: LARGE

The Decomposition

For any point x , the expected prediction error under squared loss decomposes exactly:

Expected Error = Bias² + Variance + Irreducible Noise

where:

Bias² = $E[f_{\text{hat}}(x) - f(x)]^2$
 Variance = $E[(f_{\text{hat}}(x) - E[f_{\text{hat}}(x)])^2]$
 Noise = $E[(y - f(x))^2]$ (sigma²)

- $f(x)$ is the true function
- $f_{\text{hat}}(x)$ is your model's prediction
- $E[\dots]$ is the expectation over different training sets
- y is the observed label (true function plus noise)

The noise term is irreducible. No model can do better than sigma² on noisy data. Your job is to find the right balance between bias² and variance.

Model Complexity vs Error

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/10-bias-variance>

The classic U-shaped curve:

Complexity	Bias	Variance	Total Error
Too low	HIGH	LOW	HIGH (underfitting)
Just right	MODERATE	MODERATE	LOWEST
Too high	LOW	HIGH	HIGH (overfitting)

Regularization as Bias-Variance Control

Regularization deliberately increases bias to reduce variance. It constrains the model so it cannot chase noise.

- **L2 (Ridge):** Shrinks all weights toward zero. Keeps all features but reduces their influence.
- **L1 (Lasso):** Pushes some weights exactly to zero. Performs feature selection.
- **Dropout:** Randomly disables neurons during training. Forces redundant representations.
- **Early stopping:** Stops training before the model fully fits the training data.

The regularization strength (lambda, dropout rate, number of epochs) directly controls where you sit on the bias-variance curve. More regularization means more bias, less variance.

Double Descent: The Modern Perspective

Classical theory says: after the sweet spot, more complexity always hurts. But research since 2019 has shown something unexpected. If you keep increasing model capacity far past the interpolation threshold (where the model has enough parameters to perfectly fit training data), test error can decrease again.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/10-bias-variance>

This “double descent” phenomenon explains why massively overparameterized neural networks (with far more parameters than training examples) still generalize well. The classical bias-variance tradeoff is not wrong, but it is incomplete for the modern regime.

Key observations about double descent: - It happens in linear models, decision trees, and neural networks - More data can actually hurt in the interpolation region (sample-wise double descent) - More training epochs can cause it too (epoch-wise double descent) - Regularization smooths out the peak but does not eliminate it

Why does this happen? At the interpolation threshold, the model has just enough capacity to fit all training points. It is forced into a very specific solution that threads through every point, and small perturbations in the data cause large changes in the fit. This is where variance peaks. Past the threshold, the model has many possible solutions that fit the data perfectly. The learning algorithm (e.g., gradient descent with implicit regularization) tends to pick the simplest one among them. This implicit bias toward simple solutions is why overparameterized models generalize.

Regime	Parameters vs Samples	Behavior
Underparameterized	$p \ll n$	Classical tradeoff applies
Interpolation threshold	$p \sim n$	Variance peaks, test error spikes
Overparameterized	$p \gg n$	Implicit regularization kicks in, test error drops

For practical purposes: if you are using neural networks or large tree ensembles, do not stop at the interpolation threshold. Either stay well below it (with explicit regularization) or go well past it. The worst place to be is right at the threshold.

Diagnosing Your Model

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/10-bias-variance>

Symptom	Diagnosis	Fix
High train error, high test error	Bias	More features, complex model, less regularization
Low train error, high test error	Variance	More data, regularization, simpler model, dropout
Low train error, low test error	Good fit	Ship it
Train error decreasing, test error increasing	Overfitting in progress	Early stopping

Practical Strategies

When bias is the problem: - Add polynomial or interaction features - Use a more flexible model (tree ensemble instead of linear) - Reduce regularization strength - Train longer (if not yet converged)

When variance is the problem: - Get more training data - Use bagging (random forests) - Increase regularization (higher lambda, more dropout) - Feature selection (remove noisy features) - Use cross-validation to detect it early

Ensemble Methods and Variance Reduction

Ensemble methods are the most practical tool for fighting variance.

Bagging (Bootstrap Aggregating) trains multiple models on different bootstrap samples of the training data, then averages their predictions. Each individual model has high variance, but the average has much lower variance. Random forests are bagging applied to decision trees.

Why it works mathematically: if you average N independent predictions, each with variance σ^2 , the variance of the average is σ^2 / N . The models are not truly independent (they all see similar data), so the reduction is less than $1/N$, but it is still substantial.

Boosting reduces bias by building models sequentially, where each new model focuses on the errors of the ensemble so far. Gradient boosting and AdaBoost are the main examples. Boosting can overfit if you add too many models, so you need early stopping or regularization.

Method	Primary Effect	Bias Change	Variance Change
Bagging	Reduces variance	No change	Decreases
Boosting	Reduces bias	Decreases	Can increase
Stacking	Reduces both	Depends on meta-learner	Depends on base models
Dropout	Implicit bagging	Slight increase	Decreases

Practical rule: if your base model has high variance (deep trees, high-degree polynomials), use bagging. If your base model has high bias (shallow stumps, simple linear models), use boosting.

Learning Curves

Learning curves plot training and validation error as a function of training set size. They are the most practical diagnostic tool you have. Unlike a single train/test comparison, learning curves show you the trajectory of your model and tell you whether more data will help.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/10-bias-variance>

How to read them:

Scenario	Training Error	Validation Error	Gap	What It Means	What to Do
High bias	High	High	Small	Model cannot capture the pattern	More features, complex model, less regularization
High variance	Low	High	Large	Model memorizes training data	More data, regularization, simpler model
Good fit	Moderate	Moderate	Small	Model generalizes well	Ship it
High variance, improving	Low	Decreasing with more data	Shrinking	Variance problem that data can fix	Collect more data
High bias, flat	High	High and flat	Small and flat	More data will NOT help	Change model architecture

The critical insight: if both curves have plateaued and the gap is small but both errors are high, more data is useless. You need a better model. If the gap is large and still shrinking, more data will help.

How to Generate Learning Curves

There are two approaches:

Approach 1: Vary training set size, fixed model. Hold the model and hyperparameters constant. Train on increasingly large subsets of the training data. Measure training error and validation error at each size. This is the standard learning curve.

Approach 2: Vary model complexity, fixed data. Hold the data constant. Sweep a complexity parameter (polynomial degree, tree depth, number of layers). Measure training error and validation error at each complexity. This is a validation curve and shows the bias-variance tradeoff directly.

Both approaches complement each other. The first tells you if more data will help. The second tells you if a different model will help. Run both before making decisions about your next step.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/10-bias-variance>

Interactive figure: bias-variance. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/10-bias-variance>

Build It

The code in `code/bias_variance.py` runs the full bias-variance decomposition experiment. Here is the approach, step by step.

Step 1: Generate Synthetic Data from a Known Function

We use $f(x) = \sin(1.5x) + 0.5x$ with Gaussian noise. Knowing the true function lets us compute exact bias and variance.

```
def true_function(x):
    return np.sin(1.5 * x) + 0.5 * x

def generate_data(n_samples=30, noise_std=0.5, x_range=(-3, 3), seed=None):
    rng = np.random.RandomState(seed)
    x = rng.uniform(x_range[0], x_range[1], n_samples)
    y = true_function(x) + rng.normal(0, noise_std, n_samples)
    return x, y
```

Step 2: Bootstrap Sampling and Polynomial Fitting

For each polynomial degree, we draw many bootstrap training sets, fit the polynomial, and record predictions on a fixed test grid. This gives us a distribution of predictions at each test point.

```
def fit_polynomial(x_train, y_train, degree, lam=0.0):
    X = np.column_stack([x_train ** d for d in range(degree + 1)])
    if lam > 0:
        penalty = lam * np.eye(X.shape[1])
        penalty[0, 0] = 0
        w = np.linalg.solve(X.T @ X + penalty, X.T @ y_train)
    else:
        w = np.linalg.lstsq(X, y_train, rcond=None)[0]
    return w
```

We fit on 200 different bootstrap samples. Each bootstrap sample is drawn from the same underlying distribution but contains different points.

Step 3: Computing Bias², Variance Decomposition

With 200 sets of predictions at each test point, we can compute the decomposition directly from the definition:

```
mean_pred = predictions.mean(axis=0)
bias_sq = np.mean((mean_pred - y_true) ** 2)
variance = np.mean(predictions.var(axis=0))
total_error = np.mean(np.mean((predictions - y_true) ** 2, axis=1))
```

- `mean_pred` is $E[\hat{f}(x)]$ estimated from bootstrap samples
- `bias_sq` is the squared gap between average prediction and truth
- `variance` is the average spread of predictions across bootstrap samples
- `total_error` should approximately equal $\text{bias}^2 + \text{variance} + \text{noise}$

Step 4: Learning Curves

Learning curves sweep training set size while holding model complexity fixed. They show whether your model is data-limited or capacity-limited.

```
def demo_learning_curves():
    sizes = [10, 15, 20, 30, 50, 75, 100, 150, 200, 300]
    degree = 5

    for n in sizes:
        train_errors = []
        test_errors = []
        for seed in range(50):
            x_train, y_train = generate_data(n_samples=n, seed=seed * 100)
            w = fit_polynomial(x_train, y_train, degree)
            train_pred = predict_polynomial(x_train, w)
            train_mse = np.mean((train_pred - y_train) ** 2)
            test_pred = predict_polynomial(x_test, w)
            test_mse = np.mean((test_pred - y_test) ** 2)
            train_errors.append(train_mse)
            test_errors.append(test_mse)
        # Average over runs gives the learning curve point
```

For a high-variance model (degree 5 with small data), you see: - Training error starts low and increases as more data makes memorization harder - Test error starts high and decreases as the model gets more signal - The gap shrinks with more data

For a high-bias model (degree 1), both errors converge quickly to the same high value and more data does not help.

Step 5: Regularization Sweep

The code also includes `demo_regularization_sweep()`, which fixes a high-degree polynomial (degree 15) and sweeps Ridge regularization strength from 0.001 to 100. This shows the bias-variance tradeoff from a different angle: instead of varying model complexity, we vary the constraint strength.

```
def demo_regularization_sweep():
    alphas = [0.001, 0.005, 0.01, 0.05, 0.1, 0.5, 1.0, 5.0, 10.0, 50.0, 100.0]
    for alpha in alphas:
        results = bias_variance_decomposition([15], lam=alpha)
        r = results[15]
        print(f"alpha={alpha:.3f}  bias={r['bias_sq']:.4f}  var={r['variance']:.4f}")
```

At low alpha, the degree-15 polynomial is nearly unconstrained. Variance dominates because the model chases noise in each bootstrap sample. At high alpha, the penalty is so strong that the model effectively becomes a near-constant function. Bias dominates. The optimal alpha sits between these extremes.

This is the same U-curve from varying polynomial degree, but controlled by a continuous knob instead of a discrete one. In practice, regularization is the preferred way to control the tradeoff because it allows fine-grained control without changing the feature set.

Use It

sklearn provides `learning_curve` and `validation_curve` to automate these diagnostics without writing bootstrap loops.

Validation Curve: Sweep Model Complexity

```
from sklearn.model_selection import validation_curve
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import Ridge

degrees = list(range(1, 16))
train_scores_all = []
val_scores_all = []

for d in degrees:
    pipe = make_pipeline(PolynomialFeatures(d), Ridge(alpha=0.01))
    train_scores, val_scores = validation_curve(
        pipe, X, y, param_name="polynomialfeatures__degree",
        param_range=[d], cv=5, scoring="neg_mean_squared_error"
    )
    train_scores_all.append(-train_scores.mean())
    val_scores_all.append(-val_scores.mean())
```

This gives you the bias-variance tradeoff curve directly. Where the validation score is worst relative to train score, variance dominates. Where both are bad, bias dominates.

Learning Curve: Sweep Training Set Size

```
from sklearn.model_selection import learning_curve

pipe = make_pipeline(PolynomialFeatures(5), Ridge(alpha=0.01))
train_sizes, train_scores, val_scores = learning_curve(
    pipe, X, y, train_sizes=np.linspace(0.1, 1.0, 10),
    cv=5, scoring="neg_mean_squared_error"
)
train_mse = -train_scores.mean(axis=1)
val_mse = -val_scores.mean(axis=1)
```

Plot `train_mse` and `val_mse` against `train_sizes`. The shape tells you everything about your model.

Cross-Validation with Regularization Sweep

```
from sklearn.model_selection import cross_val_score

alphas = [0.001, 0.01, 0.1, 1.0, 10.0, 100.0]
for alpha in alphas:
    pipe = make_pipeline(PolynomialFeatures(10), Ridge(alpha=alpha))
    scores = cross_val_score(pipe, X, y, cv=5, scoring="neg_mean_squared_error")
    print(f"alpha={alpha:>7.3f} MSE={-scores.mean():.4f} +/- {scores.std():.4f}")
```

This sweeps regularization strength for a fixed model complexity. You will see the same bias-variance tradeoff: low alpha means high variance, high alpha means high bias.

Putting It All Together: A Complete Diagnostic Workflow

In practice, you run these diagnostics in sequence:

1. Train your model. Compute train and test error.
2. If both are high: you have a bias problem. Skip to step 4.
3. If train is low but test is high: you have a variance problem. Generate a learning curve to see if more data will help. If not, regularize.
4. Generate a validation curve sweeping your main complexity parameter. Find the sweet spot.
5. At the sweet spot, generate a learning curve. If the gap is still large, you need more data or regularization.
6. Try Ridge/Lasso with different alpha values using `cross_val_score`. Pick the alpha where cross-validated error is lowest.

This takes 10-15 minutes of compute for most tabular datasets and saves hours of guessing.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/10-bias-variance>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/10-bias-variance/code>

1. Run the decomposition with `noise_std=0` (no noise). What happens to the irreducible error term? Does the optimal complexity change?
2. Increase the training set size from 30 to 300. How does this affect the variance component? Does the optimal polynomial degree shift?
3. Add L2 regularization (Ridge regression) to the experiment. For a fixed high-degree polynomial (degree 15), sweep `lambda` from 0 to 100. Plot bias^2 and variance as functions of `lambda`.
4. Modify the true function from a polynomial to $\sin(x)$. How does the bias-variance decomposition change? Is there still a clear optimal degree?
5. Implement a simple bootstrap aggregating (bagging) wrapper: train 10 models on bootstrap samples and average predictions. Show that this reduces variance without increasing bias much.

Key Terms

Term	What people say	What it actually means
Bias	"The model is too simple"	Systematic error from wrong assumptions. The gap between the average model prediction and truth.
Variance	"The model is overfitting"	Error from sensitivity to training data. How much predictions change across different training sets.
Irreducible error	"Noise in the data"	Error from randomness in the true data-generating process. No model can eliminate it.

Term	What people say	What it actually means
Underfitting	“Not learning enough”	Model has high bias. It misses the real pattern even on training data.
Overfitting	“Memorizing the data”	Model has high variance. It fits noise in training data that does not generalize.
Regularization	“Constraining the model”	Adding a penalty to reduce model complexity, trading bias for lower variance.
Double descent	“More parameters can help”	Test error decreases again when model capacity far exceeds the interpolation threshold.
Model complexity	“How flexible the model is”	The capacity of a model to fit arbitrary patterns. Controlled by architecture, features, or regularization.

Further Reading

- Hastie, Tibshirani, Friedman: Elements of Statistical Learning, Ch. 7 – the definitive treatment of bias-variance decomposition
- Belkin et al., Reconciling modern machine learning practice and the bias-variance trade-off (2019) – the double descent paper
- Nakkiran et al., Deep Double Descent (2019) – epoch-wise and sample-wise double descent
- Scott Fortmann-Roe: Understanding the Bias-Variance Tradeoff – clear visual explanation

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/10-bias-variance>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/10-bias-variance/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/10-bias-variance>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Ensemble Methods

A group of weak learners, combined correctly, becomes a strong learner. This is not a metaphor. It is a theorem.

Type: Build **Language:** Python **Prerequisites:** Phase 2, Lesson 10 (Bias-Variance Tradeoff)
Time: ~120 minutes

Learning Objectives

- Implement AdaBoost and gradient boosting from scratch and explain how boosting sequentially reduces bias
- Build a bagging ensemble and demonstrate how averaging decorrelated models reduces variance without increasing bias
- Compare bagging, boosting, and stacking in terms of what error component each method targets
- Evaluate ensemble diversity and explain why majority voting accuracy improves with more independent weak learners

The Problem

A single decision tree is fast to train and easy to interpret, but it overfits. A single linear model underfits on complex boundaries. You could spend days engineering the perfect model architecture. Or you could combine a bunch of imperfect models and get something better than any of them individually.

Ensemble methods do exactly this. They are the most reliable technique for winning Kaggle competitions on tabular data, they power most production ML systems, and they illustrate the bias-variance tradeoff in action. Bagging reduces variance. Boosting reduces bias. Stacking learns which models to trust on which inputs.

The Concept

Why Ensembles Work

Suppose you have N independent classifiers, each with accuracy $p > 0.5$. The majority vote has accuracy:

$$P(\text{majority correct}) = \sum_{k > N/2} C(N, k) * p^k * (1-p)^{(N-k)}$$

For 21 classifiers each with 60% accuracy, majority vote accuracy is about 74%. With 101 classifiers, it rises to 84%. The errors cancel out when the models make different mistakes.

The key requirement is **diversity**. If all models make the same errors, combining them helps nothing. Ensembles work because they produce diverse models through:

- Different training subsets (bagging)
- Different feature subsets (random forests)

- Sequential error correction (boosting)
- Different model families (stacking)

Bagging (Bootstrap Aggregating)

Bagging creates diversity by training each model on a different bootstrap sample of the training data.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/11-ensemble-methods>

A bootstrap sample is drawn with replacement from the original data, same size as the original. About 63.2% of unique samples appear in each bootstrap. The remaining 36.8% (out-of-bag samples) provide a free validation set.

Bagging reduces variance without increasing bias much. Each individual tree overfits to its bootstrap sample, but the overfitting is different for each tree, so averaging cancels out the noise.

Random Forests are bagging with an extra twist: at each split, only a random subset of features is considered. This forces even more diversity among trees. The typical number of candidate features is $\sqrt{n_features}$ for classification and $n_features / 3$ for regression.

Boosting (Sequential Error Correction)

Boosting trains models sequentially. Each new model focuses on the examples that previous models got wrong.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/11-ensemble-methods>

Boosting reduces bias. Each new model corrects the systematic errors of the ensemble so far. The final prediction is a weighted sum of all models, where better models get higher weights.

The tradeoff: boosting can overfit if you run too many rounds, because it keeps fitting harder examples, some of which may be noise.

AdaBoost

AdaBoost (Adaptive Boosting) was the first practical boosting algorithm. It works with any base learner, typically decision stumps (depth-1 trees).

The algorithm:

1. Initialize sample weights: $w_i = 1/N$ for all i
2. For $t = 1$ to T :
 - a. Train weak learner h_t on weighted data
 - b. Compute weighted error:

$$err_t = \sum(w_i * I(h_t(x_i) \neq y_i)) / \sum(w_i)$$
 - c. Compute model weight:

$$\alpha_t = 0.5 * \ln((1 - err_t) / err_t)$$
 - d. Update sample weights:

$$w_i = w_i * \exp(-\alpha_t * y_i * h_t(x_i))$$
 - e. Normalize weights to sum to 1
3. Final prediction: $H(x) = \text{sign}(\sum(\alpha_t * h_t(x)))$

Models with lower error get higher alpha. Misclassified samples get higher weights so the next model focuses on them.

Gradient Boosting

Gradient boosting generalizes boosting to arbitrary loss functions. Instead of reweighting samples, it fits each new model to the residuals (negative gradient of the loss) of the current ensemble.

1. Initialize: $F_0(x) = \operatorname{argmin}_c \sum(L(y_i, c))$
2. For $t = 1$ to T :
 - a. Compute pseudo-residuals:

$$r_i = -dL(y_i, F_{t-1}(x_i)) / dF_{t-1}(x_i)$$
 - b. Fit a tree h_t to the residuals r_i
 - c. Find optimal step size:

$$\gamma_t = \operatorname{argmin}_{\gamma} \sum(L(y_i, F_{t-1}(x_i) + \gamma * h_t(x_i)))$$
 - d. Update:

$$F_t(x) = F_{t-1}(x) + \text{learning_rate} * \gamma_t * h_t(x)$$
3. Final prediction: $F_T(x)$

For squared error loss, the pseudo-residuals are just the actual residuals: $r_i = y_i - F_{t-1}(x_i)$. Each tree literally fits the errors of the previous ensemble.

The learning rate (shrinkage) controls how much each tree contributes. Smaller learning rates require more trees but generalize better. Typical values: 0.01 to 0.3.

XGBoost: Why It Dominates Tabular Data

XGBoost (eXtreme Gradient Boosting) is gradient boosting with engineering optimizations that make it fast, accurate, and resistant to overfitting:

- **Regularized objective:** L1 and L2 penalties on leaf weights prevent individual trees from being too confident
- **Second-order approximation:** Uses both first and second derivatives of the loss, giving better split decisions
- **Sparsity-aware splits:** Handles missing values natively by learning the best direction for missing data at each split
- **Column subsampling:** Like random forests, samples features at each split for diversity
- **Weighted quantile sketch:** Efficiently finds split points for continuous features on distributed data
- **Cache-aware block structure:** Memory layout optimized for CPU cache lines

For tabular data, XGBoost (and its successor LightGBM) consistently outperforms neural networks. This is not changing anytime soon. If your data fits in a table with rows and columns, start with gradient boosting.

Stacking (Meta-Learning)

Stacking uses the predictions of multiple base models as features for a meta-learner.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/11-ensemble-methods>

The meta-learner learns which base model to trust for which inputs. If the random forest is better at certain regions and the SVM at others, the meta-learner will learn to route accordingly.

To avoid data leakage, base model predictions must be generated via cross-validation on the training set. You never train base models and generate meta-features on the same data.

Voting

The simplest ensemble. Just combine predictions directly.

- **Hard voting:** Majority vote on class labels.
- **Soft voting:** Average predicted probabilities, pick the class with highest average probability. Usually better because it uses confidence information.

Interactive figure: f3-ensemble-average. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/11-ensemble-methods>

Build It

Step 1: Decision Stump (Base Learner)

The code in `code/ensembles.py` implements everything from scratch. We start with a decision stump: a tree with a single split.

```
class DecisionStump:
    def __init__(self):
        self.feature_idx = None
        self.threshold = None
        self.polarity = 1
        self.alpha = None

    def fit(self, X, y, weights):
        n_samples, n_features = X.shape
        best_error = float("inf")

        for f in range(n_features):
            thresholds = np.unique(X[:, f])
            for thresh in thresholds:
                for polarity in [1, -1]:
                    pred = np.ones(n_samples)
                    pred[polarity * X[:, f] < polarity * thresh] = -1
                    error = np.sum(weights[pred != y])
                    if error < best_error:
                        best_error = error
                        self.feature_idx = f
                        self.threshold = thresh
                        self.polarity = polarity

    def predict(self, X):
        n = X.shape[0]
        pred = np.ones(n)
        idx = self.polarity * X[:, self.feature_idx] < self.polarity * self.threshold
```

```

pred[idx] = -1
return pred

```

Step 2: AdaBoost from Scratch

```

class AdaBoostScratch:
    def __init__(self, n_estimators=50):
        self.n_estimators = n_estimators
        self.stumps = []
        self.alphas = []

    def fit(self, X, y):
        n = X.shape[0]
        weights = np.full(n, 1 / n)

        for _ in range(self.n_estimators):
            stump = DecisionStump()
            stump.fit(X, y, weights)
            pred = stump.predict(X)

            err = np.sum(weights[pred != y])
            err = np.clip(err, 1e-10, 1 - 1e-10)

            alpha = 0.5 * np.log((1 - err) / err)
            weights *= np.exp(-alpha * y * pred)
            weights /= weights.sum()

            stump.alpha = alpha
            self.stumps.append(stump)
            self.alphas.append(alpha)

    def predict(self, X):
        total = sum(a * s.predict(X) for a, s in zip(self.alphas, self.stumps))
        return np.sign(total)

```

Step 3: Gradient Boosting from Scratch

```

class GradientBoostingScratch:
    def __init__(self, n_estimators=100, learning_rate=0.1, max_depth=3):
        self.n_estimators = n_estimators
        self.lr = learning_rate
        self.max_depth = max_depth
        self.trees = []
        self.initial_pred = None

    def fit(self, X, y):
        self.initial_pred = np.mean(y)
        current_pred = np.full(len(y), self.initial_pred)

        for _ in range(self.n_estimators):
            residuals = y - current_pred
            tree = SimpleRegressionTree(max_depth=self.max_depth)
            tree.fit(X, residuals)
            update = tree.predict(X)

```

```

        current_pred += self.lr * update
        self.trees.append(tree)

    def predict(self, X):
        pred = np.full(X.shape[0], self.initial_pred)
        for tree in self.trees:
            pred += self.lr * tree.predict(X)
        return pred

```

Step 4: Compare against sklearn

The code verifies that our from-scratch implementations produce similar accuracy to sklearn's AdaBoostClassifier and GradientBoostingClassifier, and compares all methods side by side.

Use It

When to Use Each Method

Method	Reduces	Best for	Watch out for
Bagging / Random Forest	Variance	Noisy data, many features	Does not help with bias
AdaBoost	Bias	Clean data, simple base learners	Sensitive to outliers and noise
Gradient Boosting	Bias	Tabular data, competitions	Slow to train, easy to overfit without tuning
XGBoost / LightGBM	Both	Production tabular ML	Many hyperparameters
Stacking	Both	Getting last 1-2% accuracy	Complex, risk of overfitting meta-learner
Voting	Variance	Quick combination of diverse models	Only helps if models are diverse

The Production Stack for Tabular Data

For most tabular prediction problems, this is the order to try:

1. **LightGBM or XGBoost** with default parameters
2. Tune `n_estimators`, `learning_rate`, `max_depth`, `min_child_weight`
3. If you need the last 0.5%, build a stacking ensemble with 3-5 diverse models
4. Use cross-validation throughout

Neural networks on tabular data are almost always worse than gradient boosting, despite continued research attempts. TabNet, NODE, and similar architectures occasionally match but rarely beat a well-tuned XGBoost.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/11-ensemble-methods>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/11-ensemble-methods/code>

1. Modify the AdaBoost implementation to track training accuracy after each round. Plot accuracy vs. number of estimators. When does it converge?
2. Implement a random forest from scratch by adding random feature subsampling to the regression tree. Train 100 trees with `max_features=sqrt(n_features)` and average predictions. Compare variance reduction to a single tree.
3. In the gradient boosting implementation, add early stopping: track validation loss after each round and stop when it has not improved for 10 consecutive rounds. How many trees does it actually need?
4. Build a stacking ensemble with three base models (logistic regression, decision tree, k-nearest neighbors) and a logistic regression meta-learner. Use 5-fold cross-validation to generate meta-features. Compare to each base model alone.
5. Run XGBoost on the same dataset with default parameters. Compare its accuracy to your from-scratch gradient boosting. Time both. How large is the speed difference?

Key Terms

Term	What people say	What it actually means
Bagging	"Train on random subsets"	Bootstrap aggregating: train models on bootstrap samples, average predictions to reduce variance
Boosting	"Focus on hard examples"	Train models sequentially, each correcting errors of the ensemble so far, to reduce bias
AdaBoost	"Reweight the data"	Boosting via sample weight updates; misclassified points get higher weight for the next learner
Gradient boosting	"Fit the residuals"	Boosting via fitting each new model to the negative gradient of the loss function
XGBoost	"The Kaggle weapon"	Gradient boosting with regularization, second-order optimization, and systems-level speed tricks
Stacking	"Models on top of models"	Use predictions of base models as input features for a meta-learner
Random forest	"Many randomized trees"	Bagging with decision trees, adding random feature subsampling at each split for diversity
Ensemble diversity	"Make different mistakes"	Models must be uncorrelated in their errors for the ensemble to improve over individuals
Out-of-bag error	"Free validation"	Samples not in a bootstrap draw (~36.8%) serve as a validation set without needing a holdout

Further Reading

- Schapire & Freund: Boosting: Foundations and Algorithms – the book by AdaBoost’s creators
- Friedman: Greedy Function Approximation: A Gradient Boosting Machine (2001) – the original gradient boosting paper
- Chen & Guestrin: XGBoost (2016) – the XGBoost paper
- Wolpert: Stacked Generalization (1992) – the original stacking paper
- scikit-learn Ensemble Methods – practical reference

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/11-ensemble-methods>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/11-ensemble-methods/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/11-ensemble-methods>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Hyperparameter Tuning

Hyperparameters are the knobs you turn before training starts. Turning them well is the difference between a mediocre model and a great one.

Type: Build **Language:** Python **Prerequisites:** Phase 2, Lesson 11 (Ensemble Methods)
Time: ~90 minutes

Learning Objectives

- Implement grid search, random search, and Bayesian optimization from scratch and compare their sample efficiency
- Explain why random search outperforms grid search when most hyperparameters have low effective dimensionality
- Build a Bayesian optimization loop using a surrogate model and acquisition function to guide the search
- Design a hyperparameter tuning strategy that avoids overfitting the validation set through proper cross-validation

The Problem

Your gradient boosting model has a learning rate, number of trees, max depth, min samples per leaf, subsample ratio, and column sample ratio. That is six hyperparameters. If each has 5 reasonable values, the grid has $5^6 = 15,625$ combinations. Training each takes 10 seconds. That is 43 hours of compute to try them all.

Grid search is the obvious approach and the worst one at scale. Random search does better with less compute. Bayesian optimization does even better by learning from past evaluations. Knowing which strategy to use, and which hyperparameters actually matter, saves days of wasted GPU time.

The Concept

Parameters vs Hyperparameters

Parameters are learned during training (weights, biases, split thresholds). Hyperparameters are set before training starts and control how learning happens.

Hyperparameter	What it controls	Typical range
Learning rate	Step size per update	0.001 to 1.0
Number of trees/epochs	How long to train	10 to 10,000
Max depth	Model complexity	1 to 30
Regularization (lambda)	Overfitting prevention	0.0001 to 100
Batch size	Gradient estimation noise	16 to 512
Dropout rate	Fraction of neurons dropped	0.0 to 0.5

Grid Search

Grid search evaluates every combination of specified values. It is exhaustive and easy to understand, but scales exponentially with the number of hyperparameters.

Grid for 2 hyperparameters:

```
learning_rate: [0.01, 0.1, 1.0]
max_depth:     [3, 5, 7]
```

Evaluations: $3 \times 3 = 9$ combinations

```
(0.01, 3) (0.01, 5) (0.01, 7)
(0.1, 3)  (0.1, 5)  (0.1, 7)
(1.0, 3)  (1.0, 5)  (1.0, 7)
```

Grid search has a fundamental flaw: if one hyperparameter matters and the other does not, most evaluations are wasted. You get only 3 unique values of the important parameter from 9 evaluations.

Random Search

Random search samples hyperparameters from distributions instead of a grid. With the same budget of 9 evaluations, you get 9 unique values of each hyperparameter.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/12-hyperparameter-tuning>

Why random beats grid (Bergstra & Bengio, 2012):

- Most hyperparameters have low effective dimensionality. Only 1-2 of 6 hyperparameters usually matter for a given problem.
- Grid search wastes evaluations on unimportant dimensions.
- Random search covers the important dimensions more densely for the same budget.
- At 60 random trials, you have a 95% chance of finding a point within 5% of the optimum (if one exists in the search space).

Bayesian Optimization

Random search ignores results. It does not learn that high learning rates cause divergence or that depth 3 consistently outperforms depth 10. Bayesian optimization uses past evaluations to decide where to search next.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/12-hyperparameter-tuning>

The two key components:

Surrogate model: A cheap-to-evaluate model (usually a Gaussian process) that approximates the expensive objective function. It gives both a prediction and an uncertainty estimate at any point in the search space.

Acquisition function: Decides where to evaluate next by balancing exploitation (search near known good points) and exploration (search where uncertainty is high). Common choices:

- **Expected Improvement (EI):** How much improvement over the current best do we expect at this point?
- **Upper Confidence Bound (UCB):** Prediction plus a multiple of uncertainty. Higher UCB means either promising or unexplored.

- **Probability of Improvement (PI):** What is the probability this point beats the current best?

Bayesian optimization typically finds better hyperparameters than random search with 2-5x fewer evaluations. The overhead of fitting the surrogate model is negligible compared to training the actual model.

Early Stopping

Not every training run needs to finish. If a configuration is clearly bad after 10 epochs, stop it and move on. This is early stopping in the context of hyperparameter search.

Strategies: - **Patience-based:** Stop if validation loss has not improved for N consecutive epochs - **Median pruning:** Stop if the trial's intermediate result is worse than the median of completed trials at the same step - **Hyperband:** Allocate small budgets to many configurations, then progressively increase budget for the best ones

Hyperband is particularly effective. It starts 81 configurations with 1 epoch each, keeps the top third, gives them 3 epochs, keeps the top third, and so on. This finds good configurations 10-50x faster than evaluating all configs for the full budget.

Learning Rate Schedulers

The learning rate is almost always the most important hyperparameter. Rather than keeping it fixed, schedulers adjust it during training.

Scheduler	Formula	When to use
Step decay	Multiply by 0.1 every N epochs	Classic CNN training
Cosine annealing	$lr * 0.5 * (1 + \cos(\pi * t / T))$	Modern default
Warmup + decay	Linear increase then cosine decay	Transformers
One-cycle	Increase then decrease over one cycle	Fast convergence
Reduce on plateau	Reduce by factor when metric stalls	Safe default

Hyperparameter Importance

Not all hyperparameters matter equally. Research on random forests (Probst et al., 2019) and gradient boosting shows consistent patterns:

High importance: - Learning rate (always tune first) - Number of estimators / epochs (use early stopping instead of tuning) - Regularization strength

Medium importance: - Max depth / number of layers - Min samples per leaf / weight decay - Subsample ratio

Low importance: - Max features (for random forests) - Specific activation function choice - Batch size (within reasonable range)

Tune the important ones first, leave the rest at defaults.

Practical Strategy

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/12-hyperparameter-tuning>

The concrete workflow:

1. **Start with library defaults.** They are chosen by experienced practitioners and are often 80% of the way there.
2. **Coarse random search.** Wide ranges, 20-50 trials. Use early stopping to kill bad runs fast.
3. **Analyze results.** Which hyperparameters correlate with performance? Narrow the search space.
4. **Fine search.** Bayesian optimization or focused random search in the narrowed space. 50-100 trials.
5. **Retrain on all training data** with the best hyperparameters found.

Cross-Validation Integration

Tuning hyperparameters on a single validation split is risky. The best hyperparameters might overfit to the specific validation fold. Nested cross-validation solves this by using two loops:

- **Outer loop** (evaluation): splits data into train+val and test. Reports unbiased performance.
- **Inner loop** (tuning): splits train+val into train and val. Finds best hyperparameters.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/12-hyperparameter-tuning>

Each outer fold finds its own best hyperparameters independently. The outer scores are an unbiased estimate of generalization performance.

With sklearn:

```
from sklearn.model_selection import cross_val_score, GridSearchCV
from sklearn.ensemble import GradientBoostingRegressor
```

```
inner_cv = GridSearchCV(
    GradientBoostingRegressor(),
    param_grid={
        "learning_rate": [0.01, 0.05, 0.1],
        "max_depth": [2, 3, 5],
        "n_estimators": [50, 100, 200],
    },
    cv=5,
    scoring="neg_mean_squared_error",
)

outer_scores = cross_val_score(
    inner_cv, X, y, cv=5, scoring="neg_mean_squared_error"
)

print(f"Nested CV MSE: {-outer_scores.mean():.4f} +/- {-outer_scores.std():.4f}")
```

This is expensive (5 outer folds x 5 inner folds x 27 grid points = 675 model fits), but it gives you a trustworthy performance estimate. Use it when reporting final results in papers or when the stake of the decision is high.

Practical Tips

Start with the learning rate. It is always the most important hyperparameter for gradient-based methods. A bad learning rate makes everything else irrelevant. Fix other hyperparameters at defaults and sweep learning rate first.

Use log-uniform distributions for learning rate and regularization. The difference between 0.001 and 0.01 matters as much as the difference between 0.1 and 1.0. Searching linearly wastes budget on the large end.

Use early stopping instead of tuning `n_estimators`. For boosting and neural networks, set `n_estimators` or epochs high and let early stopping decide when to stop. This removes one hyperparameter from the search.

Budget allocation. Spend 60% of your tuning budget on the top 2 most important hyperparameters. Spend the remaining 40% on everything else. The top 2 account for most of the performance variation.

Scale matters. Never search batch size on a log scale (16, 32, 64 are fine). Always search learning rate on a log scale. Match the search distribution to how the hyperparameter affects the model.

Model Type	Top Hyperparameters	Recommended Search	Budget
Random Forest	<code>n_estimators</code> , <code>max_depth</code> , <code>min_samples_leaf</code>	Random search, 50 trials	Low (fast training)
Gradient Boosting	<code>learning_rate</code> , <code>n_estimators</code> , <code>max_depth</code>	Bayesian, 100 trials + early stopping	Medium
Neural Network	<code>learning_rate</code> , <code>weight_decay</code> , <code>batch_size</code>	Bayesian or random, 100+ trials	High (slow training)
SVM	<code>C</code> , <code>gamma</code> (RBF kernel)	Grid on log scale, 25-50 trials	Low (2 params)
Lasso/Ridge	<code>alpha</code>	1D search on log scale, 20 trials	Very low
XGBoost	<code>learning_rate</code> , <code>max_depth</code> , <code>subsample</code> , <code>colsample</code>	Bayesian, 100-200 trials + early stopping	Medium

When in doubt: random search with 2x the number of hyperparameters as trials (e.g., 6 hyperparameters = 12+ trials minimum). You will be surprised how often random search with 50 trials beats carefully designed grid search.

Interactive figure: k-fold-cv. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/12-hyperparameter-tuning>

Build It

Step 1: Grid Search from Scratch

The code in `code/tuning.py` implements grid search, random search, and a simple Bayesian optimizer from scratch.

```
def grid_search(model_fn, param_grid, X_train, y_train, X_val, y_val):
    keys = list(param_grid.keys())
    values = list(param_grid.values())
    best_score = -float("inf")
```

```

best_params = None
n_evals = 0

for combo in itertools.product(*values):
    params = dict(zip(keys, combo))
    model = model_fn(**params)
    model.fit(X_train, y_train)
    score = evaluate(model, X_val, y_val)
    n_evals += 1

    if score > best_score:
        best_score = score
        best_params = params

return best_params, best_score, n_evals

```

Step 2: Random Search from Scratch

```

def random_search(model_fn, param_distributions, X_train, y_train,
                  X_val, y_val, n_iter=50, seed=42):
    rng = np.random.RandomState(seed)
    best_score = -float("inf")
    best_params = None

    for _ in range(n_iter):
        params = {k: sample(v, rng) for k, v in param_distributions.items()}
        model = model_fn(**params)
        model.fit(X_train, y_train)
        score = evaluate(model, X_val, y_val)

        if score > best_score:
            best_score = score
            best_params = params

    return best_params, best_score, n_iter

```

Step 3: Bayesian Optimization (Simplified)

The core idea: fit a Gaussian process to observed (hyperparameter, score) pairs, then use an acquisition function to decide where to look next.

```

class SimpleBayesianOptimizer:
    def __init__(self, search_space, n_initial=5):
        self.search_space = search_space
        self.n_initial = n_initial
        self.X_observed = []
        self.y_observed = []

    def _kernel(self, x1, x2, length_scale=1.0):
        dists = np.sum((x1[:, None, :] - x2[None, :, :]) ** 2, axis=2)
        return np.exp(-0.5 * dists / length_scale ** 2)

    def _fit_gp(self, X_new):
        X_obs = np.array(self.X_observed)

```

```

y_obs = np.array(self.y_observed)
y_mean = y_obs.mean()
y_centered = y_obs - y_mean

K = self._kernel(X_obs, X_obs) + 1e-4 * np.eye(len(X_obs))
K_star = self._kernel(X_new, X_obs)

L = np.linalg.cholesky(K)
alpha = np.linalg.solve(L.T, np.linalg.solve(L, y_centered))
mu = K_star @ alpha + y_mean

v = np.linalg.solve(L, K_star.T)
var = 1.0 - np.sum(v ** 2, axis=0)
var = np.maximum(var, 1e-6)

return mu, var

def _expected_improvement(self, mu, var, best_y):
    sigma = np.sqrt(var)
    z = (mu - best_y) / (sigma + 1e-10)
    ei = sigma * (z * norm_cdf(z) + norm_pdf(z))
    return ei

def suggest(self):
    if len(self.X_observed) < self.n_initial:
        return sample_random(self.search_space)

    candidates = [sample_random(self.search_space) for _ in range(500)]
    X_cand = np.array([to_vector(c) for c in candidates])
    mu, var = self._fit_gp(X_cand)
    ei = self._expected_improvement(mu, var, max(self.y_observed))
    return candidates[np.argmax(ei)]

def observe(self, params, score):
    self.X_observed.append(to_vector(params))
    self.y_observed.append(score)

```

The GP surrogate gives two things at each candidate point: a predicted score (mu) and an uncertainty (var). Expected Improvement balances these: it favors points where the model predicts high scores OR where uncertainty is high. Early on, most points have high uncertainty so the optimizer explores. Later, it focuses on the most promising region.

Step 4: Compare All Methods

Run all three methods on the same synthetic objective and compare. This comparison uses a simplified wrapper that calls each optimizer with a direct objective function (no model training), so the API differs from the model-based implementations above:

```

def synthetic_objective(params):
    lr = params["learning_rate"]
    depth = params["max_depth"]
    return -(np.log10(lr) + 2) ** 2 - (depth - 4) ** 2 + 10

param_grid = {
    "learning_rate": [0.001, 0.01, 0.1, 1.0],

```

```

    "max_depth": [2, 3, 4, 5, 6, 7, 8],
}

grid_best = None
grid_score = -float("inf")
grid_history = []
for combo in itertools.product(*param_grid.values()):
    params = dict(zip(param_grid.keys(), combo))
    score = synthetic_objective(params)
    grid_history.append((params, score))
    if score > grid_score:
        grid_score = score
        grid_best = params

param_dist = {
    "learning_rate": ("log_float", 0.001, 1.0),
    "max_depth": ("int", 2, 8),
}

rand_best = None
rand_score = -float("inf")
rand_history = []
rng = np.random.RandomState(42)
for _ in range(28):
    params = {k: sample(v, rng) for k, v in param_dist.items()}
    score = synthetic_objective(params)
    rand_history.append((params, score))
    if score > rand_score:
        rand_score = score
        rand_best = params

optimizer = SimpleBayesianOptimizer(param_dist, n_initial=5)
bayes_history = []
for _ in range(28):
    params = optimizer.suggest()
    score = synthetic_objective(params)
    optimizer.observe(params, score)
    bayes_history.append((params, score))
bayes_score = max(s for _, s in bayes_history)

print(f"{'Method':<20} {'Best Score':>12} {'Evaluations':>12}")
print("-" * 50)
print(f"{'Grid Search':<20} {grid_score:>12.4f} {len(grid_history):>12}")
print(f"{'Random Search':<20} {rand_score:>12.4f} {len(rand_history):>12}")
print(f"{'Bayesian Opt':<20} {bayes_score:>12.4f} {len(bayes_history):>12}")

```

With the same budget, Bayesian optimization usually finds the best score fastest because it does not waste evaluations in clearly bad regions. Random search covers more ground than grid search. Grid search only wins when you have very few hyperparameters and can afford to be exhaustive.

Use It

Optuna in Practice

Optuna is the recommended library for serious hyperparameter tuning. It supports pruning, distributed search, and visualization out of the box.

```
import optuna

def objective(trial):
    lr = trial.suggest_float("learning_rate", 1e-4, 1e-1, log=True)
    n_est = trial.suggest_int("n_estimators", 50, 500)
    max_depth = trial.suggest_int("max_depth", 2, 10)

    model = GradientBoostingRegressor(
        learning_rate=lr,
        n_estimators=n_est,
        max_depth=max_depth,
    )
    model.fit(X_train, y_train)
    return mean_squared_error(y_val, model.predict(X_val))

study = optuna.create_study(direction="minimize")
study.optimize(objective, n_trials=100)

print(f"Best params: {study.best_params}")
print(f"Best MSE: {study.best_value:.4f}")
```

Key Optuna features:

- `suggest_float(..., log=True)` for parameters best searched on log scale (learning rate, regularization)
- `suggest_int` for integer parameters
- `suggest_categorical` for discrete choices
- Built-in MedianPruner for early stopping of bad trials
- `study.trials_dataframe()` for analysis

Optuna with Pruning

Pruning stops unpromising trials early, saving massive compute. Here is the pattern:

```
import optuna
from sklearn.model_selection import cross_val_score

def objective(trial):
    params = {
        "learning_rate": trial.suggest_float("lr", 1e-4, 0.5, log=True),
        "max_depth": trial.suggest_int("max_depth", 2, 10),
        "n_estimators": trial.suggest_int("n_estimators", 50, 500),
        "subsample": trial.suggest_float("subsample", 0.5, 1.0),
    }

    model = GradientBoostingRegressor(**params)
    scores = cross_val_score(model, X_train, y_train, cv=3,
                             scoring="neg_mean_squared_error")
    mean_score = -scores.mean()

    trial.report(mean_score, step=0)
    if trial.should_prune():
```

```

        raise optuna.TrialPruned()

    return mean_score

pruner = optuna.pruners.MedianPruner(n_startup_trials=10, n_warmup_steps=5)
study = optuna.create_study(direction="minimize", pruner=pruner)
study.optimize(objective, n_trials=200)

```

The MedianPruner stops a trial if its intermediate value is worse than the median of all completed trials at the same step. Pruning requires calling `trial.report()` to report intermediate metrics and `trial.should_prune()` to check whether the trial should be stopped. The `n_startup_trials=10` ensures at least 10 trials complete fully before pruning kicks in. This typically saves 40-60% of total compute.

sklearn's Built-in Tuners

For quick experiments, sklearn provides GridSearchCV, RandomizedSearchCV, and HalvingRandomSearchCV:

```

from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import loguniform, randint

param_dist = {
    "learning_rate": loguniform(1e-4, 0.5),
    "max_depth": randint(2, 10),
    "n_estimators": randint(50, 500),
}

search = RandomizedSearchCV(
    GradientBoostingRegressor(),
    param_dist,
    n_iter=100,
    cv=5,
    scoring="neg_mean_squared_error",
    random_state=42,
    n_jobs=-1,
)
search.fit(X_train, y_train)
print(f"Best params: {search.best_params_}")
print(f"Best CV MSE: {-search.best_score_:.4f}")

```

Use `loguniform` from `scipy` for learning rate and regularization. Use `randint` for integer hyperparameters. The `n_jobs=-1` flag parallelizes across all CPU cores.

Common Mistakes in Hyperparameter Tuning

Data leakage through preprocessing. If you fit a scaler on the full dataset before cross-validation, information from the validation fold leaks into training. Always put preprocessing inside a Pipeline so it is fit only on the training fold.

Overfitting to the validation set. Running thousands of trials effectively trains on the validation set. Use nested cross-validation for final performance estimates, or hold out a separate test set that you never touch during tuning.

Searching too narrow a range. If your best value is at the boundary of your search space, you have not searched widely enough. The optimal value might be outside your range. Always

check if the best parameters are at the edges.

Ignoring interaction effects. Learning rate and number of estimators interact strongly in boosting. A low learning rate needs more estimators. Tuning them independently gives worse results than tuning them together.

Not using early stopping for iterative models. For gradient boosting and neural networks, set `n_estimators` or `epochs` to a high value and use early stopping. This is strictly better than tuning the number of iterations as a hyperparameter.

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/12-hyperparameter-tuning/code>

1. Run grid search and random search with the same total budget (e.g., 50 evaluations). Compare the best scores found. Run the experiment 10 times with different seeds. How often does random search win?
2. Implement Hyperband from scratch. Start with 81 configurations, each trained for 1 epoch. Keep the top 1/3 at each round and triple their budget. Compare total compute (sum of all epochs across all configs) to running 81 configs for the full budget.
3. Add a learning rate scheduler (cosine annealing) to the gradient boosting implementation from Lesson 11. Does it help compared to a fixed learning rate?
4. Use Optuna to tune a RandomForestClassifier on a real dataset (e.g., sklearn's breast cancer dataset). Use `optuna.visualization.plot_param_importances(study)` to see which hyperparameters matter most. Does it match the importance ranking from this lesson?
5. Implement a simple acquisition function (Expected Improvement) and demonstrate exploration vs exploitation. Plot the surrogate model's mean and uncertainty, and show where EI chooses to evaluate next.

Key Terms

Term	What people say	What it actually means
Hyperparameter	"A setting you choose"	A value set before training that controls the learning process, not learned from data
Grid search	"Try every combination"	Exhaustive search over a specified parameter grid. Exponential cost.
Random search	"Just sample randomly"	Sample hyperparameters from distributions. Covers important dimensions better than grid search.
Bayesian optimization	"Smart search"	Uses a surrogate model of the objective to decide where to evaluate next, balancing exploration and exploitation
Surrogate model	"A cheap approximation"	A model (usually Gaussian process) that approximates the expensive objective function from observed evaluations
Acquisition function	"Where to look next"	Scores candidate points by balancing expected improvement with uncertainty. EI and UCB are common choices.

Term	What people say	What it actually means
Early stopping	“Stop wasting time”	Terminate training early when validation performance stops improving
Hyperband	“Tournament bracket for configs”	Adaptive resource allocation: start many configs with small budgets, keep the best and increase their budgets
Learning rate scheduler	“Change lr during training”	A function that adjusts the learning rate over the course of training for better convergence

Further Reading

- Bergstra & Bengio: Random Search for Hyper-Parameter Optimization (2012) – the paper that showed random beats grid
- Snoek et al., Practical Bayesian Optimization of Machine Learning Algorithms (2012) – Bayesian optimization for ML
- Li et al., Hyperband: A Novel Bandit-Based Approach (2018) – the Hyperband paper
- Optuna: A Next-generation Hyperparameter Optimization Framework – the Optuna paper
- Probst et al., Tunability: Importance of Hyperparameters (2019) – which hyperparameters matter

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/12-hyperparameter-tuning>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/12-hyperparameter-tuning/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/12-hyperparameter-tuning>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

ML Pipelines

A model is not a product. A pipeline is. The pipeline is everything from raw data to deployed prediction, and every step must be reproducible.

Type: Build **Language:** Python **Prerequisites:** Phase 2, Lesson 12 (Hyperparameter Tuning)

Time: ~120 minutes

Learning Objectives

- Build an ML pipeline from scratch that chains imputation, scaling, encoding, and model training into a single reproducible object
- Identify data leakage scenarios and explain how pipelines prevent them by fitting transformers only on training data
- Construct a ColumnTransformer that applies different preprocessing to numeric and categorical features
- Implement pipeline serialization and demonstrate that the same fitted pipeline produces identical results in training and production

The Problem

You have a notebook that loads data, fills missing values with the median, scales features, trains a model, and prints accuracy. It works. You ship it.

A month later, someone retrains the model and gets different results. The median was computed on the full dataset including test data (data leakage). The scaling parameters were not saved, so inference uses different statistics. The feature engineering code was copy-pasted between training and serving, and the copies diverged. A categorical column gained a new value in production that the encoder has never seen.

These are not hypothetical. They are the most common reasons ML systems fail in production. Pipelines solve all of them by packaging every transformation step into a single, ordered, reproducible object.

The Concept

What a Pipeline Is

A pipeline is an ordered sequence of data transformations followed by a model. Each step takes the output of the previous step as input. The entire pipeline is fitted once on training data. At inference time, the same fitted pipeline transforms new data and produces predictions.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/13-ml-pipelines>

The pipeline guarantees: - Transformations are fitted only on training data (no leakage) - The same transformations are applied at inference time - The entire object can be serialized and deployed as one artifact - Cross-validation applies the pipeline per fold, preventing subtle leakage

Data Leakage: The Silent Killer

Data leakage happens when information from the test set or future data contaminates training. Pipelines prevent the most common forms.

Leaky (wrong):

```
X = df.drop("target", axis=1)
y = df["target"]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test = X_scaled[:800], X_scaled[800:]
y_train, y_test = y[:800], y[800:]
```

The scaler saw test data. The mean and standard deviation include test samples. This inflates accuracy estimates.

Correct:

```
X_train, X_test = X[:800], X[800:]

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

With a pipeline, you do not need to think about this. The pipeline handles it automatically.

sklearn Pipeline

sklearn's Pipeline chains transformers and an estimator. It exposes `.fit()`, `.predict()`, and `.score()` that apply all steps in order.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
```

```
pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression()),
])
```

```
pipe.fit(X_train, y_train)
predictions = pipe.predict(X_test)
```

When you call `pipe.fit(X_train, y_train)`: 1. Scaler calls `fit_transform` on `X_train` 2. Model calls `fit` on the scaled `X_train`

When you call `pipe.predict(X_test)`: 1. Scaler calls `transform` (not `fit_transform`) on `X_test` 2. Model calls `predict` on the scaled `X_test`

The scaler never sees test data during fitting. This is the whole point.

ColumnTransformer: Different Pipelines for Different Columns

Real datasets have numeric and categorical columns that need different preprocessing. ColumnTransformer handles this.

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer

numeric_pipe = Pipeline([
    ("impute", SimpleImputer(strategy="median")),
    ("scale", StandardScaler()),
])

categorical_pipe = Pipeline([
    ("impute", SimpleImputer(strategy="most_frequent")),
    ("encode", OneHotEncoder(handle_unknown="ignore")),
])

preprocessor = ColumnTransformer([
    ("num", numeric_pipe, ["age", "income", "score"]),
    ("cat", categorical_pipe, ["city", "gender", "plan"]),
])

full_pipeline = Pipeline([
    ("preprocess", preprocessor),
    ("model", GradientBoostingClassifier()),
])
```

The `handle_unknown="ignore"` in `OneHotEncoder` is critical for production. When a new category appears (a city the model has never seen), it produces a zero vector instead of crashing.

Experiment Tracking

A pipeline makes training reproducible, but you also need to track what happened across experiments: which hyperparameters were used, which dataset version, what the metrics were, which code was running.

MLflow is the most common open-source solution:

```
import mlflow

with mlflow.start_run():
    mlflow.log_param("max_depth", 5)
    mlflow.log_param("n_estimators", 100)
    mlflow.log_param("learning_rate", 0.1)

    pipe.fit(X_train, y_train)
    accuracy = pipe.score(X_test, y_test)

    mlflow.log_metric("accuracy", accuracy)
    mlflow.sklearn.log_model(pipe, "model")
```

Every run is recorded with parameters, metrics, artifacts, and the full model. You can compare runs, reproduce any experiment, and deploy any model version.

Weights & Biases (wandb) provides the same functionality with a hosted dashboard:

```
import wandb

wandb.init(project="my-pipeline")
wandb.config.update({"max_depth": 5, "n_estimators": 100})

pipe.fit(X_train, y_train)
accuracy = pipe.score(X_test, y_test)

wandb.log({"accuracy": accuracy})
```

Model Versioning

After experiment tracking, you need to manage model versions. Which model is in production? Which is staging? Which was last week's?

MLflow's Model Registry provides: - **Version tracking:** Every saved model gets a version number - **Stage transitions:** "Staging", "Production", "Archived" - **Approval workflow:** Models must be explicitly promoted to production - **Rollback:** Switch back to a previous version instantly

Data Versioning with DVC

Code is versioned with git. Data should be versioned too, but git cannot handle large files. DVC (Data Version Control) solves this.

```
dvc init
dvc add data/training.csv
git add data/training.csv.dvc data/.gitignore
git commit -m "Track training data"
dvc push
```

DVC stores the actual data in remote storage (S3, GCS, Azure) and keeps a small .dvc file in git that records the hash. When you checkout a git commit, dvc checkout restores the exact data that was used.

This means every git commit pins both the code and the data. Full reproducibility.

Reproducible Experiments

A reproducible experiment requires four things:

1. **Fixed random seeds:** Set seeds for numpy, random, and the framework (torch, sklearn)
2. **Pinned dependencies:** requirements.txt or poetry.lock with exact versions
3. **Versioned data:** DVC or similar
4. **Config files:** All hyperparameters in a config, not hardcoded

```
import numpy as np
import random

def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    try:
        import torch
        torch.manual_seed(seed)
        torch.cuda.manual_seed_all(seed)
```

```
torch.backends.cudnn.deterministic = True
except ImportError:
    pass
```

From Notebook to Production Pipeline

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/13-ml-pipelines>

The typical progression:

1. **Notebook exploration:** Quick experiments, visualizations, feature ideas
2. **Extract functions:** Move preprocessing, feature engineering, evaluation into modules
3. **Build Pipeline:** Chain transformations into a sklearn Pipeline or custom class
4. **Config management:** Move all hyperparameters into a YAML/JSON config
5. **Experiment tracking:** Add MLflow or wandb logging
6. **Data validation:** Check schema, distributions, and missing value patterns before training
7. **Tests:** Unit tests for transformers, integration tests for the full pipeline
8. **Deployment:** Serialize the pipeline, wrap in an API (FastAPI, Flask), containerize

Common Pipeline Mistakes

Mistake	Why it is bad	Fix
Fitting on full data before splitting	Data leakage	Use Pipeline with <code>cross_val_score</code>
Feature engineering outside pipeline	Different transforms at train vs serve	Put all transforms in the Pipeline
Not handling unknown categories	Production crash on new values	OneHotEncoder(handle_unknown='ignore')
Hardcoded column names	Breaks when schema changes	Use column name lists from config
No data validation	Silently wrong predictions on bad data	Add schema checks before prediction
Training/serving skew	Model sees different features in prod	One Pipeline object for both

Interactive figure: f3-pipeline-flow. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/13-ml-pipelines>

Build It

The code in `code/pipeline.py` builds a complete ML pipeline from scratch:

Step 1: Custom Transformer

```
class CustomTransformer:
    def __init__(self):
        self.means = None
        self.stds = None

    def fit(self, X):
        self.means = np.mean(X, axis=0)
        self.stds = np.std(X, axis=0)
        self.stds[self.stds == 0] = 1.0
        return self

    def transform(self, X):
        return (X - self.means) / self.stds

    def fit_transform(self, X):
        return self.fit(X).transform(X)
```

Step 2: Pipeline from Scratch

```
class PipelineFromScratch:
    def __init__(self, steps):
        self.steps = steps

    def fit(self, X, y=None):
        X_current = X.copy()
        for name, step in self.steps[::-1]:
            X_current = step.fit_transform(X_current)
        name, model = self.steps[-1]
        model.fit(X_current, y)
        return self

    def predict(self, X):
        X_current = X.copy()
        for name, step in self.steps[::-1]:
            X_current = step.transform(X_current)
        name, model = self.steps[-1]
        return model.predict(X_current)
```

Step 3: Cross-Validation with Pipeline

The code demonstrates how cross-validation with a pipeline prevents data leakage: the scaler is fit separately on each fold's training data.

Step 4: Full Production Pipeline with sklearn

A complete pipeline with ColumnTransformer, multiple preprocessing paths, and a model, trained with proper cross-validation and experiment logging.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/13-ml-pipelines>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/13-ml-pipelines/code>

1. Build a pipeline that handles a dataset with 3 numeric columns and 2 categorical columns. Use `ColumnTransformer` to apply median imputation + scaling to numerics and most-frequent imputation + one-hot encoding to categoricals. Train with 5-fold cross-validation.
2. Deliberately introduce data leakage: fit the scaler on the full dataset before splitting. Compare the cross-validation score (leaky) to the pipeline cross-validation score (clean). How large is the difference?
3. Serialize your pipeline with `joblib.dump`. Load it in a separate script and run predictions. Verify the predictions are identical.
4. Add a custom transformer to the pipeline that creates polynomial features (degree 2) for the two most important numeric columns. Where should it go in the pipeline?
5. Set up MLflow tracking for the pipeline. Run 5 experiments with different hyperparameters. Use the MLflow UI (`mlflow ui`) to compare runs and pick the best model.

Key Terms

Term	What people say	What it actually means
Pipeline	"Chain of transforms + model"	An ordered sequence of fitted transformers and a model, applied as one unit to prevent leakage
Data leakage	"Test info leaked into training"	Using information from outside the training set to build the model, inflating performance estimates
ColumnTransformer	"Different preprocessing per column"	Applies different pipelines to different subsets of columns, combining results
Experiment tracking	"Logging your runs"	Recording parameters, metrics, artifacts, and code versions for every training run
MLflow	"Track and deploy models"	Open-source platform for experiment tracking, model registry, and deployment
DVC	"Git for data"	Version control system for large data files, storing hashes in git and data in remote storage
Model registry	"Model version catalog"	A system that tracks model versions with stage labels (staging, production, archived)
Training/serving skew	"It worked in the notebook"	Differences between how data is processed during training versus inference, causing silent errors
Reproducibility	"Same code, same result"	The ability to get identical results from the same code, data, and configuration

Further Reading

- scikit-learn Pipeline docs – the official pipeline reference

- MLflow documentation - experiment tracking and model registry
- DVC documentation - data versioning
- Sculley et al., Hidden Technical Debt in Machine Learning Systems (2015) - the seminal paper on ML systems complexity
- Google ML Best Practices: Rules of ML - practical production ML advice

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/13-ml-pipelines>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/13-ml-pipelines/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/13-ml-pipelines>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Naive Bayes

The “naive” assumption is wrong, and it works anyway. That’s the beauty of it.

Type: Build **Language:** Python **Prerequisites:** Phase 2, Lessons 01-07 (classification, Bayes’ theorem) **Time:** ~75 minutes

Learning Objectives

- Implement Multinomial Naive Bayes from scratch with Laplace smoothing for text classification
- Explain why the naive independence assumption is mathematically wrong but produces correct class rankings in practice
- Compare Multinomial, Bernoulli, and Gaussian Naive Bayes variants and select the right one for a given feature type
- Evaluate Naive Bayes against logistic regression on high-dimensional sparse data and explain the bias-variance tradeoff at work

The Problem

You need to classify text. Emails into spam or not-spam. Customer reviews into positive or negative. Support tickets into categories. You have thousands of features (one per word) and limited training data.

Most classifiers choke here. Logistic regression needs enough samples to estimate thousands of weights reliably. Decision trees split on one word at a time and overfit wildly. KNN in 10,000 dimensions is meaningless because every point is equally far from every other point.

Naive Bayes handles this. It makes a mathematically wrong assumption (that every feature is independent of every other feature given the class), and it still outperforms “smarter” models on text classification, especially with small training sets. It trains in a single pass through the data. It scales to millions of features. It produces probability estimates (though often poorly calibrated due to the independence assumption).

Understanding why a wrong assumption leads to good predictions teaches you something fundamental about machine learning: the best model is not the most correct one, it is the one with the best bias-variance tradeoff for your data.

The Concept

Bayes’ Theorem (Quick Review)

Bayes’ theorem flips conditional probabilities:

$$P(\text{class} \mid \text{features}) = P(\text{features} \mid \text{class}) * P(\text{class}) / P(\text{features})$$

We want $P(\text{class} \mid \text{features})$ – the probability that a document belongs to a class given the words in it. We can compute this from: - $P(\text{features} \mid \text{class})$ – the likelihood of seeing these

words in documents of this class - $P(\text{class})$ - the prior probability of the class (how common is spam in general?) - $P(\text{features})$ - the evidence, same for all classes, so we can ignore it when comparing

The class with the highest $P(\text{class} \mid \text{features})$ wins.

The Naive Independence Assumption

Computing $P(\text{features} \mid \text{class})$ exactly requires estimating the joint probability of all features together. With a vocabulary of 10,000 words, you would need to estimate a distribution over $2^{10,000}$ possible combinations. Impossible.

The naive assumption: every feature is conditionally independent given the class.

$$P(w_1, w_2, \dots, w_n \mid \text{class}) = P(w_1 \mid \text{class}) * P(w_2 \mid \text{class}) * \dots * P(w_n \mid \text{class})$$

Instead of one impossible joint distribution, you estimate n simple per-feature distributions. Each one needs only a count.

This assumption is obviously wrong. The words “machine” and “learning” are not independent in any document. But the classifier does not need correct probability estimates. It needs correct rankings - which class has the highest probability. The independence assumption introduces systematic errors, but those errors affect all classes similarly, so the ranking stays correct.

Why It Still Works

Three reasons:

1. **Ranking over calibration.** Classification only needs the top-ranked class to be correct. Even if $P(\text{spam}) = 0.99999$ when the true probability is 0.7, the classifier still picks spam correctly. We do not need correct probabilities. We need the correct winner.
2. **High bias, low variance.** The independence assumption is a strong prior. It constrains the model heavily, which prevents overfitting. With limited training data, a model that is slightly wrong but stable beats a model that is theoretically right but wildly unstable. This is the bias-variance tradeoff in action.
3. **Feature redundancy cancels out.** Correlated features provide redundant evidence. The classifier double-counts this evidence, but it double-counts it for the correct class too. If “machine” and “learning” always appear together, both provide evidence for the “tech” class. NB counts them twice, but it counts them twice for the right class.

A fourth, practical reason: Naive Bayes is extremely fast. Training is a single pass through the data counting frequencies. Prediction is a matrix multiplication. You can train on a million documents in seconds. This speed means you can iterate faster, try more feature sets, and run more experiments than with slower models.

The Math Step by Step

Let us trace through a concrete example. Suppose we have two classes: spam and not-spam. Our vocabulary has three words: “free”, “money”, “meeting”.

Training data: - Spam emails mention “free” 80 times, “money” 60 times, “meeting” 10 times (150 total words) - Not-spam emails mention “free” 5 times, “money” 10 times, “meeting” 100 times (115 total words) - 40% of emails are spam, 60% are not-spam

With Laplace smoothing ($\alpha=1$):

$$\begin{aligned} P(\text{free} \mid \text{spam}) &= (80 + 1) / (150 + 3) = 81/153 = 0.529 \\ P(\text{money} \mid \text{spam}) &= (60 + 1) / (150 + 3) = 61/153 = 0.399 \\ P(\text{meeting} \mid \text{spam}) &= (10 + 1) / (150 + 3) = 11/153 = 0.072 \end{aligned}$$

$$\begin{aligned} P(\text{free} \mid \text{not-spam}) &= (5 + 1) / (115 + 3) = 6/118 = 0.051 \\ P(\text{money} \mid \text{not-spam}) &= (10 + 1) / (115 + 3) = 11/118 = 0.093 \\ P(\text{meeting} \mid \text{not-spam}) &= (100 + 1) / (115 + 3) = 101/118 = 0.856 \end{aligned}$$

New email contains: “free” (2 times), “money” (1 time), “meeting” (0 times).

$$\begin{aligned} \log P(\text{spam} \mid \text{email}) &= \log(0.4) + 2 \cdot \log(0.529) + 1 \cdot \log(0.399) + 0 \cdot \log(0.072) \\ &= -0.916 + 2 \cdot (-0.637) + (-0.919) + 0 \\ &= -3.109 \end{aligned}$$

$$\begin{aligned} \log P(\text{not-spam} \mid \text{email}) &= \log(0.6) + 2 \cdot \log(0.051) + 1 \cdot \log(0.093) + 0 \cdot \log(0.856) \\ &= -0.511 + 2 \cdot (-2.976) + (-2.375) + 0 \\ &= -8.838 \end{aligned}$$

Spam wins by a large margin. The word “free” appearing twice is strong evidence for spam. Note that “meeting” not appearing contributes zero to both log sums ($0 \cdot \log(P)$) – in Multinomial NB, absent words have no effect. It is Bernoulli NB that explicitly models word absence.

Three Variants

Naive Bayes comes in three flavors. Each models $P(\text{feature} \mid \text{class})$ differently.

Multinomial Naive Bayes

Models each feature as a count. Best for text data where features are word frequencies or TF-IDF values.

$$P(\text{word}_i \mid \text{class}) = (\text{count of word}_i \text{ in class} + \alpha) / (\text{total words in class} + \alpha * \text{vocab_size})$$

The alpha is Laplace smoothing (explained below). This variant is the workhorse for text classification.

Gaussian Naive Bayes

Models each feature as a normal distribution. Best for continuous features.

$$P(x_i \mid \text{class}) = (1 / \sqrt{2 * \pi * \text{var}}) * \exp(-(x_i - \text{mean})^2 / (2 * \text{var}))$$

Each class gets its own mean and variance per feature. This works well when features genuinely follow a bell curve within each class.

Bernoulli Naive Bayes

Models each feature as binary (present or absent). Best for short text or binary feature vectors.

$$P(\text{word}_i \mid \text{class}) = (\text{docs in class containing word}_i + \alpha) / (\text{total docs in class} + 2 * \alpha)$$

Unlike Multinomial, Bernoulli explicitly penalizes the absence of a word. If “free” typically appears in spam but is absent from this email, Bernoulli counts that as evidence against spam.

When to Use Each Variant

Variant	Feature Type	Best For	Example
Multinomial	Counts or frequencies	Text classification, bag-of-words	Email spam, topic classification
Gaussian	Continuous values	Tabular data with normal-ish features	Iris classification, sensor data
Bernoulli	Binary (0/1)	Short text, binary feature vectors	SMS spam, presence/absence features

Laplace Smoothing

What happens when a word appears in the test data but never appeared in the training data for a particular class?

Without smoothing: $P(\text{word} \mid \text{class}) = 0/N = 0$. One zero multiplied through the entire product makes $P(\text{class} \mid \text{features}) = 0$, regardless of all other evidence. A single unseen word destroys the entire prediction, no matter how much other evidence supports it.

Laplace smoothing adds a small count alpha (usually 1) to every feature count:

$$P(\text{word}_i \mid \text{class}) = (\text{count}(\text{word}_i, \text{class}) + \alpha) / (\text{total_words_in_class} + \alpha * \text{vocab_size})$$

With $\alpha=1$, every word gets at least a tiny probability. The word “discombobulate” appearing in a test email no longer kills the spam probability. The smoothing has a Bayesian interpretation: it is equivalent to placing a uniform Dirichlet prior on the word distributions.

Higher alpha means stronger smoothing (more uniform distributions). Lower alpha means the model trusts the data more. Alpha is a hyperparameter you tune.

The effect of alpha:

Alpha	Effect	When to use
0.001	Almost no smoothing, trust the data	Very large training set, no unseen features expected
0.1	Light smoothing	Large training set
1.0	Standard Laplace smoothing	Default starting point
10.0	Heavy smoothing, flattens distributions	Very small training set, many unseen features expected

Log-Space Computation

Multiplying hundreds of probabilities (each less than 1) causes floating-point underflow. The product becomes zero in floating point even though the true value is a very small positive number.

The solution: work in log space. Instead of multiplying probabilities, add their logarithms:

$$\log P(\text{class} \mid x_1, x_2, \dots, x_n) = \log P(\text{class}) + \sum_i \log P(x_i \mid \text{class})$$

This turns the prediction into a dot product:

```
log_scores = X @ log_feature_probs.T + log_class_priors
prediction = argmax(log_scores)
```

Matrix multiplication. That is why Naive Bayes prediction is so fast – it is the same operation as a single-layer linear model.

Naive Bayes vs Logistic Regression

Both are linear classifiers for text. The difference is in what they model.

Aspect	Naive Bayes	Logistic Regression
Type	Generative (models $P(X Y)$)	Discriminative (models $P(Y X)$)
Training	Count frequencies	Optimize loss function
Small data	Better (strong prior helps)	Worse (not enough to estimate weights)
Large data	Worse (wrong assumption hurts)	Better (flexible boundary)
Features	Assumes independence	Handles correlations
Speed	Single pass, very fast	Iterative optimization
Calibration	Poor probabilities	Better probabilities

Rule of thumb: start with Naive Bayes. If you have enough data and NB plateaus, switch to logistic regression.

Classification Pipeline

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/14-naive-bayes>

In practice, we work in log space to avoid floating-point underflow. Instead of multiplying many small probabilities, we add their logarithms:

$$\log P(\text{class} \mid \text{features}) = \log P(\text{class}) + \sum_i \log P(\text{feature}_i \mid \text{class})$$

Interactive figure: naive-bayes. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/14-naive-bayes>

Build It

The code in `code/naive_bayes.py` implements both `MultinomialNB` and `GaussianNB` from scratch.

MultinomialNB

The from-scratch implementation:

1. **fit(X, y):** For each class, count the frequency of each feature. Add Laplace smoothing. Compute log probabilities. Store class priors (log of class frequencies).
2. **predict_log_proba(X):** For each sample, compute $\log P(\text{class}) + \sum \log P(\text{feature}_i \mid \text{class})$ for all classes. This is a matrix multiplication: $X @ \log_probs.T + \log_priors$.
3. **predict(X):** Return the class with highest log probability.

```
class MultinomialNB:
    def __init__(self, alpha=1.0):
        self.alpha = alpha

    def fit(self, X, y):
        classes = np.unique(y)
        n_classes = len(classes)
```

```

n_features = X.shape[1]

self.classes_ = classes
self.class_log_prior_ = np.zeros(n_classes)
self.feature_log_prob_ = np.zeros((n_classes, n_features))

for i, c in enumerate(classes):
    X_c = X[y == c]
    self.class_log_prior_[i] = np.log(X_c.shape[0] / X.shape[0])
    counts = X_c.sum(axis=0) + self.alpha
    self.feature_log_prob_[i] = np.log(counts / counts.sum())

return self

```

The key insight: after fitting, prediction is just matrix multiplication plus a bias. This is why Naive Bayes is so fast.

GaussianNB

For continuous features, we estimate mean and variance per class per feature:

```

class GaussianNB:
    def __init__(self):
        pass

    def fit(self, X, y):
        classes = np.unique(y)
        self.classes_ = classes
        self.means_ = np.zeros((len(classes), X.shape[1]))
        self.vars_ = np.zeros((len(classes), X.shape[1]))
        self.priors_ = np.zeros(len(classes))

        for i, c in enumerate(classes):
            X_c = X[y == c]
            self.means_[i] = X_c.mean(axis=0)
            self.vars_[i] = X_c.var(axis=0) + 1e-9
            self.priors_[i] = X_c.shape[0] / X.shape[0]

        return self

```

Prediction uses the Gaussian PDF per feature, multiplied across features (added in log space).

Demo: Text Classification

The code generates synthetic bag-of-words data simulating two classes (tech articles vs sports articles). Each class has a different word frequency distribution. MultinomialNB classifies them using word counts.

The synthetic data works like this: we create 200 “words” (feature columns). Words 0-39 have high frequency in tech articles and low in sports. Words 80-119 have high frequency in sports and low in tech. Words 40-79 are medium frequency in both. This creates a realistic scenario where some words are strong class indicators and others are noise.

Demo: Continuous Features

The code generates Iris-like data (3 classes, 4 features, Gaussian clusters). GaussianNB classifies using per-class mean and variance. Each class has a different center (mean vector) and different spread (variance), mimicking real-world data where measurements differ systematically between categories.

The code also demonstrates: - **Smoothing comparison:** Training MultinomialNB with different alpha values to show the effect of smoothing strength on accuracy. - **Training size experiment:** How NB accuracy improves as training data grows from 20 to 1600 samples. NB reaches decent accuracy even with very few samples – this is its main advantage. - **Confusion matrix:** Per-class precision, recall, and F1 score to show where NB makes mistakes.

Prediction Speed

Naive Bayes prediction is a matrix multiplication. For n samples with d features and k classes:
 - MultinomialNB: one matrix multiply $(n \times d) @ (d \times k) = O(n * d * k)$ - GaussianNB: $n * k$ Gaussian PDF evaluations, each over d features = $O(n * d * k)$

Both are linear in every dimension. Compare this to KNN (which requires distance computation to all training points) or SVM with RBF kernel (which requires kernel evaluation against all support vectors). NB is faster by orders of magnitude at prediction time.

Use It

With sklearn, both variants are one-liners:

```
from sklearn.naive_bayes import GaussianNB, MultinomialNB

gnb = GaussianNB()
gnb.fit(X_train, y_train)
print(f"GaussianNB accuracy: {gnb.score(X_test, y_test):.3f}")

mnb = MultinomialNB(alpha=1.0)
mnb.fit(X_train_counts, y_train)
print(f"MultinomialNB accuracy: {mnb.score(X_test_counts, y_test):.3f}")
```

For text classification with sklearn:

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import Pipeline

text_clf = Pipeline([
    ("vectorizer", CountVectorizer()),
    ("classifier", MultinomialNB(alpha=1.0)),
])

text_clf.fit(train_texts, train_labels)
accuracy = text_clf.score(test_texts, test_labels)
```

The code in naive_bayes.py compares from-scratch implementations against sklearn on the same data to verify correctness.

TF-IDF with Naive Bayes

Raw word counts give every word equal weight per occurrence. But common words like “the” and “is” appear frequently in every class – they carry no information. TF-IDF (Term Frequency - Inverse Document Frequency) downweights common words and upweights rare, discriminative words.

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import Pipeline

text_clf = Pipeline([
    ("tfidf", TfidfVectorizer()),
    ("classifier", MultinomialNB(alpha=0.1)),
])
```

TF-IDF values are non-negative, so they work with MultinomialNB. The combination of TF-IDF + MultinomialNB is one of the strongest baselines for text classification. It frequently beats more complex models on datasets with fewer than 10,000 training samples.

BernoulliNB for Short Text

For short text (tweets, SMS, chat messages), BernoulliNB can outperform MultinomialNB. Short texts have low word counts, so the frequency information that MultinomialNB relies on is noisy. BernoulliNB only cares about presence or absence, which is more reliable with short text.

```
from sklearn.naive_bayes import BernoulliNB
from sklearn.feature_extraction.text import CountVectorizer

text_clf = Pipeline([
    ("vectorizer", CountVectorizer(binary=True)),
    ("classifier", BernoulliNB(alpha=1.0)),
])
```

The binary=True flag in CountVectorizer converts all counts to 0/1. Without it, BernoulliNB still works but is seeing counts that it was not designed for.

Calibrating NB Probabilities

NB probabilities are poorly calibrated. When NB says $P(\text{spam}) = 0.95$, the true probability might be 0.7. If you need reliable probability estimates (for example, to set a threshold or to combine with other models), use sklearn’s CalibratedClassifierCV:

```
from sklearn.calibration import CalibratedClassifierCV

calibrated_nb = CalibratedClassifierCV(MultinomialNB(), cv=5, method="sigmoid")
calibrated_nb.fit(X_train, y_train)
proba = calibrated_nb.predict_proba(X_test)
```

This fits a logistic regression on top of NB’s raw scores using cross-validation. The resulting probabilities are much closer to the true class frequencies.

Common Gotchas

1. **Negative feature values.** MultinomialNB requires non-negative features. If you have negative values (like TF-IDF with certain settings or standardized features), use Gaus-

sianNB instead, or shift the features to be positive.

2. **Zero variance features.** GaussianNB divides by variance. If a feature has zero variance for a class (all values identical), the probability computation breaks. The code adds a small smoothing term ($1e-9$) to all variances to prevent this.
3. **Class imbalance.** If 99% of emails are not-spam, the prior $P(\text{not-spam}) = 0.99$ is so strong that it overwhelms the likelihood evidence. You can set class priors manually or use `class_prior` parameter in `sklearn`.
4. **Feature scaling.** MultinomialNB does not need scaling (it works on counts). GaussianNB does not need scaling either (it estimates per-feature statistics). This is an advantage over logistic regression and SVM, which are sensitive to feature scales.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/14-naive-bayes>

Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/14-naive-bayes/code>

1. **Smoothing experiment.** Train MultinomialNB on text data with alpha values of 0.01, 0.1, 1.0, 10.0, and 100.0. Plot accuracy vs alpha. Where does performance peak? Why does very high alpha hurt?
2. **Feature independence test.** Take a real text dataset. Pick two words that are obviously correlated (“machine” and “learning”). Compute $P(\text{word1} \mid \text{class}) * P(\text{word2} \mid \text{class})$ and compare to $P(\text{word1 AND word2} \mid \text{class})$. How wrong is the independence assumption? Does it affect classification accuracy?
3. **Bernoulli implementation.** Extend the code with a BernoulliNB class. Convert bag-of-words to binary (present/absent) and compare accuracy against MultinomialNB on text data. When does Bernoulli win?
4. **NB vs Logistic Regression.** Train both on text data. Start with 100 training samples and increase to 10,000. Plot accuracy vs training set size for both. At what point does Logistic Regression overtake Naive Bayes?
5. **Spam filter.** Build a complete spam classifier: tokenize raw email text, build vocabulary, create bag-of-words features, train MultinomialNB, evaluate with precision and recall (not just accuracy – why?).

Key Terms

Term	What people say	What it actually means
Naive Bayes	“Simple probabilistic classifier”	A classifier that applies Bayes’ theorem with the assumption that features are conditionally independent given the class
Conditional independence	“Features don’t affect each other”	$P(A, B \mid C) = P(A \mid C) * P(B \mid C)$ – knowing B tells you nothing new about A once you know C

Term	What people say	What it actually means
Laplace smoothing	"Add-one smoothing"	Adding a small count to every feature to prevent zero probabilities from dominating the prediction
Prior	"What you believed before seeing data"	$P(\text{class})$ – the probability of each class before observing any features
Likelihood	"How well the data fits"	$P(\text{features} \mid \text{class})$ – the probability of observing these features if the class is known
Posterior	"What you believe after seeing data"	$P(\text{class} \mid \text{features})$ – the updated probability of the class after observing the features
Generative model	"Models how data is generated"	A model that learns $P(X \mid Y)$ and $P(Y)$, then uses Bayes' theorem to get $P(Y \mid X)$
Discriminative model	"Models the decision boundary"	A model that directly learns $P(Y \mid X)$ without modeling how X is generated
Log probability	"Avoid underflow"	Working with $\log P$ instead of P to prevent the product of many small numbers from becoming zero in floating point

Further Reading

- scikit-learn Naive Bayes docs – all three variants with mathematical details
- McCallum and Nigam, A Comparison of Event Models for Naive Bayes Text Classification (1998) – the classic comparison of Multinomial vs Bernoulli for text
- Rennie et al., Tackling the Poor Assumptions of Naive Bayes Text Classifiers (2003) – improvements to NB for text
- Ng and Jordan, On Discriminative vs. Generative Classifiers (2001) – proves NB converges faster than LR with less data

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/14-naive-bayes>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/14-naive-bayes/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/14-naive-bayes>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Time Series Fundamentals

Past performance does predict future results – if you check for stationarity first.

Type: Build **Language:** Python **Prerequisites:** Phase 2, Lessons 01-09 **Time:** ~90 minutes

Learning Objectives

- Decompose a time series into trend, seasonality, and residual components and test for stationarity
- Implement lag features and rolling statistics to convert a time series into a supervised learning problem
- Build a walk-forward validation framework that prevents future data from leaking into training
- Explain why random train/test splits are invalid for time series and demonstrate the performance gap versus proper temporal splits

The Problem

You have data ordered by time. Daily sales, hourly temperature, per-minute CPU usage, weekly stock prices. You want to predict the next value, the next week, the next quarter.

You reach for your standard ML toolkit: random train/test split, cross-validation, feature matrix in, prediction out. Every step is wrong.

Time series breaks the assumptions that standard ML relies on. Samples are not independent – today's temperature depends on yesterday's. Random splits leak future information into the past. Features that look great in backtest fail in production because they rely on patterns that shift over time.

A model that gets 95% accuracy with random cross-validation might get 55% with proper time-based evaluation. The difference is not a technicality. It is the difference between a model that works on paper and one that works in production.

This lesson covers the fundamentals: what makes time data different, how to evaluate models honestly, and how to turn a time series into features that standard ML models can consume.

The Concept

What Makes Time Series Different

Standard ML assumes i.i.d. – independent and identically distributed. Each sample is drawn from the same distribution, independently of other samples. Time series violates both:

- **Not independent.** Today's stock price depends on yesterday's. This week's sales correlate with last week's.
- **Not identically distributed.** The distribution shifts over time. Sales in December look different from sales in March.

These violations are not minor. They change how you build features, how you evaluate models, and which algorithms work.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/15-time-series>

In standard ML, samples are interchangeable. Shuffling them changes nothing. In time series, order is everything. Shuffling destroys the signal.

Components of a Time Series

Every time series is a combination of:

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/15-time-series>

- **Trend:** The long-term direction. Revenue growing 10% per year. Global temperature rising.
- **Seasonality:** Repeating patterns at fixed intervals. Retail sales spike in December. Air conditioning usage peaks in July.
- **Residual:** Whatever is left after removing trend and seasonality. If the residual looks like white noise, the decomposition captured the signal.

Stationarity

A time series is stationary if its statistical properties (mean, variance, autocorrelation) do not change over time. Most forecasting methods assume stationarity.

Why it matters: A non-stationary series has a mean that drifts. A model trained on data from January has learned a different mean than what February will show. It will be systematically wrong.

How to check: Compute rolling mean and rolling standard deviation over windows. If they drift, the series is non-stationary.

How to fix: Differencing. Instead of modeling the raw values, model the change between consecutive values:

```
diff[t] = value[t] - value[t-1]
```

If one round of differencing does not make the series stationary, apply it again (second-order differencing). Most real-world series need at most two rounds.

Example:

Original series: [100, 102, 106, 112, 120] First difference: [2, 4, 6, 8] (still trending upward)
Second difference: [2, 2, 2] (constant – stationary)

The original series had a quadratic trend. First differencing turned it into a linear trend. Second differencing made it flat. In practice, you rarely need more than two rounds.

Formal test: The Augmented Dickey-Fuller (ADF) test is the standard statistical test for stationarity. The null hypothesis is “the series is non-stationary.” A p-value below 0.05 means you can reject the null and conclude stationarity. We do not implement ADF from scratch (it requires asymptotic distribution tables), but the rolling statistics approach in our code gives a practical visual check.

Autocorrelation

Autocorrelation measures how much a value at time t correlates with the value at time $t-k$ (k steps in the past). The autocorrelation function (ACF) plots this correlation for each lag k .

ACF tells you: - How far back the series remembers. If ACF drops to zero after lag 5, values more than 5 steps ago are irrelevant. - Whether seasonality exists. If ACF spikes at lag 12 (monthly data), there is yearly seasonality. - How many lag features to create. Use lags up to where ACF becomes negligible.

PACF (Partial Autocorrelation Function) removes indirect correlations. If today correlates with 3 days ago only because both correlate with yesterday, PACF at lag 3 will be zero while ACF at lag 3 will not.

Lag Features: Turning Time Series into Supervised Learning

Standard ML models need a feature matrix X and a target y . Time series gives you a single column of values. The bridge is lag features.

Take the series [10, 12, 14, 13, 15] and create lag-1 and lag-2 features:

lag_2	lag_1	target
10	12	14
12	14	13
14	13	15

Now you have a standard regression problem. Any ML model (linear regression, random forest, gradient boosting) can predict the target from the lags.

Additional features you can engineer: - **Rolling statistics:** mean, std, min, max over the last k values - **Calendar features:** day of week, month, is_holiday, is_weekend - **Differenced values:** change from previous step - **Expanding statistics:** cumulative mean, cumulative sum - **Ratio features:** current value / rolling mean (how far from recent average) - **Interaction features:** lag_1 * day_of_week (weekday effects on momentum)

How many lags? Use the autocorrelation function. If ACF is significant up to lag 10, use at least 10 lags. If there is weekly seasonality, include lag 7 (and possibly 14). More lags give the model more history but also more features to fit, increasing the risk of overfitting.

The target alignment trap. When creating lag features, the target must be the value at time t , and all features must use values at time $t-1$ or earlier. If you accidentally include the value at time t as a feature, you have a perfect predictor - and a completely useless model. This is the most common bug in time series feature engineering.

Walk-Forward Validation

This is the most important concept in this lesson. Standard k -fold cross-validation randomly assigns samples to train and test. For time series, this leaks future information.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/15-time-series>

Walk-forward validation: 1. Train on data up to time t 2. Predict at time $t+1$ (or $t+1$ to $t+k$ for multi-step) 3. Slide the window forward 4. Repeat

Each test fold only contains data that comes after all training data. No future leakage. This gives you an honest estimate of how the model will perform when deployed.

Expanding window uses all historical data for training (window grows). **Sliding window** uses a fixed-size training window (window slides). Use expanding when you believe older data is still relevant. Use sliding when the world changes and old data hurts.

ARIMA Intuition

ARIMA is the classical time series model. It has three components:

- **AR (Autoregressive):** Predict from past values. AR(p) uses the last p values.
- **I (Integrated):** Differencing to achieve stationarity. I(d) applies d rounds of differencing.
- **MA (Moving Average):** Predict from past forecast errors. MA(q) uses the last q errors.

ARIMA(p, d, q) combines all three. You choose p, d, q based on ACF/PACF analysis or automated search (auto-ARIMA).

We will not implement ARIMA from scratch – it requires numerical optimization that is beyond the scope of this lesson. The key insight is understanding what each component does so you can interpret ARIMA results and know when to use it.

When to Use What

Approach	Best For	Handles Seasonality	Handles External Features
Lag features + ML	Tabular with many external features	With calendar features	Yes
ARIMA	Single univariate series, short-term	SARIMA variant	No (ARIMAX for limited)
Exponential smoothing	Simple trend + seasonality	Yes (Holt-Winters)	No
Prophet	Business forecasting, holidays	Yes (Fourier terms)	Limited
Neural networks (LSTM, Transformer)	Long sequences, many series	Learned	Yes

For most practical problems, lag features + gradient boosting is the strongest starting point. It handles external features naturally, does not require stationarity, and is easy to debug.

Forecasting Horizons and Strategies

Single-step forecasting predicts one time step ahead. Multi-step forecasting predicts multiple steps. There are three strategies:

Recursive (iterated): Predict one step ahead, use the prediction as input for the next step. Simple but errors accumulate – each prediction uses the previous prediction, so mistakes compound.

Direct: Train a separate model for each horizon. Model-1 predicts $t+1$, Model-5 predicts $t+5$. No error accumulation, but each model has fewer training samples and they do not share information.

Multi-output: Train one model that outputs all horizons simultaneously. Shares information across horizons but requires a model that supports multiple outputs (or a custom loss function).

For most practical problems, start with recursive for short horizons (1-5 steps) and direct for longer horizons.

Common Mistakes in Time Series

Mistake	Why it happens	How to fix
Random train/test split	Habit from standard ML	Use walk-forward or temporal split
Using future features	Feature at time t included by mistake	Audit every feature for temporal alignment
Overfitting to seasonality	Model memorizes calendar patterns	Hold out a full seasonal cycle in the test set
Ignoring scale changes	Revenue doubles but patterns stay	Model percentage change instead of absolute
Too many lag features	"More history is better"	Use ACF to determine relevant lags
Not differencing	"The model will figure it out"	Tree models handle trends; linear models need stationarity

Interactive figure: f3-series-decompose. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/15-time-series>

Build It

The code in `code/time_series.py` implements the core building blocks from scratch.

Lag Feature Creator

```
def make_lag_features(series, n_lags):
    n = len(series)
    X = np.full((n, n_lags), np.nan)
    for lag in range(1, n_lags + 1):
        X[lag:, lag - 1] = series[:-lag]
    valid = ~np.isnan(X).any(axis=1)
    return X[valid], series[valid]
```

This converts a 1D series into a feature matrix where each row has the last `n_lags` values as features, and the current value as the target.

Walk-Forward Cross-Validation

```
def walk_forward_split(n_samples, n_splits=5, min_train=50):
    assert min_train < n_samples, "min_train must be less than n_samples"
    step = max(1, (n_samples - min_train) // n_splits)
    for i in range(n_splits):
        train_end = min_train + i * step
        test_end = min(train_end + step, n_samples)
        if train_end >= n_samples:
            break
        yield slice(0, train_end), slice(train_end, test_end)
```

Each split ensures training data comes strictly before test data. The training window expands with each fold.

Simple Autoregressive Model

A pure AR model is just linear regression on lag features:

```
class SimpleAR:
    def __init__(self, n_lags=5):
        self.n_lags = n_lags
        self.weights = None
        self.bias = None

    def fit(self, series):
        X, y = make_lag_features(series, self.n_lags)
        # Solve via normal equations
        X_b = np.column_stack([np.ones(len(X)), X])
        theta = np.linalg.lstsq(X_b, y, rcond=None)[0]
        self.bias = theta[0]
        self.weights = theta[1:]
        return self
```

This is conceptually identical to linear regression from Lesson 02, but applied to time-lagged versions of the same variable.

Stationarity Check

The code computes rolling statistics to visually and numerically assess stationarity:

```
def check_stationarity(series, window=50):
    rolling_mean = np.array([
        series[max(0, i - window):i].mean()
        for i in range(1, len(series) + 1)
    ])
    rolling_std = np.array([
        series[max(0, i - window):i].std()
        for i in range(1, len(series) + 1)
    ])
    return rolling_mean, rolling_std
```

If the rolling mean drifts or the rolling std changes, the series is non-stationary. Apply differencing and check again.

The code also checks stationarity by comparing the first half and second half of the series. If the means differ by more than half a standard deviation or the variance ratio exceeds 2x, the

series is flagged as non-stationary.

Autocorrelation

```
def autocorrelation(series, max_lag=20):
    n = len(series)
    mean = series.mean()
    var = series.var()
    acf = np.zeros(max_lag + 1)
    for k in range(max_lag + 1):
        cov = np.mean((series[:n-k] - mean) * (series[k:] - mean))
        acf[k] = cov / var if var > 0 else 0
    return acf
```

Use It

With sklearn, you use lag features directly with any regressor:

```
from sklearn.linear_model import Ridge
from sklearn.ensemble import GradientBoostingRegressor
```

```
X, y = make_lag_features(series, n_lags=10)
```

```
for train_idx, test_idx in walk_forward_split(len(X)):
    model = Ridge(alpha=1.0)
    model.fit(X[train_idx], y[train_idx])
    predictions = model.predict(X[test_idx])
```

For ARIMA, use statsmodels:

```
from statsmodels.tsa.arima.model import ARIMA
```

```
model = ARIMA(train_series, order=(5, 1, 2))
fitted = model.fit()
forecast = fitted.forecast(steps=30)
```

The code in `time_series.py` demonstrates both approaches and compares them using walk-forward validation.

sklearn TimeSeriesSplit

sklearn provides `TimeSeriesSplit` which implements walk-forward validation:

```
from sklearn.model_selection import TimeSeriesSplit

tscv = TimeSeriesSplit(n_splits=5)
for train_index, test_index in tscv.split(X):
    X_train, X_test = X[train_index], X[test_index]
    y_train, y_test = y[train_index], y[test_index]
    model.fit(X_train, y_train)
    score = model.score(X_test, y_test)
```

This is equivalent to our from-scratch `walk_forward_split` but integrated into sklearn's cross-validation framework. You can use it with `cross_val_score`:

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(model, X, y, cv=TimeSeriesSplit(n_splits=5))
print(f"Mean score: {scores.mean():.4f} +/- {scores.std():.4f}")
```

Evaluation Metrics

Time series forecasting uses regression metrics, but with time-aware context:

- **MAE (Mean Absolute Error):** Average of $|y_{\text{true}} - y_{\text{pred}}|$. Easy to interpret in original units. “On average, predictions are off by 3.2 degrees.”
- **RMSE (Root Mean Squared Error):** Square root of mean squared error. Penalizes large errors more than MAE. Use when big errors are worse than many small errors.
- **MAPE (Mean Absolute Percentage Error):** Average of $|\text{error} / \text{true_value}| * 100$. Scale-independent, useful for comparing across different series. But undefined when true values are zero.
- **Naive baseline comparison:** Always compare against simple baselines. The seasonal naive baseline predicts the value from one period ago (yesterday, last week). If your model cannot beat naive, something is wrong.

Rolling Features

The code demonstrates adding rolling statistics (mean, std, min, max over windows of 7 and 14 days) to lag features. These give the model information about recent trends and volatility that lag features alone do not capture.

For example, if the rolling mean is rising, it suggests an upward trend. If the rolling std is increasing, it suggests growing volatility. These are the kinds of patterns that tree-based models can learn from but linear models cannot.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/15-time-series>

Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/15-time-series/code>

1. **Stationarity experiment.** Generate a series with a linear trend. Check stationarity with rolling statistics. Apply first differencing. Check again. How many rounds of differencing does it take for a quadratic trend?
2. **Lag selection.** Compute ACF on a seasonal series (period=7). Which lags have the highest autocorrelation? Create lag features using only those lags (not consecutive lags). Does accuracy improve compared to using lags 1 through 7?
3. **Walk-forward vs random split.** Train a Ridge regression on lag features. Evaluate with random 80/20 split and with walk-forward validation. How much does the random split overestimate performance?
4. **Feature engineering.** Add rolling mean (window=7), rolling std (window=7), and day-of-week features to the lag features. Compare accuracy with and without these extras using walk-forward validation.

5. **Multi-step forecasting.** Modify the AR model to predict 5 steps ahead instead of 1. Compare two strategies: (a) predict one step, use the prediction as input for the next step (recursive), and (b) train separate models for each horizon (direct). Which is more accurate?

Key Terms

Term	What people say	What it actually means
Stationarity	“The stats don’t change over time”	A series whose mean, variance, and autocorrelation structure are constant over time
Differencing	“Subtract consecutive values”	Computing $y[t] - y[t-1]$ to remove trends and achieve stationarity
Autocorrelation (ACF)	“How a series correlates with itself”	The correlation between a time series and a lagged copy of itself, as a function of the lag
Partial autocorrelation (PACF)	“Direct correlation only”	Autocorrelation at lag k after removing the effect of all shorter lags
Lag features	“Past values as inputs”	Using $y[t-1], y[t-2], \dots, y[t-k]$ as features to predict $y[t]$
Walk-forward validation	“Time-respecting cross-validation”	Evaluation where training data always precedes test data chronologically
ARIMA	“The classic time series model”	AutoRegressive Integrated Moving Average: combines past values (AR), differencing (I), and past errors (MA)
Seasonality	“Repeating calendar patterns”	Regular, predictable cycles in a time series tied to calendar periods (daily, weekly, yearly)
Trend	“The long-term direction”	A persistent increase or decrease in the series level over time
Expanding window	“Use all history”	Walk-forward validation where the training set grows with each fold
Sliding window	“Fixed-size history”	Walk-forward validation where the training set is a fixed-length window that slides forward

Further Reading

- Hyndman and Athanasopoulos, *Forecasting: Principles and Practice* (3rd ed.) – the best free textbook on time series forecasting
- scikit-learn Time Series Split – sklearn’s walk-forward splitter
- statsmodels ARIMA docs – ARIMA implementation with diagnostics
- Makridakis et al., *The M5 Competition* (2022) – large-scale forecasting competition showing ML methods vs statistical methods

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/15-time-series>

- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/15-time-series/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/15-time-series>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Anomaly Detection

Normal is easy to define. Abnormal is whatever doesn't fit.

Type: Build **Language:** Python **Prerequisites:** Phase 2, Lessons 01-09 **Time:** ~75 minutes

Learning Objectives

- Implement Z-score, IQR, and Isolation Forest anomaly detection methods from scratch
- Distinguish between point, contextual, and collective anomalies and select the appropriate detection method for each
- Explain why anomaly detection is framed as modeling normal data rather than classifying anomalies
- Compare unsupervised anomaly detection with supervised classification and evaluate the tradeoff between novel anomaly coverage and precision

The Problem

A credit card is used in New York at 2pm, then in Tokyo at 2:05pm. A factory sensor reads 150 degrees when the normal range is 80-120. A server sends 50,000 requests per second when the daily average is 200.

These are anomalies. Finding them matters. Fraud costs billions. Equipment failures cost downtime. Network intrusions cost data.

The challenge: you rarely have labeled examples of anomalies. Fraud makes up 0.1% of transactions. Equipment failures happen a few times per year. You cannot train a standard classifier because there is almost nothing in the "anomaly" class to learn from. Even if you have some labels, the anomalies you have seen are not the only types you will encounter. Tomorrow's fraud scheme looks different from today's.

Anomaly detection flips the problem. Instead of learning what is abnormal, learn what is normal. Anything that deviates from normal is suspicious. This works without labels, adapts to new types of anomalies, and scales to massive datasets.

The Concept

Types of Anomalies

Not all anomalies are the same:

- **Point anomalies.** A single data point that is unusual regardless of context. A temperature reading of 500 degrees. A transaction of \$50,000 from an account that normally spends \$50.
- **Contextual anomalies.** A data point that is unusual given its context. A temperature of 90 degrees is normal in summer, anomalous in winter. Same value, different context.

- **Collective anomalies.** A sequence of data points that is unusual as a group, even though each individual point might be normal. Five login failures is normal. Fifty in a row is a brute-force attack.

Most methods detect point anomalies. Contextual anomalies need time or location features. Collective anomalies need sequence-aware methods.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/16-anomaly-detection>

The Unsupervised Framing

In standard classification, you have labels for both classes. In anomaly detection, you typically have one of three situations:

1. **Fully unsupervised.** No labels at all. You fit the detector on all data and hope anomalies are rare enough not to corrupt the “normal” model.
2. **Semi-supervised.** You have a clean dataset of normal data only. You fit on this clean set and score everything else. This is the strongest setup when possible.
3. **Weakly supervised.** You have a few labeled anomalies. Use them for evaluation, not training. Train unsupervised, then measure precision/recall on the labeled subset.

The key insight: anomaly detection is fundamentally different from classification. You are modeling the distribution of normal data, not the decision boundary between two classes.

Supervised vs Unsupervised: The Tradeoff

If you do have labeled anomalies, should you use them for training (supervised classification) or for evaluation only (unsupervised detection)?

Supervised (treat as classification): - Catches the exact types of anomalies you have seen before - Higher precision on known anomaly types - Misses novel anomaly types entirely - Requires retraining when new anomaly types emerge - Needs enough anomaly examples (often too few)

Unsupervised (model normal, flag deviations): - Catches any deviation from normal, including novel types - Does not require labeled anomalies - Higher false positive rate (not everything unusual is bad) - More robust to distribution shift

In practice, the best systems combine both: unsupervised detection for broad coverage, supervised models for known high-priority anomaly types, and human review for ambiguous cases.

Z-Score Method

The simplest approach. Compute the mean and standard deviation of each feature. Flag any point more than k standard deviations from the mean.

```
z_score = (x - mean) / std
anomaly if |z_score| > threshold
```

The default threshold is 3.0 (99.7% of normal data falls within 3 standard deviations for a Gaussian distribution).

Strengths: Simple. Fast. Interpretable (“this value is 4.5 standard deviations from normal”).

Weaknesses: Assumes data is normally distributed. Sensitive to outliers in the training data (the outliers shift the mean and inflate the std, making them harder to detect). Fails on multimodal distributions.

When it works well: Single-feature monitoring where data is roughly bell-shaped. Server response times, manufacturing tolerances, sensor readings with stable baselines.

When it fails: Multi-cluster data (two office locations with different baseline temperatures), skewed data (transaction amounts where \$1000 is rare but not anomalous), data with outliers in the training set.

IQR Method

More robust than Z-score. Uses the interquartile range instead of mean and standard deviation.

```
Q1 = 25th percentile
Q3 = 75th percentile
IQR = Q3 - Q1
lower_bound = Q1 - factor * IQR
upper_bound = Q3 + factor * IQR
anomaly if x < lower_bound or x > upper_bound
```

The default factor is 1.5.

Strengths: Robust to outliers (percentiles are not affected by extreme values). Works on skewed distributions. No normality assumption.

Weaknesses: Univariate only (applies per feature independently). Cannot detect anomalies that are unusual only when features are considered together (a point might be normal in each feature individually but anomalous in the joint space).

Practical note: The 1.5 factor in IQR corresponds to the whiskers in a box plot. Points outside the whiskers are potential outliers. Using 3.0 instead of 1.5 makes the detector more conservative (fewer flags, fewer false positives). The right factor depends on your tolerance for false alarms.

Isolation Forest

The key insight: anomalies are few and different. In a random partitioning of the data, anomalies are easier to isolate – they need fewer random splits to be separated from the rest.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/16-anomaly-detection>

How it works: 1. Build many random trees (an isolation forest) 2. At each node, pick a random feature and a random split value between the feature's min and max 3. Keep splitting until every point is isolated (in its own leaf) 4. Anomalies have shorter average path lengths across all trees

Why it works: Normal points live in dense regions. Many random splits are needed to isolate one from its neighbors. Anomalies live in sparse regions. One or two random splits are enough to isolate them.

The anomaly score is based on the average path length across all trees, normalized by the expected path length of a random binary search tree:

$$\text{score}(x) = 2^{(-\text{average_path_length}(x) / c(n))}$$

Where $c(n)$ is the expected path length for n samples. Score near 1 means anomaly. Score near 0.5 means normal. Score near 0 means very normal (deep in dense clusters).

Strengths: No distribution assumptions. Works in high dimensions. Scales well (sublinear in sample size because each tree uses a subsample). Handles mixed feature types.

Weaknesses: Struggles with anomalies in dense regions (masking effect). Random splitting is less effective when many features are irrelevant.

Key hyperparameters: - `n_estimators`: Number of trees. 100 is usually enough. More trees give more stable scores but slower computation. - `max_samples`: Number of samples per tree. 256 is the default in the original paper. Smaller values make individual trees less accurate but increase diversity. The subsampling is what makes Isolation Forest fast - each tree sees a small fraction of the data. - `contamination`: Expected fraction of anomalies. Used only for setting the threshold. Does not affect the scores themselves.

Local Outlier Factor (LOF)

LOF compares the local density around a point to the density around its neighbors. A point in a sparse region surrounded by dense regions is anomalous.

How it works: 1. For each point, find its k nearest neighbors 2. Compute the local reachability density (how dense is the neighborhood) 3. Compare each point's density to its neighbors' densities 4. If a point has much lower density than its neighbors, it is an outlier

LOF score: - LOF close to 1.0 means similar density as neighbors (normal) - LOF greater than 1.0 means lower density than neighbors (potentially anomalous) - LOF much greater than 1.0 (e.g., 2.0+) means significantly lower density (likely anomaly)

The "local" part is critical. Consider a dataset with two clusters: a dense cluster of 1000 points and a sparse cluster of 50 points. A point on the edge of the sparse cluster is not globally unusual - it has 50 neighbors. But it is locally unusual if its immediate neighbors are denser than it is. LOF captures this nuance that global methods miss.

Strengths: Detects local anomalies (points that are unusual in their neighborhood, even if they are not globally unusual). Works on clusters of different densities.

Weaknesses: Slow on large datasets ($O(n^2)$ for naive implementation). Sensitive to the choice of k . Does not work well in very high dimensions (curse of dimensionality affects distance calculations).

Comparison

Method	Assumptions	Speed	Handles High Dims	Detects Local Anomalies
Z-score	Normal distribution	Very fast	Yes (per feature)	No
IQR	None (per feature)	Very fast	Yes (per feature)	No
Isolation Forest	None	Fast	Yes	Partially
LOF	Distance is meaningful	Slow	Poorly	Yes

Evaluation Challenges

Evaluating anomaly detectors is harder than evaluating classifiers:

- **Extreme class imbalance.** With 0.1% anomalies, predicting "normal" for everything gives 99.9% accuracy. Accuracy is useless.
- **AUROC is misleading.** With heavy imbalance, AUROC can look good even when the model misses most anomalies at practical thresholds.

- **Better metrics:** Precision@k (of the top k flagged items, how many are real anomalies), AUPRC (area under precision-recall curve), and recall at a fixed false positive rate.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/16-anomaly-detection>

Anomaly Detection Pipeline

In practice, anomaly detection follows this workflow:

1. **Collect baseline data.** Ideally, a period where you know there are no (or very few) anomalies.
2. **Feature engineering.** Raw features plus derived features (rolling statistics, time features, ratios).
3. **Train the detector.** Fit on the baseline data. The model learns what “normal” looks like.
4. **Score new data.** Each new observation gets an anomaly score.
5. **Threshold selection.** Choose the score cutoff. This is a business decision: higher threshold means fewer false alarms but more missed anomalies.
6. **Alert and investigate.** Flagged points go to human review or automated response.
7. **Feedback collection.** Record whether flagged items were true anomalies or false alarms. Use this data to evaluate the detector and tune the threshold over time.

The pipeline is never “done.” Data distributions shift, new anomaly types emerge, and thresholds need adjustment. Treat anomaly detection as a living system, not a one-time model.

Interactive figure: f3-anomaly-fence. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/16-anomaly-detection>

Build It

The code in `code/anomaly_detection.py` implements Z-score, IQR, and Isolation Forest from scratch.

Z-Score Detector

```
def zscore_detect(X, threshold=3.0):
    mean = X.mean(axis=0)
    std = X.std(axis=0)
    std[std == 0] = 1.0
    z = np.abs((X - mean) / std)
    return z.max(axis=1) > threshold
```

Simple and vectorized. Flags a point if any feature exceeds the threshold.

IQR Detector

```
def iqr_detect(X, factor=1.5):
    q1 = np.percentile(X, 25, axis=0)
    q3 = np.percentile(X, 75, axis=0)
    iqr = q3 - q1
    iqr[iqr == 0] = 1.0
    lower = q1 - factor * iqr
    upper = q3 + factor * iqr
```

```
outside = (X < lower) | (X > upper)
return outside.any(axis=1)
```

Isolation Forest from Scratch

The from-scratch implementation builds isolation trees that randomly partition the feature space:

```
class IsolationTree:
    def __init__(self, max_depth):
        self.max_depth = max_depth

    def fit(self, X, depth=0):
        n, p = X.shape
        if depth >= self.max_depth or n <= 1:
            self.is_leaf = True
            self.size = n
            return self
        self.is_leaf = False
        self.feature = np.random.randint(p)
        x_min = X[:, self.feature].min()
        x_max = X[:, self.feature].max()
        if x_min == x_max:
            self.is_leaf = True
            self.size = n
            return self
        self.threshold = np.random.uniform(x_min, x_max)
        left_mask = X[:, self.feature] < self.threshold
        self.left = IsolationTree(self.max_depth).fit(X[left_mask], depth + 1)
        self.right = IsolationTree(self.max_depth).fit(X[~left_mask], depth + 1)
        return self
```

The path length to isolate a point determines its anomaly score. Shorter paths mean more anomalous.

The IsolationForest class wraps multiple trees:

```
class IsolationForest:
    def __init__(self, n_estimators=100, max_samples=256, seed=42):
        self.n_estimators = n_estimators
        self.max_samples = max_samples

    def fit(self, X):
        sample_size = min(self.max_samples, X.shape[0])
        max_depth = int(np.ceil(np.log2(sample_size)))
        for _ in range(self.n_estimators):
            idx = rng.choice(X.shape[0], size=sample_size, replace=False)
            tree = IsolationTree(max_depth=max_depth)
            tree.fit(X[idx])
            self.trees.append(tree)

    def anomaly_score(self, X):
        avg_path = average path length across all trees
        scores = 2.0 ** (-avg_path / c(max_samples))
        return scores
```

The normalization factor $c(n)$ is the expected path length of an unsuccessful search in a binary search tree with n elements. It equals $2 * H(n-1) - 2*(n-1)/n$ where H is the harmonic number. This normalization ensures scores are comparable across datasets of different sizes.

Demo Scenarios

The code generates multiple test scenarios:

1. **Single cluster with outliers.** A 2D Gaussian cluster with anomalies injected far from the center. All methods should work here.
2. **Multimodal data.** Three clusters of different sizes and densities. Points between clusters are anomalous. Z-score struggles because the per-feature ranges are wide.
3. **High-dimensional data.** 50 features, but anomalies differ in only 5 of them. Tests whether methods can find anomalies in a subset of features.

Each demo compares all methods using precision, recall, F1, and Precision@k.

Use It

With sklearn (using library implementations, not from-scratch):

```
from sklearn.ensemble import IsolationForest
from sklearn.neighbors import LocalOutlierFactor

iso = IsolationForest(n_estimators=100, contamination=0.05, random_state=42)
iso.fit(X_train)
predictions = iso.predict(X_test)

lof = LocalOutlierFactor(n_neighbors=20, contamination=0.05, novelty=True)
lof.fit(X_train)
predictions = lof.predict(X_test)
```

Note contamination sets the expected fraction of anomalies. Setting it correctly matters – too low misses anomalies, too high creates false alarms.

The code in `anomaly_detection.py` compares from-scratch implementations against sklearn on the same data.

sklearn Contamination Parameter

The contamination parameter in sklearn determines the threshold for converting continuous anomaly scores into binary predictions. It does not change the underlying scores.

```
iso_5 = IsolationForest(contamination=0.05)
iso_10 = IsolationForest(contamination=0.10)
```

Both produce the same anomaly scores. But `iso_5` flags the top 5% while `iso_10` flags the top 10%. If you do not know the true anomaly rate (you usually do not), set contamination to “auto” and work with the raw scores directly. Set your own threshold based on the cost tradeoff between false positives and false negatives.

One-Class SVM

Another unsupervised anomaly detector worth knowing. One-Class SVM fits a boundary around normal data in a high-dimensional feature space (using the kernel trick).

```
from sklearn.svm import OneClassSVM

oc_svm = OneClassSVM(kernel="rbf", gamma="auto", nu=0.05)
oc_svm.fit(X_train)
predictions = oc_svm.predict(X_test)
```

The `nu` parameter approximates the fraction of anomalies. One-Class SVM works well on small to medium datasets but does not scale to very large data (the kernel matrix grows quadratically).

Autoencoder Approach (Preview)

Autoencoders are neural networks that learn to compress and reconstruct data. Train on normal data. At test time, anomalies have high reconstruction error because the network learned to reconstruct normal patterns only.

This is covered in Phase 3 (Deep Learning), but the principle is the same: model what is normal, flag what deviates.

Ensemble Anomaly Detection

Just as ensemble methods improve classification (Lesson 11), combining multiple anomaly detectors improves detection. The simplest approach:

1. Run multiple detectors (Z-score, IQR, Isolation Forest, LOF)
2. Normalize each detector's scores to [0, 1]
3. Average the normalized scores
4. Flag points above the threshold on the average score

This reduces false positives because different methods have different failure modes. A point flagged by all four methods is almost certainly anomalous. A point flagged by only one might be a quirk of that method.

More sophisticated ensembles weight each detector by its estimated reliability (measured on a validation set with known anomalies, if available).

Production Considerations

1. **Threshold drift.** As data distribution shifts, a fixed threshold becomes outdated. Monitor the distribution of anomaly scores and adjust periodically.
2. **Alert fatigue.** Too many false alarms and operators stop paying attention. Start with a high threshold (fewer, more reliable alerts) and lower it as trust builds.
3. **Ensemble approach.** In production, combine multiple detectors. Flag a point only if multiple methods agree it is anomalous. This reduces false positives significantly.
4. **Feature engineering.** Raw features are rarely enough. Add rolling statistics, ratios, time-since-last-event, and domain-specific features. A good feature set matters more than the choice of detector.
5. **Feedback loop.** When operators investigate flagged items and confirm or dismiss them, feed this back into the system. Accumulate labeled data over time to evaluate and improve the detector.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/16-anomaly-detection>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/16-anomaly-detection/code>

1. **Threshold tuning.** Run the Z-score detector with thresholds from 1.0 to 5.0 in steps of 0.5. Plot precision and recall at each threshold. Where is the sweet spot for your data?
2. **Multivariate anomalies.** Create 2D data where each feature individually looks normal, but the combination is anomalous (e.g., points far from the main cluster diagonal). Show that Z-score per feature misses these but Isolation Forest catches them.
3. **LOF from scratch.** Implement Local Outlier Factor using k-nearest neighbors. Compare against sklearn's LocalOutlierFactor on the same data. Use k=10 and k=50 - how does the choice of k affect results?
4. **Streaming anomaly detection.** Modify the Z-score detector to work in a streaming setting: update the running mean and variance as new points arrive (Welford's online algorithm). Compare to batch Z-score on the same data.
5. **Real-world evaluation.** Take a dataset with known anomalies (credit card fraud from Kaggle, for example). Evaluate all four methods using precision@100, precision@500, and AUPRC. Which method works best? Why?

Key Terms

Term	What people say	What it actually means
Anomaly	"Outlier, unusual point"	A data point that deviates significantly from the expected pattern of normal data
Point anomaly	"A single weird value"	An individual observation that is unusual regardless of context
Contextual anomaly	"Normal value, wrong context"	An observation that is unusual given its context (time, location, etc.) but might be normal in another context
Isolation Forest	"Random splits to find outliers"	An ensemble of random trees that isolates anomalies with fewer splits than normal points
Local Outlier Factor	"Compare density to neighbors"	A method that flags points whose local density is much lower than their neighbors' density
Z-score	"Standard deviations from mean"	$(x - \text{mean}) / \text{std}$, measuring how far a point is from the center in units of standard deviation
IQR	"Interquartile range"	$Q3 - Q1$, measuring the spread of the middle 50% of data, used for robust outlier detection
Contamination	"Expected fraction of anomalies"	A hyperparameter telling the detector what proportion of the data it should flag as anomalous
Precision@k	"Of the top k flags, how many are real"	Precision computed on only the k most suspicious points, useful for imbalanced anomaly detection

Term	What people say	What it actually means
AUPRC	“Area under precision-recall curve”	A metric that summarizes precision-recall performance across all thresholds, better than AUROC for imbalanced data

Further Reading

- Liu et al., Isolation Forest (2008) – the original Isolation Forest paper
- Breunig et al., LOF: Identifying Density-Based Local Outliers (2000) – the original LOF paper
- scikit-learn Outlier Detection docs – overview of all sklearn anomaly detectors
- Chandola et al., Anomaly Detection: A Survey (2009) – comprehensive survey of anomaly detection methods
- Goldstein and Uchida, A Comparative Evaluation of Unsupervised Anomaly Detection Algorithms (2016) – empirical comparison of 10 methods on real datasets

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/16-anomaly-detection>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/16-anomaly-detection/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/16-anomaly-detection>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Handling Imbalanced Data

When 99% of your data is “normal,” accuracy is a lie.

Type: Build **Language:** Python **Prerequisites:** Phase 2, Lessons 01-09 (especially evaluation metrics) **Time:** ~90 minutes

Learning Objectives

- Implement SMOTE from scratch and explain how synthetic oversampling differs from random duplication
- Evaluate imbalanced classifiers using F1, AUPRC, and Matthews Correlation Coefficient instead of accuracy
- Compare class weighting, threshold tuning, and resampling strategies and select the right approach for a given imbalance ratio
- Build a complete imbalanced data pipeline that combines SMOTE, class weights, and threshold optimization

The Problem

You build a fraud detection model. It gets 99.9% accuracy. You celebrate. Then you realize it predicts “not fraud” for every single transaction.

This is not a bug. It is the rational thing to do when only 0.1% of transactions are fraudulent. The model learns that always guessing the majority class minimizes overall error. It is technically correct and completely useless.

This happens everywhere real classification matters. Disease diagnosis: 1% positive rate. Network intrusion: 0.01% attacks. Manufacturing defects: 0.5% defective. Spam filtering: 20% spam. Churn prediction: 5% churners. The more consequential the minority class, the rarer it tends to be.

Accuracy fails because it treats all correct predictions equally. Correctly labeling a legitimate transaction and correctly catching fraud both count as one point of accuracy. But catching fraud is the entire reason the model exists. We need metrics, techniques, and training strategies that force the model to pay attention to the rare but important class.

The Concept

Why Accuracy Fails

Consider a dataset with 1000 samples: 990 negative, 10 positive. A model that always predicts negative:

	Predicted Positive	Predicted Negative
Actually Positive	0 (TP)	10 (FN)

	Predicted Positive	Predicted Negative
Actually Negative	0 (FP)	990 (TN)

$$\text{Accuracy} = (0 + 990) / 1000 = 99.0\%$$

The model catches zero fraud. Zero disease. Zero defects. But accuracy says 99%. This is why accuracy is dangerous for imbalanced problems.

Better Metrics

Precision = $TP / (TP + FP)$. Of everything flagged as positive, how many actually are? High precision means few false alarms.

Recall = $TP / (TP + FN)$. Of everything actually positive, how many did we catch? High recall means few missed positives.

F1 Score = $2 * \text{precision} * \text{recall} / (\text{precision} + \text{recall})$. The harmonic mean. Penalizes extreme imbalance between precision and recall more than the arithmetic mean would.

F-beta Score = $(1 + \text{beta}^2) * \text{precision} * \text{recall} / (\text{beta}^2 * \text{precision} + \text{recall})$. When $\text{beta} > 1$, recall matters more. When $\text{beta} < 1$, precision matters more. F2 is common in fraud detection (missing fraud is worse than a false alarm).

AUPRC (Area Under Precision-Recall Curve). Like AUC-ROC but more informative for imbalanced data. A random classifier has AUPRC equal to the positive class rate (not 0.5 like ROC). This makes improvements easier to see.

Matthews Correlation Coefficient = $(TP * TN - FP * FN) / \sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}$. Ranges from -1 to +1. Only gives a high score when the model does well on both classes. Balanced even when classes are very different sizes.

For the “always predict negative” model above: precision = 0/0 (undefined, often set to 0), recall = 0/10 = 0, F1 = 0, MCC = 0. These metrics correctly identify the model as worthless.

The Imbalanced Data Pipeline

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/17-imbalanced-data>

SMOTE: Synthetic Minority Oversampling Technique

Random oversampling duplicates existing minority samples. This works but risks overfitting because the model sees identical points repeatedly.

SMOTE creates new synthetic minority samples that are plausible but not copies. The algorithm:

1. For each minority sample x , find its k nearest neighbors among other minority samples
2. Pick one neighbor at random
3. Create a new sample on the line segment between x and that neighbor

The formula: $\text{new_sample} = x + \text{random}(0, 1) * (\text{neighbor} - x)$

This interpolates between real minority points, creating samples in the same region of feature space without just copying existing data.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/17-imbalanced-data>

Sampling Strategies Compared

Random Oversampling: duplicate minority samples to match majority count. - Pros: simple, no information loss - Cons: exact duplicates cause overfitting, increases training time

Random Undersampling: remove majority samples to match minority count. - Pros: fast training, simple - Cons: throws away potentially useful majority data, higher variance

SMOTE: create synthetic minority samples via interpolation. - Pros: generates new data points, reduces overfitting compared to random oversampling - Cons: can create noisy samples near the decision boundary, does not account for majority class distribution

Strategy	Data Changed	Risk	When to Use
Oversample	Minority duplicated	Overfitting	Small datasets, moderate imbalance
Undersample	Majority removed	Information loss	Large datasets, want fast training
SMOTE	Synthetic minority added	Boundary noise	Moderate imbalance, enough minority samples for k-NN

Class Weights

Instead of changing the data, change how the model treats errors. Assign higher weight to misclassifying the minority class.

For a binary problem with 950 negative and 50 positive samples: - Weight for negative class = $n_samples / (2 * n_negative) = 1000 / (2 * 950) = 0.526$ - Weight for positive class = $n_samples / (2 * n_positive) = 1000 / (2 * 50) = 10.0$

The positive class gets 19x the weight. Misclassifying one positive sample costs as much as misclassifying 19 negative samples. The model is forced to pay attention to the minority class.

In logistic regression, this modifies the loss function:

$$\text{weighted_loss} = -\sum(w_i * [y_i * \log(p_i) + (1-y_i) * \log(1-p_i)])$$

where w_i depends on the class of sample i .

Class weights are mathematically equivalent to oversampling in expectation, but without creating new data points. This makes them faster and avoids the overfitting risk of duplicated samples.

Threshold Tuning

Most classifiers output a probability. The default threshold is 0.5: if $P(\text{positive}) \geq 0.5$, predict positive. But 0.5 is arbitrary. When classes are imbalanced, the optimal threshold is usually much lower.

The process: 1. Train a model 2. Get predicted probabilities on the validation set 3. Sweep thresholds from 0.0 to 1.0 4. Compute F1 (or your chosen metric) at each threshold 5. Pick the threshold that maximizes your metric

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/17-imbalanced-data>

A model might output $P(\text{fraud}) = 0.15$ for a fraudulent transaction. At threshold 0.5, this is classified as not fraud. At threshold 0.10, it is correctly caught. The probability calibration

matters less than the ranking - as long as fraud gets higher probabilities than non-fraud, there exists a threshold that separates them.

Cost-Sensitive Learning

Generalization of class weights. Instead of uniform costs, assign specific misclassification costs:

	Predict Positive	Predict Negative
Actually Positive	0 (correct)	C_FN = 100
Actually Negative	C_FP = 1	0 (correct)

Missing a fraudulent transaction (FN) costs 100x more than a false alarm (FP). The model optimizes for total cost, not total error count.

This is the most principled approach when you can estimate real-world costs. A missed cancer diagnosis has a very different cost than a false alarm that leads to an extra biopsy. Making these costs explicit forces the right tradeoffs.

Decision Flowchart

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/17-imbalanced-data>

Interactive figure: class-imbalance. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/17-imbalanced-data>

Build It

Step 1: Generate an imbalanced dataset

```
import numpy as np

def make_imbalanced_data(n_majority=950, n_minority=50, seed=42):
    rng = np.random.RandomState(seed)

    X_maj = rng.randn(n_majority, 2) * 1.0 + np.array([0.0, 0.0])
    X_min = rng.randn(n_minority, 2) * 0.8 + np.array([2.5, 2.5])

    X = np.vstack([X_maj, X_min])
    y = np.concatenate([np.zeros(n_majority), np.ones(n_minority)])

    shuffle_idx = rng.permutation(len(y))
    return X[shuffle_idx], y[shuffle_idx]
```

Step 2: SMOTE from scratch

```
def euclidean_distance(a, b):
    return np.sqrt(np.sum((a - b) ** 2))
```

```

def find_k_neighbors(X, idx, k):
    distances = []
    for i in range(len(X)):
        if i == idx:
            continue
        d = euclidean_distance(X[idx], X[i])
        distances.append((i, d))
    distances.sort(key=lambda x: x[1])
    return [d[0] for d in distances[:k]]

def smote(X_minority, k=5, n_synthetic=100, seed=42):
    rng = np.random.RandomState(seed)
    n_samples = len(X_minority)
    k = min(k, n_samples - 1)
    synthetic = []

    for _ in range(n_synthetic):
        idx = rng.randint(0, n_samples)
        neighbors = find_k_neighbors(X_minority, idx, k)
        neighbor_idx = neighbors[rng.randint(0, len(neighbors))]
        t = rng.random()
        new_point = X_minority[idx] + t * (X_minority[neighbor_idx] - X_minority[idx])
        synthetic.append(new_point)

    return np.array(synthetic)

```

Step 3: Random oversampling and undersampling

```

def random_oversample(X, y, seed=42):
    rng = np.random.RandomState(seed)
    classes, counts = np.unique(y, return_counts=True)
    max_count = counts.max()

    X_resampled = list(X)
    y_resampled = list(y)

    for cls, count in zip(classes, counts):
        if count < max_count:
            cls_indices = np.where(y == cls)[0]
            n_needed = max_count - count
            chosen = rng.choice(cls_indices, size=n_needed, replace=True)
            X_resampled.extend(X[chosen])
            y_resampled.extend(y[chosen])

    X_out = np.array(X_resampled)
    y_out = np.array(y_resampled)
    shuffle = rng.permutation(len(y_out))
    return X_out[shuffle], y_out[shuffle]

def random_undersample(X, y, seed=42):
    rng = np.random.RandomState(seed)

```

```

classes, counts = np.unique(y, return_counts=True)
min_count = counts.min()

X_resampled = []
y_resampled = []

for cls in classes:
    cls_indices = np.where(y == cls)[0]
    chosen = rng.choice(cls_indices, size=min_count, replace=False)
    X_resampled.extend(X[chosen])
    y_resampled.extend(y[chosen])

X_out = np.array(X_resampled)
y_out = np.array(y_resampled)
shuffle = rng.permutation(len(y_out))
return X_out[shuffle], y_out[shuffle]

```

Step 4: Logistic regression with class weights

```

def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-np.clip(z, -500, 500)))

def logistic_regression_weighted(X, y, weights, lr=0.01, epochs=200):
    n_samples, n_features = X.shape
    w = np.zeros(n_features)
    b = 0.0

    for _ in range(epochs):
        z = X @ w + b
        pred = sigmoid(z)
        error = pred - y
        weighted_error = error * weights

        gradient_w = (X.T @ weighted_error) / n_samples
        gradient_b = np.mean(weighted_error)

        w -= lr * gradient_w
        b -= lr * gradient_b

    return w, b

def compute_class_weights(y):
    classes, counts = np.unique(y, return_counts=True)
    n_samples = len(y)
    n_classes = len(classes)
    weight_map = {}
    for cls, count in zip(classes, counts):
        weight_map[cls] = n_samples / (n_classes * count)
    return np.array([weight_map[yi] for yi in y])

```

Step 5: Threshold tuning

```
def find_optimal_threshold(y_true, y_probs, metric="f1"):
    best_threshold = 0.5
    best_score = -1.0

    for threshold in np.arange(0.05, 0.96, 0.01):
        y_pred = (y_probs >= threshold).astype(int)
        tp = np.sum((y_pred == 1) & (y_true == 1))
        fp = np.sum((y_pred == 1) & (y_true == 0))
        fn = np.sum((y_pred == 0) & (y_true == 1))

        if metric == "f1":
            precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
            recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
            score = 2 * precision * recall / (precision + recall) if (precision + recall) > 0 else 0.0
        elif metric == "recall":
            score = tp / (tp + fn) if (tp + fn) > 0 else 0.0
        elif metric == "precision":
            score = tp / (tp + fp) if (tp + fp) > 0 else 0.0

        if score > best_score:
            best_score = score
            best_threshold = threshold

    return best_threshold, best_score
```

Step 6: Evaluation functions

```
def confusion_matrix_values(y_true, y_pred):
    tp = np.sum((y_pred == 1) & (y_true == 1))
    tn = np.sum((y_pred == 0) & (y_true == 0))
    fp = np.sum((y_pred == 1) & (y_true == 0))
    fn = np.sum((y_pred == 0) & (y_true == 1))
    return tp, tn, fp, fn

def compute_metrics(y_true, y_pred):
    tp, tn, fp, fn = confusion_matrix_values(y_true, y_pred)
    accuracy = (tp + tn) / (tp + tn + fp + fn)
    precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
    recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
    f1 = 2 * precision * recall / (precision + recall) if (precision + recall) > 0 else 0.0

    denom = np.sqrt(float((tp + fp) * (tp + fn) * (tn + fp) * (tn + fn)))
    mcc = (tp * tn - fp * fn) / denom if denom > 0 else 0.0

    return {
        "accuracy": accuracy,
        "precision": precision,
        "recall": recall,
        "f1": f1,
        "mcc": mcc,
    }
```

Step 7: Compare all approaches

```
X, y = make_imbalanced_data(950, 50, seed=42)
split = int(0.8 * len(y))
X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]

# Baseline: no treatment
w_base, b_base = logistic_regression_weighted(
    X_train, y_train, np.ones(len(y_train)), lr=0.1, epochs=300
)
probs_base = sigmoid(X_test @ w_base + b_base)
preds_base = (probs_base >= 0.5).astype(int)

# Oversampled
X_over, y_over = random_oversample(X_train, y_train)
w_over, b_over = logistic_regression_weighted(
    X_over, y_over, np.ones(len(y_over)), lr=0.1, epochs=300
)
preds_over = (sigmoid(X_test @ w_over + b_over) >= 0.5).astype(int)

# SMOTE
minority_mask = y_train == 1
X_minority = X_train[minority_mask]
synthetic = smote(X_minority, k=5, n_synthetic=len(y_train) - 2 * int(minority_mask.sum()))
X_smote = np.vstack([X_train, synthetic])
y_smote = np.concatenate([y_train, np.ones(len(synthetic))])
w_sm, b_sm = logistic_regression_weighted(
    X_smote, y_smote, np.ones(len(y_smote)), lr=0.1, epochs=300
)
preds_smote = (sigmoid(X_test @ w_sm + b_sm) >= 0.5).astype(int)

# Class weights
sample_weights = compute_class_weights(y_train)
w_cw, b_cw = logistic_regression_weighted(
    X_train, y_train, sample_weights, lr=0.1, epochs=300
)
probs_cw = sigmoid(X_test @ w_cw + b_cw)
preds_cw = (probs_cw >= 0.5).astype(int)

# Threshold tuning (tune on held-out validation set, not test set)
probs_val = sigmoid(X_val @ w_cw + b_cw)
best_thresh, best_f1 = find_optimal_threshold(y_val, probs_val, metric="f1")
preds_thresh = (probs_cw >= best_thresh).astype(int)
```

The code file runs all of this in a single script and prints results.

Use It

With scikit-learn and imbalanced-learn, these techniques are one-liners:

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, f1_score
from sklearn.model_selection import train_test_split
```



```

from imblearn.over_sampling import SMOTE
from imblearn.under_sampling import RandomUnderSampler
from imblearn.pipeline import Pipeline

X_train, X_test, y_train, y_test = train_test_split(X, y, stratify=y)

model_weighted = LogisticRegression(class_weight="balanced")
model_weighted.fit(X_train, y_train)
print(classification_report(y_test, model_weighted.predict(X_test)))

smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(X_train, y_train)
model_smote = LogisticRegression()
model_smote.fit(X_resampled, y_resampled)
print(classification_report(y_test, model_smote.predict(X_test)))

pipeline = Pipeline([
    ("smote", SMOTE()),
    ("model", LogisticRegression(class_weight="balanced")),
])
pipeline.fit(X_train, y_train)
print(classification_report(y_test, pipeline.predict(X_test)))

```

The from-scratch implementations show exactly what each technique does. SMOTE is just k-NN interpolation on the minority class. Class weights multiply the loss. Threshold tuning is a for-loop over cutoffs. No magic.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/17-imbalanced-data>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/17-imbalanced-data/code>

1. **Borderline-SMOTE:** modify the SMOTE implementation to only generate synthetic samples for minority points that are near the decision boundary (those whose k-nearest neighbors include majority class samples). Compare results with standard SMOTE on a dataset where classes overlap.
2. **Cost matrix optimization:** implement cost-sensitive learning where the cost matrix is a parameter. Create a function that takes a cost matrix and returns optimal predictions that minimize expected cost. Test with different cost ratios (1:10, 1:100, 1:1000) and plot how the precision-recall tradeoff changes.
3. **Threshold calibration:** implement Platt scaling (fit a logistic regression on the model's raw outputs to produce calibrated probabilities). Compare the precision-recall curve before and after calibration. Show that calibration does not change the ranking (AUC stays the same) but makes the probabilities more meaningful.
4. **Ensemble with balanced bagging:** train multiple models, each on a balanced bootstrap sample (all minority + random subset of majority). Average their predictions. Compare this approach against a single model with SMOTE. Measure both performance and variance across runs.

5. **Imbalance ratio experiment:** take a balanced dataset and progressively increase the imbalance ratio (50/50, 70/30, 90/10, 95/5, 99/1). For each ratio, train with and without SMOTE. Plot F1 vs imbalance ratio for both approaches. At what ratio does SMOTE start making a meaningful difference?

Key Terms

Term	What people say	What it actually means
Class imbalance	“One class has way more samples”	The distribution of classes in the dataset is significantly skewed, causing models to favor the majority class
SMOTE	“Synthetic oversampling”	Creates new minority samples by interpolating between existing minority samples and their k-nearest minority neighbors
Class weights	“Making errors on rare classes more expensive”	Multiplying the loss function by class-specific weights so the model penalizes minority misclassification more heavily
Threshold tuning	“Moving the decision boundary”	Changing the probability cutoff for classification from the default 0.5 to a value that optimizes the desired metric
Precision-recall tradeoff	“You cannot have both”	Lowering the threshold catches more positives (higher recall) but also flags more false positives (lower precision), and vice versa
AUPRC	“Area under the PR curve”	Summarizes the precision-recall curve into a single number; more informative than AUC-ROC when classes are heavily imbalanced
Matthews Correlation Coefficient	“The balanced metric”	A correlation between predicted and actual labels that produces a high score only when the model performs well on both classes
Cost-sensitive learning	“Different mistakes cost different amounts”	Incorporating real-world misclassification costs into the training objective so the model optimizes for total cost, not error count
Random oversampling	“Duplicate the minority”	Repeating minority class samples to balance class counts; simple but risks overfitting to duplicated points

Further Reading

- SMOTE: Synthetic Minority Over-sampling Technique (Chawla et al., 2002) – the original SMOTE paper, still the most cited work on imbalanced learning
- Learning from Imbalanced Data (He & Garcia, 2009) – comprehensive survey covering sampling, cost-sensitive, and algorithmic approaches
- imbalanced-learn documentation – Python library with SMOTE variants, undersampling strategies, and pipeline integration

- The Precision-Recall Plot Is More Informative than the ROC Plot (Saito & Rehmsmeier, 2015) – when and why to prefer PR curves over ROC curves for imbalanced problems

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/17-imbalanced-data>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/17-imbalanced-data/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/17-imbalanced-data>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Feature Selection

More features is not better. The right features is better.

Type: Build **Language:** Python **Prerequisites:** Phase 2, Lessons 01-09, 08 (feature engineering) **Time:** ~75 minutes

Learning Objectives

- Implement filter methods (variance threshold, mutual information, chi-squared) and wrapper methods (RFE, forward selection) from scratch
- Explain why mutual information captures nonlinear feature-target relationships that correlation misses
- Compare L1 regularization (embedded selection) with RFE (wrapper selection) and evaluate their computational tradeoffs
- Build a feature selection pipeline that combines multiple methods and demonstrate improved generalization on held-out data

The Problem

You have 500 features. Your model trains slowly, overfits constantly, and nobody can explain what it learned. You add more features hoping to improve performance. It gets worse.

This is the curse of dimensionality in action. As the number of features grows, the volume of the feature space explodes. Data points become sparse. Distances between points converge. The model needs exponentially more data to find real patterns. Noise features drown out signal features. Overfitting becomes the default.

Feature selection is the antidote. Strip away the noise. Remove the redundancy. Keep the features that carry actual information about the target. The result: faster training, better generalization, and models you can actually explain.

The goal is not to use all available information. It is to use the right information.

The Concept

Three Categories of Feature Selection

Every feature selection method falls into one of three categories:

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/18-feature-selection>

Filter methods score each feature independently using a statistical measure. They do not use a model. Fast, but they miss feature interactions.

Wrapper methods train a model to evaluate feature subsets. They use model performance as the score. Better results, but expensive because they retrain the model many times.

Embedded methods select features as part of model training. L1 regularization drives weights to zero. Decision trees split on the most useful features. Selection happens during fitting, not as a separate step.

Variance Threshold

The simplest filter. If a feature barely varies across samples, it carries almost no information.

Consider a feature that is 0.0 for 999 out of 1000 samples. Its variance is near zero. No model can use it to distinguish between classes. Remove it.

$$\text{variance}(x) = \text{mean}((x - \text{mean}(x))^2)$$

Set a threshold (e.g., 0.01). Drop every feature with variance below it. This removes constant or near-constant features without looking at the target variable at all.

When to use it: as a preprocessing step before other methods. It catches obviously useless features at near-zero cost.

Limitation: a feature can have high variance and still be pure noise. Variance threshold is necessary but not sufficient.

Mutual Information

Mutual information measures how much knowing the value of feature X reduces uncertainty about target Y.

$$I(X; Y) = \sum_x \sum_y p(x, y) * \log(p(x, y) / (p(x) * p(y)))$$

If X and Y are independent, $p(x, y) = p(x) * p(y)$, so the log term is zero and $I(X; Y) = 0$. The more X tells you about Y, the higher the mutual information.

Key advantage over correlation: mutual information captures nonlinear relationships. A feature might have zero correlation with the target but high mutual information because the relationship is quadratic or periodic.

For continuous features, discretize into bins first (histogram-based estimation). The number of bins affects the estimate – too few bins lose information, too many bins add noise. A common choice: $\sqrt{n}$ bins or Sturges' rule $(1 + \log_2(n))$.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/18-feature-selection>

Recursive Feature Elimination (RFE)

RFE is a wrapper method. It uses a model's own feature importance to iteratively prune:

1. Train the model with all features
2. Rank features by importance (coefficients for linear models, impurity reduction for trees)
3. Remove the least important feature(s)
4. Repeat until the desired number of features remains

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/18-feature-selection>

RFE considers feature interactions because the model sees all remaining features together. Removing one feature changes the importance of others. This makes it more thorough than filter methods.

The cost: you train the model N - target times. With 500 features and a target of 10, that is 490 training runs. For expensive models, this is slow. You can speed it up by removing multiple features per step (e.g., remove the bottom 10% each round).

L1 (Lasso) Regularization

L1 regularization adds the absolute value of weights to the loss function:

```
loss = prediction_error + alpha * sum(|w_i|)
```

The alpha parameter controls how aggressively features are pruned. Higher alpha means more weights go to exactly zero.

Why exactly zero? The L1 penalty creates a diamond-shaped constraint region in weight space. The optimal solution tends to land at a corner of this diamond, where one or more weights are zero. L2 regularization (ridge) creates a circular constraint where weights shrink but rarely hit zero.

This is embedded feature selection: the model learns during training which features to ignore. Features with zero weight are effectively removed.

Advantages: single training run, handles correlated features (picks one and zeros the others), built into most linear model implementations.

Limitation: only works for linear models. Cannot capture nonlinear feature importance.

Tree-Based Feature Importance

Decision trees and their ensembles (random forests, gradient boosting) naturally rank features. Every split reduces impurity (Gini or entropy for classification, variance for regression). Features that produce larger impurity reductions are more important.

For a random forest with T trees:

```
importance(feature_j) = (1/T) * sum over all trees of
    sum over all nodes splitting on feature_j of
        (n_samples * impurity_decrease)
```

This gives a normalized importance score for each feature. It handles nonlinear relationships and feature interactions automatically.

Caution: tree-based importance is biased toward features with many unique values (high cardinality). A random ID column will appear important because it perfectly splits every sample. Use permutation importance as a sanity check.

Permutation Importance

A model-agnostic method:

1. Train the model and record baseline performance on validation data
2. For each feature: shuffle its values randomly, measure the drop in performance
3. The bigger the drop, the more important the feature

If shuffling a feature does not hurt performance, the model does not depend on it. If performance collapses, that feature is critical.

Permutation importance avoids the cardinality bias of tree-based importance. But it is slow: one full evaluation per feature, repeated multiple times for stability.

Comparison Table

Method	Type	Speed	Nonlinear	Feature Interactions
Variance threshold	Filter	Very fast	No	No
Mutual information	Filter	Fast	Yes	No
Correlation filter	Filter	Fast	No	No
RFE	Wrapper	Slow	Depends on model	Yes
L1 / Lasso	Embedded	Fast	No (linear)	No
Tree importance	Embedded	Medium	Yes	Yes
Permutation importance	Model-agnostic	Slow	Yes	Yes

Decision Flowchart

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/18-feature-selection>

Interactive figure: f3-feature-prune. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/18-feature-selection>

Build It

Step 1: Generate synthetic data with known feature structure

```
import numpy as np

def make_feature_selection_data(n_samples=500, seed=42):
    rng = np.random.RandomState(seed)

    x1 = rng.randn(n_samples)
    x2 = rng.randn(n_samples)
    x3 = rng.randn(n_samples)
    x4 = x1 + 0.1 * rng.randn(n_samples)
    x5 = x2 + 0.1 * rng.randn(n_samples)

    informative = np.column_stack([x1, x2, x3, x4, x5])

    correlated = np.column_stack([
        x1 * 0.9 + 0.1 * rng.randn(n_samples),
        x2 * 0.8 + 0.2 * rng.randn(n_samples),
        x3 * 0.7 + 0.3 * rng.randn(n_samples),
        x1 * 0.5 + x2 * 0.5 + 0.1 * rng.randn(n_samples),
        x2 * 0.6 + x3 * 0.4 + 0.1 * rng.randn(n_samples),
    ])
    ])
```

```

noise = rng.randn(n_samples, 10) * 0.5

X = np.hstack([informative, correlated, noise])
y = (2 * x1 - 1.5 * x2 + x3 + 0.5 * rng.randn(n_samples) > 0).astype(int)

feature_names = (
    [f"info_{i}" for i in range(5)]
    + [f"corr_{i}" for i in range(5)]
    + [f"noise_{i}" for i in range(10)]
)

return X, y, feature_names

```

We know the ground truth: features 0-4 are informative (plus 3 and 4 are correlated copies of 0 and 1), features 5-9 are correlated with informative features, features 10-19 are pure noise. A good selection method should rank 0-4 highest and 10-19 lowest.

Step 2: Variance threshold

```

def variance_threshold(X, threshold=0.01):
    variances = np.var(X, axis=0)
    mask = variances > threshold
    return mask, variances

```

Step 3: Mutual information (discrete)

```

def discretize(x, n_bins=10):
    min_val, max_val = x.min(), x.max()
    if max_val == min_val:
        return np.zeros_like(x, dtype=int)
    bin_edges = np.linspace(min_val, max_val, n_bins + 1)
    binned = np.digitize(x, bin_edges[1:-1])
    return binned

def mutual_information(X, y, n_bins=10):
    n_samples, n_features = X.shape
    mi_scores = np.zeros(n_features)

    y_vals, y_counts = np.unique(y, return_counts=True)
    p_y = y_counts / n_samples

    for f in range(n_features):
        x_binned = discretize(X[:, f], n_bins)
        x_vals, x_counts = np.unique(x_binned, return_counts=True)
        p_x = dict(zip(x_vals, x_counts / n_samples))

        mi = 0.0
        for xv in x_vals:
            for yi, yv in enumerate(y_vals):
                joint_mask = (x_binned == xv) & (y == yv)
                p_xy = np.sum(joint_mask) / n_samples
                if p_xy > 0:
                    mi += p_xy * np.log(p_xy / (p_x[xv] * p_y[yi]))

```



```

        mi_scores[f] = mi

    return mi_scores

```

Step 4: Recursive Feature Elimination

```

def simple_logistic_importance(X, y, lr=0.1, epochs=100):
    n_samples, n_features = X.shape
    w = np.zeros(n_features)
    b = 0.0

    for _ in range(epochs):
        z = X @ w + b
        pred = 1.0 / (1.0 + np.exp(-np.clip(z, -500, 500)))
        error = pred - y
        w -= lr * (X.T @ error) / n_samples
        b -= lr * np.mean(error)

    return w, b

def rfe(X, y, n_features_to_select=5, lr=0.1, epochs=100):
    n_total = X.shape[1]
    remaining = list(range(n_total))
    rankings = np.ones(n_total, dtype=int)
    rank = n_total

    while len(remaining) > n_features_to_select:
        X_subset = X[:, remaining]
        w, _ = simple_logistic_importance(X_subset, y, lr, epochs)
        importances = np.abs(w)

        least_idx = np.argmin(importances)
        original_idx = remaining[least_idx]
        rankings[original_idx] = rank
        rank -= 1
        remaining.pop(least_idx)

    for idx in remaining:
        rankings[idx] = 1

    selected_mask = rankings == 1
    return selected_mask, rankings

```

Step 5: L1 feature selection

```

def soft_threshold(w, alpha):
    return np.sign(w) * np.maximum(np.abs(w) - alpha, 0)

def l1_feature_selection(X, y, alpha=0.1, lr=0.01, epochs=500):
    n_samples, n_features = X.shape
    w = np.zeros(n_features)
    b = 0.0

```

```

for _ in range(epochs):
    z = X @ w + b
    pred = 1.0 / (1.0 + np.exp(-np.clip(z, -500, 500)))
    error = pred - y

    gradient_w = (X.T @ error) / n_samples
    gradient_b = np.mean(error)

    w -= lr * gradient_w
    w = soft_threshold(w, lr * alpha)
    b -= lr * gradient_b

    selected_mask = np.abs(w) > 1e-6
    return selected_mask, w

```

Step 6: Tree-based importance (simple decision tree)

```

def gini_impurity(y):
    if len(y) == 0:
        return 0.0
    classes, counts = np.unique(y, return_counts=True)
    probs = counts / len(y)
    return 1.0 - np.sum(probs ** 2)

def best_split(X, y, feature_idx):
    values = np.unique(X[:, feature_idx])
    if len(values) <= 1:
        return None, -1.0

    best_threshold = None
    best_gain = -1.0
    parent_gini = gini_impurity(y)
    n = len(y)

    for i in range(len(values) - 1):
        threshold = (values[i] + values[i + 1]) / 2.0
        left_mask = X[:, feature_idx] <= threshold
        right_mask = ~left_mask

        n_left = np.sum(left_mask)
        n_right = np.sum(right_mask)

        if n_left == 0 or n_right == 0:
            continue

        gain = parent_gini - (n_left / n) * gini_impurity(y[left_mask]) - (n_right / n) * gini_impurity(y[right_mask])

        if gain > best_gain:
            best_gain = gain
            best_threshold = threshold

    return best_threshold, best_gain

```

```

def tree_importance(X, y, n_trees=50, max_depth=5, seed=42):
    rng = np.random.RandomState(seed)
    n_samples, n_features = X.shape
    importances = np.zeros(n_features)

    for _ in range(n_trees):
        sample_idx = rng.choice(n_samples, size=n_samples, replace=True)
        feature_subset = rng.choice(n_features, size=max(1, int(np.sqrt(n_features))), replace=

        X_boot = X[sample_idx]
        y_boot = y[sample_idx]

        tree_imp = _build_tree_importance(X_boot, y_boot, feature_subset, max_depth)
        importances += tree_imp

    total = importances.sum()
    if total > 0:
        importances /= total

    return importances

def _build_tree_importance(X, y, feature_subset, max_depth, depth=0):
    n_features = X.shape[1]
    importances = np.zeros(n_features)

    if depth >= max_depth or len(np.unique(y)) <= 1 or len(y) < 4:
        return importances

    best_feature = None
    best_threshold = None
    best_gain = -1.0

    for f in feature_subset:
        threshold, gain = best_split(X, y, f)
        if gain > best_gain:
            best_gain = gain
            best_feature = f
            best_threshold = threshold

    if best_feature is None or best_gain <= 0:
        return importances

    importances[best_feature] += best_gain * len(y)

    left_mask = X[:, best_feature] <= best_threshold
    right_mask = ~left_mask

    importances += _build_tree_importance(X[left_mask], y[left_mask], feature_subset, max_depth,
    importances += _build_tree_importance(X[right_mask], y[right_mask], feature_subset, max_dep

    return importances

```

Step 7: Run all methods and compare

The code file runs all five methods on the same synthetic dataset and prints a comparison table showing which features each method selects.

Use It

With scikit-learn, feature selection is built into the pipeline:

```
from sklearn.feature_selection import (
    VarianceThreshold,
    mutual_info_classif,
    RFE,
    SelectFromModel,
)
from sklearn.linear_model import Lasso, LogisticRegression
from sklearn.ensemble import RandomForestClassifier

vt = VarianceThreshold(threshold=0.01)
X_filtered = vt.fit_transform(X)

mi_scores = mutual_info_classif(X, y)
top_k = np.argsort(mi_scores)[-10:]

rfe_selector = RFE(LogisticRegression(), n_features_to_select=10)
rfe_selector.fit(X, y)
X_rfe = rfe_selector.transform(X)

lasso_selector = SelectFromModel(Lasso(alpha=0.01))
lasso_selector.fit(X, y)
X_lasso = lasso_selector.transform(X)

rf = RandomForestClassifier(n_estimators=100)
rf.fit(X, y)
importances = rf.feature_importances_
```

The from-scratch implementations show exactly what happens inside each method. Variance threshold is just computing $\text{var}(X, \text{axis}=0)$ and applying a mask. Mutual information is counting joint and marginal frequencies in a contingency table. RFE is a loop that trains, ranks, and prunes. L1 is gradient descent with a soft-thresholding step. Tree importance accumulates impurity reductions across splits. No magic – just statistics and loops.

The sklearn versions add robustness (e.g., `mutual_info_classif` uses k-NN density estimation instead of binning), speed (C implementations), and pipeline integration.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/18-feature-selection>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/18-feature-selection/code>

1. **Forward selection:** implement the opposite of RFE. Start with zero features. At each step, add the feature that improves model performance the most. Stop when adding features no longer helps. Compare the selected features against RFE results. Which is faster? Which gives better results?
2. **Stability selection:** run L1 feature selection 50 times, each time on a random 80% subsample of the data, with slightly different alpha values. Count how often each feature is selected. Features selected in > 80% of runs are “stable.” Compare stable features against single-run L1 selection. Which is more reliable?
3. **Multicollinearity detection:** compute the correlation matrix for all features. Implement a function that, given a correlation threshold (e.g., 0.9), removes one feature from each highly-correlated pair (keeping the one with higher mutual information with the target). Test on the synthetic dataset and verify it removes the redundant correlated features.
4. **Feature selection pipeline:** chain variance threshold, mutual information filter, and RFE into a single pipeline. First remove near-zero-variance features, then keep the top 50% by mutual information, then run RFE on the survivors. Compare this pipeline against running RFE alone on all features. Is the pipeline faster? Is it equally accurate?
5. **Permutation importance from scratch:** implement permutation importance. For each feature, shuffle its values 10 times, measure the average drop in F1 score. Compare the ranking against tree-based importance. Find cases where they disagree and explain why (hint: correlated features).

Key Terms

Term	What people say	What it actually means
Filter method	“Score features independently”	A feature selection approach that ranks features using a statistical measure without training a model, evaluating each feature in isolation
Wrapper method	“Use the model to pick features”	A feature selection approach that evaluates feature subsets by training a model and using its performance as the selection criterion
Embedded method	“The model selects features during training”	Feature selection that happens as part of model fitting, such as L1 regularization driving weights to zero
Mutual information	“How much one variable tells you about another”	A measure of the reduction in uncertainty about Y given knowledge of X, capturing both linear and nonlinear dependencies
Recursive Feature Elimination	“Train, rank, prune, repeat”	An iterative wrapper method that trains a model, removes the least important feature(s), and repeats until a target count is reached
L1 / Lasso regularization	“Penalty that kills features”	Adding the sum of absolute weight values to the loss function, which drives unimportant feature weights to exactly zero

Term	What people say	What it actually means
Variance threshold	“Remove constant features”	Dropping features whose variance across samples falls below a specified threshold, filtering out features that carry no information
Feature importance	“Which features matter most”	A score indicating how much each feature contributes to model predictions, computed from split gains (trees) or coefficient magnitudes (linear)
Permutation importance	“Shuffle and measure the damage”	Evaluating feature importance by randomly shuffling each feature’s values and measuring the resulting drop in model performance
Curse of dimensionality	“Too many features, not enough data”	The phenomenon where adding features increases the volume of the feature space exponentially, making data sparse and distances meaningless

Further Reading

- An Introduction to Variable and Feature Selection (Guyon & Elisseeff, 2003) – the foundational survey on feature selection methods, still widely referenced
- scikit-learn Feature Selection Guide – practical reference for filter, wrapper, and embedded methods with code examples
- Stability Selection (Meinshausen & Bühlmann, 2010) – combines subsampling with feature selection for robust, reproducible results
- Beware Default Random Forest Importances (Strobl et al., 2007) – demonstrates the cardinality bias in tree-based importance and proposes conditional importance as an alternative

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/18-feature-selection>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/02-ml-fundamentals/18-feature-selection/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson.html?path=phases/02-ml-fundamentals/18-feature-selection>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.